

Verilog HDL

程序设计 教程

王金明 编著
徐志军 主审



人民邮电出版社
POSTS & TELECOM PRESS



附 光 盘
CD-ROM

Verilog HDL

程序设计教程

ISBN 7-115-11939-2



9 787115 119391 >

ISBN7-115-11939-2/TN·2224
定价:34.00 元(附光盘)

人民邮电出版社网址 www.ptpress.com.cn

Verilog HDL 程序设计教程

王金明 编著

徐志军 主审

人民邮电出版社

图书在版编目 (CIP) 数据

Verilog HDL 程序设计教程 / 王金明编著. —北京: 人民邮电出版社, 2004.1

ISBN 7-115-11939-2

I. V... II. 王... III. 硬件描述语言, VHDL—程序设计—高等学校—教材 IV. TP312

中国版本图书馆 CIP 数据核字 (2003) 第 102959 号

Verilog HDL 程序设计教程

◆ 编 著 王金明

主 审 徐志军

责任编辑 杨 凌

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街 14 号

邮编 100061 电子函件 315@ptpress.com.cn

网址 <http://www.ptpress.com.cn>

读者热线 010-67129258

北京汉魂图文设计有限公司制作

北京隆昌伟业印刷有限公司印刷

新华书店总店北京发行所经销

◆ 开本: 787×1092 1/16

印张: 20.25

字数: 484 千字

2004 年 1 月第 1 版

印数: 1-4 500 册

2004 年 1 月北京第 1 次印刷

ISBN 7-115-11939-2/TN · 2224

定价: 34.00 元 (附光盘)

本书如有印装质量问题, 请与本社联系 电话: (010) 67129223

内 容 提 要

本书对 Verilog HDL 程序设计作了系统全面的介绍，以可综合的设计为重点，同时对仿真和模拟也作了深入的阐述。

本书以 Verilog-1995 标准为基础，全面介绍了 Verilog HDL 的词法、语法、语句，可综合程序的编写，仿真程序的编写，一般数字逻辑的实现，复杂逻辑和算法的实现等，讨论了设计中的方法与技巧，并以大量经过验证的设计实例为依据，深入浅出地阐述了 Verilog 程序开发所涉及的各个方面。对 Verilog-2001 的新增语法结构也作了介绍。

着眼于实用是本书的出发点，由于 HDL 语言的学习与使用必须依托一定的 EDA 环境，因此对典型 EDA 软件的使用与接口也作了介绍。

本书可作为电子工程、通信工程及相关专业高年级本科生和研究生教学用书，也可供从事电路设计和系统开发的工程设计人员阅读参考。

前 言

在现代电子设计领域，EDA 技术已成为每个工程师和设计师必须掌握的工具，起着越来越重要的作用。从 PCB 设计到芯片版图制作，从原理图输入到 HDL 语言描述，从 FPGA 设计到 ASIC 实现，从综合到仿真等等，这些功能的实现，无不依靠 EDA 技术提供强有力的支持。

EDA 技术的发展使得现代电子系统设计从设计思想、设计工具一直到实现方式都产生了深刻的变化，因此，怎样在当前的 EDA 教学中体现出这些变化，抓住 EDA 技术的实质，则是每一个从事 EDA 教学的老师所面临和思考的问题，其根本目的是在有限的学时内将 EDA 技术最本质的内容作为讲授的重点传达给学生，使其能够掌握 EDA 技术的精髓。

HDL 语言是 EDA 技术中重要的一个方面，作者认为，要学习和掌握 EDA 技术，必须至少熟练掌握一种 HDL 语言，其中 Verilog HDL 和 VHDL 显然是首选，这两种语言是已成为 IEEE 标准的硬件描述语言，并各有特色。Verilog 语言是在 C 语言的基础上发展而来的硬件描述语言，具有简洁、高效、易用的特点，是使用最广泛的硬件设计语言之一。在 ASIC 设计领域，Verilog 语言已成为了事实上的标准。IEEE 先后推出了两个 Verilog 标准：IEEE Std. 1364-1995 (Verilog-1995) 和 IEEE Std. 1364-2001 (Verilog-2001)，后者在前者的基础上对 Verilog 语言做了若干改进和扩充，使其功能更强、使用更方便。本书以讲授 Verilog-1995 为主，系统介绍 Verilog 语言的词法、语法、语句，以可综合的设计为重点，同时兼顾仿真和模拟，通过大量的例程由浅入深地展示了 Verilog HDL 程序设计的方法与技巧。对于 Verilog-2001，由于目前的综合和仿真工具还没有全面的支持，因此只作了比较简单的介绍。

EDA 软件工具也是设计开发中重要的一环，优秀的 EDA 工具为 HDL 语言的学习和开发提供了理想的平台，它将设计者的思想高效、高质量地转化为物理电路，并以直观、便捷的形式提供了仿真模拟的手段。脱离了 EDA 工具的支持，HDL 语言就失去了它的意义，基于此考虑，本书用一定的篇幅介绍了典型 EDA 工具的使用方法，将 Verilog 程序设计和典型 EDA 设计工具的使用相结合，并尽量给出综合或仿真的结果，以便于对照。

本书所附的光盘包括了书中所有实例的源代码，这些实例全部通过了综合或仿真，所用软件为仿真软件 ModelSim、综合软件 Synplify Pro/Synplify 以及集成工具 MAX+PLUS II 等。如将这些程序移植到其他综合器和仿真器平台上，由于 Verilog 语言的标准性，同样会得出相同或相似的综合和仿真结果。

本书共分 13 章。

第 1 章对 EDA 技术的发展与特点作了综述，解释了有关的概念。第 2 章介绍流行的 EDA 软件和典型的基于 FPGA 的 EDA 设计流程。

第 3 章以完整的 Verilog 设计过程为例展示了 Verilog 程序设计的全过程，力图使读者快速建立 HDL 设计的整体印象。在第 4、5、6 章中详细介绍了 Verilog 的词法、语法、语句、模块等内容，同时对 Verilog-2001 中新增的语法结构作了简介。第 7 章介绍 Verilog 程序设计

的风格, 包括结构描述、行为描述和数据流描述三种方式。

第 8 章是有关仿真的内容, 包括系统任务和函数、UDP 元件、延时模型等, 然后是电路仿真的具体实例。第 9 章列举了常用数字电路的 Verilog 描述方法。第 10 章对 Verilog 程序设计的方法与技巧进行了探讨和总结。第 11 章和第 12 章是典型数字电路的设计举例, 以及一些较复杂数字逻辑和算法的实现方法。第 13 章详细介绍了典型 EDA 软件的使用方法和相互间的接口。

写作本书的目的在于将作者学习使用 Verilog 语言的心得体会以及一些积累与读者共享, 假如本书对您学习使用 Verilog 语言能有所帮助的话, 则是作者最大的荣幸。

徐志军教授对全书进行了审阅, 并提出了修改意见, 张雄伟教授也给予了具体建议, 杭州电子工业学院的潘松老师和黄继业老师提供了诸多帮助, 作者在此对他们一并表示诚挚的谢意。

此外, 还要感谢 Altera 公司中国区项目经理徐平波先生和骏龙公司曾雪峰工程师对作者的大力支持。

由于水平所限, 加上时间仓促, 书中错误与疏漏之处在所难免, 真诚希望同行和广大读者给予批评指正。

电子邮件地址: wjm_ice@sina.com。

作者
2003 年 10 月于南京

目 录

第 1 章 EDA 技术综述	1
本章内容简介	1
1.1 引言	1
1.2 EDA 技术及其发展	2
1.3 设计方法与设计技术	3
1.3.1 Top-down 设计	3
1.3.2 Bottom-up 设计	5
1.3.3 IP 复用技术与 SOC	5
1.4 EDA 设计的实现	6
1.5 硬件描述语言	7
思考与练习	9
第 2 章 EDA 设计软件与设计流程	10
本章内容简介	10
2.1 EDA 软件工具概述	10
2.1.1 集成的 CPLD/FPGA 开发工具	10
2.1.2 输入工具 (Design Input Tools)	11
2.1.3 逻辑综合工具 (Synthesis Tools)	12
2.1.4 仿真工具 (Simulation Tools)	12
2.1.5 IC 版图工具	13
2.1.6 其他 EDA 工具	14
2.2 EDA 设计的流程	14
2.2.1 输入 (Design Input)	14
2.2.2 综合 (Synthesis)	15
2.2.3 适配 (Fitter)	16
2.2.4 仿真 (Simulation)	16
2.2.5 编程 (Program)	17
思考与练习	17
第 3 章 Verilog HDL 设计初步	18

本章内容简介	18
3.1 Verilog 语言的历史及与 C 语言的比较	18
3.2 完整的 Verilog HDL 设计	20
3.2.1 4 位全加器和 4 位计数器	20
3.2.2 综合	21
3.2.3 仿真	22
3.3 Verilog 模块基本结构剖析	27
3.3.1 Verilog 模块的结构	27
3.3.2 逻辑功能定义	29
思考与练习	30
第 4 章 Verilog HDL 语言要素	32
本章内容简介	32
4.1 词法	32
4.1.1 空白符和注释	32
4.1.2 数字与字符串 (Numbers & Strings)	33
4.1.3 标识符 (Identifiers)	35
4.1.4 运算符 (Operators)	36
4.1.5 关键字 (Keywords)	36
4.2 数据类型	36
4.2.1 连线型 (Net Type)	36
4.2.2 寄存器型 (Register Type)	38
4.2.3 parameter	38
4.3 寄存器和存储器	39
4.3.1 寄存器	39
4.3.2 存储器	39
4.4 运算符	40
4.4.1 运算符	40
4.4.2 运算符的优先级	44
思考与练习	44
第 5 章 Verilog HDL 行为语句	45
本章内容简介	45
5.1 概述	45
5.2 过程语句	46
5.2.1 always 过程语句	46
5.2.2 initial 语句	50
5.3 块语句	51
5.3.1 串行块 begin-end	51

5.3.2 并行块 fork-join	52
5.4 赋值语句	53
5.4.1 持续赋值与过程赋值	53
5.4.2 阻塞赋值与非阻塞赋值	55
5.5 条件语句	56
5.5.1 if-else 语句	56
5.5.2 case 语句	58
5.5.3 条件语句使用要点	60
5.6 循环语句	61
5.6.1 for 语句	61
5.6.2 repeat 语句	62
5.6.3 while 和 forever 语句	63
5.7 编译向导	65
5.7.1 宏替换`define	66
5.7.2 文件包含`include	66
5.7.3 条件编译`ifdef、`else、`endif	67
思考与练习	68
第 6 章 进程、任务与函数	69
本章内容简介	69
6.1 进程 (process)	69
6.1.1 进程	69
6.1.2 进程间的通信	70
6.2 任务 (task)	71
6.3 函数 (function)	74
6.3.1 函数	74
6.3.2 任务与函数的区别	79
6.4 顺序执行与并发执行	79
6.5 Verilog-2001 新增语法结构简介	81
思考与练习	84
第 7 章 Verilog HDL 的描述风格	85
本章内容简介	85
7.1 概述	85
7.2 结构描述	86
7.2.1 Verilog HDL 内置门元件	86
7.2.2 门级结构描述	89
7.3 行为描述方式	90
7.4 数据流描述方式	91

7.5 不同描述风格的例子	93
7.5.1 半加器	93
7.5.2 1 位全加器	95
7.5.3 4 位全加器	98
思考与练习	100
第 8 章 仿真	101
本章内容简介	101
8.1 概述	101
8.2 系统任务与系统函数	102
8.2.1 \$display 与 \$write	102
8.2.2 \$monitor 与 \$strobe	103
8.2.3 \$time 与 \$realtime	104
8.2.4 \$finish 与 \$stop	105
8.2.5 \$readmemh 与 \$readmemb	105
8.2.6 \$random	106
8.2.7 文件输出	106
8.3 用户自定义元件 (UDP)	107
8.3.1 组合电路 UDP 元件	107
8.3.2 时序逻辑 UDP 元件	110
8.3.3 UDP 元件缩记符	111
8.4 延时模型的表示	112
8.4.1 时间标尺定义 `timescale	112
8.4.2 延时的表示方法	113
8.4.3 延时说明块 (specify 块)	114
8.5 测试平台 (Test Bench)	114
8.5.1 Test Bench	114
8.5.2 测试程序的编写	116
8.6 仿真示例	119
8.6.1 组合电路的仿真	119
8.6.2 时序电路的仿真	124
思考与练习	125
第 9 章 Verilog HDL 设计进阶	126
本章内容简介	126
9.1 基本组合电路的设计	126
9.1.1 简单门电路	126
9.1.2 编译码器	129
9.1.3 数据选择器	132

9.1.4 用组合电路实现的 ROM	133
9.2 基本时序电路的设计	134
9.2.1 D 触发器与 JK 触发器	134
9.2.2 锁存器与寄存器	136
9.2.3 计数器	139
9.2.4 ROM/RAM 模块	140
9.2.5 串/并转换	141
9.3 简单的微处理器	142
9.3.1 设计实现	142
9.3.2 仿真	143
9.4 乘累加器 (MAC) 的设计	144
思考与练习	147
第 10 章 设计方法与设计技巧的探讨	148
本章内容简介	148
10.1 可综合的设计技术的讨论	148
10.1.1 可综合的 Verilog HDL 结构	148
10.1.2 可综合设计的要点	150
10.2 流水线设计技术 (Pipeline Design)	151
10.3 资源共享 (Resource Sharing)	155
10.4 有限状态机 (FSM) 设计	157
10.4.1 基于状态机的设计	157
10.4.2 频率计控制器设计举例	160
10.4.3 几点讨论	162
10.5 多层次结构电路的设计	164
10.5.1 图形与文本混合设计	164
10.5.2 文本设计	165
10.6 阻塞与非阻塞赋值使用要点	167
10.6.1 进程的设计	167
10.6.2 阻塞赋值与非阻塞赋值	168
10.6.3 应用要点	171
10.7 片内存储器的使用	172
10.7.1 片内存储器	172
10.7.2 设计举例	173
10.8 如何消除毛刺	175
10.8.1 毛刺的产生	176
10.8.2 毛刺的消除	177
思考与练习	179

第 11 章 Verilog HDL 综合设计实践	180
本章内容简介	180
11.1 数字跑表	180
11.1.1 设计输入与编译	180
11.1.2 仿真	183
11.2 4 位数字频率计	183
11.2.1 功能与原理	183
11.2.2 设计实现	184
11.3 交通灯控制器	187
11.3.1 功能要求	187
11.3.2 设计实现	188
11.4 乐曲演奏电路	191
11.4.1 音调的控制	192
11.4.2 音长的控制	193
11.4.3 乐曲演奏电路源程序	193
11.5 自动售饮料机的设计	197
11.6 实用多功能数字钟	199
11.6.1 功能	200
11.6.2 源程序	200
11.7 计费器设计	205
思考与练习	209
第 12 章 算法与复杂逻辑的实现	210
本章内容简介	210
12.1 加法器设计	210
12.1.1 级连加法器	211
12.1.2 并行加法器	212
12.1.3 超前进位加法器	212
12.1.4 流水线加法器	219
12.2 乘法器设计	219
12.2.1 并行乘法器	219
12.2.2 移位相加乘法器	220
12.2.3 查找表乘法器	220
12.2.4 加法树乘法器	223
12.3 FIR 滤波器的设计	225
12.3.1 FIR 滤波器的结构	225
12.3.2 抽头系数编码	226
12.3.3 源代码及仿真	227

12.4 数字相关器	231
12.5 信道编译码器	233
12.5.1 线性分组码编译码器	233
12.5.2 循环码编译码器	237
12.6 CRC 校验码	241
思考与练习	244
第 13 章 EDA 软件使用指南	245
本章内容简介	245
13.1 概述	245
13.2 Synplify Pro 使用指南	246
13.2.1 Synplify Pro 的性能特点	247
13.2.2 Synplify Pro 使用指南	248
13.2.3 Synplify Pro 与 MAX+PLUS II 的接口	251
13.3 Synplify 使用指南	253
13.3.1 Synplify 的功能特点	253
13.3.2 Synplify 使用举例	254
13.4 ModelSim 使用指南	256
13.4.1 ModelSim 功能仿真	257
13.4.2 ModelSim 时序仿真	262
13.5 ispLEVER 使用指南	264
13.5.1 ispLEVER 的特点	264
13.5.2 基于 ispLEVER 的 Verilog 设计	264
13.6 MAX+PLUS II 使用指南	268
13.7 Quartus II 使用指南	274
13.7.1 创建工程文件	274
13.7.2 编译	278
13.7.3 仿真	281
13.7.4 Synplify Pro 与 Quartus II 的接口	284
13.8 结束语	285
附录 A Verilog HDL (IEEE Std 1364-1995) 关键字	286
附录 B Synplify Pro/Synplify 可综合的 Verilog 结构	287
附录 C MAX+PLUS II 软件支持的 Verilog 结构	288
附录 D 形式化句法定义	289
附录 E 附带光盘内容及说明	304
参考文献	308

第 1 章 EDA 技术综述

本章内容简介

- ◆ EDA 技术的发展
- ◆ EDA 技术的特点
- ◆ Top-down 设计
- ◆ Bottom-up 设计
- ◆ IP 复用 (IP reuse) 技术
- ◆ 系统芯片 (SOC)
- ◆ 可编程逻辑器件 (PLD)
- ◆ 专用集成电路 (ASIC)
- ◆ 硬件描述语言 (HDL)

本章对 EDA 技术进行综述, 介绍 EDA 技术的发展及有关的概念。

EDA 技术仍在不断地向前发展, 其自动化和智能化程度不断提高。超大规模集成电路集成度和工艺水平的提高, 嵌入式系统的开发与应用, 对 EDA 技术提出了更高的要求, 也促使其不断进步。现有的 HDL 语言还只能提供行为级或功能级的描述, 尚无法完成复杂的系统级的抽象描述, 人们正在这方面努力寻求突破。

1.1 引言

我们已经进入了数字化和信息化的时代, 其特点是各种数字产品的广泛应用。现代数字产品在性能提高、复杂度增大的同时, 其更新换代的步伐也越来越快, 实现这种进步的因素在于生产制造技术和电子设计技术的进步。

生产制造技术以微细加工技术为代表, 目前已进展到深亚微米阶段, 可以在几平方厘米的芯片上集成数千万个晶体管。摩尔曾经对半导体集成技术的发展做出预言: 大约每 18 个月, 芯片的集成度提高 1 倍, 功耗下降一半, 称为摩尔定律 (Moore's law)。几十年来, 集成电路的发展与这个预言基本一致。数字芯片经历了从 SSI、MSI、LSI 到 VLSI, 直到现在的系统芯片 (SOC, System On Chip), 我们已经能够把一个完整的电子系统集成在一个芯片上。此外, 还有一种器件的发明与使用极大地改变了我们设计制作电子系统的方式与方法, 这就是可编程逻辑器件 (PLD, Programmable Logic Device)。PLD 器件是 20 世纪 70 年代后发展

起来的一种器件,它经历了从可编程逻辑阵列(PLA, Programmable Logic Array)、通用阵列逻辑(GAL, Generic Array Logic)等简单形式到现场可编程门阵列(FPGA, Field Programmable Gate Array)和复杂可编程逻辑器件(CPLD, Complex Programmable Logic Device)等高级形式的发展,它的广泛使用不仅简化了电路设计,降低了成本,提高了系统的可靠性,而且给数字系统的设计方式也带来了革命性的变化。PLD 器件目前仍在朝密度更高、速度更快、功耗更低、功能更强的方向发展。

电子设计技术的发展也是日新月异的,电子系统的设计理念和设计方法在过去的几十年时间里也发生了深刻的变化。从电子 CAD (Computer Aided Design)、电子 CAE (Computer Aided Engineering)到电子设计自动化(EDA, Electronic Design Automation),设计的自动化程度越来越高,设计的复杂性也越来越强。

目前,EDA 技术已成为现代电子设计技术的核心,没有 EDA 技术的支持,想要完成超大规模集成电路的设计制造是不可想象的。反过来,生产制造技术的进步又不断对 EDA 技术提出新的要求,促使其不断向前发展。

1.2 EDA 技术及其发展

在现代的电子设计中,EDA 技术已经成为一种普遍的工具。对设计者而言,熟练地掌握 EDA 技术,可以大大提高工作效率,起到事半功倍的效果。

EDA 即电子设计自动化。EDA 技术的发展是以计算机科学、微电子技术的发展为基础,并融合了应用电子技术、智能技术以及计算机图形学、拓扑学、计算数学等众多学科的最新成果发展起来的。简单地说,EDA 就是立足于计算机工作平台而开发出来的一整套先进的设计电子系统的软件工具。

一般认为,EDA 技术经历了下面 3 个发展阶段。

1. CAD 阶段

电子 CAD 阶段是 EDA 技术发展的早期阶段。在这个阶段,一方面,计算机的功能还比较有限,个人计算机还没有普及;另一方面,电子设计软件的功能也较弱。人们主要是借助于计算机对所设计电路的性能进行一些模拟和预测;另外,就是完成 PCB 板的布局布线、简单版图的绘制等工作。

2. CAE 阶段

随着集成电路规模的扩大,电子系统设计的逐步复杂,电子 CAD 的工具逐步完善和发展,尤其是人们在设计方法学、设计工具集成化方面取得了长足的进步,就进入了电子 CAE 阶段。在这个阶段,各种单点设计工具、各种设计单元库逐渐完备,并且开始将许多单点工具集成在一起使用,大大提高了工作效率。

3. EDA 阶段

20 世纪 90 年代以来,微电子工艺有了惊人的发展,工艺水平已经达到了深亚微米级,在一个芯片上已经可以集成上百万、上千万乃至上亿个晶体管,芯片的工作速度达到了吉比特级,这样就对电子设计的工具提出了更高的要求,也促进了设计工具的发展。

在今天,EDA 技术已经成为电子设计的普及工具。无论是设计芯片还是设计系统,没有

EDA 工具的支持, 都是难以完成的。EDA 工具已经成为设计师必不可少的武器, 起着越来越重要的作用。

从发展的过程看, EDA 技术一直是滞后于制造工艺的发展的, 它在制造技术的驱动下, 不断地向前进步; 从长远看, EDA 技术将随着微电子技术、计算机技术的不断发展而发展。

总的来说, EDA 技术将向着智能性更高、功能更强、高层综合的方向发展。另外一个发展方向就是支持软、硬件协同设计。其次, 随着芯片规模的扩大, 一些新的问题也需要 EDA 工具加以解决。比如, 在大型的系统设计中, 设计验证工作将变得比设计本身还要艰巨, 因此, 更好、更快的仿真验证工具也是 EDA 需要加以完善的地方。

EDA 的发展是没有止境的, 它将在诸多因素的推动下不断前进。

1.3 设计方法与设计技术

近 20 年来, 电子系统的设计方法和设计技术都发生了深刻的变化。在以前, 数字系统大多是采用搭积木式的方式设计的, 即由一些固定功能的器件加上一定的外围电路构成模块, 由这些模块再进一步形成各种功能电路。构成系统的“积木块”是各种标准芯片, 如 74/54 系列 (TTL)、4000/4500 系列 (CMOS) 芯片等等, 这些芯片的功能是固定的, 用户只能根据需求从这些标准器件中选出最适合的, 并按照推荐的电路搭成系统。在设计时, 几乎没有灵活性可言, 设计一个系统所需的芯片种类多且数量大。

PLD 器件和 EDA 技术的出现改变了传统的设计思路, 使人们可以通过设计芯片来实现各种不同的功能。新的设计方法能够由设计者自己定义器件的内部逻辑和管脚, 将原来由电路板设计完成的工作大部分放在芯片的设计中进行。这样不仅可以通过芯片设计实现各种数字逻辑功能, 而且由于管脚定义的灵活性, 大大减轻了原理图和印制板设计的工作量和难度, 增加了设计的自由度, 提高了效率。同时, 这种设计减少了所用芯片的种类和数量, 缩小了体积, 降低了功耗, 提高了系统的可靠性。

在基于 EDA 技术的设计中, 有两种基本的设计思路, 一种是自顶向下的设计思路, 一种是自底向上的设计思路。下面分别进行介绍。

1.3.1 Top-down 设计

Top-down 设计, 即自顶向下的设计。这种设计方法首先从系统设计入手, 在顶层进行功能方框图的划分和结构设计。在功能级进行仿真、纠错, 并用硬件描述语言对高层次的系统行为进行描述, 然后用综合工具将设计转化为具体门电路网表, 其对应的物理实现可以是 PLD 器件或专用集成电路 (ASIC)。由于设计的主要仿真和调试过程是在高层次上完成的, 这不仅有利于早期发现结构设计上的错误, 避免设计工作的浪费, 而且也减少了逻辑功能仿真的工作量, 提高了设计的一次成功率。

在 Top-down 设计中, 将设计分成系统级、功能级、门级、开关级等几个不同的层次, 按照自上而下的顺序, 在不同的层次上, 对系统进行设计与仿真。

图 1.1 是这种设计方式的示意图。如图所示, 在 Top-down 的设计过程中, 需要有 EDA 工具的支持, 有些步骤 EDA 工具可以自动完成, 比如综合等, 有些步骤 EDA 工具为用户提

供了操作平台。

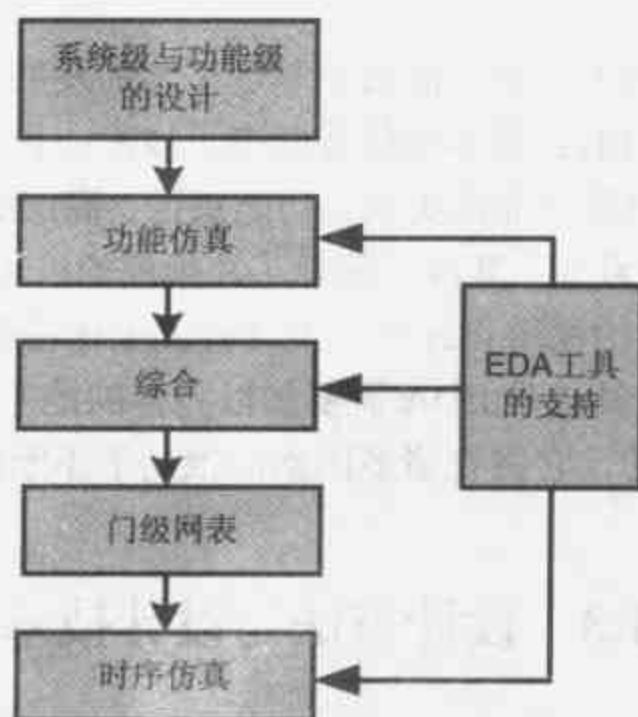


图 1.1 Top-down 设计方式示意图

Top-down 设计须经过“设计—验证—修改设计—再验证”的过程，不断反复，直到得到的结果能够完全实现所要求的逻辑功能，并且在速度、功耗、价格和可靠性方面实现较为合理的平衡为止。不过，这种设计也并非绝对的，在设计过程中，有时也需要用到自下而上的方法，就是在系统划分和分解的基础上，先进行底层单元设计，然后再逐步向上进行功能块、子系统的设计，直至构成整个的系统。

如图 1.2 所示是用 Top-down 的设计方式设计一个 CPU 的示意图。首先在系统级进行划分，将整个 CPU 划分为几个模块，如 ALU、PC、RAM 模块等，对每个模块再分别进行设计与描述，然后通过 EDA 工具将整个设计综合为门级网表实现之。在设计过程中，需要进行多次仿真和验证，不断修改设计。

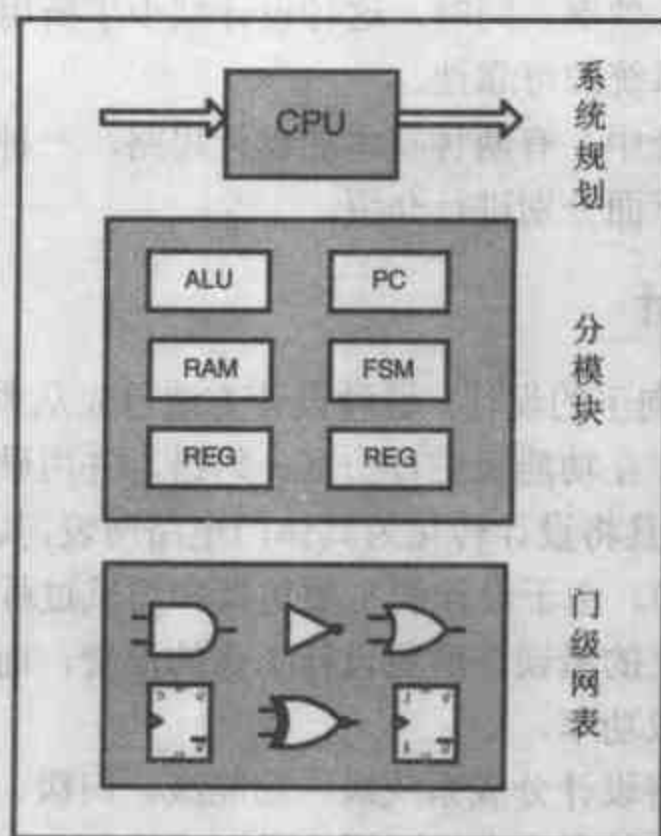


图 1.2 CPU 的 Top-down 设计方式示意图

1.3.2 Bottom-up 设计

Bottom-up 设计, 即自底向上的设计, 这是一种传统的设计思路。这种设计方式一般是设计者选择标准的集成电路, 或者将各种基本单元, 如各种门电路以及像加法器、计数器等模块做成基本单元库, 调用这些基本单元, 逐级向上组合, 直到设计出满足自己需要的系统为止。这样的设计方法就如同一砖一瓦建造金字塔, 不仅效率低、成本高, 而且容易出错。

目前, Top-down 设计更为人们所接受, 也为更多的 EDA 工具所支持, 这是由于这种设计思想更符合人们逻辑思维的习惯, 也容易使设计者对复杂的系统进行合理的划分与不断的优化。而 Bottom-up 设计往往使设计者关注了细节, 却对整个的系统缺乏规划, 当设计出现问题时, 如果要修改的话, 就会很麻烦, 甚至前功尽弃, 不得不从头再来。因此, 作为设计者, 应该培养 Top-down 的设计习惯。

1.3.3 IP 复用技术与 SOC

当电子系统的设计越来越向高层发展的时候, 基于 IP 复用 (IP reuse) 的设计技术越来越显示出其优越性。IP (Intellectual Property), 其原来的含义是指知识产权、著作权等, 在 IC 设计领域可将其理解为实现某种功能的设计, IP 核 (IP 模块) 则是指完成某种功能的设计模块。

一般将 IP 核分为硬核、固核和软核 3 种类型。软核指的是在寄存器级或门级对电路功能用 HDL 进行描述, 表现为 Verilog HDL 或 VHDL 代码, 用户在使用软核的时候可以修改, 以满足自己所需要的功能。软核主要用于接口、算法、编译码和加密等模块的设计。

硬核指的是以版图形式描述的设计模块, 它基于一定的设计工艺, 而且用户不能改动, 用户得到的硬核仅是产品的功能, 而不是产品的设计。常用的硬核有存储器、模拟器件和一些接口等。固核介于硬核和软核之间, 它允许用户重新定义关键的性能参数, 内部连线也可以重新优化。

一般软核的开发成本较低, 使用灵活, 但其可预测性差, 延时不一定能达到要求; 而硬核的开发成本相对较高, 但可预测性强, 可靠性高, 能够很快地投入使用; 固核的性能介于硬核和软核之间。

如图 1.3 所示, 由微处理器核 (MPU core)、数字信号处理器核 (DSP core)、存储器 (RAM/ROM) 核、A/D 核、D/A 核以及 USB 接口核等构成了一个系统芯片 (SOC)。

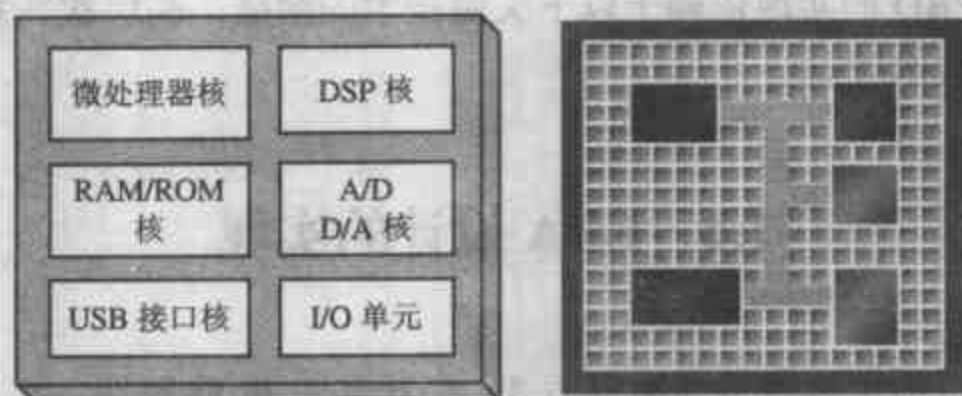


图 1.3 系统芯片 (SOC) 示意图

用户在设计一个系统时, 可以自行设计各个功能模块, 也可以用 IP 模块来构建。IP 核

作为一种商品,已经在 Internet 上广泛销售,而且还有专门的组织——虚拟插座接口协会 (Virtual Socket Interface Association) 来制定关于 IP 产品的标准与规范。作为设计者来说,想要在短时间内开发出新产品,一个比较好的方法就是使用 IP 核完成设计。

如上所述,基于 IP 复用的开发给设计者带来了诸多好处,如节省时间、缩短开发周期、避免重复劳动等。当然,IP 的发展还存在一些问题,比如 IP 版权的保护、IP 的保密及 IP 间的集成等。但基于 IP 复用的设计技术无疑会成为电子系统开发的重要手段之一。

系统芯片 (SOC),或者称为芯片系统,是指把一个完整的系统集成在一个芯片上;简单地说,就是用一个芯片实现一个功能完整的系统。

系统芯片可以采用全定制的方式来实现,把设计的网表文件提交给半导体厂家流片就可得到,但采用这种方式风险性高,费用大,周期长。还有一种方式就是采用可编程逻辑器件来实现。CPLD 和 FPGA 的集成度越来越高,速度也越来越快,设计者可以在其上通过编程完成自己的设计。今天,已不仅能用它们实现一般的逻辑功能,还可以把微处理器、DSP、存储器、标准接口等功能部件全部集成在其中,真正实现 System On a Chip。

微电子制造工艺的进步为 SOC 的实现提供了硬件基础,而 EDA 软件技术的提高则为 SOC 创造了必要的开发平台。目前,EDA 的新工具、新标准和新方法正在向着高层化发展,过去已将设计从晶体管级提高到了逻辑门级,后来又提高到了寄存器传输级,现在则越来越多地在系统级完成。

在数字系统进入 SOC 时代后,设计方法也随之产生变化。如果把器件的设计看成是设计者根据设计规则用软件来搭接已有的不同模块,那么早期的设计是基于晶体管的 (Transistor Based Design)。在这一阶段,设计者最关心的是怎样减小芯片的面积,所以又称为面积驱动的设计 (ADD, Area Driving Design)。随着设计方法的改进,出现了以门级模块为基础的设计 (GBD, Gate Based Design)。在这一阶段,设计者在考虑芯片面积的同时,更多关注门级模块之间的延时,所以这种设计又称为时延驱动的设计 (TDD, Time Driving Design)。20 世纪 90 年代以来,芯片的集成度进一步提高, SOC 的出现使得以 IP 模块复用为基础的设计逐渐流行,这种设计方法称为基于模块的设计方法 (BBD, Block Based Design)。在应用 BBD 方法进行设计的过程中,逐渐产生的一个问题是,在开发完一个产品后,怎么能尽快开发出其系列产品。这样就产生了新的概念——PBD,它是基于平台的设计方法 (Platform Based Design)。

PBD 是一种基于 IP 的、面向特定应用领域的 SOC 设计环境,可以在更短的时间内设计出满足需要的电路,PBD 的实现依赖于以下关键技术的突破:高层次、系统级的设计工具和软件、硬件协同设计技术等。

1.4 EDA 设计的实现

电子系统的实现目前主要有下面两种方式,一种基于 PLD 器件,另一种用专用集成电路 (ASIC) 来实现。这两种方式各有优点,下面分别进行介绍。

1. 可编程逻辑器件 (PLD)

20 多年来,PLD 器件有了迅速的发展,以 Altera 的 CPLD 和 Xilinx 的 FPGA 为代表的

PLD 器件不断推陈出新。1985 年, Xilinx 推出自己的第一片 FPGA, 至今, FPGA 已从最初的 1000 多个可用门发展到现在的百万门以上, 工艺尺寸也达到了 $0.15\mu\text{m}$ 的深亚微米级, 金属布线层数达到了七层以上。Altera 先后推出了 MAX 7000、MAX 9000、FLEX 10K、ACEX 1K、APEX 20K、APEX-E、Mercury Devices、Stratix Devices、Cyclone 等器件系列, 其集成度不断提高, 可用门数已达到了数百万门。在器件中, 除集成各种逻辑门和寄存器外, 还植入了嵌入式系统块 (ESB, Embedded System Block), 可以构成 RAM、ROM、FIFO (先进先出存储器) 或者 CAM (内容寻址存储器) 等存储器, 也可以用作乘积项以实现一般的逻辑功能。

采用 PLD 器件, 用户只要对它编程就可实现所需要的功能, 而且可以反复修改、反复编程, 直到完全满足要求, 具有其他方法无可比拟的方便性和灵活性。

目前的 EDA 开发软件对 PLD 器件也提供了强有力的支持, 所以用 PLD 器件实现设计是一种便捷实用的选择。

2. 专用集成电路

这里的专用集成电路 (ASIC) 是指用全定制方法来实现设计的方式, 它在最底层, 即物理版图级实现设计。采用 ASIC 能得到最高速度、最低功耗和最省面积的设计。它要求设计者必须使用版图编辑工具从晶体管的版图尺寸、位置及连线开始进行设计, 以期得到芯片的最优性能。

在运用全定制法设计芯片时, 当芯片的功能、性能、面积和成本确定后, 设计人员要对芯片结构、逻辑、电路进行精心的设计, 对不同的方案反复比较, 对单元电路的结构、晶体管的参数反复优化。在版图设计时, 设计人员要手工设计版图并精心地布局布线, 以获得最佳的性能和最小的面积。版图设计完成后, 还要进行完整的检查和验证, 包括设计规则检查、电学规则检查、连接性检查、版图与电路图一致性检查等, 全部通过后, 才能将得到的标准格式的版图文件 (一般为 CIF、GDSII 格式) 交与半导体厂家进行流片。

由此可见, 采用全定制法可以设计出高速度、低功耗、省面积的芯片, 但设计的周期长, 设计成本高, 只适用于对性能要求很高或批量很大的芯片。

以上两种实现方式各有自己的优点, 作为设计者, 应选择适合自己的方式, 才能最合理地实现所设计的系统。当然也可用一种方式实现, 等时机成熟, 再转化成别的方式, 以取得最优的性价比。

本书主要讨论基于 PLD 器件的数字系统实现方式, 这种方式方便易行, 应用广泛。

1.5 硬件描述语言

电子系统的设计主要有原理图输入法和硬件描述语言设计两种方式。图形输入法是设计规模较小的电路时经常采用的方法, 这种方法直接把设计的系统用原理图的方式表现出来, 具有直观、形象的优点, 尤其对表现层次结构、模块化结构更为方便。但图形设计方式要求设计工具提供必要的元件库, 以供调用。它适于描述连接关系和接口关系, 不适于描述逻辑功能。同时, 如果所设计系统的规模比较大, 或设计软件不能提供设计者所需的库单元时, 这种方法就显得很受限制了。而且用原理图表示的设计, 通用性、可移植性也弱一些, 所以

在现代的设计中,越来越多地采用基于硬件描述语言的设计方式。

硬件描述语言(HDL, Hardware Description Language)是一种用文本形式来描述和设计电路的语言。设计者可利用 HDL 来描述自己的设计,然后利用 EDA 工具进行综合和仿真,最后变为某种目标文件,再用 ASIC 或 FPGA 来具体实现。这种设计方法已被普遍采用。

硬件描述语言的发展至今不过 20 年左右的历史,已成功应用于系统开发的各个阶段:设计、综合、仿真、验证等。到 20 世纪 80 年代时,已出现了数 10 种硬件描述语言,它们对设计自动化起了促进和推动的作用。但是,这些语言一般面向特定的设计领域与层次,而且众多的语言使用户无所适从,因此急需一种面向多领域、多层次、并得到普遍认同的标准 HDL 语言。进入 80 年代后期,硬件描述语言向着标准化、集成化的方向发展。最终, VHDL 和 Verilog HDL 适应了这种趋势的要求,先后成为 IEEE 标准。

1. Verilog HDL 语言

Verilog HDL 语言是应用最为广泛的硬件描述语言之一,可用来进行各种层次的逻辑设计,也可以进行仿真验证、时序分析等。

Verilog HDL 适合算法级(Algorithm Level)、寄存器传输级(RTL, Register Transfer Level)、门级(Gate Level)和版图级(Layout Level)等各个层次的设计和描述,见表 1.1。

表 1.1 不同层次的描述方式

设计层次	行为描述	结构描述
系统级	系统算法	系统逻辑框图
RTL 级	数据流图、真值表、状态机	寄存器、ALU、ROM 等分模块描述
门级	布尔方程、真值表	逻辑门、触发器、锁存器构成的逻辑图
版图级	几何图形	图形连接关系

在采用 Verilog HDL 进行设计时,由于 Verilog 语言的标准化,可以很容易地把完成的设计移植到不同厂家的不同芯片中去。用 Verilog HDL 完成的设计,其信号参数也容易改变,可以任意修改,以适应不同的应用。在仿真验证时,测试向量也可用该语言描述。此外,采用 Verilog HDL 进行设计还具有工艺无关性,这使得工程师在功能设计、逻辑验证阶段可以不必过多考虑门级及工艺实现的具体细节,而只需根据系统设计的要求,施加不同的约束条件,即可设计出实际电路。

2. Verilog HDL 与 VHDL

Verilog HDL 和 VHDL 都是用于电路设计的硬件描述语言,并且都已成为 IEEE 标准。两者的共同点在于:能形式化地、抽象地表示电路的结构和行为,支持逻辑设计中层次与领域的描述,可借用高级语言的特点来简化电路的描述,具有电路仿真与验证机制以保证设计的正确,支持电路描述由高层到低层的综合和转换,便于文档管理,易于理解和移植重用。

Verilog HDL 和 VHDL 又各有自己的优势和特点。Verilog HDL 语言早在 1983 年就已推出,至今已有近 20 年的应用历史,因而拥有广泛的设计群体,其设计的资源也比 VHDL 丰富。与 VHDL 相比,Verilog HDL 的语法结构自由,也相对容易掌握一些,可使用户集中精力于设计工作中,而不必花费太多的时间在语言和语法的学习上。

思考与练习

1. 谈谈自己对 EDA 技术的认识。
2. 现代 EDA 技术的特点有哪些？
3. 什么是 Top-down 设计方式？
4. EDA 设计的实现方式有哪些？各有什么优缺点？
5. 什么是 IP 复用技术？IP 核对 EDA 技术的应用和发展有什么意义？
6. 用硬件描述语言设计电路有什么优势？
7. 解释什么是 ASIC，它有什么特点。

第 2 章 EDA 设计软件与设计流程

本章内容简介

- ◆ 集成的 CPLD/FPGA 开发工具
- ◆ 逻辑综合工具
- ◆ 仿真工具
- ◆ 版图设计工具
- ◆ 输入 (Design Input)
- ◆ 综合 (Synthesis)
- ◆ 适配 (Fitter)
- ◆ 仿真 (Simulation)
- ◆ 编程 (Program)

本章介绍常用的 EDA 设计软件和基于 FPGA/CPLD 的设计流程。有关 EDA 软件的具体使用操作将在第 13 章中介绍。

2.1 EDA 软件工具概述

EDA 软件是进行 EDA 设计必需的平台,也是 EDA 技术的核心所在,本节将根据设计的不同阶段和层次对常用的 EDA 软件工具进行介绍。

2.1.1 集成的 CPLD/FPGA 开发工具

集成的 CPLD/FPGA 开发工具大都是由 CPLD/FPGA 芯片生产厂家提供的,这些工具可以完成从设计输入(原理图或 HDL)、逻辑综合、模拟仿真到适配下载等全部工作。

常用的集成 CPLD/FPGA 开发工具见表 2.1,这些开发工具多数是将一些专业的第三方软件也集成在一起,大大提高了软件的功能。

表 2.1

常用的集成 CPLD/FPGA 开发工具

软 件	简 介
 MAX+PLUS II	MAX+PLUS II 是 Altera 的集成开发软件,使用广泛,支持 Verilog HDL、VHDL 和 AHDL 多种语言

续表

软 件	简 介
	Quartus II 是 Altera 继 MAX+PLUS II 后的新一代开发工具, 能够支持百万门级的设计, 适合大规模 FPGA 的开发。Quartus II 提供了更优化的综合和适配功能, 改善了对第三方仿真和时域分析工具的支持。 软件包含 SOPC Builder, 可自动添加、参数化和链接 IP 核, 包括嵌入处理器、协处理器、外设和用户定义逻辑等
	ISE 是 Xilinx 公司最新的 FPGA/CPLD 集成开发软件, 它提供给用户一个从设计输入到综合、布线、仿真、下载的全套解决方案, 并能很方便地同其他 EDA 工具接口。其中, 原理图输入用的是第三方软件 ECS, HDL 综合可以使用 Xilinx 公司开发的 XST、Synopsys 的 FPGA Express 和 Synplicity 的 Synplify/Synplify Pro, 测试输入是图形化的 HDL Benchner, 状态图输入用的是 StateCAD, 前、后仿真则可以使用 ModelSim XE(Xilinx Edition)或 ModelSim SE
	Foundation 是 Xilinx 公司的 PLD 开发软件
	ispLEVER 是 Lattice 公司继 ispDesignEXPERT 后的新一代集成开发工具, 该软件同时集成了许多第三方专业工具, 如综合工具 Synplify/Synplify Pro 和 Leonardo Spectrum, 仿真软件 ModelSim 等

为了提高设计效率, 优化设计结果, 很多 EDA 公司提供了各种专业软件, 用以进行更复杂和更高效的设计, 这些软件可以组合使用, 相互之间具有良好的兼容性和互操作性, 下面按功能的不同分别进行介绍。

2.1.2 输入工具 (Design Input Tools)

输入工具主要是帮助用户完成原理图和 HDL 文本的编辑和输入工作。好的输入工具应能够支持多种输入方式, 包括原理图、HDL 文本、波形图、状态机、真值表等。输入工具可帮助用户提高输入效率。多数人习惯使用集成开发软件或者综合/仿真工具中自带的文本编辑器, 也可以直接使用普通文本编辑器, 如 UltraEdit 等。

常用的设计输入工具见表 2.2。

表 2.2

常用的输入工具

软 件	简 介
HDL Designer Series	Mentor 公司的设计输入工具, 包含于 FPGA Advantage 软件中, 可以接受 HDL 文本、原理图、状态图、表格等多种设计输入方式, 并将其转化为 HDL 文本表达方式, 功能强
UltraEdit	一个通用的编辑器
HDL Turbo Writer	VHDL/Verilog HDL 专用编辑器, 可大小写自动转换, 缩进、折叠、格式编排很方便, 可直接使用 FPGA Advantage 做后端处理
Visual HDL	Innovada 公司的可视化 VHDL/Verilog HDL 编辑工具, 可以通过画流程图等可视化方法生成一部分 HDL 代码

2.1.3 逻辑综合工具 (Synthesis Tools)

逻辑综合是将设计者在 EDA 平台上编辑输入的 HDL 文本、原理图或状态图描述, 依据给定的硬件结构和约束控制条件进行编译、优化、转换和综合, 最终获得门级电路甚至更底层的电路描述网表文件的过程。

逻辑综合工具能够自动完成上述的功能, 产生优化的电路结构网表, 输出.edf 文件, 导给 CPLD/FPGA 厂家的软件进行适配和布局布线。专业的逻辑综合软件通常比 CPLD/FPGA 厂家的集成开发软件中自带的逻辑综合功能更强, 能得到更优化的结果。

最著名的用于 FPGA/CPLD 设计的 HDL 综合工具有如下 3 个:

- ◆ Synopsys 公司的 FPGA Express、FPGA Compiler 和 FPGA Compiler II。
- ◆ Synplicity 的 Synplify 和 Synplify Pro。
- ◆ Mentor 的 Leonardo Spectrum。

表 2.3 对这 3 个综合工具的性能作了介绍。

表 2.3 常用的综合工具

软 件	简 介
	Synplify/Synplify Pro 是 Synplicity 公司推出的 Verilog HDL/VHDL 综合软件, 使用广泛。Synplify Pro 除了具有原理图生成器、延时分析器外, 还带了一个 FSM Compiler (有限状态机编译器), 能从 HDL 设计文本中提出存在的 FSM 设计模块, 并用状态图的方式显示出来, 用表格来说明状态的转移条件及输出
	FPGA Compiler II 是 Synopsys 公司的 Verilog HDL/VHDL 综合软件。Synopsys 是最早推出 HDL 综合器的公司, 它改变了早先 HDL 语言只能用于电路的模拟仿真的状况。Synopsys 的综合器包括 FPGA Express、FPGA Compiler, 目前其最新的综合软件为 FPGA Compiler II
	Leonardo Spectrum 是 Mentor 的子公司 Exemplar Logic 出品的 Verilog HDL/VHDL 综合软件, 并作为 FPGA Advantage 软件的一个组成部分。 Leonardo Spectrum 可同时用于 CPLD/FPGA 和 ASIC 设计两类目标, 性能稳定

2.1.4 仿真工具 (Simulation Tools)

仿真工具提供了对设计进行模拟仿真的手段, 包括布线以前的功能仿真 (前仿真) 和布线以后包含延时的时序仿真 (后仿真)。在一些复杂的设计中, 验证仿真将会比设计本身还要耗时费力, 以至于有人认为仿真是 EDA 的精髓所在, 所以仿真器的仿真速度、仿真的准确性、易用性等成为衡量仿真器性能的重要指标。

仿真器按对设计语言的不同处理方式分为两类: 编译型仿真器和解释型仿真器。编译型仿真器的仿真速度快, 但需要预处理, 因此不能即时修改; 解释型仿真器的仿真速度要慢一些, 但可以随时修改仿真环境和仿真条件。

另外按处理的 HDL 语言类型的不同, HDL 仿真器可分为 Verilog HDL 仿真器、VHDL 仿真器和混合仿真器。混合仿真器能够同时处理 Verilog HDL 和 VHDL。

常用的 HDL 仿真软件见表 2.4。

表 2.4

常用的仿真软件

软 件	简 介
 ModelSim	ModelSim 是 Mentor 的子公司 Model Technology 的一个出色的 VHDL/Verilog HDL 混合仿真软件, 它属于编译型仿真器, 仿真速度快, 功能强
 NC-Verilog / NC-VHDL / NC-Sim Verilog-XL	这几个软件都是 Cadence 公司的 Verilog HDL/VHDL 仿真工具, 其中 NC-Verilog 的前身是著名的 Verilog 仿真软件 Verilog-XL, 用于对 Verilog HDL 程序进行仿真; NC-VHDL 用于 VHDL 仿真; 而 NC-Sim 则能够对 Verilog HDL/VHDL 进行混合仿真
 VCS / Scirocco	VCS 是 Synopsys 公司的 Verilog HDL 仿真软件, Scirocco 是 Synopsys 的 VHDL 仿真软件
Active HDL	Active HDL 是 Aldec 的 Verilog HDL/ VHDL 仿真软件, 界面友好, 简单易用

ModelSim 能够提供很好的 Verilog 和 VHDL 混合仿真; NC-Verilog 和 VCS 是基于编译技术的仿真软件, 能够胜任行为级、RTL 级和门级各种层次的仿真, 速度快; 而 Verilog-XL 是基于解释的仿真工具, 速度要慢一些。

2.1.5 IC 版图工具

著名的提供 IC 版图设计工具的公司有 Cadence、Mentor、Avant!、Synopsys 等, 这些公司各有所长, Avant! 因其模拟仿真工具 HSPICE 出名, Synopsys 的优势在于其逻辑综合工具, 而 Mentor 和 Cadence 则能够在设计的各个层次提供全套的开发工具。

在晶体管级或基本门级提供图形输入工具的有 Cadence 的 composer、Viewlogic 的 Viewdraw 等, 均可根据不同的厂家库生成与输入晶体管或门电路相对应的模拟网表。

专门应用于 IC 的综合工具有 Synopsys 的 Design Compiler 和 Behavioral Compiler; Synplicity 的 Synplify ASIC; Cadence 的 Synergy 等。

IC 仿真工具的关键在于晶体管物理模型的建立, 与实际工艺中晶体管物理特性相符的模型必然得到与实际电路更符合的工作波形。随着 IC 集成度的日益提高以及线宽的日趋缩小, 晶体管的模型也日趋复杂。任何的电路仿真都是基于一定的厂家库的, 在这些库文件中, 制造厂家为设计者提供了相应的工艺参数。

SPICE 是最著名的模拟电路仿真工具, SPICE 最早产生于 Berkley 大学, 经历了数十年的发展, 随晶体管线宽的不断缩小, SPICE 也引入了更多的参数和更复杂的晶体管模型, 使其在亚微米和深亚微米工艺的今天依旧是模拟电路仿真的主要工具之一。Avant! 的 HSPICE 因其在亚微米和深亚微米工艺中的出色表现而在近年得到了广泛的应用。Cadence 的 spectre 也是模拟仿真软件, 但应用远不及 SPICE 和 HSPICE 广泛。

此外, 还有其他一些 IC 版图工具, 如自动布局布线 (Auto Place & Route) 工具、版图输入工具、物理验证 (Physical Validate) 和参数提取 (LVS) 工具等等。

半导体集成技术还在不断地向前发展, 相应的 IC 设计工具也不断地更新换代, 以提供

对 IC 设计的全方位支持。应该说, 没有 EDA 工具, 就没有 IC。

2.1.6 其他 EDA 工具

除了上面介绍的 EDA 软件外, 应用广泛的其他一些工具见表 2.5。

近年来, 由于数字信号处理的广泛应用, 一些 EDA 公司加大了在该方面的工具支持, 比如 Altera 公司推出了 DSP Builder 工具, 它允许设计者在 MATLAB 中设计算法, 在 Simulink 中运行系统集成, 自动导出 HDL 设计文本到 Quartus II 软件中进行实现, 从而完成了从常用的 DSP 开发工具到 EDA 工具的无缝连接。Xilinx 公司则推出了 System Generator 工具, 可完成类似的功能。

表 2.5

其他 EDA 工具

软 件	简 介
	Mentor 公司推出的 Verilog HDL/VHDL 完整开发系统, 可以完成除了适配和编程以外所有的工作, 包括 3 套软件: HDL Designer Series (输入及项目管理)、Leonardo Spectrum (逻辑综合) 和 ModelSim (模拟仿真)
DSP Builder	Altera 的 DSP 开发工具, 能快速、有效地完成数字信号处理的仿真和最终 FPGA 实现, 实现了常用的 DSP 开发工具 (如 MATLAB) 到 EDA 工具 (Quartus II) 的无缝连接
System Generator	Xilinx 的 DSP 开发工具, 实现 ISE 与 MATLAB 的接口, 能快速、有效地完成数字信号处理的仿真和最终 FPGA 实现
SOPC Builder	配合 QuartusII, 可以完成系统集成、IP 复用、系统验证等工作

2.2 EDA 设计的流程

总的来说, 要实现一个数字系统, 有两种常用的方式, 一种基于 PLD 器件, 另外一种则采用 ASIC 来实现。具体采用什么方式实现, 需要根据实际情况来决定。对于一般的设计开发而言, 采用 PLD 器件来实现, 是一种设计周期短、投入资金少、风险小的选择。对于一些成熟的设计来说, 可以考虑把系统中的某些模块, 甚至于整个系统, 采用专用芯片 (ASIC) 的形式实现之。

本节主要讨论基于 FPGA/CPLD 器件实现数字系统的步骤与要点。

基于 FPGA/CPLD 器件的 EDA 设计流程如图 2.1 所示, 主要包括设计输入、综合、FPGA/CPLD 适配、仿真和编程下载等步骤。

2.2.1 输入 (Design Input)

设计输入是设计者将所要设计的电路以开发软件要求的某种形式表达出来, 并输入到相应软件中的过程。设计输入有多种表达方式, 最常用的是原理图方式和 HDL 文本方式两种。

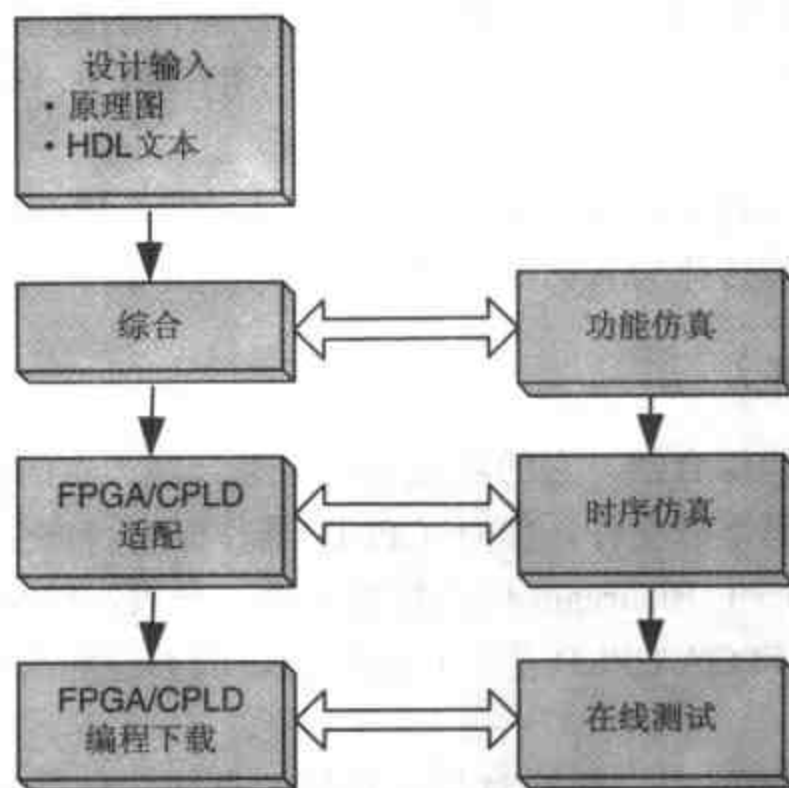


图 2.1 基于 FPGA/CPLD 的 EDA 设计流程

1. 原理图输入

原理图是图形化的表达方式，使用元件符号和连线等形式来描述设计。其特点是适合描述连接关系和接口关系，而描述逻辑功能则比较繁琐。原理图输入对用户来讲很直观，尤其对表现层次结构、模块化结构更为方便。但它要求设计工具提供必要的元件库或逻辑宏单元。如果输入的是较为复杂的逻辑或者元件库中不存在的模型，采用原理图输入方式往往很不方便。此外，原理图方式的设计可重用性、可移植性也差一些。

2. HDL 文本输入

用 HDL 文本来描述设计，其逻辑描述能力强，但描述接口和连接关系则不如图形方式直观。流行的硬件描述语言有 Verilog HDL 和 VHDL 等，Verilog HDL 和 VHDL 的功能比较强，属于行为描述语言，能描述和仿真复杂的逻辑设计，并且都已被采用为国际标准，被绝大多数的 EDA 工具所支持。

PLD 器件的设计往往采用层次化的设计方法，分模块、分层次地进行设计描述。描述器件总体功能的模块放置在最上层，称为顶层设计；描述器件最基本功能的模块放置在最下层，称为底层设计。顶层和底层之间的关系类似于软件中的主程序和子程序之间的关系。层次化设计的方法比较自由，可以在任何层次使用原理图或硬件描述语言来进行描述。一般的做法是：在顶层设计中，使用图形法表达连接关系和芯片内部逻辑到管脚的接口；在底层设计中，使用硬件描述语言描述各个模块的逻辑功能。

2.2.2 综合 (Synthesis)

综合 (Synthesis) 是一个很重要的步骤，综合指的是将较高层次的设计描述自动转化为较低层次描述的过程。综合有下面几种形式。

◆ 从算法表示、行为描述转换到寄存器传输级 (RTL)，即从行为描述到结构描述，称为行为综合。

◆ RTL 级描述转换到逻辑门级 (可包括触发器)，称为逻辑综合。

◆ 从逻辑门表示转换到版图表示, 或转换到 PLD 器件的配置网表表示, 称为版图综合或结构综合。根据版图信息能够进行 ASIC 生产, 有了配置网表, 可完成基于 PLD 器件的系统实现。

综合器就是能够自动实现上述转换的软件工具。或者说, 综合器是能够将原理图或 HDL 语言表达或描述的电路功能转化为具体的电路结构网表的工具。

2.2.3 适配 (Fitter)

适配器有时也称为结构综合器, 它的功能是将由综合器产生的网表文件配置于指定的目标器件中, 并产生最终的可下载文件, 如对 CPLD 器件而言, 产生熔丝图文件, 即 JEDEC 文件; 对 FPGA 器件, 则产生 Bitstream 位流数据文件。一般, 综合器可由第三方的 EDA 公司提供, 而适配则只能由 FPGA/CPLD 生产厂家提供, 因为适配的过程直接与器件的具体结构相对应。

利用适配器将综合后的网表文件针对某一具体的目标器件进行逻辑映射操作, 包括底层器件配置、逻辑分割、逻辑优化、布局布线等。映射是把设计分为多个适合器件内部逻辑资源实现的逻辑小块的过程。如针对 Lattice 的 ispLSI 器件的设计, 要适合用 GLB 和 IOC 实现; 针对 Xilinx 器件的设计, 要适合用 CLB 和 IOB 实现; 针对 Altera 的 PLD 器件的设计, 要适合用 LC 实现。布局是将已分割的逻辑小块放到器件内部逻辑资源的具体位置, 并使它们易于连线, 且连线最少; 布线则是利用器件的布线资源完成各功能块之间和反馈信号之间的连接。

适配器产生如下一些重要的文件:

- ① 适配报告: 包括芯片内部资源的利用情况、设计的布尔方程描述情况等。
- ② 面向其他 EDA 工具的输出文件, 如 EDIF 格式文件等。
- ③ 适配后的仿真模型, 包括延时信息等, 以便于进行精确的时序仿真, 因为已经得到器件的实际硬件特性(如时延特性), 所以仿真结果能比较精确地预测未来芯片的实际性能。如果仿真结果达不到设计要求, 就需要修改源代码或选择不同速度的器件, 直至满足设计要求。
- ④ 器件编程文件: 如用于 CPLD 编程的 JEDEC、POF 等格式的文件; 用于 FPGA 配置的 SOF、JAM、BIT 等格式的文件。

2.2.4 仿真 (Simulation)

仿真, 也称为模拟, 是对所设计电路的功能验证。用户可以在设计的过程中对整个系统和各个模块进行仿真, 即在计算机上用软件验证功能是否正确, 各部分的时序配合是否准确。如果有问题, 可以随时进行修改, 从而避免了逻辑错误。高级的仿真软件还可以对整个系统设计的性能进行估计。规模越大的设计, 越需要进行仿真。仿真不消耗资源, 不浪费时间, 可避免不必要的损失。

仿真包括功能仿真和时序仿真。不考虑信号时延等因素的仿真称为功能仿真, 又叫前仿真; 时序仿真又称后仿真, 它是在选择了具体器件并完成了布局布线后进行的包含定时关系的仿真。由于不同器件的内部时延不一样, 不同的布局、布线方案也给延时造成了很大的影响, 因此在设计实现后, 对网络和逻辑块进行时延仿真, 分析定时关系, 估计设计性能是非

常有必要的。

2.2.5 编程 (Program)

把适配后生成的编程文件装入到 PLD 器件中的过程称为下载。通常将对基于 E²PROM 工艺的非易失结构 PLD 器件的下载称为编程 (Program)，而将基于 SRAM 工艺结构的 PLD 器件的下载称为配置 (Configure)。编程需要满足一定的条件，如编程电压、编程时序和编程算法等。有两种常用的编程方式：在系统编程 (ISP, In-System Programmable) 和用专用的编程器编程，现在的 PLD 器件一般都支持在系统编程，因此在设计数字系统和做 PCB 印制板时，应预留器件的下载接口。

思考与练习

1. 结合自己的使用情况谈谈对 EDA 工具的认识。
2. 基于 CPLD/FPGA 的 EDA 设计流程包括哪些步骤？
3. 什么是综合？常用的综合工具有哪些？
4. 什么是适配？
5. 功能仿真与时序仿真有什么区别？

第3章 Verilog HDL 设计初步

本章内容简介

- ◆ Verilog 语言的发展
- ◆ Verilog 语言与 C 语言的比较
- ◆ 4 位全加器的设计过程
- ◆ 4 位计数器的设计过程
- ◆ Verilog 模块的结构
- ◆ 逻辑功能的定义

本章通过数个简单完整的 Verilog HDL 设计示例,使读者对用 Verilog 语言进行电路设计有一个初步的了解,并对模块结构以及一些基本的语法现象进行介绍。力图通过本章使读者能迅速地从总体上把握 Verilog HDL 程序的基本结构和基本特点,达到快速入门的目的。

3.1 Verilog 语言的历史及与 C 语言的比较

Verilog HDL 是一种应用广泛的硬件描述语言,可用于从算法级、门级到开关级的多种抽象层次的数字系统设计。

Verilog HDL 是 1983 年由 GDA (Gateway Design Automation) 公司的 Phil Moorby 首创的,之后 Moorby 又设计了 Verilog-XL 仿真器,Verilog-XL 仿真器大获成功,也使得 Verilog HDL 得到推广使用。1989 年, Cadence 收购了 GDA; 1990 年, Cadence 公开发表了 Verilog HDL, 并成立了 OVI (Open Verilog International) 组织专门负责 Verilog HDL 的发展。由于这种语言具有简洁、高效、易学易用、功能强等优点,因此逐渐为众多设计者接受和喜爱。Verilog 于 1995 年成为 IEEE 标准,即 IEEE Standard 1364-1995。

Verilog 语言是在 C 语言的基础上发展而来的。从语法结构上看, Verilog 语言与 C 语言有许多相似之处,继承和借鉴了 C 语言的很多语法结构,表 3.1 中列举了两种语言中一些相同或相似的语句。从表 3.2 则可以看出, Verilog 语言与 C 语言的运算符几乎完全相同。当然, Verilog HDL 作为一种硬件描述语言,与 C 语言还是有着本质的区别的。

表 3.1 Verilog 与 C 语言的比较

C 语言	Verilog 语言
function	module,function
if-then-else	if-then-else
for	for
while	while
case	case
break	break
define	define
printf	printf
int	int

Verilog HDL 还具有下述特点。

- ◆ 既能进行面向综合的电路设计，也可用于电路的模拟仿真。
- ◆ 能够在多个层次上对所设计的系统加以描述，从开关级、门级、寄存器传输级（RTL）到行为级等，都可以胜任；设计的规模是任意的，语言不对设计的规模施加任何限制。
- ◆ Verilog 语言具有混合建模能力，即在一个设计中，各个模块可以在不同的设计层次上建模和描述。
- ◆ 灵活多样的电路描述风格，可进行行为描述，也可进行结构描述或者数据流描述。
- ◆ Verilog HDL 的行为描述语句，如条件语句、赋值语句和循环语句等，类似于软件高级语言，便于学习和使用。

表 3.2 Verilog 与 C 语言运算符的比较

C 语言	Verilog 语言	功 能
+	+	加
-	-	减
*	*	乘
/	/	除
%	%	取模
!	!	逻辑非
&&	&&	逻辑与
		逻辑或
>	>	大于
<	<	小于
>=	>=	大于等于
<=	<=	小于等于
==	==	等于
!=	!=	不等于
~	~	取反

续表

C 语言	Verilog 语言	功 能
&	&	按位与
		按位或
^	^	按位异或
<<	<<	左移
>>	>>	右移
?:	?:	等同于 if-else

◆ 内置各种基本逻辑门，如 and、or 和 nand 等，可方便地进行门级结构描述。

◆ 内置各种开关级元件，如 pmos、nmos 和 cmos 等，可进行开关级的建模。

◆ 用户定义原语（UDP）创建的灵活性。用户定义的原语既可以是组合逻辑，也可以是时序逻辑。

◆ 可通过编程语言接口（PLI）机制进一步扩展 Verilog 语言的描述能力。PLI 是允许外部函数访问 Verilog 模块内信息，并允许设计者与模拟器交互的例程集合。

另外，Verilog HDL 易学易用，可以使设计者将主要精力放在系统设计上，而不至于在语言和语法的学习上耗时费力。

Verilog HDL 的功能强，可满足各个层次设计人员的需要。正是以上优良的性能，使得它广泛流行。在 ASIC 设计领域，Verilog HDL 一直都是事实上的标准。

3.2 完整的 Verilog HDL 设计

首先来看两个简单的 Verilog HDL 程序。

3.2.1 4 位全加器和 4 位计数器

1. 4 位全加器的 Verilog HDL 源代码

【例 3.1】4 位全加器

```

module adder4(cout,sum,ina,inb,cin);
output [3:0] sum;
output cout;
input [3:0] ina,inb;
input cin;
    assign {cout,sum}=ina+inb+cin;
endmodule

```

2. 4 位计数器的 Verilog HDL 源代码

【例 3.2】4 位计数器

```

module count4(out,reset,clk);
output [3:0] out;

```



```

input  reset,clk;
reg[3:0] out;
always @(posedge clk)
begin
    if (reset)      out<=0;      //同步复位
    else      out<=out+1;      //计数
end
endmodule

```

从上面的例子可以看出, Verilog 程序具有以下一些共性。

- (1) Verilog 程序是由模块构成的。每个模块的内容都嵌在 module 和 endmodule 两个关键字之间; 每个模块实现特定的功能; 模块是可以进行层次嵌套的。
- (2) 每个模块首先要进行端口定义, 并说明输入端口和输出端口 (input、output 或 inout), 然后对模块的功能进行逻辑描述。
- (3) Verilog 程序书写格式自由, 一行可以写几个语句, 一个语句也可以分多行写。
- (4) 除了 endmodule 等少数语句外, 每个语句的最后必须有分号。
- (5) 可以用 /*.....*/ 或 //..... 对 Verilog 程序作注释。好的源程序都应当加上必要的注释, 以增强程序的可读性和可维护性。

3.2.2 综合

将 4 位全加器和 4 位计数器的源代码分别进行综合, 所用的综合器为 Synplify Pro, Synplify Pro 的使用请参考第 13 章的相关内容。图 3.1 所示是 4 位全加器 RTL 级综合的结果, 图 3.2 所示是 4 位计数器 RTL 级综合的结果。

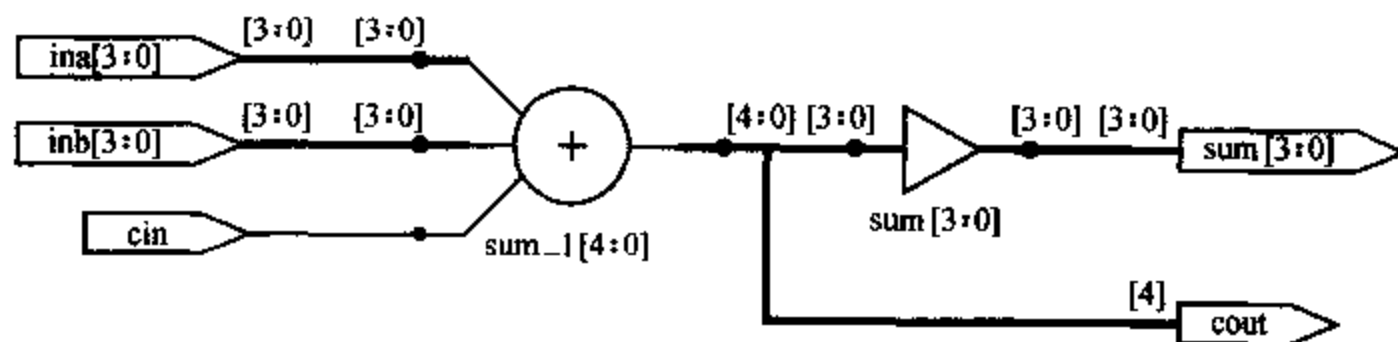


图 3.1 4 位全加器 RTL 级综合的结果

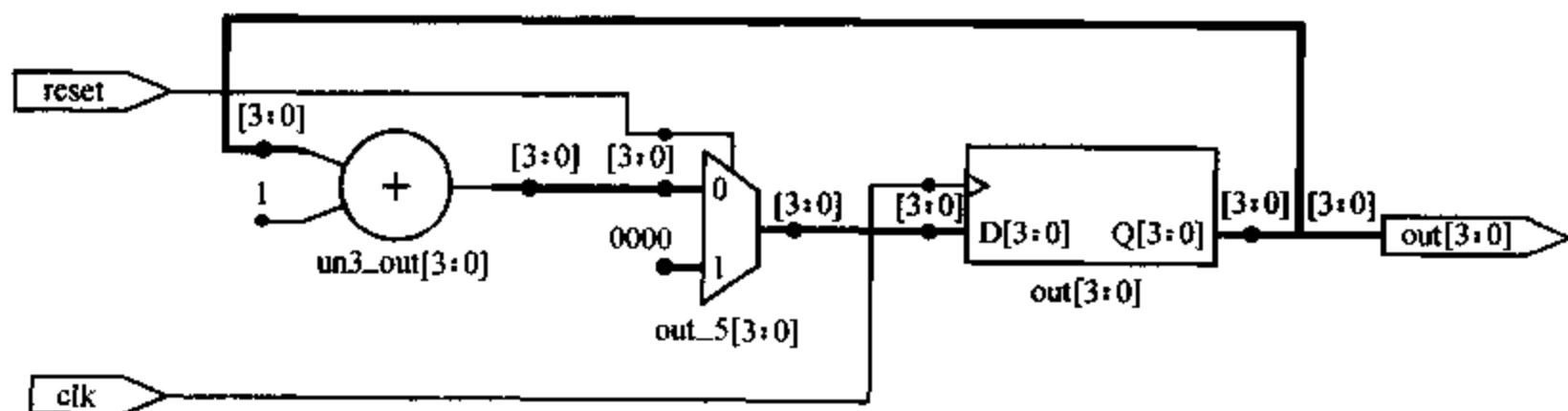


图 3.2 4 位计数器 RTL 级综合的结果

图 3.3 所示是 4 位全加器的门级综合视图, 图 3.4 所示是 4 位计数器的门级综合视图(选择的器件都是 Altera 的 EPF10K10)。

3.2.3 仿真

为了对 4 位全加器和 4 位计数器的功能进行验证, 需对其进行仿真。首先必须编写仿真程序, 下面是 4 位加法器的测试程序(有关仿真程序的编写, 可参考第 8 章的相关内容)。

【例 3.3】4 位全加器的仿真程序

```
'timescale 1ns/1ns
'include "adder4.v"

module adder_tp;                                //测试模块的名字
reg[3:0] a,b;                                   //测试输入信号定义为 reg 型
reg cin;
wire[3:0] sum;                                  //测试输出信号定义为 wire 型
wire cout;
integer i,j;
adder4 adder(sum,cout,a,b,cin);                //调用测试对象

always #5 cin=~cin;                             //设定 cin 的取值

initial
begin
a=0;b=0;cin=0;
for(i=1;i<16;i=i+1)
#10 a=i;                                         //设定 a 的取值
end

initial
begin
for(j=1;j<16;j=j+1)
#10 b=j;                                         //设定 b 的取值
end

initial                                         //定义结果显示格式
begin
$monitor($time,, "%d + %d + %b = {%b,%d}",a,b,cin,cout,sum);
#160 $finish;
end
endmodule
```

将上面的测试程序用 ModelSim 软件进行编译和仿真(ModelSim 软件的使用请参考第 13 章的相关内容), 如图 3.5 所示是 4 位全加器仿真输出的波形。

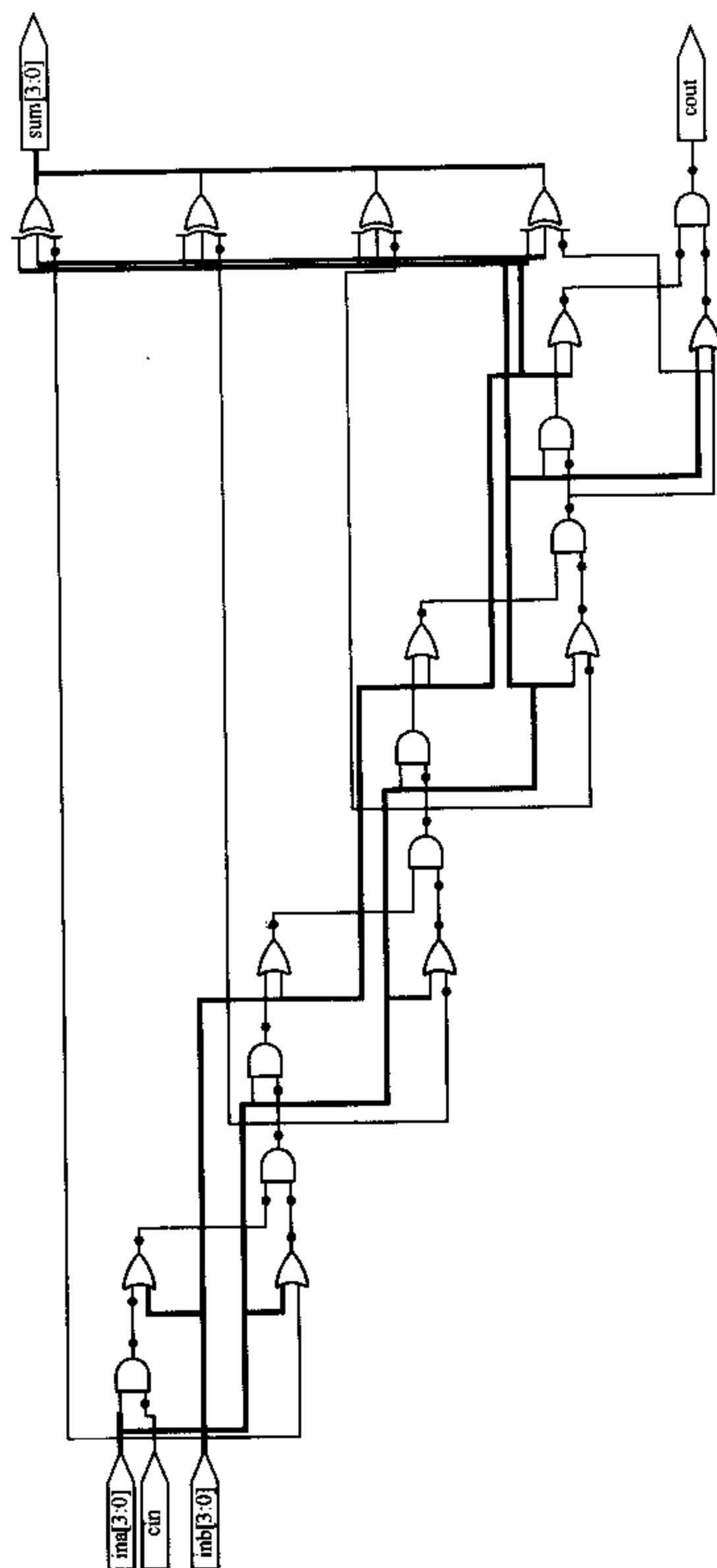


图 3.3 4 位全加器门级综合视图

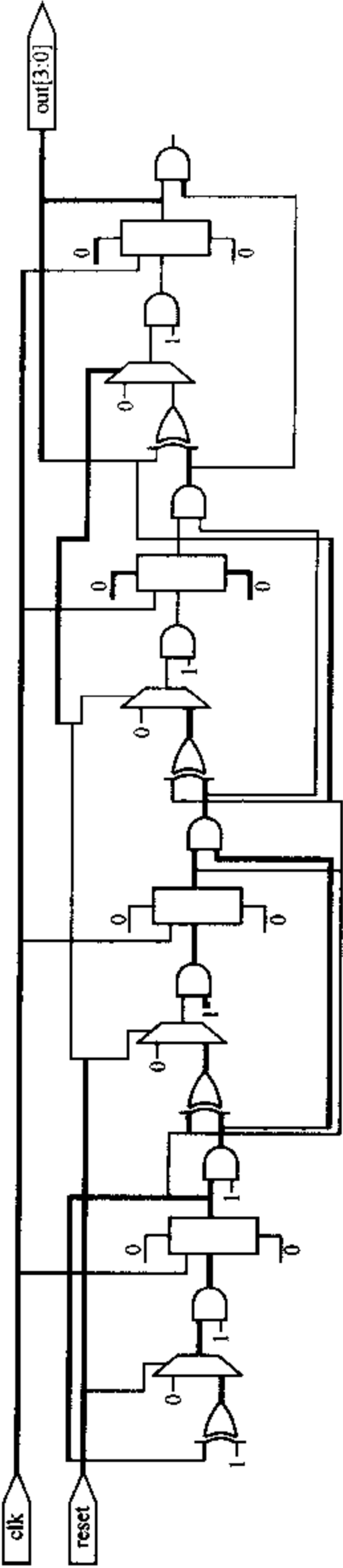


图 3.4 4 位计数器门级综合视图



图 3.5 4 位全加器的功能仿真结果

文本方式的输出结果如下所示。

0	0 + 0 + 0 = {0, 0}	80	8 + 8 + 0 = {1, 0}
5	0 + 0 + 1 = {0, 1}	85	8 + 8 + 1 = {1, 1}
10	1 + 1 + 0 = {0, 2}	90	9 + 9 + 0 = {1, 2}
15	1 + 1 + 1 = {0, 3}	95	9 + 9 + 1 = {1, 3}
20	2 + 2 + 0 = {0, 4}	100	10 + 10 + 0 = {1, 4}
25	2 + 2 + 1 = {0, 5}	105	10 + 10 + 1 = {1, 5}
30	3 + 3 + 0 = {0, 6}	110	11 + 11 + 0 = {1, 6}
35	3 + 3 + 1 = {0, 7}	115	11 + 11 + 1 = {1, 7}
40	4 + 4 + 0 = {0, 8}	120	12 + 12 + 0 = {1, 8}
45	4 + 4 + 1 = {0, 9}	125	12 + 12 + 1 = {1, 9}
50	5 + 5 + 0 = {0, 10}	130	13 + 13 + 0 = {1, 10}
55	5 + 5 + 1 = {0, 11}	135	13 + 13 + 1 = {1, 11}
60	6 + 6 + 0 = {0, 12}	140	14 + 14 + 0 = {1, 12}
65	6 + 6 + 1 = {0, 13}	145	14 + 14 + 1 = {1, 13}
70	7 + 7 + 0 = {0, 14}	150	15 + 15 + 0 = {1, 14}
75	7 + 7 + 1 = {0, 15}	155	15 + 15 + 1 = {1, 15}

下面是 4 位计数器的仿真程序。

【例 3.4】4 位计数器的仿真程序

```
'timescale 1ns/1ns
'include "count4.v"
module coun4_tp;
    reg clk, reset;
    wire [3:0] out;
    parameter DELY=100;

    count4 mycount(out, reset, clk);

    always #(DELY/2) clk = ~clk;
    initial
    begin
        clk = 0; reset = 0;
        #DELY reset = 1;
    end
```

//测试输入信号定义为 reg 型
//测试输出信号定义为 wire 型
//调用测试对象
//产生时钟波形
//激励信号定义

```

#DELY    reset=0;
#(DELY*20) $finish;

end

//定义结果显示格式
initial $monitor($time,,,"clk=%d reset=%d out=%d", clk, reset,out);
endmodule

```

将上面的仿真程序用 ModelSim 软件进行编译和仿真,如图 3.6 所示是 4 位计数器功能仿真波形图的一部分。

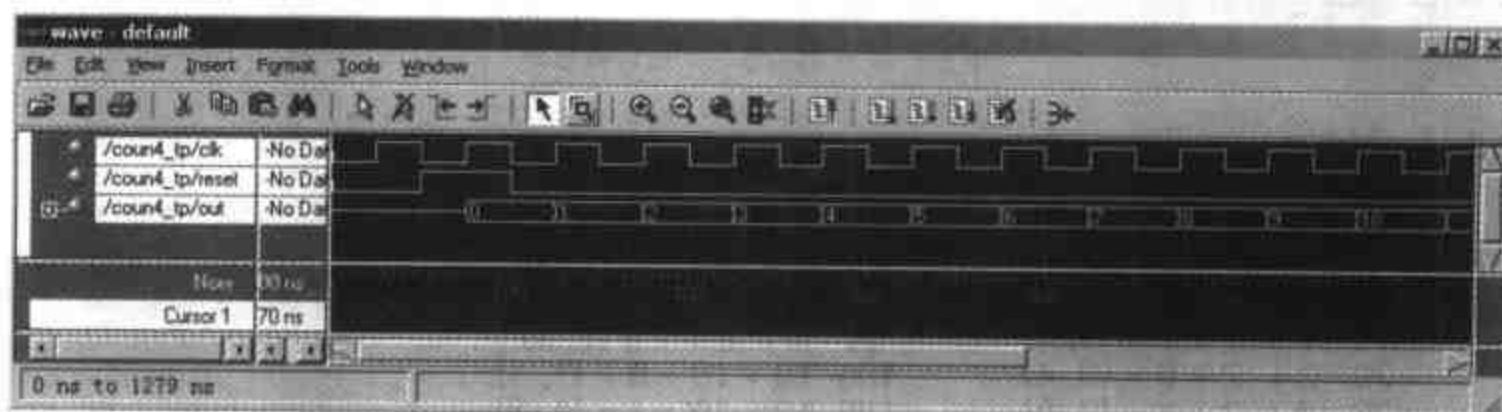


图 3.6 4 位计数器的功能仿真部分波形

文本方式的输出结果如下所示 (部分结果)。

```

0 clk=0 reset=0 out= x
50 clk=1 reset=0 out= x

100 clk=0 reset=1 out= x
150 clk=1 reset=1 out= 0

200 clk=0 reset=0 out= 0
250 clk=1 reset=0 out= 1

300 clk=0 reset=0 out= 1
350 clk=1 reset=0 out= 2

400 clk=0 reset=0 out= 2
450 clk=1 reset=0 out= 3

500 clk=0 reset=0 out= 3
550 clk=1 reset=0 out= 4

600 clk=0 reset=0 out= 4
650 clk=1 reset=0 out= 5

700 clk=0 reset=0 out= 5
750 clk=1 reset=0 out= 6

800 clk=0 reset=0 out= 6
850 clk=1 reset=0 out= 7

900 clk=0 reset=0 out= 7
950 clk=1 reset=0 out= 8

1000 clk=0 reset=0 out= 8
1050 clk=1 reset=0 out= 9

1100 clk=0 reset=0 out= 9
1150 clk=1 reset=0 out=10

.....

```

本节用一个组合电路和一个时序电路的例子介绍了 Verilog HDL 程序设计、综合和仿真的全过程。在后面的章节中将对这些过程进行更为详细的介绍。

3.3 Verilog 模块基本结构剖析

本节对 Verilog 模块的基本结构和基本语法进行剖析。

3.3.1 Verilog 模块的结构

Verilog 程序的基本设计单元是“模块”(module)。一个模块由几个部分组成,下面通过一个具体的示例来对模块的基本结构进行剖析。

如图 3.7 所示是一个简单的“与—或—非”门电路。

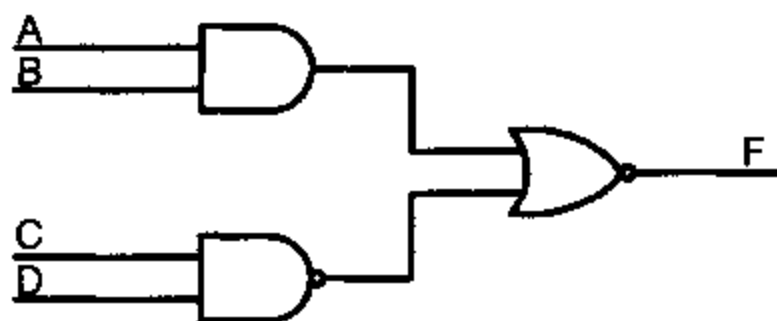


图 3.7 “与—或—非”门电路

该门电路表示的逻辑函数可以写为： $F = \overline{AB + CD}$ 。

用 Verilog 语言对该门电路描述如下。

【例 3.5】“与—或—非”门电路

```
module AOI (A,B,C,D,F);           //模块名为 AOI(端口列表 A, B, C, D, F)
input  A,B,C,D;                   //模块的输入端口为 A, B, C, D
output F;                         //模块的输出端口为 F
wire  A,B,C,D,F;                 //定义信号的数据类型
    assign F = ~(A&B | C&D);     //逻辑功能描述
endmodule
```

将上面的 Verilog 程序与前面的原理图进行对照,可以对 Verilog 程序模块有一个比较直观的认识。该程序的第 1 行为模块的名字、模块的端口列表;第 2、3 行为输入、输出端口声明;第 4 行定义了端口的数据类型;在第 5 行中对输入、输出信号间的逻辑关系进行了描述。

Verilog 模块的基本结构如图 3.8 所示。

Verilog 模块结构完全嵌在 module 和 endmodule 关键字之间,每个 Verilog 程序包括 4 个主要部分:模块声明、端口定义、信号类型说明和逻辑功能描述。

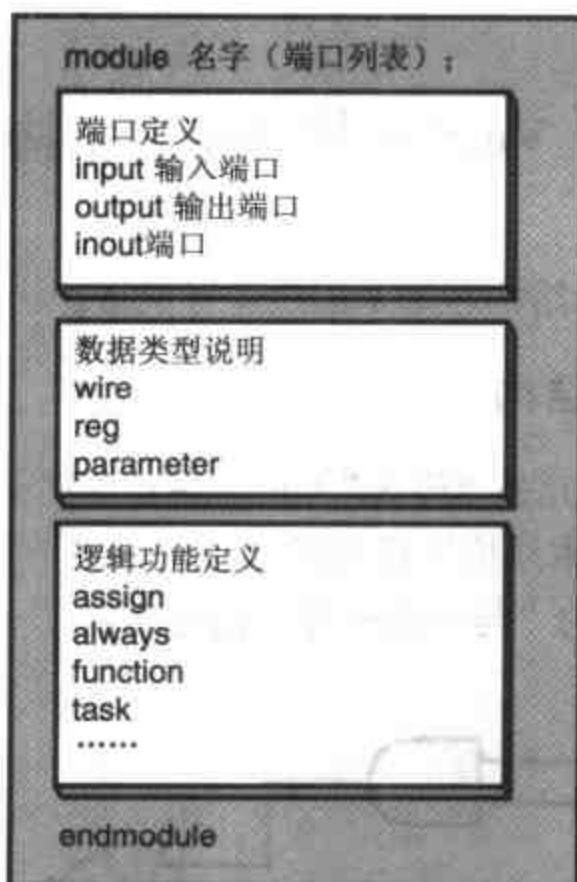


图 3.8 Verilog 模块的基本结构

1. 模块声明

模块声明包括模块名字和模块输入、输出端口列表。其格式如下：

module 模块名 (端口 1, 端口 2, 端口 3,) ;

模块结束的标志为关键字: **endmodule**。

2. 端口 (Port) 定义

对模块的输入、输出端口要明确说明，其格式为：

input 端口名 1, 端口名 2, 端口名 N; //输入端口

output 端口名 1, 端口名 2, 端口名 N; //输出端口

inout 端口名 1, 端口名 2, 端口名 N; //输入/输出端口

端口是模块与外界或其他模块连接和通信的信号线，图 3.9 中对一个模块的端口进行了图示。有 3 种端口类型，分别是输入端口 (**input**)、输出端口 (**output**) 和输入/输出端口 (**inout**)。



图 3.9 模块的端口示意图

对于端口应注意以下几点：

- ◆ 每个端口除了要声明是输入、输出还是双向端口外，还要声明其数据类型，是连线

型 (wire) 还是寄存器型 (reg), 如果没有声明, 则综合器将其默认为是 wire 型。

- ◆ 输入和双向端口不能声明为寄存器型。
- ◆ 在测试模块中不需要定义端口。

3. 信号类型声明

对模块中所用到的所有信号 (包括端口信号、节点信号等) 都必须进行数据类型的定义。

Verilog 语言提供了各种信号类型, 分别模拟实际电路中的各种物理连接和物理实体。

下面是定义信号数据类型的几个例子:

```
reg cout;                //定义信号 cout 的数据类型为 reg 型
reg[3:0] out;            //定义信号 out 的数据类型为 4 位 reg 型
wire A,B,C,D,F;          //定义信号 A,B,C,D,F 为 wire (连线) 型
```

如果信号的数据类型没有定义 (缺省), 则综合器将其默认为是 wire 型。关于数据类型的更详细的说明可参考第 4 章。

4. 逻辑功能定义

模块中最核心的部分是逻辑功能定义。有多种方法可在模块中描述和定义逻辑功能, 还可以调用函数 (function) 和任务 (task) 来描述逻辑功能。下面介绍定义逻辑功能的几种基本方法。

3.3.2 逻辑功能定义

1. 用 “assign” 持续赋值语句定义

如: assign F = ~((A&B)|(C&D));

“assign” 语句一般用于组合逻辑的赋值, 称为持续赋值方式。这种方法简单, 只需将逻辑表达式放在关键字 “assign” 后即可。

2. 调用元件 (元件例化)

调用元件的方法类似于在电路图输入方式下调入图形符号来完成设计, 这种方法侧重于电路的结构描述。在 Verilog 语言中, 可通过调用如下元件的方式来描述电路的结构:

- ◆ 调用 Verilog 内置门元件 (门级结构描述)
- ◆ 调用开关级元件 (开关级结构描述)
- ◆ 用户定义元件 UDP (也在门级)

在多层次结构的电路设计中, 不同模块的调用也可以认为是结构描述。下面是元件调用的例子:

```
and myand3(out,a,b,c);    //调用门元件, 定义了一个三输入的与门
and c2 (out,in1,in2);     //二输入与门, 名字为 c2
```

3. 用 “always” 过程块赋值

在例 3.2 的计数器模块中:

```
always @(posedge clk)     //每当 clk 上升沿到来时执行一遍 begin-end 块内的语句
begin
    if(reset) out=0;
    else    out=out+1;
```

end

上面的代码用“always”块来描述逻辑功能，一般称其为行为描述方式，这种方法一般多用于描述时序逻辑。这个例子中用“always”过程生成了一个带有同步复位的计数器。在“always”过程块中可用很多种行为语句来表达逻辑，如本例中就用了 if-else 语句来表达逻辑关系。

综上所述，可给出 Verilog HDL 模块的模板如下（这里主要指用于逻辑综合的模块结构，用于逻辑模拟的模块结构请参见第 8 章）：

```

module <顶层模块名> (<输入输出端口列表>);
output 输出端口列表;           //输出端口声明
input 输入端口列表;           //输入端口声明
/*定义数据、信号的类型，任务、函数声明，用关键字 wire、reg、task、function 等定义*/
wire 信号名;
reg 信号名;

//逻辑功能定义
assign <结果信号名>=<表达式>; //使用 assign 语句定义逻辑功能

//用 always 块描述逻辑功能
always @ (<敏感信号表达式>)
begin
    //过程赋值
    //if-else
    //case 语句
    //while, repeat, for 循环语句
    //task, function 调用
end

//调用其他模块
<调用模块名 module_name > <例化模块名> (<端口列表 port_list >);
//门元件例化
    门元件关键字 <例化门元件名> (<端口列表 port_list>);
endmodule

```

后面章节中将分别对模块的各个部分作详细说明。

思考与练习

1. 用 Verilog 设计一个简单的门电路。

2. 用 Verilog 设计一个 8 位加法器，并进行综合和仿真，查看综合结果和仿真结果。
3. 用 Verilog 设计一个 8 位计数器，并进行综合和仿真，查看综合结果和仿真结果。
4. 对例 3.5 进行综合，并查看综合结果。
5. 简要总结用 Verilog 进行电路设计的过程。

第4章 Verilog HDL 语言要素

本章内容简介

- ◆ 词法
- ◆ 空白符 (White space)
- ◆ 注释 (Comments)
- ◆ 数字 (Numbers)
- ◆ 字符串 (Strings)
- ◆ 操作符 (Operators)
- ◆ 标识符 (Identifiers)
- ◆ 关键字 (Keywords)
- ◆ 数据类型 (Data Type)
- ◆ parameter
- ◆ 寄存器和存储器

本章介绍 Verilog 语言的基本要素, 包括数字、字符串、标识符、运算符、数据类型和保留字等, 这些语言要素既有与软件编程语言 (如 C 语言) 相同和相似之处, 也有 Verilog HDL 作为一种硬件描述语言所特有的地方。比如: Verilog 语言的运算符与 C 语言的很相像, 而数据类型 (wire、reg 等) 则是 Verilog HDL 所特有的。必须深入理解硬件描述语言与软件编程语言的本质区别。

4.1 词 法

Verilog HDL 程序是由各种符号流构成的, 这些符号包括空白符 (White space)、操作符 (Operators)、数字 (Numbers)、字符串 (Strings)、注释 (Comments)、标识符 (Identifiers) 和关键字 (Keywords) 等, 我们将其称为词法, 并分别予以介绍。

4.1.1 空白符和注释

1. 空白符 (White space)

在 Verilog HDL 代码中, 空白符包括空格、tab、换行和换页。空白符使代码错落有致, 阅读起来更方便。在综合时, 空白符被忽略。

Verilog 程序可以不分行，也可以加入空白符采用多行编写。

例如：`initial begin ina = 3' b001; inb= 3' b011; end`

这段程序等同于下面的书写格式：

```
initial
    begin                //加入空格、换行等，使代码错落有致，提高可读性
        ina= 3' b001;
        inb= 3' b011;
    end
```

2. 注释 (Comments)

在 Verilog HDL 中有两种形式的注释。

◆ 单行注释：以“//”开始到本行结束，不允许续行。

◆ 多行注释：多行注释以“/*”开始，到“*/”结束。

4.1.2 数字与字符串 (Numbers & Strings)

Verilog HDL 有下面 4 种基本的逻辑状态：

- ◆ 0：低电平、逻辑 0 或逻辑非；
- ◆ 1：高电平、逻辑 1 或“真”；
- ◆ x 或 X：不确定或未知的逻辑状态；
- ◆ z 或 Z：高阻态。

Verilog HDL 中的常量是由以上这 4 类基本值组成的。x 和 z 都不区分大小写，也就是说，值 0x1z 与值 0X1Z 是等同的。

Verilog HDL 中有 3 种类型的常量：

- ◆ 整数；
- ◆ 实数；
- ◆ 字符串。

1. 整数 (Integer)

整数按如下方式书写：`+/-<size> '<base><value>`

即`+/-<位宽>'<进制><数字>`

size 为对应二进制数的宽度；base 为进制；value 是基于进制的数字序列。其中，进制有如下 4 种表示形式：

- ◆ 二进制 (b 或 B)；
- ◆ 十进制 (d 或 D 或缺省)；
- ◆ 十六进制 (h 或 H)；
- ◆ 八进制 (o 或 O)。

另外，在书写中，十六进制中的 a 到 f 与值 x 和 z 一样，不区分大小写。

下面是一些合法的整数的例子：

```
8'b11000101    //位宽为 8 位的二进制数 11000101
8'hd5          //位宽为 8 位的十六进制数 d5
5'o27          //5 位八进制数 27
```



```

4'D2           //4 位十进制数 2
4'B1x_01       //4 位二进制数 1x01
5'Hx           //5 位十六进制数 x (扩展的 x), 即 xxxxx
4'hZ           //4 位十六进制数 z, 即 zzzz
8'h 2 A        //在位宽和字符之间, 以及进制和数值之间允许出现空格

```

下面是一些不正确的书写整数的例子:

```

4'd-4          //非法: 数值不能为负, 有负号应放最左边
3' b001        //非法: ' 和进制 b 之间不允许出现空格
(3+2)'b10      //非法: 位宽不能为表达式

```

在书写和使用数字中需注意下面一些问题:

① 在较长的数之间可用下划线分开, 如 16'b1010_1101_0010_1001。

下划线符号 “_” 可以随意用在整数或实数中, 它们本身没有意义, 只是用来提高可读性; 但下划线符号不能用作首字符。

② 当数字不说明位宽时, 默认值为 32 位。

③ x (或 z) 在二进制中代表 1 位 x (或 z), 在八进制中代表 3 位 x (或 z), 在十六进制值中代表 4 位 x (或 z), 其代表的宽度取决于所用的进制。

例如:

```

8'b1001xxxx    //等价于 8'h9x
8'b1010zzzz    //等价于 8'haz

```

④ 如果没有定义一个整数的位宽, 其宽度为相应值中定义的位数。

例如:

```

'o721          //9 位八进制数
'hAF           //8 位十六进制数

```

⑤ 如果定义的位宽比实际的位数长, 通常在左边填 “0” 补位。但如果数最左边一位为 x 或 z, 就相应地用 x 或 z 在左边补位。

例如:

```

10'b10         //左边补 0, 0000000010
10'bx0x1       //左边补 x, xxxxxxxx0x1

```

如果定义的位宽比实际的位数小, 那么最左边的位相应地被截断。

例如:

```

3'b1001_0011   //与 3'b011 相等
5'H0FFF        //与 5'H1F 相等

```

⑥ “?” 是高阻态 z 的另一种表示符号。在数字的表示中, 字符 “?” 和 Z (或 z) 是完全等价的, 可互相替代。

⑦ 整数可以带符号 (正、负号), 并且正、负号应写在最左边。负数通常表示为二进制补码的形式。

⑧ 缺省位宽与缺省进制代表十进制的数。

例如:

```

32             //代表十进制数 32

```

-15 //十进制数-15

⑨ 数字中不能有空格，但在表示进制的字母两侧可以有空格。

2. 实数 (Real)

实数有以下两种表示法。

(1) 十进制表示法

例如：

2.0

5.678

0.1 //以上3例是合法的实数表示形式

2. //非法：小数点两侧都必须有数字

(2) 科学计数法

例如：

43_5.1e2 //其值为 43510.0

9.6E2 //960.0 (e 与 E 相同)

5E-4 //0.0005

Verilog HDL 定义了实数转换为整数的方法。实数通过四舍五入被转换为最相近的整数。

例如：

42.446, 42.45 //若转换为整数都是 42

92.5, 92.699 //若转换为整数都是 93

-15.62 //若转换为整数是-16

-26.22 //若转换为整数是-26

3. 字符串 (Strings)

字符串是双引号内的字符序列。字符串不能分成多行书写。例如：

"INTERNAL ERROR"

"this is an example for Verilog HDL"

字符串中的特殊字符必须用字符“\”来说明，比如：

\n 换行符

\t Tab 键

\\ 字符“\”本身

\ " 双引号"

\206 八进制数 206 对应的 ASCII 值

4.1.3 标识符 (Identifiers)

Verilog HDL 中的标识符可以是任意一组字母、数字以及符号“\$”和“_”(下划线)的组合，但标识符的第一个字符必须是字母或者下划线。另外，标识符是区分大小写的。

以下是几个合法的标识符的例子：

count

COUNT //COUNT 与 count 是不同的

_A1_d2 //以下划线开头

R56_68

FIVE

而下面几个例子则是不正确的:

30count //非法: 标识符不允许以数字开头

out* //非法: 标识符中不允许包含字符*

标识符还可以是以符号“\”开头, 以空白符结尾的任何字符序列, 但反斜线和结束空白符并不是标识符的一部分。比如:

\7400

\OutGate // \OutGate 与 OutGate 相同

由于反斜线和结束空白符并不是标识符的一部分, 因此, 标识符 \OutGate 和标识符 OutGate 恒等。

4.1.4 运算符 (Operators)

与 C 语言类似, Verilog HDL 提供了丰富的运算符。有关运算符的内容将在 4.4 节详细介绍。

4.1.5 关键字 (Keywords)

Verilog 语言内部已经使用的词称为关键字或保留字, 这些关键字用户不能随便使用。附录 A 列出了 Verilog HDL 中的所有关键字。

需注意的是, 所有关键字都是小写的。例如, ALWAYS (标识符) 不是关键字, 它与 always (关键字) 是不同的。

4.2 数据类型

在硬件描述语言中, 数据类型 (Data Type) 是用来表示数字电路中的物理连线、数据存储和传送单元等物理量的。

Verilog HDL 中共有 19 种数据类型, 包括 wire 型、reg 型、integer 型、parameter 型、large 型、medium 型、scalared 型、time 型、small 型、tri 型、trio 型、tril 型、triand 型、trior 型、trioreg 型、vectored 型、wand 型和 wor 型等。

在上面的数据类型中, 最基本的是两类: 一类称为连线型 (Net Type), 另一类称为寄存器型 (Register Type)。在这两类中最常用的是: wire 型、reg 型、integer 型和 parameter 型 4 种数据类型。本节着重介绍 wire 型、reg 型和 parameter 型数据类型的使用。

4.2.1 连线型 (Net Type)

连线型数据相当于硬件电路中的各种物理连接, 其特点是输出的值紧跟输入值的变化而变化。对连线型有两种驱动方式, 一种方式是在结构描述中将其连接到一个逻辑门或模块的输出端; 另一种方式是用持续赋值语句 assign 对其进行赋值。

Verilog HDL 提供了多种 net 型变量, 见表 4.1, 在表中, 对综合器支持的数据类型, 用

符号“√”表示。

表 4.1 常用的 net 型变量

类 型	功 能 说 明	可综合性说明
wire, tri	连线类型	√
wor, trior	具有线或特性的连线	
wand, triand	具有线与特性的连线	
tri1, tri0	分别为上拉电阻和下拉电阻	
supply1, supply0	分别为电源（逻辑1）和地（逻辑0）	√

wire 是最常用的连线型变量，因此下面着重加以介绍。

wire 型数据常用来表示以 assign 语句赋值的组合逻辑信号。Verilog HDL 模块中的输入/输出信号类型缺省时自动定义为 wire 型。wire 型信号可以用作任何表达式的输入，也可以用作“assign”语句和实例元件的输出。对于综合器而言，其取值可为 0、1、X、Z。

wire 型变量的定义格式如下：

wire 数据名 1, 数据名 2, ……，数据名 n;

例如： wire a,b; //定义了两个 wire 型变量 a 和 b

上面两个变量 a、b 的宽度都是一位，若定义一个多位的 wire 型数据（如总线），可按如下方式：

wire[n-1:0] 数据名 1, 数据名 2, ……，数据名 n; //数据的宽度为 n 位

或：

wire[n:1] 数据名 1, 数据名 2, ……，数据名 n;

如下面的例子定义了 8 位宽的数据总线，20 位宽的地址总线：

wire[7:0] databus; // databus 的宽度是 8 位

wire[19:0] addrbus; // addrbus 的宽度是 20 位

或：

wire[8:1] databus;

wire[20:1] addrbus;

这种多位的 wire 型数据也称为 wire 型向量（Vector），它可按下面的方式使用：

例如：

wire[7:0] in,out; //定义了两个 8 位 wire 型向量 in 和 out

assign out=in;

若只使用其中某几位，可直接选中这几位，但应注意宽度要一致，如：

wire[7:0] out;

wire[3:0] in

assign out[5:2]=in; //out 向量的第 2 到 5 位与 in 向量相等

它等效于：

assign out[5]=in[3];

assign out[4]=in[2];

assign out[3]=in[1];

assign out[2]=in[0];

4.2.2 寄存器型 (Register Type)

寄存器型变量对应的是具有状态保持作用的电路元件，如触发器、寄存器等。register 型变量与 net 型变量的根本区别在于：register 型变量需要被明确地赋值，并且 register 型变量在被重新赋值前一直保持原值。在设计中必须将寄存器型变量放在过程语句（如 initial、always）中，通过过程赋值语句赋值。另外，在 always、initial 等过程块内，被赋值的每一个信号都必须定义成寄存器型。

Verilog HDL 有 4 种寄存器型变量，见表 4.2，在表中，对综合器支持的数据类型，用符号“√”表示。

integer、real 和 time 三种寄存器型变量都是纯数学的抽象描述，不对应任何具体的硬件电路。reg 型变量是最常用的一种寄存器型变量，下面着重对其进行介绍。

reg 型变量的定义格式类似于 wire 型，如下所示：

```
reg 数据名 1, 数据名 2, …… , 数据名 n;
```

表 4.2 常用的 register 型变量及可综合性说明

类 型	功 能 说 明	可综合性说明
reg	常用的寄存器型变量	√
integer	32 位带符号整型变量	√
real	64 位带符号实型变量	
time	无符号时间变量	

例如：

```
reg a,b; //定义了两个 reg 型变量 a, b
```

上面两个变量 a、b 的宽度都是一位，若定义一个多位的 reg 型向量 (Vector)，可按以下方式：

```
reg[n-1:0] 数据名 1, 数据名 2, …… , 数据名 n;
```

```
reg[n:1] 数据名 1, 数据名 2, …… , 数据名 n; //数据的宽度为 n 位
```

如下面的语句定义了 8 位宽的 reg 型向量：

```
reg[7:0] qout; //定义 qout 为 8 位宽的 reg 型向量
```

或 reg[8:1] qout;

4.2.3 parameter

在 Verilog HDL 中，用 parameter 来定义符号常量，即用 parameter 来定义一个标志符代表一个常量。其定义格式如下：

```
parameter 参数名 1=表达式 1, 参数名 2=表达式 2, 参数名 3=表达式 3, …… ;
```

例如：

```
parameter sel=8,code=8'ha3;
```

//分别定义参数 sel 代表常数 8 (10 进制)，参数 code 代表常量 a3 (16 进制)

又如：

```
parameter datawidth=8, addrwidth= datawidth * 2;
```

//定义参数 datawidth 代表常数 8，参数 addrwidth 代表常量 16 (8*2)

4.3 寄存器和存储器

用 reg 类型变量可构成寄存器和存储器。

4.3.1 寄存器

下面的语句即定义了一个名为 mybit 的 1 位寄存器。

```
reg mybit;
```

而下面的语句则定义了一个名为 mybyte 的 8 位寄存器。

```
reg[7:0] mybyte;
```

1 位寄存器是标量，多位寄存器称为向量 (vector)。

在表达式中，可任意选中寄存器中的一位或相邻几位，分别称为位选择和域选择。

例如：

```
A=mybyte[6]; //将 mybyte 的第 6 位赋值给变量 A，位选择
```

```
B=mybyte[5:2]; //将 mybyte 的第 5，4，3，2 位的值赋给变量 B，域选择
```

再如：

```
reg[7:0] a,b;
```

```
reg[3:0] c;
```

```
reg d;
```

```
d=a[7]&b[7]; //位选择
```

```
c=a[7:4]+b[3:0]; //域选择
```

4.3.2 存储器

若干个相同宽度的向量构成数组 (array)，也就是存储器。

例如：

```
reg[7:0] mymem[1023:0];
```

上面的语句定义了一个 1024 个字节、每个字节宽度为 8bit 的存储器。

再如：

```
reg [0:3] Amem [0:63] //Amem 为 64 个 4 位寄存器的存储器
```

```
reg Bmem[1:5] //Bmem 为 5 个 1 位寄存器的存储器
```

也可以用 parameter 参数定义存储器的尺寸，比如：

```
parameter wordwidth=8,memsize=1024;
```

```
reg[wordwidth-1:0] mymem[memsize-1:0];
```

上面的语句定义了一个宽度为 8bit、1024 个存储单元的存储器，该存储器的名字是 mymem。若对该存储器中的某一单元赋值的话，可采用如下方式：

```
mymem[8]=1; //mymem 存储器中的第 8 个单元被赋值为 1
```

```
mymem[25]=65; //mymem 存储器中的第 25 个单元被赋值为 65
```

不允许对存储器进行位选择和域选择。不过，可以首先将存储器的值赋给寄存器，然后对寄存器进行位选择和域选择。

为存储器赋值的另一种方法是使用系统任务（仅限于电路仿真中使用）：

\$readmemb （加载二进制值）

\$readmemb （加载十六进制值）

这些系统任务从指定的文本文件中读取数据并加载到存储器中。文本文件必须包含相应的二进制或者十六进制数。具体的使用方法可参考第 8 章的相关内容。

4.4 运算符

Verilog HDL 提供了丰富的运算符，按功能分的话，包括算术运算符、逻辑运算符、关系运算符、等式运算符、缩减运算符、条件运算符、位运算符、移位运算符和拼接运算符等 9 类；如果按运算符所带操作数的个数来区分，运算符可分为 3 类，分别为：

- ◆ 单目运算符（unary operators）：运算符可带一个操作数。
- ◆ 双目运算符（binary operators）：运算符可带两个操作数。
- ◆ 三日运算符（ternary operators）：运算符可带三个操作数。

下面按功能的不同对这些运算符分别进行介绍。

4.4.1 运算符

1. 算术运算符（Arithmetic Operators）

常用的算术运算符包括：

+	加
-	减
*	乘
/	除
%	求模

以上的算术运算符都属于双目运算符。符号“+ - * /”分别表示常用的加、减、乘、除四则运算，%是求模运算符，或称为求余运算符，比如 9%4 的值为 1，6%3 的值为 0。

2. 逻辑运算符（Logical Operators）

&&	逻辑与
	逻辑或
!	逻辑非

如 A 的非表示为 !A；A 和 B 的与表示为 A&&B；A 和 B 的或表示为 A||B。

在逻辑运算符的运算中，若操作数是一位的，则逻辑运算的真值表见表 4.3。

表 4.3 逻辑运算符的真值表

a	b	a&& b	a b	!a	!b
1	1	1	1	0	0
1	0	0	1	0	1
0	1	0	1	1	0
0	0	0	0	1	1

如果操作数不只一位的话,则应将操作数作为一个整体来对待,即如果操作数是全“0”,则相当于逻辑0,但只要某一位是“1”,则操作数就应该整体看作逻辑1。逻辑运算符的操作结果是一位。

3. 位运算符 (Bitwise Operators)

位运算,即将两个操作数按对应位分别进行逻辑运算。位运算符包括:

~	按位取反
&	按位与
	按位或
^	按位异或
^~,~^	按位同或 (符号^~与~^是等价的)

按位与、按位或、按位异或的真值表分别见表4.4、表4.5和表4.6。

表 4.4 按位与的真值表

&	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

表 4.5 按位或的真值表

	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

表 4.6 按位异或的真值表

^	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

例如:若 $A=5'b11001$; $B=5'b10101$, 则:

$\sim A=5'b00110$

$A\&B=5'b10001$

$A|B=5'b11101$

$A^B=5'b01100$

需要注意的一点是:两个不同长度的数据进行位运算时,会自动地将两个操作数按右端对齐,位数少的操作数会在高位用“0”补齐。

4. 关系运算符 (Relational Operators)

<	小于
<=	小于或等于
>	大于
>=	大于或等于

注:其中“<=”操作符也用于表示信号的一种赋值操作。

在进行关系运算时,如果声明的关系是假,则返回值是0;如果声明的关系是真,则返回值是1;如果某个操作数的值不定,则关系的结果是模糊的,返回值是不定值。

5. 等式运算符 (Equality Operators)

等式运算符有 4 种, 分别为:

==	等于
!=	不等于
===	全等
!==	不全等

这 4 种运算符都是双目运算符, 得到的结果是 1 位的逻辑值, 如果得到 1, 说明声明的关系为真; 如得到 0, 说明声明的关系为假。

相等运算符 (==) 和全等运算符 (===) 的区别是: 参与比较的两个操作数必须逐位相等, 其相等比较的结果才为 1, 如果某些位是不定态或高阻值, 其相等比较得到的结果是不定值; 而全等比较 (===) 则是对这些不定态或高阻值的位也进行比较, 两个操作数必须完全一致, 其结果才是 1, 否则结果是 0。

相等运算符 (==) 和全等运算符 (===) 的真值表分别见表 4.7 和表 4.8。

比如: 设寄存器变量 $a=5'b11x01$, $b=5'b11x01$, 则“ $a==b$ ”得到的结果为不定值 x , 而“ $a===b$ ”得到的结果为 1。

表 4.7 相等运算符 (==) 真值表

==	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

表 4.8 全等运算符 (===) 真值表

===	0	1	x	z
0	1	0	0	0
1	0	1	0	0
x	0	0	1	0
z	0	0	0	1

6. 缩位运算符 (Reduction Operators)

缩位运算符是单目运算符, 它包括下面几种:

&	与
~&	与非
	或
~	或非
^	异或
^~, ~^	同或

缩位运算符与位运算符的逻辑运算法则一样, 但缩位运算是针对单个操作数进行与、或、非递推运算的, 它放在操作数的前面。缩位运算符将一个矢量缩减为一个标量。例如:

```
reg[3:0] a;
```

```
b=&a; //等效于 b=((a[0]&a[1])&a[2])&a[3];
```

再比如:

若 $A=5'b11001$, 则有:

```
&A=0; //只有 A 的各位都为 1 时, 其与缩减运算的值才为 1
```

```
|A=1; //只有 A 的各位都为 0 时, 其或缩减运算的值才为 0
```

```
~|A=0;
```

7. 移位运算符 (Shift Operators)

>> 右移

<< 左移

Verilog HDL 的移位运算符只有两个：左移和右移。其用法为：

$A \gg n$ 或 $A \ll n$

表示把操作数 A 右移或左移 n 位。该移位是逻辑移位，移出的位用 0 添补。

例如：

若 $A = 5'b11001$ ，则：

$A \gg 2$ 的值为 $5'b00110$ ； //将 A 右移 2 位，用 0 添补移出的位

$A \ll 2$ 的值为 $5'b00100$ ； //将 A 左移 2 位，用 0 添补移出的位

Verilog-1995 中没有指数运算符。但是，移位操作符可用于支持部分指数操作。例如，若 $A = 8'b0000_0100$ ，则二进制的 A^3 可以使用移位操作实现：

$A \ll 3$ //执行后，A 的值变为：8'b0010_0000

在 Verilog-2001 中增加了指数运算符 (**)，具体在 6.5 节中介绍。

8. 条件运算符 (Conditional Operators)

?:

这是一个三目运算符，对 3 个操作数进行运算，其定义同 C 语言中的定义一样，方式如下：

```
signal=condition?true_expression:false_expression;
```

即：信号=条件?表达式 1:表达式 2;

当条件成立时，信号取表达式 1 的值，反之取表达式 2 的值。

例如对如图 4.1 所示的 2 选 1 MUX 可用条件运算符描述为：

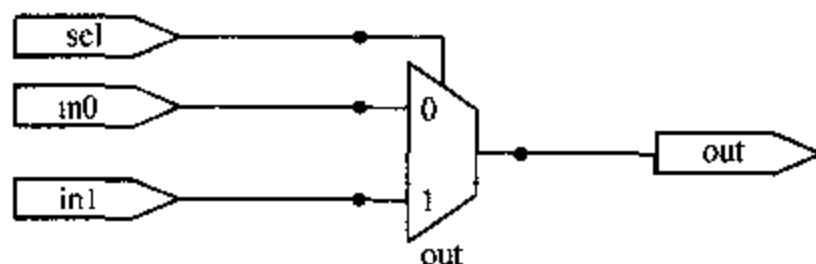


图 4.1 2 选 1 MUX

```
out=sel?in1:in0;            //sel=1 时 out=in1; sel=0 时 out=in0
```

或者：

```
out=(sel==0)?in0:in1;        //该语句完成与上句相同的功能
```

9. 位拼接运算符 (Concatenation Operators)

{ }

该运算符将两个或多个信号的某些位拼接起来，使用如下：

{信号 1 的某几位, 信号 2 的某几位, ……, 信号 n 的某几位}

例如，在进行加法运算时，可将和与进位输出拼接在一起使用：

```

output[3:0] sum;           //sum 代表和
output cout;              //cout 为进位输出
input[3:0] ina,inb;       //ina,inb 为两个加数
input cin;                //cin 为进位输入
assign {cout,sum}=ina+inb+cin; //进位与和拼接在一起

```

位拼接可以嵌套使用，还可以用重复法来简化书写，如：
`{3{a,b}}` 等同于 `{a,b},{a,b},{a,b}`，也等同于 `{a,b,a,b,a,b}`。

4.4.2 运算符的优先级

以上运算符的优先级见表 4.9。但不同的综合开发工具在执行这些优先级时可能有微小的差别，因此在书写程序时建议用括号（）来控制运算的优先级，这样也能有效地避免错误，同时增加了程序的可读性。

表 4.9 运算符的优先级

运 算 符	优 先 级
! ~	高优先级  低优先级
* / %	
+ -	
<< >>	
< <= > >=	
== != === !==	
& ~&	
^ ^~	
~	
&&	
?:	

思考与练习

- 下列标识符哪些是合法的，哪些是错误的？
`Cout`, `8sum`, `\a*b`, `_data`, `\wait`, `initial`
- 下列数字的表示是否正确？
`6'd18`, `'Bx0`, `5'b0x110`, `'da30`, `10'd2`, `'hzF`
- `reg` 型和 `wire` 型变量有什么本质的区别？
- 能否对存储器进行位选择和域选择？

第5章 Verilog HDL 行为语句

本章内容简介

- ◆ 过程语句 (initial、always)
- ◆ 块语句 (begin-end、fork-join)
- ◆ 赋值语句 (assign、=、<=)
- ◆ 条件语句 (if-else、case、casez、casex)
- ◆ 循环语句 (for、forever、repeat、while)
- ◆ 编译向导语句 (`define、`include、`ifdef、`else、`endif)

本章介绍 Verilog HDL 行为语句，在学习和使用这些语句时应注意下面几点：

所有的 HDL 语句都可用于仿真，但通常可综合的只是 HDL 语言的一个优化子集，所以应对语句的可综合性有所了解。不同的综合器支持的 HDL 语句集也是不同的，应具体参考软件的帮助文档。

要充分理解硬件描述语言和硬件电路的关系，并熟练掌握常用的 HDL 语句。基本的 HDL 语句就可以完成绝大多数电路与系统的设计，所以对常用语句用心钻研、深入理解，对完成电路设计将是非常有用的。

此外，对语言中的难点，如阻塞赋值和非阻塞赋值的区别等，必须深入理解和领会。

5.1 概 述

Verilog HDL 支持许多高级行为语句，使其成为结构化和行为性的语言。Verilog HDL 语句包括：过程语句、块语句、赋值语句、条件语句、循环语句、编译向导语句等，见表 5.1 中。符号“√”表示该语句能够为综合工具所支持，是可综合的。

表 5.1 Verilog HDL 语句

类 别	语 句	可 综 合 性
过程语句	initial	
	always	√
块语句	串行块 begin-end	√
	并行块 fork-join	
赋值语句	持续赋值 assign	√

续表

类 别	语 句	可 综 合 性
	过程赋值=、<=	√
条件语句	if-else	√
	case, casez, casex	√
循环语句	for	√
	repeat	
	while	
	forever	
编译向导语句	`define	√
	`include	√
	`ifdef, `else, `endif	√

学习行为语句时，应该对语句的可综合性有所了解。Verilog 标准中的所有 HDL 语句都可以用于仿真，并得到仿真器的支持，但通常能被综合的语句只是其一个子集，而且可综合的子集还没有标准化，不同的综合器支持的 HDL 语句也有所差别，因此要了解所用综合器的性能。

其次，要充分理解 HDL 语句和硬件电路的关系。编写 HDL 程序，就是在描述一个电路，每一段程序都对应着相应的硬件电路结构，必须深入理解两者间的关系。有些综合器，如 Synplify Pro/Synplify，可将 HDL 文本对应的硬件电路以图形的方式呈现出来，非常直观，便于学习者建立 HDL 语言与硬件电路之间的对应关系。

下面对表 5.1 中所列的语句分别介绍，着重介绍综合器可综合的语句。

5.2 过程语句

Verilog HDL 中的多数过程模块都从属于以下 2 种过程语句：

- ◆ initial
- ◆ always

在一个模块（module）中，使用 initial 和 always 语句的次数是不受限制的。initial 语句常用于仿真中的初始化，initial 过程块中的语句仅执行一次；always 块内的语句则是不断重复执行的。always 过程语句是可综合的，在可综合的电路设计中广泛采用。

5.2.1 always 过程语句

always 过程语句使用模板如下：

```
always @(<敏感信号表达式 event-expression>)
begin
//过程赋值
```



```
//if-else, case, casex, casez 选择语句
//while, repeat, for 循环
//task, function 调用
end
```

“always”过程语句通常是带有触发条件的，触发条件写在敏感信号表达式中，只有当触发条件满足时，其后的“begin-end”块语句才能被执行。因此，此处首先讨论敏感信号表达式“event-expression”的含义以及如何写敏感信号表达式。

1. 敏感信号表达式

所谓敏感信号表达式，又称事件表达式或敏感信号列表，即当该表达式中变量的值改变时，就会引发块内语句的执行。因此，敏感信号表达式中应列出影响块内取值的所有信号。若有两个或两个以上信号，它们之间用“or”连接。

例如：

```
@(a)                //当信号 a 的值发生改变时
@(a or b)            //当信号 a 或信号 b 的值发生改变时
@(posedge clock)     //当 clock 的上升沿到来时
@(negedge clock)     //当 clock 的下降沿到来时
@(posedge clk or negedge reset) //当 clk 的上升沿到来或 reset 信号的下降沿到来时
```

比如下面例 5.1 中用 case 语句描述的 4 选 1 数据选择器，只要输入信号 in0、in1、in2、in3 或选择信号 sel[1:0]中的任一个改变，则输出改变，所以敏感信号表达式写为：

```
@ (in0 or in1 or in2 or in3 or sel)
```

【例 5.1】用 case 语句描述的 4 选 1 数据选择器

```
module mux4_1(out,in0,in1,in2,in3,sel);
output out;
input in0,in1,in2,in3;
input[1:0] sel;
reg out;
always @(in0 or in1 or in2 or in3 or sel)    //敏感信号列表
    case(sel)
        2'b00: out=in0;
        2'b01: out=in1;
        2'b10: out=in2;
        2'b11: out=in3;
        default: out=2'bx;
    endcase
endmodule
```

敏感信号可以分为两种类型：一种为边沿敏感型，一种为电平敏感型。每一个 always 过程最好只由一种类型的敏感信号来触发，而不要将边沿敏感型和电平敏感型信号列在一起，比如下面的例子。

```

always @(posedge clk or posedge clr)
    //两个敏感信号都是边沿敏感型
always @(A or B)
    //两个敏感信号都是电平敏感型
always @(posedge clk or clr)
    //不建议这样用，最好不要将边沿敏感型和电平敏感型信号列在一起

```

2. posedge 与 negedge 关键字

对于时序电路，事件通常是由时钟边沿触发的。为表达边沿这个概念，Verilog HDL 提供了 posedge 和 negedge 两个关键字来进行描述。比如：

【例 5.2】同步置数、同步清零的计数器

```

module count(out,data,load,reset,clk);
output[7:0] out;
input[7:0] data;
input load,clk,reset;
reg[7:0] out;
always @(posedge clk)                //clk 上升沿触发
begin
    if (!reset)    out = 8'h00;        //同步清 0，低电平有效
    else if (load) out = data;        //同步预置
    else          out = out + 1;      //计数
end
endmodule

```

在上面的例子中，posedge clk 表示时钟信号 clk 的上升沿作为触发条件，而 negedge clk 表示时钟信号 clk 的下降沿作为触发条件。

在上例中，敏感信号表达式中没有列出输入信号 load、reset，这是因为它们是同步置数、同步清零，这些信号要起作用，必须有时钟的上升沿来到。对于异步的清零/置数，应按以下格式书写敏感信号表达式。

例：假如时钟信号为 clk，clear 为异步清零信号，则敏感信号表达式写为：

```

always @(posedge clk or posedge clear)
    // clear 信号上升沿到来时清零，故高电平清零有效
always @(posedge clk or negedge clear)
    // clear 信号下降沿到来时清零，故低电平清零有效

```

若有其他异步控制信号，可按此方式加入。注意，块内的逻辑描述要与敏感信号表达式中信号的有效电平一致。例如，下面的描述是错误的：

```

always @(posedge clk or negedge clear)    //低电平清零有效
begin
    if (clear)                            //与敏感信号表达式中低电平清零有效矛盾，应改为 if(!clear)
        out=0;
    else    out=in;
end

```


3. 用 always 过程块实现较复杂的组合逻辑电路

always 过程语句通常用来对寄存器类型的数据进行赋值，但 always 过程语句也可以用来设计组合逻辑。在有的情况下，使用 assign 来实现组合逻辑电路，很多地方会显得冗长且效率低下，而适当地采用 always 过程语句来设计组合逻辑，能收到更好的效果。

下面是一个简单的指令译码电路的设计示例。该电路通过对指令的判断，对输入数据执行相应的操作，包括加、减、求与、求或、求反，并且无论是指令作用的数据还是指令本身发生变化，结果都应做出及时的反应。显然，这是一个较为复杂的组合逻辑电路，如果采用 assign 语句描述，表达起来非常复杂。在本例中使用了电平敏感的 always 块，所谓电平敏感的触发条件，是指@后的括号内敏感信号列表中的任何一个信号电平发生变化，就能触发 always 块的动作，并且运用了 case 结构来进行分支判断，不但设计思想得到了直观的体现，而且代码看起来整齐有序，便于理解。

【例 5.3】用 always 过程语句描述的简单算术逻辑单元

```
'define add      3'd0
'define minus    3'd1
'define band     3'd2
'define bor      3'd3
'define bnot     3'd4
module alu(out,opcode,a,b);
output[7:0] out;
reg[7:0] out;
input[2:0] opcode;           //操作码
input[7:0] a,b;              //操作数
always@(opcode or a or b)    //电平敏感的 always 块
begin
    case(opcode)
        'add: out = a+b;      //加操作
        'minus: out = a-b;    //减操作
        'band: out = a&b;     //求与
        'bor: out = a|b;      //求或
        'bnot: out=~a;        //求反
        default: out=8'hx;     //未收到指令时，输出任意态
    endcase
end
endmodule
```

图 5.1 是上例的 RTL 综合结果。

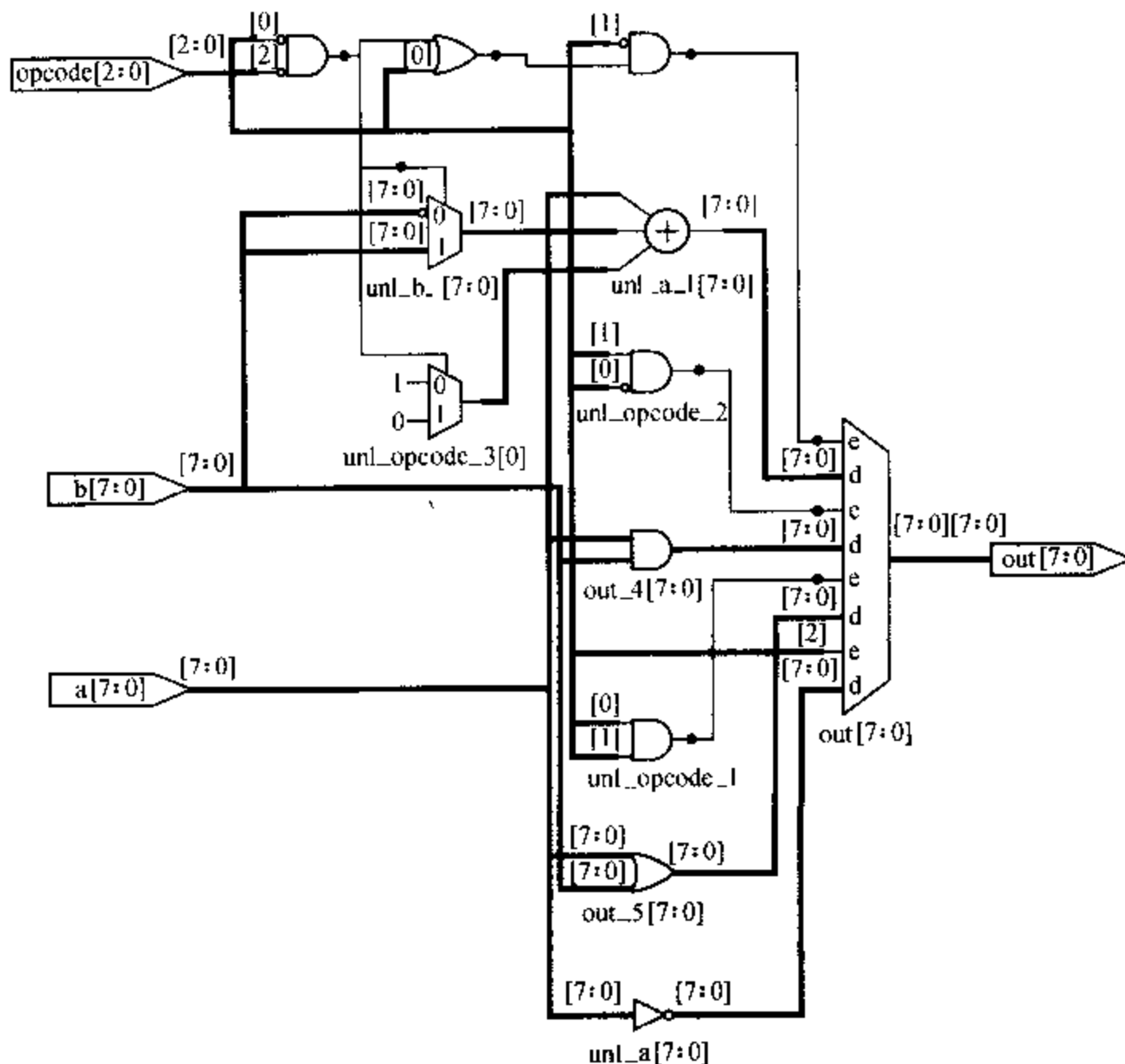


图 5.1 例 5.3 的 RTL 综合结果

5.2.2 initial 语句

initial 语句的使用格式如下:

```

initial
  begin
    语句 1;
    语句 2;
    .....
  end

```

initial 语句不带触发条件, initial 过程中的块语句沿时间轴只执行一次。initial 语句通常用于仿真模块中对激励向量的描述, 或用于给寄存器变量赋初值, 它是面向模拟仿真的过程语句, 通常不能被逻辑综合工具所支持。

下面对 initial 语句的使用举例进行说明。比如, 下面的测试模块中利用 initial 语句完成对测试变量 A、B、C 的赋值。

【例 5.4】用 initial 过程语句对测试变量 A、B、C 赋值

```

`timescale 1ns/1ns
module test;

```

```

reg A,B,C;
initial
begin
    A = 0; B = 1; C = 0;
#50  A = 1; B = 0;
#50  A = 0; C = 1;
#50  B = 1;
#50  B = 0; C = 0;
#50  $finish;
end
endmodule

```

上面例子对 A、B、C 的赋值相当于描述了如图 5.2 所示的波形。

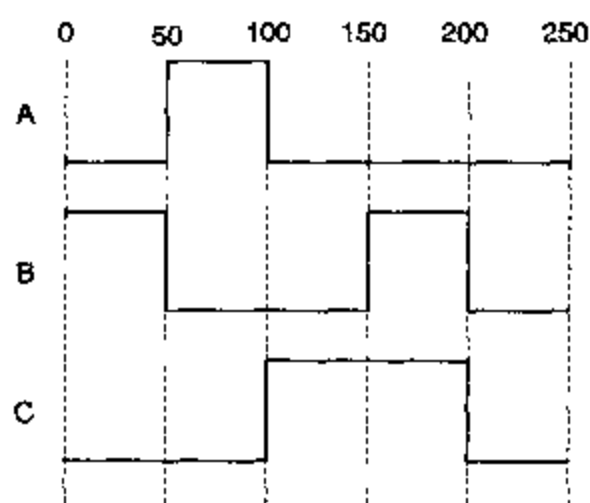


图 5.2 例 5.4 所定义的波形

在下面的例子中，用 initial 语句对存储器进行初始化。

```

initial
begin
    for(addr=0;addr<size;addr=addr+1)
        memory[addr]=0;        //对 memory 存储器进行初始化
    end

```

在上面的例子中，使用 initial 语句对 memory 存储器进行初始化，将其所有的存储单元的初始值都置为“0”。

5.3 块 语 句

块语句是由块标志符 begin-end 或 fork-join 界定的一组语句，当块语句只包含一条语句时，块标志符可以缺省。下面分别介绍串行块 begin-end 和并行块 fork-join。

5.3.1 串行块 begin-end

begin-end 串行块中的语句按串行方式顺序执行。

比如: **begin**
 regb=rega;
 regc=regb;
end

由于 begin-end 块内的语句顺序执行, 在最后, 将 regb、regc 的值都更新为 rega 的值。该 begin-end 块执行完后, regb、regc 的值是相同的。

仿真时, begin-end 块中的每条语句前面的延时都是相对于前一条语句执行结束的相对时间。比如: 下面的模块产生了一段周期为 10 个时间单位的信号波形。

【例 5.5】用 begin-end 串行块产生信号波形

```
'timescale 10ns/1ns
module wavel;
reg wave;
parameter cycle=10;
initial
begin
    wave=0;
    #(cycle/2) wave=1;
    #(cycle/2) wave=0;
    #(cycle/2) wave=1;
    #(cycle/2) wave=0;
    #(cycle/2) wave=1;
    #(cycle/2) $finish ;
end
initial $monitor($time,,, "wave=%b", wave);
endmodule
```

上面的程序用 ModelSim 编译仿真后, 可得到一段周期为 10 个时间单位的信号波形, 如图 5.3 所示。

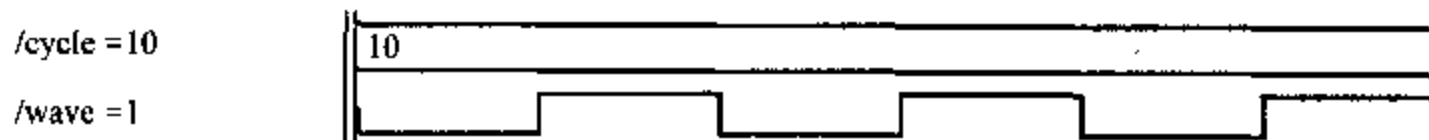


图 5.3 例 5.5 所描述的波形

5.3.2 并行块 fork-join

并行块 fork-join 中的所有语句是并发执行的。

比如: **fork**
 regb=rega;
 regc=regb;
join

由于 fork-join 并行块中的语句是同时执行的, 在上面的块语句执行完后, regb 更新为 rega 的值, 而 regc 的值更新为没有改变前的 regb 的值, 故执行完后, regb 与 regc 的值是不同的。

在进行仿真时, fork-join 并行块中的每条语句前面的延时都是相对于该并行块的起始执行时间的。

比如要用 fork-join 并行块产生一段与例 5.5 中相同的信号波形, 应该像下面的例子这样标注延时。

【例 5.6】用 fork-join 并行块产生信号波形

```
'timescale 10ns/1ns
module wave2;
reg wave;
parameter cycle=5;
initial
    fork
        wave=0;
        #(cycle)    wave=1;
        #(2*cycle)  wave=0;
        #(3*cycle)  wave=1;
        #(4*cycle)  wave=0;
        #(5*cycle)  wave=1;
        #(6*cycle)  $finish ;
    join
initial $monitor($time,,, "wave=%b", wave);
endmodule
```

上面的程序用 ModelSim 编译仿真后, 可得到与图 5.3 完全相同的信号波形。两者的输出也相同, 都如下所示。

```
0 wave=0
5 wave=1

10 wave=0
15 wave=1

20 wave=0
25 wave=1
```

将例 5.5 和例 5.6 进行对比, 可体会 begin-end 串行块和 fork-join 并行块的区别。

5.4 赋值语句

5.4.1 持续赋值与过程赋值

Verilog HDL 有以下赋值方式和赋值语句。

1. 持续赋值语句 (Continuous Assignments)

`assign` 为持续赋值语句，主要用于对 `wire` 型变量的赋值。

比如：`assign c=a&b;`

在上面的赋值中，`a`、`b`、`c` 三个变量皆为 `wire` 型变量，`a` 和 `b` 信号的任何变化都将随时反映到 `c` 上来。

下面的例子是用持续赋值方式定义的 2 选 1 数据选择器。

【例 5.7】持续赋值方式定义的 2 选 1 多路选择器

```
module MUX21_1(out,a,b,sel);
input a,b,sel;
output out;
assign out= (sel==0) ? a : b ;
        //持续赋值，如果 sel 为 0，则 out= a；否则 out= b
endmodule
```

2. 过程赋值语句 (Procedural Assignments)

过程赋值语句多用于对 `reg` 型变量进行赋值。过程赋值有阻塞 (`blocking`) 赋值和非阻塞 (`non_blocking`) 赋值两种方式。

(1) 非阻塞 (`non_blocking`) 赋值方式

赋值符号为 “`<=`”，如：`b<= a;`

非阻塞赋值在整个过程块结束时才完成赋值操作，即 `b` 的值并不是立刻就改变的。

(2) 阻塞 (`blocking`) 赋值方式

赋值符号为 “`=`”，如：`b= a;`

阻塞赋值在该语句结束时就立即完成赋值操作，即 `b` 的值在该条语句结束后立刻改变。如果在一个块语句中有多条阻塞赋值语句，那么在前面的赋值语句没有完成之前，后面的语句就不能被执行，仿佛被阻塞了 (`blocking`) 一样，因此称为阻塞赋值方式。

例 5.8 是用阻塞赋值方式定义的 2 选 1 多路选择器。

【例 5.8】阻塞赋值方式定义的 2 选 1 多路选择器

```
module MUX21_2(out,a,b,sel);
input a,b,sel;
output out;
reg out;
always@(a or b or sel)
begin
    if(sel==0) out=a;           //阻塞赋值
    else      out=b;
end
endmodule
```


5.4.2 阻塞赋值与非阻塞赋值

阻塞赋值方式和非阻塞赋值方式的区别常给设计人员带来问题，问题主要是对“always”模块内的 reg 型变量的赋值不易把握。为区分非阻塞赋值与阻塞赋值，可看下面两例。

【例 5.9】非阻塞赋值

```
module non_block(c,b,a,clk);
output c,b;
input clk,a;
reg c,b;
always @(posedge clk)
begin
b<=a;
c<=b;
end
endmodule
```

【例 5.10】阻塞赋值

```
module block(c,b,a,clk);
output c,b;
input clk,a;
reg c,b;
always @(posedge clk)
begin
b=a;
c=b;
end
endmodule
```

将上面两段代码用 MAX+PLUS II 软件进行综合，并仿真，可分别得到如下波形，如图 5.4（非阻塞赋值波形图）和图 5.5（阻塞赋值波形图）所示。

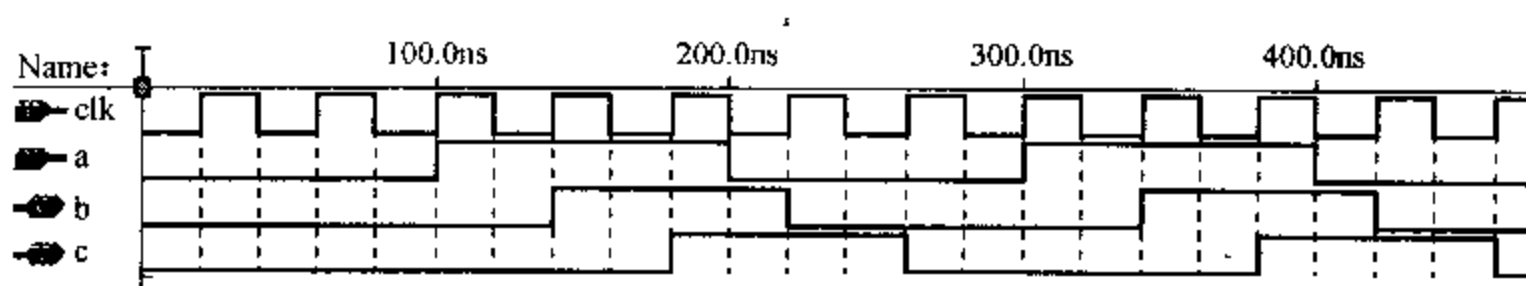


图 5.4 例 5.9 非阻塞赋值仿真波形图

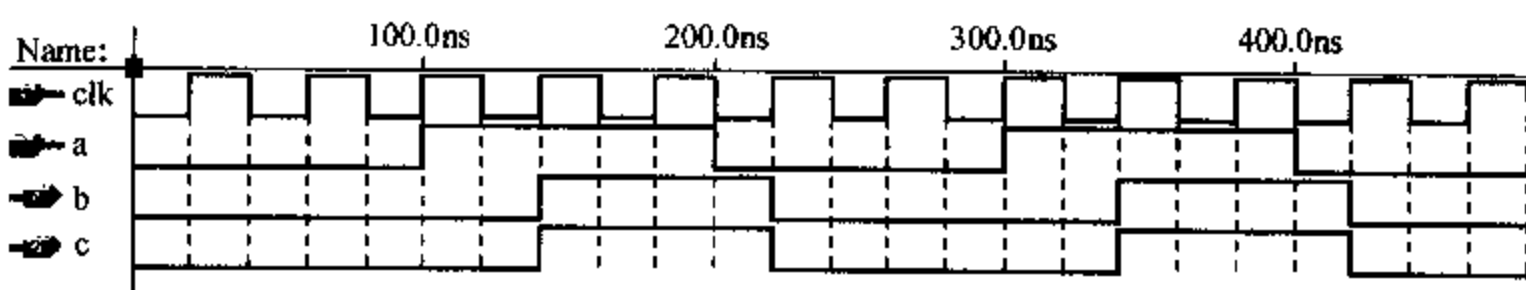


图 5.5 例 5.10 阻塞赋值仿真波形图

从图中不难看出二者的区别：

对于非阻塞赋值，c 的值落后 b 的值一个时钟周期，这是因为该“always”块中两条语句是同时执行的。因此，每次执行完后，b 的值得到更新，而 c 的值仍是上一时钟周期的 b 值。

对于阻塞赋值，c 的值和 b 的值一样，这是因为 b 的值是立即更新的。

上述两例综合后的电路结构分别如图 5.6 和图 5.7 所示。

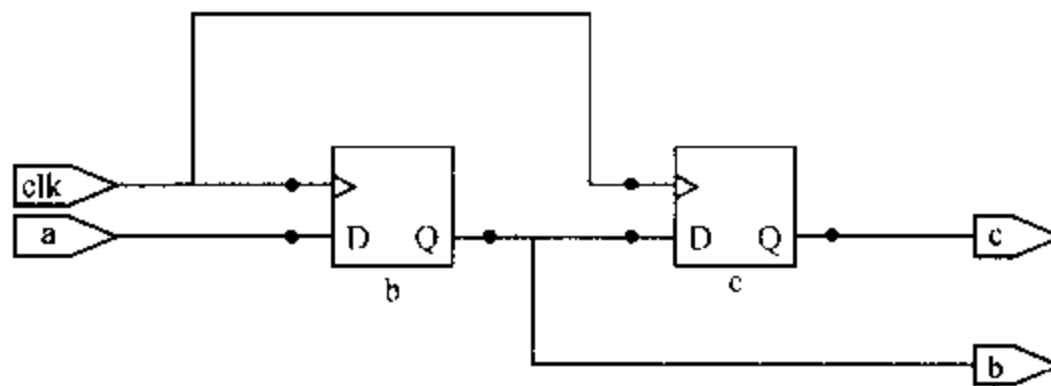


图 5.6 例 5.9 非阻塞赋值综合结果

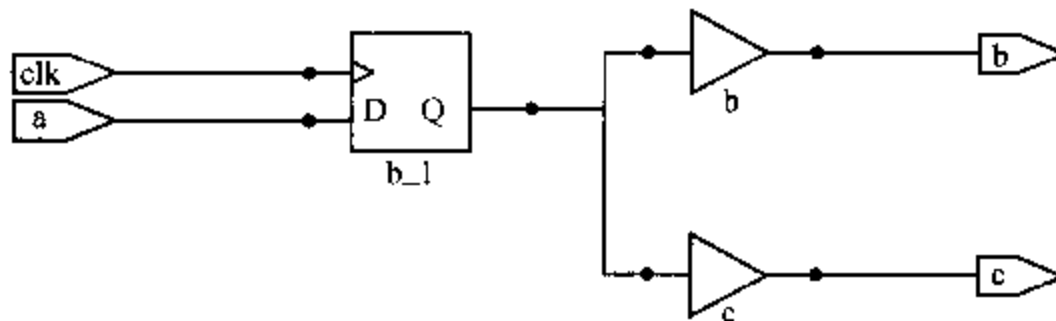


图 5.7 例 5.10 阻塞赋值综合结果

为避免出错，在同一块内，最好不要将输出再作为输入使用。为了用阻塞赋值方式完成与上述非阻塞赋值同样的功能，可采用两个“always”块来实现，如下所示。在下面的例子中，两个 always 过程块是并发执行的。

```

module non_block(c,b,a,clk);
output c,b;
input clk,a;
reg c,b;
always @(posedge clk)
    begin b=a; end
always @(posedge clk)
    begin c=b; end
endmodule

```

阻塞赋值与非阻塞赋值是学习 Verilog 语言的难点之一，关于这两种赋值方式的使用要点，将在 10.6 节中进一步讨论。

5.5 条件语句

条件语句有 if-else 语句和 case 语句两种，它们都是顺序语句，应放在“always”块内。下面对这两种语句分别介绍。

5.5.1 if-else 语句

其格式与 C 语言中的 if-else 语句类似，使用方法有以下 3 种。


```

(1) if (表达式) 语句 1;
(2) if (表达式) 语句 1;
    else 语句 2;
(3) if (表达式 1) 语句 1;
    else if (表达式 2) 语句 2;
    else if (表达式 3) 语句 3;
    .....
    else if (表达式 n) 语句 n;
    else 语句 n+1;

```

在上述 3 种方式中,“表达式”一般为逻辑表达式或关系表达式,也可能是一位变量。系统对表达式的值进行判断,若为 0、x、z,按“假”处理;若为 1,按“真”处理,执行指定语句。语句可是单句,也可多句,多句时用“begin-end”块语句括起来。对于 if 语句的嵌套,若不清楚 if 和 else 的匹配,最好用“begin-end”语句括起来。

下例是用 if-else 语句实现的模为 60 的 BCD 码计数器。

【例 5.11】模为 60 的 BCD 码加法计数器

```

module count60(qout,cout,data,load,cin,reset,clk);
output [7:0] qout;
output cout;                                //进位输出
input [7:0] data;
input load,cin,clk,reset;
reg [7:0] qout;
always @(posedge clk)                       //clk 上升沿时刻计数
begin
    if (reset) qout<=0;                       //同步复位
    else if (load) qout<=data;                 //同步置数
    else if (cin)
        begin
            if (qout[3:0]==9)                 //低位是否为 9,是则
                begin
                    qout[3:0]<=0;              //回 0,并判断高位是否为 5
                    if (qout[7:4]==5) qout[7:4]<=0;
                end
            else
                qout[7:4]<=qout[7:4]+1;        //高位不为 5,则加 1
            end
            else                               //低位不为 9,则加 1
                qout[3:0]<=qout[3:0]+1;
        end
    end
assign cout=((qout==8'h59)&cin)?1:0;         //产生进位输出信号

```

```
endmodule
```

5.5.2 case 语句

相对于 if 语句只有两个分支而言, case 语句是一种多分支语句, 故 case 语句可用于多条件译码电路, 如描述译码器、数据选择器、状态机及微处理器的指令译码等。case 语句有 case、casez、casex 三种表示方式, 这里分别予以说明。

1. case 语句

case 语句的使用格式如下。

```
case (敏感表达式)
    值 1: 语句 1;           //case 分支项
    值 2: 语句 2;
    .....
    值 n: 语句 n;
    default: 语句 n+1;
endcase
```

当敏感表达式的值为值 1 时, 执行语句 1; 为值 2 时, 执行语句 2; 依此类推。如果敏感表达式的值与上面列出的值都不相同的话, 则执行 default 后面的语句。如果前面已列出了敏感表达式所有可能的取值, 则 default 语句可以省略。

下面的例子是用 case 语句编写的 BCD 码—七段数码管译码电路。

【例 5.12】BCD 码—七段数码管显示译码器

```
module decode4_7(decodeout, indec);
    output[6:0] decodeout;
    input[3:0] indec;
    reg[6:0] decodeout;
    always @(indec)
        begin
            case(indec)                                //用 case 语句进行译码
                4'd0: decodeout=7'b1111110;
                4'd1: decodeout=7'b0110000;
                4'd2: decodeout=7'b1101101;
                4'd3: decodeout=7'b1111001;
                4'd4: decodeout=7'b0110011;
                4'd5: decodeout=7'b1011011;
                4'd6: decodeout=7'b1011111;
                4'd7: decodeout=7'b1110000;
                4'd8: decodeout=7'b1111111;
                4'd9: decodeout=7'b1111011;
                default: decodeout=7'bx;
            endcase
        end
```

```
end
endmodule
```

2. casez 与 casex 语句

在 case 语句中, 敏感表达式与值 1~值 n 间的比较是一种全等比较, 必须保证两者的对应位全等。casez 与 casex 语句是 case 语句的两种变体, 在 casez 语句中, 如果分支表达式某些位的值为高阻 z, 那么对这些位的比较就不予考虑, 因此只需关注其他位的比较结果。而在 casex 语句中, 则把这种处理方式进一步扩展到对 x 的处理, 即如果比较的双方有一方的某些位的值是 x 或 z, 那么这些位的比较就都不予考虑。

表 5.2、表 5.3 和表 5.4 分别列出了 case、casez 和 casex 在进行比较时的规则。

此外, 还有另外一种标识 x 或 z 的方式, 即用表示无关值的符号“?”来表示。

比如: case(a)

```
2'b1x: out=1;    //只有 a=1x, 才有 out=1
```

```
casex(a)
```

```
2'b1x: out=1;    //如果 a=10、11、1x、1z 等, 都有 out=1
```

```
casez(a)
```

```
3'b1??: out=1;   //如果 a=100、101、110、111 或 1xx、1zz 等, 都有 out=1
```

```
3'b01?: out=1;   //如果 a=010、011、01x、01z, 都有 out=1
```

表 5.2 case 语句比较规则

case	0	1	x	z
0	1	0	0	0
1	0	1	0	0
x	0	0	1	0
z	0	0	0	1

表 5.3 casez 语句比较规则

casez	0	1	x	z
0	1	0	0	1
1	0	1	0	1
x	0	0	1	1
z	1	1	1	1

表 5.4 casex 语句比较规则

casex	0	1	x	z
0	1	0	1	1
1	0	1	1	1
x	1	1	1	1
z	1	1	1	1

例 5.13 是一个采用 casez 语句描述并使用了符号“?”的数据选择器的例子。

【例 5.13】用 casez 描述的数据选择器

```
module mux_casez (out, a, b, c, d, select);
output out;
input a, b, c, d;
input[3:0] select;
reg out;
always @(select or a or b or c or d)
```

```

begin
    casez (select)
        4'b???1: out = a;
        4'b??1?: out = b;
        4'b?1??: out = c;
        4'b1???: out = d;      //不需再加 default 语句
    endcase
end
endmodule

```

5.5.3 条件语句使用要点

在使用条件语句时，应注意列出所有条件分支，否则，编译器认为条件不满足时，会引进一个触发器保持原值。这一点可用于设计时序电路，例如在计数器的设计中，条件满足则加 1，否则保持不变；而在组合电路设计中，应避免这种隐含触发器的存在。当然，一般不可能列出所有分支，因为每一变量至少有 4 种取值 0、1、z、x。为包含所有分支，可在 if 语句最后加上 else；在 case 语句的最后加上 default 语句。

下面是一个隐含锁存器的例子。

【例 5.14】隐含锁存器举例

```

module buried_ff(c,b,a);
output c;
input b,a;
reg c;
always @(a or b)
begin
    if((b==1)&&(a==1)) c=a&b;
end
endmodule

```

设计者原意是设计一个二输入与门，但因 if 语句中无 else 语句，在对此语句逻辑综合时会认为 else 语句为：“c=c;”，即保持不变，所以形成了一个隐含锁存器，该例的综合结果如图 5.8 所示。

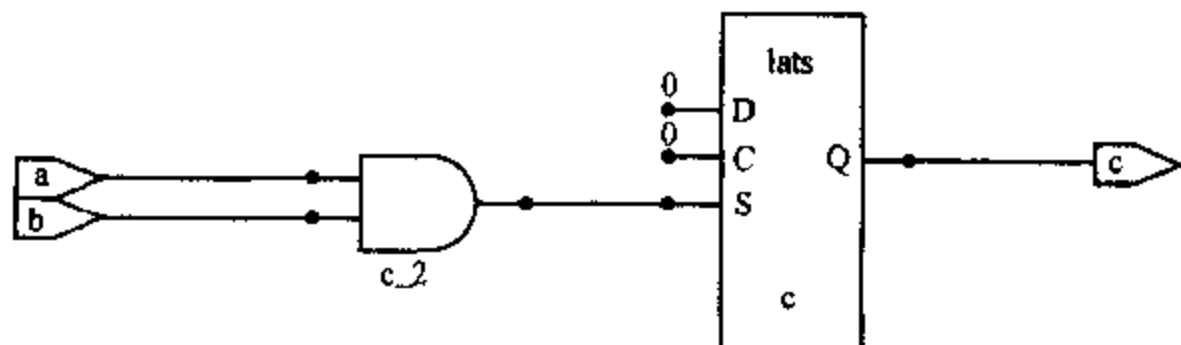


图 5.8 隐含锁存器

在对上例进行仿真时，只要出现“a=1”及“b=1”时，“c=1”后，c 的值一直都维持为

1. 为改正此错误, 仅需加上下面的语句即可。

```
else c=0;
```

即“always”过程块变为:

```
always @(a or b)
```

```
begin
```

```
    if((b==1)&&(a==1)) c= a&b ;
```

```
    else c=0;
```

```
end
```

在利用“if-else”语句设计组合电路时, 要防止此类错误的发生。

5.6 循环语句

在 Verilog HDL 中存在 4 种类型的循环语句, 可用来控制语句的执行次数。这 4 种语句分别为:

- (1) forever: 连续地执行语句; 多用在“initial”块中, 以生成时钟等周期性波形。
- (2) repeat: 连续执行一条语句 n 次。
- (3) while: 执行一条语句, 直到某个条件不满足。
- (4) for: 有条件的循环语句。

5.6.1 for 语句

for 语句的使用格式如下 (同 C 语言):

for (表达式 1; 表达式 2; 表达式 3) 语句;

即: for (循环变量赋初值; 循环结束条件; 循环变量增值) 执行语句;

下面通过“七人表决器”的例子来说明 for 语句的使用, 通过一个循环语句统计赞成的人数, 若超过四人赞成则通过。用 vote[6:0] 表示 7 人的投票情况, “1”代表赞成, 即 vote[i] 为 “1” 代表第 i 个人赞成, pass=“1”表示表决通过。

【例 5.15】用 for 语句描述的七人投票表决器

```
module voter7(pass,vote);
```

```
output pass;
```

```
input [6:0] vote;
```

```
reg [2:0] sum;
```

```
integer i;
```

```
reg pass;
```

```
always @(vote)
```

```
begin
```

```
    sum=0;
```

```
    for(i=0;i<=6;i=i+1)           //for 语句
```

```
        if(vote[i]) sum=sum+1;
```

```

        if(sum[2]) pass=1;           //若超过 4 人赞成, 则 pass=1
        else      pass=0;

    end

endmodule

```

下面的这个例子中使用 for 循环语句实现了两个 8 位二进制数的乘法操作。

【例 5.16】 用 for 语句实现 2 个 8 位数相乘

```

module mult_for(outcome, a, b);
parameter size=8;
input[size:1]  a,b;           //两个操作数
output[2*size:1] outcome;     //结果
reg[2*size:1]  outcome;
integer i;

always @(a or b)
begin
    outcome=0;
    for(i=1; i<=size; i=i+1)    // for 语句
        if(b[i]) outcome=outcome +(a << (i-1));
    end
endmodule

```

图 5.9 是该乘法器的功能仿真波形图 (图中的数据显示为十进制形式)。

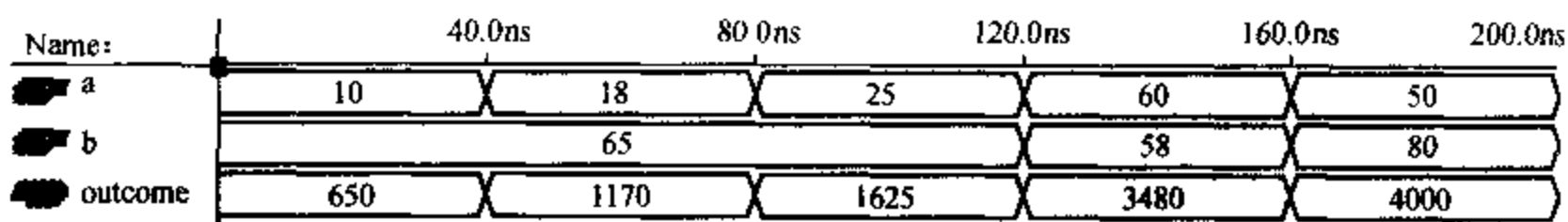


图 5.9 乘法器的功能仿真波形图

绝大多数的综合器都支持 for 循环语句, 在可综合的设计中, 若需要用到循环语句, 应首先考虑用 for 语句来实现。

5.6.2 repeat 语句

repeat 语句的使用格式为:

```

repeat (循环次数表达式) 语句;
或 repeat (循环次数表达式) begin
    .....
end

```

在下面的例子中, 利用 repeat 循环语句和移位操作实现了两个 8 位二进制数的乘法。

【例 5.17】 用 repeat 实现 8 位二进制数的乘法

```

module mult_repeat(outcome, a, b);

```



```

parameter size=8;
input[size:1] a,b;
output[2*size:1] outcome;
reg[2*size:1] temp_a,outcome;
reg[size:1] temp_b;
always @(a or b)
begin
    outcome=0;
    temp_a=a;
    temp_b=b;
    repeat(size)                //repeat 语句, size 为循环次数
    begin
        if(temp_b[i])           //如果 temp_b 的最低位为 1, 就执行下面的加法
        outcome=outcome +temp_a;
        temp_a=temp_a<<1;       //操作数 a 左移一位
        temp_b=temp_b>>1;       //操作数 b 右移一位
    end
end
endmodule

```

上面的模块用 Synplify 软件进行编译, 目标器件选择 EPF10K10 (FLEX 10K 系列), 编译后生成.edf 文件, 再转到 MAX+PLUS II 软件中进行仿真。如图 5.10 所示是该乘法器的功能仿真波形图 (图中的数据显示为十六进制形式)。

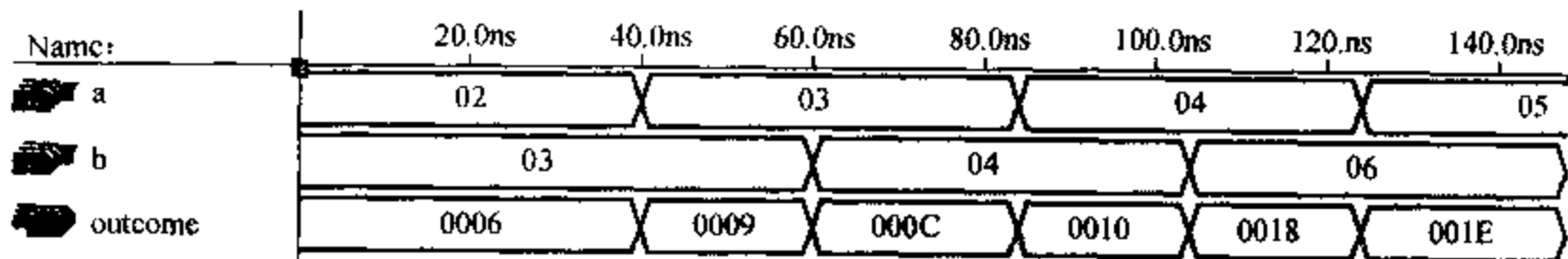


图 5.10 例 5.17 的功能仿真波形

5.6.3 while 和 forever 语句

1. while 语句

while 语句的使用格式如下:

```

while (循环执行条件表达式) 语句;
或 while (循环执行条件表达式) begin
    .....
end

```

while 语句在执行时, 首先判断循环执行条件表达式是否为真, 若为真, 执行后面的语句或语句块, 然后再回头判断循环执行条件表达式是否为真, 为真的话, 再执行一遍后面的语

句，如此不断，直到条件表达式不为真。因此，在执行语句中，必须有一条改变循环执行条件表达式的值的语句。

2. forever 语句

forever 语句的使用格式如下：

```

        forever      语句;
或      forever      begin
        .....
        end

```

forever 循环语句连续不断地执行后面的语句或语句块，常用来产生周期性的波形，作为仿真激励信号。forever 语句一般用在 initial 过程语句中，若要用它来进行模块描述，可用 disable 语句进行中断。

下面的 3 个程序段分别用 for、repeat、while 语句实现同一个循环。

```

.....
initial
begin
for(i=0; i<=4; i=i+1)
out = out +1;
end
.....

```

```

.....
initial
begin
repeat(5)
out = out +1;
end
.....

```

```

.....
initial
begin
i=0;
while(i<5)
begin
i=i+1;
out = out +1;
end
end
.....

```

下面的例子中，也用不同的语句实现了同一循环。

【例 5.18】同一循环的不同实现方式

```

module loop1;                                //方式 1
integer i;
initial
    for(i=0; i<4; i=i+1)                    //for 语句
    begin
        $display ("i=%h",i);
    end
endmodule

```

```

module loop2;                                //方式 2
integer i;

```



```

initial begin
    i=0;
    while (i<4)                //while 语句
    begin
        $display ("i=%h",i);
        i=i+1;
    end
end
endmodule

module loop3;                  //方式3
integer i;
initial begin
    i=0;
    repeat (4)                 //repeat 语句
    begin
        $display ("i=%h",i);
        i=i+1;
    end
end
endmodule

```

上述三个程序分别用 for、while 和 repeat 语句实现了显示一个 32 位整数的循环。用 ModelSim 软件进行仿真，三个程序的输出结果都是：

```

i=00000001      //i 是 32 位整数
i=00000002
i=00000003
i=00000004

```

5.7 编译向导

Verilog 语言和 C 语言一样也提供了编译向导功能。Verilog HDL 允许在程序中使用特殊的编译向导（Compiler Directives）语句，编译时，通常先对这些向导语句进行“预处理”，然后再将预处理的结果和源程序一起进行编译。

向导语句以符号“`”开头，以区别于其他语句。Verilog HDL 提供了十几条编译向导语句，如`define、`ifdef、`else、`endif、`restall 等。比较常用的有`define、`include 和`ifdef、`else、`endif 等，下面就这些常用语句进行说明。

5.7.1 宏替换`define

`define 语句用于将一个简单的名字或标志符（或称为宏名）来代替一个复杂的名字或字符串，其使用格式为：

`define 宏名（标志符） 字符串

如：

`define sum ina+inb+inc+ind

在上面的语句中，用简单的宏名 sum 来代替了一个复杂的表达式 ina+inb+inc+ind，采用了这样的定义形式后，在后面的程序中就可以直接用 sum 来代表表达式 ina+inb+inc+ind 了。比如程序中若出现：

assign out='sum+ine;

它实际上与 **assign** out= ina+inb+inc+ind+ine; 是完全等价的。

再比如：

`define WORDSIZE 8

reg['WORDSIZE:1] data; //相当于定义 reg[8:1] data;

从上面的例子可以看出：

- (1) `define 用于将一个简单的宏名来代替一个字符串或一个复杂的表达式。
- (2) 宏定义语句行末不加分号，这一点需要注意。
- (3) 在引用已定义的宏名时，不要忘了在宏名的前面加上符号“`”，以表示该名字是一个宏定义的名字。
- (4) 采用宏定义，可以简化程序的书写，也便于修改，若需要改变某个变量，只需改变`define 定义行，一改全改。比如在上面的例子中，定义 data 是一个 8 位的寄存器变量，若要将其改为 16 位，只须将定义行改为`define WORDSIZE 16 即可。

5.7.2 文件包含`include

`include 是文件包含语句，它可将一个文件全部包含到另一个文件中。其格式为：

`include “文件名”

下面是一个 16 位加法器的例子。在该例中，adder16 模块使用`include 语句调用了一个通用的加法器 adder 模块。或者说，adder16 模块包含了 adder 模块。

【例 5.19】使用了`include 语句的 16 位加法器

```
`include "adder.v"

module adder16(cout, sum, a, b, cin);
output cout;
parameter my_size = 16;
output[my_size - 1: 0] sum;
input[my_size - 1: 0] a, b;
input cin;
```

```

adder my_adder (cout, sum, a, b, cin);           //调用 adder 模块
endmodule
//下面是 adder 模块代码
module adder(cout, sum, a, b, cin);
parameter size = 16;
output cout;
output[size-1:0] sum;
input cin;
input[size-1:0] a, b;
    assign {cout, sum} = a + b + cin;
endmodule

```

用 MAX+PLUS II 软件对上面的程序编译仿真，图 5.11 是功能仿真的波形图（图中数据为十六进制表示）。

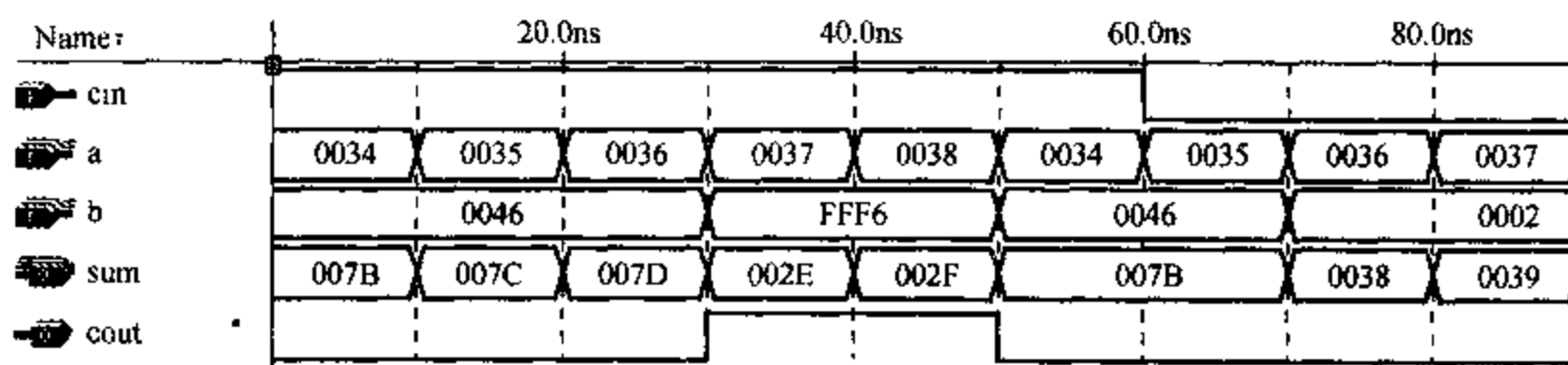


图 5.11 16 位加法器功能仿真波形图

使用`include 语句时应注意以下几点：

- (1) 一个`include 语句只能指定一个被包含的文件。
- (2) `include 语句可以出现在源程序的任何地方。被包含的文件若与包含文件不在同一个子目录下，必须指明其路径名。
- (3) 文件包含允许多重包含，比如文件 1 包含文件 2，文件 2 又包含文件 3 等。

5.7.3 条件编译`ifdef、`else、`endif

条件编译命令`ifdef、`else、`endif 可以指定仅对程序中的部分内容进行编译，这三个命令有如下两种使用形式。

- (1) `ifdef 宏名
语句块

`endif

这种形式的意思是：当宏名在程序中被定义过的话（用`define 语句定义），则下面的语句块参与源文件的编译，否则，该语句块将不参与源文件的编译。

- (2) `ifdef 宏名
语句块 1
`else 语句块 2
`endif

这种形式的意思是：当宏名在程序中被定义过的话（用`define 语句定义），则语句块 1 将被编译到源文件中，否则，语句块 2 将被编译到源文件中。

例如：

【例 5.20】条件编译举例

```
module compile(out, A, B);  
    output out;  
    input A, B;  
    `ifdef add //宏名为 add  
        assign out=A+B;  
    `else  
        assign out=A-B;  
    `endif  
endmodule
```

在上面的例子中，如果在程序中定义了“`define add”，则执行“assign out=A+B;”操作，若没有该语句，则执行“assign out=A-B;”操作。

思考与练习

1. 阻塞赋值和非阻塞赋值有什么本质的区别？
2. 了解行为语句的可综合性。
3. 用持续赋值语句描述一个 4 选 1 数据选择器。
4. 用行为语句设计一个 8 位计数器，每次在时钟的上升沿，计数器加 1，当计数器溢出时，自动从零开始重新计数。另外，计数器有同步复位端。
5. 设计一个 4 位移位寄存器。
6. initial 语句与 always 语句的关键区别是什么？

第6章 进程、任务与函数

本章内容简介

- ◆ 进程 (process)
- ◆ 进程间的通信
- ◆ 任务 (task)
- ◆ 函数 (function)
- ◆ 任务与函数的区别
- ◆ 顺序执行与并发执行
- ◆ Verilog-2001 新增语法结构简介

本章介绍 Verilog 进程、任务和函数的使用方法，并对语句的顺序执行和并行执行等进行必要的讨论，最后对 Verilog-2001 标准中新增的语法结构进行了简要的介绍。

6.1 进程 (process)

在 Verilog 硬件设计中，进程是一个很重要的概念。

6.1.1 进程

行为模型的本质是进程。一个进程可以被看作是一个独立的运行单元，它可能很简单，也可能很复杂，我们可以将数字系统的行为看作很多有机结合的进程的集合。

在 Verilog HDL 中，描述进程的基本语句是 `always` 和 `initial`。`always` 过程反复执行其中的块语句，而 `initial` 过程中的语句块只执行一次。除了 `always` 和 `initial` 过程块外，一个 `assign` 赋值语句、一个实例元件的调用都可以看作是一个进程。所以，一个 Verilog 模块中有如下表示进程的方式。

- ◆ `always` 过程块
- ◆ `initial` 过程
- ◆ `assign` 赋值语句
- ◆ 元件例化，如 `and al(out, a, b);`

比如下面的加法计数器中包含了两个进程：一个是 `always` 进程，执行复位、预置和加法计数等操作，这些操作必须有时钟的上升沿触发才能执行；另一个是 `assign` 持续赋值语句，

用于产生进位输出。

【例 6.1】加法计数器中的进程

```

module count(data, clk, reset, load, cout, qout);
output cout;
output[3:0] qout;
reg[3:0] qout;
input[3:0] data;
input clk, reset, load;

always @ (posedge clk)                                //进程 1, always 过程块
begin
    if (!reset)      qout= 4'h00;                        //同步清 0, 低电平有效
    else if (load)    qout= data;                          //同步预置
    else              qout=qout + 1;                      //加法计数
end

assign cout=(qout==4'hf)?1:0;                          //进程 2, 用持续赋值产生进位信号
endmodule

```

上面程序中的两个进程是并发执行的，两个进程在程序中的位置并不影响其执行的结果。进程 1 需要有 clk 的上升沿来启动，而对于进程 2，只要表达式 `cout=(qout==4'hf)?1:0` 中 qout 的值产生任何的变化，均会启动该进程的重新执行，产生新的 cout 的值。

进程具有以下一些重要的特点：

- ◆ 进程只有两种状态，即执行状态和等待状态。进程是否进入执行状态，取决于是否满足特定的条件，如敏感变量是否发生变化。一旦满足条件，进程即进入执行状态。当该进程执行完毕或遇到停止语句后，即停止执行，自动返回到起始语句，进入等待状态。

- ◆ 进程一般由敏感信号的变化来启动。

- ◆ 各个进程之间通过信号线进行通信。多个进程之所以能同步并发运行，一个很重要的原因就是有进程之间的信号线的通信和协调。

- ◆ 一个进程中只允许描述对应于一个时钟信号的同步时序逻辑。

- ◆ 进程之间是并发执行的。两个或更多个“always”过程块、“assign”持续赋值语句、实例元件调用等操作都是同时执行的。比如，在例 6.1 的加法计数器的描述中，进位输出信号的生成语句为：

```
assign cout=(qout==4'hf)?1:0;
```

若把该语句放在“always”模块之前，并不影响其功能，因为该赋值语句与 always 过程块是并发执行的，而与其所处的位置无关。

6.1.2 进程间的通信

在进行数字系统设计时，设计进程时应注意以下几点：

- ◆ 将硬件电路的行为以合理的方式映射为一些进程，对每个进程，以最有效的方式进

行设计，以最合理的方式描述并实现。

- ◆ 将组合逻辑实现的电路和用时序逻辑实现的电路应尽量分配到不同的进程中。
- ◆ 多个进程之间通过信号线进行通信。在设计中，为了使多个进程协调运行，我们可以设置一些握手信号，在进程中检测这些握手信号的状态，以决定是否进行必要的操作。在有的设计中，这种握手信号的协调是必不可少的。

当一个系统中包含有几个进程时，我们必须使它们同步，尤其是当不同的进程有共享数据的时候，这一点显得尤其重要。原因是进程 A 并不知道进程 B 当前正处于什么状态，其数据是否已处理完毕，因此进程 B 最好能给出一个信号来指明状态信息，在进程 A 中检测该信号的状态，以进行合适的操作。由此可以考虑采用一种通信机模型，用握手信号来实现两个进程间的通信与协调，如图 6.1 所示。

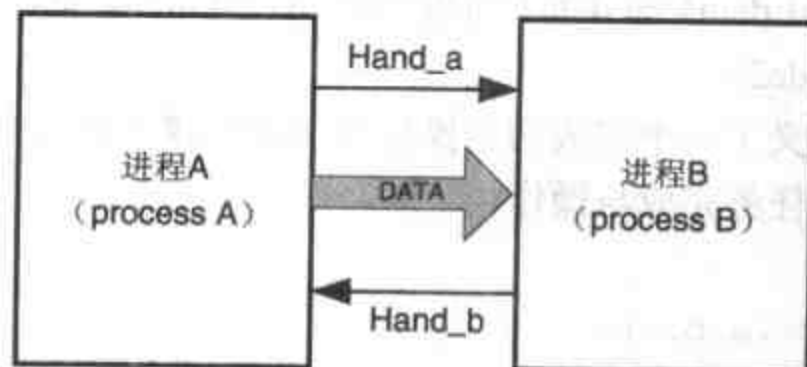


图 6.1 用握手信号实现两个进程间的通信与协调

在图 6.1 中，进程 A 对数据进行必要的处理后，发给进程 B 作进一步的处理，进程 A 与进程 B 是同时执行的，因此相互之间需要进行协调，否则，进程 B 并不知道输入进来的数据是否是进程 A 已经处理完的。因此，可考虑设置两个握手信号：Hand_a 和 Hand_b，进程 A 将输入数据处理完毕，即将 Hand_a 信号置位，在进程 B 中检测 Hand_a 信号的状态，若有上跳沿来到，即表明一组新的数据已经准备好，即可对输入数据进行操作，操作完毕后，同样将 Hand_b 信号置位，告诉进程 A 本次数据已处理完毕。如此，模仿简单的通信机终端，实现数据的有效处理。

6.2 任务 (task)

任务和函数的关键字分别是 task 和 function，利用任务和函数可以把一个大的程序模块分解成许多小的任务和函数，以方便调试，并且能使写出的程序结构更清晰。

任务 (task) 定义与调用的格式分别如下：

定义： **task** <任务名>; //注意无端口列表

端口及数据类型声明语句；

其他语句；

endtask

任务调用的格式为：

<任务名> (端口 1, 端口 2,);

需要注意的是：任务调用变量和定义时说明的 I/O 变量是一一对应的。比如下面是一个

定义任务的例子。

```
task test;
input in1,in2;
output out1,out2;
```

```
#1 out1=in1 & in2;
#1 out2=in1 | in2;
```

```
endtask
```

当调用该任务时，可使用如下语句：

```
test (data1,data2, code1,code2);
```

调用任务 test 时，变量 data1 和 data2 的值赋给 in1 和 in2，而任务执行完后，out1 和 out2 的值则赋给了 code1 和 code2。

在下面的例子中，定义了一个完成两个操作数按位与操作的任务，然后在后面的算术逻辑单元的描述中，调用该任务完成与操作。

【例 6.2】任务举例

```
module alutask(code,a,b,c);
input[1:0] code;
input[3:0] a,b;
output[4:0] c;
reg[4:0] c;
```

```
task my_and;                                //任务定义，注意无端口列表
input[3:0] a,b;                             //a,b,out 名称的作用域范围为 task 任务内部
output[4:0] out;
integer i;
begin
for(i=3;i>=0;i=i-1)
out[i]=a[i]&b[i];                          //按位与
end
endtask
```

```
always@(code or a or b)
```

```
begin
```

```
case(code)
```

```
2'b00: my_and(a,b,c);
```

/* 调用任务 my_and，需注意端口列表的顺序应与任务定义中的一致，这里的 a,b,c 分别对应任务定义中的 a,b,out */

```
2'b01: c=a|b;                             //或
```

```
2'b10: c=a-b;                             //相减
```



```

        2'b11: c=a+b;          //相加
    endcase
end
endmodule

```

上面的程序用 Synplify Pro 软件进行综合，图 6.2 是其 RTL 原理图。

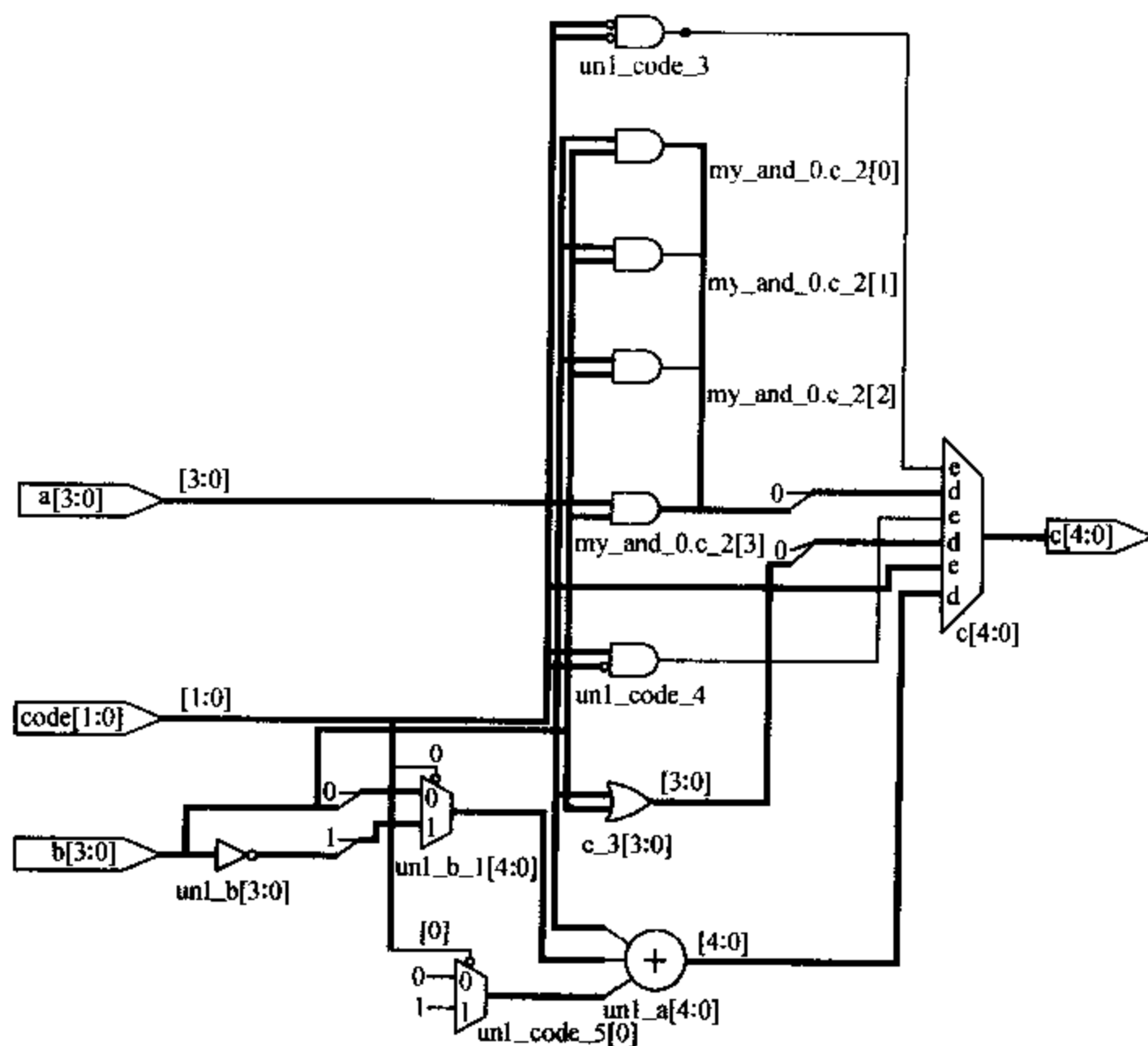


图 6.2 RTL 原理图

为了检验程序的功能，编写下面的测试模块，对其进行仿真。

【例 6.3】测试程序

```

#include "alutask.v"
module alu_tp;
    reg[3:0] a,b;
    reg[1:0] code;
    wire[4:0] c;
    parameter DELY = 100;

    alutask ADD(code,a,b,c);          //调用被测试模块

    initial begin

```

```

        code=4'd0; a= 4'b0000; b= 4'b1111;
#DELY    code=4'd0; a= 4'b0111; b= 4'b1101;
#DELY    code=4'd1; a= 4'b0001; b= 4'b0011;
#DELY    code=4'd2; a= 4'b1001; b= 4'b0011;
#DELY    code=4'd3; a= 4'b0011; b= 4'b0001;
#DELY    code=4'd3; a= 4'b0111; b= 4'b1001;
#DELY    $finish;
end
initial $monitor($time,, "code=%b a=%b b=%b c=%b", code,a,b,c);
endmodule

```

用 ModelSim 软件对上面的程序编译和仿真，输出结果如下：

```

0, code=00 a=0000 b=1111 c=x0000
100, code=00 a=0111 b=1101 c=x0101
200, code=01 a=0001 b=0011 c=00011
300, code=10 a=1001 b=0011 c=00110
400, code=11 a=0011 b=0001 c=00100
500, code=11 a=0111 b=1001 c=10000

```

通过验证，证明程序完成了既定的功能。

通过上面的例子，需强调的是在使用任务时，应注意以下几点：

- ◆ 任务的定义与调用须在一个 module 模块内。
- ◆ 定义任务时，没有端口名列表，但需要紧接着进行输入、输出端口和数据类型的说明。
- ◆ 当任务被调用时，任务被激活。任务的调用与模块调用一样通过任务名调用来实现。调用时，需列出端口名列表，端口名的排序和类型必须与任务定义中的相一致。
- ◆ 一个任务可以调用别的任务和函数，可以调用的任务和函数个数不限。

6.3 函数 (function)

本节介绍函数的使用方法。

6.3.1 函数

函数的目的是返回一个值，以用于表达式的计算。函数的定义格式为：

```

function <返回值位宽或类型说明> 函数名;
    端口声明;
    局部变量定义;
    其他语句;

```

endfunction

<返回值位宽或类型说明>是一个可选项,如果缺省,则返回值为一位寄存器类型的数据。

下面的例子中定义了一个函数。

【例 6.4】函数

```
function[7:0] get0;
input[7:0] x;
reg[7:0] count;
integer i;
begin
count=0;
for (i=0;i<=7;i=i+1)
if (x[i]=1'b0) count=count+1;
get0=count;
end
endfunction
```

上面的 get0 函数循环核对输入数据 x 的每一位,统计出 x 中“0”的个数,并返回一个适当的值。

函数的定义中蕴含了一个与函数同名的、函数内部的寄存器。在函数定义时,将函数返回值所使用的寄存器名称设为与函数同名的内部变量,因此函数名被赋予的值就是函数的返回值。例如在上面的例子中, get0 最终赋予的值即为函数的返回值。

函数的调用是通过将函数作为表达式中的操作数来实现的。调用格式如下:

<函数名> (<表达式><表达式>);

比如使用持续赋值语句调用函数 get0 时,可以采用如下语句:

```
assign out=is_legal ? get0 (in) : 1'b0;
```

函数的使用与任务相比有更多的限制和约束。例如,函数不能启动任务,在函数中不能包含有任何的时间控制语句,同时定义函数时至少要有一个输入参量等,这些都需要在使用时注意。

下面的例子用函数定义了一个 8-3 编码器。

【例 6.5】用函数和 case 语句描述的编码器 (不含优先顺序)

```
module code_83(din, dout);
input[7:0] din;
output[2:0] dout;

function[2:0] code;                                //函数定义
input[7:0] din;                                     //函数只有输入,输出为函数名本身
    casex (din)
        8'b1xxx_xxxx : code = 3'h7;
        8'b01xx_xxxx : code = 3'h6;
        8'b001x_xxxx : code = 3'h5;
```

```

    8'b0001_xxxx : code = 3'h4;
    8'b0000_1xxx : code = 3'h3;
    8'b0000_01xx : code = 3'h2;
    8'b0000_001x : code = 3'h1;
    8'b0000_000x : code = 3'h0;
    default: code = 3'hx;
  endcase
endfunction

assign dout = code(din); //函数调用
endmodule

```

如图 6.3 所示是例 6.5 的 RTL 综合视图。

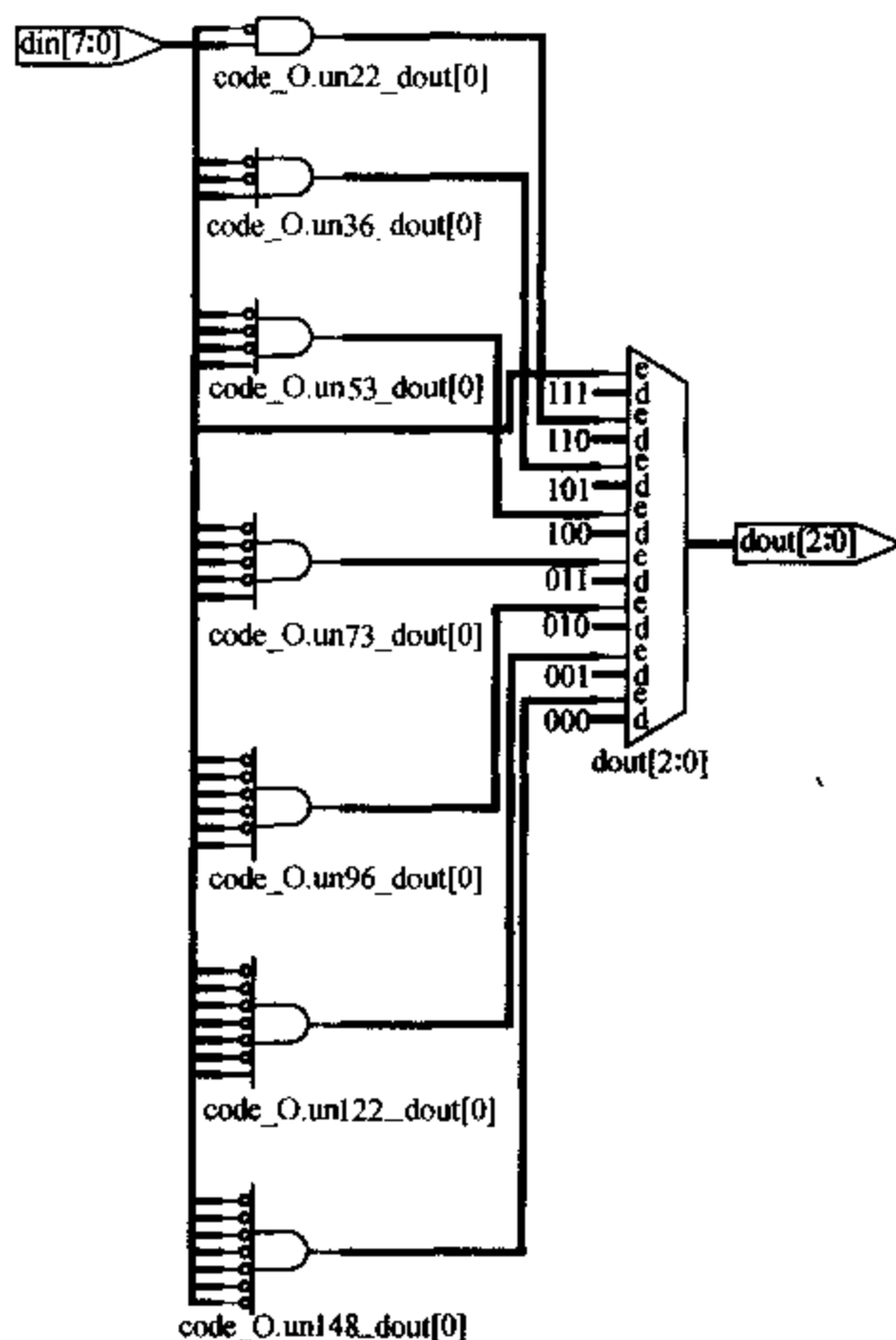


图 6.3 例 6.5 的 RTL 综合视图

与 C 语言相类似，Verilog HDL 使用函数，以应对对不同操作数采取同一运算的操作。函数在综合时被转换成具有独立运算功能的电路，每调用一次函数，相当于改变这部分电路的输入，以得到相应的计算结果。

下面的例子中定义了一个实现阶乘运算的函数 `factorial`，该函数返回一个 32 位的寄存器类型的值。采用同步时钟触发运算的执行，每个 `clk` 时钟周期都会执行一次运算。

【例 6.6】阶乘运算函数

```

module funct(clk, n, result, reset);
output[31:0] result;
input[3:0] n;
input reset, clk;
reg[31:0] result;
always @(posedge clk)                                //在 clk 的上升沿时执行运算
begin
    if(!reset) result<=0;                                //复位
    else begin
        result <= 2 * factorial(n);                    //调用 factorial 函数
    end
end

function[31:0] factorial;                                //阶乘运算函数定义（注意无端口列表）
input[3:0] opa;                                          //函数只能定义输入端，输出端口为函数名本身
reg [3:0] i;
    begin
        factorial = opa ? 1 : 0;
        for(i= 2; i <= opa; i = i+1)                //该句若要综合通过，opa 应赋具体的数值，如
                                                                9 等
        factorial = i* factorial;                        //阶乘运算
    end
endfunction
endmodule

```

为了验证该函数的功能，编写下面的测试模块，通过调用系统任务 `$display` 在时钟的下降沿显示每次计算的结果。

【例 6.7】测试程序

```

'define clk_cycle 50
'include "funct.v"
module funct_tp;
reg[3:0] n;
reg reset, clk;
wire[31:0] result;
    initial                                                //定义激励向量
    begin

```

```

n=0; reset=1; clk=0;
for(n=0;n<=15;n=n+1)
#100 n=n;
end

initial $monitor($time,,"n=%d result=%d", n,result);
                                     //定义输出显示格式
always # `clk_cycle clk=~clk;      //产生时钟信号

funct funct_try (.clk(clk),.n(n),.result(result),.reset(reset));
                                     //调用被测试模块

endmodule

```

用 ModelSim 软件对上面的程序进行编译和仿真，输出结果如下：

```

0 n= 0 result=    x
50 n= 0 result=    0

100 n= 1 result=    0
150 n= 1 result=    2

200 n= 2 result=    2
250 n= 2 result=    4

300 n= 3 result=    4
350 n= 3 result=   12

400 n= 4 result=   12
450 n= 4 result=   48

500 n= 5 result=   48
550 n= 5 result=  240

600 n= 6 result=  240
650 n= 6 result= 1440

700 n= 7 result= 1440
750 n= 7 result=10080

```

可以发现，该函数完成了预想的功能。

通过上面的例子，需强调指出的是在使用函数时，应注意以下几点：

- ◆ 函数的定义与调用须在一个 module 模块内。
- ◆ 函数只允许有输入变量，且必须至少有一个输入变量，输出变量由函数名本身担任，比如在上面的例子中，函数名 factorial 即是输出变量，在调用该函数时：`result <= 2 * factorial(n);` 自动将 n 的值赋给函数的输入变量 opa，完成函数计算后，将结果通过 factorial 名字本身返回，作为一个操作数参与 result 表达式的计算。因此，在定义函数时，需对函数名说明其类型和位宽。
- ◆ 定义函数时，没有端口名列表，但调用函数时，需列出端口名列表，端口名的排序和类型必须与定义时的相一致，这一点与任务相同。
- ◆ 函数可以出现在持续赋值 assign 的右端表达式中。
- ◆ 函数不能调用任务，而任务可以调用别的任务和函数，且调用任务和函数个数不受限制。

6.3.2 任务与函数的区别

表 6.1 对任务与函数的区别进行了比较。

本节介绍了 task 和 function 的使用, 合理使用 task 和 function 会使程序显得结构清晰而简洁, 一般的综合器对 task 和 function 都是支持的, 但也有的综合器不支持 task, 需要在使用的时候加以注意。

表 6.1 任务与函数的区别

比较项目	任务 (task)	函数 (function)
输入与输出	可有任意个各种类型的参数	至少有一个输入, 不能将 inout 类型作为输出
调用	任务只可在过程语句中调用, 而不能在连续赋值语句 assign 中调用	函数可作为表达式中的一个操作数来调用, 在过程赋值和连续赋值语句中均可以调用
定时和事件控制(#, @和 wait)	任务可以包含定时和事件控制语句	函数不能包含这些语句
调用其他任务和函数	任务可调用其他任务和函数	函数可调用其他函数, 但不可以调用其他任务
返回值	任务不同表达式返回值	函数向调用它的表达式返回一个值

6.4 顺序执行与并发执行

用 Verilog HDL 设计电路时, 必须搞清楚哪些操作是同时发生的, 哪些是顺序发生的。

首先, 进程之间是并发执行的。两个或更多个“always”过程块、“assign”持续赋值语句、实例元件调用等操作都是同时执行的。

比如, 在例 6.1 的加法计数器的描述中, 进位输出信号的生成语句为:

```
assign cout=(qout==4'hf)?1:0;
```

若把该语句放在“always”模块之前, 并不影响其功能, 因为该赋值语句与 always 过程块是并发执行的, 而与其所处的位置无关。

在“always”模块内部, 其语句如果是非阻塞赋值, 也是并发执行的; 而如果是阻塞赋值, 则语句是按照指定的顺序执行的, 语句的书写顺序对程序的执行结果有着直接的影响。比如下面的例子。

【例 6.8】顺序执行模块 1

```
module serial1(q,a,clk);
output q,a;
input clk;
reg q,a;
always @(posedge clk)
    begin
        q=~q;
        a=~q;
    end
endmodule
```

【例 6.9】顺序执行模块 2

```
module serial2(q,a,clk);
output q,a;
input clk;
reg q,a;
always @(posedge clk)
    begin
        a=~q;
        q=~q;
    end
endmodule
```

上面的两个例子，其区别只是在“always”模块内，把两个赋值语句的顺序相互颠倒。分别对上面的两个模块进行模拟，得到的波形如图 6.4 和图 6.5 所示。

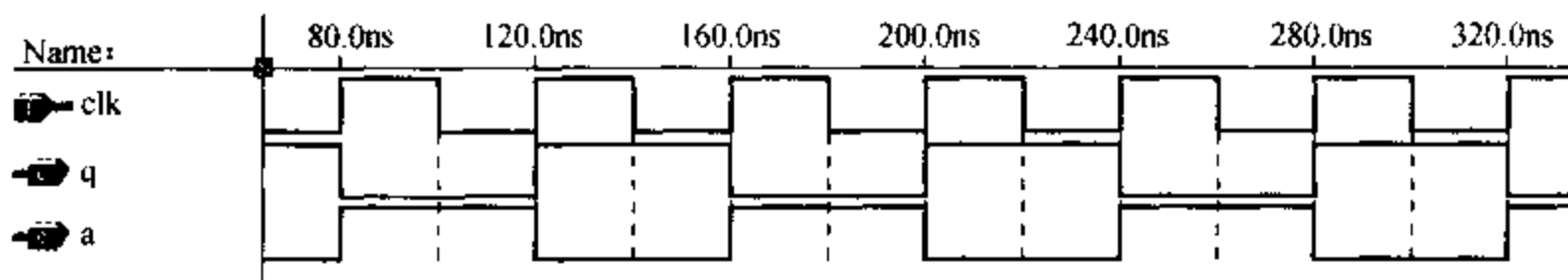


图 6.4 例 6.8 的仿真波形图

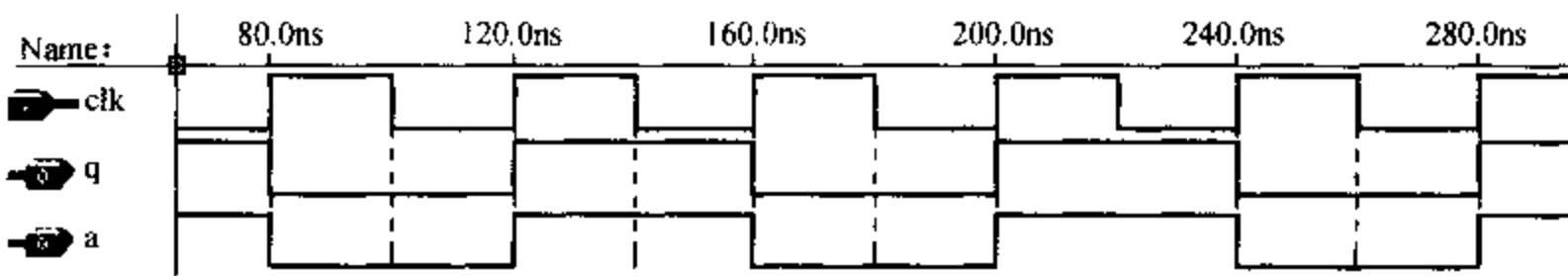


图 6.5 例 6.9 的仿真波形图

从波形图我们可以看到，在模块 1 中，q 先取反，然后再取反给 a；在模块 2 中，q 取反后赋值给 q 和 a，a 和 q 的波形是完全一样的。

例 6.8 和例 6.9 综合后的电路分别如图 6.6 和图 6.7 所示。

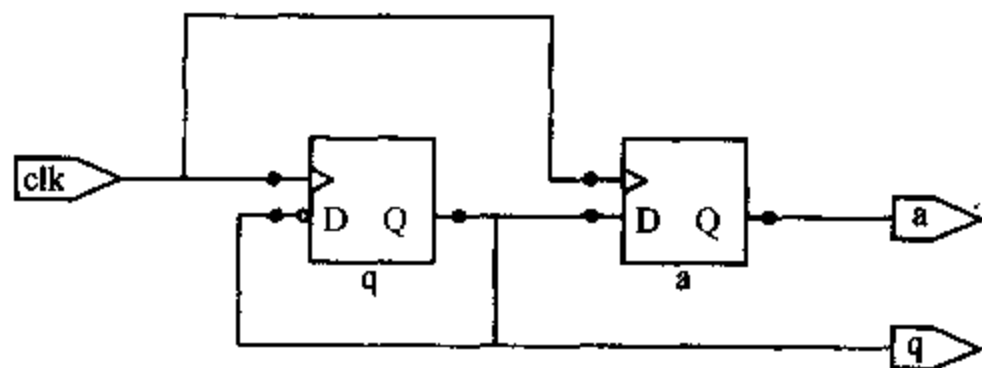


图 6.6 例 6.8 的综合结果

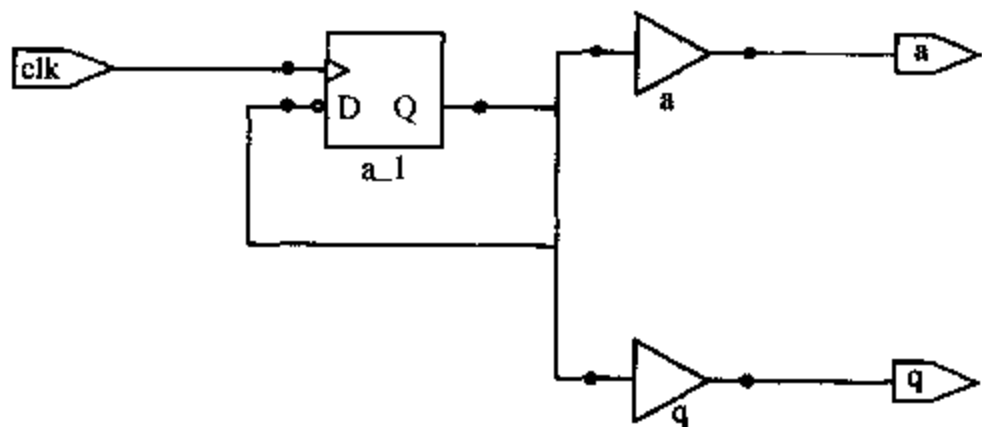


图 6.7 例 6.9 的综合结果

如果将上述两句赋值语句分别放在两个“always”模块中，如下面例 6.10 和例 6.11 两个程序所示，则经过仿真可以发现：这两个“always”模块放置的顺序对结果并没有影响，因为这两个模块是并发执行的。

【例 6.10】并行执行模块 1

```

module para11(q,a,clk);
output q,a;
input clk;
reg q,a;
always @(posedge clk)
    begin
        q=~q;
    end
always @(posedge clk)
    begin
        a=~q;
    end
endmodule

```

【例 6.11】并行执行模块 2

```

module para12(q,a,clk);
output q,a;
input clk;
reg q,a;
always @(posedge clk)
    begin
        a=~q;
    end
always @(posedge clk)
    begin
        q=~q;
    end
endmodule

```

例 6.10 和例 6.11 的仿真波形与上面例 6.8 的仿真波形是完全相同的（如图 6.4 所示）。

6.5 Verilog-2001 新增语法结构简介

前面介绍的 Verilog 语法结构主要是基于 IEEE 标准“1364-1995”（简称为 Verilog-1995），现在，IEEE “1364-2001” Verilog 标准（这里简称 Verilog-2001）也已经获得通过，并有很多 Verilog 综合器、仿真器等已经支持部分 Verilog-2001 语法结构，如 Synplify Pro。本节将对一些 Verilog-2001 新的语法结构进行介绍，重点介绍综合器支持的结构。

Verilog-2001 对 Verilog 语言作了很多扩充和增强，在可配置的 IP 模型、深亚微米结构和设计管理方面增加了更多的支持，另外，增加和修改了一些语法结构，使其更加好用，这些增强包括：增加了一些运算符，如指数运算符、算术移位运算符等；增加了一些关键字；增加了一些系统函数和系统任务；增强了对符号的支持；增强了对 SDF 的支持；编程语言接口（PLI）的功能也得到了加强。

下面对部分新增的语法结构举例进行说明。

（1）数据类型和端口类型组合声明

Verilog-2001 中允许对端口类型和数据类型组合在一起进行定义，并直接放在模块的端口列表中。比如下面的例子可对比 Verilog-1995 和 Verilog-2001 的不同。

```

module adder8 (sum, cout, cin, a, b);           //Verilog-1995
output [7:0] sum;
output cout;
input [7:0] a, b;
input cin;
reg [7:0] sum;

```

```
reg cout;
wire[7:0] a, b;
wire cin;
```

上面的声明程序段在 Verilog-2001 中可用下面的语句代替, 使程序更简洁。

```
module adder8 (output reg[7:0] sum, output reg cout,
               input wire cin, input wire[7:0] a, b);
```

(2) 敏感信号列表中用逗号分隔敏感信号

在 Verilog-1995 中, 我们书写敏感信号列表时, 通常用 “or” 来连接敏感信号, 比如:

```
always @(a or b or cin)
    {cout,sum}=a+b+cin;
always @(posedge clk or negedge clr)
    if (!clr)    q <= 0;
    else        q <= d;
```

在 Verilog-2001 中可用逗号分隔敏感信号, 上面的语句可写为下面的形式。

```
always @(a, b, cin)                //用逗号分隔信号
    {cout,sum}=a+b+cin;
always @(posedge clock, negedge clr)
    if (!clr)    q <= 0;
    else        q <= d;
```

(3) 在敏感信号列表中使用通配符 “*”

用 always 过程块描述组合逻辑时, 应在敏感信号列表中列出所有的输入信号, 在 Verilog-2001 中可用通配符 “*” 来表示包括该过程块中的所有信号变量。

比如: 在 Verilog-1995 中, 一般这样写敏感信号列表:

```
always @(a or b or cin)
    {cout,sum}=a+b+cin;
```

上面的敏感信号列表在 Verilog-2001 中可表示为如下两种形式。

```
always @(*)                //形式 1
    {cout,sum}=a+b+cin;
always @*                  //形式 2
    {cout,sum}=a+b+cin;
```

上面两种形式是等价的。

(4) 对符号的扩展

在 Verilog-1995 中, 整数可以带正负号, 但 net 型和 reg 型的变量是不允许带符号的。

在 Verilog-2001 中, 对带符号的运算作了如下一些扩充:

- ◆ net 型和 reg 型的变量可以声明为符号 (signed) 变量。
- ◆ 函数的返回值可以带符号。
- ◆ 任意宽度的整数都可以带符号。
- ◆ 操作数可以从无符号数转换为有符号数。
- ◆ 增加了算术移位操作符。

“signed”是 Verilog-1995 中的保留字，但没有使用，在 Verilog-2001 中，用“signed”保留字来定义数据类型、端口、整数、函数等，比如下面的例子。

```
wire signed[3:0] a;
reg signed[7:0] out;
output signed[15:0] sum;
function signed[31:0] alu;
16'sh54af
parameter p0 = 2'sb00, p1 = 2'sb01;
```

Verilog-2001 还增加了两个系统函数 \$signed 和 \$unsigned，以实现无符号数转化为有符号数，或者有符号数转化为无符号数。

比如：

```
c = $signed (s);          /*将 s 转化为带符号数，赋给 c */
d = $unsigned (s);        /*将 s 转化为不带符号数，赋给 d */
```

(5) 新增运算符

Verilog-2001 增加了算术移位操作符“>>>”和“<<<”，对于有符号数，执行算术移位操作时，将符号位填补移出的位，比如，如果定义有符号数 $A = 8'sb10100011$ ，则执行逻辑移位操作和算术移位操作后的值分别如下：

```
A>>>3;          //逻辑右移后其值为 8'b00010100
A>>>>3;         //算术右移后其值为 8'b11110100
```

Verilog-2001 还增加了指数运算符“**”，执行指数运算。比如：

```
Result=base**exponent;
```

一般更多使用的是底数为 2 的指数运算，如 2^n 。

(6) 任务和函数中增加关键字“automatic”

Verilog-2001 增加了一个关键字“automatic”，可用于任务和函数的定义中。如用于函数的话，表示函数的迭代调用，比如下面的例子中，通过函数自身的迭代调用，实现了 32 位无符号整数的阶乘运算 ($n!$)。

```
function automatic [63:0] factorial;
input [31:0] n;
if (n == 1)
factorial = 1;
else
factorial = n * factorial(n-1);          //迭代调用
endfunction
```

(7) generate 语句

Verilog-2001 新增了语句“generate”，通过 generate 循环，可以产生一个对象（比如一个元件或一个模块等）的多个例化，为可变尺度的设计提供了方便。generate 语句一般在循环和条件语句中使用。为此，Verilog-2001 增加了四个关键字 generate、endgenerate、genvar 和 localparam。genvar 是一个新的数据类型，用在 generate 循环中的标尺变量必须定义为 genvar 型数据。

下面是一个用 generate 循环描述的多位加法器的例子，它采用了一个 generate for 循环产生元件的例化和元件间的连接关系。

```

module nbit_add(a, b, cin, sum, cout);
parameter size=4;
input[size-1:0] a, b;
input cin;
output[size-1:0] sum;
output cout;
wire[size:0] c;

genvar i;
assign c[0]=cin;
assign cout=c[size];

generate
for(i=0;i<size;i=i+1)
begin
wire n1,n2,n3;
xor g1(n1, a[i], b[i]);
xor g2(sum[i], n1, c[i]);
and g3(n2, a[i], b[i]);
and g4(n3, n1, c[i]);
or g5(c[i+1], n2, n3);
end
endgenerate
endmodule

```

思考与练习

1. 分别用任务和函数描述一个 4 选 1 多路选择器。
2. 总结任务和函数的区别。
3. 在 Verilog 中，进程有哪些表现形式？
4. 在 Verilog 中，哪些操作是并发执行的，哪些操作是顺序执行的？
5. 了解所使用综合器对 Verilog-2001 语法结构的支持情况，用 Verilog-2001 语法结构编写模块并综合。

第7章 Verilog HDL 的描述风格

本章内容简介

- ◆ 结构 (Structural) 描述
- ◆ 行为 (Behavioural) 描述
- ◆ 数据流 (Data Flow) 描述
- ◆ Verilog 门原语

本章介绍 Verilog HDL 的描述风格, 包括门级结构描述、行为描述、数据流描述以及混合描述等。在设计电路与系统时, 一般优先选择高层次的描述方式。门级结构描述的电路一般综合器很难再进行优化, 所以除非一些关键路径的设计, 一般较少采用。而行为级和更高级别的描述则给综合器提供了可优化的空间。在设计时, 可以灵活选用最适宜的设计风格。

7.1 概 述

Verilog HDL 是一种用于数字逻辑电路设计的语言。用 Verilog HDL 描述的电路就是该电路的 Verilog HDL 模型。

Verilog HDL 既是一种行为描述语言, 也是一种结构描述语言, 也就是说, 既可以用电路的功能描述也可以用元器件和它们之间的连接来建立所设计电路的 Verilog HDL 模型。

Verilog HDL 是一种能够在多个级别对数字电路和数字系统进行描述的语言, Verilog HDL 模型可以是实际电路不同级别的抽象, 这些抽象级别可分为下面一些层次:

- (1) 系统级 (System Level);
- (2) 算法级 (Algorithm Level);
- (3) 寄存器传输级 (RTL, Register Transfer Level);
- (4) 门级 (Gate Level);
- (5) 开关级 (Switch Level)。

其中, 前 3 种属于高级别的描述方法, 是多数设计所采用的方式, 门级描述主要是利用逻辑门以及逻辑门之间的连接来构筑电路模型, 而开关级的模型则主要是描述器件中三极管和存储节点以及它们之间的连接关系。Verilog HDL 在开关级也提供了完整的原语 (primitive), 可以精确地建立 MOS 器件的底层模型。

Verilog 程序有下面 3 种描述设计的方式:

- ◆ 结构 (Structural) 描述;
- ◆ 行为 (Behavioural) 描述;
- ◆ 数据流 (Data Flow) 描述方式。

还可以采用上述方式的混合形式来描述设计, 下面分别介绍这三种常用的设计描述方式。

7.2 结 构 描 述

在 Verilog 程序中可通过如下方式来描述电路的结构:

- ◆ 调用 Verilog 内置门元件 (门级结构描述);
- ◆ 调用开关级元件 (开关级结构描述);
- ◆ 用户自定义元件 UDP (也在门级)。

此外, 在多层次结构电路的设计中, 不同模块间的调用也可以认为是结构描述。在上述的结构描述方式中, 用户自定义元件 (UDP) 由于主要与仿真有关, 因此放在第 8 章中介绍, 而开关级结构描述不是本书讨论的重点, 本节主要介绍门级结构描述。

7.2.1 Verilog HDL 内置门元件

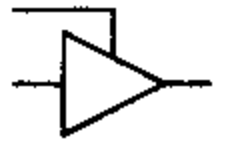
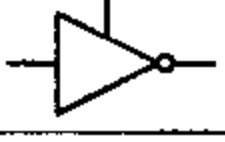
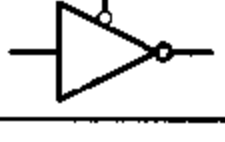
Verilog HDL 内置 26 个基本元件 (Basic Primitive), 其中 14 个是门级元件 (Gate-level Primitive), 12 个为开关级元件 (Switch-level Primitive), 这 26 个基本元件及其分类见表 7.1。

表 7.1 Verilog 语言内置基本元件

类 型		元 件
基本门 (Basic Gate)	多输入门	and, nand, or, nor, xor, xnor
	多输出门	buf, not
三态门 (Three State Drivers)	允许定义驱动强度	buf0, buf1, notif0, notif1
MOS 开关 (MOS Switches)	无驱动强度	nmos, pmos, cmos, rmos, rpmos, rcmos
双向开关 (Bi-directional switches)	无驱动强度	tran, tranif0, tranif1
	无驱动强度	rtran, rtranif0, rtranif1
上拉、下拉电阻	允许定义驱动强度	pullup, pulldown

这里重点介绍门级元件。Verilog HDL 中丰富的门元件为电路的门级结构描述提供了方便。Verilog 语言中的门元件见表 7.2。

表 7.2 Verilog HDL 常用的内置门元件

类 别	关 键 字	符号示意图	门 名 称
多输入门	and		与门
	nand		与非门
	or		或门
	nor		或非门
	xor		异或门
	xnor		异或非门
多输出门	buf		缓冲器
	not		非门
三态门	bufif1		4 种三态门
	buiif0		
	notif1		
	notif0		

1. 基本门的逻辑真值表

在下面的表格中将各种基本门的真值表进行了罗列，表 7.3 至表 7.7 分别是与非门、或非门、异或门、异或非门、缓冲器和非门的真值表。

表 7.3 nand (与非门) 真值表

nand	0	1	x	z
0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

表 7.4 nor (或非门) 真值表

nor	0	1	x	z
0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

表 7.5 xor (异或门) 真值表

xor	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

表 7.6 xnor (异或非门) 真值表

xnor	0	1	x	z
0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

表 7.7 buf (缓冲器) 与 not (非门) 真值表

buf		not	
输入	输出	输入	输出
0	0	0	1
1	1	1	0
x	x	x	x
z	x	z	x

bufif1、bufif0、notif1、notif0 4 种三态门的真值表分别见表 7.8 至表 7.11。表中的 L 代表 0 或 z, H 代表 1 或 z。

表 7.8 bufif1 的真值表

bufif1		Enable (使能端)			
		0	1	x	z
输入	0	z	0	L	L
	1	z	1	H	H
	x	z	x	x	x
	z	z	x	x	x

表 7.9 bufif0 的真值表

bufif0		Enable (使能端)			
		0	1	x	z
输入	0	0	z	L	L
	1	1	z	H	H
	x	x	z	x	x
	z	x	z	x	x

表 7.10 notif1 的真值表

notif1		Enable (使能端)			
		0	1	x	z
输入	0	z	1	H	H
	1	z	0	L	L
	x	z	x	x	x
	z	z	x	x	x

表 7.11 notif0 的真值表

Notif0		Enable (使能端)			
		0	1	x	z
输入	0	1	z	H	H
	1	0	z	L	L
	x	x	z	x	x
	z	x	z	x	x

2. 门元件的调用

调用门元件的格式为:

门元件名字 <例化的门名字> (<端口列表>)

其中, 普通门的端口列表按下面的顺序列出:

(输出, 输入 1, 输入 2, 输入 3,);

例如:

```
and a1(out,in1,in2,in3);           //三输入与门, 其名字为 a1
and a2 (out,in1,in2);             //二输入与门, 其名字为 a2
```

对于三态门, 则按如下顺序列出输入、输出端口:

(输出, 输入, 使能控制端);

例如:

```
bufif1 mytri1(out,in,enable);      //高电平使能的三态门
bufif0 mytri2(out,a,ctrl);         //低电平使能的三态门
```

对于 **buf** 和 **not** 两种元件的调用, 需要注意的是: 它们允许有多个输出, 但只能有一个输入。

例如:

```
not N1(out1,out2,in);              //1 个输入 in, 2 个输出 out1,out2
buf B1(out1,out2,out3,in);         //1 个输入 in, 3 个输出 out1,out2,out3
```

7.2.2 门级结构描述

如图 7.1 所示是一个用基本门实现的 4 选 1 数据选择器 (MUX) 的原理图。

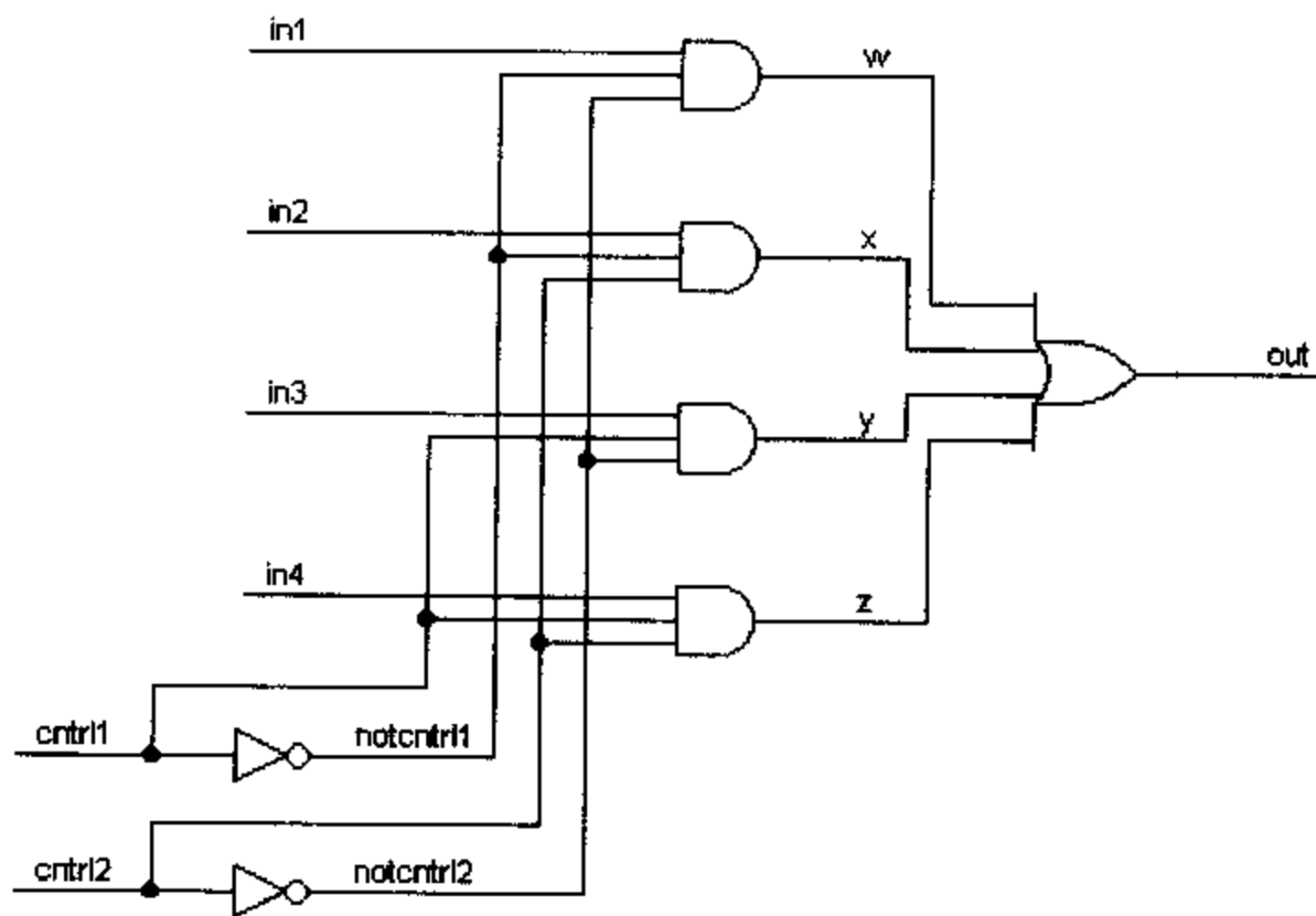


图 7.1 用基本门实现的 4 选 1 MUX 原理图

对于该电路, 采用门级 Verilog HDL 描述如下。

【例 7.1】调用门元件实现的 4 选 1 MUX

```
module mux4_1a(out,in1,in2,in3,in4,ctrl1,ctrl2);
output out;
```

```

input  in1,in2,in3,in4,cntrl1,cntrl2;
wire  notcntrl1,notcntrl2,w,x,y,z;
not  (notcntrl1,cntrl2),
      (notcntrl2,cntrl2);
and  (w,in1,notcntrl1,notcntrl2),
      (x,in2,notcntrl1,cntrl2),
      (y,in3,cntrl1,notcntrl2),
      (z,in4,cntrl1,cntrl2);
or   (out,w,x,y,z);
endmodule

```

7.3 行为描述方式

硬件电路的结构描述侧重于表示一个电路由哪些基本元件组成，以及这些基本元件的相互连接关系。而硬件电路的行为特性则主要反映该电路输入、输出信号间的相互关系，这种相互关系以何种方式表达更为合适，取决于所处的阶段或者说取决于抽象的级别。

在可综合的电路设计中，一般采用 `always` 过程语句来描述电路的行为特性，这种行为描述方式既适合于设计时序逻辑电路，也可以用于组合逻辑电路的设计。

结构级的描述在进行仿真（simulation）时要优于行为性描述；而行为描述在进行综合（synthesis）时则更优越一些，也更有利于综合工具开展工作。在电路的规模相当大或者需要描述复杂的时序关系的时候，我们通常侧重于使用行为描述方式。

下面用行为语句 `case` 来实现上例中的 4 选 1 MUX。

【例 7.2】用 `case` 语句描述的 4 选 1 MUX

```

module  mux4_1b(out,in1,in2,in3,in4,cntrl1,cntrl2);
output  out;
input  in1,in2,in3,in4,cntrl1,cntrl2;
reg  out;
always@(in1 or in2 or in3 or in4 or cntrl1 or cntrl2)
    case({cntrl1,cntrl2})
        2'b00:out=in1;
        2'b01:out=in2;
        2'b10:out=in3;
        2'b11:out=in4;
        default:out=2'bx;
    endcase
endmodule

```

下面是用行为描述方式实现的 4 位计数器。

【例 7.3】 行为描述方式实现的 4 位计数器

```

module count4(clk, clr, out);
  input clk, clr;
  output[3:0] out;
  reg[3:0] out;
  always @(posedge clk or posedge clr)
    begin
      if (clr)    out<=0;
      else      out<=out+1;
    end
endmodule

```

在用 Verilog 语言编写仿真测试程序时，我们也多数采用行为描述方式，采用“initial”和“always”过程来定义激励向量，定义输出显示格式等。

比如：

```

parameter cycle = 100 ;
always # (cycle/2) clk = ~clk;           // always 块产生时钟波形

```

再比如：

```

initial begin                               //用 initial 过程设定激励波形
    A = 0; B = 1; C = 0;
    #100    A = 1 ; B = 0;
    #100    A = 0;
    #100    B = 1; C = 0 ;
    #100    $finish;
end

```

对于设计者而言，采用的描述级别越高，设计越容易，而对于综合器而言，行为级的描述为综合器的优化提供了更大的空间，较门级结构描述更能发挥综合器的性能，所以在电路设计中，除非一些关键路径的设计采用门级结构描述外，一般更多地采用行为建模方式。

7.4 数据流描述方式

通常将采用持续赋值语句描述的设计称为数据流描述方式，即：

```

assign LHS_net = RHS_expression;

```

右边表达式中的操作数无论何时发生变化都会引起其值的重新计算，并将重新计算后的值赋予左边表达式的线网变量。

若上节中的 4 选 1 数据选择器采用数据流方式描述的话，如下所示。

【例 7.4】 数据流方式描述的 4 选 1 MUX

```

module mux4_1c(out,in1,in2,in3,in4,cntrl1,cntrl2);

```

```

output out;
input in1,in2,in3,in4,cntrl1,cntrl2;
assign out=(in1 & ~cntrl1 & ~cntrl2) | (in2 & ~cntrl1 & cntrl2) |
           (in3 & cntrl1 & ~cntrl2) | (in4 & cntrl1 & cntrl2);
endmodule

```

若用条件运算符来描述的话, 则代码更简洁。

【例 7.5】用条件运算符描述的 4 选 1 MUX

```

module mux4_1d(out,in1,in2,in3,in4,cntrl1,cntrl2);
output out;
input in1,in2,in3,in4,cntrl1,cntrl2;
assign out=cntrl1?(cntrl2?in4:in3):(cntrl2?in2:in1);
endmodule

```

持续赋值语句是并发执行的, 也就是说, 各语句的执行顺序与其在描述中出现的顺序是无关的。

下面用一个 2 选 1 多路选择器的例子来对几种不同的描述方式进行一下比较。图 7.2 是 2 选 1 选择器的示意图。下面是对它的 3 种描述方式。

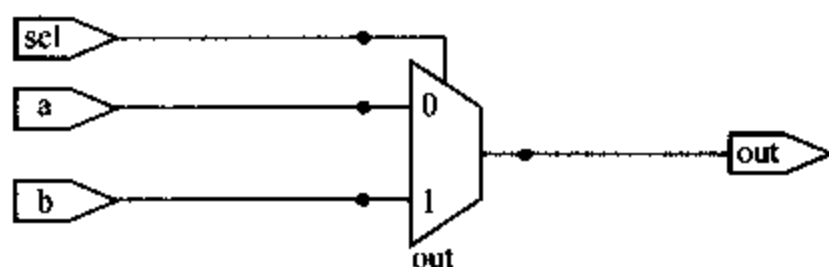


图 7.2 2 选 1 数据选择器示意图

调用门元件实现的 2 选 1 数据选择器, 其原理图如图 7.3 所示, 下面是其源代码。

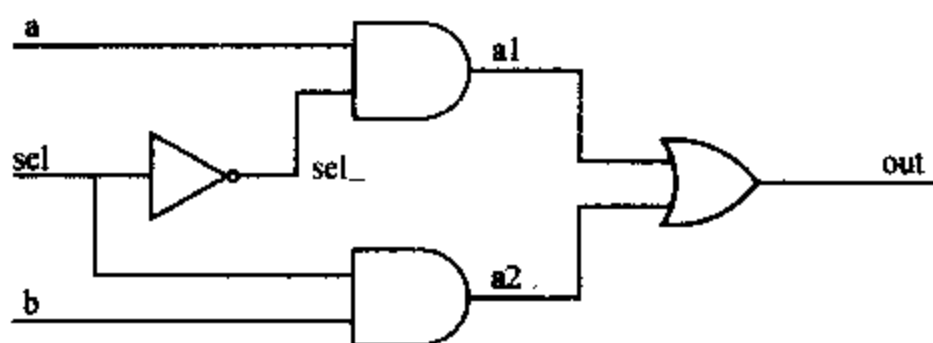


图 7.3 2 选 1 数据选择器门级原理图

【例 7.6】门级结构描述的 2 选 1 MUX

```

module MUX2_1a(out, a, b, sel);
output out;
input a, b, sel;
not (sel_, sel);
and (a1, a, sel_),

```

```

    (a2, b, sel);
or (out, a1, a2);
endmodule

```

【例 7.7】行为描述的 2 选 1 MUX

```

module mux2_1b(out, a, b, sel);
output out;
input a, b, sel;
reg out;
always @(a or b or sel)
begin
    if(sel)    out = b;
    else      out = a;
end
endmodule

```

【例 7.8】数据流描述的 2 选 1 MUX

```

module MUX2_1c(out, a, b, sel);
output out;
input a, b, sel;
    assign out = sel ? b : a;
endmodule

```

7.5 不同描述风格的例子

本节用几个简单的例子进一步说明同一设计的不同描述方法。

7.5.1 半加器

首先以 1 位半加器的描述为例对几种不同的描述风格加以比较。半加器的真值表见表 7.12。

表 7.12 1 位半加器的真值表

输 入		输 出	
a	b	sum	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

由此可得其门级原理图如图 7.4 所示。

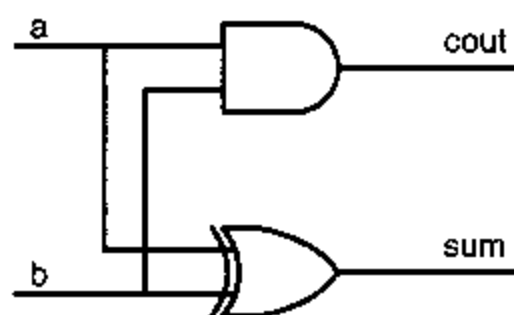


图 7.4 1 位半加器门级结构原理图

下面的结构描述通过调用门元件实现了上面的 1 位半加器。

【例 7.9】 调用门元件实现的 1 位半加器

```
module half_add1(a, b, sum, cout);
  input a, b;
  output sum, cout;
  and (cout, a, b);
  xor (sum, a, b);
endmodule
```

用数据流方式描述的 1 位半加器如下所示。

【例 7.10】 数据流方式描述的 1 位半加器

```
module half_add2 (a, b, sum, cout);
  input a, b;
  output sum, cout;
  assign sum=a^b;
  assign cout=a&b;
endmodule
```

还可以用行为方式描述，下面的代码用 case 语句将半加器的真值表直接表示出来。

【例 7.11】 采用行为描述的 1 位半加器

```
module half_add3 (a, b, sum, cout);
  input a, b;
  output sum, cout;
  reg sum, cout;
  always @ ( a or b )
  begin
    case ((a,b)) //真值表描述
      2'b00: begin sum=0; cout=0; end
      2'b01: begin sum=1; cout=0; end
      2'b10: begin sum=1; cout=0; end
      2'b11: begin sum=0; cout=1; end
    endcase
  end
end
```


endmodule

也可以用下面的行为描述方式设计半加器。

【例 7.12】 采用行为描述的 1 位半加器

```

module half_add4 (a, b, sum, cout);
input a, b;
output sum, cout;
reg sum, cout;
always @ ( a or b )
    begin
        sum= a^b;
        cout=a&b;
    end
endmodule

```

7.5.2 1 位全加器

下面以 1 位全加器为例说明用不同的方式设计 1 位全加器的方法。

1. 门级结构描述

1 位全加器的门级结构原理图如图 7.5 所示。

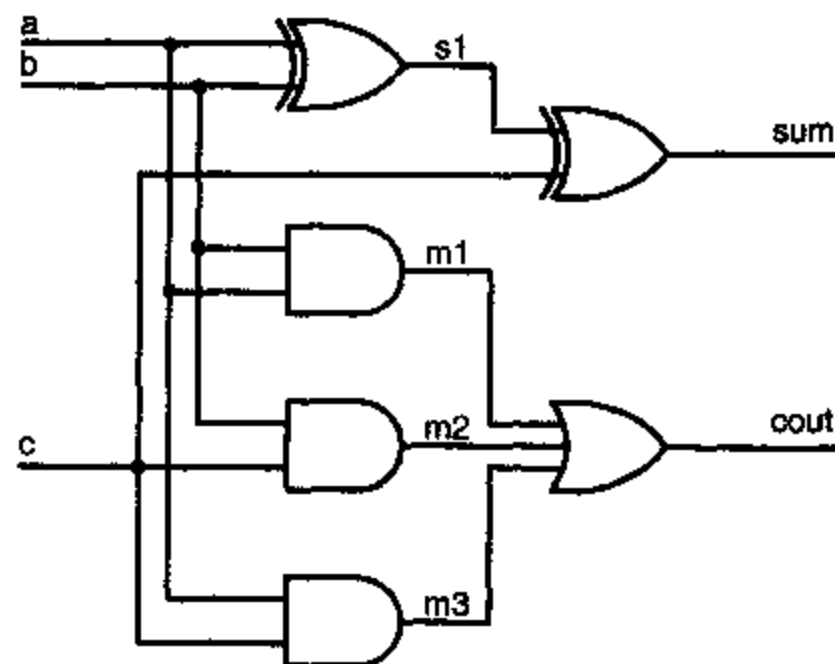


图 7.5 1 位全加器门级结构原理图

下面的结构描述通过调用门元件实现了上面的全加器电路。

【例 7.13】 调用门元件实现的 1 位全加器

```

module full_add1(a, b, cin, sum, cout);
input a, b, cin;
output sum, cout;
wire s1, m1, m2, m3;
and (m1, a, b),
    (m2, b, cin),

```

```

        (m3, a, cin);
    xor (s1, a, b),
        (sum, s1, cin);
    or  (cout, m1, m2, m3);
endmodule

```

在本例中，通过调用 xor、and 和 or 等基本门元件，实现了电路的门级描述。门元件的调用顺序是任意的。该 1 位全加器综合后的门级结构原理图如图 7.6 所示，可见与图 7.5 完全一致。

2. 数据流描述

用 assign 语句对 1 位全加器电路建模。

【例 7.14】数据流描述的 1 位全加器

```

module full_add2(a, b, cin, sum, cout);
input a, b, cin;
output sum, cout;
assign sum = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (cin & a);
endmodule

```

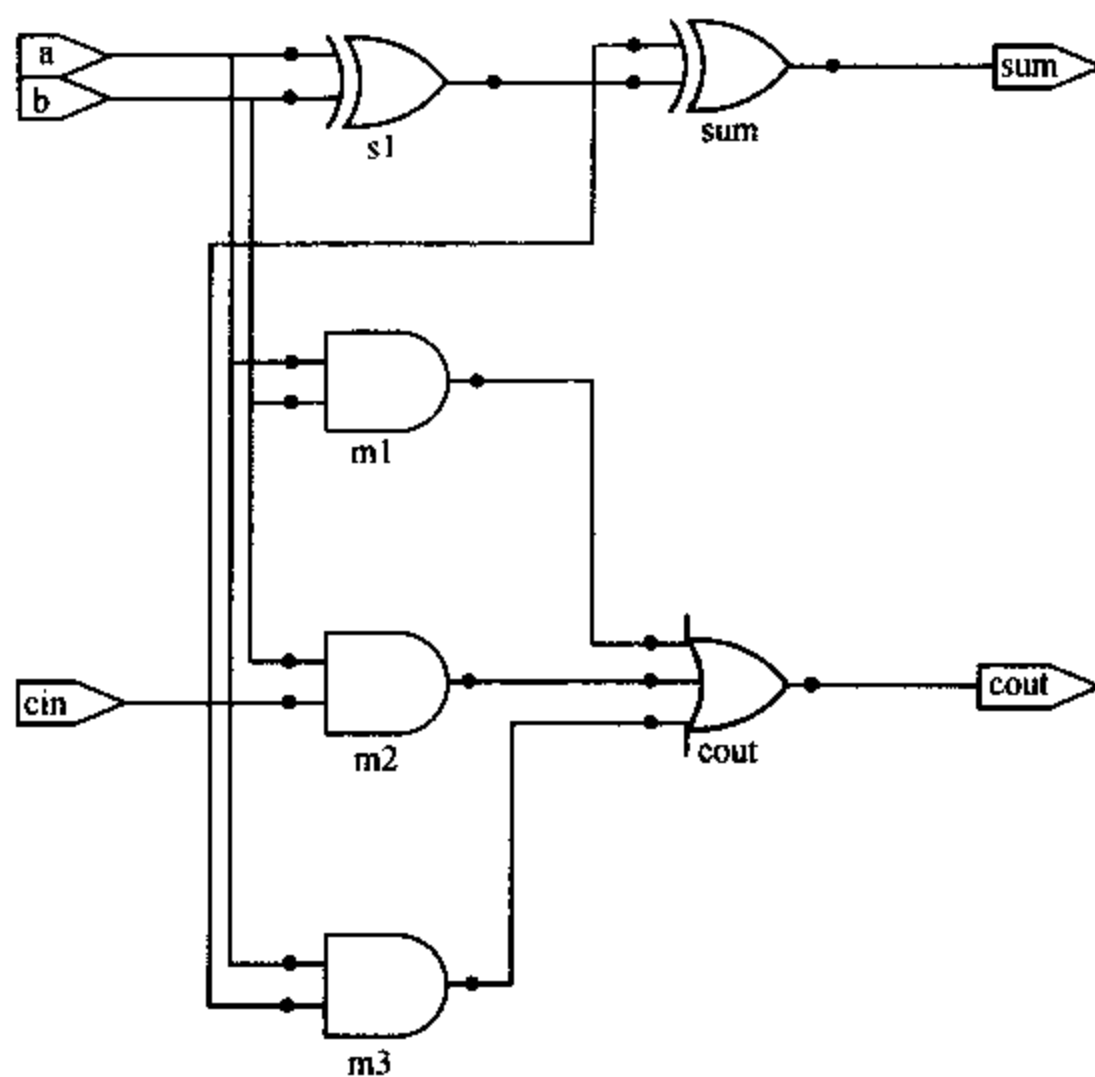


图 7.6 1 位全加器综合后的门级结构原理图

或者像例 7.15。

【例 7.15】1 位全加器

```

module full_add3(a, b, cin, sum, cout);

```



```

input a, b, cin;
output sum, cout;
assign {cout, sum} = a+b+cin;
endmodule

```

3. 行为描述

用 always 过程语句对 1 位全加器电路进行行为建模。

【例 7.16】行为描述的 1 位全加器

```

module full_add4(a, b, cin, sum, cout);
input a, b, cin;
output sum, cout;
reg sum, cout; //在 always 块中被赋值的变量应定义为 reg 型
reg m1, m2, m3;
always @ ( a or b or cin )
begin
    sum = (a ^ b) ^ cin;
    m1 = a & b;
    m2 = b & cin;
    m3 = a & cin;
    cout = (m1 | m2) | m3;
end
endmodule

```

由于 sum、cout、m1、m2 和 m3 在 always 语句中被赋值，所以必须定义为 reg 型。a、b 或 cin 被定义为敏感事件，只要 a、b 或 cin 中的任一个值发生变化，就会引起赋值过程执行一次。

4. 混合描述

在模块中，可以将结构描述和行为描述混合，也就是说，模块描述中可以调用门元件，调用其他模块，可包含持续赋值语句以及 always 语句和 initial 语句的混合，它们之间可以相互包含。来自 always 语句和 initial 语句的值能够驱动门或开关，而来自于门或持续赋值语句（只能驱动线网）的值也能够用于触发 always 语句和 initial 语句。

下面是用混合方式描述的 1 位全加器。

【例 7.17】混合描述的 1 位全加器

```

module full_add5(a, b, cin, sum, cout);
input a,b, cin;
output sum, cout;
reg cout, m1, m2, m3; //在 always 块中被赋值的变量应定义为 reg 型
wire s1;
xor x1(s1, a, b); //调用门元件
always @ ( a or b or cin ) //always 块语句
begin

```

```

    m1 = a & b;
    m2 = b & cin;
    m3 = a & cin;
    cout = (m1 | m2) | m3;
    end

    assign sum = s1 ^ cin;           //assign 持续赋值语句
endmodule

```

只要 a 或 b 的值发生变化, 门语句即被执行; 只要 a、b 或 cin 的值发生变化, 就执行 always 块内的语句; 而只要 s1 或 cin 产生变化, 就会执行持续赋值语句。

7.5.3 4 位全加器

我们同样分别用结构描述、行为描述、数据流描述方式来设计 4 位全加器。

结构描述基于前面的 1 位全加器设计, 用 4 个 1 位全加器按如图 7.7 所示进行级连即构成 4 位全加器, 其描述如下所示。

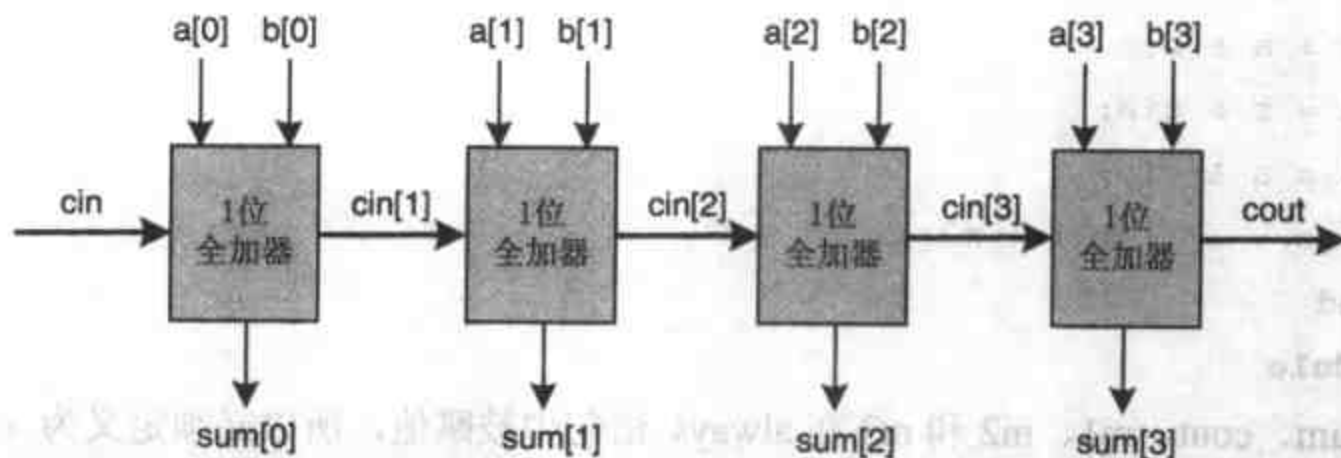


图 7.7 4 位全加器结构示意图

【例 7.18】结构描述的 4 位级连全加器

```

`include "full_add1.v"
module add4_1(sum, cout, a, b, cin);
    output [3:0] sum;
    output cout;
    input [3:0] a, b;
    input cin;

    full_add1 f0 (a[0], b[0], cin, sum[0], cin1);
    full_add1 f1 (a[1], b[1], cin1, sum[1], cin2);
    full_add1 f2 (a[2], b[2], cin2, sum[2], cin3);
    full_add1 f3 (a[3], b[3], cin3, sum[3], cout);
endmodule

```

//级连描述

【例 7.19】数据流描述的 4 位全加器

```

module add4_2(cout, sum, a, b, cin);

```

```

output[3:0] sum;
output cout;
input[3:0] a,b;
input cin;
assign {cout,sum}=a+b+cin;
endmodule

```

【例 7.20】行为描述的 4 位全加器

```

module add4_3(cout,sum,a,b,cin);
output[3:0] sum;
output cout;
input[3:0] a,b;
input cin;
reg[3:0] sum;
reg cout;
always @ ( a or b or cin )
begin
    {cout,sum}=a+b+cin;
end
endmodule

```

结构描述的 4 位全加器的 RTL 综合原理图如图 7.8 所示。

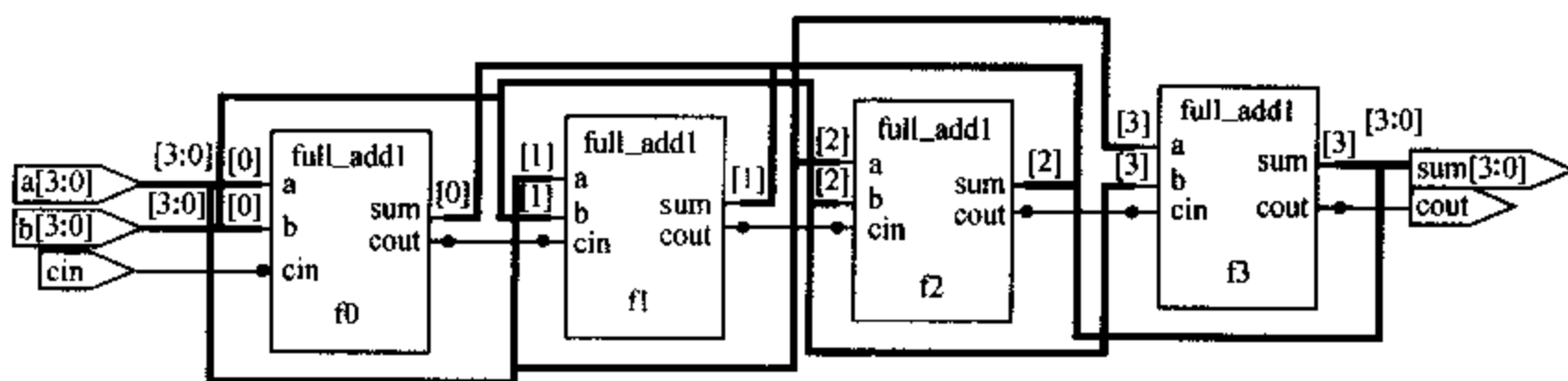


图 7.8 结构描述的 4 位全加器的 RTL 综合原理图

行为描述和数据流描述的 4 位全加器的 RTL 综合原理图如图 7.9 所示。

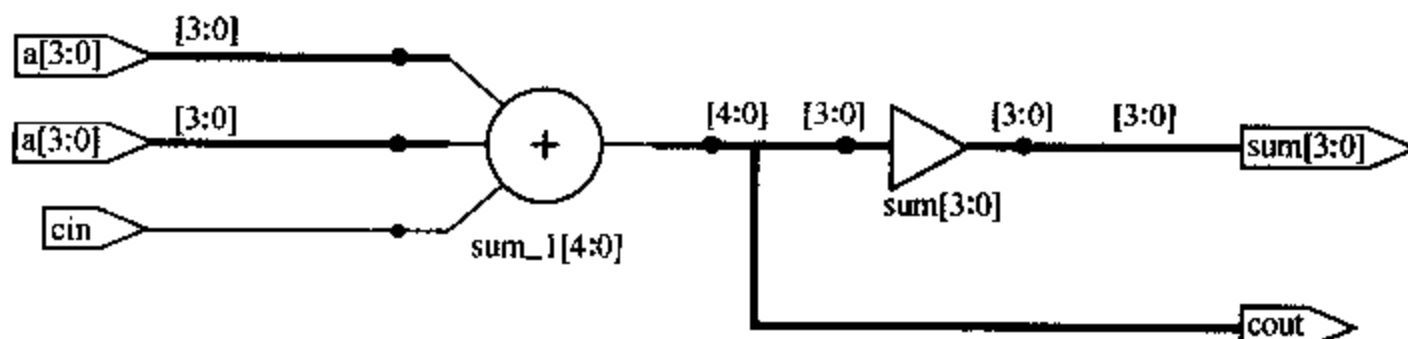


图 7.9 行为描述和数据流描述的 4 位全加器的 RTL 综合原理图

思考与练习

1. Verilog HDL 支持哪三种基本描述方式？
2. 在数据流描述方式中使用什么语句描述设计？
3. 分别用结构描述、数据流描述和行为描述方式设计一个基本的 D 触发器，并进行综合。
4. 分别用结构描述、数据流描述和行为描述方式设计一个 RS 触发器，并进行综合。

第 8 章 仿 真

本章内容简介

- ◆ 仿真 (Simulation)
- ◆ 系统任务和系统函数
- ◆ 用户自定义元件 (UDP)
- ◆ 延时的表示
- ◆ 测试平台 (Test Bench)
- ◆ 仿真程序的编写
- ◆ 组合电路的仿真
- ◆ 时序电路的仿真

Verilog HDL 不仅提供了设计描述的能力, 而且提供对激励、控制、存储响应和设计验证的建模能力。Verilog HDL 最先只是一种用于电路模拟和仿真的语言, 后来, 由于 Verilog 综合器的出现, 才使得它具有了硬件设计和综合的能力。Verilog HDL 能够提供完备的仿真验证功能。

本章介绍与 Verilog HDL 仿真相关的内容, 包括 Verilog HDL 的系统任务和系统函数、用户自定义元件 (UDP)、延时的表示等, 并通过若干实例具体说明 Verilog 仿真程序的编写, 以及如何对组合电路和时序电路进行仿真等。

8.1 概 述

仿真 (Simulation), 或者称为模拟, 是对所设计电路或系统的一种检测和验证。

仿真包括功能仿真和时序仿真。在设计输入阶段进行的仿真, 不考虑信号时延等因素, 称为功能仿真, 又叫前仿真; 时序仿真又称后仿真, 它是在选择了具体器件并完成了布局布线后进行的包含定时关系的仿真。由于不同器件的内部时延不一样, 不同的布局、布线方案也给延时造成了很大的影响, 因此在设计实现后, 对网络和逻辑块进行时延仿真, 分析定时关系, 估计设计性能是非常必要的。

要进行电路仿真, 必须有仿真器的支持。按对设计语言的不同处理方式, 可将仿真器分为两类: 编译型仿真器和解释型仿真器。编译型仿真器的仿真速度快, 但需要预处理, 因此不能即时修改; 解释型仿真器的仿真速度相对慢一些, 但可以随时修改仿真环境和仿真条件。

按处理的 HDL 语言类型, 仿真器可分为: Verilog HDL 仿真器、VHDL 仿真器和混合仿真器。混合仿真器能够处理 Verilog HDL 和 VHDL 混合编程的仿真程序。

常用的 Verilog HDL 仿真器包括: ModelSim、Verilog-XL、NC-Verilog 和 VCS 等。

ModelSim 能够提供很好的 Verilog 和 VHDL 混合仿真; NC-Verilog 和 VCS 是基于编译技术的仿真软件, 能够胜任行为级、RTL 级和门级等各种层次的仿真, 速度快; 而 Verilog-XL 是基于解释的仿真工具, 速度相对慢一些。

在一些复杂的设计中, 仿真验证比设计本身还要耗时费力, 所以仿真器的仿真速度、仿真的准确性、易用性等成为衡量仿真器性能的重要指标。

8.2 系统任务与系统函数

Verilog HDL 的系统任务和系统函数主要用于仿真。本节介绍常用的系统任务和系统函数。这些系统任务和系统函数提供了各种功能, 比如实时显示当前仿真时间 (\$time)、显示信号的值 (\$display、\$monitor)、暂停仿真 (\$stop)、结束仿真 (\$finish) 等等。

系统任务和系统函数有以下一些特点。

- ◆ 系统任务和系统函数一般以符号 “\$” 开头, 如 \$monitor、\$readmemh 等。
- ◆ 使用系统任务和系统函数, 可以显示模拟结果, 对文件进行操作, 以及控制模拟的执行过程等。
- ◆ 使用不同的 Verilog HDL 仿真工具 (如 VCS、Verilog-XL、ModelSim 等) 进行仿真时, 这些系统任务和系统函数在使用方法上可能存在差异, 应根据使用手册来使用。
- ◆ 一般在 initial 或 always 过程块中调用系统任务和系统函数。
- ◆ 用户可以通过编程语言接口 (PLI) 将自己定义的系统任务和系统函数加到语言中, 以方便仿真和调试。

由于系统任务和系统函数比较多, 有的开发工具还有自己定义的系统任务和函数, 因此本节中只介绍常用的系统任务和系统函数, 这些任务和函数大多数仿真工具都支持, 且基本能够满足一般的仿真测试的需要。

8.2.1 \$display 与 \$write

\$display 和 \$write 是两个系统任务, 两者的功能相同, 都用于显示模拟结果, 其区别是 \$display 在输出结束后能自动换行, 而 \$write 不能。

\$display 和 \$write 的使用格式为:

\$display (“格式控制符”, 输出变量名列表);

\$write (“格式控制符”, 输出变量名列表);

例如:

```
$display ($time,,, "a=%h b=%h c=%h", a, b, c );
```

上面的语句定义了信号显示的格式, 即以十六进制格式显示信号 a、b、c 的值, 两个相邻的逗号 “,” 表示加入一个空格。

显示格式的控制符见表 8.1。

表 8.1 格式控制符

格式控制符	说 明
%h 或%H	以十六进制形式显示
%d 或%D	以十进制形式显示
%o 或%D	以八进制形式显示
%b 或%B	以二进制形式显示
%c 或%C	以 ASCII 字符形式显示
%s 或%S	以字符串形式输出
%v 或%V	输出连线型数据的驱动强度
%e 或%E	以指数形式显示实数
%f 或%F	以浮点方式显示实数

也可以用\$display 显示字符串, 比如:

```
$display ( "it's a example for display\n" );
```

上面的语句将直接输出引号中的字符串, 而“\n”是转义字符, 表示换行。表 8.2 中列举了常用的转义字符, 这些转义字符也用于输出格式的定义。

表 8.2 转义字符

转义字符	说 明
\n	换行
\t	TAB 键
\\	符号\
\"	符号"
\ddd	八进制的 ASCII 字符

8.2.2 \$monitor 与\$strobe

\$monitor、\$strobe 与\$display、\$write 一样, 也属于输出控制类的系统任务, \$monitor 与\$strobe 都提供了监控和输出参数列表中字符或变量的值的功能, 其使用格式为:

```
$monitor ( "格式控制符", 输出变量名列表 );
```

```
$strobe ( "格式控制符", 输出变量名列表 );
```

这里的格式控制符、输出变量名列表等与\$display、\$write 中定义的完全相同。

例如: \$monitor(\$time, "a=%b b=%h", a, b);

每次 a 或 b 信号的值发生变化都会激活上面的语句, 并显示当前仿真时间、二进制格式的 a 信号和十六进制格式的 b 信号。

可将\$monitor 想象为一个持续监控器, 一旦被调用, 就相当于启动了一个实时监控器, 如果输出变量列表中的任何变量发生了变化, 则系统将按照\$monitor 语句中所规定的格式将

结果输出一次。而\$strobe 相当于是选通监控器，只有在模拟时间发生改变时，并且所有的事件都已处理完毕后，\$strobe 才将结果输出。\$strobe 更多地用来显示用非阻塞方式赋值的变量的值。

例如：

```
$monitor($time,,, "A=%d B=%d C=%d", A, B, C);
//只要 A、B、C 三个变量的值发生任何变化，都会将 A、B、C 的值输出一次
```

8.2.3 \$time 与\$realtime

\$time、\$realtime 是属于显示仿真时间标度的系统函数。这两个函数被调用时，都返回当前时刻距离仿真开始时刻的时间量值，所不同的是，\$time 函数以 64 位整数值的形式返回模拟时间，\$realtime 函数则以实数型数据返回模拟时间。

通过下面的例子可清楚地看出\$time 与\$realtime 的区别。

【例 8.1】\$time 与\$realtime 的区别

```
'timescale 10ns/1ns
module time_dif;
reg ts;
parameter delay=2.6;
initial
begin
#delay ts=1;
#delay ts=0;
#delay ts=1;
#delay ts=0;
end
initial $monitor($time,,, "ts=%b", ts);    //使用函数$time
endmodule
```

上面的例子用 ModelSim 仿真的话，其输出为：

```
0    ts=x
3    ts=1
5    ts=0
8    ts=1
10   ts=0
```

每行开始处时间的显示都是整数形式的。

如果将上面程序中的\$time 改为\$realtime，则仿真输出变为：

```
0      ts=x
2.6    ts=1
5.2    ts=0
7.8    ts=1
10.4   ts=0
```


每行中时间的显示都变为实数形式。

从上面的例子不难看出两者的区别。

8.2.4 \$finish 与 \$stop

系统任务\$finish 与\$stop 用于对仿真过程进行控制，分别表示结束仿真和中断仿真。

\$finish 与\$stop 的使用格式如下：

```
$stop;
$stop(n);
$finish;
$finish(n);
```

n 是\$finish 和\$stop 的参数，n 可以是 0、1、2 等值，分别表示如下含义。

0: 不输出任何信息；

1: 给出仿真时间和位置；

2: 给出仿真时间和位置，还有其他一些运行统计数据。

如果不带参数，则默认的参数值是 1。

当仿真程序执行到\$stop 语句时，将暂时停止仿真，此时设计者可以输入命令，对仿真器进行交互控制。而当仿真程序执行到\$finish 语句时，则终止仿真，结束整个的仿真过程，返回主操作系统。

以下是使用\$finish 与\$stop 的例子。

```
如:      if(...)
          $stop ;                      //在一定的条件下,中断仿真

再如:    # STEP ...
          # STEP $finish ;             //在某一时刻,结束仿真
          .....
```

8.2.5 \$readmemh 与 \$readmemb

\$readmemh 与\$readmemb 属于文件读写控制的系统任务，其作用都是从外部文件中读取数据并放入存储器中。两者的区别在于两者读取数据的格式不同，\$readmemh 为读取十六进制数据，而\$readmemb 为读取二进制数据。

\$readmemh 与\$readmemb 的使用格式为：

```
$readmemh ( "数据文件名" , 存储器名, 起始地址, 结束地址 );
$readmemb ( "数据文件名" , 存储器名, 起始地址, 结束地址 );
```

其中，起始地址和结束地址均可以缺省，如果缺省起始地址，表示从存储器的首地址开始存储；如果缺省结束地址，表示一直存储到存储器的结束地址。

下面是使用\$readmemh 的例子。

```
reg[7:0] my_mem[0:255] ;           //首先定义一个 256 个地址的存储器 my_mem
initial begin
$readmemh ( "mem.hex" , my_mem ) ;
end
```

```
//将 mem.hex 中的数据装载到存储器 my_mem 中, 起始地址从 0 开始
initial begin
  $readmemh ( "mem.hex" , my_mem ,80) ;
end
```

```
//将 mem.hex 中的数据装载到存储器 my_mem 中, 起始地址从 80 开始
```

在程序的仿真中, 如果测试向量很多的话, 可以先将测试向量写入一个文件, 然后在仿真程序中使用 readmemb 或 readmemh 将测试向量读入即可。

8.2.6 \$random

\$random 是产生随机数的系统函数, 每次调用该函数, 将返回一个 32 位的随机数, 该随机数是一个带符号的整数。

例如, 下面是一个产生随机数的简单程序, 用于说明 \$random 的功能。

【例 8.2】\$random 函数的使用

```
'timescale 10ns/1ns
module random_tp;
  integer data;
  integer i;
  parameter delay=10;

  initial $monitor($time,,, "data=%b",data);
  initial begin
    for (i=0; i<=100; i=i+1)
      #delay data=$random;          //每次产生一个随机数
    end
endmodule
```

用 ModelSim 仿真的话, 其输出大致如下所示, 只不过每次显示的数据都是随机的。

```
0 data=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
10 data=1110100110000011110100000010000
20 data=0100011000111100100111111111111
30 data=11011111100110100101101000011001
40 data=00100100001011000101101110101111
```

8.2.7 文件输出

与 c 语言类似, Verilog HDL 也提供了很多文件输出类的系统任务, 可以将结果输出到文件中。这类任务有: \$fdisplay、\$fwrite、\$fmonitor、\$fstrobe、\$fopen 和 \$fclose 等。

\$fopen 用于打开某个文件并准备写操作, \$fclose 用于关闭文件, 而 \$fdisplay、\$fwrite、\$fmonitor 等系统任务则用于把文本写入到文件中。

```

如: fd=$fopen("filename");
    $fclose(fd);
/*fd 必须是 32 位的变量, 之前应该定义成 integer 或 reg 型, 如 reg [31:0] fd; 或 integer
fd; 调用 $fopen, 它返回一个 32 位的无符号整数或 0 值, 0 值表示文件不能打开。*/

```

8.3 用户自定义元件 (UDP)

利用 UDP (User Defined Primitives), 用户可以自己定义基本逻辑元件的功能, 可以像调用基本门元件一样来调用这些自己定义的元件。UDP 元件不能用于可综合的设计描述中, 而只能用于仿真程序中。

UDP 模块与一般的模块类似, 其关键词为 **primitive** 和 **endprimitive**。与一般的模块相比, UDP 模块具有下面一些特点:

- ◆ UDP 的输出端口只能有一个, 且必须位于端口列表的第一项。只有输出端口能被定义为 **reg** 类型。

- ◆ UDP 的输入端口可有多, 一般时序电路 UDP 的输入端口可多至 9 个, 组合电路 UDP 的输入端口可多至 10 个。

- ◆ 所有的端口变量必须是 1 位标量。

- ◆ 在 **table** 表项中, 只能出现 0、1、x 三种状态, 不能出现 z 状态。

定义 UDP 的语法如下。

primitive 元件名 (输出端口, 输入端口 1, 输入端口 2, ...)

output 输出端口名;

input 输入端口 1, 输入端口 2, ...;

reg 输出端口名;

initial begin

 输出端口或内部寄存器赋初值 (0, 1 或 x);

end

table

 //输入 1 输入 2 : 输出

 真值列表

endtable

endprimitive

UDP 定义的差别主要体现在 **table** 表项的描述上, 下面分别介绍组合电路和时序电路 UDP 元件的定义。

8.3.1 组合电路 UDP 元件

下面首先以一个 1 位全加器进位输出 UDP 基元为例来介绍组合电路 UDP 元件的描述与定义。

【例 8.3】1 位全加器进位输出 UDP 元件

```

primitive carry_udp(cout, cin, a, b);
input cin, a, b;
output cout;

  table
    // cin a b : cout          //真值表
    0  0  0 :  0;
    0  1  0 :  0;
    0  0  1 :  0;
    0  1  1 :  1;
    1  0  0 :  0;
    1  0  1 :  1;
    1  1  0 :  1;
    1  1  1 :  1;

  endtable
endprimitive

```

在上面的 UDP 描述中, 没有考虑输入为 x 的情况, 如果某一个输入端 (cin, a, b) 的值为 x, 则由于 table 表中没有对应的描述项, 所以输出也将是不定态 x。考虑了输入为 x 的情况的 1 位全加器进位输出 UDP 元件如下所示。

【例 8.4】包含 x 态输入的 1 位全加器进位输出 UDP 元件

```

primitive carry_udpx1(cout, cin, a, b);
input cin, a, b;
output cout;

  table
    // cin a b : cout          //真值表
    0  0  0 :  0;
    0  1  0 :  0;
    0  0  1 :  0;
    0  1  1 :  1;
    1  0  0 :  0;
    1  0  1 :  1;
    1  1  0 :  1;
    1  1  1 :  1;
    0  0  x :  0;          //只要有二个输入为 0, 则进位输出肯定为 0
    0  x  0 :  0;
    x  0  0 :  0;
    1  1  x :  1;          //只要有二个输入为 1, 则进位输出肯定为 1
    1  x  1 :  1;
    x  1  1 :  1;

  endtable

```

endprimitive

在上面的程序中可以发现：只要有二个输入为 0，则不管第 3 个输入为何值，进位输出肯定为 0；同时，若有二个输入为 1，则不管第 3 个输入为何值，进位输出肯定为 1。在这种情况下，Verilog 提供了符号“？”进行简缩，符号“？”可用来表示 0、1、x 等几种取值，也就是说，该位的值不管是 0、1、x 都不会影响到输出结果的取值时，即可用该符号来表示该位，这样就使程序的表达更清晰简洁。例 8.4 若采用简缩符“？”来表述的话，如下所示。

【例 8.5】用简缩符“？”表述的 1 位全加器进位输出 UDP 元件

```
primitive carry_udpx2(cout, cin, a, b);
input cin, a, b ;
output cout ;

table
    // cin  a   b   :   cout           //真值表
    ?   0   0   :   0;               //只要有二个输入为 0，则进位输出肯定为 0
    0   ?   0   :   0;
    0   0   ?   :   0;
    ?   1   1   :   1;               //只要有二个输入为 1，则进位输出肯定为 1
    1   ?   1   :   1;
    1   1   ?   :   1;

endtable
endprimitive
```

可以看出，简缩符“？”使表达式的书写更简练，增强了程序的可读性。下面是一个采用简缩符“？”来表述的 3 选 1 多路选择器 UDP 元件的例子。

【例 8.6】3 选 1 多路选择器 UDP 元件

```
primitive mux31 (Y, in0, in1, in2, s2, s1);
input in0, in1, in2, s2, s1;
output Y;

table
    //in0 in1 in2  s2  s1 :  Y
    0   ?   ?   0   0 :  0;           //当 s2s1=00 时, Y=in0
    1   ?   ?   0   0 :  1;
    ?   0   ?   0   1 :  0;           //当 s2s1=01 时, Y=in1
    ?   1   ?   0   1 :  1;
    ?   ?   0   1   ? :  0;           //当 s2s1=1? 时, Y=in2
    ?   ?   1   1   ? :  1;
    0   0   ?   0   ? :  0;
    1   1   ?   0   ? :  1;
    0   ?   0   ?   0 :  0;
    1   ?   1   ?   0 :  1;
    ?   0   0   ?   1 :  0;
```

```

        ? 1 1 ? 1 : 1;
    endtable
endprimitive

```

8.3.2 时序逻辑 UDP 元件

UDP 元件也可以用来描述电平敏感或边沿敏感的时序逻辑。时序逻辑元件的输出除了与当前的输入有关外，还与它当前所处的状态有关。因此，对应的 UDP 元件描述中应增加对内部状态的考虑。

例 8.7 定义了一个电平敏感的 1 位数据锁存器 UDP 元件。

【例 8.7】电平敏感的 1 位数据锁存器 UDP 元件

```

primitive latch (Q, clk, reset, D);
input clk, reset, D;
output Q;
reg Q;
initial Q = 1'b1;    //初始化
    table
    // clk reset D : state : Q
        ? 1 ? : ? : 0;    //reset=1, 则不管其他端口为什么值, 输出都为 0
        0 0 0 : ? : 0;    //clk=0, 锁存器把 D 端的输入值输出
        0 0 1 : ? : 1;
        1 0 ? : ? : -;    //clk=1, 锁存器的输出保持原值, 用符号“-”表示
    endtable
endprimitive

```

数据锁存器 UDP 与前面的组合电路元件相比，多了一列对元件内部状态 (state) 的描述，内部状态两边用冒号与输入、输出隔开。同时，增加了新的符号“-”，表示保持原值的意思。

例 8.8 是上升沿触发的 D 触发器的 UDP 元件的例子。

【例 8.8】上升沿触发的 D 触发器 UDP 元件

```

primitive DFF(Q, D, clk);
output Q;
input D, clk;
reg Q;
    table
    // clk D : state : Q
        (01) 0 : ? : 0;    //上升沿到来, 输出 Q=D
        (01) 1 : ? : 1;
        (0x) 1 : 1 : 1;
        (0x) 0 : 0 : 0;
        (?0) ? : ? : -;    //没有上升沿到来, 输出 Q 保持原值
        ? (??) : ? : -;    //时钟不变, 输出也不变
    endtable
endprimitive

```


endtable

endprimitive

在上面的例子中，括号内的两个数字表示状态间的转变，也就是不同的边沿，(01)表示上升沿；(10)表示下降沿；(?0)表示从任何状态（0、1、x）到 0 的跳变，即排除了上升沿的可能性。

上例中 table 列表第 3、4 行的意思是：当时钟从 0 状态变化到不确定状态（x）时，如输入数据与当前状态（state）一致，则输出也是定态。

table 列表中最后一行的意思是：如果时钟处于某一确定状态（这里的“?”表示是 0 或者是 1，不包括 x），则不管输入数据有什么变化（(??)表示任何可能的变化），D 触发器的输出都将保持原值不变（用符号“-”表示）。

8.3.3 UDP 元件缩记符

为便于描述、增强可读性，Verilog HDL 在 UDP 元件的定义中引入了很多缩记符号，前面已经介绍了一些，表 8.3 进一步对这些缩记符做了总结。

例 8.9 是采用了上述缩记符表示的一个带有异步置 1 和异步清零的上升沿触发的 D 触发器的 UDP 元件的例子。

表 8.3 UDP 中的缩记符

缩 记 符	含 义	说 明
x	不定态	
?	0、1 或 x	只能表示输入
b	0 或 1	只能表示输入
-	保持不变	只用于时序元件的输出
(vy)	代表(01)、(10)、(0x)、(1x)、(x1)、(x0)、(?1)等	从逻辑 v 到逻辑 y 的转变
*	同(??)	表示输入端有任何变化
R 或 r	同(01)	表示上升沿
F 或 f	同(10)	表示下降沿
P 或 p	(01)、(0x)或(x1)	包含 x 态的上升沿跳变
N 或 n	(10)、(1x)或(x0)	包含 x 态的下降沿跳变

【例 8.9】带异步置 1 和异步清零的上升沿触发的 D 触发器 UDP 元件

```

primitive DFF_UDP (Q, D, clk, clr, set);
output Q;
input D, clk, clr, set;
reg Q;

  table
    //  clk  D   clr  set  : state  : Q
        (01) 1   0   0   :   ?    : 0;
        (01) 1   0   x   :   ?    : 0;
  
```

```

        ?    ?    0    x    :    0    :    0;
    (01)  0    0    0    :    ?    :    1;
    (01)  0    x    0    :    ?    :    1;
        ?    ?    x    0    :    1    :    1;
    (x1)  1    0    0    :    0    :    0;
    (x1)  0    0    0    :    1    :    1;
    (0x)  1    0    0    :    0    :    0;
    (0x)  0    0    0    :    1    :    1;

        ?    ?    1    ?    :    ?    :    1;           //异步复位
        ?    ?    0    1    :    ?    :    0;           //异步置 1
        n    ?    0    0    :    ?    :    -;
        ?    *    ?    ?    :    ?    :    -;
        ?    ?    (?0)  ?    :    ?    :    -;
        ?    ?    ?    (?0) :    ?    :    -;
        ?    ?    ?    ?    :    ?    :    x;

    endtable
endprimitive

```

8.4 延时模型的表示

在仿真中,还涉及到延时表示的问题。延时包括门延时、assign 赋值延时和连线延时等。门延时为从门的输入端发生变化到输出发生变化的延迟时间; assign 赋值延时指等号右端某个值发生变化到等号左端发生相应变化的延迟时间;连线延时则体现了信号在连线上的传输延时。如果没有定义时延值,缺省时延为 0。

本节首先介绍模拟时间定标语句 timescale 的使用方法。

8.4.1 时间标尺定义 timescale

timescale 语句用于定义模块的时间单位和时间精度,其使用格式如下:

```
'timescale <time_unit> /<time_precision>
```

```
'timescale <时间单位>/<时间精度>
```

其中用来表示时间度量的符号有: s、ms、μs、ns、ps 和 fs,分别表示秒、 10^{-3} 秒、 10^{-6} 秒、 10^{-9} 秒、 10^{-12} 秒和 10^{-15} 秒。

例如:

```
'timescale 1ns/100ps
```

上面的语句表示时延单位为 1ns,时延精度为 100ps。timescale 编译器指令在模块说明外部出现,并且影响后面所有的时延值。例如:

【例 8.10】timescale 使用举例 1

```
'timescale 1ns/ 100ps
```



```

module Andgate (out, A, B);
output out;
input A, B;
and # (4.34, 5.86) A1 (out, A, B);
    // # (4.34, 5.86)规定了上升及下降时延值
endmodule

```

在上面的例子中,`timescale 指令定义时延以 ns 为单位,并且时延精度为 100ps(1/10ns)。因此,时延值 4.34 对应 4.3ns,时延 5.86 对应 5.9ns。如果将`timescale 指令定义为:

```
`timescale 10ns/1ns
```

那么 4.34 对应 43ns, 5.86 对应 59ns。

再比如:

【例 8.11】`timescale 使用举例 2

```

`timescale 10ns/1ns
.....
reg sel;
initial
begin
    #10 sel=0;        //在 100ns(10ns×10)时, sel 被赋值为 0
    #10 sel=1;        //在 200ns(10ns×10+10ns×10)时, sel 被赋值为 1
end
.....

```

在例 8.11 中,`timescale 语句定义了本模块的时间单位为 10ns,时间精度为 1ns。以 10ns 为计量单位,在不同的时刻,寄存器型变量 sel 被赋予了不同的值。

8.4.2 延时的表示方法

延时的表示方法有下面几种:

```

# delaytime
# (d1, d2)
# (d1,d2,d3)

```

delaytime 表示延迟时间为 delaytime, d1 表示上升延迟, d2 表示下降延迟, d3 则表示转换到高阻态 z 的延迟,这些延迟的具体时间由时间定义语句`timescale 确定。

时延定义了右边表达式操作数变化与左边表达式产生变化之间的延迟时间。如果没有定义时延值,缺省时延为 0。

例如:

```

not #4 gate1(out,in);           //延迟时间为 4 的非门
and #(5,7) gate2(out,a,b);      //与门的上升延迟为 5,下降延迟为 7
or #5 gate3(out,a,b);           //或门的上升延迟和下降延迟都为 5
bufif0 #(3,4,6) gate4(out,in,enable)

```

```
// buff0 门的上升延迟为 3, 下降延迟为 4, 高阻延迟为 6
```

8.4.3 延时说明块 (specify 块)

Verilog HDL 可对模块中某一指定的路径进行延迟定义, 这一路径连接模块的输入端口 (或 inout 端口) 与输出端口 (或 inout 端口), 利用延迟定义块在一个独立的块结构中定义模块的延时。在延迟定义块中, 要描述模块中的不同路径并给这些路径赋值。

延迟定义块的内容应放在关键字 **specify** 与 **endspecify** 之间, 并且必须放在一个模块中, 还可以使用 **specparam** 关键字定义参数。举例说明如下:

假如信号模型如图 8.1 所示。

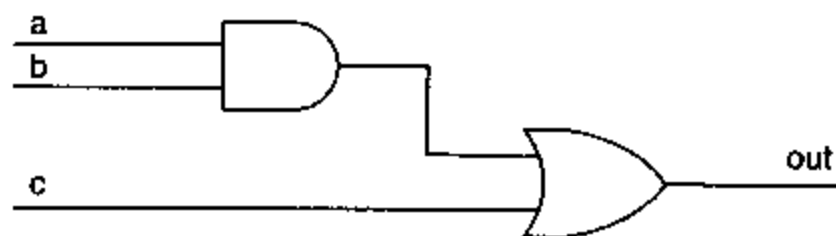


图 8.1 信号模型示意图

进行路径延迟定义如下:

【例 8.12】延迟定义块举例

```
module delay(out, a, b, c);
  output out;
  input a, b, c;
  and a1(n1, a, b);
  or o1(out, c, n1);

  specify
    (a=>out) =2;
    (b=>out) =3;
    (c=>out) =1;
  endspecify
endmodule
```

在上面的模块中, 定义从 a 到 out 的延迟为 2, 从 b 到 out 的延迟为 3, 而从 c 到 out 延迟时间为 1。

8.5 测试平台 (Test Bench)

本节介绍测试平台的概念以及测试程序的编写。

8.5.1 Test Bench

测试平台 (Test Bench 或 Test Fixture) 或者称为测试基准, 它为测试或仿真一个 Verilog 程序搭建了一个平台, 在这个平台上我们给被测试的模块施加激励信号, 通过观察被测试模

块的输出响应, 从而判断其逻辑功能和时序关系的正确与否。

如图 8.2 所示是测试平台 (Test Bench) 的示意图。

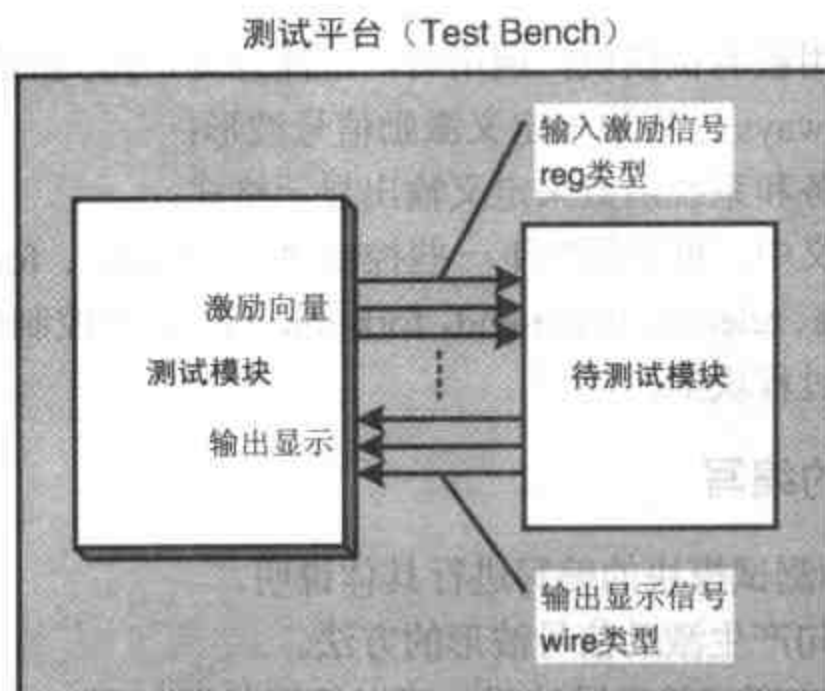


图 8.2 测试平台 (Test Bench) 示意图

从图中可以看出, 测试模块向待测试模块施加激励信息, 激励信号必须定义成 reg 类型, 以保持信号值。待测试模块在激励信息的作用下产生输出, 输出信号必须定义为 wire 类型。测试模块中将待测试模块在激励向量作用下产生的输出信息以规定的格式用文本或图形的方式显示出来, 供用户检验。

测试程序的一般结构如图 8.3 所示。

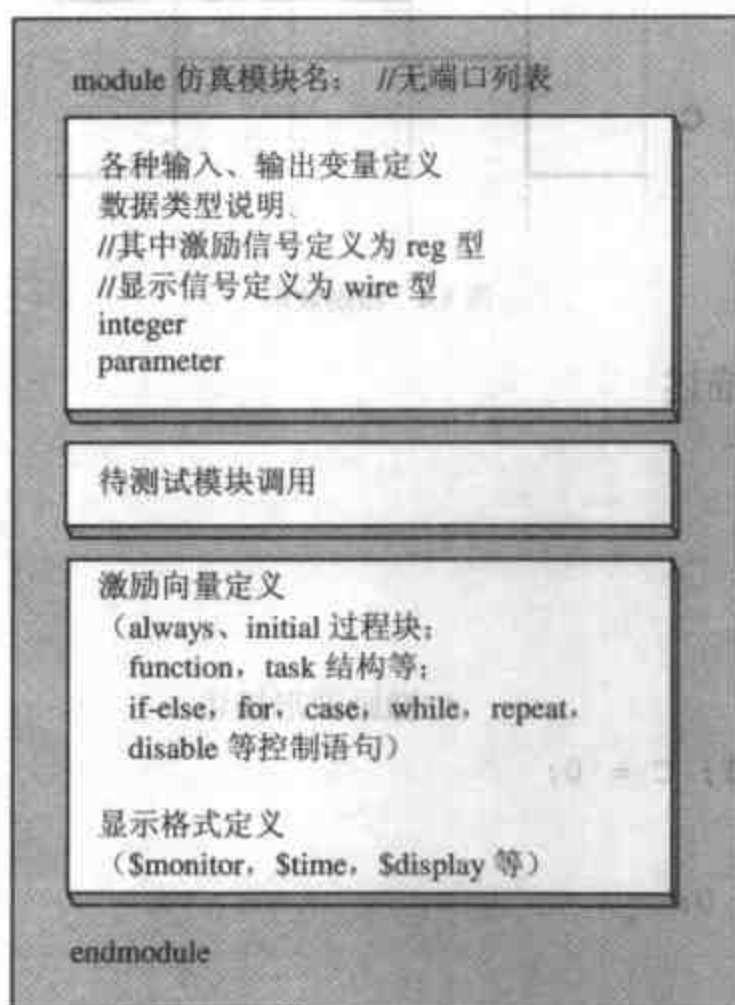


图 8.3 测试程序的一般结构

测试程序与一般的 Verilog 模块没有根本的区别, 其特点主要表现为:

- ◆ 测试模块只有模块名字，没有端口列表；
- ◆ 输入信号（激励信号）必须定义为 reg 型，以保持信号值；输出信号（显示信号）必须定义为 wire 型；
- ◆ 在测试模块中调用被测试模块，调用时，应注意端口排列的顺序与模块定义时一致；
- ◆ 一般用 initial、always 过程块来定义激励信号波形；
- ◆ 一般使用系统任务和系统函数来定义输出显示格式；
- ◆ 在激励信号的定义中，可使用如下一些控制语句：if-else、for、forever、case、while、repeat、wait、disable、force、release、begin-end、fork-join 等，这些控制语句一般只用在 always、initial、function、task 等过程块中。

8.5.2 测试程序的编写

这里通过一些实例对测试模块的编写进行具体说明。

首先介绍用 initial 语句产生激励信号波形的方法。

例如：要产生如图 8.4 所示的激励波形，可以编写代码如下。

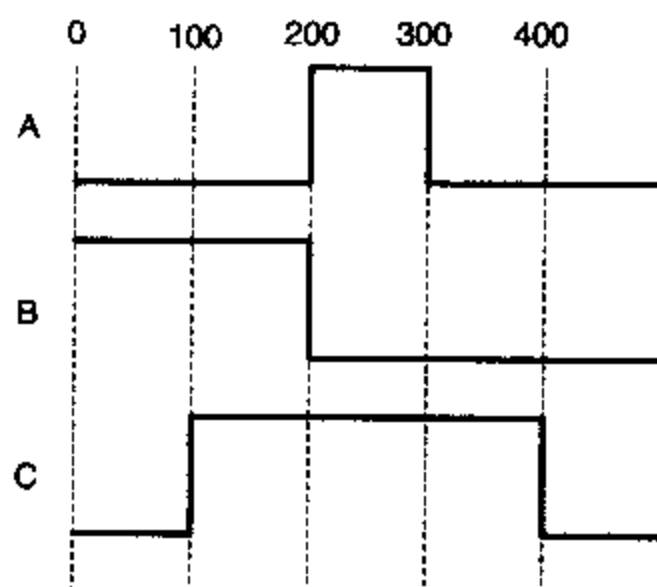


图 8.4 激励波形

【例 8.13】激励波形的描述

```
'timescale 1ns/1ns
module test1;
  reg A,B,C;
  initial
    begin                                //激励波形描述
      A = 0; B = 1; C = 0;
      #100 C = 1;
      #100 A = 1; B = 0;
      #100 A = 0;
      #100 C = 0;
      #100 $finish;
    end
```

```
initial $monitor($time,,, "A=%d B=%d C=%d", A, B, C);    //显示
endmodule
```

上面例子的运行结果为:

```
0   A=0 B=1 C=0
100 A=0 B=1 C=1
200 A=1 B=0 C=1
300 A=0 B=0 C=1
400 A=0 B=0 C=0
```

可见, 其运行结果与图 8.4 的波形完全吻合。

在下面的例子中, 用 `always` 过程块描述所需的时钟波形。

【例 8.14】 `always` 语句用于时钟波形的描述

```
'timescale 1ns/1ns
.....
reg clk;
parameter CYCLE = 100;                //一个时钟周期 100ns
always # (CYCLE/2) clk = ~clk;        //always 语句产生时钟波形
initial clk = 1;
.....
```

【例 8.15】 用 `always` 过程块产生两个时钟信号

```
module test2;
reg clk1, clk2;
parameter CYCLE = 100;
always
begin
    {clk1,clk2} = 2'b10;
    #(CYCLE /4) {clk1,clk2} = 2'b01;
    #(CYCLE /4) {clk1,clk2} = 2'b11;
    #(CYCLE /4) {clk1,clk2} = 2'b00;
    #(CYCLE /4) {clk1,clk2} = 2'b10;
end
initial $monitor($time,,, "clk1=%b clk2=%b", clk1, clk2);
endmodule
```

上例用 ModelSim 编译仿真后的波形如图 8.5 所示。



图 8.5 仿真输出波形。

该例的输出结果为:

```

0  clk1=1 clk2=0
25 clk1=0 clk2=1
50 clk1=1 clk2=1
75 clk1=0 clk2=0

100 clk1=1 clk2=0
125 clk1=0 clk2=1
150 clk1=1 clk2=1
175 clk1=0 clk2=0

200 clk1=1 clk2=0
225 clk1=0 clk2=1
250 clk1=1 clk2=1
275 clk1=0 clk2=0

300 clk1=1 clk2=0
325 clk1=0 clk2=1
350 clk1=1 clk2=1
375 clk1=0 clk2=0

.....

```

for 循环语句也经常用在测试模块的编写中。在下面的例子中, 用 for 语句实现数据的读入。

【例 8.16】for 循环语句的使用

```

integer i ;
parameter STEP = 100 ;
initial
begin
    for (i=0; i<256; i=i+1 )
    begin
        # STEP data = MEM[i] ;    //每经过 STEP 延时, 从 MEM 存储器读入 一个数据
        .....
    end
    .....
end

```

在程序的仿真中, 如果测试向量很多的话, 可以先将测试向量写入一个文件, 然后在仿真程序中使用 readmemb 或 readmemh 将测试向量读入。下面是一个例子, 在该例中, 首先定义了一个存储器 mem[0:255], 然后用 \$readmemh 函数将 rom.hex 文件中的数据读入该存储器。

【例 8.17】存储器在仿真程序中的应用

```

module ROM(addr, data, oe);
    output[7:0] data;           //数据信号
    input[14:0] addr;           //地址信号
    input oe;                   //读使能信号, 低电平有效

    reg[7:0] mem[0:255];       //存储器定义
    parameter DELAY = 100;

```



```

assign #DELAY data = (oe==0) ? mem[addr] : 8'hzz ;

initial $readmemb("rom.hex",mem);    //从文件中读入数据
endmodule

```

8.6 仿 真 示 例

本节通过若干实例介绍如何进行程序的仿真。

8.6.1 组合电路的仿真

首先，以一个8位乘法器的仿真为例来介绍组合电路的仿真。

1. 8位乘法器的仿真

【例 8.18】8 位乘法器的仿真程序

```

'timescale 10ns/1ns

module mult_tp;                                //测试模块的名字
reg[7:0]  a,b;                                //测试输入信号定义为 reg 型
wire[15:0] out;                              //测试输出信号定义为 wire 型
integer i,j;

mult8 m1(out,a,b);                            //调用测试对象
                                              //激励波形设定

initial
    begin
        a=0;b=0;
        for(i=1;i<255;i=i+1)
            #10 a=i;
        end
initial
    begin
        for(j=1;j<255;j=j+1)
            #10 b=j;
        end

initial                                        //定义结果显示格式
    begin
        $monitor($time,, "%d * %d = %d",a,b,out);
        #2560 $finish;
    end

```

```
endmodule
```

```
module mult8(out, a, b);           //8 位乘法器源代码
parameter size=8;
input[size:1] a,b;                //两个操作数
output[2*size:1] out;             //结果
assign out=a*b;                   //乘法运算符
endmodule
```

上面的例子用 ModelSim 编译, 其功能仿真波形如图 8.6 所示。

/mult_tp/a	99	14	15	16	17	18	19	20	21	22	23	24	25	26	27
/mult_tp/b	99	14	15	16	17	18	19	20	21	22	23	24	25	26	27
/mult_tp/out	9801	196	225	256	289	324	361	400	441	484	529	576	625	676	729

图 8.6 8 位乘法器的功能仿真波形图

主窗口中的文本显示如下所示:

0	0 * 0 =	0
10	1 * 1 =	1
20	2 * 2 =	4
30	3 * 3 =	9
40	4 * 4 =	16
50	5 * 5 =	25
60	6 * 6 =	36
70	7 * 7 =	49
80	8 * 8 =	64
90	9 * 9 =	81
100	10 * 10 =	100
110	11 * 11 =	121
120	12 * 12 =	144
130	13 * 13 =	169
140	14 * 14 =	196
150	15 * 15 =	225
160	16 * 16 =	256
170	17 * 17 =	289
180	18 * 18 =	324
190	19 * 19 =	361
200	20 * 20 =	400
210	21 * 21 =	441
220	22 * 22 =	484
230	23 * 23 =	529
240	24 * 24 =	576
250	25 * 25 =	625
260	26 * 26 =	676
270	27 * 27 =	729
280	28 * 28 =	784
290	29 * 29 =	841
300	30 * 30 =	900
310	31 * 31 =	961
320	32 * 32 =	1024
330	33 * 33 =	1089
340	34 * 34 =	1156
350	35 * 35 =	1225
360	36 * 36 =	1296
370	37 * 37 =	1369
380	38 * 38 =	1444
390	39 * 39 =	1521

2. 8 位加法器的仿真

如图 8.7 所示是加法器仿真的示意图。

下面是 8 位加法器的仿真程序。

【例 8.19】8 位加法器的仿真程序

```
'timescale 1ns/1ns
module add8_tp;                //仿真模块无端口列表
reg[7:0] A,B;                  //输入激励信号定义为 reg 型
reg cin;
wire[7:0] SUM;                 //输出信号定义为 wire 型
wire cout;
parameter DELY = 100;
```

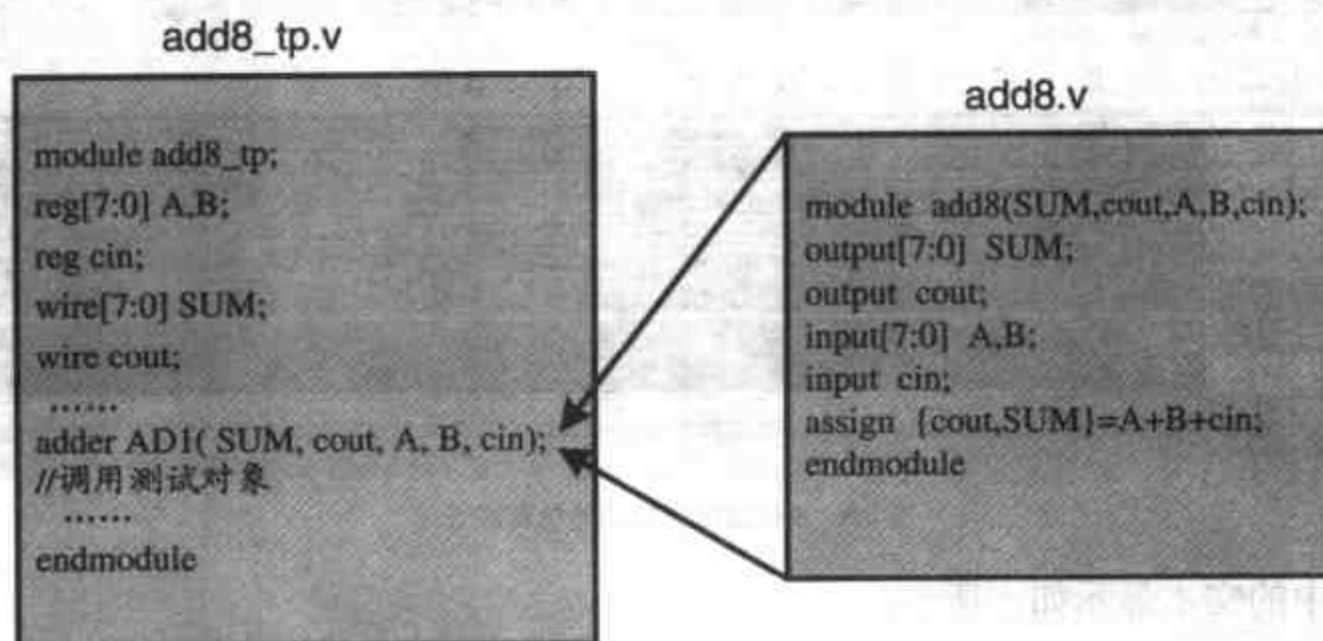



图 8.7 加法器仿真的示意图

```

add8 AD1(SUM,cout,A,B,cin); //调用测试对象

initial begin //激励波形设定
    A= 8'd0;    B= 8'd0;    cin=1'b0;
#DELY A= 8'd100; B= 8'd200; cin=1'b1;
#DELY A= 8'd200; B= 8'd88;
#DELY A= 8'd210; B= 8'd18;  cin=1'b0;
#DELY A= 8'd12;  B= 8'd12;
#DELY A= 8'd100; B= 8'd154;
#DELY A= 8'd255; B= 8'd255; cin=1'b1;
#DELY $finish;
end
//输出格式定义
initial $monitor($time,,, "%d + %d + %b = (%b, %d)", A, B, cin, cout, SUM);
endmodule

module add8(SUM,cout,A,B,cin); //待测试的8位加法器模块
output[7:0] SUM;
output cout;
input[7:0] A,B;
input cin;
assign {cout,SUM}=A+B+cin;
endmodule

```

上面的例子用 ModelSim 编译仿真后的输出波形如图 8.8 所示。

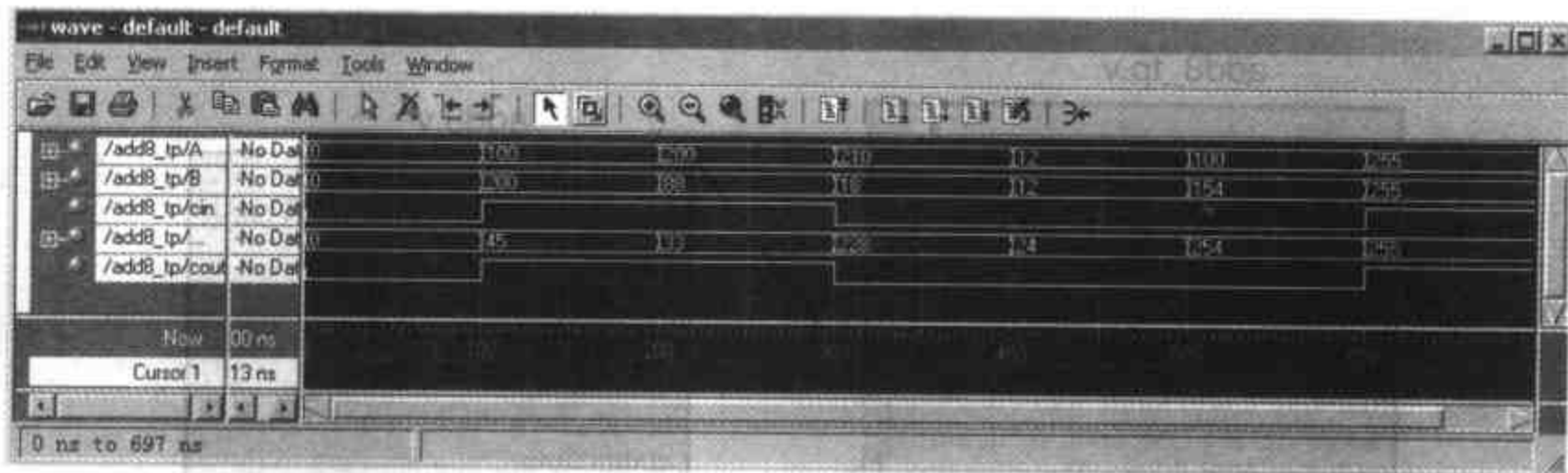


图 8.8 8 位加法器的功能仿真波形

主窗口中的文本显示如下所示:

```

0 0 + 0 + 0 = {0, 0}
100 100 + 200 + 1 = {1, 45}
200 200 + 88 + 1 = {1, 33}
300 210 + 18 + 0 = {0, 228}
400 12 + 12 + 0 = {0, 24}
500 100 + 154 + 0 = {0, 254}
600 255 + 255 + 1 = {1, 255}

```

3. 2 选 1 数据选择器的仿真

下面是一个 2 选 1 多路选择器仿真的例子。

【例 8.20】2 选 1 多路选择器的仿真

```

`timescale 1ns/1ns
module mux_tp;
  reg a,b,sel;
  wire out;

  MUX2_1 m1(out,a,b,sel); //调用待测试模块

initial
begin
  a=1'b0; b=1'b0; sel=1'b0;
  #5 sel=1'b1;
  #5 a=1'b1; sel=1'b0;
  #5 sel=1'b1;
  #5 a=1'b0; b=1'b1; sel=1'b0;
  #5 sel=1'b1;
  #5 a=1'b1; b=1'b1; sel=1'b0;
  #5 sel=1'b1;

```

```

end
initial $monitor($time,,,"a=%b b=%b sel=%b out=%b", a, b, sel, out);
endmodule

```

2选1多路选择器的源代码如下所示,采用门级结构描述,图8.9是其门级原理图。

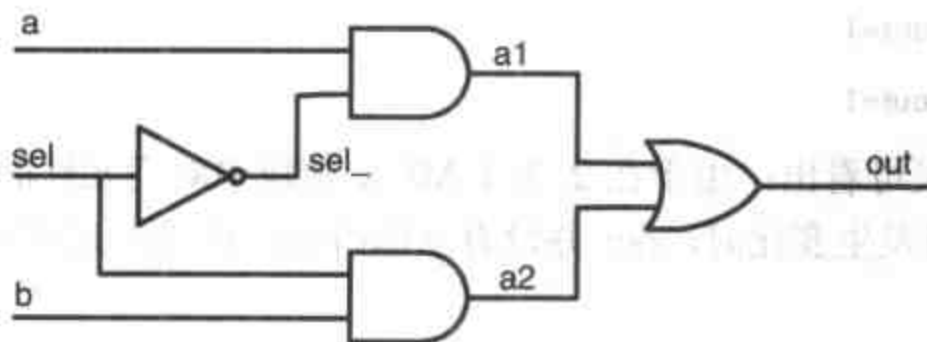


图 8.9 2选1 MUX 门级原理图

```

module MUX2_1(out,a,b,sel); //待测试的2选1MUX模块
input a, b, sel;
output out;
not #(0.4,0.3) (sel_,sel ); // #(0.4,0.3)为门延时
and #(0.7,0.6) (a1 ,a ,sel_ );
and #(0.7,0.6) (a2, b ,sel );
or #(0.7,0.6) (out ,a1 , a2 );
endmodule

```

上面的程序用 ModelSim 编译,仿真波形如图 8.10 所示。

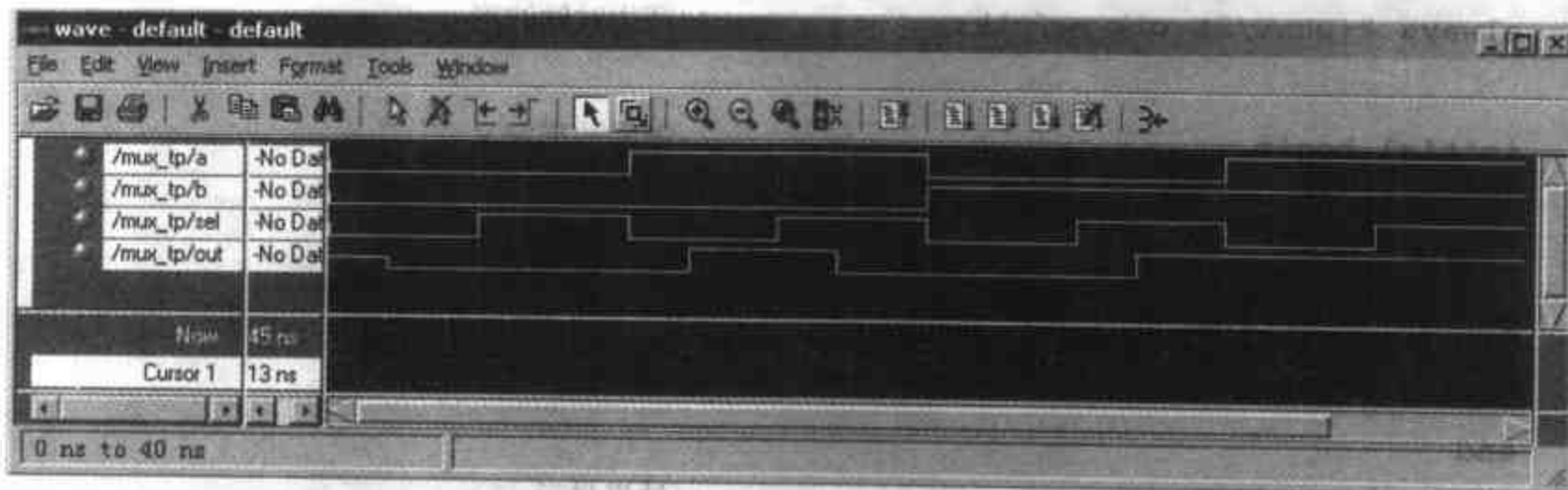


图 8.10 2选1多路选择器的功能仿真波形

主窗口中的文本显示如下所示:

```

0 a=0 b=0 sel=0 out=0
2 a=0 b=0 sel=0 out=0
5 a=0 b=0 sel=1 out=0
10 a=1 b=0 sel=0 out=0
12 a=1 b=0 sel=0 out=1

```

```

15 a=1 b=0 sel=1 out=1
17 a=1 b=0 sel=1 out=0

20 a=0 b=1 sel=0 out=0

25 a=0 b=1 sel=1 out=0
27 a=0 b=1 sel=1 out=1

30 a=1 b=1 sel=0 out=1

35 a=1 b=1 sel=1 out=1

```

从上面的输出结果可看出：由于在 2 选 1 MUX 模块中对门元件的延时作了定义，因此当输入 a、b、sel 的值发生变化时，out 并没有立即改变，而是经过相应的门延时后，out 的值才改变。

8.6.2 时序电路的仿真

以一个带有同步复位的 8 位计数器的仿真为例，介绍时序电路的仿真。

【例 8.21】8 位计数器的仿真

```

`timescale 10ns/1ns
module count8_tp;
    reg clk, reset;                //输入激励信号定义为 reg 型
    wire[7:0] qout;                //输出信号定义为 wire 型
    parameter DELY=100;

    counter C1(qout,reset,clk);    //调用测试对象

    always #(DELY/2) clk = ~clk;  //产生时钟波形

    initial begin                  //激励波形定义
        clk =0; reset=0;
        #DELY reset=1;
        #DELY reset=0;
        #(DELY*300) $finish;
    end

    //结果显示
    initial $monitor($time,,,"clk=%d reset=%d qout=%d", clk, reset,qout);
endmodule

module counter(qout,reset,clk);    //待测试的 8 位计数器模块
    output[7:0] qout;
    input clk,reset;
    reg[7:0] qout;
    always @(posedge clk)

```



```

begin    if (reset) qout<=0;
          else      qout<=qout+1;
        end
endmodule

```

上面的例子用 ModelSim 编译仿真后的波形如图 8.11 所示。

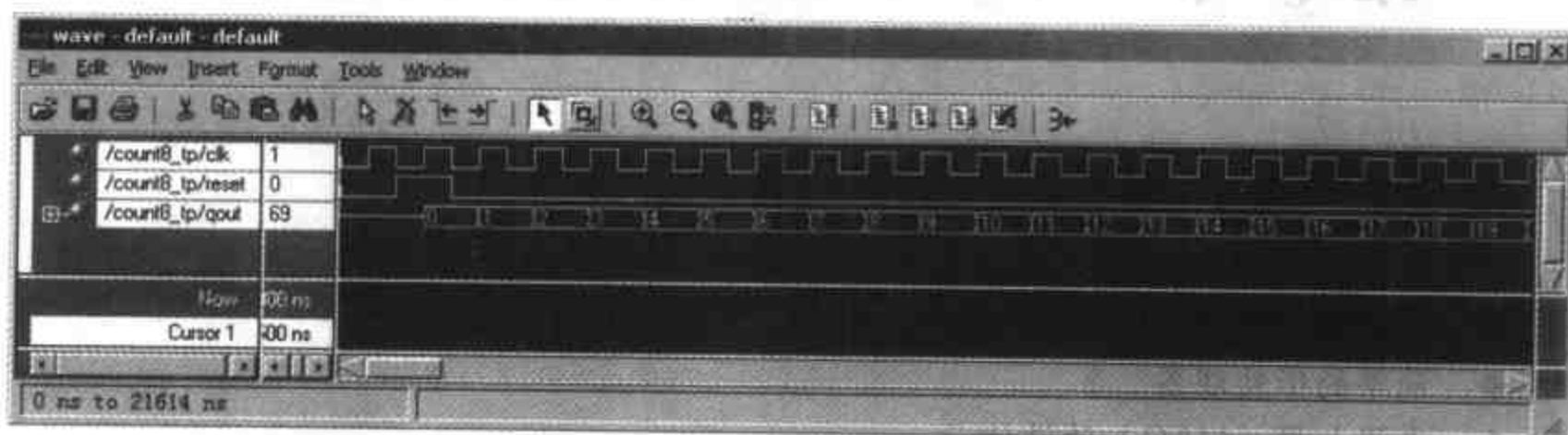


图 8.11 计数器的仿真波形

主窗口中的文本显示如下所示:

0 clk=0 reset=0 qout= x	950 clk=1 reset=0 qout= 8
50 clk=1 reset=0 qout= x	1000 clk=0 reset=0 qout= 8
100 clk=0 reset=1 qout= x	1050 clk=1 reset=0 qout= 9
150 clk=1 reset=1 qout= 0	1100 clk=0 reset=0 qout= 9
200 clk=0 reset=0 qout= 0	1150 clk=1 reset=0 qout= 10
250 clk=1 reset=0 qout= 1	1200 clk=0 reset=0 qout= 10
300 clk=0 reset=0 qout= 1	1250 clk=1 reset=0 qout= 11
350 clk=1 reset=0 qout= 2	1300 clk=0 reset=0 qout= 11
400 clk=0 reset=0 qout= 2	1350 clk=1 reset=0 qout= 12
450 clk=1 reset=0 qout= 3	1400 clk=0 reset=0 qout= 12
500 clk=0 reset=0 qout= 3	1450 clk=1 reset=0 qout= 13
550 clk=1 reset=0 qout= 4	1500 clk=0 reset=0 qout= 13
600 clk=0 reset=0 qout= 4	1550 clk=1 reset=0 qout= 14
650 clk=1 reset=0 qout= 5	1600 clk=0 reset=0 qout= 14
700 clk=0 reset=0 qout= 5	1650 clk=1 reset=0 qout= 15
750 clk=1 reset=0 qout= 6	1700 clk=0 reset=0 qout= 15
800 clk=0 reset=0 qout= 6	1750 clk=1 reset=0 qout= 16
850 clk=1 reset=0 qout= 7	1800 clk=0 reset=0 qout= 16
900 clk=0 reset=0 qout= 7	
.....	

思考与练习

1. 什么是仿真? 常用的 Verilog 仿真器有哪些?
2. 写出 1 位全加器本位和 (SUM) 的 UDP 描述。
3. 写出 4 选 1 多路选择器的 UDP 描述。
4. `timescale 指令的作用是什么? 举例说明。
5. 编写一个 4 位的比较器, 并对其进行测试。
6. 编写一个测试程序, 对 D 触发器的逻辑功能进行测试。

第9章 Verilog HDL 设计进阶

本章内容简介

- ◆ 基本组合电路的设计
- ◆ 基本时序电路的设计
- ◆ 简单的微处理器
- ◆ 乘累加器 (MAC) 的设计

本章通过若干可综合的实例来介绍基本数字电路的 Verilog 描述, 这些基本的数字模块通常可以用多种方式来设计, 如行为描述、结构描述或者数据流描述等。这些基本的数字模块为更复杂的设计提供了方便。

9.1 基本组合电路的设计

9.1.1 简单门电路

1. 基本门电路的描述

图 9.1 所示是一个基本门电路, 下面用几种方式对其进行描述。

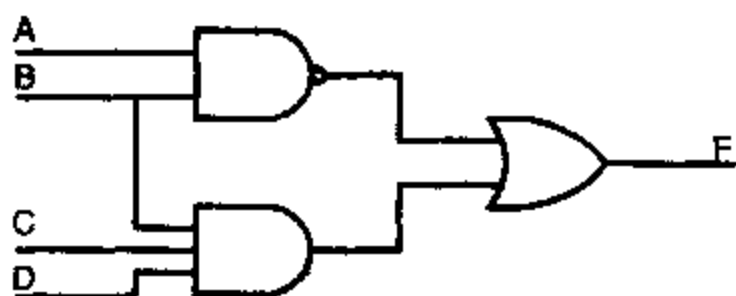


图 9.1 简单的门电路

【例 9.1】基本门电路的几种描述方法

(1) 门级结构描述

```
module gate1(F,A,B,C,D);  
input  A,B,C,D;  
output F;  
nand(F1,A,B);           //调用门元件  
and(F2,B,C,D);
```

```
or(F,F1,F2);
```

```
endmodule
```

(2) 数据流描述

```
module gate2(F,A,B,C,D);
```

```
input A,B,C,D;
```

```
output F;
```

```
assign F=(A&B)|(B&C&D); //assign 持续赋值
```

```
endmodule
```

(3) 行为描述

```
module gate3(F,A,B,C,D);
```

```
input A,B,C,D;
```

```
output F;
```

```
reg F;
```

```
always @(A or B or C or D) //过程赋值
```

```
begin
```

```
F=(A&B)|(B&C&D);
```

```
end
```

```
endmodule
```

2. 三态门

(1) 用 bufif1 关键字描述的三态门

例 9.2 中调用 Verilog 语言的门元件 bufif1 描述了一个三态门, 该三态门当 en 控制端为高电平时, 输出 out=in; 当 en 端为低电平时, 输出 out 为高阻态。

【例 9.2】用 bufif1 关键字描述的三态门

```
module tri_1(in, en, out);
```

```
input in, en;
```

```
output out;
```

```
tri out;
```

```
bufif1 b1(out, in, en); //注意三态门端口的排列顺序
```

```
endmodule
```

(2) 用数据流方式描述的三态门

例 9.3 用数据流方式描述了一个功能与上例相同的三态门。

【例 9.3】用 assign 语句描述的三态门

```
module tri_2(out, in, en);
```

```
output out;
```

```
input in, en;
```

```
assign out = en ? in : 'bz;
```

```
//若 en=1, 则 out=in; 若 en=0, 则 out 为高阻态
```

```
endmodule
```

例 9.3 的 RTL 综合结果如图 9.2 所示。

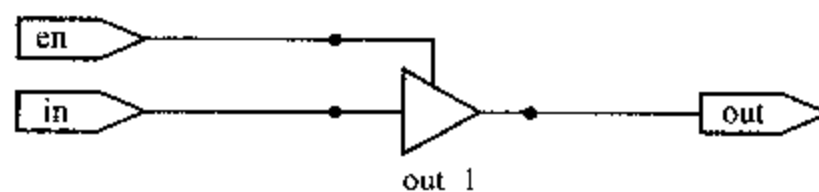


图 9.2 三态门的综合结果

3. 三态双向驱动器

下面程序描述的是一个三态双向驱动器。

【例 9.4】三态双向驱动器

```

module bidir(tri_inout, out, in, en, b);
inout tri_inout;
output out;
input in, en, b;
assign tri_inout = en ? in : 'bz;
assign out = tri_inout ^ b;
endmodule

```

该程序的综合结果如图 9.3 所示。

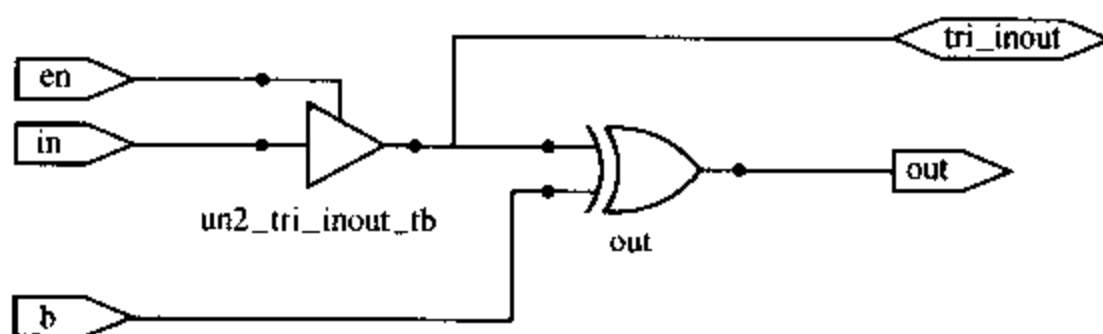


图 9.3 三态双向驱动器的综合结果

下面也是一个三态双向驱动器的例子。

【例 9.5】三态双向驱动器

```

module bidir2(bidir, en, clk);
inout[7:0] bidir;
input en, clk;
reg[7:0] temp;
assign bidir = en ? temp : 8'bz;
always @(posedge clk)
begin
    if(en) temp = bidir;
    else temp = temp + 1;
end
endmodule

```

该程序的综合结果如图 9.4 所示。

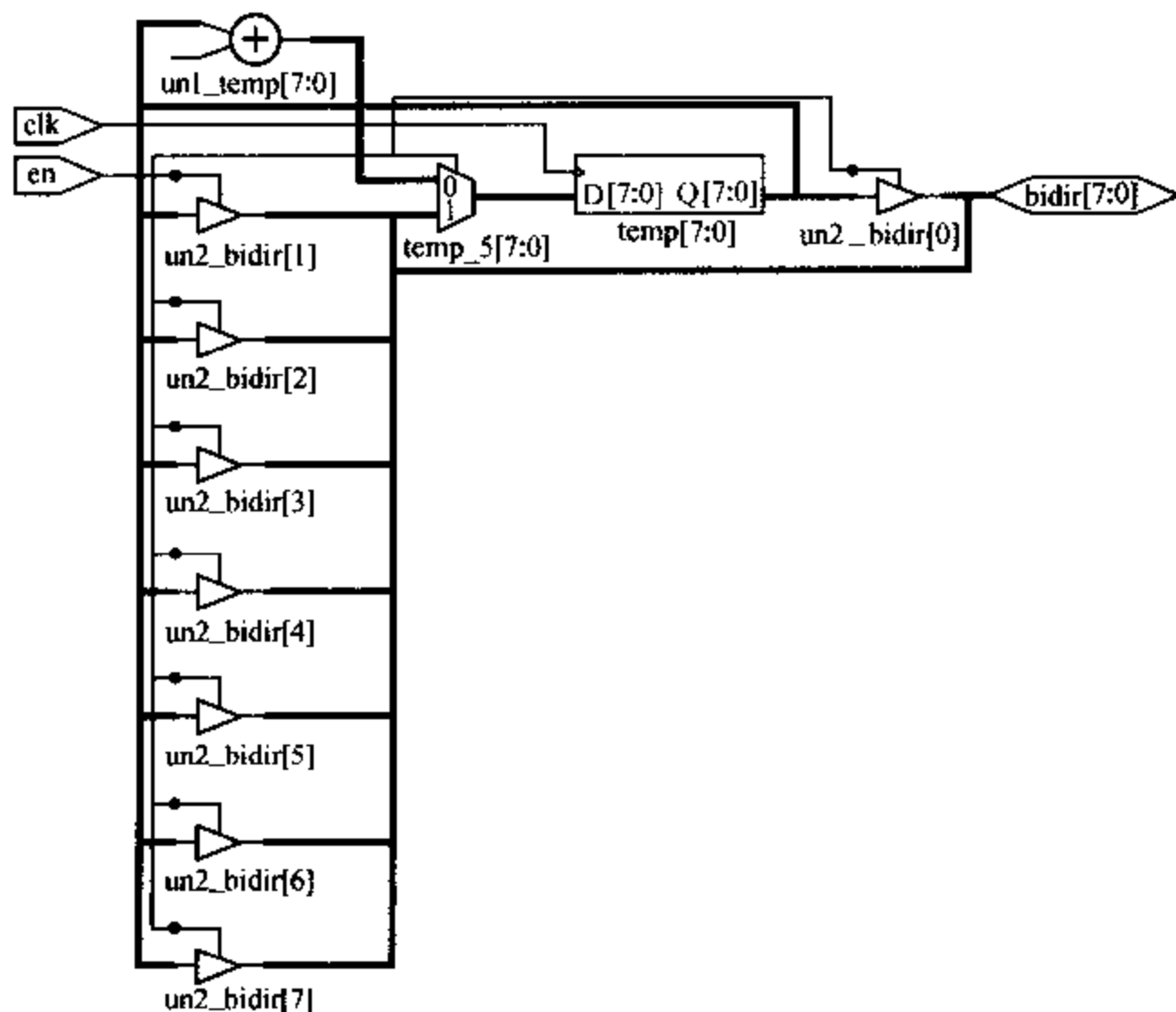


图 9.4 例 9.5 的综合结果

9.1.2 编译码器

1. 3-8 译码器

下面的例子用 **case** 语句描述了一个 3-8 译码器。

【例 9.6】3-8 译码器

```

module decoder_38(out, in);
output[7:0] out;
input[2:0] in;
reg[7:0] out;
always @(in)
begin
    case(in)
        3'd0: out=8'b11111110;
        3'd1: out=8'b11111101;
        3'd2: out=8'b11111011;
        3'd3: out=8'b11110111;
        3'd4: out=8'b11101111;
        3'd5: out=8'b11011111;
        3'd6: out=8'b10111111;
        3'd7: out=8'b01111111;
    endcase
end

```

```

    endcase
  end
endmodule

```

2. 8-3 优先编码器

作为条件语句，if-else 语句的分支是有优先顺序的。利用 if-else 语句的特点，实现优先编码器的设计。

下面的例子描述了一个 8-3 优先编码器，该编码器的输入为 a、b、c、d、e、f、g、h，输入信号高电平有效，h 的优先级最高。

【例 9.7】8-3 优先编码器

```

module encoder8_3(none_on,outcode,a,b,c,d,e,f,g,h);
output none_on;
output[2:0] outcode;
input a,b,c,d,e,f,g,h;
reg[3:0] outtemp;
assign {none_on,outcode}=outtemp;
always @(a or b or c or d or e or f or g or h)
begin
    if(h) outtemp=4'b0111;
    else if(g) outtemp=4'b0110;
    else if(f) outtemp=4'b0101;
    else if(e) outtemp=4'b0100;
    else if(d) outtemp=4'b0011;
    else if(c) outtemp=4'b0010;
    else if(b) outtemp=4'b0001;
    else if(a) outtemp=4'b0000;
    else outtemp=4'b1000;
end
endmodule

```

3. 用函数定义的 8-3 优先编码器

下面的例子用函数（function）定义了一个 8-3 优先编码器，该编码器同样采用 if-else 语句描述。

【例 9.8】用函数定义的 8-3 优先编码器

```

module code_83(din, dout) ;
input[7:0] din ;
output[2:0] dout;

function [2:0] code; //函数定义
input [7:0] din; //函数只有输入端口，输出为函数名本身
if (din[7]) code = 3'd7;

```

```

else if (din[6])    code = 3'd6;
else if (din[5])    code = 3'd5;
else if (din[4])    code = 3'd4;
else if (din[3])    code = 3'd3;
else if (din[2])    code = 3'd2;
else if (din[1])    code = 3'd1;
else                code = 3'd0;
endfunction

```

```

assign dout = code(din); //函数调用
endmodule

```

4. 七段数码管译码器

下面是用 case 语句编写的七段数码管译码器源代码。

【例 9.9】七段数码管译码器

```

module decode47(a,b,c,d,e,f,g,D3,D2,D1,D0);
output  a,b,c,d,e,f,g;
input   D3,D2,D1,D0;           //输入的 4 位 BCD 码
reg     a,b,c,d,e,f,g;
always @(D3 or D2 or D1 or D0)
begin
    case ({D3,D2,D1,D0})       //用 case 语句进行译码
        4'd0: {a,b,c,d,e,f,g}=7'b1111110;
        4'd1: {a,b,c,d,e,f,g}=7'b0110000;
        4'd2: {a,b,c,d,e,f,g}=7'b1101101;
        4'd3: {a,b,c,d,e,f,g}=7'b1111001;
        4'd4: {a,b,c,d,e,f,g}=7'b0110011;
        4'd5: {a,b,c,d,e,f,g}=7'b1011011;
        4'd6: {a,b,c,d,e,f,g}=7'b1011111;
        4'd7: {a,b,c,d,e,f,g}=7'b1110000;
        4'd8: {a,b,c,d,e,f,g}=7'b1111111;
        4'd9: {a,b,c,d,e,f,g}=7'b1111011;
        default: {a,b,c,d,e,f,g}=7'bx;
    endcase
end
endmodule

```

5. 奇偶校验位产生器

下面的例子对并行输入的 8 位数据 input 进行奇偶校验，并产生奇校验位 odd_bit 和偶校验位 even_bit。

【例 9.10】奇偶校验位产生器

```

module parity(even_bit, odd_bit, input_bus);
output even_bit, odd_bit;
input [7:0] input_bus;
assign odd_bit = ^ input_bus;           //产生奇校验位
assign even_bit = ~odd_bit;            //产生偶校验位
endmodule

```

图 9.5 是该例的综合结果。

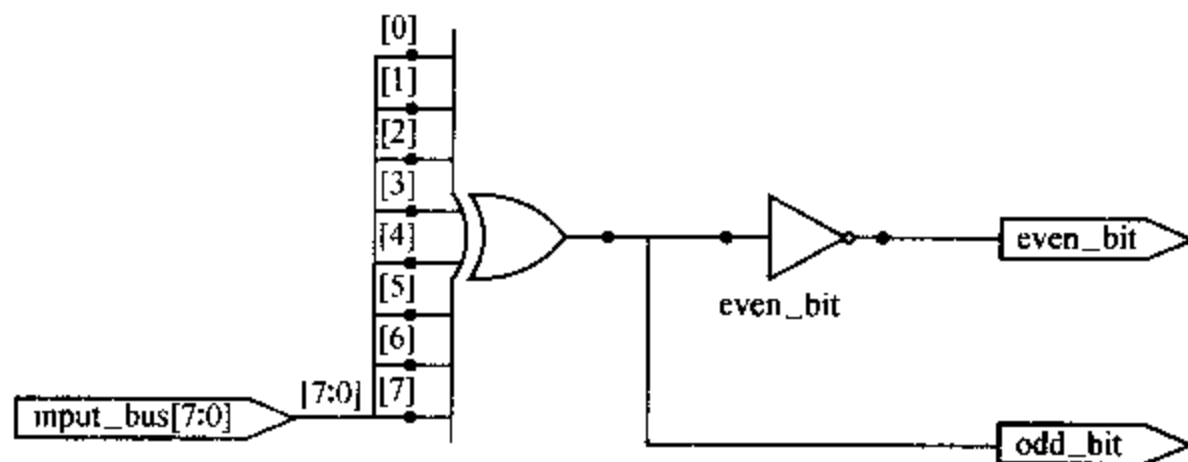


图 9.5 例 9.10 的综合结果

9.1.3 数据选择器

1. 用 if-else 语句描述 4 选 1 MUX

【例 9.11】用 if-else 语句描述的 4 选 1 MUX

```

module mux_if(out,in0,in1,in2,in3,sel);
output out;
input in0,in1,in2,in3;
input [1:0] sel;
reg out;
always @(in0 or in1 or in2 or in3 or sel)
begin
if(sel==2'b00) out=in0;
else if(sel==2'b01) out=in1;
else if(sel==2'b10) out=in2;
else out=in3;
end
endmodule

```

2. 用 case 语句描述的 4 选 1 MUX

【例 9.12】用 case 语句描述的 4 选 1 MUX

```

module mux_case (out,in0,in1,in2,in3,sel);
output out;
input in0,in1,in2,in3;

```

```

input[1:0] sel;
reg out;
always @(in0 or in1 or in2 or in3 or sel)
    begin
        case(sel)
            2'b00: out=in0;
            2'b01: out=in1;
            2'b10: out=in2;
            default: out=in3;
        endcase
    end
endmodule

```

9.1.4 用组合电路实现的 ROM

下面的例子是用组合电路实现的 ROM，该存储器用 16 个存储单元存储了 16 个结果，分别是 0~15 共 16 个整数的平方，根据地址输出相应的结果。

【例 9.13】用组合电路实现的 ROM

```

module rom(addr, data);
input [3:0] addr;
output [7:0] data;

function [7:0] romout;
input [3:0] addr;
    case (addr)
        0 : romout = 0;
        1 : romout = 1;
        2 : romout = 4;
        3 : romout = 9;
        4 : romout = 16;
        5 : romout = 25;
        6 : romout = 36;
        7 : romout = 49;
        8 : romout = 64;
        9 : romout = 81;
        10 : romout = 100;
        11 : romout = 121;
        12 : romout = 144;
        13 : romout = 169;
        14 : romout = 196;
    endcase
endfunction

```

```

    15 : romout = 225;
    default: romout = 3'hxx;
    endcase
endfunction
assign data = romout(addr);
endmodule

```

例 9.13 的综合结果如图 9.6 所示。

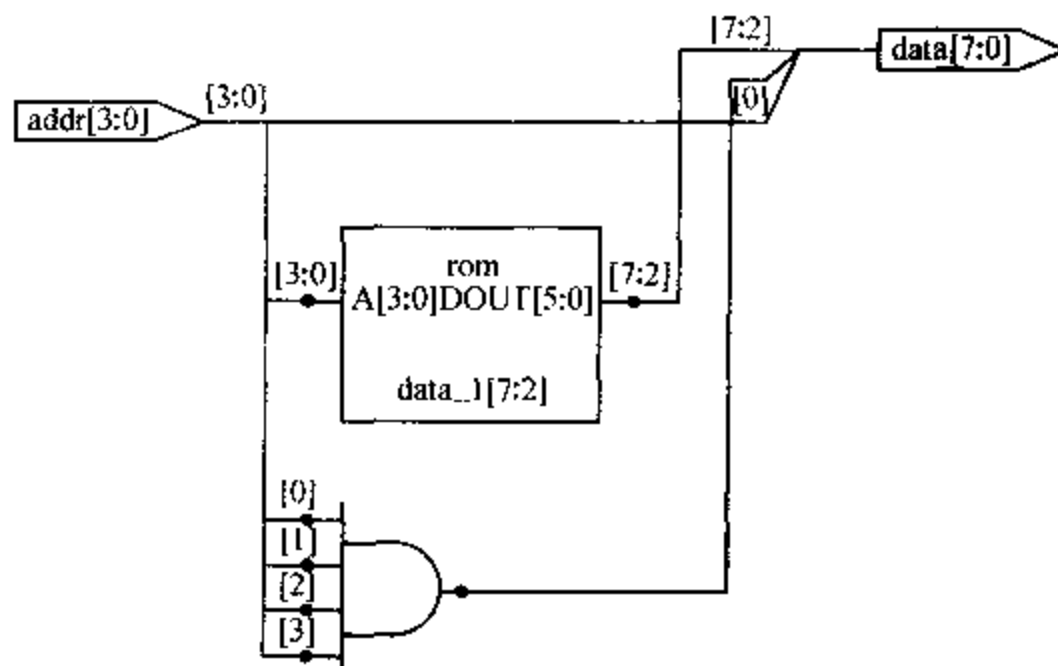


图 9.6 例 9.13 的综合结果

9.2 基本时序电路的设计

本节介绍常用时序电路的 Verilog HDL 设计。

9.2.1 D 触发器与 JK 触发器

1. 最简单的 D 触发器

下面的代码设计了一个最简单的 D 触发器，该触发器只有信号输入端、输出端和时钟。

【例 9.14】基本 D 触发器

```

module DFF(Q, D, CLK);
output Q;
input D, CLK;
reg Q;
always @(posedge CLK)
begin
    Q <= D;
end
endmodule

```


该例的综合结果如图 9.7 所示。

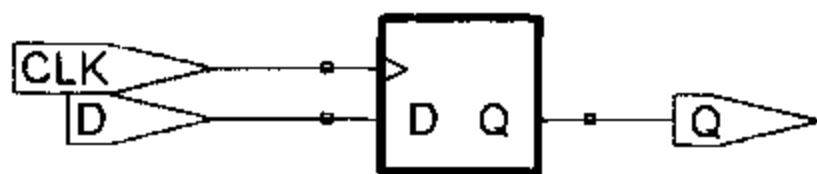


图 9.7 D 触发器

2. 带异步清 0、异步置 1 的 D 触发器

下例为带异步清 0、异步置 1（低电平有效）功能的 D 触发器的描述。

【例 9.15】带异步清 0、异步置 1 的 D 触发器

```

module DFF1(q, qn, d, clk, set, reset);
input d, clk, set, reset;
output q, qn;
reg q, qn;
always @(posedge clk or negedge set or negedge reset)
begin
    if (!reset) begin
        q <= 0;           //异步清 0, 低电平有效
        qn <= 1;
    end
    else if (!set) begin
        q <= 1;           //异步置 1, 低电平有效
        qn <= 0;
    end
    else begin
        q <= d;
        qn <= ~d;
    end
end
endmodule

```

3. 带同步清 0、同步置 1 的 D 触发器

下面的代码描述了一个带同步清 0、同步置 1（高电平有效）功能的 D 触发器。

【例 9.16】带同步清 0、同步置 1 的 D 触发器

```

module DFF2(q, qn, d, clk, set, reset);
input d, clk, set, reset;
output q, qn;
reg q, qn;
always @(posedge clk)

```

```

begin
    if (reset)      begin
        q <= 0; qn <= 1;      //同步清 0, 高电平有效
    end
    else if (set)    begin
        q <= 1; qn <= 0;      //同步置 1, 高电平有效
    end
    else            begin
        q <= d; qn <= ~d;
    end
end
endmodule

```

4. JK 触发器

下例为带异步清 0、异步置 1（低电平有效）功能的 JK 触发器的描述。

【例 9.17】带异步清 0、异步置 1 的 JK 触发器

```

module JK_FF (CLK, J, K, Q, RS, SET);
input CLK, J, K, SET, RS;
output Q;
reg Q;
always @ (posedge CLK or negedge RS or negedge SET)
begin
    if (!RS) Q <= 1'b0;
    else if (!SET) Q <= 1'b1;
    else case ({J,K})
        2'b00 : Q <= Q;
        2'b01 : Q <= 1'b0;
        2'b10 : Q <= 1'b1;
        2'b11 : Q <= ~Q;
        default: Q <= 1'bx;
    endcase
end
endmodule

```

9.2.2 锁存器与寄存器

1. 数据锁存器 1

在下面的例子中，用数据流方式描述了一个电平敏感的数据锁存器，该锁存器只锁存 1 位数据。

【例 9.18】电平敏感的 1 位数据锁存器

```

module latch_1(q, d, clk);

```



```

output q;
input d, clk;
assign q = clk ? d : q;      //时钟信号为高电平时，将输入端数据锁存
endmodule

```

图 9.8 是其综合结果。

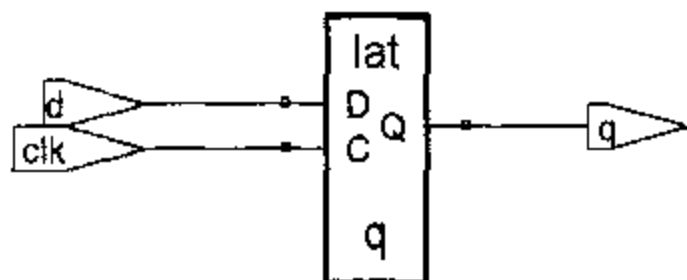


图 9.8 1 位锁存器的综合结果

2. 数据锁存器 2

下面描述的是一个带置位和复位端的电平敏感型的 1 位数据锁存器，同样采用的是数据流描述方式。

【例 9.19】带置位和复位端的 1 位数据锁存器

```

module latch_2(q, d, clk, set, reset);
output q;
input d, clk, set, reset;
assign q = reset ? 0 : (set ? 1 : (clk ? d : q));
endmodule

```

3. 数据锁存器 3

下面的程序中用 always 过程语句描述了一个电平敏感型数据锁存器，该锁存器一次锁存 8 位数据。

【例 9.20】8 位数据锁存器

```

module latch_8(qout,data,clk);
output[7:0] qout;
input[7:0] data;
input clk;
reg[7:0] qout;
always @(clk or data)
begin
if (clk) qout<=data;
end
endmodule

```

4. 数据寄存器

首先来看一下数据锁存器和数据寄存器的区别。从寄存数据的角度来看，锁存器和寄存器的功能是相同的，两者的区别在于：锁存器一般由电平信号来控制，属于电平敏感型，而寄存器一般由同步时钟信号控制。两者有不同的使用场合，主要取决于控制方式及控制信号

和数据信号之间的时序关系：若数据有效滞后于控制信号有效，则只能使用锁存器；若数据提前于控制信号，并要求同步操作，则可以选择寄存器来存放数据。

下面是 8 位数据寄存器的设计，该寄存器每次对 8 位并行输入的数据信号（in_data）进行同步寄存，还具有异步清零端（clear），只要该端口为高电平，寄存器输出端立刻变为零。

【例 9.21】8 位数据寄存器

```

module reg8(out_data,in_data,clk,clr);
output[7:0] out_data;
input[7:0] in_data;
input clk,clr;
reg[7:0] out_data;
always @(posedge clk or posedge clr)
begin
    if(clr)    out_data <=0;
    else      out_data <= in_data;
end
endmodule

```

图 9.9 是 8 位数据寄存器的 RTL 综合视图。

上述程序稍加修改，可将寄存器变为 10 位、16 位或 32 位宽度，也可把异步复位改为同步复位。

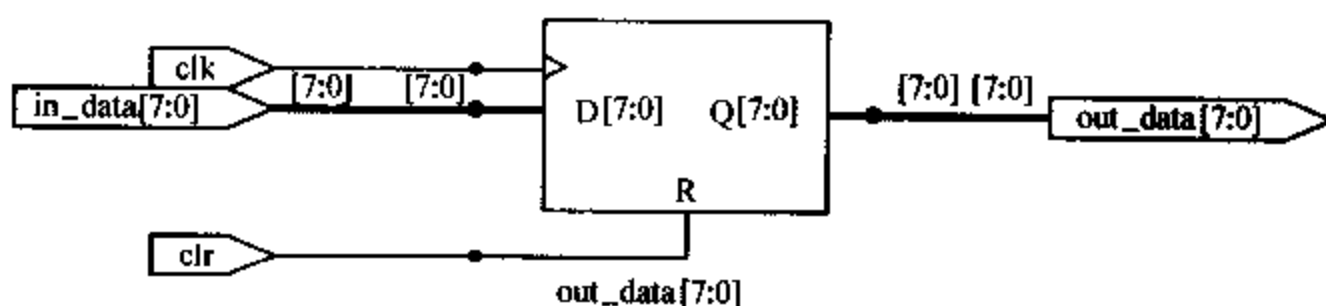


图 9.9 8 位数据寄存器的 RTL 综合视图

5. 移位寄存器

下面的程序设计了一个 8 位移位寄存器。该寄存器有一个时钟输入端（clk）、一个同步清零端（clr）。信号串行输入（din），8 位并行输出（dout），每个时钟周期内输出信号左移一位，同时将串行输入的一位补充到输出信号的最低位。

【例 9.22】8 位移位寄存器

```

module shifter(din, clk, clr, dout);
input din, clk, clr;
output[7:0] dout;
reg[7:0] dout;
always @(posedge clk)
begin
    if (clr) dout <= 8'b0;           //同步清 0，高电平有效
    else

```

```

        begin
            dout <= dout << 1;           //输出信号左移一位
            dout[0] <= din;              //输入信号补充到输出信号的最低位
        end
    end
endmodule

```

8 位移位寄存器的综合结果如图 9.10 所示。

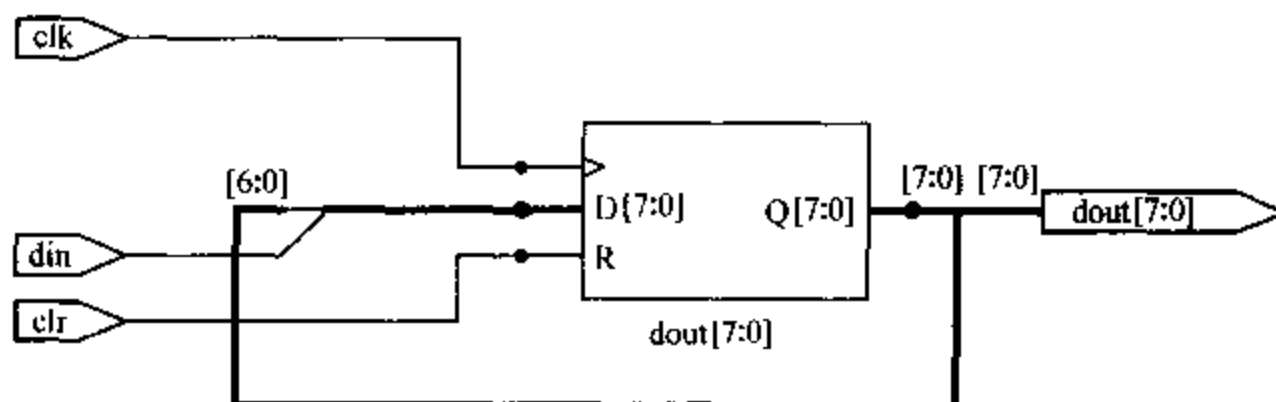


图 9.10 8 位移位寄存器的综合结果

9.2.3 计数器

下面描述了一个可加/可减计数器。该计数器有一个加/减控制端 `up_down`，当该控制端为高电平时，实现加法计数；为低电平时，实现减法计数。`load` 为同步预置端，`clear` 则是同步清 0 端（低电平有效）。

【例 9.23】可变模加法/减法计数器

```

module updown_count(d, clk, clear, load, up_down, qd);
    input [7:0] d;
    input clk;
    input clear;
    input load;
    input up_down;
    output [7:0] qd;
    reg [7:0] cnt;
    assign qd = cnt;
    always @ (posedge clk)
        begin
            if (!clear)                cnt = 8'h00;           //同步清 0，低电平有效
            else if (load)              cnt = d;               //同步预置
            else if (up_down)           cnt = cnt + 1;         //加法计数
            else                         cnt = cnt - 1;         //减法计数
        end
endmodule

```

下面的例子描述了一个 4 位约翰逊 (Johnson) 计数器。

【例 9.24】4 位 Johnson 计数器 (异步复位)

```

module johnson (clk, clr, out );
input clk, clr;
output [3:0] out;
reg [3:0] out;
always @ (posedge clk or posedge clr)
begin
    if (clr)        out<= 4'h0 ;
    else
        begin      out<= out<< 1 ;
                    out[0]<= ~out[3] ;
        end
    end
end
endmodule

```

如图 9.11 所示是 4 位 Johnson 计数器的门级综合结果。

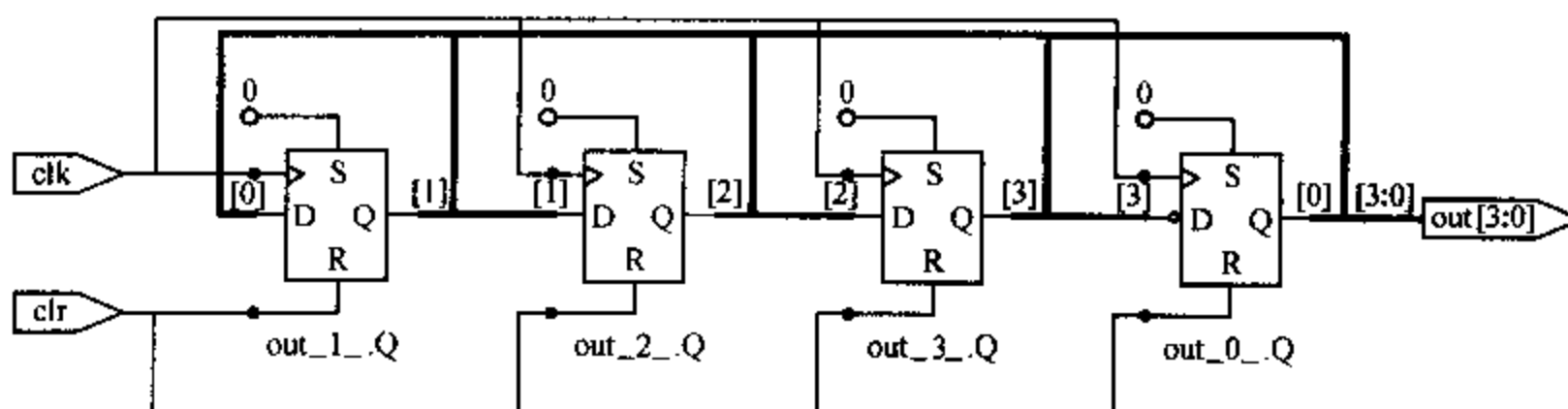


图 9.11 4 位 Johnson 计数器的门级综合结果

9.2.4 ROM/RAM 模块

下面的例子利用 MAX+PLUS II 软件 LPM 宏单元库中的 lpm_ram_dq 函数设计了一个 RAM 模块, 该 RAM 块的尺寸为 256×8 , 即数据总线和地址总线的宽度都是 8 位。

【例 9.25】 256×8 RAM 模块

```

module ram256x8 ( data, address, we, inclock, outclock, q);
input [7:0] data;
input [7:0] address;
input we, inclock, outclock;
output [7:0] q;
lpm_ram_dq myram(.q (q), .data(data), .address(address),
                  .we(we), .inclock(inclock), .outclock(outclock));
defparam myram.lpm_width = 8; //定义数据宽度
defparam myram.lpm_widthad = 8; //定义地址宽度

```

endmodule

在该 RAM 块中, we (write enable) 为写使能, inclock 为输入时钟端, outclock 为输出时钟端, 工作于异步方式。

利用 LPM 宏单元库的 lpm_ram_dq 可以方便地构成任意大小的 RAM 块, 如下面设计的 RAM 块为 256×16 。

【例 9.26】 256×16 RAM 块

```
module map_lpm_ram (dataout, datain, addr, we, inclk, outclk);
input[15:0] datain;           //端口定义
input[7:0] addr;
input we, inclk, outclk;
output[15:0] dataout;

    // lpm_ram_dq 元件例化
lpm_ram_dq ram (.data(datain), .address(addr), .we(we), .inclock(inclk),
               .outclock(outclk), .q(dataout));

defparam ram.lpm_width = 16;           // 参数赋值
defparam ram.lpm_widthad = 8;
defparam ram.lpm_indata = "REGISTERED";
defparam ram.lpm_outdata = "REGISTERED";
defparam ram.lpm_file = "map_lpm_ram.mif"; //RAM 块中的内容从该文件输入
endmodule
```

上面 RAM 块的内容存储于 .mif (Memory Initialization File) 文件中, .mif 文件是一个 ASCII 文本文件, 可以手工创建, 也可以是模拟仿真的输出结果。

9.2.5 串/并转换

下面的例子设计了一个 4 位串/并转换器。

【例 9.27】4 位串/并转换器

```
module serial_pal (clk, reset, en, in, out);
input clk, reset, en, in;
output[3:0] out;
reg[3:0] out;
always @ (posedge clk)
begin
    if (reset)        out<= 4'h0;
    else if (en)      out<= {out, in};           //使用连接运算符
end
endmodule
```

9.3 简单的微处理器

在本节中，设计实现一个简单的微处理器，该处理器根据输入的指令能实现 4 种操作，分别为两数相加、两数相减、操作数加 1、操作数减 1。操作码和操作数均从输入指令中提取。

9.3.1 设计实现

【例 9.28】用函数实现简单的处理器

```

module mpc(instr,out);


```



```

    default:
        begin
            add_func=0;
            opr2=8'd1;          //实现-1 操作
        end
    endcase
    code_add={add_func,opr2,opr1};
end
endfunction

always @(instr)
begin
    {func,op2,op1}=code_add(instr); //调用函数
    if(func==1) out=op1+op2;        //实现两数相加、操作数1 加 1 操作
    else      out=op1-op2;          //实现两数相减、操作数1 减 1 操作
end
endmodule

```

简单微处理器的 RTL 综合结果如图 9.12 所示。

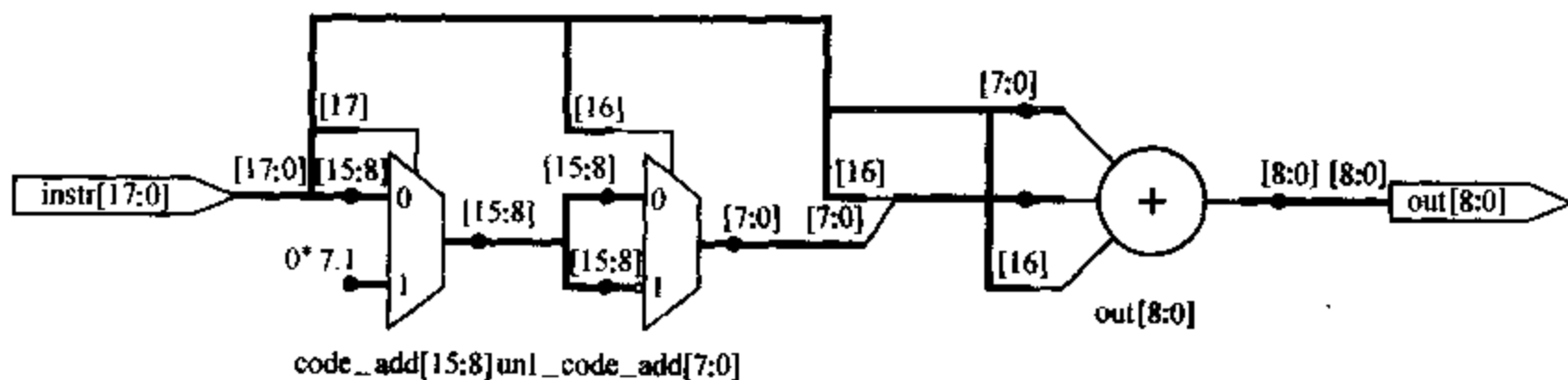


图 9.12 简单微处理器的 RTL 综合结果

9.3.2 仿真

为对上面的微处理器的功能进行检验，编写下面的测试模块。

【例 9.29】微处理器的测试代码

```

`timescale 10ns/1ns
`include "mpc.v"
module mpc_tp;
    reg[17:0] instr;
    wire[8:0] out;

    parameter DELY=10;
    mpc m1(instr,out);          //调用待测试模块
    initial begin

```

```

instr=18'd0;
#DELY instr=18'b00_01001101_00101111;
#DELY instr=18'b00_11001101_11101111;
#DELY instr=18'b01_01001101_11101111;
#DELY instr=18'b01_01001101_00101111;
#DELY instr=18'b10_01001101_00101111;
#DELY instr=18'b11_01001101_00101111;
#DELY instr=18'b00_01001101_00101111;
#DELY $finish;
end
initial $monitor($time,,"instr=%b out=%b", instr, out);
endmodule

```

用 ModelSim 软件对上面的测试程序进行编译和仿真,得到下面的输出显示。通过分析,证明该处理器实现了所设计的 4 种简单的功能。

```

0, instr=00000000000000000000 out=000000000
100, instr=000100110100101111 out=001111100
200, instr=001100110111101111 out=110111100
300, instr=010100110111101111 out=010100010
400, instr=010100110100101111 out=111100010
500, instr=100100110100101111 out=000110000
600, instr=110100110100101111 out=000101110
700, instr=000100110100101111 out=001111100

```

对该微处理器可以很方便地进行功能扩充,实现更为复杂的逻辑操作和数值计算。

9.4 乘累加器 (MAC) 的设计

本节设计一个简单的乘累加器 (MAC, Multiply Accumulator), 图 9.13 是该乘累加器的框图。

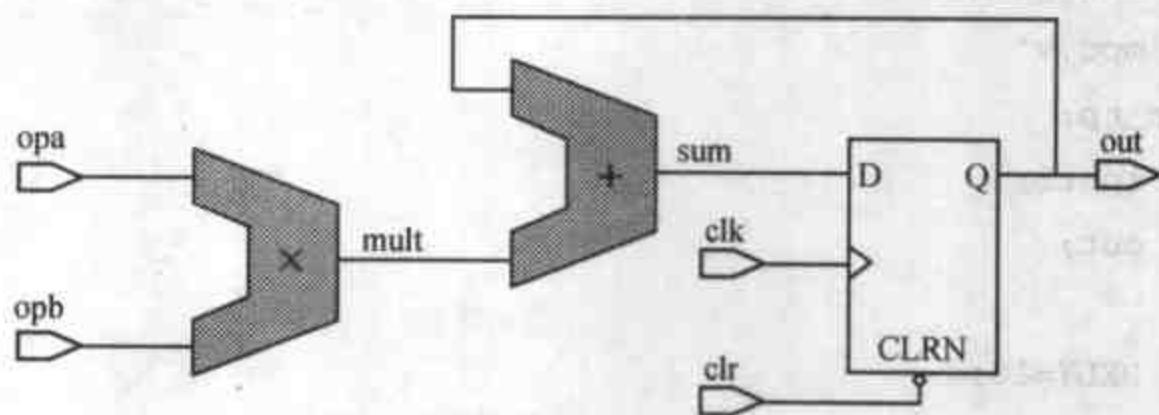


图 9.13 乘累加器的框图

【例 9.30】乘累加器 (MAC) 代码

```

module MAC(out,opa,opb,clk,clr);
    output[15:0] out;
    input[7:0] opa,opb;
    input clk,clr;
    wire[15:0] sum;
    reg[15:0] out;

    function[15:0] mult;           //函数定义, mult 函数完成乘法操作
    input[7:0] opa,opb;           //函数只能定义输入端, 输出端口为函数名本身
    reg[15:0] result;
    integer i;

    begin
        result = opa[0]? opb : 0;
        for(i= 1; i <= 7; i = i+1)
            begin
                if (opa[i]==1) result=result+(opb<<(i-1));
            end
        mult=result;
    end
endfunction

    assign sum=mult(opa,opb)+out;

    always @(posedge clk or posedge clr)
        begin
            if(clr) out<=0;
            else out<=sum;
        end
endmodule

```

为了对上面乘累加器的功能进行检验, 编写下面的测试程序。

【例 9.31】乘累加器的测试代码

```

`timescale 1ns/1ns
`include "mac.v"
module mac_tp;
    reg[7:0] opa,opb;           //测试输入信号用 reg 型变量
    reg clr,clk;
    wire[15:0] out;           //测试输出信号用 wire 型变量

```

```

parameter DELY = 100;
//测试对象调用
MAC m1(out,opa,opb,clk,clr);

always #(DELY) clk = ~clk;      //产生时钟波形

initial begin                  //激励波形定义
    clr=1;clk=0;opa=8'd0; opb= 8'd0;
    #DELY clr=0;opa=8'd1; opb= 8'd10;
    #DELY opa=8'd2; opb= 8'd10;
    #DELY opa=8'd3; opb= 8'd10;
    #DELY opa=8'd4; opb= 8'd10;
    #DELY opa=8'd5; opb= 8'd10;
    #DELY opa=8'd6; opb= 8'd10;
    #DELY opa=8'd7; opb= 8'd10;
    #DELY opa=8'd8; opb= 8'd10;
    #DELY opa=8'd9; opb= 8'd10;
    #DELY opa=8'd10; opb= 8'd10;
    #DELY $finish;
end

//结果显示
initial $monitor($time,,,"clr=%b opa=%d opb=%d out=%d",clr, opa, opb, out);
endmodule

```

用 ModelSim 软件对上面的测试程序进行编译和仿真,得到下面的输出显示,可见所设计的乘累加器实现了乘累加的功能,但在计算上有一定的延时。

```

0, clr=1 opa= 0 opb= 0 out= 0
100, clr=0 opa= 1 opb= 10 out= 0
200, clr=0 opa= 2 opb= 10 out= 0
300, clr=0 opa= 3 opb= 10 out= 10
400, clr=0 opa= 4 opb= 10 out= 10
500, clr=0 opa= 5 opb= 10 out= 30
600, clr=0 opa= 6 opb= 10 out= 30
700, clr=0 opa= 7 opb= 10 out= 60
800, clr=0 opa= 8 opb= 10 out= 60
900, clr=0 opa= 9 opb= 10 out= 100
1000, clr=0 opa= 10 opb= 10 out= 100
1100, clr=0 opa= 11 opb= 10 out= 150
1200, clr=0 opa= 12 opb= 10 out= 150

```

1300, clr=0 opa= 23 opb= 10 out= 210

1400, clr=0 opa= 14 opb= 10 out= 210

思考与练习

1. 采用不同的描述方式设计本章中的组合电路模块，并综合。
2. 采用不同的描述方式设计本章中的时序电路模块，并综合。
3. 用状态机设计一个带同步复位的十进制计数器，并写出测试程序。
4. 设计实现一个功能类似 74LS160 的电路。
5. 设计一个可预置的十六进制计数器，并仿真。
6. 设计一个“1101”序列检测器。
7. 用 Verilog 语言设计一个最多允许 90 人参加的会议电子表决器，只有当赞成票不少于总数的三分之二时，议案被通过；否则被否决，要求输出表决结果及相应票数。

第 10 章 设计方法与设计技巧的探讨

本章内容简介

- ◆ 可综合的设计技术的讨论
- ◆ 可综合的 Verilog 结构
- ◆ 流水线设计技术 (Pipeline Design)
- ◆ 资源共享 (Resource Sharing)
- ◆ 有限状态机 (FSM) 设计
- ◆ 多层次结构电路的设计
- ◆ 阻塞与非阻塞赋值
- ◆ 如何消除毛刺

本章对数字系统设计中的方法与技巧进行探讨, 并通过若干实例对这些方法进行说明与验证。对设计进行优化总的说是针对综合结果的面积、速度和功耗等问题, 就是怎样使设计面积最省、速度最高而功耗最低。当然也包括其他一些方面, 比如提高系统仿真速度的设计技巧以及如何提高系统可测性的设计技巧等。

10.1 可综合的设计技术的讨论

这里所讲的可综合指的是所设计的指令和代码能转化为具体的电路网表结构。Verilog HDL 既支持电路的设计综合, 也支持电路的模拟仿真, 对面向综合的设计技术进行一些探讨是非常有必要的。

10.1.1 可综合的 Verilog HDL 结构

作为 HDL 语言, 有两种基本的用途: 系统仿真和设计实现。如果程序只用于仿真, 那么几乎所有的语法和编程语句都可以使用; 但如果程序是用于硬件实现(如用于 FPGA 设计), 那么我们就必须保证程序的可综合性, 即所编写的程序能被综合器转化为相应的电路结构, 不可综合的 HDL 语句在用综合工具综合时将被忽略或者报错。所以, 作为设计者, 应该清楚: 所有的 HDL 描述都可用于仿真, 但并非所有的 HDL 描述都可综合。通常, 综合器所支持的 HDL 语言只是该语言的一个优化子集, 故有必要对一般综合器所支持的 Verilog 语言结构作一些讨论和总结。

表 10.1 对多数 EDA 综合工具支持的 Verilog HDL 结构进行了总结, 并对某些结构和语句的使用限制也作了说明 (表中的符号“√”表示可综合)。

表 10.1 一般综合工具支持的 Verilog HDL 结构

Verilog HDL 结构	可综合性说明
module, macromodule	√
wire, reg, integer 数据类型	√
parameter	√
端口类型说明 input, output, inout	√
运算符 +, -, *, %, &, ~&, , ~ , ^, ^~, ==, !=, &&, , !, ~, &, , ^, ^~, >>, <<, ?:, {}	大部分可综合: 全等运算符 (===, !==) 不支持; 多数工具对除法 (/) 和求模 (%) 有限制; 如对除法 (/) 操作, 只有当除数是常数, 且是 2 的指数时才支持
基本门元件 and, nand, nor, or, xor, xnor, buf, not, bufif1, bufif0, notif1, notif0, pullup, pulldown	全部可综合: 但某些工具对取值为 x 和 z 有所限制
持续赋值 assign	√
过程赋值: 阻塞赋值 (=) 非阻塞赋值 (<=)	支持, 但对同一 reg 型变量只能采用阻塞和非阻塞赋值中的一种进行赋值
条件语句: if-else, case, casex, casez, endcase	一般支持, 有的综合器不支持 casex、casez
for 循环语句	√
always 过程语句 begin-end 块语句	√
function, endfunction	√
task, endtask	一般支持, 少数综合器不支持
编译向导: `include, `define, `ifdef, `else, `endif	√

有一些 Verilog 语法结构在综合工具中将被忽略, 如延时信息等。表 10.2 对一般 EDA 综合工具忽略的 Verilog HDL 结构进行了总结。

表 10.2 一般综合工具忽略的 Verilog HDL 结构

Verilog HDL 结构	可综合性说明
延时控制	这些结构和语句在综合时全被忽略
scalared, vectored	
specify	
small, large, medium	
weak1, weak0, highz0, highz1, pull0, pull1	
time	有些综合工具将其视为整数 (integer)
wait	有些综合工具有限制地支持

表 10.3 对多数 EDA 综合工具不支持的 Verilog HDL 结构进行了总结。

表 10.3 一般综合工具不支持的 Verilog HDL 结构

Verilog HDL 结构	可综合性说明
在 assign 持续赋值中, 等式左边含有变量的位选择	多数综合器对这些结构和语句不支持, 用这些语句描述的程序代码不能转化为具体的电路网表结构, 但这些结构都能够被仿真工具 (如 ModelSim 等) 所支持
全等运算符 <code>===</code> , <code>!==</code>	
cmos, nmos, rcmos, rmos, pmos, rmos	
deassign, defparam, event	
force, release	
fork, join	
forever, while, repeat	
initial	
rtran, tran, tranif0, tranif1, rtranif0, rtranif1	
table, endtable, primitive, endprimitive	

10.1.2 可综合设计的要点

在用 Verilog HDL 进行可综合的设计时, 需要特别注意如下一些要点:

- ◆ 不使用初始化语句。
- ◆ 不使用带有延时的描述。
- ◆ 不使用循环次数不确定的循环语句, 如 forever、while 等。
- ◆ 尽量采用同步方式设计电路。
- ◆ 除非是关键路径的设计, 一般不采用调用门级元件来描述设计的方法, 建议采用行为语句来完成设计。
- ◆ 用 always 过程块描述组合逻辑, 应在敏感信号列表中列出所有的输入信号。
- ◆ 所有的内部寄存器都应该能够被复位, 在使用 FPGA 实现设计时, 应尽量使用器件的全局复位端作为系统总的复位。
- ◆ 在 Verilog 模块中, 任务 (task) 通常被综合成组合逻辑的形式; 每个函数 (function) 在调用时通常也被综合为一个独立的组合电路模块。

◆ 用户自定义原语 (UDP 元件) 是不可综合的, 它只能用来建立门级元件的仿真模型。

10.2 流水线设计技术 (Pipeline Design)

流水线设计是用于提高所设计系统运行速度的一种有效的方法。为了保障数据的快速传输, 必须使系统运行在尽可能高的频率上, 但如果某些复杂逻辑功能的完成需要较长的延时, 就会使系统很难运行在高的频率上, 在这种情况下, 可使用流水线技术, 即在长延时的逻辑功能块中插入触发器, 使复杂的逻辑操作分步完成, 减小每个部分的处理延时, 从而使系统的运行频率得以提高。流水线设计的代价是增加了寄存器逻辑, 即增加了芯片资源的耗用。

流水线操作的概念可用图 10.1 来说明, 在图中, 假定某个复杂逻辑功能的实现需要较长的延时, 我们可以将其分解为几个 (如 3 个) 步骤来实现, 每一步的延时变为原来的 $1/3$, 并在各步之间加入寄存器, 以暂存中间结果, 这样的话, 将使整个系统的最高工作频率得到很大的提高。

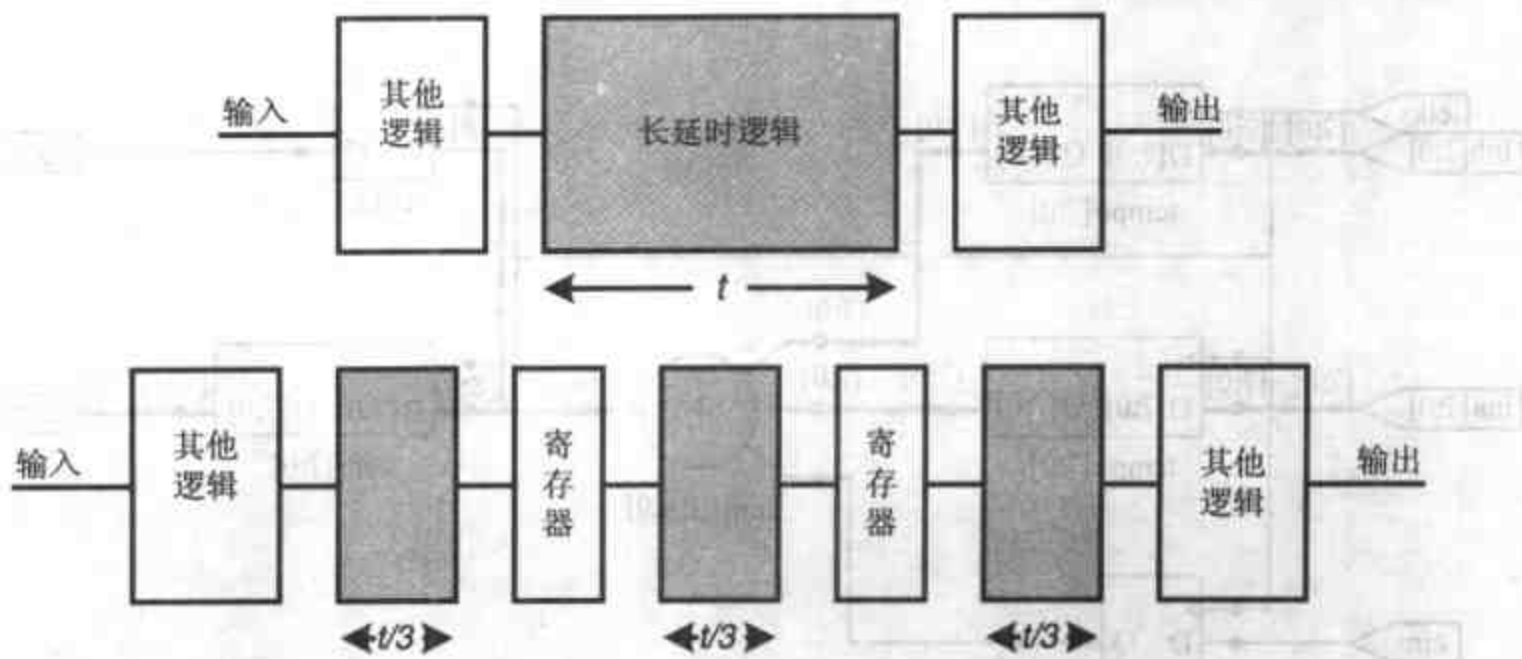


图 10.1 流水线操作的概念示意图

流水线设计技术可以有效提高系统的工作频率, 尤其是对于 FPGA 器件。FPGA 与 CPLD 相比, 单个的逻辑单元规模小得多, 但逻辑单元的个数多, FPGA 中触发器的数量比 CPLD 多。一般 FPGA 的逻辑单元中实现组合逻辑的为 4~5 变量的查找表 (LUT), 上千个触发器, 因此在 FPGA 的设计中采用流水线技术可以有效提高系统的速度。

例如, 下面是 8 位全加器的设计, 一个是直接实现, 一个采用 4 级流水线实现。

1. 非流水线实现方式

下面是非流水线方式实现的 8 位全加器, 其输入、输出端都带有锁存器。

【例 10.1】非流水线方式的 8 位全加器

```
module adder8(cout,sum,ina,inb,cin,clk);
output [7:0] sum;
output cout;
input [7:0] ina,inb;
```

```

input  cin,clk;
reg[7:0]  tempa,tempb,sum;
reg cout;
reg tempc;
always @(posedge clk)
begin
    tempa=ina; tempb=inb; tempc=cin;    //输入数据锁存
end
always @(posedge clk)
begin
    {cout,sum}=tempa+tempb+tempc;
end
endmodule

```

图 10.2 是上例用综合器综合后的 RTL 视图。从综合视图可以清楚地看到，全加器的输入、输出端都带有锁存器。

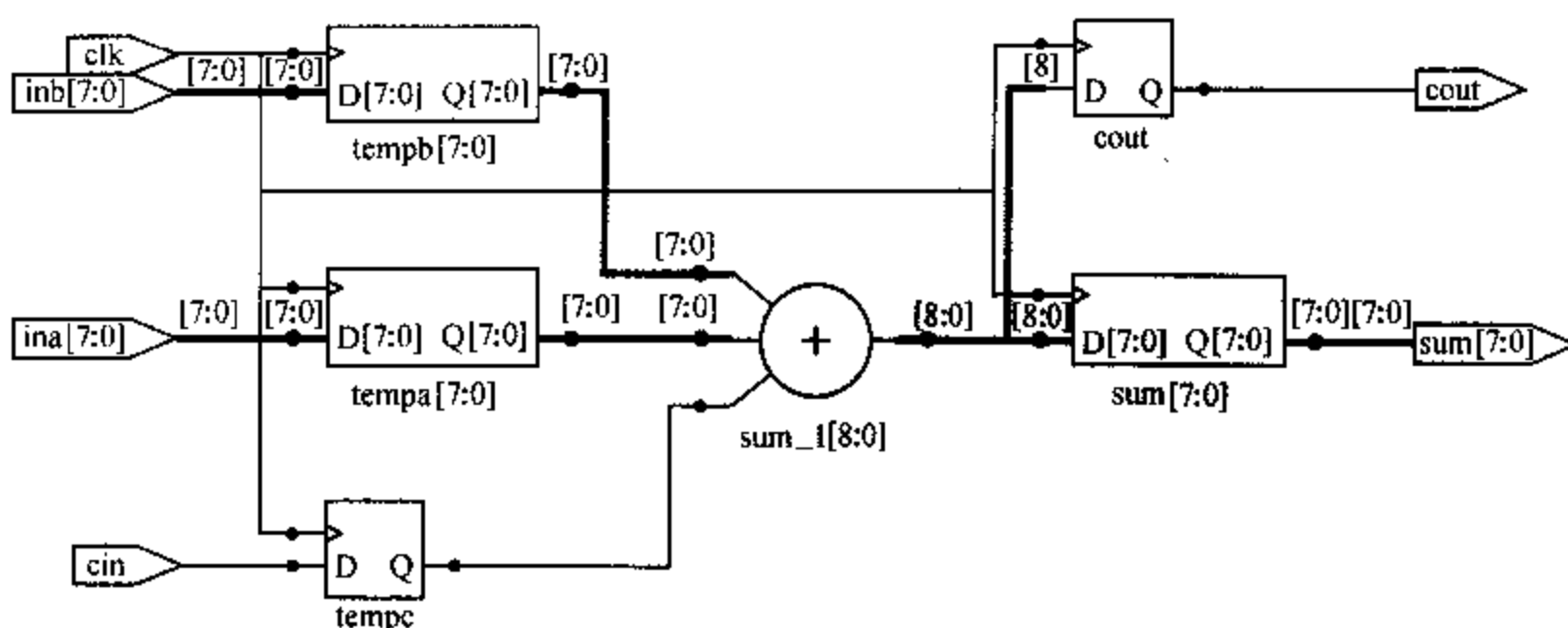


图 10.2 普通 8 位加法器的 RTL 综合视图

2. 采用 4 级流水线方式实现

【例 10.2】4 级流水线方式的 8 位全加器

```

module pipeline(cout,sum,ina,inb,cin,clk);
output[7:0]  sum;
output  cout;
input[7:0]  ina,inb;
input  cin,clk;
reg[7:0]  tempa,tempb,sum;
reg  tempci,firstco,secondco,thirdco, cout;
reg[1:0]  firsts, thirda,thirdb;
reg[3:0]  seconda, secondb, seconds;

```



```

reg[5:0]  firsta, firstb, thirds;

always @(posedge clk)
begin
    tempa=ina; tempb=inb; tempci=cin;          //输入数据缓存
end
always @(posedge clk)
begin
    {firstco,firsts}=tempa[1:0]+tempb[1:0]+tempci;
                                //第一级加（低 2 位）
    firsta=tempa[7:2];          //未参加计算的数据缓存
    firstb=tempb[7:2];
end
always @(posedge clk)
begin
    {secondco,seconds}={firsta[1:0]+firstb[1:0]+firstco,firsts};
                                //第二级加（第 2、3 位相加）
    seconda=firsta[5:2];        //数据缓存
    secondb=firstb[5:2];
end
always @(posedge clk)
begin
    {thirdco,thirds}={seconda[1:0]+secondb[1:0]+secondco,seconds};
                                //第三级加（第 4、5 位相加）
    thirda=seconda[3:2];        //数据缓存
    thirdb=secondb[3:2];
end
always @(posedge clk)
begin
    {cout,sum}={thirda[1:0]+thirdb[1:0]+thirdco,thirds};
                                //第四级加（高两位相加）
end
endmodule

```

图 10.3 是上述 4 级流水线加法器的框图。从该图可以比较清楚地看出，上面的加法器采用了 5 级锁存、4 级加法，每一个加法器实现 2 位数据和一个进位的相加，整个加法器只受 2 位全加器工作速度的限制，平均完成一个加法运算只需一个时钟周期的时间。

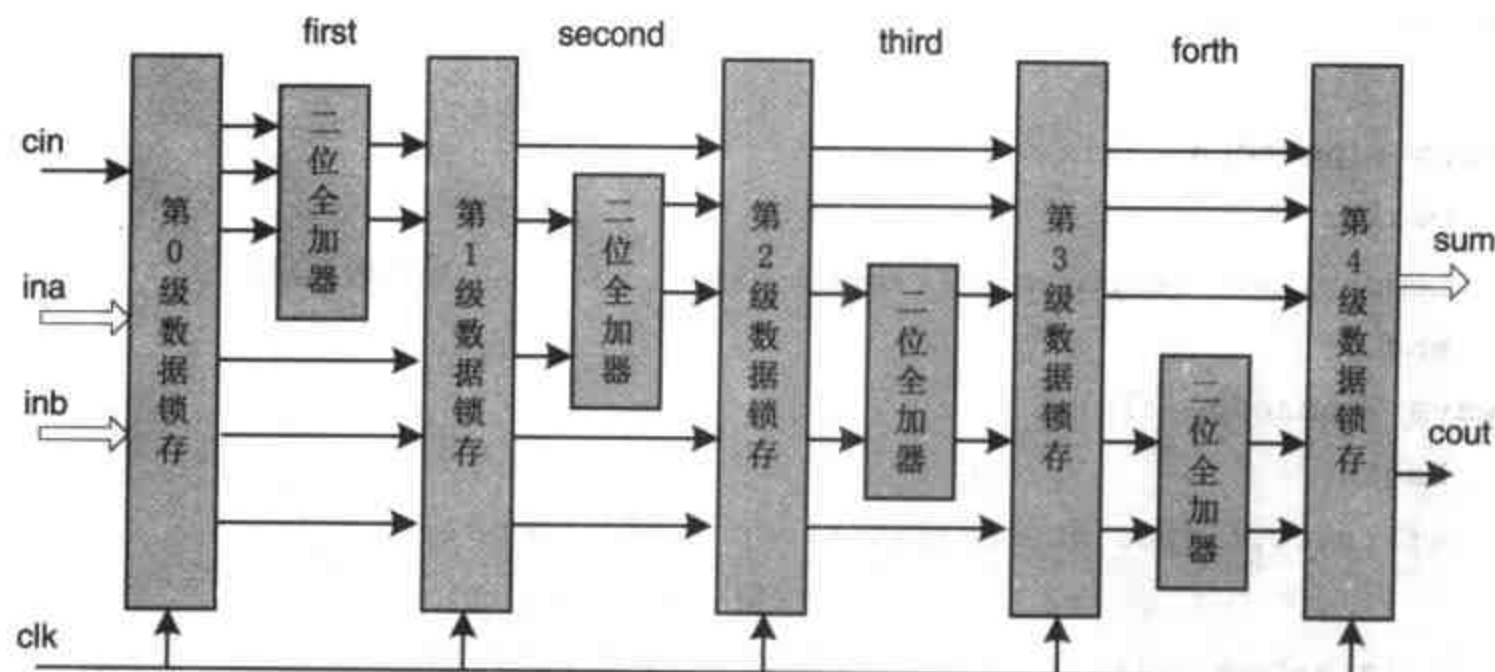


图 10.3 8 位加法器的 4 级流水线实现方框图

将以上基于流水线的设计综合到 Altera 的 EPF10K10-3 器件中, 测试其最大工作频率。具体步骤为: 用 MAX+PLUSII 软件对源程序进行编译, 编译通过后, 选择 Utilities/Analyze Timing 选项, 之后, 在菜单 Anaysis 中选择 Registers Performance; 此时, 出现如图 10.4 所示的 Timing Analyzer 窗口, 按 Start 就开始测量, 可以看到, 最后的测试结果为 99MHz。



图 10.4 流水线设计允许的最大时钟频率

采取上述步骤, 分别将以上非流水线的设计和基于流水线的设计综合到 Altera 的 EPF10K10-3 (FPGA) 和 EPM7128S-6 (CPLD) 器件中, 测试其允许的最高工作频率, 可以得到如表 10.4 所示的比较结果。

从表中可以清楚地看出, 流水线设计有效提高了系统的最高运行频率, 提高了工作速度, 尤其是对于 FPGA 器件, 效果更明显。

表 10.4 两个设计综合到不同器件中的最高工作频率

器 件	非流水线设计 (MHz)	流水线设计 (MHz)
EPF10K10-3 (FPGA)	37.03	99.00
EPM7128S-6 (CPLD)	46.94	62.89

10.3 资源共享 (Resource Sharing)

尽量减少系统所耗用的器件资源也是我们进行电路设计所追求的一个目标。在这方面,资源共享是一个较好的方法,尤其是将一些耗用资源较多的模块进行共享,能有效降低整个系统所占用的芯片面积。

这里是一个有关资源耗用的例子,比如要实现这样的功能:当 $sel=0$ 时, $sum=a+b$; 当 $sel=1$ 时, $sum=c+d$; a 、 b 、 c 、 d 的宽度可变,在本例中定义为 4 位。有两种实现的方式,如下所示。

1. 方式一: 先加后选

这种方式需要两个加法器、一个数据选择器,如图 10.5 所示。

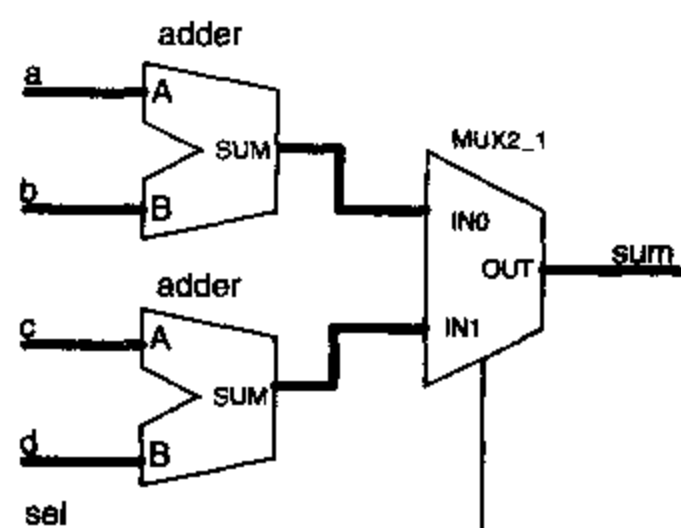


图 10.5 用两个加法器和一个数据选择器实现

该实现方式用 Verilog 语言描述如下。

【例 10.3】两个加法器和一个数据选择器的实现方式

```

module resource1(sum,a,b,c,d,sel);
parameter size=4;
output[size:0] sum;
input sel;
input[size-1:0] a,b,c,d;
reg[size:0] sum;
always @(a or b or c or d or sel)
begin
if(sel)    sum=a+b;

```

```

    else    sum=c+d;
  end
endmodule

```

2. 方式二：先选后加

这种实现方式需要两个数据选择器、一个加法器，如图 10.6 所示。

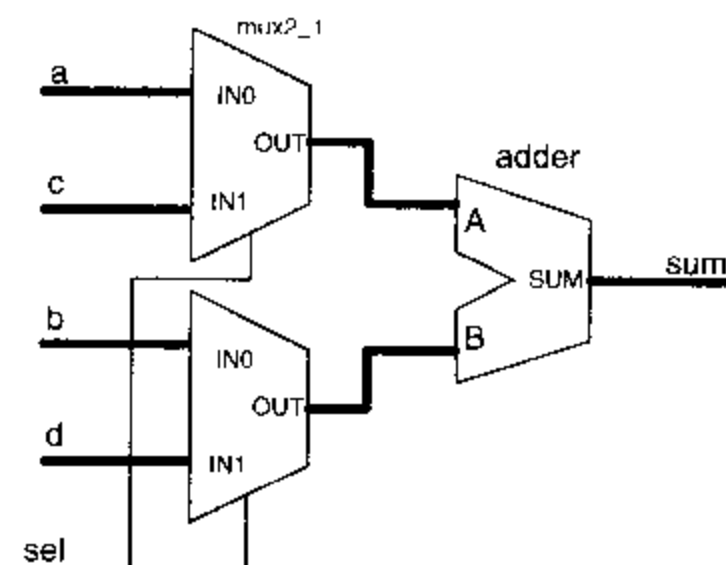


图 10.6 用两个数据选择器和一个加法器实现

用 Verilog 语言描述如下。

【例 10.4】两个数据选择器和一个加法器的实现方式

```

module resource2(sum,a,b,c,d,sel);
parameter size=4;
output[size-1:0] sum;
input sel;
input[size-1:0] a,b,c,d;
reg[size-1:0] atemp,btemp;
reg[size:0] sum;
always @(a or b or c or d or sel)
begin
  if(sel)    begin atemp=a; btemp=b; end
  else      begin atemp=c; btemp=d; end
  sum=atemp+btemp;
end
endmodule

```

将上面两个程序分别综合到 EPF10K10 器件中，编译后打开报告文件 resource1.rpt 和 resource2.rpt，比较器件资源的消耗情况，如下所示。

Chip/PGF	Device	Input Pins	Output Pins	Bidir Pins	Memory Bits	Memory % Utilized	LCs	% Utilized
resource1	EPF10K10LC84-3	17	5	0	0	0 %	18	3 %

Chip/PoF	Device	Input Pins	Output Pins	Bidir Pins	Memory Bits	Memory % Utilized	Memory LCs	Memory % Utilized
resource2	EPF10K10LC84-3	17	5	0	0	0 %	15	2 %

从中可以看出, 方式一耗用了 18 个逻辑单元 (LC), 而方式二则耗用了 15 个 LC, 可见, 方式二更节省资源。将上面设计文件中的 size 的值改变, 更改整个设计的规模, 再比较资源消耗的情况, 得到的结果可见表 10.5。

表 10.5 器件资源消耗的比较

设计规模 (size)	方式一 (resource1)	方式二 (resource2)
4 位	18 个 LC	15 个 LC
8 位	38 个 LC	31 个 LC
10 位	48 个 LC	39 个 LC

在上面的例子中, 方式一需要两个加法器, 而方式二通过增加一个 MUX, 共享一个加法器, 由于加法器耗用的资源比 MUX 多, 因此方式二更节省资源。所以在电路设计中, 应尽可能使硬件代价高的功能模块资源共享, 从而降低整个系统的成本。

在节省资源的设计中应注意以下要点:

- ◆ 尽量共享复杂的运算单元, 可以采用函数和任务来定义这些共享的数据处理模块。
- ◆ 模块数据的宽度应该尽量小, 以能满足设计要求为准。

10.4 有限状态机 (FSM) 设计

有限状态机 (FSM, Finite State Machine) 设计是时序电路设计中经常采用的一种方式, 尤其适合于设计数字系统的控制模块。用 Verilog HDL 的 case、if-else 等语句能很好地描述基于状态机的设计。

10.4.1 基于状态机的设计

状态机可以认为是组合逻辑和寄存器逻辑的特殊组合, 它一般包括两个部分: 组合逻辑部分和寄存器逻辑部分。寄存器用于存储状态, 组合电路用于状态译码和产生输出信号。状态机的下一个状态及输出不仅与输入信号有关, 而且还与寄存器当前所处的状态有关。

根据输出信号产生方法的不同, 状态机可以分为两类: 米里型 (Mealy) 和摩尔型 (Moore)。摩尔型状态机的输出只是当前状态的函数, 而米里型状态机的输出则是当前状态和当前输入的函数。

实用的状态机一般都设计为同步时序方式, 它在时钟信号的触发下完成各个状态之间的转移, 并产生相应的输出。

状态机有 3 种表示方法: 状态图、状态表和流程图, 这 3 种表示方法是等价的, 相互之间可以转换。其中, 状态图是最常用的一种表示方式。米里型状态图的表示如图 10.7 所示, 图中的每个圆圈表示状态机的一个状态, 而每个箭头表示状态之间的一次转移, 引起转换的

输入信号及产生的输出信号标注在箭头上。



图 10.7 米里型状态图的表示

对于有限状态机的设计，一般首先根据所设计电路的功能画出其状态转移图，然后用 Verilog 语言的 case、if-else 等语句对状态机的转换进行描述。比如下面是一个状态机设计的例子，图 10.8 是其状态转移图，共有 4 个状态：state0、state1、state2、state3，每个状态下的输出分别为：out=001，out=010，out=100，out=111，缺省输出为：out=001。

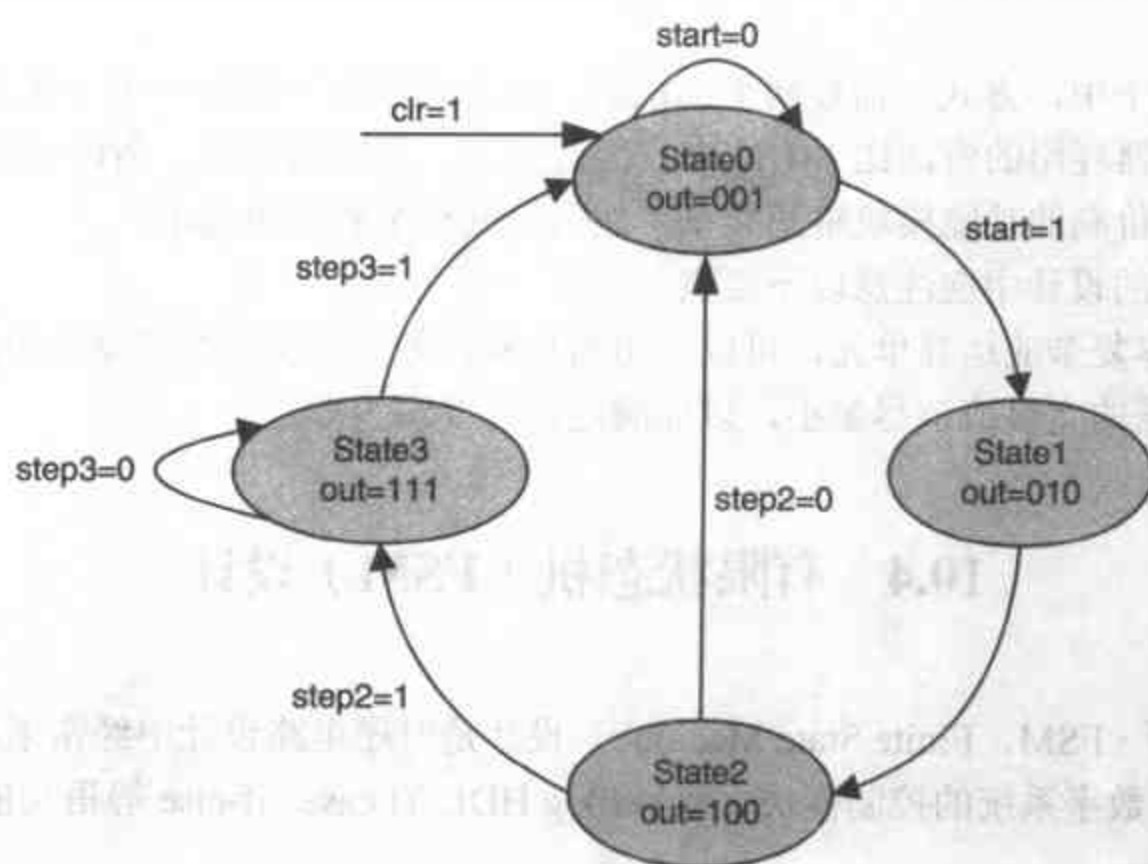


图 10.8 状态转移图

上面的状态转移图用 Verilog 语言描述如下。

【例 10.5】状态机设计的例子

```
module FSM(clk,clr,out,start,step2,step3);
input  clk,clr,start,step2,step3;
output [2:0] out;
reg [2:0] out;
reg [1:0] state, next_state;

parameter state0=2'b00,state1=2'b01,
           state2=2'b11,state3=2'b10;
           /*状态编码，采用格雷 (Gray) 编码方式*/

always @(posedge clk or posedge clr) /*该进程定义起始状态*/
```



```

begin
    if (clr) state <= state0;
    else state <- next_state;
end

always @(state or start or step2 or step3)          /*该进程实现状态的转换*/
begin
    case (state)
        state0: begin
            if (start) next_state<=state1;
            else        next_state <=state0;
            end
        state1: begin
                                next_state <= state2;
            end
        state2: begin
            if (step2) next_state <=state3;
            else        next_state <=state0;
            end
        state3: begin
            if (step3) next_state <=state0;
            else        next_state <=state3;
            end
        default:        next_state <=state0;          /*default 语句*/
    endcase
end

always @(state)                                     /*该进程定义组合逻辑 (FSM 的输出) */
begin
    case (state)
        state0: out=3'b001;
        state1: out=3'b010;
        state2: out=3'b100;
        state3: out=3'b111;
        default:out=3'b001;                          /*default 语句, 避免锁存器的产生*/
    endcase
end

endmodule

```

10.4.2 频率计控制器设计举例

在这里用状态机来设计一个自动转换量程的频率计控制器，用 case 语句进行描述。

图 10.9 是该频率计控制器的状态转移图（摩尔型状态机），该状态转移图用 Verilog 语言描述如下。

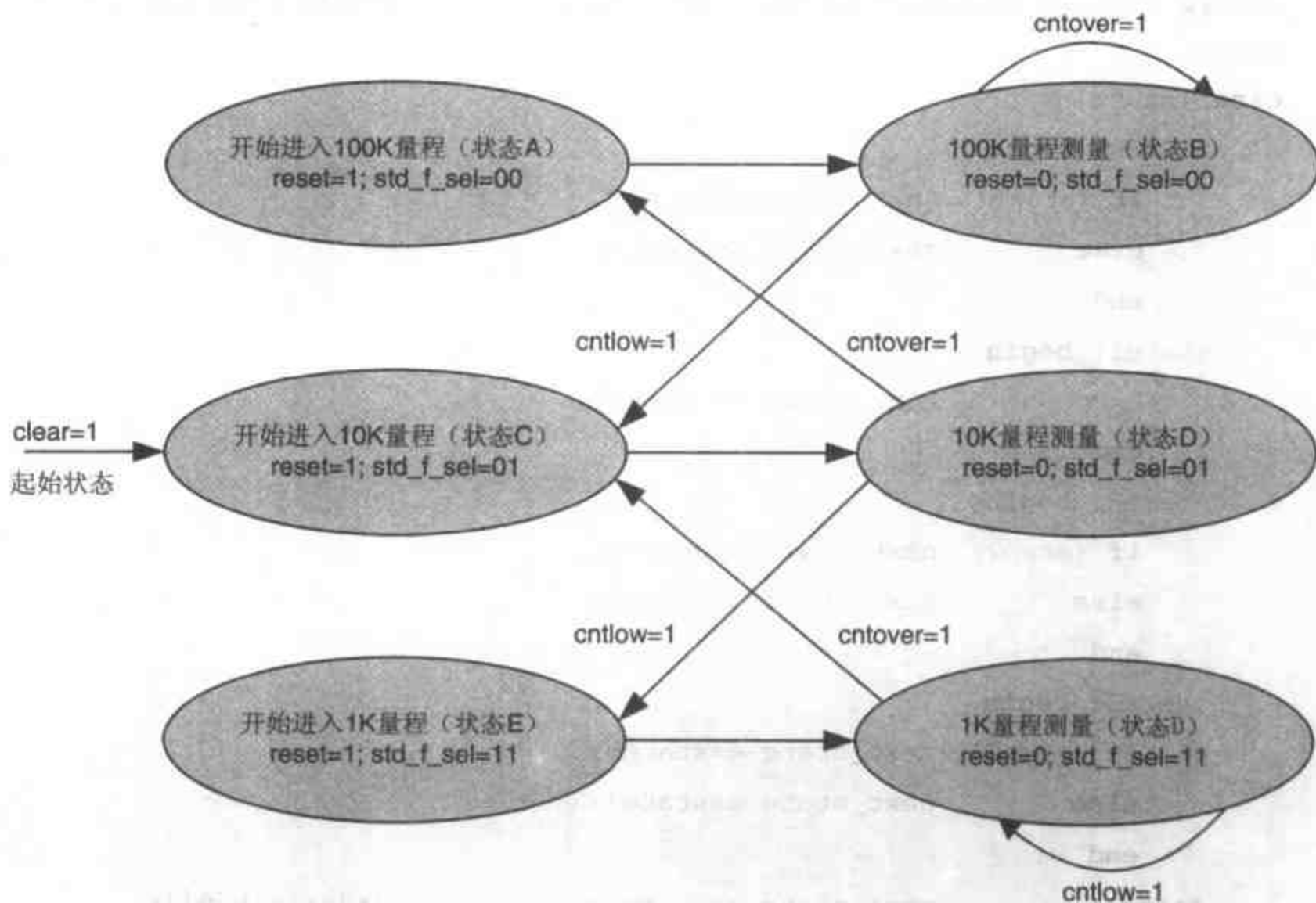


图 10.9 频率计控制器的状态转移图

【例 10.6】自动转换量程频率计控制器

/*信号定义:

clk: 输入时钟;

clear: 为整个频率计的异步复位信号;

reset: 用来在量程转换开始时复位计数器;

std_f_sel: 用来选择标准时基;

cntover: 代表超量程;

cntlow: 代表欠量程。

状态 A, B, C, D, E, F 采用一位热码编码 */

```
module control (std_f_sel, reset, clk, clear, cntover, cntlow);
```

```
output [1:0] std_f_sel;
```

```
output reset;
```

```
input clk, clear, cntover, cntlow;
```



```

reg[1:0] std_f_sel;
reg reset;
reg[5:0] present,next;           //用于保存当前状态和次态的中间变量
parameter start_fl00k=6'b000001, //状态 A 编码, 采用 1 位热码
           fl00k_cnt=6'b000010,   //状态 B
           start_fl10k=6'b000100, //状态 C
           fl10k_cnt=6'b001000,   //状态 D
           start_flk=6'b010000,   //状态 E
           flk_cnt=6'b100000;     //状态 F

always @(posedge clk or posedge clear)
begin
    if(clear) present<=start_fl10k; //start_fl10k 为起始状态
    else present<=next
end

always @(present or cntover or cntlow)
begin
    case(present) //用 case 语句描述状态转换
        start_fl00k: next=fl00k_cnt;
        fl00k_cnt:
            begin
                if(cntlow) next<=start_fl10k;
                else next<=fl00k_cnt;
            end
        start_fl10k: next=fl10k_cnt;
        fl10k_cnt:
            begin
                if(cntlow) next=start_flk;
                else if(cntover) next=start_fl00k;
                else next=fl10k_cnt;
            end
        start_flk: next=flk_cnt;
        flk_cnt:
            begin
                if(cntover) next=start_fl10k;
                else if next=flk_cnt;
            end
        default: next=start_fl10k; //缺省状态为起始状态
    endcase
end

```

```

        endcase
    end

    always @(present) //该进程产生各状态下的输出
    begin
        case (present)
            start_fl00k:    begin    reset=1;  std_f_sel=2'b00;  end
            fl00k_cnt:      begin    reset=0;  std_f_sel=2'b00;  end
            start_fl0k:     begin    reset=1;  std_f_sel=2'b01;  end
            fl0k_cnt:       begin    reset=0;  std_f_sel=2'b01;  end
            start_flk:      begin    reset=1;  std_f_sel=2'b11;  end
            flk_cnt:        begin    reset=0;  std_f_sel=2'b11;  end
            default:        begin    reset=1;  std_f_sel=2'b01;  end
        endcase
    end
endmodule

```

10.4.3 几点讨论

在基于状态机的设计中，有以下几点需要引起注意。

1. 起始状态的选择

起始状态是指电路复位后所处的状态，选择一个合理的起始状态将使整个系统简洁、高效。在例 10.5 中，起始状态是 state0；在例 10.6 中，起始状态是状态 C。有的 EDA 软件会自动为基于状态机的设计选择一个最佳的起始状态。

2. 状态编码

状态编码的方式主要有二进制编码、格雷编码和一位热码编码等。

(1) 二进制编码 (Binary State Machine)

二进制编码采用普通的二进制数代表每个状态。比如：有 4 个状态为 state0、state1、state2、state3，其二进制编码每个状态所对应的码字为 00、01、10、11。二进制编码的缺点是在从一个状态转换到相邻状态时，有可能有多个比特位发生变化，比如：01→10 有两个比特发生变化，瞬变次数多，容易产生毛刺，引起逻辑错误。

(2) 格雷编码 (Gray Code State Machine)

比如，在例 10.5 中，state0、state1、state2、state3 四个状态编码为 00、01、11、10 即为格雷编码方式。格雷码节省逻辑单元，而且在状态的顺序转换中 (state0→state1→state2→state3→state0...)，相邻状态每次只有一个比特位产生变化，如 01→11，这样就减少了瞬变的次数，也减少了产生毛刺和一些暂态的可能。

(3) 一位热码编码 (One-Hot State Machine Encoding)

一位热码编码即采用 n 位 (或 n 个触发器) 来编码具有 n 个状态的状态机。比如对于 state0、state1、state2、state3 四个状态，可用 4 位二进制来编码：1000、0100、0010、0001。在例 10.6 中，有 A、B、C、D、E、F 共 6 个状态，若用普通二进制编码只需 3 位即可实现状态编码，

但用一位热码编码则需用6位进行编码,分别为6'b000001、6'b000010、6'b000100、6'b001000、6'b010000、6'b100000。

表10.6是对8个状态分别用上述3种编码方式进行编码的对比。

采用一位热码编码,虽然多用了触发器,但可以有效节省和简化组合电路。对于寄存器数量多、而门逻辑相对缺乏的FPGA器件来说,采用一位热码编码可以有效提高电路的速度和可靠性,也有利于提高器件资源的利用率。因此,对于FPGA器件,建议采用该编码方式。

表 10.6 3 种编码方式的对比

状 态	二进制编码	格 雷 编 码	一 位 热 码
state0	000	000	00000001
state1	001	001	00000010
state2	010	011	00000100
state3	011	010	00001000
state4	100	110	00010000
state5	101	111	00100000
state6	110	101	01000000
state7	111	100	10000000

有的编译软件有一位热码编码选项,用户选择了该选项后,编译器能够自动为用户设计项目中的状态机选择该种编码方式。如在MAX+PLUS II软件中,选择菜单Assign/Global Project Logic Synthesis,出现逻辑综合设置对话框,在该对话框中,选中One-Hot State Machine Encoding选项,即为你的设计选择了一位热码编码。

采用一位热码编码后,会出现一些多余的状态,即一些无效的状态。因此,如果用case语句来描述的话,需要增加default分支项,以便在这些状态下能自动回到起始状态。

3. 状态编码的定义

有两种方式可用于定义状态编码,分别用parameter和'define语句。比如要为state0、state1、state2、state3四个状态分别定义码字为00、01、11、10,可采用下面两种方式。

方式1: 用parameter参数定义

```
parameter state0 = 2'b00, state1 = 2'b01,
           state2 = 2'b11, state3 = 2'b10;
```

.....

```
case(state)
```

```
    state0: ...;           //调用
```

```
    state1; ...;
```

.....

方式2: 用'define语句定义

```
'define state0 2'b00      //不要加分号“;”
```

```
'define state1 2'b01
```

```
'define state2 2'b11
```

```

`define state3 2'b10
.....
case(state)
    'state0: ...;           //调用, 不要漏掉符号“'”
    'state1: ...;
    .....

```

要注意两种方式定义与调用时的区别, 一般情况下, 更倾向于采用方式 1 来定义状态编码。在例 10.5 和例 10.6 中, 都采用了方式 1 来定义状态编码。

4. 状态转换的描述

一般使用 case、casex 和 casez 语句来描述状态之间的转换, 用 case 语句表述比用 if-else 语句更清晰明了。此外, 在 case 语句的最后, 不要忘了加上 default 分支语句, 以避免锁存器的产生。

状态机一般都应设计为同步方式, 并由一个时钟信号来触发。实用的状态机都应该设计为由唯一时钟边沿触发的同步运行方式。

10.5 多层次结构电路的设计

有的数字系统比较复杂, 可采用“Top-down”的方法进行设计。首先把系统分为几个模块, 每个模块再分为几个子模块, 以此类推, 直到易于实现为止。

这种“Top-down”的方法能够把复杂的设计分解为许多简单的设计来实现, 同时也适合于多人进行合作开发, 如同用 C 语言编写大型软件一样。Verilog HDL 能够很好地支持这种“Top-down”的设计方法。

本节通过示例介绍多层次结构电路的设计方法。多层次结构电路的描述既可以采用文本方式, 也可以用图形和文本混合设计的方式。用一个简单的 8 位累加器 (ACC) 的设计为例来说明这两种设计方式。

10.5.1 图形与文本混合设计

设计一个 8 位累加器 ACC, 用于对输入的 8 位数据进行累加。我们把累加器分为两个模块, 一个是 8 位全加器, 一个是 8 位寄存器。全加器负责对不断输入的数据和进位进行累加, 寄存器负责暂存累加和, 并把累加和输出并反馈到全加器的输入端, 以进行下一次的累加。划分好模块后, 再把每个模块的端口和连接关系设计好, 就可以设计各个功能模块。

下面首先设计全加器和寄存器。

1. 8 位全加器

【例 10.7】8 位全加器

```

module add8(sum,cout,b,a,cin);
    output[7:0] sum;
    output cout;
    input[7:0] a,b;

```

```

input cin;
    assign {cout,sum}=a+b+cin;
endmodule

```

2. 8 位寄存器

【例 10.8】8 位寄存器

```

module reg8(qout,in,clk,clear);
output[7:0] qout;
input[7:0] in;
input clk,clear;
reg[7:0] qout;
always @(posedge clk or posedge clear)
begin
    if(clear)    qout=0;        //异步清 0
    else        qout=in;
end
endmodule

```

3. 顶层连接

对 8 位全加器和 8 位寄存器模块用 MAX+PLUS II 软件分别进行编译,生成两个模块符号,再把两个模块连接,加上输入、输出端,就构成了完整的累加器电路,如图 10.10 所示。顶层连接用图形方式设计具有端口连接清晰、直观的优点。

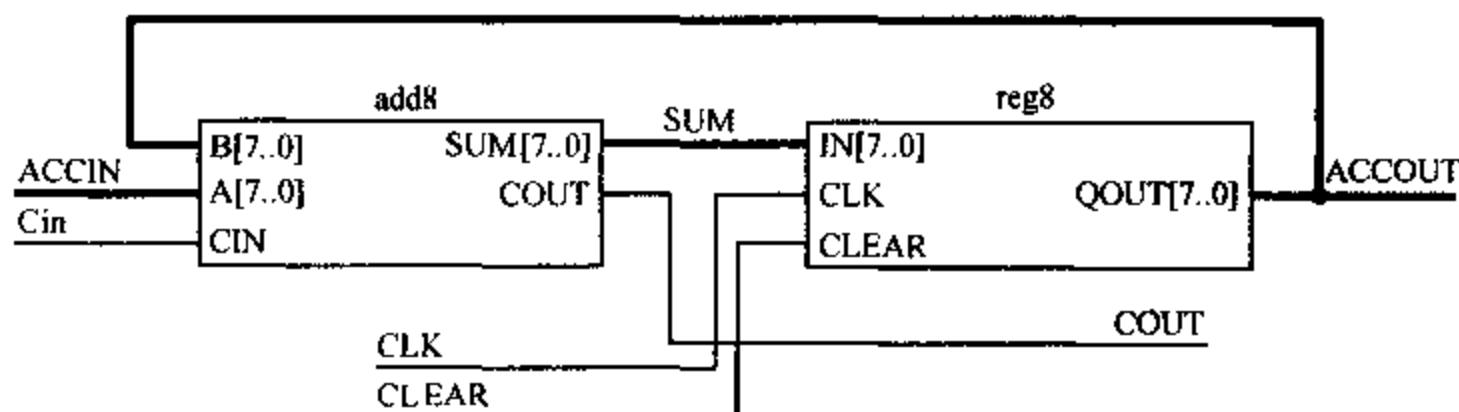


图 10.10 累加器顶层电路

10.5.2 文本设计

对于顶层电路的连接关系,也可以用文本方式描述,这样就实现了完全文本方式的设计,如下所示。

【例 10.9】累加器顶层连接文本描述

```

module acc(accout,cout,accin,cin,clk,clear);
output[7:0] accout;
output cout;
input[7:0] accin;
input cin,clk,clear;

```

```

wire [7:0] sum;

add8 accadd8(sum,cout, accout,accin, cin);           //调用 add8 子模块
reg8 accreg8(accout,sum,clk,clear);                 //调用 reg8 子模块

endmodule

```

该文本描述与上面图形连接的效果完全相同。在调用 add8 和 reg8 两个模块时, 需注意模块端口信号的一一对应。图 10.11 是该 8 位累加器的 RTL 综合结果。

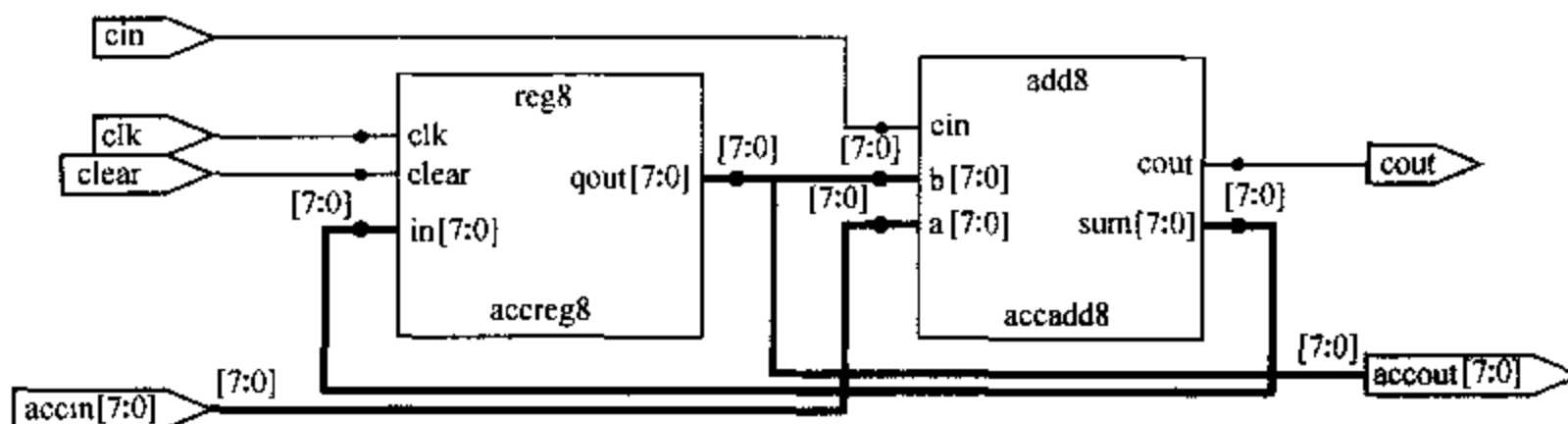


图 10.11 8 位累加器的 RTL 综合结果

还有一点就是, 如何让综合器知道此处调用的 add8 和 reg8 就是前面所编写的模块呢? 这和所用的综合器有关。一般有以下几种指定的方式。

1. 文件复制方式

将 add8 和 reg8 的代码复制到 acc.v 中, 在综合时指明顶层模块。在 MAX+PLUS II 软件中, 文件名对应的模块就是顶层模块。

2. 使用`include 语句

用`include 语句将低层文件 add8.v 和 reg8.v 包含到顶层文件 acc.v 中, 方式同 C 语言中的 #include, 其格式如下。

【例 10.10】用`include 描述的累加器

```

`include "add8.v"
`include "reg8.v"

module accn(accout,cout,accin,cin,clk,clear);
output [7:0] accout;
output cout;
input [7:0] accin;
input cin,clk,clear;
wire [7:0] sum;

add8 accadd8(sum,cout, accout,accin, cin);           //调用 add8 子模块
reg8 accreg8(accout,sum,clk,clear);                 //调用 reg8 子模块
endmodule

```


对于上面的模块调用,可采用位置对应的方式,即调用时模块端口列表中信号的排列顺序与模块定义时端口列表中的信号排列顺序相同;也可以采用信号名对应方式,此时不必按顺序,例如上面对 reg8 的调用:

```
module reg8(qout, in, clk, clear);           //reg8 的模块声明
reg8 accreg8(accout, sum, clk, clear);      //调用方式 1, 位置对应
reg8 accreg8(.qout(accout), .clear(clear), .in(sum), .clk(clk));
                                           //调用方式 2, 信号名对应
```

3. 库管理方式

在有的综合器中提供了库管理功能,由用户指明所调用的低层文件所在的目录即可。

比如 MAX+PLUS II 软件,即提供了库管理功能。假设文件 add8.v 和 reg8.v 都在目录 d:\verilog\acc 下,只要在综合 acc.v 项目文件时选择菜单 options/User libraries, 打开如图 10.12 所示的用户库管理对话框,将目录 d:\verilog\acc 加到“Existing Directories...”中即可,这样在综合 acc.v 时会自动调用 add8.*和 reg8.* 等文件。

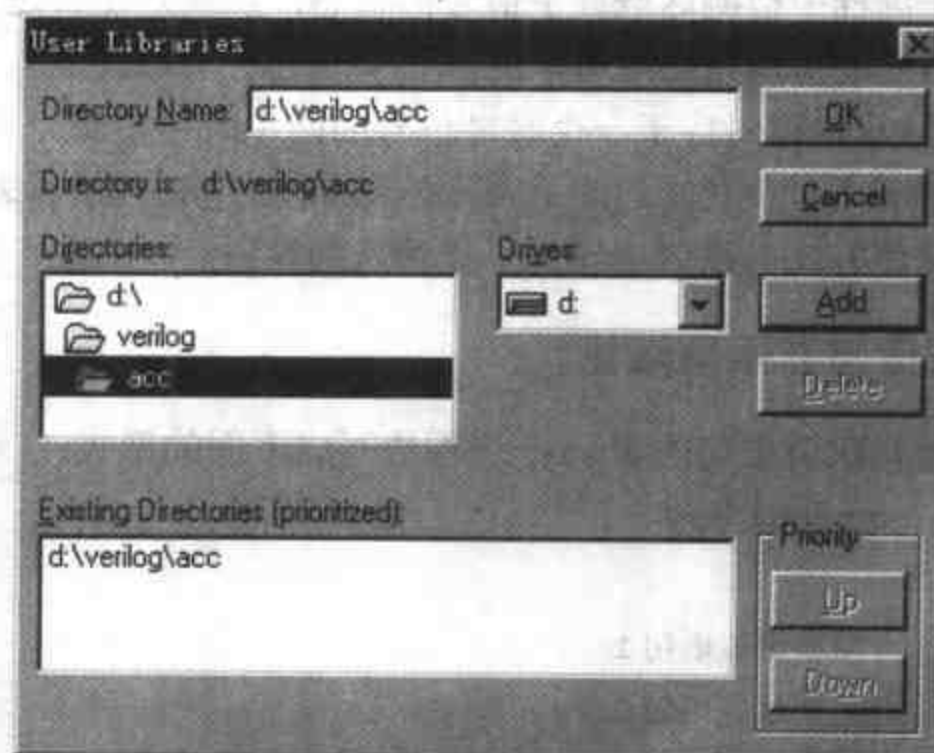


图 10.12 MAX+PLUS II 软件的库管理

10.6 阻塞与非阻塞赋值使用要点

阻塞与非阻塞赋值是 Verilog 语言的难点之一,在使用中也是容易出错的地方,在程序设计中,必须对阻塞与非阻塞赋值有深入的理解。

阻塞与非阻塞赋值一般使用在进程中,包括 always 和 initial 进程、assign 赋值等操作中,因此本节首先介绍进程的设计。

10.6.1 进程的设计

有关进程的概念前面已经介绍过。一个进程可以被看作是一个独立的运行单元,它可以很简单,也可以很复杂,可以将数字系统的行为看作是很多有机结合的进程的集合,各进程

间通过信号线彼此协调。

在 Verilog HDL 中, 描述进程的基本语句是 `always` 和 `initial`。`always` 过程反复执行其中的块语句, 而 `initial` 过程语句只执行一次。此外, 一个 `assign` 赋值语句、一个实例元件的调用也都是一个独立的进程。

进程只有两种状态, 即执行状态和等待状态, 一旦满足特定的条件 (如敏感变量发生变化), 进程即进入执行状态; 执行完毕或遇到停止语句后, 即停止执行, 自动返回到起始语句, 进入等待状态。

在进行数字系统设计的时候, 应注意以下几点:

- ◆ 将硬件电路的行为以合理的方式映射为一些进程, 对每个进程, 以最合理的方式描述并实现。
 - ◆ 将组合逻辑实现的电路和用时序逻辑实现的电路尽量分配到不同的进程中。
 - ◆ 进程中输出的信号在各种可能的情况下都应该被赋值, 且赋值号右端不再出现该信号。
 - ◆ 多个进程之间通过信号线进行通信, 在设计中, 为了达到多个进程协调运行, 可以设置一些握手信号, 在进程中检测这些握手信号的状态, 以决定是否进行必要的操作。在有的设计中, 这种握手信号的协调是必不可少的。
 - ◆ 一个进程中一般只描述对应于一个时钟信号的同步时序逻辑。
 - ◆ 进程必须由敏感信号的变化来启动, 因此必须精心选择进程敏感表达式中的敏感变量。
- 下面对进程中的赋值方式与赋值语句进行更深入的探讨。

10.6.2 阻塞赋值与非阻塞赋值

我们可以将采用阻塞赋值语句的 `always` 进程块写成下面的形式:

```
always @(event-expression)
begin
    <LHS1=RHS1>    //阻塞赋值语句 1
    <LHS2=RHS2>    //阻塞赋值语句 2
    .....
end
```

同样可将采用非阻塞赋值方式的 `always` 进程块写成下面的形式:

```
always @(event-expression)
begin
    <LHS1 <= RHS1>    //非阻塞赋值语句 1
    <LHS2 <= RHS2>    //非阻塞赋值语句 2
    .....
end
```

其中的 `<LHS=RHS assignments>` 和 `<LHS<=RHS assignments>` 表示赋值语句, LHS (Left-Hand Side) 指赋值符号左边的变量或表达式, RHS (Right-Hand Side) 指赋值符号右端的变量或表达式。

阻塞赋值 “=” 与非阻塞赋值 “<=” 的本质区别在于: 非阻塞赋值语句右端表达式计算完后并不立即赋给左端, 而是同时启动下一条语句继续执行, 我们可以将其理解为所有的右

端表达式 RHS1、RHS2 等在进程开始时同时计算，计算完后，等进程结束时同时分别赋给左端变量 LHS1、LHS2 等。

而阻塞赋值语句在每个右端表达式计算完后立即赋给左端变量，即赋值语句 LHS1=RHS1 执行完后 LHS1 是立即更新的，同时只有 LHS1=RHS1 执行完后才可执行语句 LHS2=RHS2，依次类推。前一条语句的执行结果直接影响到后面语句的执行结果。

非阻塞赋值不能用于“assign”持续赋值中，而只能用于对寄存器型变量进行赋值，只能用在“initial”和“always”等过程块中。阻塞赋值则既能用于“assign”持续赋值，也能用于“initial”和“always”等过程赋值中。

对于时序逻辑描述和建模，应尽量使用非阻塞赋值方式。关于这一点，我们通过一个移位寄存器的设计具体进行说明。

在下面的例子中，用阻塞赋值的方式描述了一个移位寄存器。

【例 10.11】阻塞赋值方式描述的移位寄存器 1

```
module block1(Q0,Q1,Q2,Q3,din,clk);
output Q0,Q1,Q2,Q3;
input clk,din;
reg Q0,Q1,Q2,Q3;
always @(posedge clk)
begin
    Q3=Q2;           //注意赋值语句的顺序
    Q2=Q1;
    Q1=Q0;
    Q0=din;
end
endmodule
```

该例的 RTL 综合结果如图 10.13 所示。

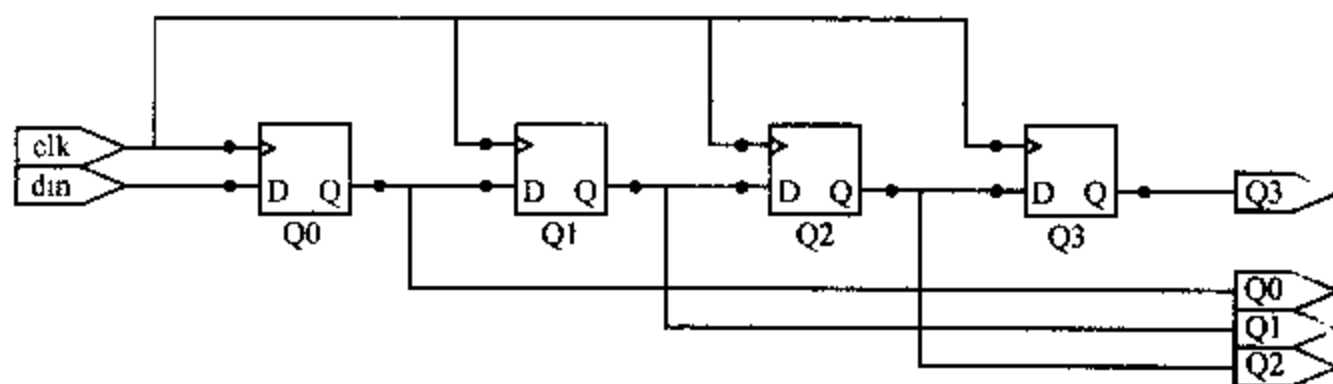


图 10.13 例 10.11 的 RTL 综合结果

而如果把上面例子中的阻塞赋值语句保持不变，仅仅将其两条语句的排列顺序改一下的话，则综合器综合的结果就会大不相同。

【例 10.12】阻塞赋值方式描述的移位寄存器 2

```
module block2(Q0,Q1,Q2,Q3,din,clk);
output Q0,Q1,Q2,Q3;
```

```

input  clk,din;
reg  Q0,Q1,Q2,Q3;
always @(posedge clk)
begin
  Q3=Q2;
  Q1=Q0;           //该句与下句的顺序与例 10.11 颠倒
  Q2=Q1;
  Q0=din;
end
endmodule

```

该例的 RTL 综合结果如图 10.14 所示。

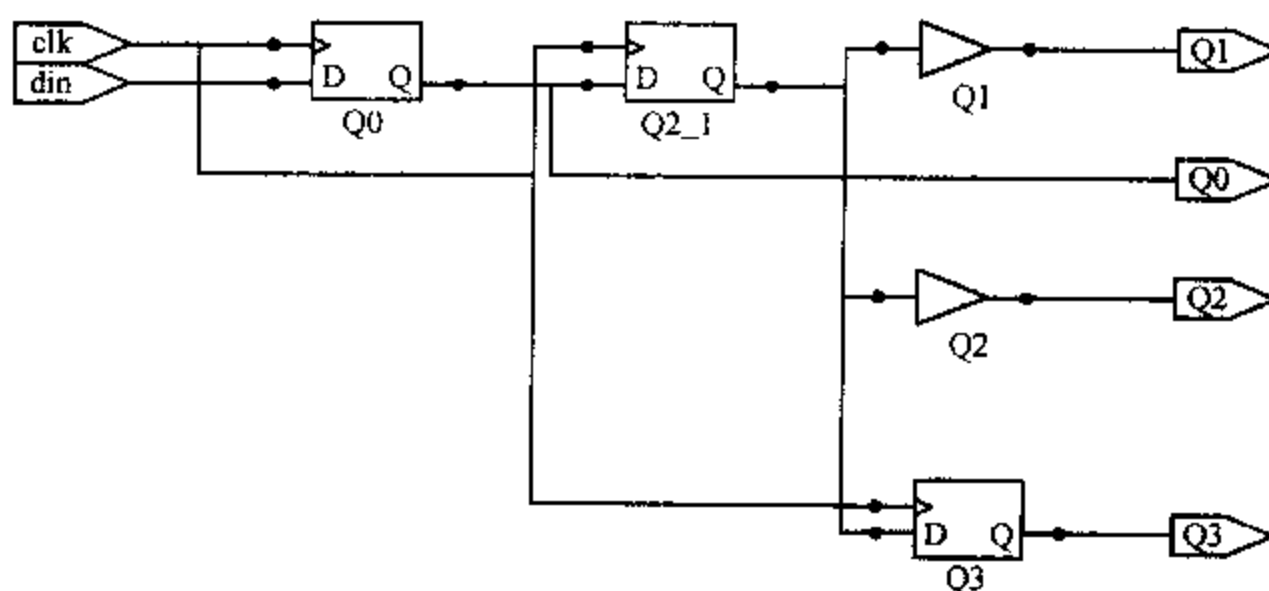


图 10.14 例 10.12 的 RTL 综合结果

如果将 4 条阻塞赋值语句的顺序完全颠倒的话，如例 10.13 中所示，则综合器综合的结果就变成图 10.15 中的效果，可以看出，整个描述实际上等效成一个触发器。

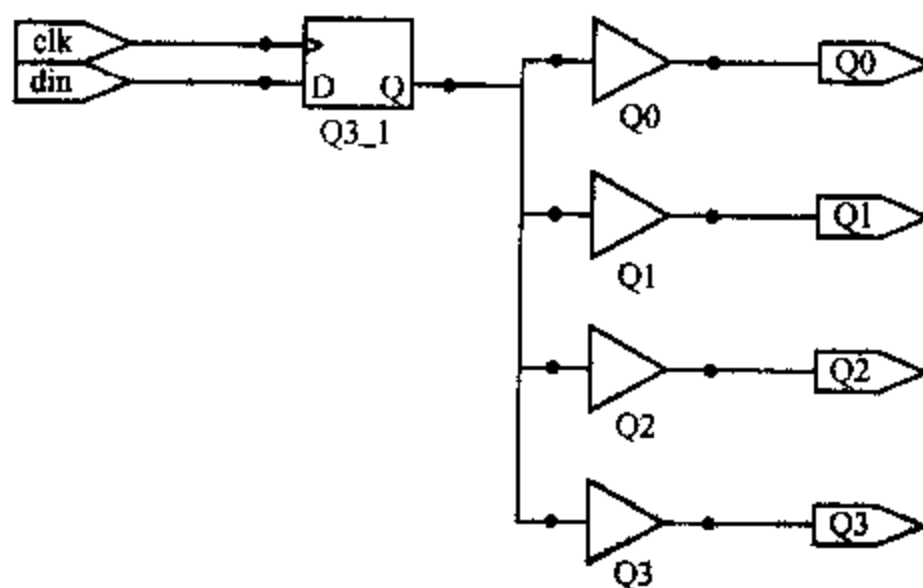


图 10.15 例 10.13 的 RTL 综合结果

【例 10.13】阻塞赋值方式描述的移位寄存器 3

```

module block3(Q0,Q1,Q2,Q3,din,clk);

```

```

output  Q0,Q1,Q2,Q3;
input   clk,din;
reg     Q0,Q1,Q2,Q3;
always @(posedge clk)
    begin
        Q0=din;           //4 条赋值语句的顺序与例 10.11 完全颠倒
        Q1=Q0;
        Q2=Q1;
        Q3=Q2;
    end
endmodule

```

由此可见，对于阻塞赋值来说，赋值语句的顺序对最后的综合结果有着直接的影响。

而如果采用非阻塞赋值方式来描述的话，则可以不考虑赋值语句的排列顺序，只需将其连接关系描述清楚即可。

【例 10.14】非阻塞赋值方式描述的移位寄存器

```

module block4(Q0,Q1,Q2,Q3,din,clk);
output  Q0,Q1,Q2,Q3;
input   clk,din;
reg     Q0,Q1,Q2,Q3;
always @(posedge clk)
    begin
        Q3<=Q2;
        Q1<=Q0;
        Q2<=Q1;
        Q0<=din;
    end
endmodule

```

对于上面的例子，无论将其“always”过程块中 4 条赋值语句的顺序怎样变动，均不会影响其综合结果，其综合结果与例 10.11 相同。所以对于时序逻辑描述和建模，应尽量使用非阻塞赋值方式。此外，若在同一个人“always”过程块中描述时序和组合逻辑混合电路时，也最好使用非阻塞赋值方式。

10.6.3 应用要点

在可综合的程序设计中，在使用阻塞和非阻塞赋值语句方面，应注意以下一些原则：

- ◆ 非阻塞赋值不能用于“assign”持续赋值中，一般只出现在“initial”和“always”等过程块中，对 reg 型变量进行赋值。像 assign out<=a+b; 这样的语句是错误的。

- ◆ 当用“always”块来描述组合逻辑时，既可以用阻塞赋值，也可以采用非阻塞赋值。但在同一个过程块中，最好不要同时用阻塞赋值和非阻塞赋值，虽然同时使用这两种赋值方式在综合时并不一定会出错。

- ◆ 在向函数 (function) 的返回值赋值时, 应使用阻塞赋值 “=”。
- ◆ 不能在一个以上的 “always” 过程块中对同一个变量赋值, 这样会引起冲突, 在综合时会报错。
- ◆ 在一个模块中, 严禁对同一个变量既进行阻塞赋值, 又进行非阻塞赋值, 这样在综合时会报错。
- ◆ 对时序逻辑描述和建模, 应尽量使用非阻塞赋值方式, 此外, 若在同一 “always” 过程块中描述时序和组合逻辑混合电路时, 也最好使用非阻塞赋值方式。

10.7 片内存储器的使用

在进行数字信号处理 (DSP)、数据加密或数据压缩等复杂数字逻辑设计时, 不可避免地要用到存储器。将存储模块 (RAM 或 FIFO 等) 嵌入 PLD 芯片, 不仅可简化设计, 提高设计的灵活性, 同时也降低了数据存储的成本, 使芯片内、外数据的交换更可靠。目前很多 FPGA 器件都集成了片内 RAM。这种片内 RAM 的速度快, 读操作的时间可以达到 3~4ns, 写操作的时间大约为 5ns, 甚至更短。

10.7.1 片内存储器

FPGA 的片内 RAM 可以分为两类: 一类是块 RAM, 一类是分布式 RAM。

Xilinx 的 Spartan/XL、XC4000 器件中的片内 RAM 是一种分布式 RAM, 它主要由排列成分布式阵列的各个可配置逻辑块 (CLB) 组成, CLB 中的函数发生器可以被定义为 RAM 使用, 根据需要, 可以将 CLB 配置成 16×2 位或是 32×1 位的读/写存储器, 这时, 函数发生器的 F1~F4 和 G1~G4 作为存储器的地址线, CLB 的其他控制脚也被重新定义。

分布式片内 RAM 可用来实现很多功能, 如累加器、状态寄存器、变址寄存器、DMA 计数器、LIFO 堆栈和 FIFO 缓冲器等。

Xilinx 新一代 FPGA 器件 Virtex、Virtex-E 芯片内既集成了许多分布式 RAM, 也有大量的块式 RAM, 以满足 DSP、视频处理等设计中对 RAM 的需要。以视频处理为例, 对按字节存储的视频行数据, 可以存储在块 RAM 中; 对按位存储的像素和系数等数据, 则可存储在分布式 RAM 中, 以实现快速而灵活的数据存取。

Altera 的 APEX 20K/20KE、FLEX 10K、ACEX 1K 等器件内部的 RAM 称为嵌入式系统块 (ESB)。ESB 可以完成乘法功能, 也能够实现各种类型的存储器, 如 RAM、ROM、FIFO 和 CAM 等。ESB 含有输入、输出寄存器。输入寄存器支持同步写操作, 输出寄存器支持流水线设计, 以提高系统的速度。ESB 还提供双端口方式, 该方式支持在两个不同的时钟频率下同时对存储体进行读写操作。此外, ESB 还可以用作同步、异步 RAM。

如图 10.16 所示是用 ESB 实现 CAM 的示意图。CAM 可以认为是逆 RAM。在读操作时, RAM 根据一个给定地址输出数据; 相反的, CAM 可以根据一个给定的数据字输出地址。CAM 可用于高速查表的应用系统中, 例如网络、通信、数据压缩和缓存管理等。在一个 RAM 块中, 按串行方式读取数据。这样, 查找一个特定的字需要多个时钟周期。CAM 并行地搜索所有的地址, 并输出存储这个特定字的地址。当找到匹配项后, 匹配标志位置高。

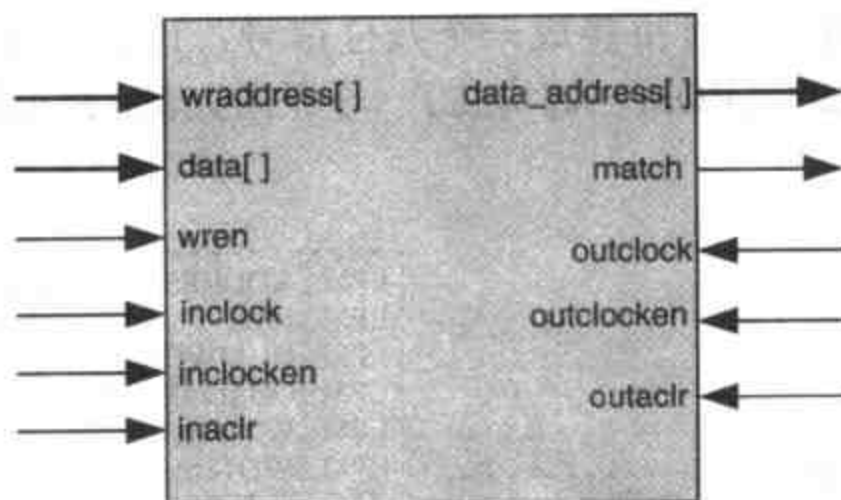


图 10.16 用 ESB 实现 CAM

APEX 20KE 的 CAM 提供了比传统分立 CAM 更快的系统性能。集成的 CAM 和逻辑结构消除了片间的延时，提高了性能。在 CAM 模式下，一个 ESB 可以实现 32 位 32 字的 CAM。更宽、更深的 CAM 可以通过组合多个 CAM 和用 LE 实现的辅助逻辑来实现。Quartus II 设计软件可以自动地组合 ESB 和 LE 生成更大的 CAM。

ESB 还可以在只读存储器 (ROM) 方式下通过生成大的查找表 (LUT) 来实现逻辑功能。使用 LUT 实现组合逻辑功能比一般算法更快。ESB 的时间快速通道进一步提高了这个性能。ESB 的强大功能允许设计者可以在同级逻辑中实现复杂功能，而不需要因连接 LE 和分散的 RAM 块而产生布线延时。参数化逻辑功能块，如 LPM 功能块会自动使用 ESB 的优点。另外，Quartus II 软件可以在适用 ESB 的地方也会自动使用 ESB 实现设计。

EAB 单元的灵活性还体现在以下方面：

- ◆ 位数配置的灵活性：EAB 可以根据设计要求配置成多种字宽，数据线和地址线的宽度也可以灵活变化。
- ◆ 级连使用：如果需要使用更宽的数据宽度，MAX+PLUS II 软件可以通过自动级连更多的 EAB 来完成单个 EAB 无法完成的功能。
- ◆ 深度扩展：每个 EAB 最多具有 11 位地址线，其 RAM 区深度为 2048 个字。但当需要 4096 个字或更多时，MAX+PLUS II 会自动选用 EAB 单元的数量，并产生相应的片选信号。

10.7.2 设计举例

下面用一个乘法器的例子来说明 EAB 的使用方法，这个例子采用 FLEX10K10 器件中的 EAB 构成一个 4 位×4 位的乘法器，利用查表方法完成乘法功能。

设计采用 MUX+PLUS II 软件中的 lpm 存储器模块来完成。首先在图形编辑器中双击空白处，在 lpm 库中选择符号 lpm_rom。选择 lpm_rom 器件后，接着出现参数设置对话框。

视需要并参考在线帮助设置对话框中的参数和端口。由于此处仅将其作为 ROM 使用，这样只使用两个端口：地址输入端口 address[] 和数据输出端口 q[]。

地址端口宽度设为 8，参数设置为 LPM_WIDTHAD=8，LPM_NUMWORDS=256，数据输出端口宽度也为 8 位，即 LPM_WIDTH=8。此外须指明初始化 RAM 所用的数据文件，此处采用 .mif 格式，输入数据文件名 LPM_FILE=“mul4_rom.mif”，注意不要漏掉双引号。这样，ROM 即成为用户定制的存储器。

由于查找表中乘数与被乘数作为地址的高位和低位来处理，为清楚起见，增加一个模块

deput, 该模块将两个四位信号拼接成一个八位信号, 比如 $a=1000$, $b=0110$, 则输出 $o=1000,0110$ 。该模块与 LPM_ROM 一起构成了一个采用查表方式实现的 4×4 乘法器, 如图 10.17 所示。

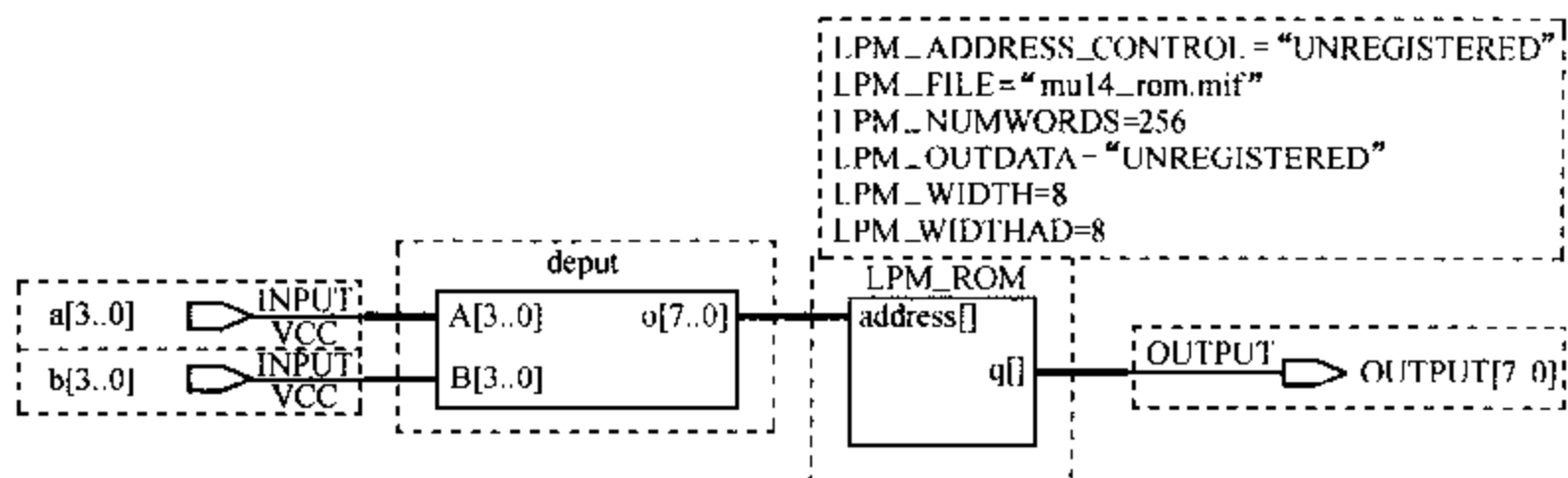


图 10.17 利用 LPM_ROM 构成 4×4 乘法器

mul4_rom.mif 文件用来初始化 LPM_ROM 的数据文件, 它实际上是在 EAB 中存入了一个乘法器的真值表, 用查表的方式完成 4×4 乘法运算。mul4_rom.mif 文件可以用任何不带格式的文本编辑器来编辑, 其内容如下:

```
-----MAX+PLUS II ---generated Memory Initialization File
WIDTH =8;
DEPTH =256;
ADDRESS_RADIX = DEC;
DATA_RADIX = DEC;
CONTENT BEGIN
0    :    000;
1    :    000;
2    :    000;
3    :    000;
4    :    000;
    (注: 此处略去多行;
地址    内容)
64   :    000;
65   :    004;
66   :    008;
67   :    012;
68   :    016;
69   :    020;
.....
250  :    150;
251  :    165;
252  :    180;
```



```

253 : 195;
254 : 210;
255 : 225;
END

```

模块 `deput` 的源代码如下:

```

module deput(o,a,b);
output[7:0] o;
input[3:0] a,b;
wire[7:0] o;
assign o={a,b};
endmodule

```

将这个基于 LPM 的乘法器设计存为 `mult_rom.gdf`, 进行编译后功能仿真的结果如图 10.18 所示。(注: 图中数字为十六进制形式)。

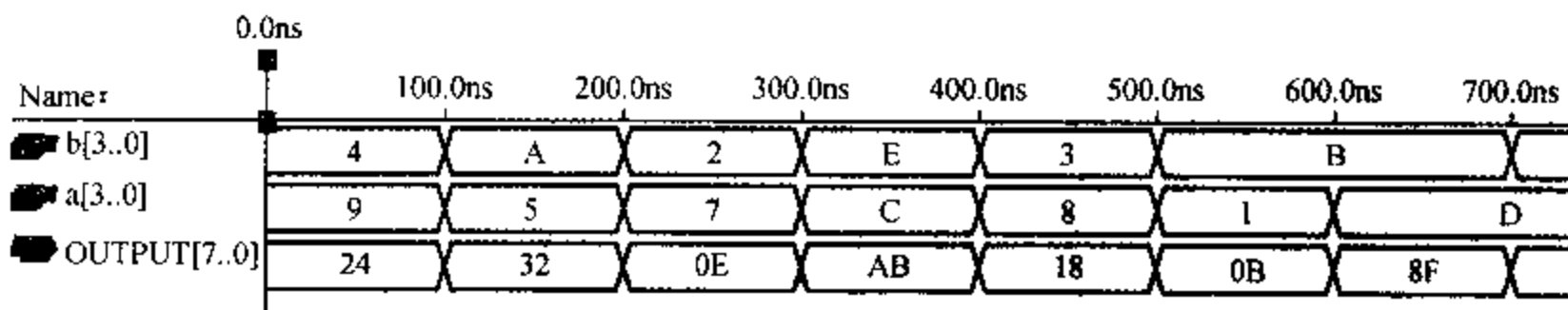


图 10.18 基于 LPM_ROM 的乘法器功能仿真图

10.8 如何消除毛刺

信号在 **FPGA** 器件内部通过连线和逻辑门时, 都有一定的延时。延时的大小与连线的长短和门单元的数目有关, 同时还受器件的制造工艺、工作电压、温度等条件的影响。此外, 信号的高/低电平转换也需要一定的过渡时间。由于存在这些因素的影响, 多路信号的电平值发生变化时, 在信号变化的瞬间, 组合逻辑的输出有先后顺序, 并不是同时变化, 往往会出现一些不正确的“毛刺”, 称为“冒险”现象。这些几纳秒的毛刺在电路板的设计中由于 **PCB** 走线时存在分布电感和分布电容, 所以许多毛刺能够被自然滤除, 而在 **PLD** 内部没有分布电感和电容, 这些毛刺将被完整地保留并向下一级传递, 所以在 **FPGA** 设计中, 如何消除毛刺就变得很重要。

任何一个门电路只要有二个输入信号同时向相反的方向变化, 其输出端都有可能产生毛刺。判断一个逻辑电路在某些输入信号发生变化时是否会产生冒险, 首先要判断信号是否会同时变化, 然后判断在信号同时变化的时候是否会产生冒险。

冒险往往会影响到逻辑电路的稳定性。时钟端口、清零和置位端口对毛刺信号十分敏感, 任何一点毛刺都可能会使系统出错, 因此判断逻辑电路中是否存在冒险以及如何避免冒险是设计人员必须要考虑的问题。

我们可以通过改变设计来破坏毛刺产生的条件,从而减少毛刺的发生。例如,在数字电路设计中,常常采用格雷码计数器取代普通的二进制计数器,这是因为格雷码计数器的输出每次只有一位跳变,消除了竞争冒险的发生条件,避免了毛刺的产生。

毛刺并不是对所有的输入都有危害,例如 D 触发器的 D 输入端,只要毛刺不出现在时钟的上升沿并且满足数据的建立和保持时间,就不会对系统造成影响,即 D 触发器的 D 输入端对毛刺不敏感。利用 D 触发器的这一特性,可以有效地消除毛刺。

本节将通过一个实际开发中遇到的简单例子来介绍这种根据 D 触发器的 D 输入端对毛刺不敏感的特点而消除毛刺的方法。

10.8.1 毛刺的产生

在平时的设计开发中,经常需要产生各种时钟波形,也就是各种占空比的矩形波,本节将设计一个简单的波形信号产生器。比如在数字通信中,PCM 编码是语音信号常用的源编码方式,MC145480 是单片语音 PCM 编码芯片,MC145480 芯片工作时需要两种时钟信号,即帧同步时钟和位同步时钟。这里取帧同步时钟为 8kHz,而位同步时钟取为 2.048MHz,帧同步时钟和位同步时钟的相位关系主要有两种方式,一种称为长帧同步方式,如图 10.19 所示;一种称为短帧同步方式,如图 10.20 所示。

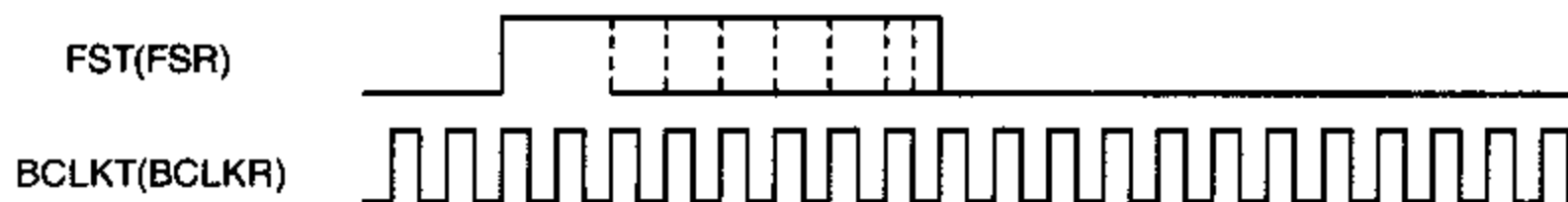


图 10.19 长帧同步时序关系



图 10.20 短帧同步时序关系

在本例中将要产生这两种相位关系的时钟信号,假定外输入时钟是 2.048MHz 的方波,那么在长帧方式中,位同步时钟可直接采用外输入时钟,而帧同步时钟的一个周期中包含 8 个位同步时钟周期的高电平、24 个位同步时钟周期的低电平。短帧方式同步时钟一个周期包含 1 个位同步时钟周期的高电平、31 个位同步时钟周期的低电平。

例 10.15 是由位同步时钟产生长帧同步时钟的 Verilog 代码,而短帧同步时钟的产生只需将参数 delay 设为 1 即可。

【例 10.15】长帧同步时钟的产生

```
module longframe1(clk, strb);
    parameter delay=8;
    input clk;
    output strb;
    reg strb;
```



```

reg [7:0] counter;
always@(posedge clk)
begin
    if(counter==255)    counter=0;
        else    counter=counter+1;
    end
always@(counter)
begin
    if(counter<=(delay-1))    strb=1;
    else    strb=0;
    end
endmodule

```

10.8.2 毛刺的消除

上面的设计虽然完成了波形产生的任务，但在进行时序仿真的时候会发现输出波形中有毛刺，如图 10.21 所示，在进行实际电路的调试时也会发现电路有明显的噪声，主要是由于帧同步时钟中的毛刺引起的，因此要考虑消除毛刺。实际上，只要在波形输出端加一个触发器，即可解决毛刺问题，其源代码如例 10.16 所示。

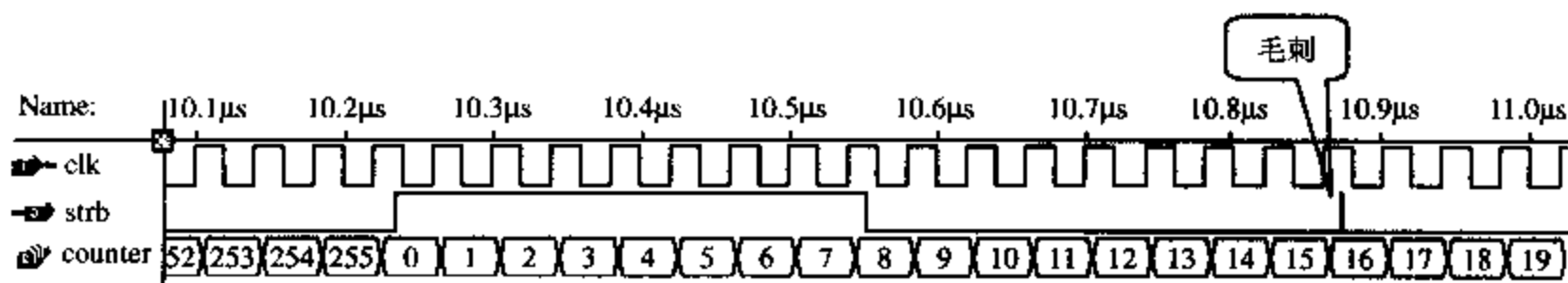


图 10.21 时序仿真输出波形

【例 10.16】引入了 D 触发器的长帧同步时钟的产生

```

module longframe2(clk, strb);
parameter delay=8;
input clk;
output strb;
reg [7:0] counter;
reg temp;
reg strb;
always@(posedge clk)
begin
    if(counter==255)    counter=0;
        else    counter=counter+1;
    end
end

```

```

always@(posedge clk)
begin
    strb=temp;          //引入一个触发器
end
always@(counter)
begin
    if(counter<=(delay-1)) temp=1;
    else temp=0;
end
endmodule

```

如图 10.22 所示是消除毛刺后的时序仿真输出波形。可以看出, 信号 strb 比图 10.21 中的仿真波形拖后了一个时钟周期, 但 strb 信号中的毛刺去掉了。

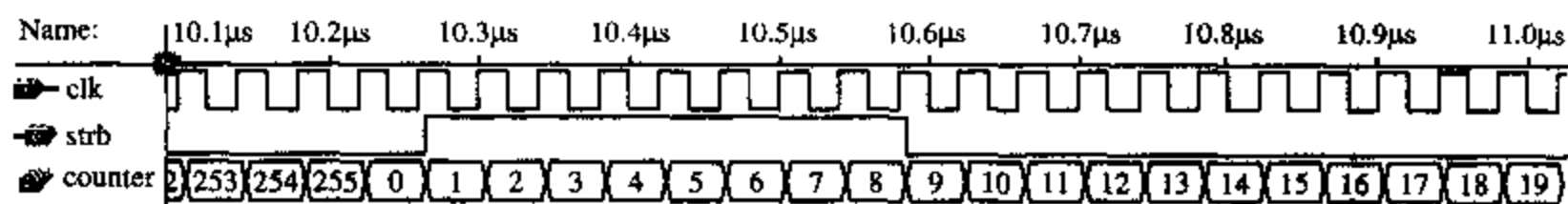


图 10.22 消除毛刺后的时序仿真输出波形

如图 10.23 和图 10.24 所示分别是例 10.15 和例 10.16 的 RTL 综合视图, 从这两个 RTL 视图我们可以很清楚地看出, 后者只在前者的基础上, 在输出端加入了一个触发器, 即有效去除了输出信号中的毛刺。

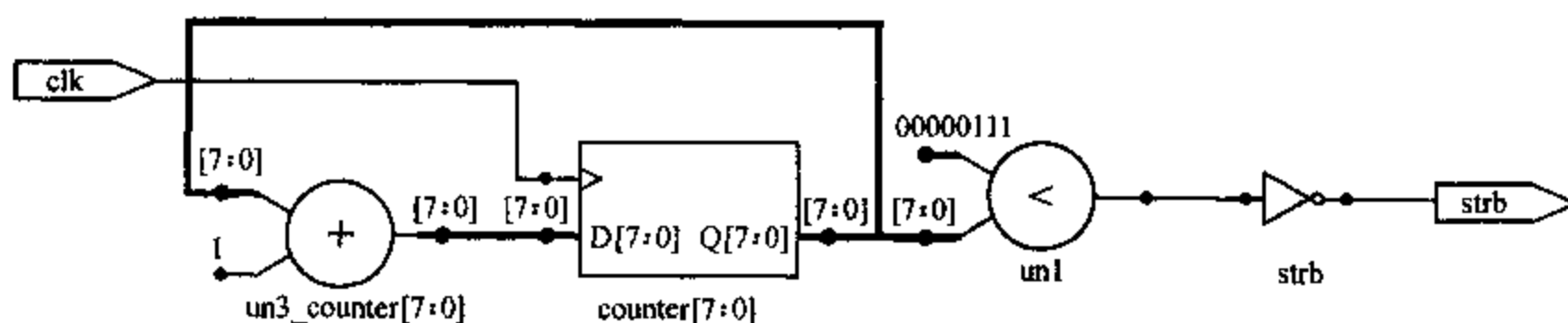


图 10.23 例 10.15 的 RTL 综合结果

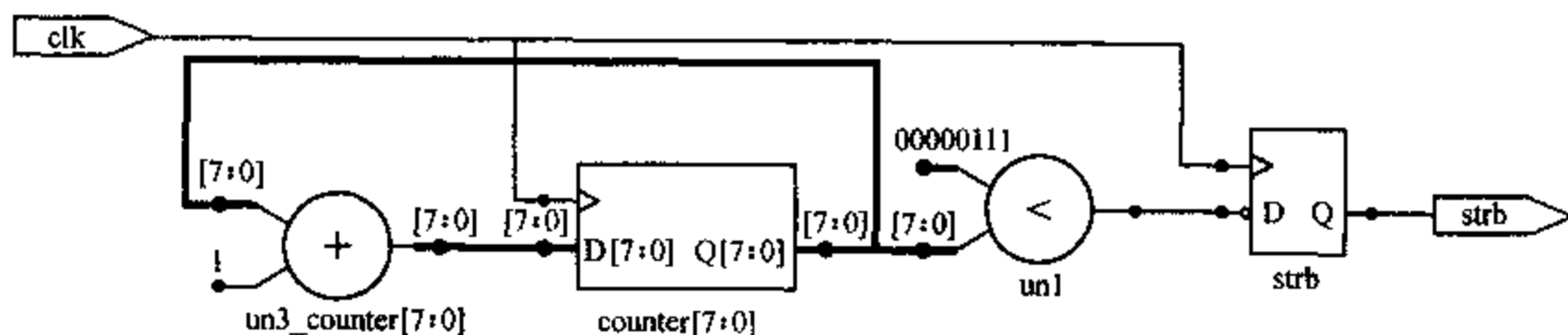


图 10.24 例 10.16 的 RTL 综合结果

本节介绍的消除毛刺的方法简单有效, 当然还有很多其他方法也可用于消除毛刺, 比如流水线设计技术。由于流水线技术是在复杂组合逻辑中加入寄存器, 因此可以有效减少毛刺的产生。

思考与练习

1. 利用状态机, 设计一个序列检测器, 检测器在连续 4 个或 4 个以上的“1”输入时输出为“1”; 其他输入情况下, 输出为“0”。画出状态转移图, 并用 Verilog 语言描述实现。
2. 阻塞赋值与非阻塞赋值有什么本质的区别, 在使用中应注意哪些方面, 结合自己的设计实践进行总结。
3. 流水线设计技术为什么能提高数字系统的工作频率?
4. 设计一个加法器, 实现 $\text{sum}=\text{a0}+\text{a1}+\text{a2}+\text{a3}$, a0 、 a1 、 a2 、 a3 宽度都是 8 位。如用下面两种方法实现, 哪种方法更好一些。
 - (1) $\text{sum}=(\text{a0}+\text{a1})+\text{a2}+\text{a3}$
 - (2) $\text{sum}=(\text{a0}+\text{a1})+(\text{a2}+\text{a3})$
5. 用流水线技术对上例中的 $\text{sum}=(\text{a0}+\text{a1})+\text{a2}+\text{a3}$ 的实现方式进行优化, 对比最高工作频率。
6. 在 FPGA 设计开发中, 还有哪些技术可提高设计性能?

第 11 章 Verilog HDL 综合设计实践

本章内容简介

- ◆ 数字跑表
- ◆ 4 位数字频率计
- ◆ 十字路口交通灯控制器
- ◆ 乐曲演奏电路
- ◆ 自动售饮料机
- ◆ 实用多功能数字钟
- ◆ 计费器

本章通过一些多功能数字电路的设计进一步讨论基于 Verilog HDL 的开发实践, 多数设计用文本方式完成, 少数采用文本与图形混合输入的方式实现。这些设计实例经常在数字电路和相关课程中遇到。还可以把这些设计与实际的实验开发装置相结合, 将设计文件生成的可配置数据下载到实验板中, 实际观察设计效果。

11.1 数字跑表

11.1.1 设计输入与编译

设计一个数字跑表, 该跑表具有复位、暂停、秒表计时等功能。图 11.1 是该数字跑表的结构示意图。



图 11.1 数字跑表示意图

首先对数字跑表进行结构和功能的划分。跑表设三个输入端, 分别为时钟输入 (CLK)、复位 (CLR) 和启动/暂停 (PAUSE) 按键。复位信号高电平有效, 可对跑表异步清零; 当启动/暂停键为低电平时, 跑表开始计时, 为高电平时暂停, 变低后在原来的数值基础上继续计

数。这两个控制信号的作用见表 11.1。

表 11.1	两个控制信号的作用	
控 制 端	取 值	功 能
复位 (CLR)	1	异步清零
	0	计数
计数/暂停键 (PAUSE)	1	暂停
	0	计数

为便于显示,百分秒、秒和分钟信号皆采用 BCD 码计数方式,并直接输出到 6 个数码管显示出来。

根据上面的设计要求,用 Verilog 语言对数字跑表描述如下。

【例 11.1】数字跑表

/*信号定义:

CLK: CLK 为时钟信号;

CLR: 为异步复位信号;

PAUSE: 为暂停信号;

MSH, MSL: 百分秒的高位和低位;

SH, SL: 秒信号的高位和低位;

MH, ML: 分钟信号的高位和低位。 */

module paobiao(CLK,CLR,PAUSE,MSH,MSL,SH,SL,MH,ML);

input CLK,CLR;

input PAUSE;

output [3:0] MSH,MSL,SH,SL,MH,ML;

reg [3:0] MSH,MSL,SH,SL,MH,ML;

reg cn1,cn2; //cn1 为百分秒向秒的进位, cn2 为秒向分的进位

//百分秒计数进程, 每计满 100, cn1 产生一个进位

always @(posedge CLK or posedge CLR)

begin

if(CLR) **begin** //异步复位

{MSH,MSL}<=8'h00;

cn1<=0;

end

else if(!PAUSE) //PAUSE 为 0 时正常计数, 为 1 时暂停计数

begin

if(MSL==9) **begin**

MSL<=0;

```
                if(MSH==9)
                    begin MSH<=0;  cn1<=1;  end
                    else MSH<=MSH+1;
                    end
            else    begin
                    MSL<=MSL+1;  cn1<=0;
                    end
        end
    end
end
```

//秒计数进程, 每计满 60, cn2 产生一个进位

```
always @(posedge cn1 or posedge CLR)
```

```
begin
```

```
    if(CLR) begin          //异步复位
```

```
        {SH,SL}<=8'h00;
```

```
        cn2<=0;
```

```
    end
```

```
    else if(SL==9)         //低位是否为 9
```

```
        begin
```

```
            SL<=0;
```

```
            if(SH==5) begin SH<=0;  cn2<=1;  end
```

```
            else          SH<=SH+1;
```

```
            end
```

```
        else
```

```
            begin SL<=SL+1;  cn2<=0;  end
```

```
    end
```

//分钟计数进程, 每计满 60, 系统自动清零

```
always @(posedge cn2 or posedge CLR)
```

```
begin
```

```
    if(CLR)
```

```
        begin {MH,ML}<=8'h00;  end    //异步复位
```

```
    else if(ML==9) begin
```

```
        ML<=0;
```

```
        if(MH==5)  MH<=0;
```

```
        else      MH<=MH+1;
```

```
        end
```

```
    else  ML<=ML+1;
```

```
end
```


endmodule

如图 11.2 所示是数字跑表的端口示意图。

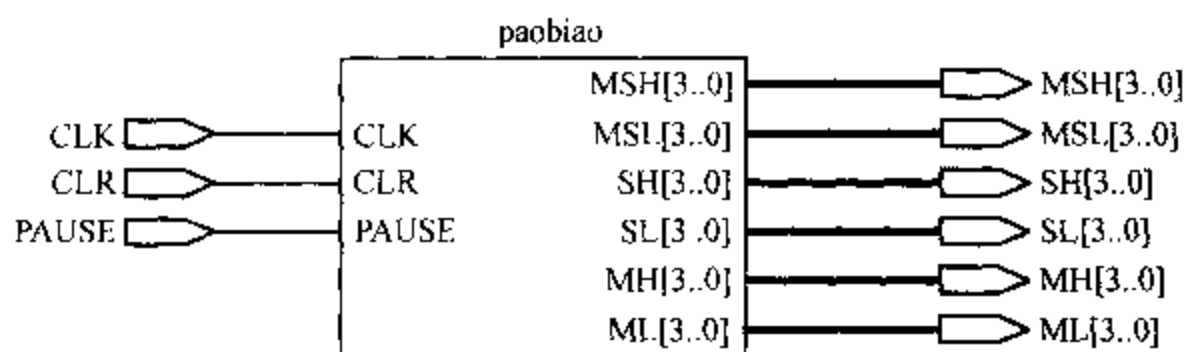


图 11.2 数字跑表的端口连接

11.1.2 仿真

用 MAX+PLUS II 软件对所设计的跑表程序进行编译和仿真。在编译时, 选择 Processing/Functional SNF Extractor 选项, 以进行功能仿真。用波形编辑器在跑表的输入端加上必要的激励信号, 然后查看输出波形, 图 11.3 是其仿真波形的一部分。由仿真可知, 该数字跑表实现了预想的各种功能。

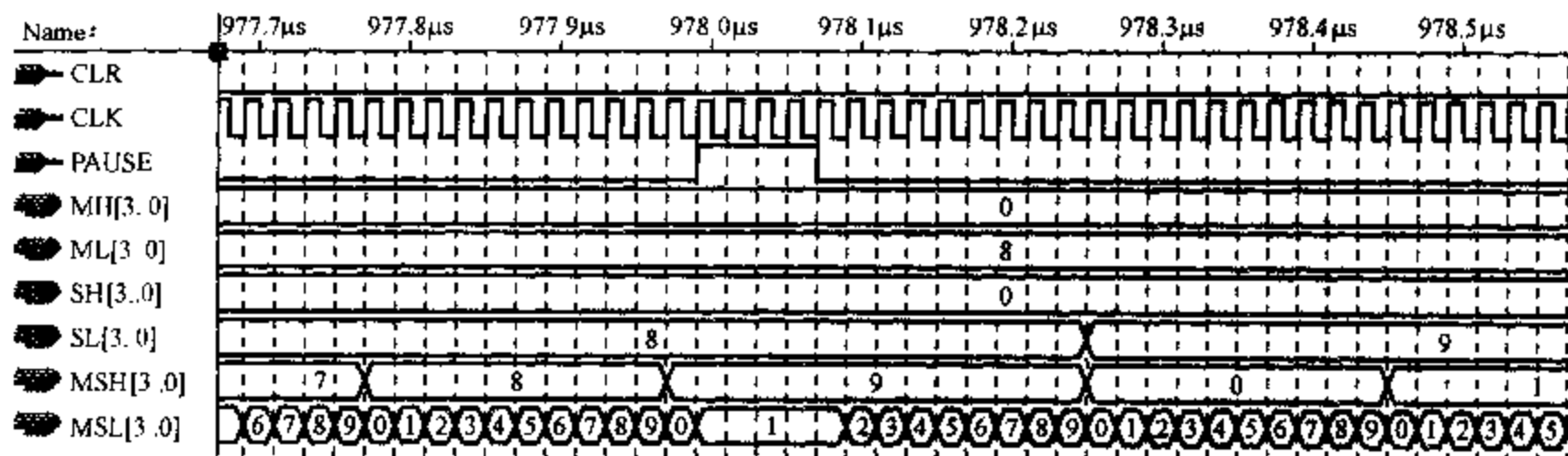


图 11.3 数字跑表功能仿真波形

11.2 4 位数字频率计

本节设计一个 4 位频率计, 可以测量 1~9999Hz 的信号频率, 并将被测信号的频率在 4 个数码管上显示出来。采用文本和图形混合设计的方法, 设计软件用 MAX+PLUS II。

11.2.1 功能与原理

采用一个标准的基准时钟, 在单位时间 (如 1s) 里对被测信号的脉冲数进行计数, 即为信号的频率。4 位数字频率计的顶层框图如图 11.4 所示, 整个系统分为 3 个模块: 控制模块、计数测量模块和数据锁存器。

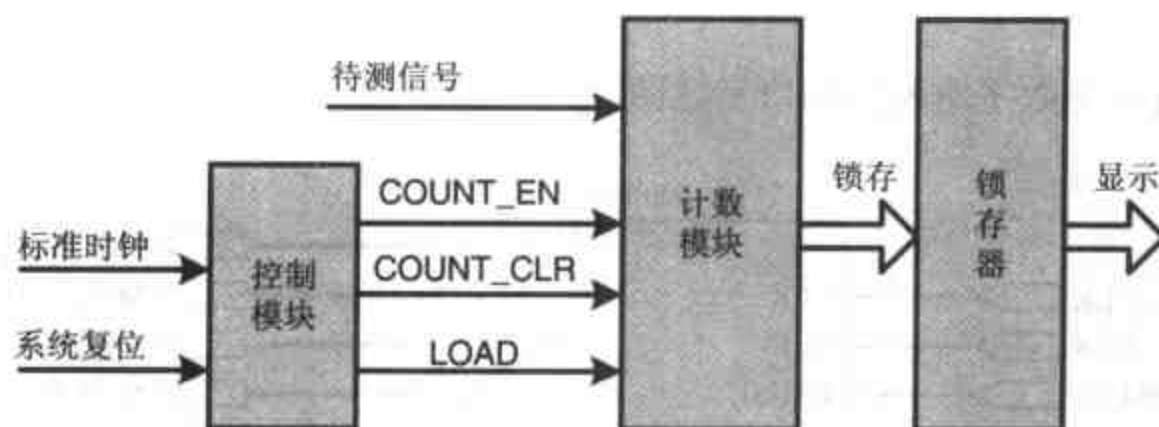


图 11.4 4 位数字频率计的框图

1. 控制模块

控制模块的作用是产生测频所需要的各种控制信号。控制模块的标准输入时钟为 1Hz，每两个时钟周期进行一次频率测量。该模块产生 3 个控制信号，分别为 COUNT_EN、COUNT_CLR 和 LOAD。COUNT_CLR 信号用于在每一次测量开始时对计数模块进行复位，以清除上次测量的结果。复位信号高电平有效，持续半个时钟周期的时间。COUNT_EN 信号为计数允许信号，在 COUNT_EN 信号的上升沿时刻，计数模块开始对输入信号的频率进行测量，测量时间恰为一个时钟周期（正好为单位时间 1s），在此时间里对被测信号的脉冲数进行计数，即为信号的频率。然后将该值锁存，并送到数码管显示出来。设置锁存器的好处是，显示的数据稳定，不会由于周期性的清零信号而不断闪烁。在每一次测量开始时，都必须重新对计数模块清零。

控制模块所产生的几个控制信号的时序关系如图 11.5 所示。从图中可以看出，计数使能信号 COUNT_EN 在 1s 的高电平后利用其反相值的上跳沿产生一个锁存信号 LOAD，随后产生清零信号上跳沿 COUNT_CLR。

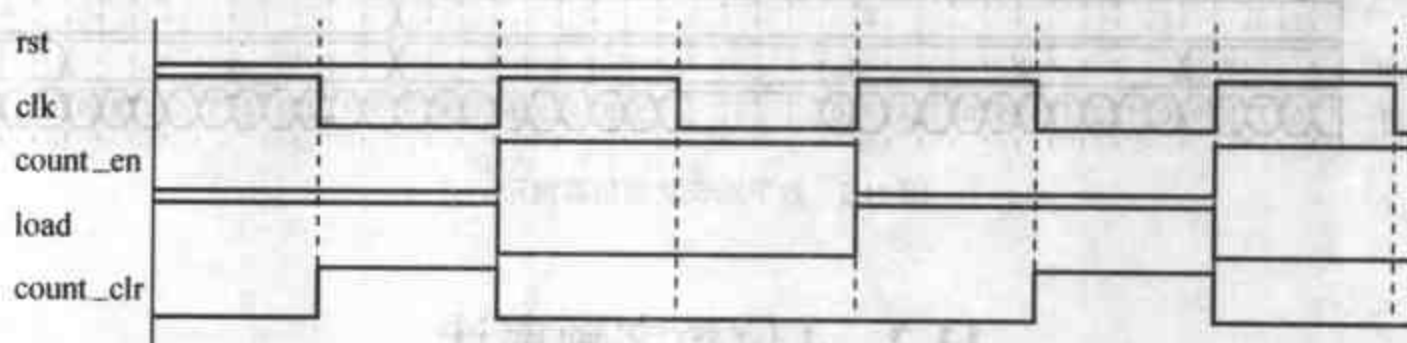


图 11.5 控制信号的时序关系

2. 锁存器模块

锁存器模块也是必不可少的，测频模块测量完后，在 LOAD 信号的上升沿时刻将测量值锁存到寄存器中，然后输出，送到试验板上的数码管上显示出相应的数值。如果所用的下载实验板没有 BCD 码—七段数码管译码电路，则还需要设计译码电路。

3. 计数模块

计数模块用于在单位时间内对输入信号的脉冲进行计数，该模块必须有计数允许、异步清 0 等端口，以便于控制模块对其进行控制。

11.2.2 设计实现

如图 11.6 所示是 4 位数字频率计的顶层原理图。

其中 `fre_ctrl` 是控制模块, `count10` 是计数模块, `latch_16` 是 16 位锁存模块。这 3 个模块都采用文本方式设计实现。`SIGNAL` 为被测输入信号, `CLK` 为输入的基准时钟, `QO[15..0]` 为输出到数码管显示的频率值。

1. `fre_ctrl` 控制模块

`fre_ctrl` 控制模块的作用前面已经讲过, 它用于产生 3 个控制信号。`fre_ctrl` 模块用 Verilog 语言描述如下。

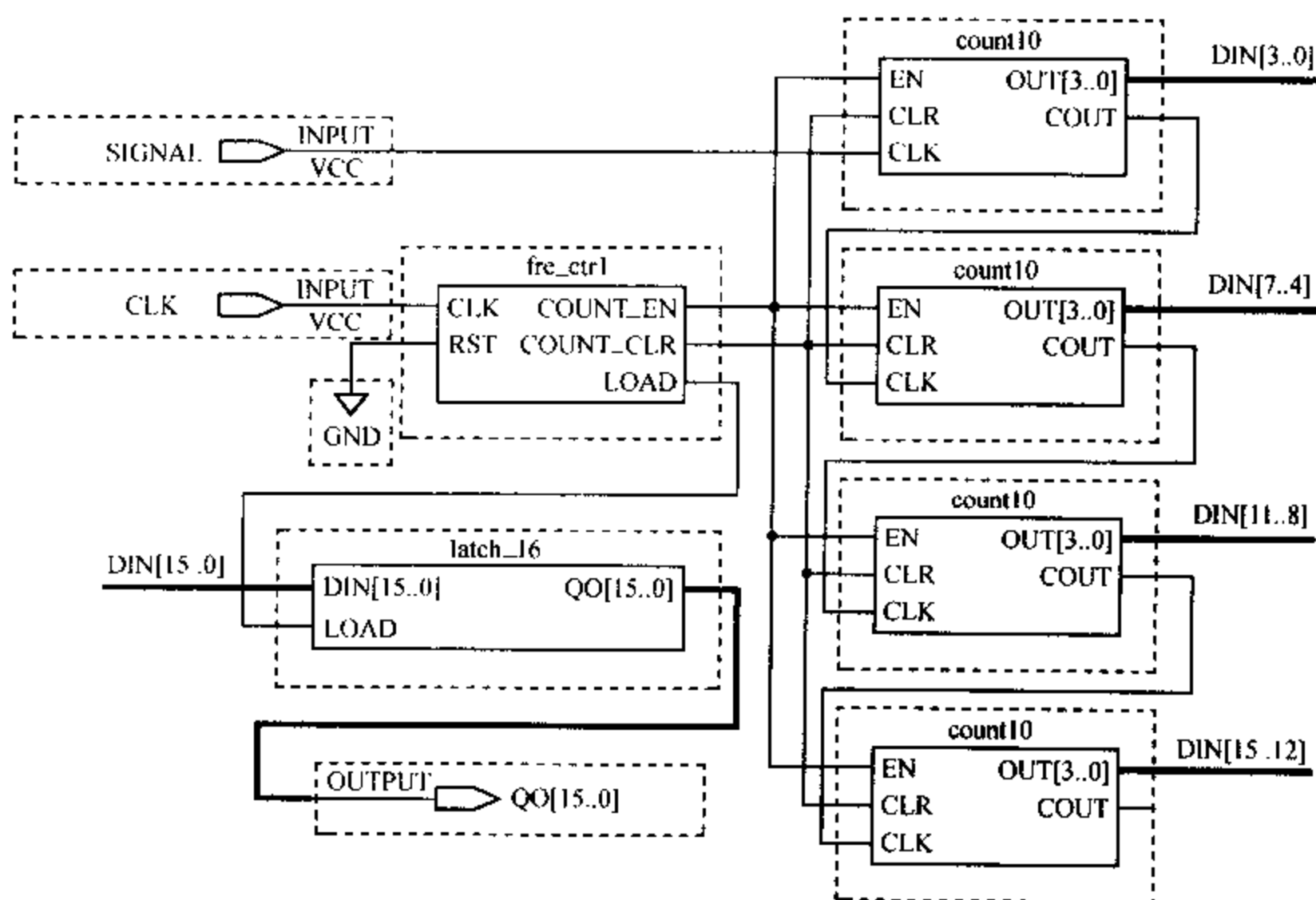


图 11.6 4 位数字频率计顶层原理图

【例 11.2】4 位数字频率计控制模块

```

module fre_ctrl(clk,rst,count_en,count_clr,load);
output count_en,count_clr,load;
input clk,rst;
reg count_en,load;
always @(posedge clk)
begin
    if(rst) begin count_en=0; load=1; end
    else begin
        count_en=~count_en;
        load=~count_en;           // load 信号的产生
    end

```

```

    end
    assign count_clr=~clk&load;           // count_clr 信号的产生
endmodule

```

2. count10 计数模块

count10 是一个带有计数使能输入端 (EN) 和异步清零端 (CLR) 的模为 10 的计数模块。当 EN 为高电平时开始计数，为低电平时停止计数。CLR 为异步清零端，当它为高电平时，计数器输出为零。

【例 11.3】4 位数字频率计计数子模块

```

module count10(out, cout, en, clr, clk);
    output[3:0] out;
    output cout;
    input en,clr,clk;
    reg[3:0] out;

    always @(posedge clk or posedge clr)
    begin
        if (clr) out = 0;           //异步清 0
        else if(en)
            begin if(out==9) out=0;
                  else out = out+1;
            end
    end

    end

    assign cout = ((out==9)&en)?1:0; //产生进位信号
endmodule

```

3. 16 位锁存器模块 latch_16

【例 11.4】频率计锁存器模块

```

module latch_16(qo,din,load);
    output[15:0] qo;
    input[15:0] din;
    input load;
    reg[15:0] qo;
    always @(posedge load)
        begin qo=din; end
endmodule

```

将整个图形和文本设计进行综合和仿真，如图 11.7 所示是其功能仿真波形的一部分。从仿真波形图可以看出，改变输入信号的频率，频率计都能准确测出其频率，如图中所示，分别为 20Hz、17Hz 和 8Hz（图中假定 clk 一个周期为 1s）。

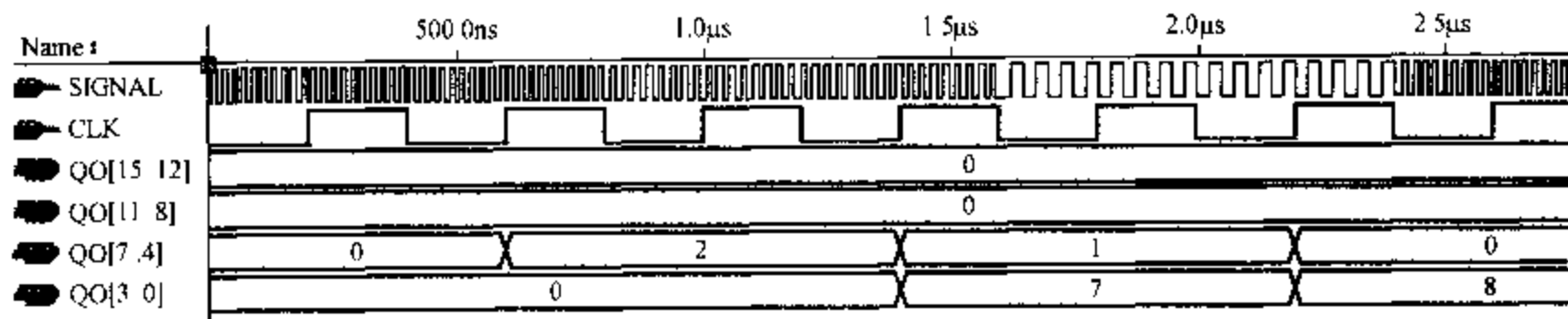


图 11.7 频率计的功能仿真波形图

11.3 交通灯控制器

用 Verilog 语言设计一个十字路口交通灯控制器，其示意图如图 11.8 所示，A 方向和 B 方向各设红（R）、黄（Y）、绿（G）和左拐（L）4 盏灯，4 种灯按合理的顺序亮灭，并能将灯亮的时间以倒计时的形式显示出来。

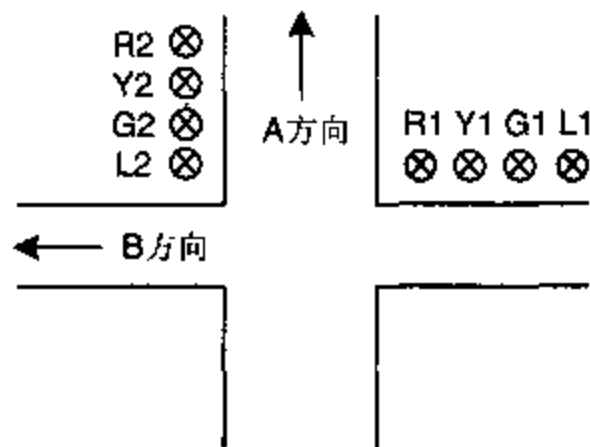


图 11.8 十字路口交通灯控制器示意图

11.3.1 功能要求

两个方向各种灯亮的时间能够进行设置和修改，此外，假设 A 方向是主干路，车流量大，因此 A 方向通行的时间应比 B 方向长。

交通灯控制器的状态转换见表 11.2。表中，1 表示灯亮，0 表示灯灭。A 方向和 B 方向的红、黄、绿和左拐灯分别用 R1、Y1、G1、L1 和 R2、Y2、G2、L2 来表示。

表 11.2 交通灯控制器的状态转换表

A 方向				B 方向			
绿灯 (G1)	黄灯 (Y1)	左拐灯 (L1)	红灯 (R1)	绿灯 (G2)	黄灯 (Y2)	左拐灯 (L2)	红灯 (R2)
1	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1
0	0	1	0	0	0	0	1
0	1	0	0	0	0	0	1
0	0	0	1	1	0	0	0
0	0	0	1	0	1	0	0
0	0	0	1	0	0	1	0
0	0	0	1	0	1	0	0

从状态转换表中可以看出, 每个方向 4 种灯依次按如下顺序点亮, 并不断循环:

绿灯→黄灯→左拐灯→黄灯→红灯。

并且每个方向红灯亮的时间应该与另一方向绿、黄、左拐、黄灯亮的时间相等。黄灯所起的作用是用来在绿灯和左拐灯后进行缓冲, 以提醒行人该方向马上要禁行了。

11.3.2 设计实现

1. 设计思路

根据交通灯控制器要实现的功能, 考虑用两个并行执行的 always 模块(两个进程)来分别控制 A 和 B 两个方向的 4 盏灯。这两个 always 模块使用同一个时钟信号, 以进行同步, 也就是说, 两个进程的敏感信号是同一个。

每个 always 模块控制一个方向的 4 种灯按如下顺序点亮, 并往复循环:

绿灯→黄灯→左拐灯→黄灯→红灯。

每种灯亮的时间采用一个减法计数器进行计数, 计数器用同步预置法设计, 这样只需改变预置数据, 就能改变计数器的模, 因此每个方向只要一个计数器进行计时即可。

为便于显示灯亮的时间, 计数器的输出均采用 BCD 码, 显示由 4 个数码管来完成, A 方向和 B 方向各用两个数码管。

2. 源程序

本例中设定 A 方向红、绿、黄、左拐灯亮的时间分别为 65s、40s、5s 和 15s, B 方向红、绿、黄、左拐灯亮的时间分别为: 55s、30s、5s 和 15s。假如要改变这些时间, 只需改变计数器的预置数即可。

以下是交通灯控制器的源程序。

【例 11.5】交通灯控制器

/* 信号定义与说明:

CLK: 为同步时钟;

EN: 使能信号, 为 1 的话, 则控制器开始工作;

LAMPA: 控制 A 方向 4 盏灯的亮灭; 其中, LAMPA0~LAMPA3 分别控制 A 方向的左拐灯、绿灯、黄灯和红灯;

LAMPB: 控制 B 方向 4 盏灯的亮灭; 其中, LAMPB0~LAMPB3 分别控制 B 方向的左拐灯、绿灯、黄灯和红灯;

ACOUNT: 用于 A 方向灯的时间显示, 8 位, 可驱动两个数码管;

BCOUNT: 用于 B 方向灯的时间显示, 8 位, 可驱动两个数码管。 */

```
module traffic(CLK, EN, LAMPA, LAMPB, ACOUNT, BCOUNT);
```

```
output [7:0] ACOUNT, BCOUNT;
```

```
output [3:0] LAMPA, LAMPB;
```

```
input CLK, EN;
```

```
reg [7:0] numa, numb;
```

```
reg tempa, tempb;
```

```
reg [2:0] counta, countb;
```

```

reg[7:0]  ared,ayellow,agreen,aleft,bred,byellow,bgreen,bleft;
reg[3:0]  LAMPA,LAMPB;

always @(EN)
  if(!EN)
    begin
      //设置各种灯的计数器的预置数
      ared    <=8'd55;      //55 秒
      ayellow <=8'd5;       //5 秒
      agreen  <=8'd40;      //40 秒
      aleft   <=8'd15;      //15 秒
      bred    <=8'd65;      //65 秒
      byellow <=8'd5;       //5 秒
      bleft   <=8'd15;      //15 秒
      bgreen  <=8'd30;      //30 秒
    end
  assign ACOUNT=numa;
  assign BCOUNT=numb;

always @(posedge CLK) //该进程控制 A 方向的 4 种灯
  begin
    if(EN)
      begin
        if(!tempa)
          begin
            tempa<=1;
            case(counta) //控制亮灯的顺序
              0: begin numa<=agreen;    LAMPA<=2; counta<=1; end
              1: begin numa<=ayellow;    LAMPA<=4; counta<=2; end
              2: begin numa<=aleft;      LAMPA<=1; counta<=3; end
              3: begin numa<=ayellow;    LAMPA<=4; counta<=4; end
              4: begin numa<=ared;       LAMPA<=8; counta<=0; end
              default:
                LAMPA<=8;
            endcase
          end
        else begin //倒计时
          if(numa>1)
            if(numa[3:0]==0) begin
              numa[3:0]<=4'b1001;
              numa[7:4]<=numa[7:4]-1;
            end
          end
        end
      end
    end
  end

```

```
                                end
                                else
                                    numa[3:0]<=numa[3:0]-1;
                                if (numa==2) tempa<=0;
                                end
                            end
                        else begin
                            LAMPA<=4'b1000;
                            counta<=0; tempa<=0;
                        end
                    end

always @(posedge CLK)           //该进程控制 B 方向的 4 种灯
begin
    if (EN)
        begin
            if(!tempb)
                begin
                    tempb<=1;
                    case (countb)           //控制亮灯的顺序
                        0: begin numb<=bred;      LAMPB<=8; countb<=1; end
                        1: begin numb<=bgreen;    LAMPB<=2; countb<=2; end
                        2: begin numb<=byellow;   LAMPB<=4; countb<=3; end
                        3: begin numb<=bleft;     LAMPB<=1; countb<=4; end
                        4: begin numb<=byellow;   LAMPB<=4; countb<=0; end
                        default:
                            LAMPB<=8;
                        endcase
                    end
                else
                    begin
                        //倒计时
                        if(numb>1)
                            if(!numb[3:0]) begin
                                numb[3:0]<=9;
                                numb[7:4]<=numb[7:4]-1;
                                end
                            else
                                numb[3:0]<=numb[3:0]-1;
                                if(numb==2) tempb<=0;
                            end
                        end
                    else begin
```



```

        LAMPB<=4'b1000;
        tempb<=0;    countb<=0;
    end

end
endmodule

```

图 11.9 是交通灯控制器的端口示意图。

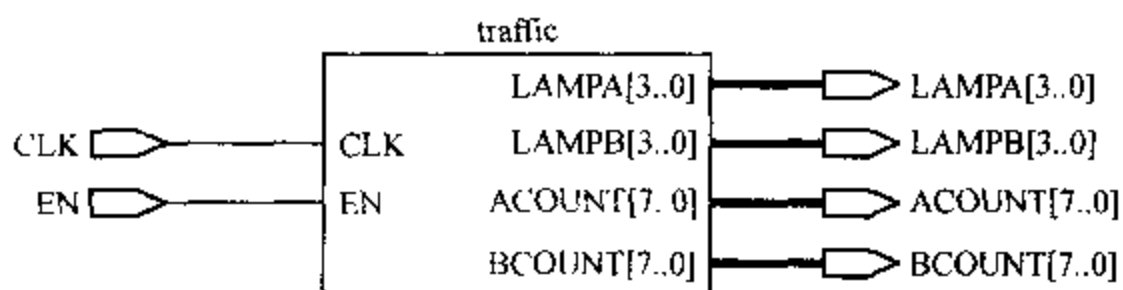


图 11.9 交通灯控制器的端口示意图

对交通灯控制器进行时序仿真，图 11.10 是其仿真波形的一部分。该图表示了两个方向的 4 种灯亮灭转换的时序关系。

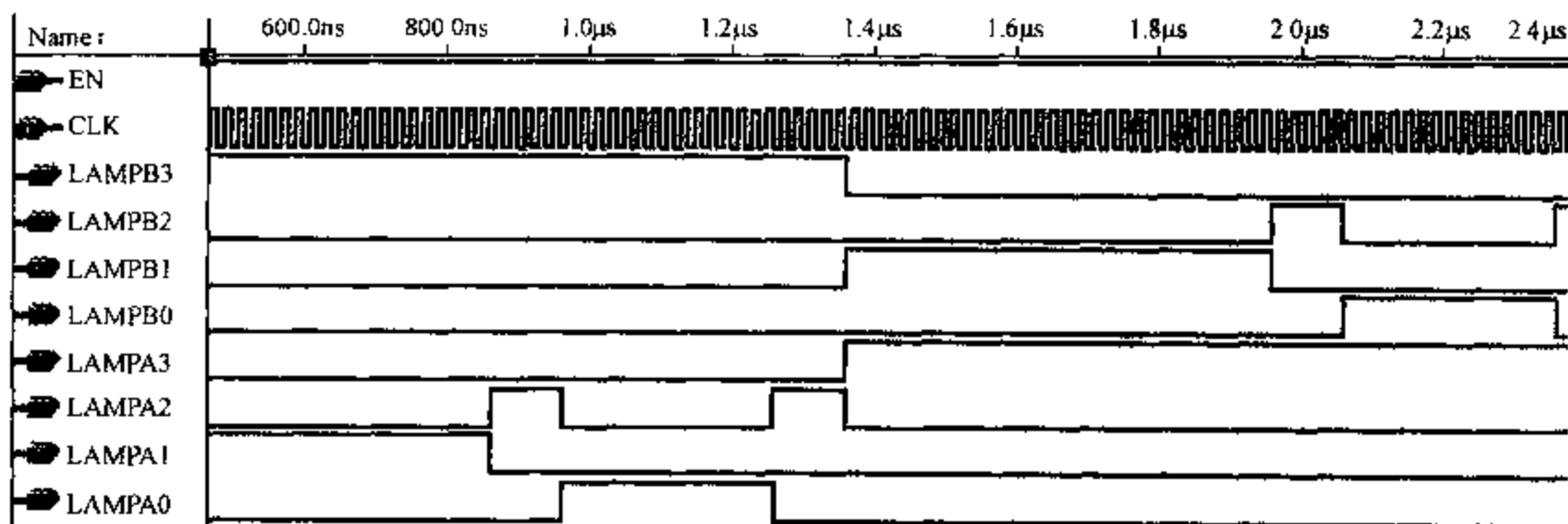


图 11.10 交通灯控制器仿真波形

11.4 乐曲演奏电路

在本节中，采用 FPGA 器件驱动扬声器构成一个乐曲演奏电路，FPGA 器件选择 Altera 的 EPF10K10，音乐选择“梁祝”片段，其曲谱如下。

3 - 5 · 6 | 1 · 2 6 1 5 | 5 1 6 5 3 5 | 2 - - - |

2 · 3 7 6 | 5 · 6 1 2 | 3 1 6 5 6 1 | 5 - - - |

乐曲演奏的原理是这样的：组成乐曲的每个音符的频率值（音调）及其持续的时间（音长）是乐曲能连续演奏所需的两个基本数据，因此只要控制输出到扬声器的激励信号的频率的高低和持续的时间，就可以使扬声器发出连续的乐曲声。首先来看一下怎样控制音调的高

低变化。

11.4.1 音调的控制

频率的高低决定了音调的高低。音乐的十二平均率规定：每两个八度音（如简谱中的中音 1 与高音 1）之间的频率相差一倍。在两个八度音之间又可分为十二个半音，每两个半音的频率比为 $\sqrt[12]{2}$ 。另外，音名 A（简谱中的低音 6）的频率为 440Hz，音名 B 到 C 之间、E 到 F 之间为半音，其余为全音。由此可以计算出简谱中从低音 1 至高音 1 之间每个音名对应的频率，见表 11.3。

表 11.3 简谱中的音名与频率的关系

音 名	频率(Hz)	音 名	频率(Hz)	音 名	频率(Hz)
低音 1	261.6	中音 1	523.3	高音 1	1046.5
低音 2	293.7	中音 2	587.3	高音 2	1174.7
低音 3	329.6	中音 3	659.3	高音 3	1318.5
低音 4	349.2	中音 4	698.5	高音 4	1396.9
低音 5	392	中音 5	784	高音 5	1568
低音 6	440	中音 6	880	高音 6	1760
低音 7	493.9	中音 7	987.8	高音 7	1975.5

所有不同频率的信号都是从同一个基准频率分频得到的。由于音阶频率多为非整数，而分频系数又不能为小数，因此必须将计算得到的分频数四舍五入取整。若基准频率过低，则由于分频比太小，四舍五入取整后的误差较大；若基准频率过高，虽然误差变小，但分频数将变大。实际的设计应综合考虑这两方面的因素，在尽量减小频率误差的前提下取合适的基准频率。本例中选取 6MHz 为基准频率。若无 6MHz 的时钟频率，则可以先分频得到 6MHz 或换一个新的基准频率。实际上，只要各个音名间的相对频率关系不变，C 作 1 与 D 作 1 演奏出的音乐听起来都不会“走调”。

本例需要演奏的是梁祝乐曲，该乐曲各音阶频率及相应的分频比见表 11.4。为了减小输出的偶次谐波分量，最后输出到扬声器的波形应为对称方波，因此在到达扬声器之前有一个二分频的分频器。表 11.4 中的分频比就是在从 6MHz 频率二分频得到的 3MHz 频率基础上计算得出的。如果用正弦波代替对称方波驱动扬声器将会有更好的效果。

从表 11.4 可以看出，最大的分频系数为 9102，故采用 14 位二进制计数器分频可满足需要。在表 11.4 中，除给出了分频比以外，还给出了对应于各个音阶频率时计数器不同的预置数。对于不同的分频系数，只要加载不同的预置数即可。采用加载预置数实现分频的方法比采用反馈复零法节约资源，实现起来也更容易一些。

表 11.4 各音阶频率对应的分频比及预置数

音 名	分 频 比	预 置 数	音 名	分 频 比	预 置 数
低音 3	9102	7281	中音 2	5111	11272
低音 5	7653	8730	中音 3	4552	11831
低音 6	6818	9565	中音 5	3827	12556
低音 7	6073	10310	中音 6	3409	12974
中音 1	5736	10647	高音 1	2867	13516

此外,对于乐曲中的休止符,只要将分频系数设为 0,即初始值为 $2^{14}-1=16383$ 即可,此时扬声器将不会发声。

11.4.2 音长的控制

音符的持续时间须根据乐曲的速度及每个音符的节拍数来确定。

本例演奏的梁祝片段,最短的音符为四分音符,如果将全音符的持续时间设为 1s 的话,则只需要再提供一个 4Hz 的时钟频率即可产生四分音符的时长。

乐曲演奏电路的原理框图如图 11.11 所示。

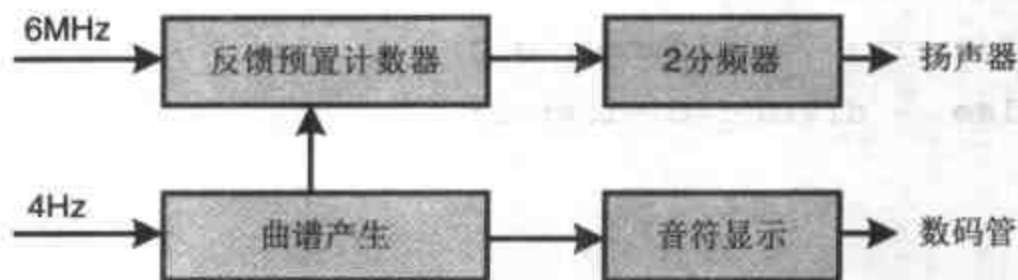


图 11.11 乐曲演奏电路原理框图

其中,乐谱产生电路用来控制音乐的音调和音长。控制音调通过设置计数器的预置数来实现,预置不同的数值就可以使计数器产生不同频率的信号,从而产生不同的音调。控制音长是通过控制计数器预置数的停留时间来实现的,预置数停留的时间越长,则该音符演奏的时间也就越长。每个音符的演奏时间都是 0.25s 的整数倍,对于节拍较长的音符,如 2 分音符,在记谱时将该音名连续记录两次即可。

音名显示电路用来显示乐曲演奏时对应的音符。可以用 3 个数码管分别显示高、中、低音的音名,实现演奏的动态显示,十分直观。在本例中, HIGH[3:0]、MED[3:0]、LOW[3:0] 等信号分别用于显示高音、中音和低音音符。

为了使演奏能循环进行,需另外设置一个时长计数器,当乐曲演奏完成时,保证能自动从头开始演奏。

11.4.3 乐曲演奏电路源程序

“梁祝”乐曲片段演奏电路的源文件如下。

【例 11.6】“梁祝”乐曲演奏电路

//信号定义与说明:

// clk_4Hz: 用于控制音长(节拍)的时钟频率;

```

// clk_6MHz:    用于产生各种音阶频率的基准频率;
// speaker:     用于激励扬声器的输出信号, 本例中为方波信号;
// high, med, low: 分别用于显示高音、中音和低音音符, 各驱动一个数码管来显示。

```

```

module song (clk_6MHz, clk_4Hz, speaker, high, med, low);
input  clk_6MHz, clk_4Hz;
output speaker;
output[3:0] high, med, low;
reg[3:0] high, med, low;
reg[13:0] divider, origin;
reg[7:0] counter;
reg speaker;
wire carry;

```

```

assign carry=(divider==16383);

```

```

always @(posedge clk_6MHz)
    begin    if(carry)    divider=origin;
            else        divider=divider+1;
    end

```

```

always @(posedge carry)
    begin
        speaker=~speaker;           //2 分频产生方波信号
    end

```

```

always @(posedge clk_4Hz)
    begin
        case ({high, med, low})           //分频比预置
            'b000000000011: origin=7281;
            'b000000000101: origin=8730;
            'b000000000110: origin=9565;
            'b000000000111: origin=10310;
            'b000000010000: origin=10647;
            'b000000100000: origin=11272;
            'b000000110000: origin=11831;
            'b000001010000: origin=12556;
            'b000001100000: origin=12974;
            'b000100000000: origin=13516;

```

```

'b00000000000000:origin=16383;
endcase
end

always @(posedge clk_4Hz)
begin
    if(counter==63)    counter=0;           //计时, 以实现循环演奏
    else                counter=counter+1;

    case(counter)      //记谱
    0: {high,med,low}='b0000000000011;      //低音“3”
    1: {high,med,low}='b0000000000011;      //持续 4 个时钟节拍
    2: {high,med,low}='b0000000000011;
    3: {high,med,low}='b0000000000011;
    4: {high,med,low}='b0000000000101;      //低音“5”
    5: {high,med,low}='b0000000000101;      //发 3 个时钟节拍
    6: {high,med,low}='b0000000000101;
    7: {high,med,low}='b0000000000110;      //低音“6”
    8: {high,med,low}='b000000010000;      //中音“1”
    9: {high,med,low}='b000000010000;      //发 3 个时钟节拍
    10: {high,med,low}='b000000010000;
    11: {high,med,low}='b000000100000;      //中音“2”
    12: {high,med,low}='b0000000000110;      //低音“6”
    13: {high,med,low}='b000000010000;
    14: {high,med,low}='b0000000000101;
    15: {high,med,low}='b0000000000101;

    16: {high,med,low}='b000001010000;      //中音“5”
    17: {high,med,low}='b000001010000;      //发 3 个时钟节拍
    18: {high,med,low}='b000001010000;
    19: {high,med,low}='b000100000000;      //高音“1”
    20: {high,med,low}='b000001100000;
    21: {high,med,low}='b000001010000;
    22: {high,med,low}='b000000110000;
    23: {high,med,low}='b000001010000;
    24: {high,med,low}='b000000100000;      //中音“2”
    25: {high,med,low}='b000000100000;      //持续 11 个时钟节拍
    26: {high,med,low}='b000000100000;
    27: {high,med,low}='b000000100000;
    28: {high,med,low}='b000000100000;

```

```

29: {high,med,low}='b0000000100000;
30: {high,med,low}='b0000000100000;
31: {high,med,low}='b0000000100000;

32: {high,med,low}='b0000000100000;
33: {high,med,low}='b0000000100000;
34: {high,med,low}='b0000000100000;
35: {high,med,low}='b0000000110000;           //中音“3”
36: {high,med,low}='b0000000000111;           //低音“7”
37: {high,med,low}='b0000000000111;
38: {high,med,low}='b0000000000110;           //低音“6”
39: {high,med,low}='b0000000000110;
40: {high,med,low}='b0000000000101;           //低音“5”
41: {high,med,low}='b0000000000101;
42: {high,med,low}='b0000000000101;
43: {high,med,low}='b0000000000110;           //低音“6”
44: {high,med,low}='b0000000010000;           //中音“1”
45: {high,med,low}='b0000000010000;
46: {high,med,low}='b0000000100000;           //中音“2”
47: {high,med,low}='b0000000100000;

48: {high,med,low}='b0000000000011;           //低音“3”
49: {high,med,low}='b0000000000011;
50: {high,med,low}='b0000000010000;           //中音“1”
51: {high,med,low}='b0000000010000;
52: {high,med,low}='b0000000000110;
53: {high,med,low}='b0000000000101;           //低音“5”
54: {high,med,low}='b0000000000110;
55: {high,med,low}='b0000000010000;           //中音“1”
56: {high,med,low}='b0000000000101;           //低音“5”
57: {high,med,low}='b0000000000101;           //持续8个时钟节拍
58: {high,med,low}='b0000000000101;
59: {high,med,low}='b0000000000101;
60: {high,med,low}='b0000000000101;
61: {high,med,low}='b0000000000101;
62: {high,med,low}='b0000000000101;
63: {high,med,low}='b0000000000101;

```

```
endcase
```

```
end
```

endmodule

限于篇幅，仅选取了“梁祝”乐曲中的一个片段进行演奏，读者可对照乐谱将完整的曲子补齐。

采用与本设计相同的方法，可以设计出演奏其他乐曲的电路。

11.5 自动售饮料机的设计

自动售饮料机是一个典型的利用状态机进行电路设计的例子。本例采用有限状态机设计，使用 **case** 语句来描述各个状态之间的转移关系。

先回顾一下 **case** 语句的使用。当多路选择的控制条件集中在某个变量的变化上时，用 **case** 语句加以表达十分方便且直观。**case** 语句最适于描述有限状态机，比如像 CPU 的译码部件的描述等。**case** 语句的用法如下：

```
case (敏感表达式)
    值 1: 块语句 1;
    值 2: 块语句 2;
    .....
    值 n: 块语句 n;
    default: 块语句 n+1;
endcase
```

当 **case** 语句中敏感表达式的值与值 1 相同时，执行块语句 1；当敏感表达式的值与值 2 相同时，执行块语句 2；依此类推，最后，如果敏感表达式的值与上面列出的值 1 到值 n 都不相同的话，则执行 **default** 后面的块语句 n+1。值 1 至值 n 必须各不相同，一旦判断与某值相同并执行相应块语句后，**case** 语句便告结束。如果某几个连续排列的值执行的是同一条语句，则这几个项间可用逗号隔开，而将执行语句放在这几个项的最后一个中。

下面用 **case** 语句设计一个自动售饮料机。假定每瓶饮料售价为 2.5 元，可使用 2 种硬币，即 5 角 (**half_dollar**)、1 元 (**one_dollar**)，机器有找零功能。

图 11.12 是该自动售饮料机的示意图。

机器设两个投币孔，分别接受 1 元和 5 角两种硬币，因硬币识别装置牵涉到传感器，在实验板上用两个按键来代替。有两个输出口，分别输出饮料和找零，还设两个灯，提示用户取走饮料和零钱，也可用声音来提醒。另外可设两个数码管，用于显示已投入的币值。在本设计中，该显示模块省略，如需要的话，读者可自行加上。

在下面的程序中，各种信号代表的含义如程序注释中所示。

Parameter 参数定义了 5 个状态，D 代表不同时刻的不同状态。在系统复位后机器开始运行，在每一次出售饮料的过程中由 D 记录其状态，表示投币者已投入钱币数目的变化，在下次售出前，首先由 **reset** 将系统清零。

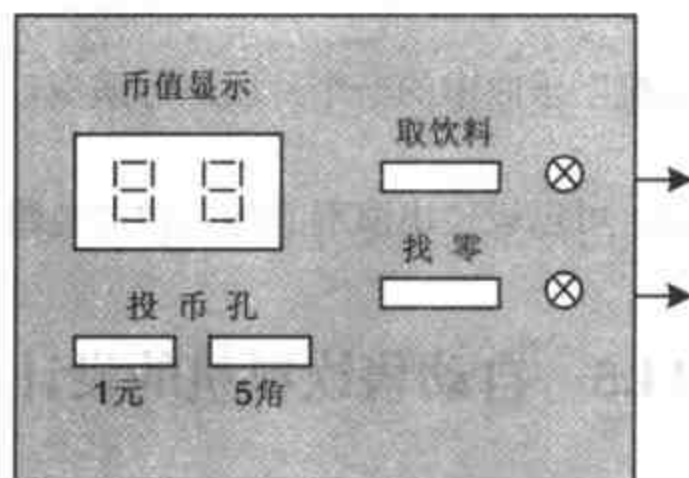


图 11.12 自动售饮料机示意图

程序中用 if-else 语句判断输入币值的变化。整个描述用一个过程块加以表示。有限状态机从本质上讲是由寄存器和组合逻辑构成的时序电路，各个状态之间的转移总是在时钟的触发下进行的，也可在设计时将时序逻辑部分和组合逻辑部分分在两个“always”过程块中进行描述，这样在综合时可以减少一些不必要的寄存器。

【例 11.7】自动售饮料机

/*信号定义:

clk: 时钟输入;

reset: 为系统复位信号;

half_dollar: 代表投入 5 角硬币;

one_dollar: 代表投入 1 元硬币;

half_out: 表示找零信号;

dispense: 表示机器售出一瓶饮料;

collect: 该信号用于提示投币者取走饮料。 */

```
module sell (one_dollar, half_dollar, collect, half_out, dispense, reset, clk);
```

```
parameter idle=0, one=2, half=1, two=3, three=4;
```

```
// idle, one, half, two, three 为中间状态变量, 代表投入币值的几种情况
```

```
input one_dollar, half_dollar, reset, clk;
```

```
output collect, half_out, dispense;
```

```
reg collect, half_out, dispense;
```

```
reg[2:0] D;
```

```
always @(posedge clk)
```

```
begin
```

```
if(reset)
```

```
begin
```

```
dispense=0; collect=0;
```

```
half_out=0; D=idle;
```

```
end
```

```
case (D)
```



```

        idle:
            if(half_dollar) D=half;
            else if(one_dollar)
                D=one;
        half:
            if(half_dollar) D=one;
            else if(one_dollar)
                D=two;
        one:
            if(half_dollar) D=two;
            else if(one_dollar)
                D=three;
        two:
            if(half_dollar) D=three;
            else if(one_dollar)
                begin
                    dispense=1;      //售出饮料
                    collect=1;      D=idle;
                end
        three:
            if(half_dollar)
                begin
                    dispense=1;      //售出饮料
                    collect=1;      D=idle;
                end
            else if (one_dollar)
                begin
                    dispense=1;      //售出饮料
                    collect=1;
                    half_out=1; D=idle;
                end
        endcase
    end
endmodule

```

11.6 实用多功能数字钟

本节用 Verilog HDL 设计一个实用的多功能数字钟。

11.6.1 功能

所设计数字钟具有下述功能：

- (1) 计时功能：包括时、分、秒的计时。
- (2) 定时与闹钟功能：能在设定的时间发出闹铃音。
- (3) 校时功能：对小时、分钟和秒能手动调整，以校准时间。
- (4) 整点报时功能：每逢整点产生“嘀嘀嘀嘀”四短一长的报时音。

图 11.13 是数字钟的结构示意图。

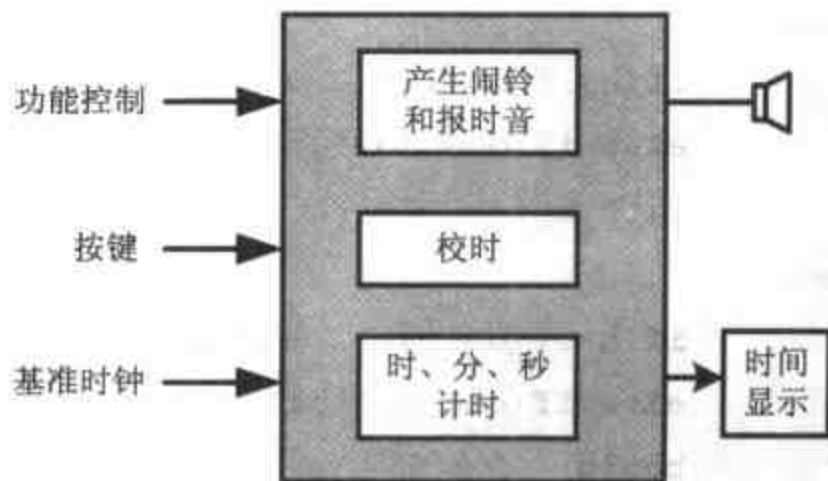


图 11.13 多功能数字钟示意图

11.6.2 源程序

以下是多功能数字钟的源程序。程序中各信号表示的含义如注释中所示。

【例 11.8】多功能数字钟

/* 信号定义：

```

clk:           标准时钟信号，本例中，其频率为 4Hz;
clk_1k:        产生闹铃音、报时音的时钟信号，本例中其频率为 1024Hz;
mode:          功能控制信号；为 0：计时功能；
               为 1：闹钟功能；
               为 2：手动校时功能；
turn:          接按键，在手动校时功能时，选择是调整小时还是分钟；
               若长时间按住该键，还可使秒信号清零，用于精确调时；
change:        接按键，手动调整时，每按一次，计数器加 1；
               如果长按，则连续快速加 1，用于快速调时和定时；
hour,min,sec:  此三信号分别输出并显示时、分、秒信号，
               皆采用 BCD 码计数，分别驱动 6 个数码管显示时间；
alert:         输出到扬声器的信号，用于产生闹铃音和报时音；
               闹铃音为持续 20 秒的急促的“嘀嘀嘀”音，若按住“change”键，
               则可屏蔽该音；整点报时音为“嘀嘀嘀嘀”四短一长音；
LD_alert:      接发光二极管，指示是否设置了闹钟功能；
LD_hour:       接发光二极管，指示当前调整的是小时信号；
LD_min:        接发光二极管，指示当前调整的是分钟信号。

```



```

*/
module clock(clk,clk_1k,mode,change,turn,alert,hour,min,sec,LD_alert,LD_
    hour,LD_min);
input  clk,clk_1k,mode,change,turn;
output alert,LD_alert,LD_hour,LD_min;
output[7:0]  hour,min,sec;
reg[7:0]  hour,min,sec,hour1,min1,sec1,ahour,amin;
reg[1:0]  m,fm,num1,num2,num3,num4;
reg[1:0]  loop1,loop2,loop3,loop4,sound;
reg  LD_hour,LD_min;
reg  clk_1Hz,clk_2Hz,minclk,hclk;
reg  alert1,alert2,ear;
reg  count1,count2,counta,countb;
wire  ct1,ct2,cta,ctb,m_clk,h_clk;

always @(posedge clk)
begin
    clk_2Hz<=~clk_2Hz;
    if(sound==3) begin sound<=0; ear<=1; end
                                //ear 信号用于产生或屏蔽声音
    else begin sound<=sound+1; ear<=0; end
end

always @(posedge clk_2Hz)      //由 4Hz 的输入时钟产生 1Hz 的时基信号
    clk_1Hz<=~clk_1Hz;

always @(posedge mode)        //mode 信号控制系统在三种功能间转换
begin if(m==2) m<=0; else m<=m+1; end

always @(posedge turn)
    fm<=~fm;

always                                //产生 count1,count2,counta,countb 四个信号
begin
    case(m)
        2:begin if(fm)
                begin count1<=change; {LD_min,LD_hour}<=2; end
                else
                begin counta<=change; {LD_min,LD_hour}<=1; end
                {count2,countb}<=0;
            end
    endcase
end

```

```
    end
    1:begin    if(fm)
                begin    count2<=change;  {LD_min,LD_hour}<=2;  end
                else
                begin    countb<=change; {LD_min,LD_hour}<=1;  end
                {count1,counta}<=2'b00;
    end
    default: {count1,count2,counta,countb,LD_min,LD_hour}<=0;
    endcase
end

always @(negedge clk)
    //如果长时间按下“change”键，则生成“num1”信号用于连续快速加1
    if(count2) begin
        if(loop1==3) num1<=1;
        else
        begin loop1<=loop1+1; num1<=0;  end
        end
    else begin loop1<=0; num1<=0;  end
always @(negedge clk)                //产生 num2 信号
    if(countb) begin
        if(loop2==3) num2<=1;
        else
        begin loop2<=loop2+1; num2<=0;  end
        end
    else begin loop2<=0; num2<=0;  end
always @(negedge clk)
    if(count1) begin
        if(loop3==3) num3<=1;
        else
        begin loop3<=loop3+1; num3<=0;  end
        end
    else begin loop3<=0; num3<=0;  end
always @(negedge clk)
    if(counta) begin
        if(loop4==3) num4<=1;
        else
        begin loop4<=loop4+1; num4<=0;  end
        end
    end
```

```

    else begin loop4<=0; num4<=0; end

    assign ct1=(num3&clk)|(!num3&m_clk);           //ct1 用于计时、校时中的分钟计数
    assign ct2=(num1&clk)|(!num1&count2);          //ct2 用于在定时状态下调整分钟信号
    assign cta=(num4&clk)|(!num4&h_clk);           //cta 用于计时、校时中的小时计数
    assign ctb=(num2&clk)|(!num2&countb);          //ctb 用于在定时状态下调整小时信号

    always @(posedge clk_1Hz)                      //秒计时和秒调整进程
    begin
        if(!(sec1^8'h59)|turn&(!m))
            begin
                sec1<=0; if(!(turn&(!m))) minclk<=1;
            end
            //按住“turn”按键一段时间，秒信号可清零，该功能用于手动精确调时
        else begin
            if(sec1[3:0]==4'b1001)
                begin sec1[3:0]<=4'b0000; sec1[7:4]<=sec1[7:4]+1; end
            else sec1[3:0]<=sec1[3:0]+1; minclk<=0;
            end
        assign m_clk=minclk||count1;

        always @(posedge ct1)                      //分计时和分调整进程
        begin
            if(min1==8'h59) begin min1<=0; hclk<=1; end
            else begin
                if(min1[3:0]==9)
                    begin min1[3:0]<=0; min1[7:4]<=min1[7:4]+1; end
                else min1[3:0]<=min1[3:0]+1; hclk<=0;
                end
            end
        assign h_clk=hclk||counta;

        always @(posedge cta)                      //小时计时和小时调整进程
        begin
            if(hour1==8'h23) hour1<=0;
            else if(hour1[3:0]==9)
                begin hour1[7:4]<=hour1[7:4]+1; hour1[3:0]<=0; end
            else hour1[3:0]<=hour1[3:0]+1;

            always @(posedge ct2)                  //闹钟定时功能中的分钟调节进程
            begin
                if(amin==8'h59) amin<=0;
            end
        end
    end

```

```

        else if(amin[3:0]==9)
            begin amin[3:0]<=0; amin[7:4]<=amin[7:4]+1; end
        else amin[3:0]<=amin[3:0]+1;

always @(posedge ctb) //闹钟定时功能中的小时调节进程
    if(ahour==8'h23) ahour<=0;
    else if(ahour[3:0]==9)
        begin ahour[3:0]<=0; ahour[7:4]<=ahour[7:4]+1; end
    else ahour[3:0]<=ahour[3:0]+1;

always //闹铃功能
    if((min1==amin)&&(hour1==ahour)&&(amin|ahour)&&(!change))
        //若按住“change”键不放,可屏蔽闹铃音

    if(sec1<8'h20) alert1<=1; //控制闹铃的时间长短
    else alert1<=0;
    else alert1<=0;

always //时、分、秒的显示控制
    case(m)
        3'b00: begin hour<=hour1; min<=min1; sec<=sec1; end
            //计时状态下的时、分、秒显示
        3'b01: begin hour<=ahour; min<=amin; sec<=8'hzz; end
            //定时状态下的时、分、秒显示
        3'b10: begin hour<=hour1; min<=min1; sec<=8'hzz; end
            //校时状态下的时、分、秒显示
    endcase

assign LD_alert=(ahour|amin)?1:0; //指示是否进行了闹铃定时
assign alert=((alert1)?clk_1k&clk:0)|alert2; //产生闹铃音或整点报时音

always //产生整点报时信号 alert2
    begin
        if((min1==8'h59)&&(sec1>8'h54)||(!min1|sec1))
            if(sec1>8'h54) alert2<=ear&clk_1k; //产生短音
            else alert2<=!ear&clk_1k; //产生长音
            else alert2<=0;
    end
endmodule

```

如图 11.14 所示是多功能数字钟的端口示意图。

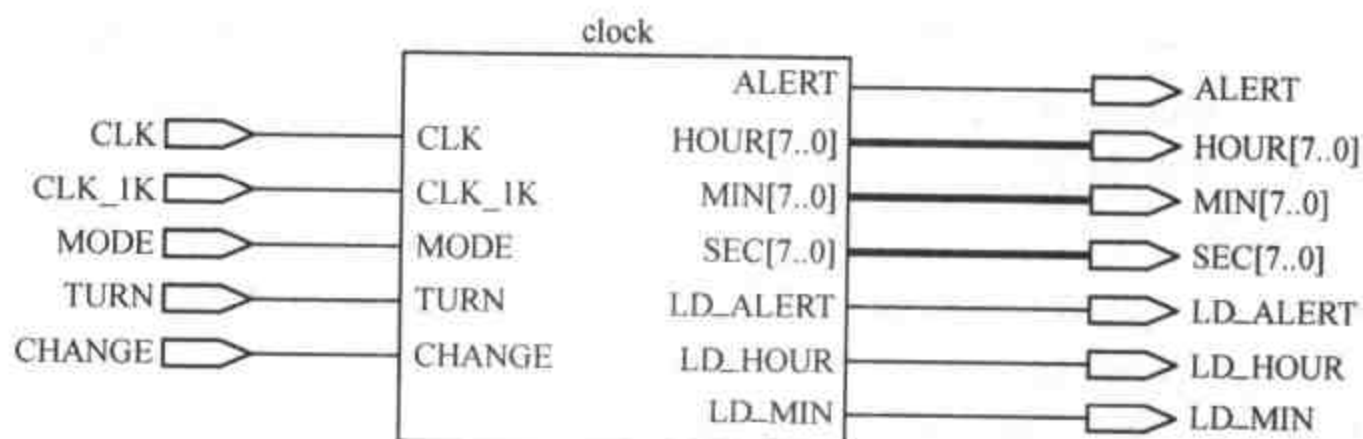


图 11.14 多功能数字钟端口示意图

可在该设计的基础上进一步增加功能，如增加日历功能，使之能够显示年、月、日等；增加跑表功能，使之能对百分秒进行计数。

11.7 计数器设计

在本节中，设计一个模拟 IC 卡电话计费的计费器模型，该计费器的端口如图 11.15 所示。话卡插入后，计费器能将卡中的币值读出并显示出来，在通话过程中，根据话务种类计话费并将话费从卡值中扣除，卡值余额每分钟更新一次，计时与计费数据均以十进制形式显示。

简单地将话务只分为三类：市话、长话和特话，其中市话按每分钟 3 角钱计费，长话按每分钟 6 角钱计费，特话不收费。当卡上余额不足时产生告警信号，告警时间达到一定长度则自动切断当前通话。

以下是电话计费器模型的源程序。

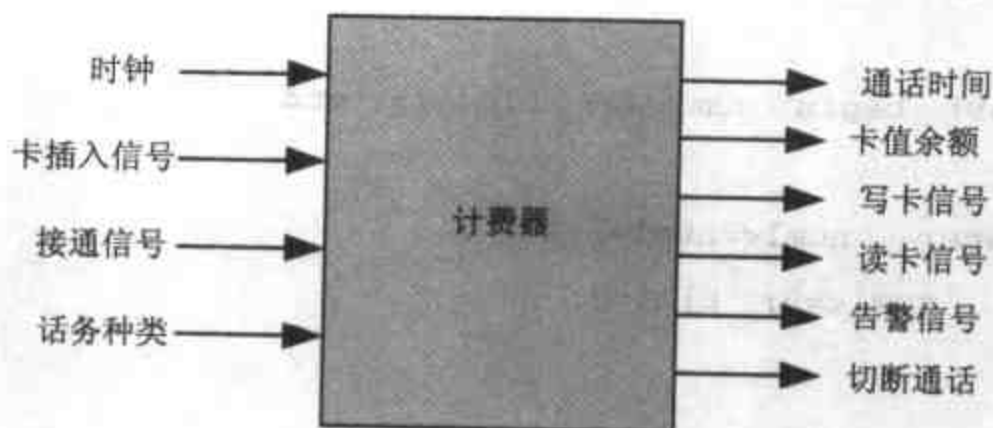


图 11.15 电话计费器端口示意图

【例 11.9】电话计费器程序

/*信号定义:

clk: 时钟信号，本例中其频率值为 1Hz;

decide: 电话局反馈回来的信号，代表话务种类，“01”表示市话，“10”表示长话，“11”表示特话;

dispmoney: 用来显示卡内余额，其单位为角，这里假定能显示的最大数额为 50 元（500 角）;

disptime: 显示本次通话的时长;

write,read: 当 write 信号下降沿到来时写卡，当话卡插入，read 信号变高时读卡;

warn: 余额过少时的告警信号。本例中, 当打市话时, 余额少于 3 角, 打长
 话时, 余额少于 6 角, 即会产生告警信号;
 cut: 当告警时间过长时自动切断通话信号。 */

```

module account(state,clk,card,decide,disptime,dispmoney,
               write,read,warn,cut);
output write,read,warn,cut;
input state,clk,card;
input[2:1] decide;
output[10:0] dispmoney;
output[8:0] disptime;
reg[10:0] money;
reg[8:0] dtime;
reg warn,cut,write,t1m;                //t1m 为分时钟
reg set,reset_ena;
integer num1,temp;

assign dispmoney=card?money:0;
assign disptime=dtime;
assign read=card?1:0;

                                   //产生分时钟
always @(posedge clk)
begin
    if (num1==59) begin num1<=0; t1m<=1; end
    else begin
        if(state) num1<=num1+1;
        else num1<=0; t1m<=0;
    end
end

always @(negedge clk)                //该进程完成电话计费功能
begin
    if(!set)
        begin money<=11'h500; set<=1; end
    if(card&state)
        if(t1m)
            case ({state,decide})
                3'b101: if(money<3)
                    begin warn<=1; write<=0; reset_ena<=1; end
            endcase
        end
    end

```



```

else
begin
//市话计费
if (money[3:0]<4'b0011)
begin
money[3:0]<=money[3:0]+7;
if (money[7:4]!=0)
money[7:4]<=money[7:4]-1;
else
begin money[7:4]<=9; money[10:8]<=money[10:8]-1; end
end
else money[3:0]<=money[3:0]-3; write<=1;

//市话通话计时
if (dtime[3:0]==9)
begin
dtime[3:0]<=0;
if (dtime[7:4]==9)
begin dtime[7:4]<=0; dtime[8]<=dtime[8]+1; end
else dtime[7:4]<=dtime[7:4]+1;
end
else
begin dtime[3:0]<=dtime[3:0]+1; warn<=0; reset_ena<=0;
end
end
3'b110: if (money<6)
begin warn<=1; write<=0; reset_ena<=1; end
else begin

//通话计时
if (dtime[3:0]==9)
begin
dtime[3:0]<=0; if (dtime[7:4]==9)
begin dtime[7:4]<=0; dtime[8]<=dtime[8]+1; end
else dtime[7:4]<=dtime[7:4]+1;
end
else dtime[3:0]<=dtime[3:0]+1;

//长话计费
if (money[3:0]<4'b0110)
begin
money[3:0]<=money[3:0]+4;
if (!money[7:4])

```

```

        begin money[7:4]<=9; money[10:8]<=money[10:8]-1; end
        else money[7:4]<=money[7:4]-1;
        end
        else money[3:0]<=money[3:0]-6;
        write<=1; reset_ena<=0; warn<=0;
        end
    endcase

    else write<=0;

    else begin dtime<=0; warn<=0; write<=0; reset_ena<=0; end
            //取卡后对一些信号进行复位

end

always @(posedge clk) //该进程在告警时间过长的情况下切断本次通话
begin
    if(warn) temp<=temp+1;
    else temp<=0;
    if(temp==15)
        begin cut<=1; temp<=0; end
    if(!card||!reset_ena)
        begin
            cut<=0; //复位 cut 信号
            temp<=0;
        end
    end
end
endmodule

```

将上面的源程序进行编译和仿真，图 11.16 所示是其仿真波形的一段，从该段波形可看出，当卡中余额过少时，就会产生告警信号，告警信号持续数秒后，便会自动切断当前通话。还可对计费器的其他各项功能进行仿真和验证。

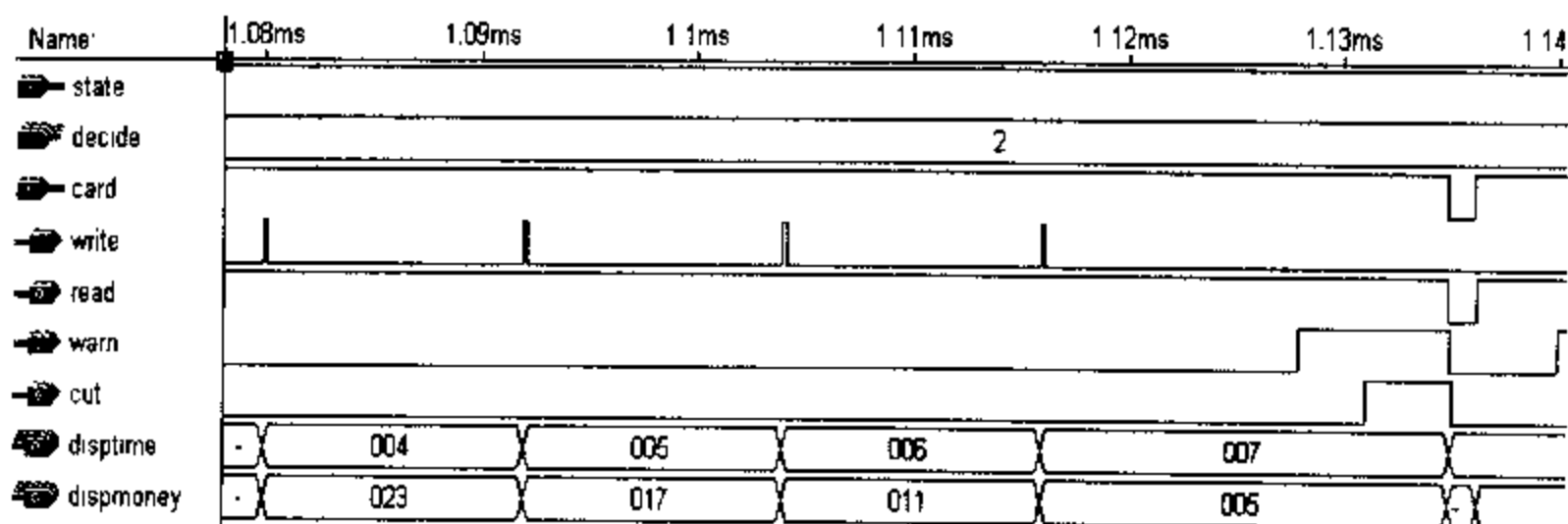


图 11.16 计费器仿真波形

思考与练习

1. 编写一个交替显示 26 个英文字母的电路。
2. 设计一个乐曲演奏电路，实现乐曲“铃儿响叮当”的循环演奏。
3. 编写一个乒乓球游戏电路，自己规定游戏规则。
4. 设计一个 8 层楼的电梯控制器，要求每层楼都应该有请求开关，电梯内有上、下和开门开关。
5. 设计一个功能类似 8255 芯片的电路。
6. 为 11.6 节的数字钟增加日历功能，使之能够显示年、月、日。
7. 试设计一个用 FPGA 产生正弦波（sin）信号的电路，并将 11.4 节乐曲演奏电路中驱动扬声器的方波信号改用正弦波信号驱动，并比较效果。

第 12 章 算法与复杂逻辑的实现

本章内容简介

- ◆ 级连加法器
- ◆ 并行加法器
- ◆ 超前进位加法器
- ◆ 流水线加法器
- ◆ 并行乘法器
- ◆ 移位相加乘法器
- ◆ 查找表乘法器
- ◆ 加法树乘法器
- ◆ FIR 滤波器
- ◆ 数字相关器
- ◆ 信道编译码器
- ◆ CRC 校验码

本章讨论算法以及一些复杂逻辑的实现方法，并以加法器、乘法器、FIR 滤波器及信道编译码器的设计为例介绍在数字信号处理和数字通信等领域中常用数字模块的设计实现方法。

12.1 加法器设计

加法、乘法作为一种基本运算，大量应用在数字信号处理和数字通信的各种算法中。由于加法器、乘法器使用频繁，因此其速度往往影响着整个系统的运行速度。如果可以实现快速加法器和快速乘法器的设计，则可以提高整个系统的处理速度。

加法运算是最基本的算术运算，在多数情况下，无论是乘法、除法还是减法以及 FFT 等运算，最终都可以分解为加法运算来实现，因此对加法运算的实现进行一些研究是非常必要的。实现加法运算有如下一些方法：

- ◆ 级连加法器；
- ◆ 并行加法器；
- ◆ 超前进位加法器；

◆ 流水线加法器。

上述方法各有其特点，下面分别进行介绍。

12.1.1 级连加法器

级连加法器是由1位全加器级连构成的，本级的进位输出作为下一级的进位输入，如图12.1所示是8位级连加法器的结构示意图，它由8个1位全加器构成。

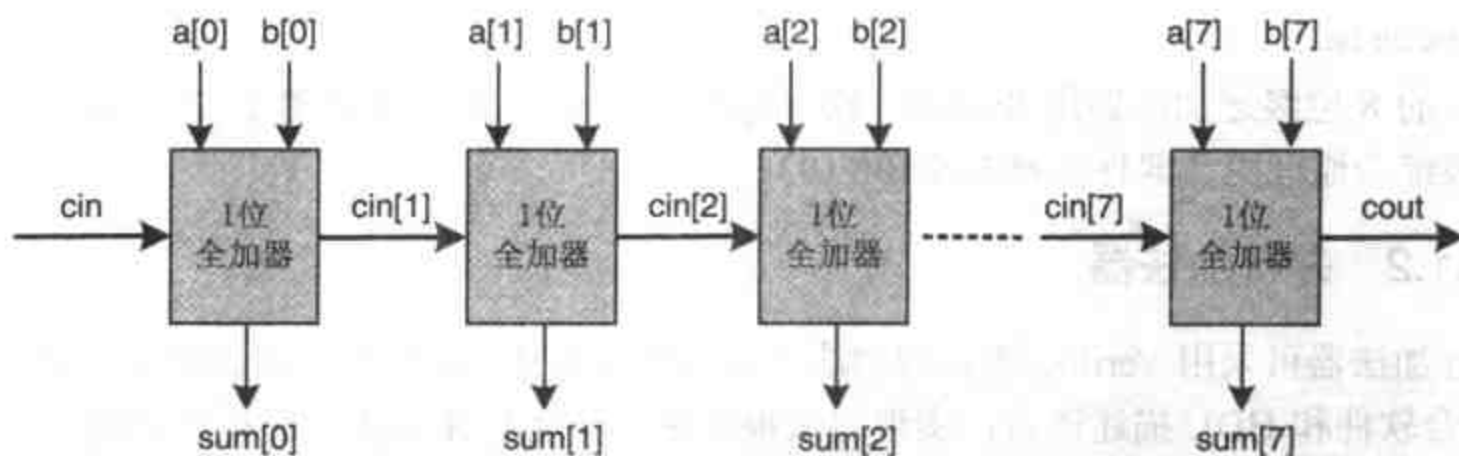


图 12.1 8位级连加法器的结构示意图

级连加法器的结构简单，但 N 位级连加法运算的延时是1位全加器的 N 倍，延时主要是由于进位信号级连造成的。在需要高性能的设计中，这种加法结构不宜采用。

如下例所示是8位级连加法器的设计程序，它采用调用1位全加器来构成。

【例 12.1】8位级连加法器

```
module add_jl(sum, cout, a, b, cin);
```

```
output [7:0] sum;
```

```
output cout;
```

```
input [7:0] a, b;
```

```
input cin;
```

```
full_add1 f0 (a[0],b[0],cin,sum[0],cin1);
```

//级连描述

```
full_add1 f1 (a[1],b[1],cin1,sum[1], cin2);
```

```
full_add1 f2 (a[2],b[2],cin2,sum[2], cin3);
```

```
full_add1 f3 (a[3],b[3],cin3,sum[3], cin4);
```

```
full_add1 f4 (a[4],b[4],cin4,sum[4], cin5);
```

```
full_add1 f5 (a[5],b[5],cin5,sum[5], cin6);
```

```
full_add1 f6 (a[6],b[6],cin6,sum[6], cin7);
```

```
full_add1 f7 (a[7],b[7],cin7,sum[7], cout);
```

```
endmodule
```

```
module full_add1(a, b, cin, sum, cout);
```

//1位全加器

```
input a, b, cin;
```

```
output sum, cout;
```

```

wire s1,m1, m2, m3;
and (m1, a, b),
    (m2, b, cin),
    (m3, a, cin);
xor (s1, a, b),
    (sum, s1, cin);
or (cout, m1, m2, m3);
endmodule

```

上面的 8 位级连加法器用 Synplify 软件综合后的 RTL 原理图如图 12.2 所示, 而图 12.3 是其门级综合原理图 (器件选择 EPF10K10)。

12.1.2 并行加法器

并行加法器可采用 Verilog 语言的加法运算符直接描述, 或称为数据流描述方式, 借助于 EDA 综合软件和 HDL 描述语言, 实现起来很容易, 其运算速度快, 但耗用资源多, 尤其是当加法运算的位数较宽时, 其耗用的资源将会比较大。例 12.2 是用数据流描述方式设计的加法器, 它只需要一条语句即可实现。

【例 12.2】8 位并行加法器

```

module add_bx(cout,sum,a,b,cin);
output[7:0] sum;
output cout;
input[7:0] a,b;
input cin;
    assign {cout,sum}=a+b+cin;
endmodule

```

上面的并行加法器用 Synplify 软件综合, 如图 12.4 所示是其 RTL 综合原理图, 而图 12.5 是其门级综合原理图 (器件选择 EPF10K10)。

12.1.3 超前进位加法器

级连加法器的延时主要是由于进位的延时造成的, 因此要加快加法器的运算速度, 就必须减小进位延迟, 超前进位链能有效地减少进位的延迟。

超前进位的推导在很多书籍和资料中都能找到, 这里只以 4 位超前进位链的推导为例介绍超前进位的概念。

首先, 对于 1 位全加器, 其本位值和与进位输出可表示如下:

$$\text{sum} = a \oplus b \oplus \text{cin}$$

$$\text{cout} = (a \cdot b) + (a \cdot \text{cin}) + (b \cdot \text{cin}) = ab + (a+b)\text{cin}$$

从上面的式子可以看出: 如 a 和 b 都为 1, 则进位输出为 1; 如 a 和 b 有一个为 1, 则进位输出等于 cin 。

令 $G=ab$, $P=a+b$, 则有:

$$\text{cout} = ab + (a+b)\text{cin} = G + P \cdot \text{cin}$$

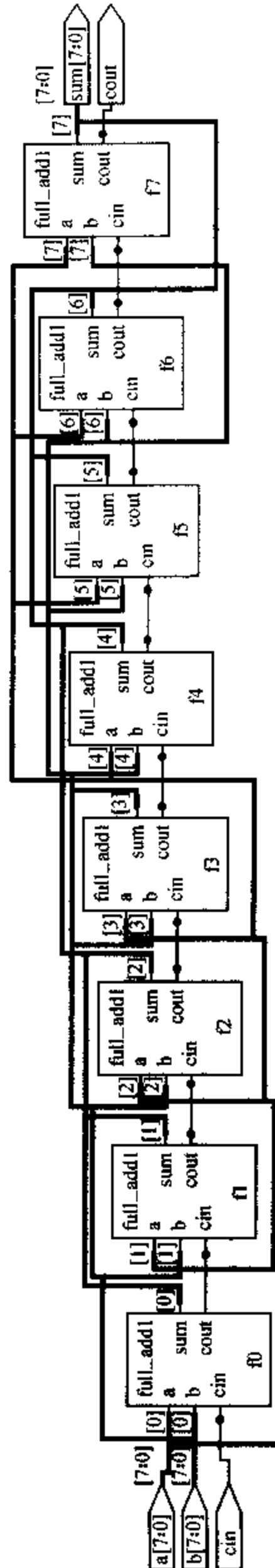


图 12.2 8 位级连加法器 RTL 综合原理图

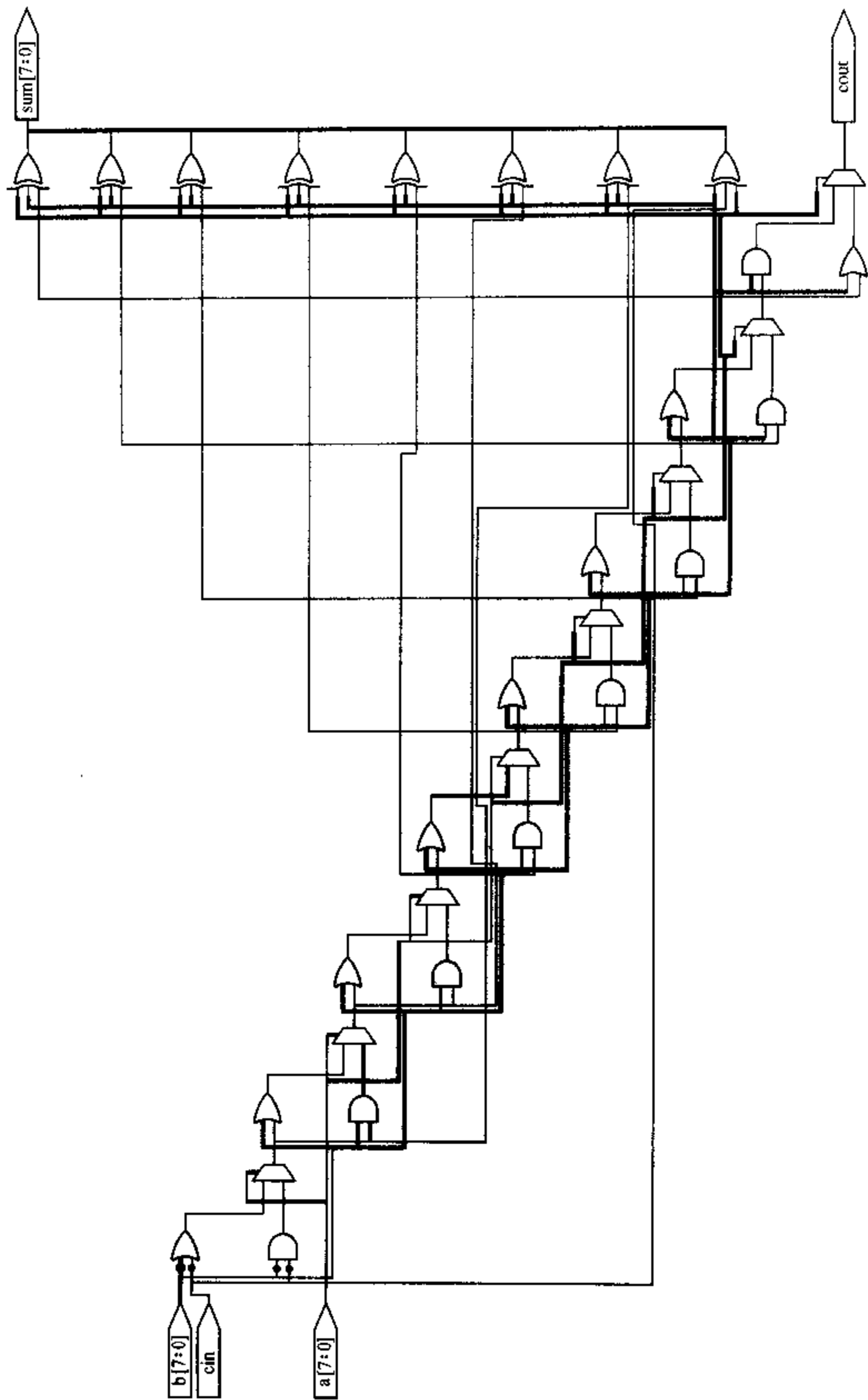


图 12.3 8 位级连加法器的门级综合原理图

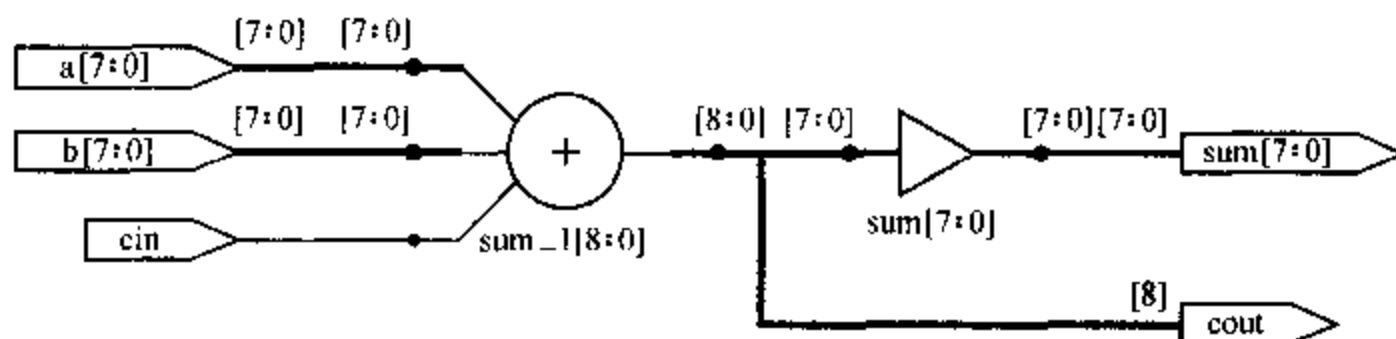


图 12.4 8 位并行加法器 RTL 综合原理图

由此可以得出用 G 和 P 来写出 4 位超前进位链如下（设定 4 位被加数和加数为 A 和 B ，进位输入为 C_{in} ，进位输出为 C_{out} ，进位产生 $G_i=A_iB_i$ ，进位传输 $P_i=A_i+B_i$ ）：

$$C_0=C_{in}$$

$$C_1=G_0+P_0C_0=G_0+P_0C_{in}$$

$$C_2=G_1+P_1C_1=G_1+P_1(G_0+P_0C_{in})=G_1+P_1G_0+P_1P_0C_{in}$$

$$C_3=G_2+P_2C_2=G_2+P_2(G_1+P_1C_1)=G_2+P_2G_1+P_2P_1G_0+P_2P_1P_0C_{in}$$

$$C_4=G_3+P_3C_3=G_3+P_3(G_2+P_2C_2)=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0+P_3P_2P_1P_0C_{in}$$

$$C_{out}=C_4$$

由上面的超前进位链可以看出：各个进位彼此独立产生，将进位级连传播给去掉了，因此，减小了进位产生的延迟时间。

同样，可推出下面的式子：

$$SUM=A \oplus B \oplus C_{in}=(AB) \oplus (A+B) \oplus C_{in} \quad U=G \oplus P \oplus C_{in}$$

下例是超前进位链 8 位加法器的 Verilog HDL 描述。

【例 12.3】8 位超前进位加法器

```
module add_ahead(sum, cout, a, b, cin);
```

```
output[7:0] sum;
```

```
output cout;
```

```
input [7:0] a, b;
```

```
input cin;
```

```
wire[7:0] G,P;
```

```
wire[7:0] C,sum;
```

```
assign G[0]=a[0]&b[0];
```

```
//产生第 0 位本位值和进位值
```

```
assign P[0]=a[0]|b[0];
```

```
assign C[0]=cin;
```

```
assign sum[0]=G[0]^P[0]^C[0];
```

```
assign G[1]=a[1]&b[1];
```

```
//产生第 1 位本位值和进位值
```

```
assign P[1]=a[1]|b[1];
```

```
assign C[1]=G[0]|(P[0]&cin);
```

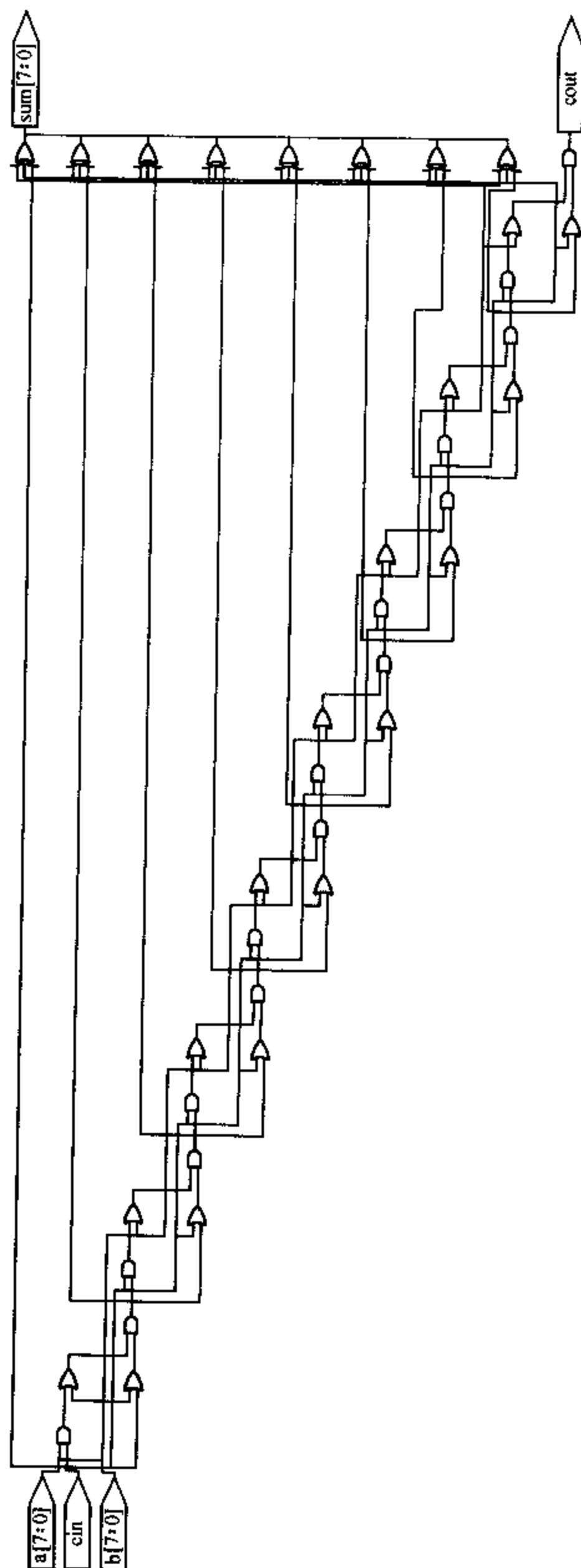


图 12.5 8 位并行加法器的门级综合原理图


```

assign sum[1]=G[1]^P[1]^C[1];
assign G[2]=a[2]&b[2];           //产生第 2 位本位值和进位值
assign P[2]=a[2]|b[2];
assign C[2]=G[1]|(P[1]&C[1]);
assign sum[2]=G[2]^P[2]^C[2];

assign G[3]=a[3]&b[3];           //产生第 3 位本位值和进位值
assign P[3]=a[3]|b[3];
assign C[3]=G[2]|(P[2]&C[2]);
assign sum[3]=G[3]^P[3]^C[3];

assign G[4]=a[4]&b[4];           //产生第 4 位本位值和进位值
assign P[4]=a[4]|b[4];
assign C[4]=G[3]|(P[3]&C[3]);
assign sum[4]=G[4]^P[4]^C[4];

assign G[5]=a[5]&b[5];           //产生第 5 位本位值和进位值
assign P[5]=a[5]|b[5];
assign C[5]=G[4]|(P[4]&C[4]);
assign sum[5]=G[5]^P[5]^C[5];

assign G[6]=a[6]&b[6];           //产生第 6 位本位值和进位值
assign P[6]=a[6]|b[6];
assign C[6]=G[5]|(P[5]&C[5]);
assign sum[6]=G[6]^P[6]^C[6];

assign G[7]=a[7]&b[7];           //产生第 7 位本位值和进位值
assign P[7]=a[7]|b[7];
assign C[7]=G[6]|(P[6]&C[6]);
assign sum[7]=G[7]^P[7]^C[7];

assign cout=G[7]|(P[7]&C[7]);    //产生最高位进位输出
endmodule

```

上面设计的超前进位加法器用 Synplify 软件综合, 如图 12.6 所示是其 RTL 综合原理图。

将上面 3 种加法器综合到 EPF10K10LC84-3 器件中, 并比较它们耗用的资源, 结果如表 12.1 中所示, 可发现, 并行加法器耗用资源最多。

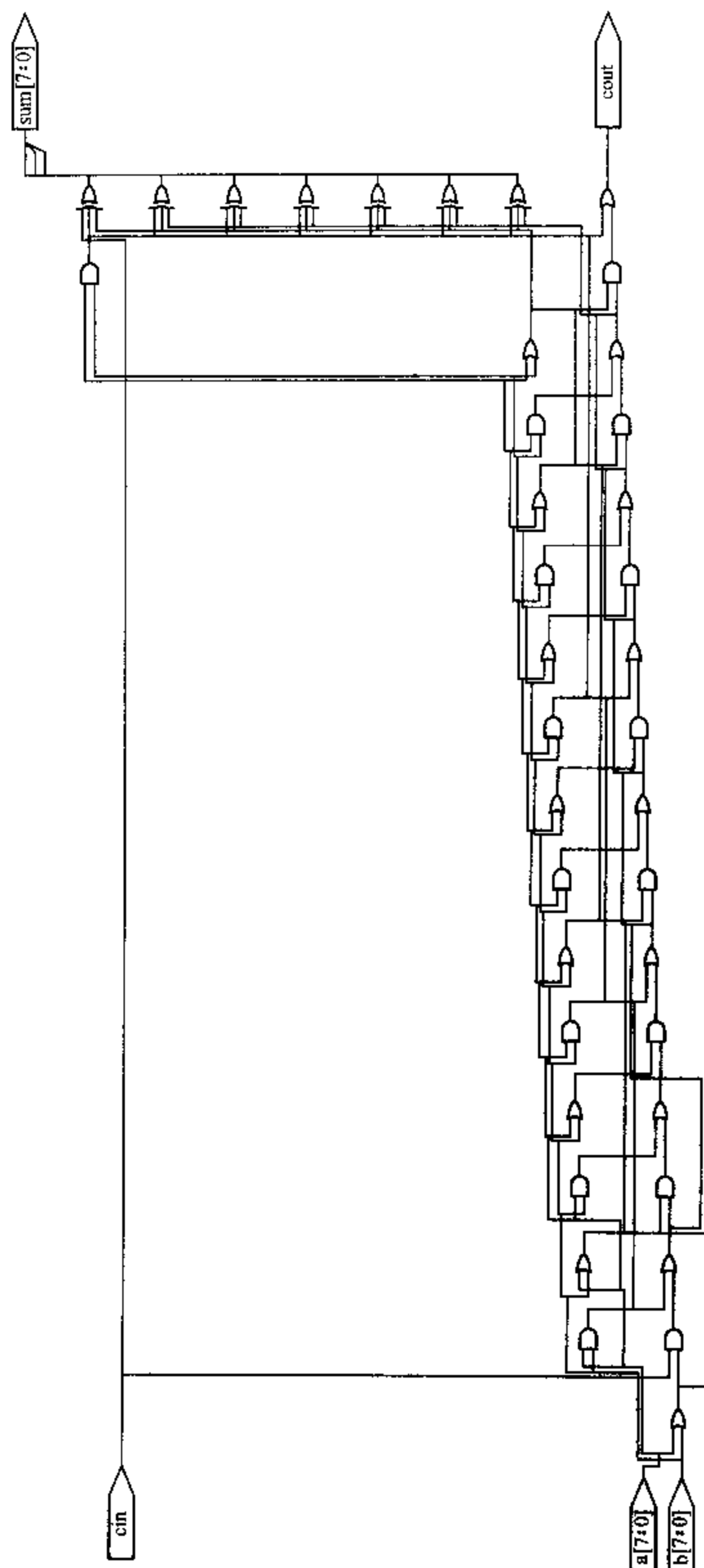


图 12.6 8 位超前进位加法器 RTL 综合原理图

表 12.1 3种加法器耗用资源的比较

	级连加法器	并行加法器	超前进位加法器
资源耗用 (LC)	16	22	16

12.1.4 流水线加法器

以上实现的加法器虽然尽量减少了延时,但还是不可避免地有多级门延时和布线延时,随着位数的增加,延迟还会增加。为了设计更快的加法器,可以采用流水线设计技术(Pipeline),以提高系统的运行频率。其基本思想是在逻辑电路中加入若干寄存器来暂存中间结果,虽然多用了一些寄存器资源,但减少了每一级组合电路的时延,因此可提高整个加法器的运行频率。

流水线加法器的设计可参看 10.2 节的有关内容,这里不再重复。

12.2 乘法器设计

乘法器也频繁使用在数字信号处理和数字通信的各种算法中,并且往往影响着整个系统的运行速度,所以如果能实现快速乘法器的设计,则可提高整个系统的处理速度。

本节用如下几种方法实现乘法运算:

- ◆ 并行乘法器;
- ◆ 移位相加乘法器;
- ◆ 查找表乘法器;
- ◆ 加法树乘法器。

这些方法各有其优缺点,分别进行介绍。

12.2.1 并行乘法器

并行乘法器是纯组合类型的乘法器,完全由逻辑门实现。Verilog 语言支持乘法运算,有乘法操作符,因此用 Verilog 语言设计并行乘法器非常简单,只需要一条语句即可实现,如下所示。

【例 12.4】8 位并行乘法器

```

module mult(outcome, a, b);
parameter size=8;
input [size:1] a,b;           //两个操作数
output [2*size:1] outcome;    //结果
assign outcome=a*b;          //乘法运算符
endmodule
```

上面设计的并行乘法器借助于 EDA 综合软件和 HDL 描述语言,实现起来很容易,其运算速度快,但耗用资源多,尤其是当乘法运算的位数较宽时,其耗用的资源将会很庞大。如图 12.7 所示是例 12.4 用 Synplify 软件综合后的 RTL 视图,而图 12.8 是其门级综合结果。从

这个综合结果可以看出，该 8 位乘法器耗用资源较多，因此这种乘法器在实用中受到了一定的限制，尤其是当操作数位数较宽时。

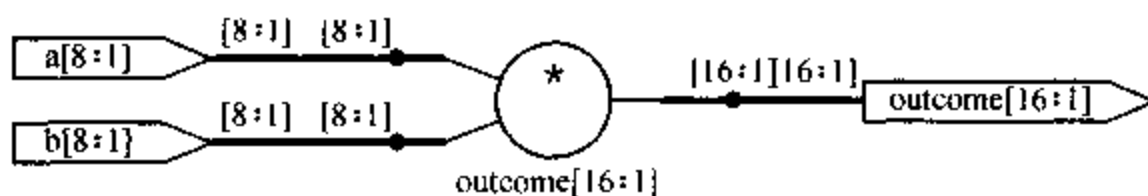


图 12.7 8 位并行乘法器的 RTL 综合原理图

12.2.2 移位相加乘法器

移位相加乘法器是一种最节省资源的乘法器设计方法，其设计思路是：乘法通过逐项移位相加来实现，根据乘数的每一位是否为 1 进行计算，若为 1 则将被乘数移位相加。这种方法硬件资源耗用少，8 位的乘法器只需要一个 16 位移位寄存器和一个 16 位加法器即可实现。

12.2.3 查找表乘法器

查找表乘法器将乘积直接存放在存储器中，将操作数（乘数和被乘数）作为地址访问存储器，得到的输出数据就是乘法运算的结果。查找表乘法器的速度只局限于所使用存储器的存取速度。但由于查找表的规模随着操作数位数的增加而迅速增大，因此不宜用于实现位数高的乘法操作。

在小型查找表的基础上结合加法器可以构成位数较高的乘法器。例如，8 位乘法器 $Y=a \times b$ 可以分解成两个半字节，其中 $a=A_I \times 2^4 + A_{II}$, $b=B_I \times 2^4 + B_{II}$ 。由此，乘式可写成：

$$Y=(A_I \times 2^4 + A_{II}) \times (B_I \times 2^4 + B_{II}) = A_I \times B_I \times 2^8 + A_{II} \times B_I \times 2^4 + A_I \times B_{II} \times 2^4 + A_{II} \times B_{II}$$

这样就把 8 位 $\times$ 8 位乘法运算转化为 4 个 4 位 $\times$ 4 位乘法运算。相对而言，减小了查找表的尺寸。

下面的例子基于查找表实现了一个 4 位 $\times$ 4 位的乘法器。

【例 12.5】4 $\times$ 4 查找表乘法器

```
module mult4x4 (out,a,b,clk);
    output [7:0] out;
    input [3:0] a,b;
    input clk;
    reg [7:0] out;
    reg [1:0] firsta,firstb;
    reg [1:0] seconda,secondb;
    wire [3:0] outa,outb,outc,outd;
    always @(posedge clk)
    begin
        firsta = a[3:2]; seconda = a[1:0];
        firstb = b[3:2]; secondb = b[1:0];
    end
end
```

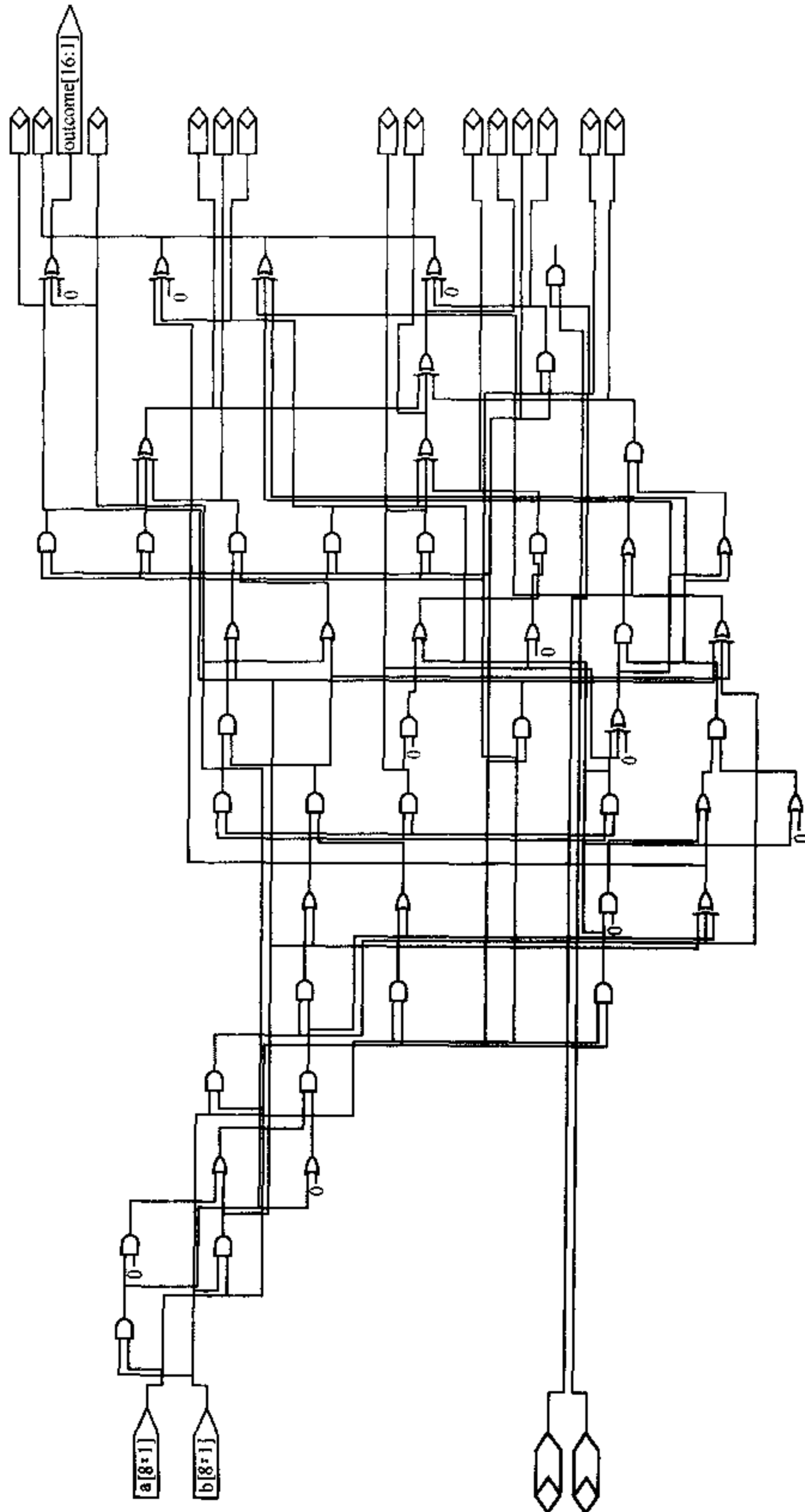


图 12.8 8 位并行乘法器的门级综合原理图

```
lookup m1(outa,firsta,firstb,clk),
      m2(outb,firsta,secondb,clk),
      m3(outc,seconda,firstb,clk),
      m4(outd,seconda,secondb,clk);    //模块调用

always @(posedge clk)
begin
    out = (outa << 4) + (outb << 2) + (outc << 2) + outd;
end
endmodule
```

```
module lookup(out,a,b,clk);                //用查找表方式实现 2×2 乘法
output[3:0] out;
input[1:0] a,b;
input clk;
reg[3:0] out;
reg[3:0] address;
always @(posedge clk)
begin
    address = {a,b};
    case(address)
        4'h0 : out = 4'b0000;
        4'h1 : out = 4'b0000;
        4'h2 : out = 4'b0000;
        4'h3 : out = 4'b0000;
        4'h4 : out = 4'b0000;
        4'h5 : out = 4'b0001;
        4'h6 : out = 4'b0010;
        4'h7 : out = 4'b0011;
        4'h8 : out = 4'b0000;
        4'h9 : out = 4'b0010;
        4'ha : out = 4'b0100;
        4'hb : out = 4'b0110;
        4'hc : out = 4'b0000;
        4'hd : out = 4'b0011;
        4'he : out = 4'b0110;
        4'hf : out = 4'b1001;
        default : out = 'bx;
    endcase
end
```



```

end
endmodule

```

图 12.9 是用 MAX+PLUS II 软件对查找表乘法器进行功能仿真的波形图 (图中数据为十六进制形式)。

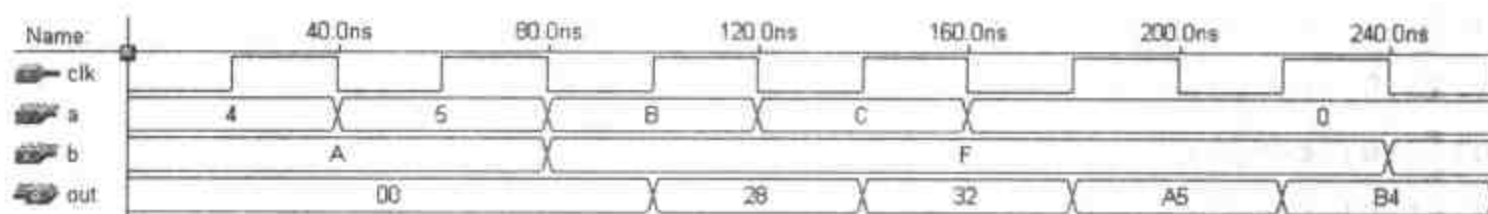


图 12.9 查找表乘法器仿真波形

12.2.4 加法树乘法器

加法树结合了移位相加乘法器和查找表乘法器的优点, 其示意图如图 12.10 所示。

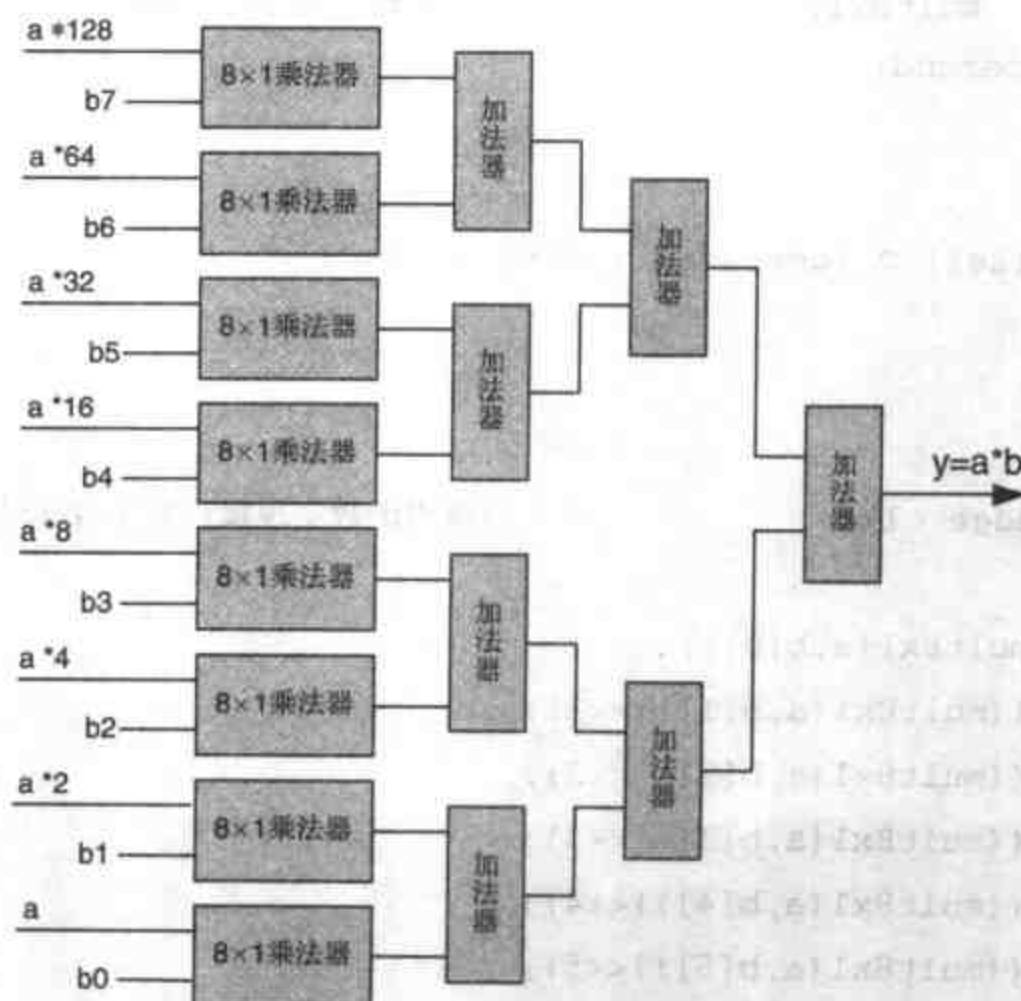


图 12.10 加法树乘法器示意图

加法树乘法器需要增加若干寄存器以暂存数据, 增加了芯片资源的耗用, 但加法树乘法器能在一个时钟周期内完成两数相乘, 提高了运算速度。

下面是加法树乘法器的 Verilog HDL 源代码。

【例 12.6】8 位加法树乘法器

```

module add_tree(out,a,b,clk);
output[15:0] out;
input[7:0] a,b;
input clk;
wire[15:0] out;

```

```

    wire[14:0] out1,c1;
    wire[12:0] out2;
    wire[10:0] out3,c2;
    wire[8:0] out4;
    reg[14:0] temp0;
    reg[13:0] temp1;
    reg[12:0] temp2;
    reg[11:0] temp3;
    reg[10:0] temp4;
    reg[9:0] temp5;
    reg[8:0] temp6;
    reg[7:0] temp7;

    function[7:0] mult8x1;           //该函数实现 8×1 乘法
    input[7:0] operand;
    input sel;

    begin
        mult8x1= (sel) ? (operand) : 8'b00000000;
    end
endfunction

always @(posedge clk)             //调用函数实现操作数 b 各位与操作数 a 的相乘
begin
    temp7<= mult8x1(a,b[0]);
    temp6<= ((mult8x1(a,b[1]))<<1);
    temp5<= ((mult8x1(a,b[2]))<<2);
    temp4<= ((mult8x1(a,b[3]))<<3);
    temp3<= ((mult8x1(a,b[4]))<<4);
    temp2<= ((mult8x1(a,b[5]))<<5);
    temp1<= ((mult8x1(a,b[6]))<<6);
    temp0<= ((mult8x1(a,b[7]))<<7);
end

assign out1 = temp0 + temp1;       //加法器树运算
assign out2 = temp2 + temp3;
assign out3 = temp4 + temp5;
assign out4 = temp6 + temp7;
assign c1 = out1 + out2;
assign c2 = out3 + out4;
assign out = c1 + c2;

```


endmodule

图 12.11 是用 MAX+PLUS II 软件对加法树乘法器进行功能仿真的波形图。和移位相加乘法器相比，这种方法在一个时钟周期内就可完成乘法运算，延时要小得多。

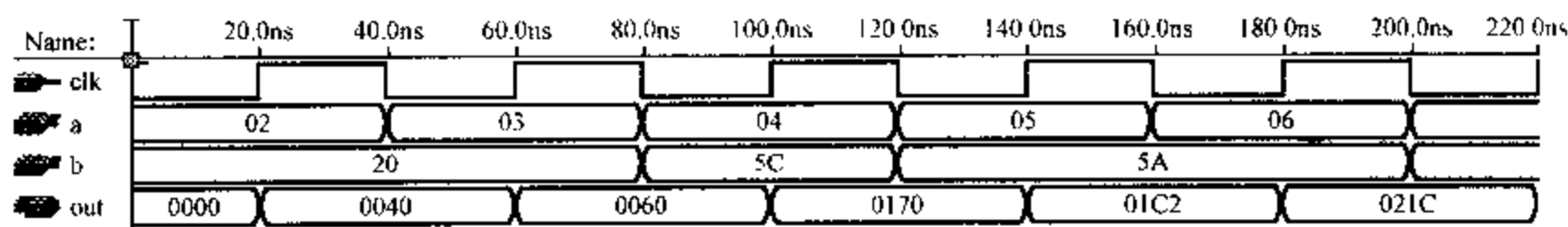


图 12.11 加法树乘法器的仿真波形图

12.3 FIR 滤波器的设计

FIR 有限脉冲响应滤波器是数字滤波器的一种，它的特点是单位脉冲响应是一个有限长序列，因此系统函数一般写成如下形式：

$$H(z) = \sum_{n=0}^{N-1} h(n)z^{-n}$$

其中 N 是 $h(n)$ 的长度，也即 FIR 滤波器的抽头数。

由于 $H(z)$ 是 z^{-1} 的 $(N-1)$ 次多项式，它在 z 平面上有 $(N-1)$ 个零点，原点 $z=0$ 是 $(N-1)$ 阶重极点，因此 $H(z)$ 肯定是稳定的。此外，线性相位特性也是 FIR 滤波器的一个优点，常用的线性相位 FIR 滤波器，其单位脉冲响应均为实数，且满足偶对称或奇对称的条件，即：

$$h(n) = h(N-1-n) \text{ 或 } h(n) = -h(N-1-n)$$

12.3.1 FIR 滤波器的结构

FIR 滤波器的直接型结构如图 12.12 所示，其输出可表示为： $y(n) = \sum_{i=0}^{N-1} h(i)x(n-i)$ 。

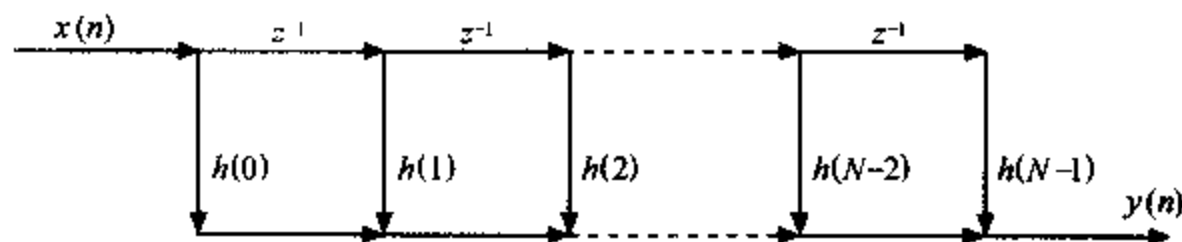


图 12.12 直接型 FIR 滤波器结构示意图

用加法器和乘法器不难实现这种结构的 FIR 滤波器，图 12.13 即为一个直接型 FIR 滤波器的实现方案。但这种直接实现的 FIR 滤波器不论在速度上还是在资源耗用上都不理想。

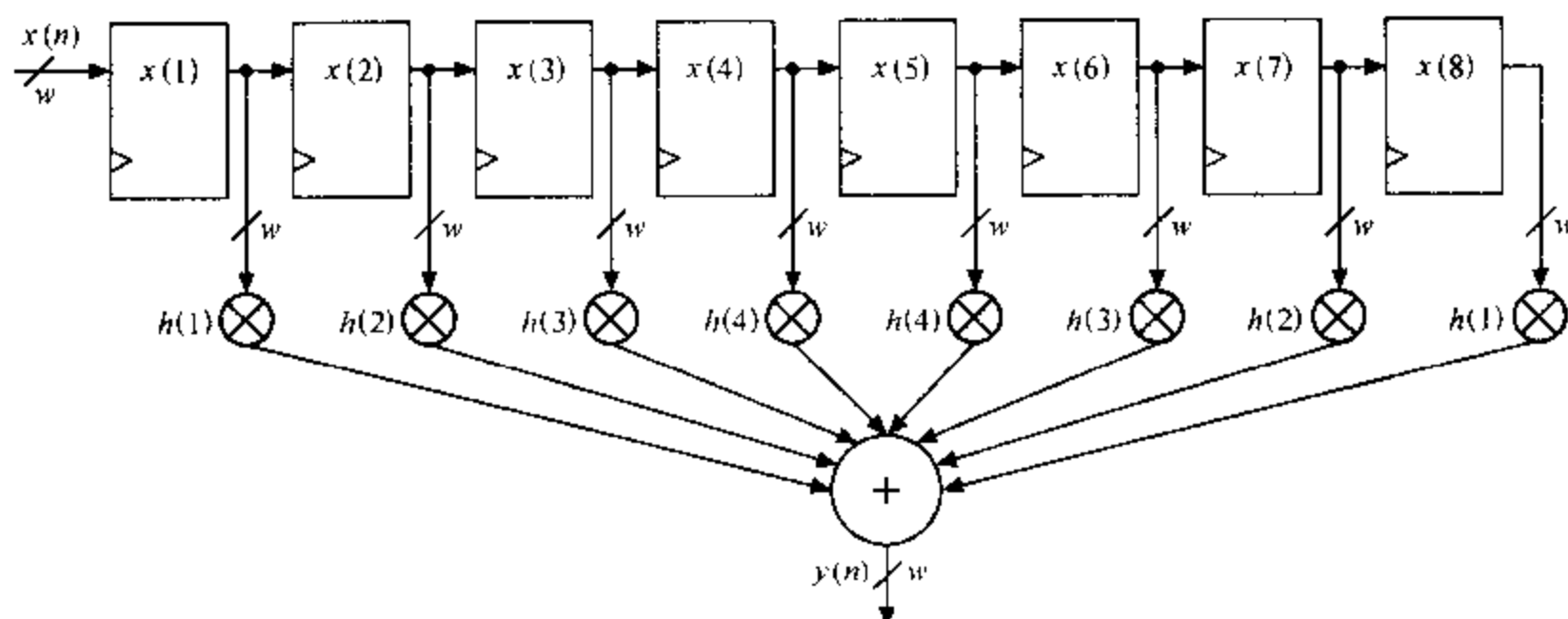


图 12.13 普通 FIR 滤波器实现方案

在使用 FIR 滤波器的实际系统中，多应用了 FIR 滤波器线性相位的特性，因此根据线性相位 FIR 滤波器的系数具有对称性这一特点，可用如图 12.14 所示的结构来实现滤波器。

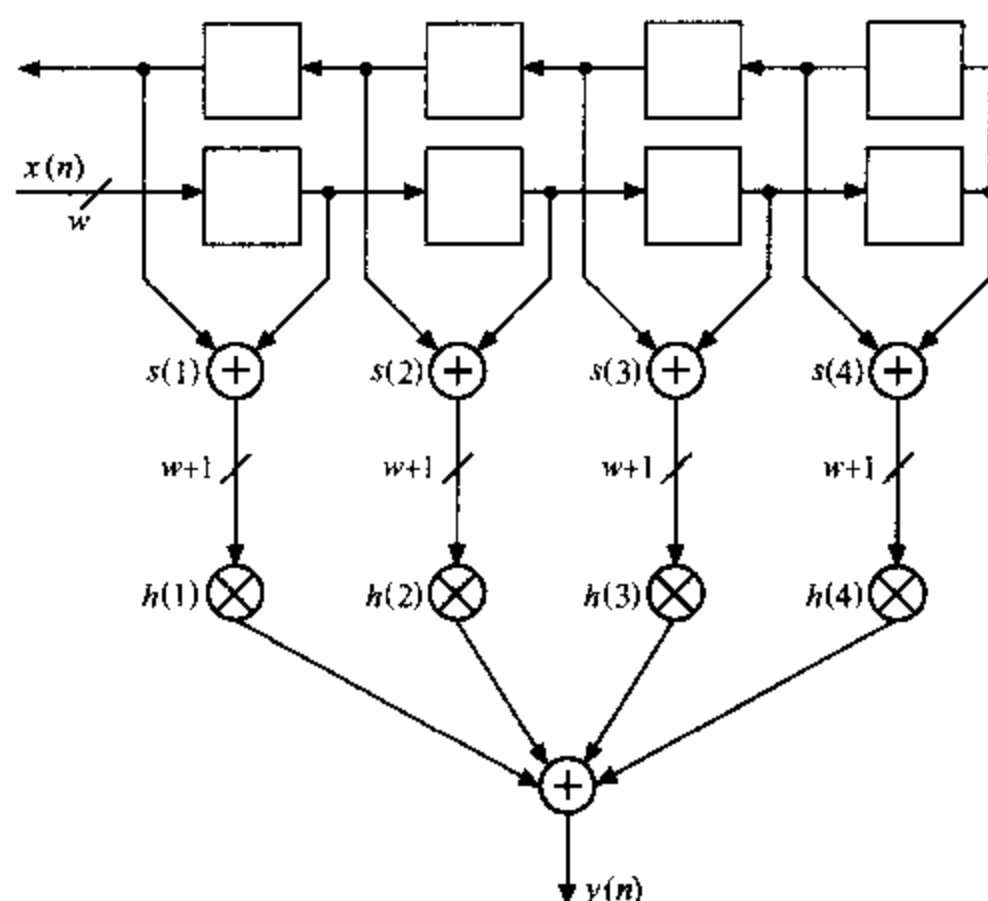


图 12.14 线性相位的 FIR 滤波器结构图

在图 12.14 所示的结构中，滤波器的输出可以写成下面的形式：

$$y(n) = \sum_{n=0}^{(N-1)/2} (x(n) + x(N-1-n))h(n)$$

这种形式的滤波器根据对称性减少了乘法器的数量，从而节省了器件的资源。

12.3.2 抽头系数编码

FIR 滤波器的抽头系数多为小数，且有符号，因此抽头系数的编码也是必须考虑的一个

问题。常用的编码方式有：二进制补码、反码、有符号数值表示法等。

这里采用如下的编码方式：将十进制数用 2^n 数相加减的形式表示出来，这种编码方式在有的书中称为 SD (Signed Digit numbers) 编码，该编码与传统的二进制编码不同，它使用 3 个值来表示数字，即 0、1、 $\bar{1}$ ，其中 $\bar{1}$ 经常写为 $\bar{1}$ 。例如：

$$27_{10} = 32_{10} - 4_{10} - 1_{10} = 100000_2 - 100_2 - 1_2 = 100\bar{1}0\bar{1}_{SD}$$

通常可以通过非零元素的数量来估计乘法的效率，比如乘法操作： $A * x[n]$ ，其具体实现过程如下：

若： $A_{10} = (a_{k-1}a_{k-2}\dots a_0)_2$ ，则：

$$\begin{aligned} A_{10} * x[n] &= (a_{k-1}a_{k-2}\dots a_0)_2 * x[n] \\ &= a_{k-1}2^{k-1}x[n] + a_{k-2}2^{k-2}x[n] + \dots + a_0x[n] \end{aligned}$$

可以明显看到乘法器的成本与 A 中非零元素 a_k 的数量有直接的关系。而 SD 编码表示法可有效降低乘法成本。比如： $93_{10} = 1011101_2 = 10\bar{1}00\bar{1}01_{SD}$ ，用普通二进制编码需要 4 个加法器，用 SD 编码则需要 3 个加法器。

SD 编码通常不是唯一的，比如：

$$15_{10} = 16_{10} - 1_{10} = 1000\bar{1}_{SD}$$

$$15_{10} = 16_{10} - 2_{10} + 1_{10} = 100\bar{1}1_{SD}$$

$$15_{10} = 16_{10} - 4_{10} + 2_{10} + 1_{10} = 10\bar{1}11_{SD}$$

在上面的 SD 编码中，由于第一种方式具有最少数量的 1 和 $\bar{1}$ ，因此它的乘法成本最低，应该尽量减少编码中 1 和 $\bar{1}$ 的数量，以将乘法器实现的成本降到最低，通常将这种包含最少 1 和 $\bar{1}$ 数量的 SD 编码称为最佳 SD 编码。

12.3.3 源代码及仿真

利用 MATLAB 软件求出所设计滤波器的系数。设计一个简单的对模拟语音信号进行滤波的低通 FIR 滤波器，其指标为：采样频率为 8kHz，通带截止频率为 3.4kHz，阻带衰减大约为 10dB 左右，输入、输出数据宽度均为 8 位，滤波器阶数为 11 阶。

根据以上指标，首先用 MATLAB 软件仿真并确定抽头系数。MATLAB 软件仿真的滤波器的抽头系数及其幅度频响特性如图 12.15 所示。抽头系数分别为：0.0036, -0.0127, 0.0417, -0.0878, 0.1318, 0.8500, 0.1318, -0.0878, 0.0417, -0.0127, 0.0036。

抽头系数是奇对称的，即： $h(0) = h(10) = 0.0036$, $h(1) = h(9) = -0.0127$,

$h(2) = h(8) = 0.0417$, $h(3) = h(7) = -0.0878$, $h(4) = h(6) = 0.1318$, $h(5) = 0.8500$

FIR 滤波器采用对称结构，每个抽头的输出分别乘以相应加权的二进制值，再将结果相加。同时，利用滤波器系数奇对称的特性，对输入信号 $x(n)$ 进行如下等效：

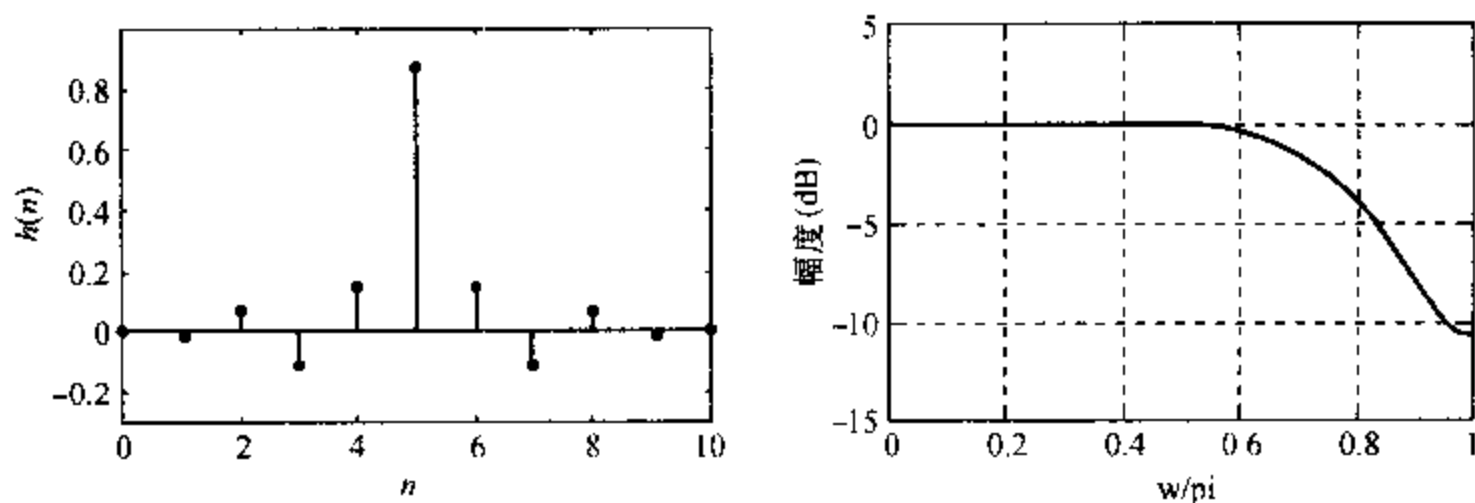


图 12.15 11 阶 FIR 滤波器的抽头系数和幅频特性曲线

$$t5 = x(5), t4 = x(4) + x(6), t3 = x(3) + x(7),$$

$$t2 = x(2) + x(8), t1 = x(1) + x(9), t0 = x(0) + x(10)。$$

在滤波器系数的处理上, 采用优化 SD 编码方式, 以减少对器件资源的耗用。滤波器系数的 SD 编码如下, 每个抽头系数均先左移 7 位 (乘以 128)。

$$128 \cdot h(0) = 128 \times 0.0036 = 0.4608 = 0.5 - 0.03125 = (0.10001)_{SD}$$

$$128 \cdot h(1) = 128 \times 0.0127 = 1.6256 = 1 + 0.5 + 0.125 = (1.101)_{SD}$$

$$128 \cdot h(2) = 128 \times 0.0417 = 5.3376 = 4 + 1 + 0.25 + 0.0625 + 0.03125 = (101.01011)_{SD}$$

$$128 \cdot h(3) = 128 \times 0.0878 = 11.2384 = 8 + 4 - 1 + 0.25 = 11.25 = (1101.01)_{SD}$$

$$128 \cdot h(4) = 128 \times 0.1318 = 16.8704 = 16 + 1 - 0.125 = (10001.001)_{SD}$$

$$128 \cdot h(5) = 128 \times 0.8500 = 108.800 = 128 - 16 - 4 + 0.5 + 0.25 + 0.0625 = (10010100.1101)_{SD}$$

这样, 11 阶 FIR 滤波器的输出就可以用下面的算式得到:

$$\begin{aligned} \text{sum} = & (t5 \ll 7) - ((t5 \ll 2) \ll 2) - (t5 \ll 2) + \{t5[7], t5[7:1]\} + \{t5[7], t5[7], t5[7:2]\} \\ & + \{t5[7], t5[7], t5[7], t5[7], t5[7:4]\} + (t4 \ll 4) + t4 - \{t4[7], t4[7], t4[7], t4[7:3]\} \\ & - (t3 \ll 3) - (t3 \ll 2) + t3 - \{t3[7], t3[7], t3[7:2]\} + (t2 \ll 2) + t2 + \{t2[7], t2[7], t2[7:2]\} \\ & + \{t2[7], t2[7], t2[7], t2[7], t2[7:4]\} + \{t2[7], t2[7], t2[7], t2[7], t2[7], t2[7:5]\} - t1 \\ & - \{t1[7], t1[7:1]\} - \{t1[7], t1[7], t1[7], t1[7:3]\} \\ & + \{t0[7], t0[7:1]\} - \{t0[7], t0[7], t0[7], t0[7], t0[7], t0[7:5]\} \end{aligned}$$

得到结果后, 再将结果右移 7 位, 即得到正确的结果。根据以上设计思路, 用 Verilog 语言设计的 11 阶 FIR 数字滤波器源代码如下所示。

【例 12.7】11 阶 FIR 数字滤波器

```
module fir(clk, x, y);
  input [7:0] x;
```

```

input clk;
output[15:0] y;
reg[15:0] y;
reg[7:0] tap0,tap1,tap2,tap3,tap4,tap5,tap6,tap7,tap8,tap9,tap10;
reg[7:0] t0,t1,t2,t3,t4,t5;
reg[15:0] sum;

always@(posedge clk)
begin
    t0<=tap5;
    t1<=tap4+tap6;
    t2<=tap3+tap7;
    t3<=tap2+tap8;
    t4<=tap1+tap9;
    t5<=tap0+tap10;          //利用对称性

sum<= (t1<<4)+{t1[7],t1[7:1]}+{t1[7],t1[7],t1[7:2]}+{t1[7],t1[7],t1[7],
    t1[7:3]}-(t2<<3)-(t2<<2)+t2-{t2[7],t2[7],t2[7:2]}
    +(t3<<2)+t3+{t3[7],t3[7],t3[7:2]}+{t3[7],t3[7],t3[7],t3[7],t3[7:4]}
    +{t3[7],t3[7],t3[7],t3[7],t3[7],t3[7:5]}
    -t4-{t4[7],t4[7:1]}-{t4[7],t4[7],t4[7],t4[7:3]}
    +{t5[7],t5[7:1]}-{t5[7],t5[7],t5[7],t5[7],t5[7],t5[7:5]}
    +(t0<<7)-{(t0<<2)<<2}-(t0<<2)+{t0[7],t0[7:1]}
    +{t0[7],t0[7],t0[7:2]}+{t0[7],t0[7],t0[7],t0[7],t0[7:4]};
//16+0.5+0.25+0.125=16.875
//8+4-1+0.25=11.25
//4+1+0.25+0.0625+0.03125=5.34375
//1+0.5+0.125=1.625
//0.5-0.03125=0.46875
//128-4*4-4+0.5+0.25+0.0625=108.8125
/* 0.0036, -0.0127, 0.0417, -0.0878, 0.1318, 0.8500, 0.1318, -0.0878,
0.0417, -0.0127, 0.0036, 0.4608, -1.6256, 5.3376, -11.2384, 16.8704,
108.800, 16.8704, -11.238, 5.3376, -1.6256, 0.4608 */

tap10<=tap9;
tap9<=tap8;
tap8<=tap7;
tap7<=tap6;
tap6<=tap5;

```

```

    tap5<=tap4;
    tap4<=tap3;
    tap3<=tap2;
    tap2<=tap1;
    tap1<=tap0;
    tap0<=x;

    y<={sum[15],sum[15],sum[15],sum[15],sum[15],sum[15],sum[15],sum[15:7]};
end
endmodule

```

如图 12.16 所示是该 FIR 滤波器用 Synplity Pro 综合后的 RTL 视图。

用 MAX+PLUS II 软件仿真该程序的结果则如图 12.17 所示（注：负数以补码形式给出）。

大多数情况下，我们希望能够根据应用的不同场合来改变滤波器的参数和性能，这就需要我们能够对滤波器的系数进行在线配置。可编程滤波器能满足这一要求。可编程滤波器在数据处理上分为两个过程：第一步完成滤波器系数的配置，将系数配置到抽头延迟线上；第二步执行乘积和的计算，对乘积进行一位有符号扩展，并加到前面的部分乘积上。

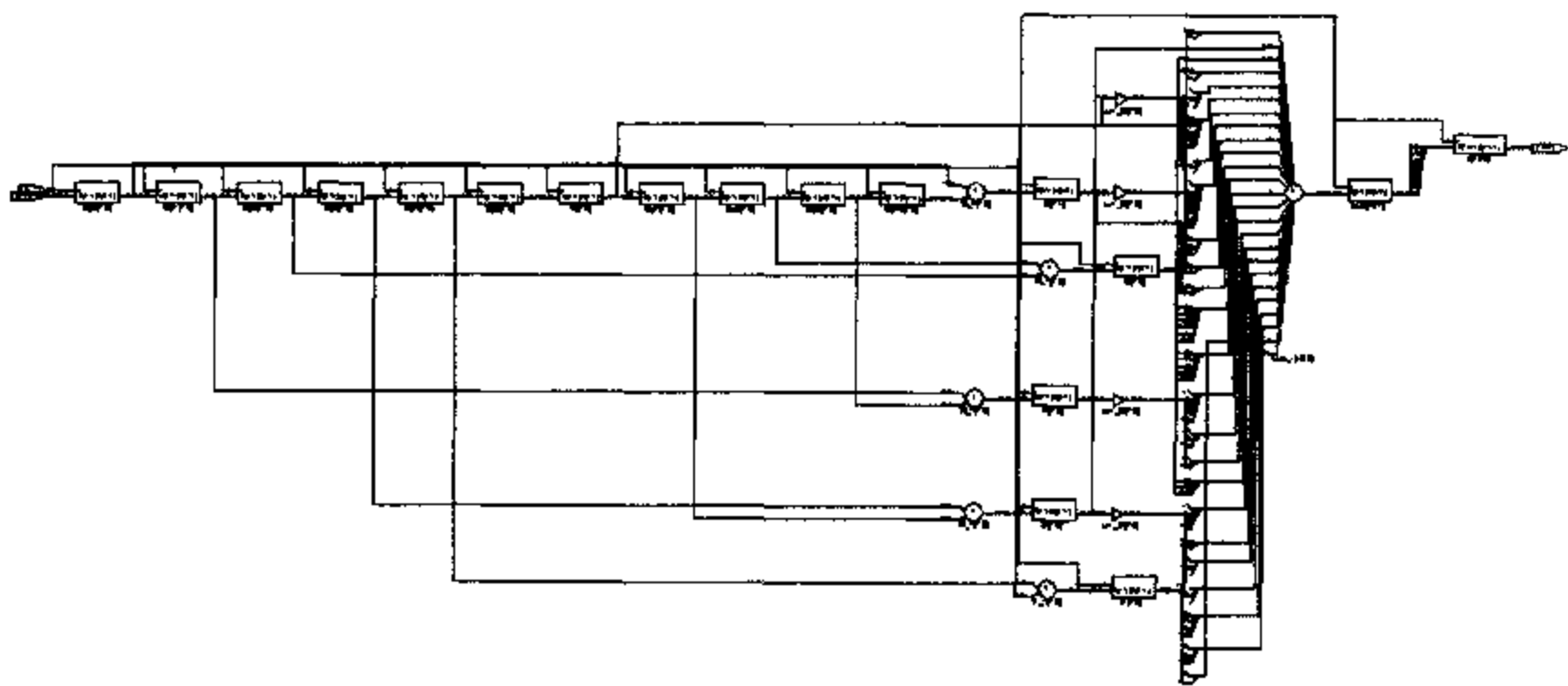


图 12.16 FIR 滤波器的 RTL 综合视图

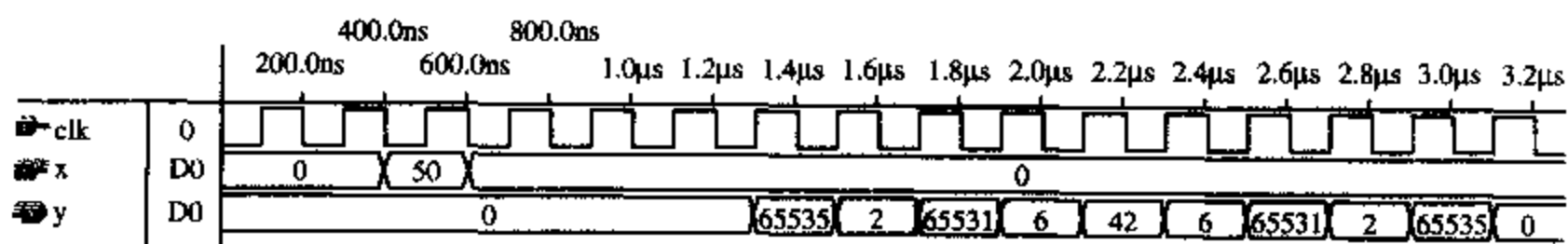


图 12.17 11 阶 FIR 滤波器仿真结果

12.4 数字相关器

在数字通信系统中, 常用一个特定的序列作为数据开始的标志, 称为帧同步字。发送端在发送数据前插入帧同步字; 接收端如果收到帧同步字, 就可以确定帧的起始位置, 从而实现发送和接收数据的格式同步。

数字相关器的作用是实现两个数字信号之间的相关运算, 即比较等长度的两个数据间相等的位数。因此, 在通信系统中, 常用数字相关器作为同步序列检测器使用。

最基本的相关器是异或门。例如 $y = a \oplus b$, 当 $H(z)$ 时, $y = 0$, 表示数据位相同; 当 $a \neq b$ 时, $y = 1$, 表示数据位不同。

多位数字相关器可以由 1 位相关器级连构成, N 位数字相关器的运算通常可以分解为以下两个步骤:

- (1) 对应位进行异或运算, 得到 N 个 1 位相关运算结果;
- (2) 统计 N 位相关结果中“0”或“1”的数目, 得到 N 位数字中相同位和不同位的数目。

下面用 Verilog 语言设计并实现一个 16 位的数字相关器。

由于实现 16 位并行相关器需要的乘积项、或门过多, 因此为降低耗用资源, 将其分解为 4 个 4 位相关器, 然后用两级加法器相加得到全部 16 位的相关结果, 如图 12.18 所示。

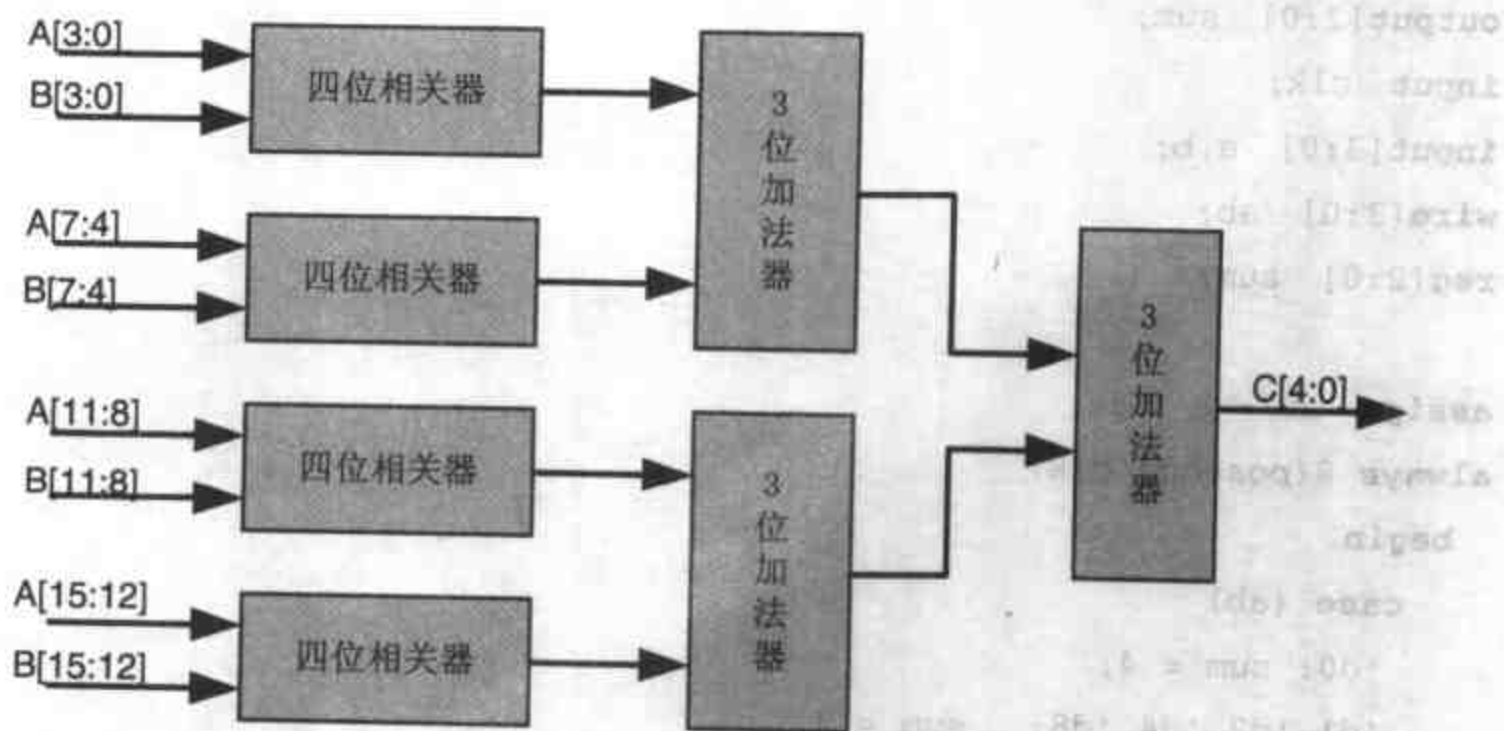


图 12.18 16 位高速数字相关器结构图

如果直接实现这个电路, 则整个运算至少需要经过 3 级门延时。随着相关位数目的增加, 速度还将随之降低。为提高速度, 采用“流水线技术”进行设计。模块中对每一步运算结果进行锁存, 按照时钟的节拍, 逐级完成运算的全过程。虽然每组输入值需要经过 3 个节拍后才能得到运算结果, 但是, 由于每个节拍都有一组新值输入到第一级运算电路, 每级运算电路上都有一组数据同时进行运算, 所以总的来看, 每步运算花费的时间只有一个时钟周期, 从而使系统工作速度等于时钟频率。

利用流水线技术的 16 位相关器的 Verilog 源代码如下。

【例 12.8】16 位高速数字相关器

```

module correlator(out,a,b,clk);
    output[4:0] out;
    input [15:0] a,b;
    input clk;
    wire[2:0] sum1,sum2,sum3,sum4;
    wire[3:0] temp1,temp2;

    detect u1(sum1,a[3:0],b[3:0],clk),           //模块调用
           u2(sum2,a[7:4],b[7:4],clk),
           u3(sum3,a[11:8],b[11:8],clk),
           u4(sum4,a[15:12],b[15:12],clk);
    add3 u5(temp1,sum1,sum2,clk),
         u6(temp2,sum3,sum4,clk);
    add4 u7(out,temp1,temp2,clk);
endmodule

module detect(sum,a,b,clk);                     //该模块实现 4 位相关器
    output[2:0] sum;
    input clk;
    input [3:0] a,b;
    wire[3:0] ab;
    reg[2:0] sum;

    assign ab = a ^ b;
    always @(posedge clk)
        begin
            case (ab)
                'd0: sum = 4;
                'd1,'d2,'d4,'d8: sum = 3;
                'd3,'d5,'d6,'d9,'d10,'d12: sum = 2;
                'd7,'d11,'d13,'d14: sum = 1;
                'd15: sum = 0;
            endcase
        end
endmodule

module add3(add,a,b,clk);                       //3 位加法器

```



```

output[3:0] add;
input[2:0] a,b;
input clk;
reg[3:0] add;
always @(posedge clk)
    begin add = a + b; end
endmodule

```

```

module add4(add,a,b,clk);           //4位加法器
output[4:0] add;
input[3:0] a,b;
input clk;
reg[4:0] add;
always @(posedge clk)
    begin add = a + b; end
endmodule

```

该设计用器件 EPF10K10LC84-3 实现时, 最高工作速度可达 90MHz。

12.5 信道编译码器

在实际的通信系统中, 信道中存在的大量噪声和干扰使得经信道传输后接收到的码字与发送码字之间存在着差异。为减小这种差异的产生, 保证数字通信系统的传输质量, 现代数字通信系统大量采用了差错控制技术。差错控制编码又称为信道编码, 其基本思想是通过对信息序列做某种变换, 比如增加校验码元等, 使原来相关性很小的信息码元产生某种相关性, 从而在接收端利用这种相关性来检查进而纠正信息码元在传输中产生的差错。

信道编码的方式很多, 从增加的校验码元和信息码元的关系出发, 可以将信道编码大致分为两类: 分组码和非分组码。如果每个码字的校验码元只与该码字的信息码元相关, 则这种码称为分组码。如果分组码内的规律是线性的, 则称为线性分组码, 否则称为非线性分组码。如果采用的规则使若干个相邻的码字都具有了一定的相关性, 则称这种码字为非分组码。常用的卷积码就是非分组码中重要的一类。

下面分别说明线性分组码编译码器和循环码编译码器的实现。首先介绍线性分组码编译码器的实现。

12.5.1 线性分组码编译码器

线性分组码中的线性是指码组中码元间的约束关系是线性的, 而分组则是对于编码方法而言的。线性分组编码时, 将每 k 个信息位分为一组进行独立处理, 按一定规则给每个信息组增加 $n-k$ 个校验码元, 组成长度为 n 的二进制码组, 这种编码方式记为 (n, k) 分组码。

这时，信息位和校验位之间存在着线性关系。

若线性分组码的输入信息位为 $\vec{u} = (u_0, u_1, \dots, u_k)$ ，编码输出为 $\vec{c} = (c_0, c_1, \dots, c_n)$ ，则有以下关系：

$$\begin{cases} \vec{c} = \vec{u} \cdot G \\ H \cdot \vec{c}^T = O^T \end{cases}$$

其中 G 为线性分组码的生成矩阵， H 为监督矩阵。当 G 确定后，编码的方法就完全确定了；而 H 给定后，编码时校验位和信息位的关系就确定了。因此在设计编码器时，常讨论生成矩阵 G ；设计译码器时，常讨论监督矩阵 H 。

设发送的码字为 $\vec{c} = (c_0, c_1, \dots, c_n)$ ， $\vec{e}$ 表示传输中的差错， y 表示接收的码字。如果码组在传输中没有出现差错，则有 $Hy^T = H\vec{c}^T = 0$ ，否则出错时 $Hy^T = H(\vec{c} \oplus \vec{e})^T = H\vec{e}^T = s^T$ ， $s = \vec{e}H^T$ 。其中 s 称为伴随子，又称为校正子。由于 s 只和序列传输中的差错 $\vec{e}$ 有关，因此在错码的纠错能力之内，一定的 $\vec{e}$ 序列的组合必然对应着一定的 s 序列组合，因此可以预先在接收机中做好对应表，然后根据序列 s 准确判断出差错位置。再根据出错的位置进行纠正，即可得到正确的译码输出。

该译码方法的步骤如下：

- (1) 计算接收矢量 y 的伴随子 $s = y \cdot H^T$ 。
- (2) 由伴随子 s 决定相应的错误图样集 $\hat{e}$ 。
- (3) 将 y 译成 $\hat{c} = y - \hat{e}$ 。

下面以 (7, 4) 分组码为例，实现编码器和译码器。

通常采用的 (7, 4) 分组码的生成矩阵 G 为：

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

监督矩阵 H 为：

$$H = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

由生成矩阵可知，(7, 4) 分组码的编码结构简单，只要在输入编码器的每组 4 个码元后附加上 3 个监督码元就可得到完整的 7 个码元的码字。下面是编码器的源代码。

【例 12.9】(7, 4) 线性分组码编码器

```
module linear(c,u,clk);
    output[6:0] c;           //c 为编码输出码字
    input[3:0] u;
    input clk;
    reg[6:0] c;
```

```

always @(posedge clk)
begin
    c[6] = u[3];
    c[5] = u[2];
    c[4] = u[1];
    c[3] = u[0];
    c[2] = u[1] ^ u[2] ^ u[3];
    c[1] = u[0] ^ u[1] ^ u[2];
    c[0] = u[0] ^ u[2] ^ u[3];
end
endmodule

```

由于伴随子 s 是一个 $(n-k)$ 维矢量，而传输中的差错 $\bar{e}$ 是一个 n 维矢量，因此两者之间的对应关系不是唯一的。所以译码器必须根据一定的规则从各种候选错误矢量中选出真正的错误矢量。各种规则对应的译码方式很多，在二进制对称信道条件下，我们采用译码平均错误概率最小的准则。这时，对应的最可能错误图样是译码汉明距离最小的接收码组，即非零个数最小的码组。因此可以得到伴随子和错误图样的对应关系。(7, 4) 线性分组码的伴随子和错误图样的对应关系见表 12.2。

表 12.2 (7, 4) 线性分组码的伴随子和错误图样集

伴随子 s	错误图样集 $\hat{e}$
000	0000000
100	1000000
010	0100000
001	0010000
110	0001000
011	0000100
111	0000010
101	0000001

若发送码字为 (1001011)，接收的码字为 (1001001)，根据公式 $s = y \cdot H^T$ 可求出 $s = (1, 1, 1)$ ，其具体运算和纠错过程如图 12.19 所示，图中同时显示了检测错误图样集的逻辑电路。

$$e_0 = s_0 \cdot \bar{s}_1 \cdot \bar{s}_2, \quad e_1 = \bar{s}_0 \cdot s_1 \cdot \bar{s}_2,$$

$$e_2 = \bar{s}_0 \cdot \bar{s}_1 \cdot s_2, \quad e_3 = s_0 \cdot s_1 \cdot \bar{s}_2,$$

$$e_4 = \bar{s}_0 \cdot s_1 \cdot s_2, \quad e_5 = s_0 \cdot s_1 \cdot s_2,$$

$$e_6 = s_0 \cdot \bar{s}_1 \cdot s_2$$

以下是 (7, 4) 线性分组码译码器的 Verilog 源代码。

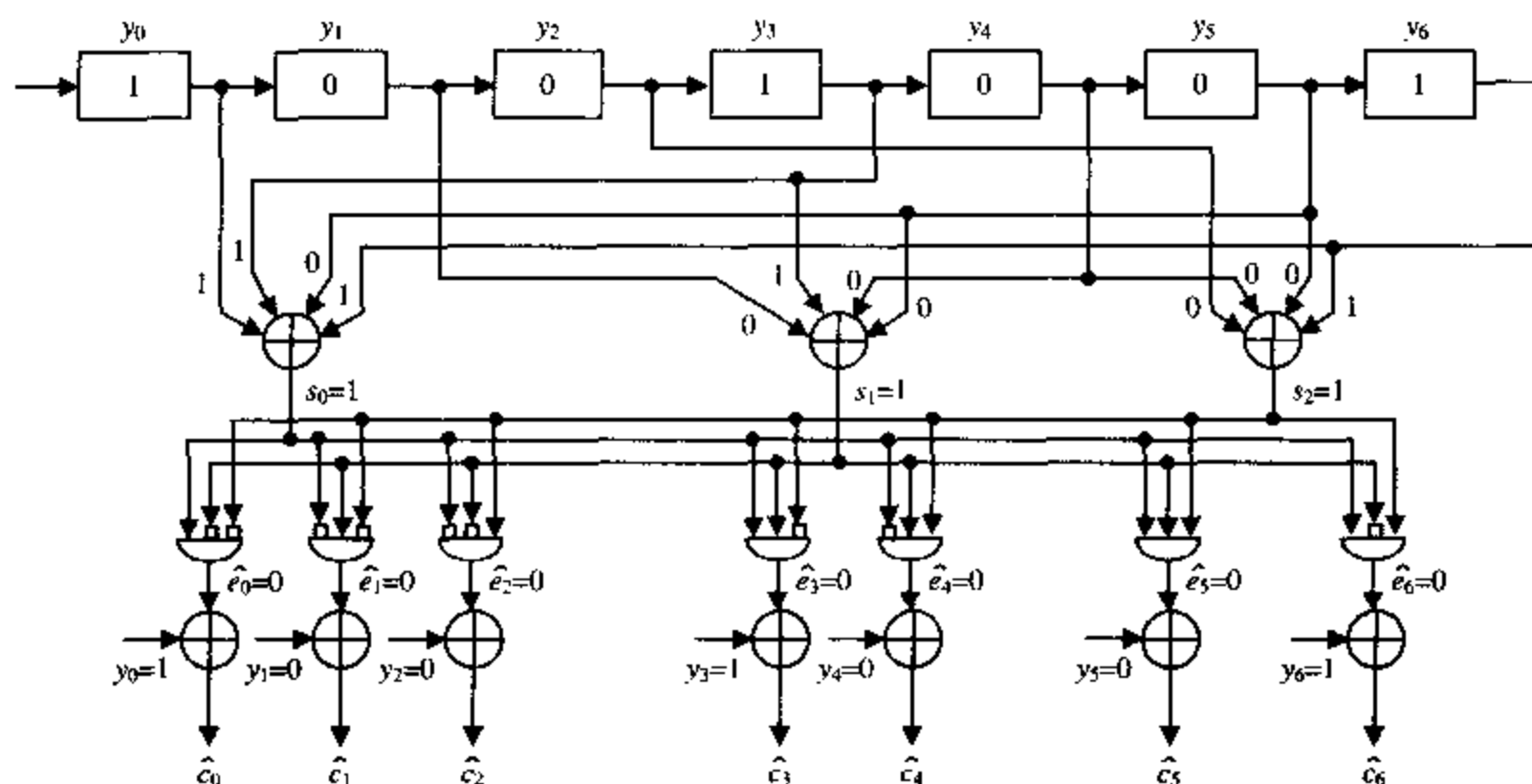


图 12.19 分组码纠错过程示意图

【例 12.10】(7, 4) 线性分组码译码器

```
module decoder1(c,y,clk);
```

```
output[6:0] c;
```

```
input[6:0] y;
```

```
input clk;
```

```
reg[2:0] s;
```

```
reg[6:0] e,c;
```

```
always @(posedge clk)
```

```
begin
```

```
    s[0] = y[0] ^ y[3] ^ y[5] ^ y[6];
```

```
    s[1] = y[1] ^ y[3] ^ y[4] ^ y[5];
```

```
    s[2] = y[2] ^ y[4] ^ y[5] ^ y[6];
```

```
//s[0]~ s[2]为伴随子
```

```
    e[0] = s[0] & (~s[1]) & (~s[2]);
```

```
    e[1] = (~s[0]) & s[1] & (~s[2]);
```

```
    e[2] = (~s[0]) & (~s[1]) & s[2];
```

```
    e[3] = s[0] & s[1] & (~s[2]);
```

```
    e[4] = (~s[0]) & s[1] & s[2];
```

```
    e[5] = s[0] & s[1] & s[2];
```

```
    e[6] = s[0] & (~s[1]) & s[2];
```

```
//e[0]~ e[6]为错误图样
```

```
    c = e ^ y;
```

```
//c 为输出码字
```

```
end
```

```
endmodule
```

12.5.2 循环码编译码器

循环码是线性分组码中常用的一种。如果一个 (n, k) 线性分组码的每个码字经任意循环后仍然是一个线性分组码, 则称为循环码。循环码具有两个主要特点:

- (1) 可以用反馈线性移位寄存器很容易地实现其编码和伴随子计算。
- (2) 可以根据它本身固有的许多代数结构找到各种简单实用的译码方法。

由于具有这两个特点, 因此循环码的编译码方法和一般的线性分组码有所不同。

循环码通常采用码多项式描述, 例如一个 $(7, 4)$ 循环码的码字为 (1001000) , 就可用 $T(x) = 1 + x^3$ 表示。特别的, 用 $g(x)$ 表示前 $k-1$ 位皆为零的码组, 称为生成多项式。由循环码的性质可知, 循环码的生成矩阵可由码的生成多项式 $g(x)$ 及其循环移位构成。例如, $(7, 4)$ 循环码生成多项式 $g(x) = 1 + x + x^3$, 则:

$$G(x) = \begin{bmatrix} x^3 g(x) \\ x^2 g(x) \\ x g(x) \\ g(x) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

根据生成矩阵, 就可利用上节介绍的方法设计编码器了, 但我们还可以利用循环码自身的循环性更方便地实现编码器。

仍然以 $(7, 4)$ 循环码为例, 若输入信息码元为 $u(x) = 1 + x^3$, 则:

$$\begin{aligned} x^{n-k} \cdot u(x) &= x^{7-4} (1 + x^3) = x^3 + x^6 \\ &\equiv x + x^2 \pmod{1 + x + x^3} \end{aligned}$$

因此, 码多项式为 $c(x) = r(x) + x^{n-k} u(x)$

$$\begin{aligned} &= x + x^2 + x^{7-4} (1 + x^3) \\ &= x + x^2 + x^3 + x^6 \end{aligned}$$

其中 $r(x) = x + x^2$ 为信息码元置为高位后整除生成多项式后的余式, 它正好对应于校验序列。因此, 对应的输出码字为 $c = (0111001)$, 其中最右边的 4 位是信息元。

循环码编码器的结构可用图 12.20 所示的框图来表示。

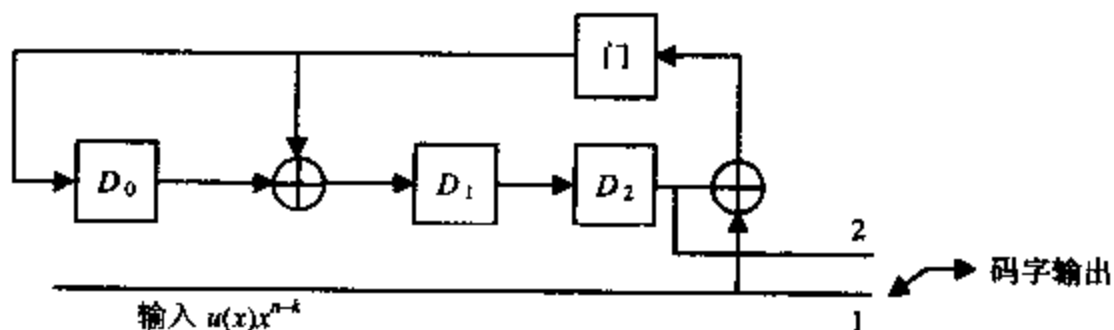


图 12.20 循环码编码器结构图

利用该结构进行编码的过程如下:

(1) 三级移位寄存器初值为 000, 这时门选通, 信息组以 $u_3, \dots, u_0$ 次序分两路输入, 一路直接输出, 另一路送入 $g(x)$ 除法电路。

(2) 经过四次移位后, 信息组全部输出。另一路则将全部信息元送入除法电路, 并完成除法运算, 这时移位寄存器中的状态就是码的监督元 (c_0, c_1, c_2)。

(3) 输出开关倒向第二个支路, 经三次移位后, 移位器由监督元跟在信息元后依次输出编码码字 $\bar{c}$ 。

(4) 重复上述过程, 进行下一组信息码元编码。

根据以上思路, 设计编码器源代码如下。其中输入和输出码元均采用并行方式。

【例 12.11】(7, 4) 循环码编码器

```

module cycle(c,u,clk);
output [6:0] c;
input [3:0] u;
input clk;
reg [2:0] i;
reg d0,d1,d2,temp;
reg [6:0] c;

always @(posedge clk)
begin
    d0=0;    d1=0;    d2=0;                //初始化
    for (i=0;i<4;i=i+1)                    //该 for 循环计算码组的前 4 个码元
        begin
            temp = d2 ^ c[i];
            d2 = d1;    d1 = d0 ^ temp;
            d0 = temp; c[i] = u[i];
        end
    for (i=4;i<7;i=i+1)                    //该 for 循环计算码组的后 3 个码元
        begin
            temp = d2;
            d2 = d1;    d1 = d0 ^ temp;
            d0 = temp; c[i] = temp;
        end
    end
endmodule

```

由于循环码的任一多项式 $T(x)$ 均能被 $g(x)$ 整除, 因此可以利用接收到的码组 $y(x)$ 去除以生成多项式 $g(x)$ 来检错。若码元无错, 则 $y(x)$ 可被 $g(x)$ 整除, 余项为零。若 $y(x)$ 有错, 则整除后的余项不为零。若要进一步纠错, 可利用伴随子和错误图样集之间的关系。

例如上面的例子中, 二元 (7, 4) 循环码可以纠正 7 个码元中的任何单个错误, 利用伴

随子进行译码的过程见表 12.3。

表 12.3 $g(x) = 1 + x + x^3$ 生成 (7, 4) 循环码译码过程

错误图样 $e(x)$	伴随式 $s(x)$	伴随矢量 $s_0 \ s_1 \ s_2$	移位次数 i	移位后伴随子矢量 $s_0 \ s_1 \ s_2$
e_6	$1 + x^2$	1 0 1	0	1 0 1
e_5	$1 + x + x^2$	1 1 1	1	1 0 1
e_4	$x + x^2$	0 1 1	2	1 0 1
e_3	$1 + x^2$	1 1 0	3	1 0 1
e_2	x^2	0 0 1	4	1 0 1
e_1	x	0 1 0	5	1 0 1
e_0	1	1 0 0	6	1 0 1

若错误图样 $e(x) = x^6$ ，即 x^6 位置上有错，则接收矢量 $y(x)$ 全部进入伴随式寄存器后，伴随式为 (1 0 1)。检测到这种伴随式说明 x^6 上的数据必须纠正。如果某个单个错误出现在 x^i 位置上，则 $y(x)$ 进入伴随式寄存器后，伴随式经过 i 次移位仍然是 (1 0 1)，且缓存器输出的接收数据是一个错误数据，所以译码时只需要检测 (1 0 1) 伴随式。

根据上述分析，利用接收码字中单个错误出现在首位的错误图样及其相应的伴随式，设计译码器的框图如图 12.21 所示。

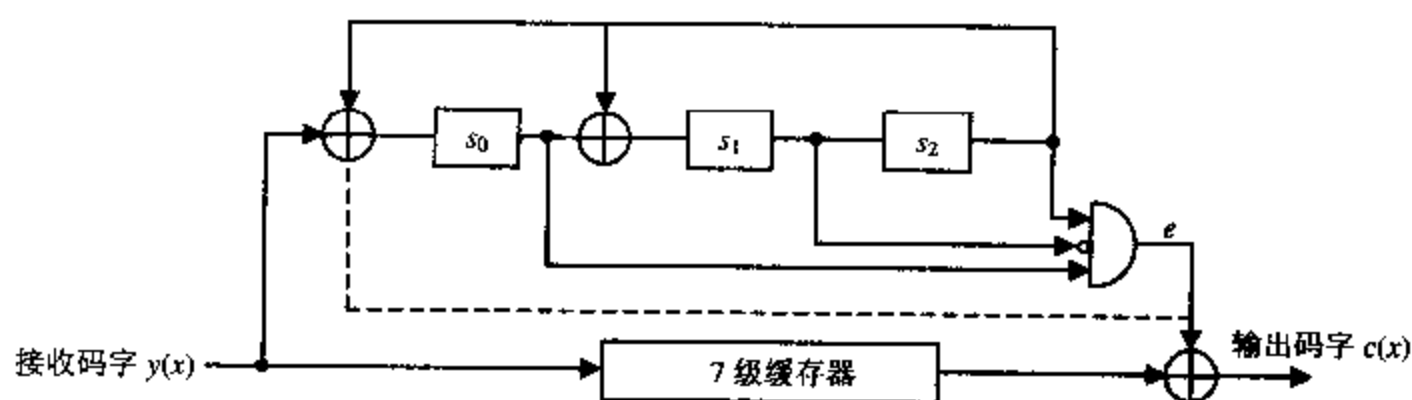


图 12.21 循环码译码器结构框图

译码器的工作步骤如下：

- (1) 将移位寄存器清零。
- (2) 接收码字 y 分两路进入译码器（高次项在前）：一路进入缓存器，另一路进入伴随式计算电路，并分别计算 s_0 、 s_1 、 s_2 。
- (3) 在输出 c 的同时， s_0 、 s_1 、 s_2 依次循环移位。若无错，则错误检测电路无输出；若有错，则错误检测电路输出为“1”。它与 y 中的相应错误位进行模 2 加（异或），纠错后输出。同时，该纠错信号也被送到伴随式计算电路对寄存器清零（如图 12.21 中虚线所示）。

其 Verilog 源程序如下所示。

【例 12.12】(7, 4) 循环码纠错译码器

```

module decoder2(c,y,clk);
    output[6:0] c;                                //c 为输出码字, c[6] 为高次项
    input[6:0] y;                                //y 为接收码字, y[6] 为高次项
    input clk;
    reg[6:0] c, c_buf, buffer;
    reg temp;
    reg s0,s1,s2;                                //伴随式电路寄存器
    reg e;                                        //错误检测输出信号
    integer i;

    always @ (posedge clk)
    begin
        s0=0; s1=0; s2=0;                        //初始化
        temp=0;
        buffer=y;                                //接收码字移入缓存

        for (i=6;i>=0;i=i-1)                    //接收码字进入除法电路
        begin
            e=s0&(~s1)&temp;
            temp=s2;
            s2=s1;
            s1=s0^temp;
            s0=y[i]^temp^e;
        end

        for (i=6;i>=0;i=i-1)                    //输出纠错译码后的码字
        begin
            e=s0&(~s1)&temp;
            temp=s2;
            s2=s1;
            s1=s0^temp;
            s0=temp^e;
            c_buf[i]=buffer[i]^e;
            if (e==1)                            //若出错, 对缓存进行清零
            begin
                s0=0; s1=0; s2=0;
            end
        end
    end

```



```

always @ (posedge clk)
begin
c=c_buf;
end
endmodule

```

12.6 CRC 校验码

循环冗余校验码 (CRC, Cyclic Redundancy Check) 也是常用的信道编码方式, 广泛应用于帧校验。

1. CRC 编码与检错原理

设待传送的 k 位信息码为 $(m_{k-1}, m_{k-2}, \dots, m_1, m_0)$, 对应的多项式为:

$$M(x) = m_{k-1}x^{k-1} + m_{k-2}x^{k-2} + \dots + m_1x + m_0$$

其中 $m_i = 0$ 或 1 , x 的幂次对应于各码元的位置。

所谓编码, 就是根据 k 位信息码元, 计算出与这 k 位信息位码元相关的 r 位监督码元, 形成 (n, k) ($n=k+r$) 格式的码字 $C(x)$, $C(x)$ 就是要发送的码元序列。 $C(x)$ 的码字结构如图 12.22 所示。

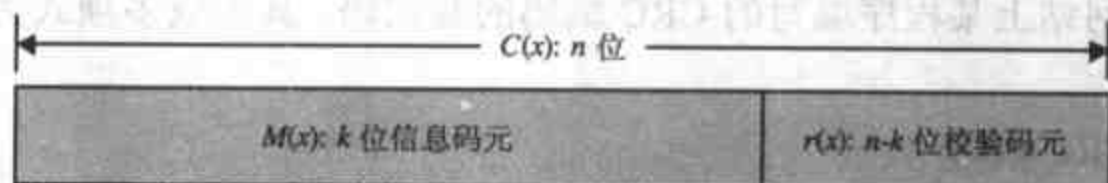


图 12.22 CRC 码结构

CRC 编码的步骤为:

- (1) 将信息多项式 $M(x)$ 左移 $r=n-k$ 位, 即 $M(x)$ 乘以 x^{n-k} , 得 $x^{n-k}M(x)$;
- (2) 将 $x^{n-k}M(x)$ 除以生成多项式 $g(x)$, 得余式 $r(x)$ 。 $r(x)$ 为 CRC 校验码多项式, 对应 r 位监督码元;

- (3) 码字 $C(x) = x^{n-k}M(x) + r(x)$ 。

下面举例说明。

已知: $M(x) = x^7 + x^5 + x^3 + x + 1 = 10101011$, 设 $g(x) = x^4 + x^3 + 1 = 11001$ 。

$n-k=4$ (总是比 $g(x)$ 少一位), 因此可得:

$$x^{n-k}M(x) = x^4(x^7 + x^5 + x^3 + x + 1) = x^{11} + x^9 + x^7 + x^5 + x^4 = 101010110000$$

$$\frac{x^{n-k}M(x)}{g(x)} = \frac{x^{11} + x^9 + x^7 + x^5 + x^4}{x^4 + x^3 + 1} = \frac{101010110000}{11001} = 11000101 + \frac{1101}{11001}$$

注意上面的除法为模 2 除法。所以可得:

商值: $Q(x) = x^7 + x^6 + x^2 + 1 = 11000101$

余式: $r(x) = x^3 + x^2 + 1 = 1101$

将 $x^{n-k}M(x)$ 与 $r(x)$ 相加, 即得 CRC 校验序列:

$$x^{11} + x^9 + x^7 + x^5 + x^4 + x^3 + x^2 + 1 = 101010111101$$

CRC 码检错的方法是: 在接收端把收到的数字序列 $R(x)$ 除以生成多项式 $g(x)$, 得余式 $S(x)$ 。若 $S(x) = 0$, 可认为无错码; 若 $S(x) \neq 0$, 则肯定有错码。例如在上面的例子中, 检验式为: $\frac{101010111101}{11001} = 11000101$ 。

CRC 检错码的检错能力与其生成多项式 $g(x)$ 密切相关。 (n, k) 循环码的生成多项式 $g(x)$ 的一般形式为: $g(x) = x^{n-k} + g_{n-k-1}x^{n-k-1} + \dots + g_1x + 1$ 。

$g(x)$ 的首项系数为 1, 末项系数也必须为 1。 $g(x)$ 的次数越高, 其检错能力越强。

国际上通行的 CRC 码生成多项式有如下几个:

CRC-ITU-T: $g(x) = x^{16} + x^{12} + x^5 + 1$

CRC-12: $g(x) = x^{12} + x^{11} + x^3 + x^2 + x + 1$

CRC-16: $g(x) = x^{16} + x^{15} + x^2 + 1$

CRC-32: $g(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

CRC-12 码通常用来传送 6bit 字符串, CRC-16 及 CRC-ITU-T 码则多用来传送 8bit 字符, CRC-16 为美国采用, 而 CRC-ITU-T 为欧洲国家所采用, CRC-32 码多被采用在点到点 (Point-to-Point) 通信中。可根据实际需要选择其中的一个。

2. 源代码

下面是根据网站上某程序编写的 CRC 编码的源代码, 其生成多项式为 $g(x) = x^{16} + x^{12} + x^5 + 1$ 。

【例 12.13】CRC 编码

```
module crc (crc_reg, crc, d, calc, init, d_valid, clk, reset);
    output[15:0] crc_reg;
    output[7:0] crc;
    input[7:0] d;
    input calc;
    input init;
    input d_valid;
    input clk;
    input reset;

    reg[15:0] crc_reg;
    reg[7:0] crc;
    wire[15:0] next_crc;

    always @ (posedge clk or posedge reset)
```

```

begin
    if (reset) begin
        crc_reg <= 16'h0000;
        crc      <= 8'h00;
    end

    else if (init) begin
        crc_reg <= 16'h0000;
        crc      <= 8'h00;
    end

    else if (calc & d_valid) begin
        crc_reg <= next_crc;
        crc      <= ~(next_crc[8], next_crc[9], next_crc[10], next_crc[11],
                      next_crc[12], next_crc[13], next_crc[14], next_crc[15]);
    end

    else if (~calc & d_valid) begin
        crc_reg <= {crc_reg[7:0], 8'h00};
        crc      <= ~{crc_reg[0], crc_reg[1], crc_reg[2], crc_reg[3],
                      crc_reg[4], crc_reg[5], crc_reg[6], crc_reg[7]};
    end

end

assign next_crc[0] = crc_reg[12] ^ d[7] ^ crc_reg[8] ^ d[3];
assign next_crc[1] = crc_reg[13] ^ d[6] ^ d[2] ^ crc_reg[9];
assign next_crc[2] = d[5] ^ crc_reg[14] ^ d[1] ^ crc_reg[10];
assign next_crc[3] = d[4] ^ crc_reg[15] ^ d[0] ^ crc_reg[11];
assign next_crc[4] = crc_reg[12] ^ d[3];
assign next_crc[5] = crc_reg[12] ^ crc_reg[13] ^ d[7] ^ crc_reg[8] ^ d[2] ^ d[3];
assign next_crc[6] = crc_reg[13] ^ d[6] ^ crc_reg[14] ^ d[1] ^ d[2] ^ crc_reg[9];
assign next_crc[7] = d[5] ^ crc_reg[14] ^ crc_reg[15] ^ d[0] ^ d[1] ^ crc_reg[10];
assign next_crc[8] = d[4] ^ crc_reg[15] ^ d[0] ^ crc_reg[0] ^ crc_reg[11];
assign next_crc[9] = crc_reg[12] ^ crc_reg[1] ^ d[3];
assign next_crc[10] = crc_reg[13] ^ d[2] ^ crc_reg[2];
assign next_crc[11] = crc_reg[3] ^ crc_reg[14] ^ d[1];
assign next_crc[12] = crc_reg[12] ^ crc_reg[4] ^ d[7] ^ crc_reg[15] ^ d[0]
                    ^ crc_reg[8] ^ d[3];
assign next_crc[13] = crc_reg[13] ^ d[6] ^ crc_reg[5] ^ d[2] ^ crc_reg[9];

```

```
assign next_crc[14] = d[5] ^ crc_reg[14] ^ crc_reg[6] ^ d[1] ^ crc_reg[10];
assign next_crc[15] = d[4] ^ crc_reg[15] ^ d[0] ^ crc_reg[7] ^ crc_reg[11];
endmodule
```

思考与练习

1. 设计一个先进先出存储器 (FIFO)，其尺寸为 8×256 ，端口包括时钟信号，读、写使能信号，数据输入、输出信号和存储器状态指示信号（空、满）等。
2. 设计一个移位相加乘法器，其设计思路是：乘法通过逐项移位相加来实现，根据乘数的每一位是否为 1 进行计算，若为 1 则将被乘数移位相加。
3. 改变 12.3 节中 FIR 滤波器的指标，设计并实现一个 15 阶的 FIR 滤波器。
4. 设计实现一个 8 点的 FFT 运算模块。
5. 设计一个长度为 15 的 M 序列产生电路。

第 13 章 EDA 软件使用指南

本章内容简介

- ◆ 综合工具 Synplify Pro
- ◆ 综合工具 Synplify
- ◆ 仿真工具 ModelSim
- ◆ 集成工具 ispLEVER
- ◆ 集成工具 MAX+PLUS II
- ◆ 集成工具 Quartus II

本章介绍典型 EDA 工具软件的使用方法, 熟练掌握这些开发工具, 对于完成 Verilog HDL 程序设计具有重要的意义。这些开发工具不仅功能强, 而且相互之间还具有良好的接口, 可以配合使用, 为高质量的系统设计提供了有力的平台支持。比如我们可以用 Synplify 取代集成工具中的 Verilog/VHDL 综合功能, 用 Verilog/VHDL 混合仿真器 ModelSim 对 MAX+PLUS II 或 ispLEVER 等输出的 HDL 代码进行行为仿真和时序仿真。

EDA 工具对于 HDL 设计开发具有非常重要的作用, 只有在这些高质量 EDA 工具的作用下, 才能将 HDL 文本设计转化为具体的电路结构网表, 实现作者的设计思想; 同时, 只有基于这些工具的支持, 才能对一些大型的系统设计进行仿真和预测, 以发现问题, 修改设计方案, 避免损失。可见, 好的 EDA 工具对于提高设计效率、改善优化质量、增强仿真测试的排错能力、加强设计成果的可移植性和可重用性以及扩展目标器件的选择范围等, 都有着直接的影响。此外, EDA 工具为学习和使用 HDL 语言提供了环境支持, 我们学习和使用 HDL 语言, 无不基于某一开发平台, 有些开发工具, 如 Synplify Pro/Synplify, 可将 HDL 语言设计的电路以图形的方式呈现出来, 非常直观, 便于学习者建立 HDL 语言与硬件电路之间的对应关系, 加深理解。因此, 熟练掌握这些工具的使用方法和使用技巧, 对于语言的学习不无好处。

13.1 概 述

在一般的设计开发中, 我们常使用由 PLD 公司提供的集成 EDA 软件, 这些集成的开发软件提供了设计输入编辑器、HDL 综合器、FPGA/CPLD 适配器、门级仿真器和编程下载器等一整套完整的开发工具, 如 Lattice 的 ispLEVER、Altera 的 MAX+PLUS II 和 Quartus II、

Xilinx 的 ISE 等等。但有时为了完成更大型的设计项目,我们往往要使用具有更优秀功能、更强大综合与仿真测试能力的专业 EDA 工具,通过在不同的综合与模拟层次上选用更适合的 EDA 工具进行系统设计来提高项目的设计效率 and 设计质量,是当前 EDA 工程中十分常用的设计流程。

不同的工具如何配合使用,以最有效的方式完成设计任务就成为我们必须清楚并熟练掌握的内容。事实上,优秀的 EDA 软件都含有相应的接口功能,相互之间能组合使用,共享数据,为用户的设计提供了便利。

图 13.1 展示了用不同设计工具相互配合完成 EDA 设计的流程图。首先,在将 Verilog/VHDL 设计文本通过文本编辑器输入后,没有使用集成开发工具自带的综合器,而选用第三方 EDA 公司提供的专业综合器,如 Synplify Pro/Synplify、FPGA Compiler II 或 Leonardo Spectrum 等。综合结束后再将综合产生的网表文件返回到主设计软件的适配器中,进行针对某种目标器件的适配;对于适配结果的仿真测试,也没有使用自带的门级仿真器来完成,而是将适配器输出的仿真文件提供给 HDL 通用仿真器(如 ModelSim)进行时序仿真,并根据仿真的结果对源文件进行修改完善,直至完全满足设计的要求为止。

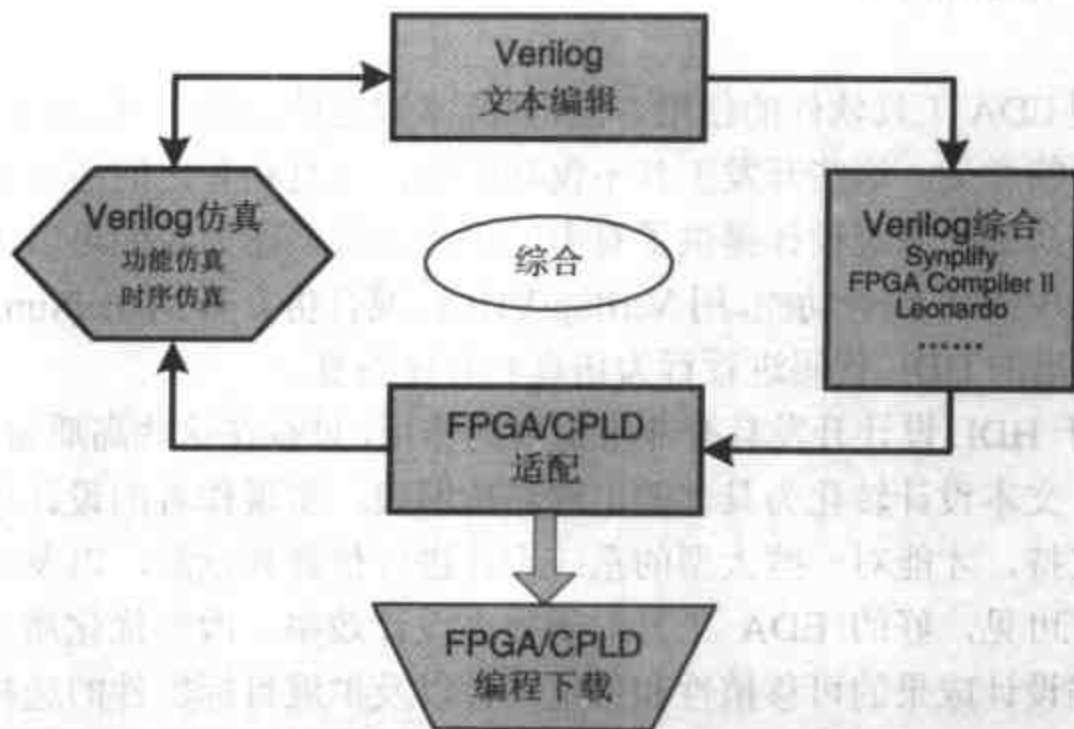


图 13.1 不同设计工具配合完成的 EDA 设计流程

输入设计文件后,也可以直接利用仿真器进行行为仿真和功能仿真,在设计的最初即可将得到的设计结果也下载到 FPGA 中进行硬件调试了。

13.2 Synplify Pro 使用指南

Synplify Pro/Synplify 是由位于美国加州 Sunnyvale 的 Synplicity 公司推出的专门用于 FPGA/CPLD 的逻辑综合工具,它支持 Verilog HDL 和 VHDL 高层次设计描述,在综合优化方面性能优异,应用广泛。

Synplify Pro/Synplify 支持 Verilog 1364-1995 标准和 VHDL 1076-1993 标准,能以很高的效率将 Verilog/VHDL 设计文件转换为针对选定器件的标准网表,并提供相应设计环境的配置

文件，在综合后还可以生成 Verilog 和 VHDL 仿真网表，以便对原设计进行功能仿真。本节介绍 Synplify Pro7.2 版本的基本功能和使用方法。

13.2.1 Synplify Pro 的性能特点

高性能的 FPGA 综合技术随着设计复杂性的提高、FPGA 器件复杂性的提高而不断增强。FPGA 器件现在已经达到几百万门的规模，速度也达到 200MHz 以上，如此复杂的设计对综合工具的性能提出了更高的要求。Synplify Pro 所独有的功能特点和极快的编译速度使它成为强有力的综合工具，而且它还附加了调试、优化功能以及项目管理功能等，因此用 Synplify Pro 进行设计可以提高 FPGA 设计的性能，缩短开发周期，优化设计效果。表 13.1 中对 Synplify Pro 的功能特点进行了总结，下面对一些关键技术作进一步的说明。

表 13.1 Synplify Pro 的功能与特点

Synplify Pro 的功能	特 点
特有的 B.E.S.T 算法	Synplify Pro 的核心算法称为 B.E.S.T (Behavior Extracting Synthesis Technology) 算法，与传统工具相比，它能在很短的时间内完成整个设计的优化
快速的编译速度	减少了综合的时间，适合大型设计
SCOPE 对设计多层次的约束	使设计者能够完全控制综合过程
支持标准的 HDL 语言	支持 Verilog HDL、VHDL 及混合语言的设计
语言的敏感编辑器	可以自动对 Verilog HDL、VHDL 进行语法检查
自动识别 RAM	减少了手工例化 RAM 的麻烦
第三方工具接口	可以与流行的仿真工具和输入工具之间实现数据共享
有限状态机开发器 (FSM Explorer)	FSM Explorer 可以自动识别有限状态机，并且对状态机的不同编码方式做评估，然后根据约束条件选择最佳的编码方式
有限状态机编译器 (FSM Compiler)	可以将 FSM 设计显示成状态图方式，使结果更加易读
自动 Retiming 技术	能自动在组合逻辑中移动寄存器，以平衡延迟，提高性能
乘法器和 ROM 的优化	自动对 ROM 和乘法器做流水线，以达到更快的性能
创建探针	允许把任何信号连到芯片的引脚做测试而不改变源码
关键路径的互相标识	可以在第三方工具的时序报告和 HDL 分析器中互相标识
HDL 分析器 (HDL Analyst)	寄存器传输级的分析调试工具，能从 HDL 代码中产生 RTL 模块框图，帮助识别关键路径和调试电路功能

◆ **Retiming 技术:** Synplify Pro 具有 Retiming 技术, 允许自动移动寄存器之间的组合逻辑, 以达到寄存器之间路径延迟的平衡, 这样可将电路的性能提高 20% 以上。Retiming 可以作为一个全局的选项, 也可以只针对部分电路。

◆ **HDL 分析器 (HDL Analyst):** HDL Analyst 是 RTL 图形分析和调试工具, 它可以提供门级和更高层的视图, 也可以连接到 HDL 源代码, 用 HDL Analyst 可以方便、快速地调试 HDL 源代码, 以提高性能。

◆ **有限状态机开发器:** Synplify Pro 的一个非常有用的特性就是它的有限状态机开发器 (FSM Explorer) 和增强的有限状态机编译器 (FSM Compiler)。FSM Explorer 可以自动识别有限状态机, 并且对状态机的不同编码方式做评估, 然后根据约束条件选择最佳的编码方式。FSM Compiler 可以将 FSM 设计显示成状态图方式, 使结果更加易读, 这种图形界面对调试非常有用。

◆ **流水线 (Pipelining) 设计技术:** 采用 Pipelining 设计技术可以提高算术操作的性能, Synplify Pro 可以通过自动移动 ROM 和乘法器内部的寄存器来创造流水线。

◆ **Amplify Physical Optimizer:** Amplify 是高性能的物理综合器, 它允许利用 RTL 图来对设计做出物理约束, 这种同时考虑布局和逻辑优化的新的算法比单独做逻辑综合最多可以提高 40% 以上的性能。

◆ **设计工具接口:** Synplify Pro 和很多仿真器 (如 NC-Verilog、NC-VHDL、Active-HDL、ModelSim 以及 Speedwave 等) 都有接口, 可以实现组合使用。

◆ Synplify Pro 还具有图形调试功能, 即将 HDL 设计文件编译和综合后, 产生相应的电路原理图, 以使用户从电路图中观察结果, 并且在图形显示时, 可交叉参考对应的 HDL 源代码。产生的原理图的表达方式有两种, 即 RTL 图和门级结构 Technology 视图。

RTL 图是综合的通用原理图描述, 不随选定的目标器件的不同而改变, 它仅与 Verilog/VHDL 代码描述的功能有关。Synplify Pro 可以将 Verilog 文件转成 RTL 图形的功能十分有利于 Verilog 语言的学习。

Technology 视图是针对选定器件的门级结构原理图, 并随着选定器件的不同而不同。

◆ Synplify Pro 能够生成几乎所有主流 FPGA/CPLD 器件的网表, 即能与这些器件相关的所有公司的适配器接口, 这些公司包括 Actel、Altera、Lattice (含 Vantis 公司)、Atmel、QuickLogic、Xilinx 等。

13.2.2 Synplify Pro 使用指南

下面以 1 位全加器的综合为例, 介绍 Synplify Pro 对 Verilog HDL 设计的综合以及与 MAX+PLUS II 的接口方法。需要注意的是, 对于较复杂的设计, LPM 和 EAB 的使用以及引脚的锁定都需要在 Synplify Pro 综合前的原文件中进行设置和说明。

1. 输入设计

首先进入 Synplify Pro 集成环境, 如图 13.2 所示。

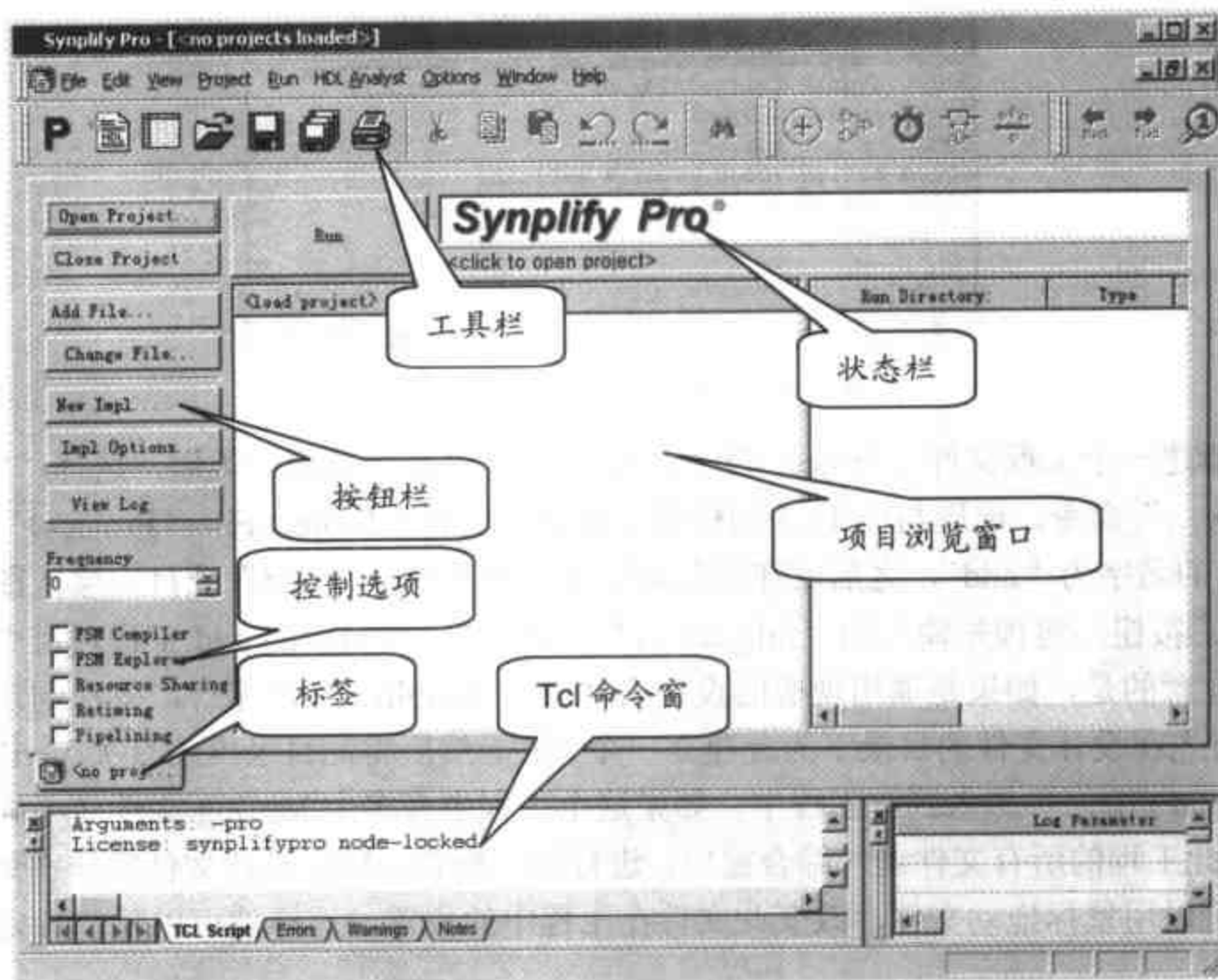


图 13.2 Synplify Pro 的工作界面

选择“File”菜单下的“New...”命令，弹出如图 13.3 所示的对话框，选择“Verilog File”选项，填写 Verilog 程序的名字为“full_add”，选择 Verilog 程序存储的目录，然后进入 Verilog 文本编辑器，在如图 13.4 所示的文本编辑器中输入 1 位全加器的源程序。

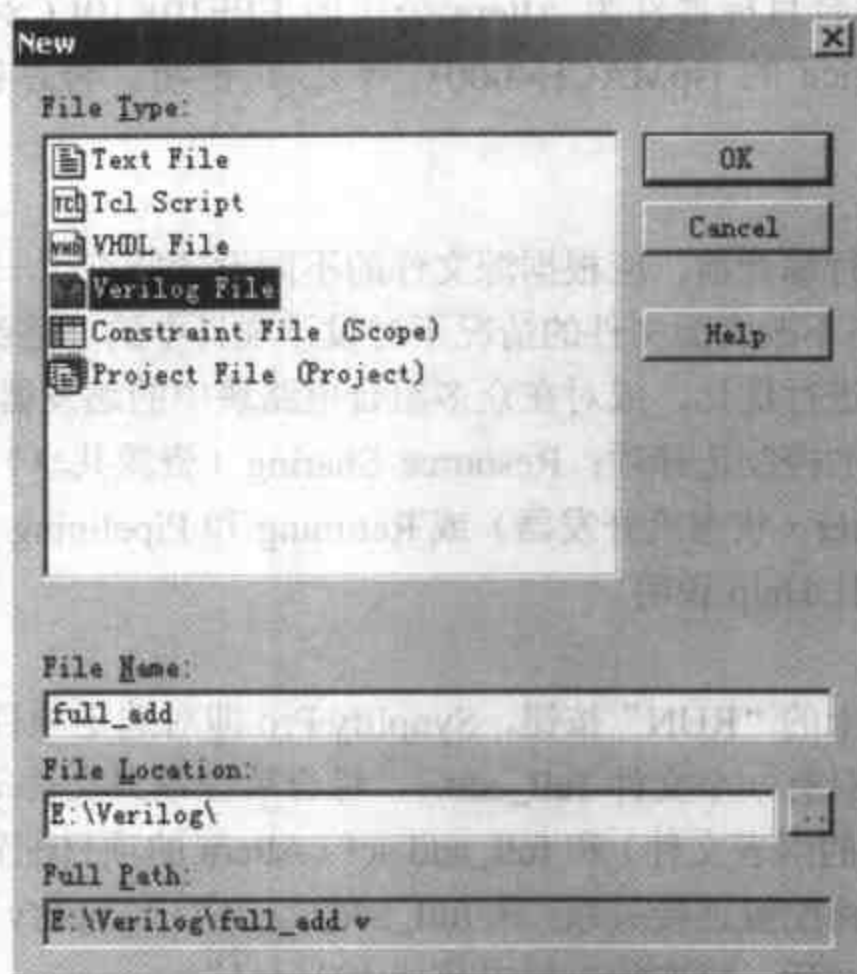


图 13.3 Synplify Pro 新建文件、新建项目对话框

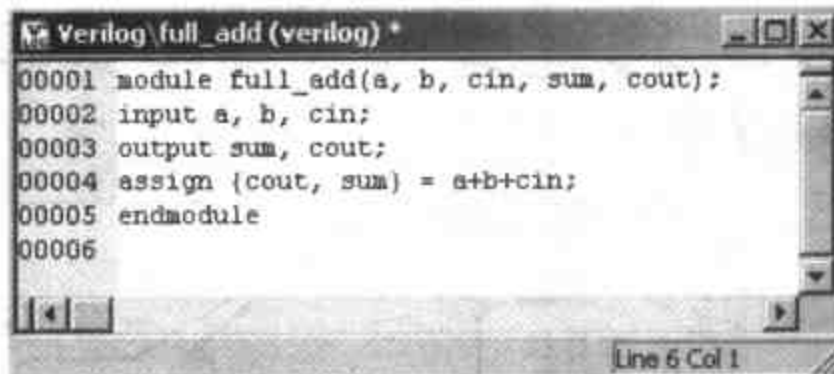


图 13.4 在文本编辑器中输入 1 位全加器源程序

还要新建一个工程文件 (Project File), 与上面新建文件的操作类似, 选择 “File” 菜单下的 “New...” 命令, 出现与图 13.3 同样的对话框, 选择 “Project File (Project)” 选项, 填写 Project 的名字为 “add”, 之后就在项目浏览窗口中出现了 “add” 项目的层次图。接着单击 **Add File...** 按钮, 将预先输入的 “full_add.v” 文件加入到当前的 add 工程项目中。

另需注意的是, 如果是调用现成的设计文件进入 Synplify 进行综合, 方法同这里介绍的类似, 需首先在设计文件的目录下为此建立一个工程, 然后将此目录内的有关 Verilog 文件通过按键 “Add File...” 加入到此工程中。如果这个工程中有多个 Verilog 文件 (不同结构层次的), 应将此工程的所有文件调入综合窗口, 进行统一综合。顶层设计文件应排在最下面。如果不是, 可以用鼠标拖动文件, 改变此文件在工程中的位置, 这样就可以随意指定顶层设计文件了。

2. 选择目标器件

按下 **Impl Options...** 按钮 (Implementation Option), 在弹出的对话框 “Options for Implementation” 中的 “Device” 标签页中选择 “Technology”: “Altera” 的 “FLEX10K” 系列; Part (型号) 选择 “10K10”, Speed (速度) 为 -3; Package (封装) 为 LC84, 再按 “OK” 按钮, 表明此设计的综合目标器件为 Altera 公司的 EPF10K10LC84-3, (当然也可以选用其他器件系列, 如 Lattice 的 ispMACH4000V 等)。这说明综合是针对某一系列特定的器件来进行的。

3. 综合前控制设置

在对输入的文件进行综合前, 应根据源文件的不同设计特点作一些针对改善综合方式的控制。例如设计者希望在不改变源文件的情况下对设计项目中的电路结构进行资源共享优化, 或对其中的有限状态机进行优化, 或对在众多组合电路块中的触发器重新放置以提高运行速度, 可以分别选中左栏的控制选择项: Resource Sharing (资源共享)、FSM Compiler (状态机编译器)、FSM Explorer (状态机开发器) 或 Retiming 和 Pipelining (流水线设计)。它们的详细功能可以参考该软件 Help 说明。

4. 综合

最后按下状态栏边上的 “RUN” 按钮, Synplify Pro 即对选定项目 add.prj 中的 Verilog 文件进行综合。在本例中只有一个文件 full_add.v, 综合后在项目文件 add.prj 所在的目录生成 full_add.edf (EDIF 格式的网表文件) 和 full_add.acf (Altera 的项目配置文件)。在 full_add.edf 中描述了对应器件的电路配置连接关系; 在 full_add.acf 中, Synplify 对 Altera 的编译环境进行了优化配置。综合完成后, 将在状态栏中显示 “Done!”。

如果在综合中发现错误, 将显示于项目浏览窗口中, 用鼠标双击出错文件, 即可进入文

件编辑器进行修改。

5. 查看结果

在综合完成后, 可以选择“HDL Analyst”→“RTL”→“Hierarchical View”菜单, 进入 RTL 级原理图观察窗, 通过此窗可以浏览经过综合生成的 RTL (寄存器传输级) 方式的电路原理图, 本例生成的 1 位全加器的 RTL 原理图如图 13.5 所示, 仔细核对此电路与设计程序的功能描述是否吻合。这是根据源程序的逻辑描述生成的原理图, 可用于定性检查原设计正确与否。此功能对调试 Verilog 程序非常有用。

为了得到设计综合后的门级电路图, 可在“HDL Analyst”菜单中选择“Technology”→“Flattened to Gates View”命令。注意, 对于不同的综合目标器件, 尽管逻辑功能一样, 但其门级电路结构是不一样的。1 位全加器的门级原理图如图 13.6 所示。

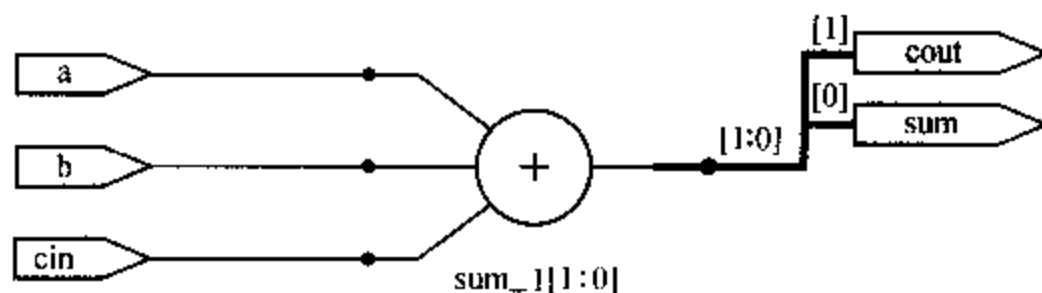


图 13.5 Synplify Pro 综合后的 RTL 级原理图

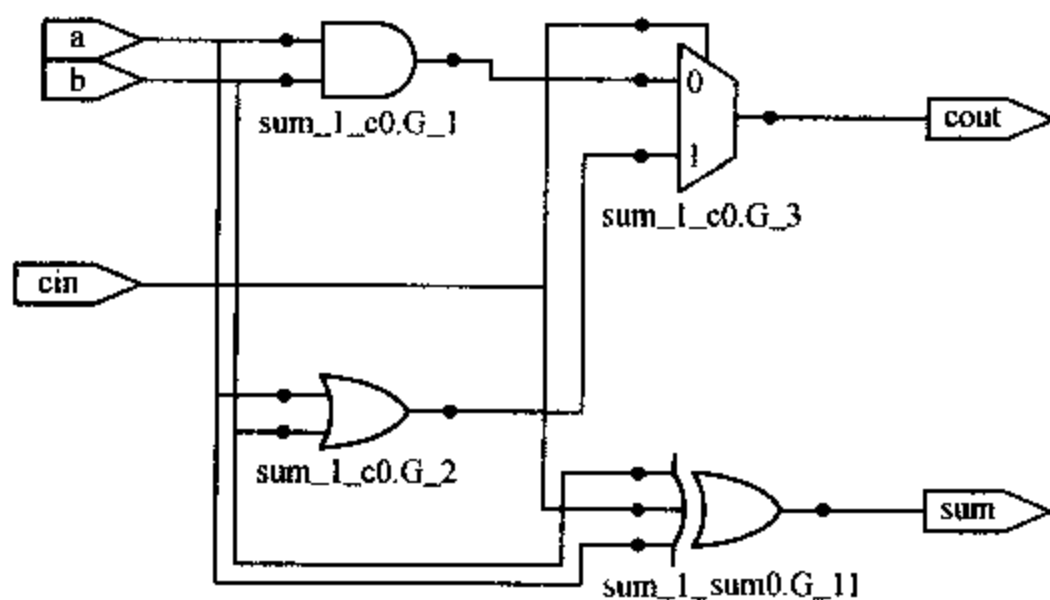


图 13.6 Synplify Pro 综合后的门级原理图

这样, 1 位全加器的 Synplify Pro 综合过程已基本完成, 下面介绍用 MAX+PLUS II 对 Synplify Pro 生成的.edf 文件作进一步处理 (适配、仿真等) 的过程。

13.2.3 Synplify Pro 与 MAX+PLUS II 的接口

1. 设定 EDF 文件为工程

MAX+PLUS II 具有良好的接口性能, 能接受来自多个 EDA 公司综合器产品的 EDIF 标准网表文件, 这些公司包括 Viewlogic、Synopsys、Synplicity、OrCAD、Cadence、Mentor Graphics 和 Exemplar Logic 等。

通过 Synplify Pro 综合器获得对应某选定器件 (如 EPF10K10) 的 EDIF 网表文件 full_add.edf 和用于 MAX+PLUS II 的配置文件 full_add.acf 后, 用 MAX+PLUS II 导入 EDIF

网表文件，将其映射到实际芯片中进行布线适配，形成最后可用于器件下载的编程文件。

打开 MAX+PLUS II 集成环境，进入 Compiler 环境界面，选择“File”→“Projects”下的“Name...”命令。在随后弹出的“Project Name”对话框中（先把 ☐ Show Only Tops of Hierarchies 方框中的“√”去掉）选择由 Synplify Pro 综合出的 full_add.edf 文件，将其设定为工程文件。

2. 选定 EDF 文件来源

仍然在编译器 (Compiler) 界面，在 Interfaces 菜单中选择“EDIF Netlist Reader Settings...”选项，然后在弹出的窗口（如图 13.7 所示）中的“Vendor”下拉列表框中选择用于生成当前 EDF 文件的综合器的提供商，这里选 Synplicity。这一步，MAX+PLUS II 一般会自动选择好，而不需要手动选择。

有时可能需要改变 EDIF 导入时的库映射文件.lmf (Library Mapping File)，这需要在图 13.7 的窗口中单击“Customize”，并如图选择对应的.lmf 文件。

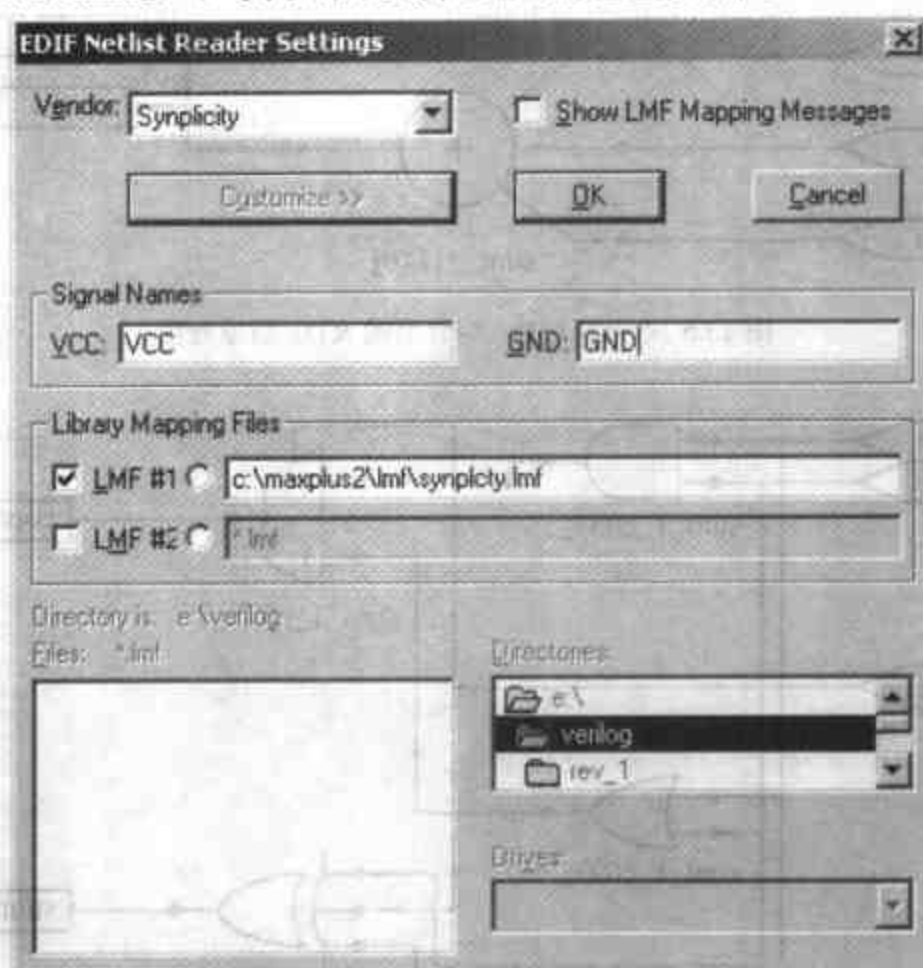


图 13.7 EDIF Netlist Reader Settings 对话框

3. 选定目标器件

通过 Assign→Device，选择目标器件。注意，此器件必须与在 Synplify Pro 中选择的器件一致。

4. 编译适配

到了适配和下载阶段，已和外部的 EDA 工具没有任何关系了，MAX+PLUS II 的操作流程可参考 13.6 节的内容。

5. 小结

对上述步骤作个小结，图 13.8 给出了 Synplify Pro 和 Altera 接口详细流程。图中 VHDL/VerilogHDL 的设计描述在进入 Synplify Pro 综合器进行综合以前还必须将 Altera 器件工艺中有关目标器件的硬件结构工艺提供给综合器，此外还要提供包括优化方式、速度指标和资源利用情况等的设计约束文件。对于综合的结果，可以利用仿真软件进行测试（以上介

绍的流程是适配后进行仿真)。

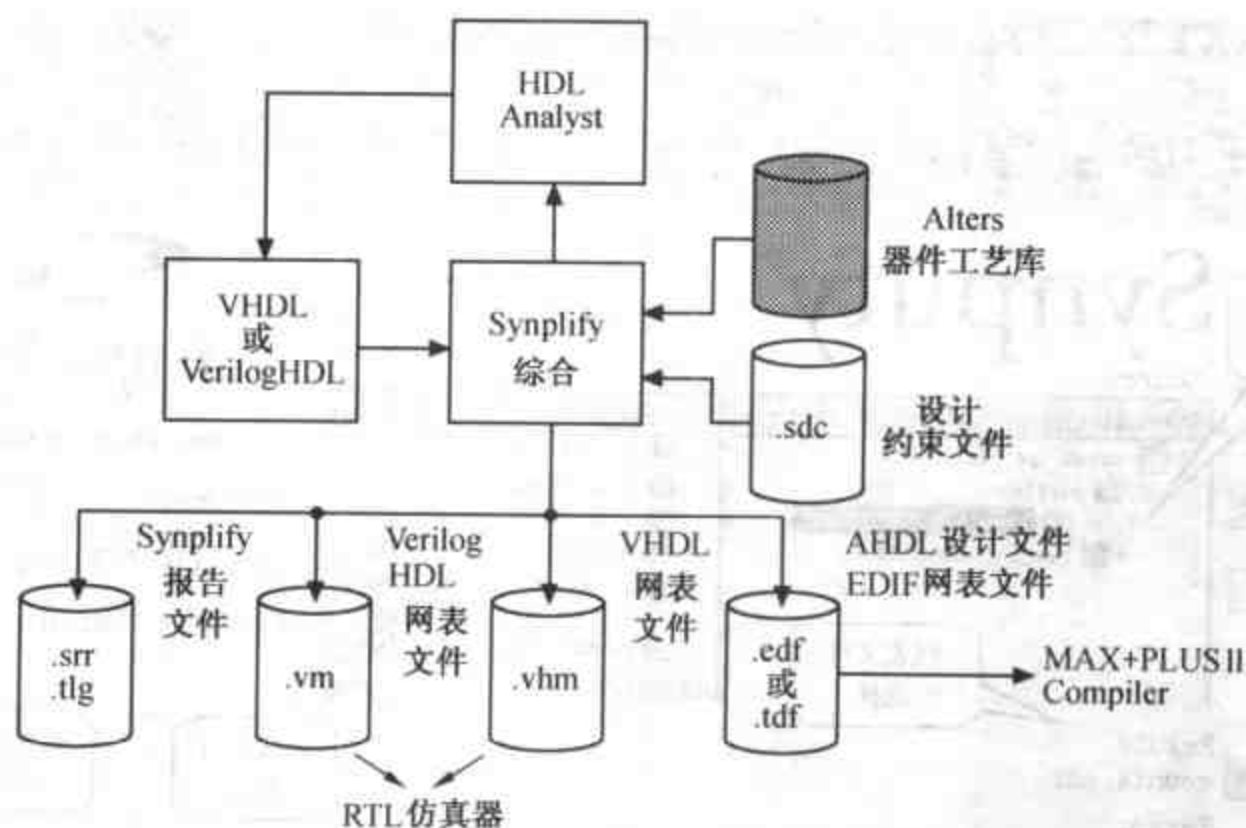


图 13.8 Synplify Pro 与 Altera 的接口流程

综合后除产生综合报告和仿真网表文件外，更重要的是产生 EDIF 格式的网表文件，它为 MAX+PLUS II 的编译器提供适配文件。

13.3 Synplify 使用指南

Synplify 的性能特点与 Synplify Pro 类似，Synplify 也是优秀的 FPGA/CPLD 逻辑综合工具，Synplify 支持 VHDL 1076-1993 标准和 Verilog 1364-1995 标准。Synplify 能高效地将 Verilog/VHDL 设计文件转换为针对选定器件的标准网表，并提供相应设计环境的配置文件。Synplify 在综合后生成 Verilog 和 VHDL 仿真网表，以便对原设计进行功能仿真。Synplify 具有资源共享优化功能，并含符号化的 FSM（有限状态机）编译器，可以实现高级的状态机优化，此外还配备了一个内置的语法敏感编辑器，可以在 HDL 源文件中高亮显示综合后的错误和警告，使用户能迅速定位并纠正所出现的问题。

13.3.1 Synplify 的功能特点

如图 13.9 所示是 Synplify 软件启动后的界面，下面对界面中标注的几个地方作重点的介绍。

① project（工程文件树状列表）：显示当前的工程文件，并显示工程文件中的所有源文件，也可以通过左面的“Add”、“Change”和“Edit”按键添加 Verilog HDL 源文件、改变目录或对源文件进行修改（修改时应选中要修改的.v 文件，再点击“Edit”按钮）。

② Result（综合结果）：综合结果一般保存为.edf 格式的文件，该格式的文件能够被 FPGA 厂家的适配软件接受，以进行适配、仿真和下载等操作。

③ Target（目标器件选择）：指定将设计文件综合到哪家公司的哪种型号的器件中，用

户可通过左边的“Change”按键来改变器件。

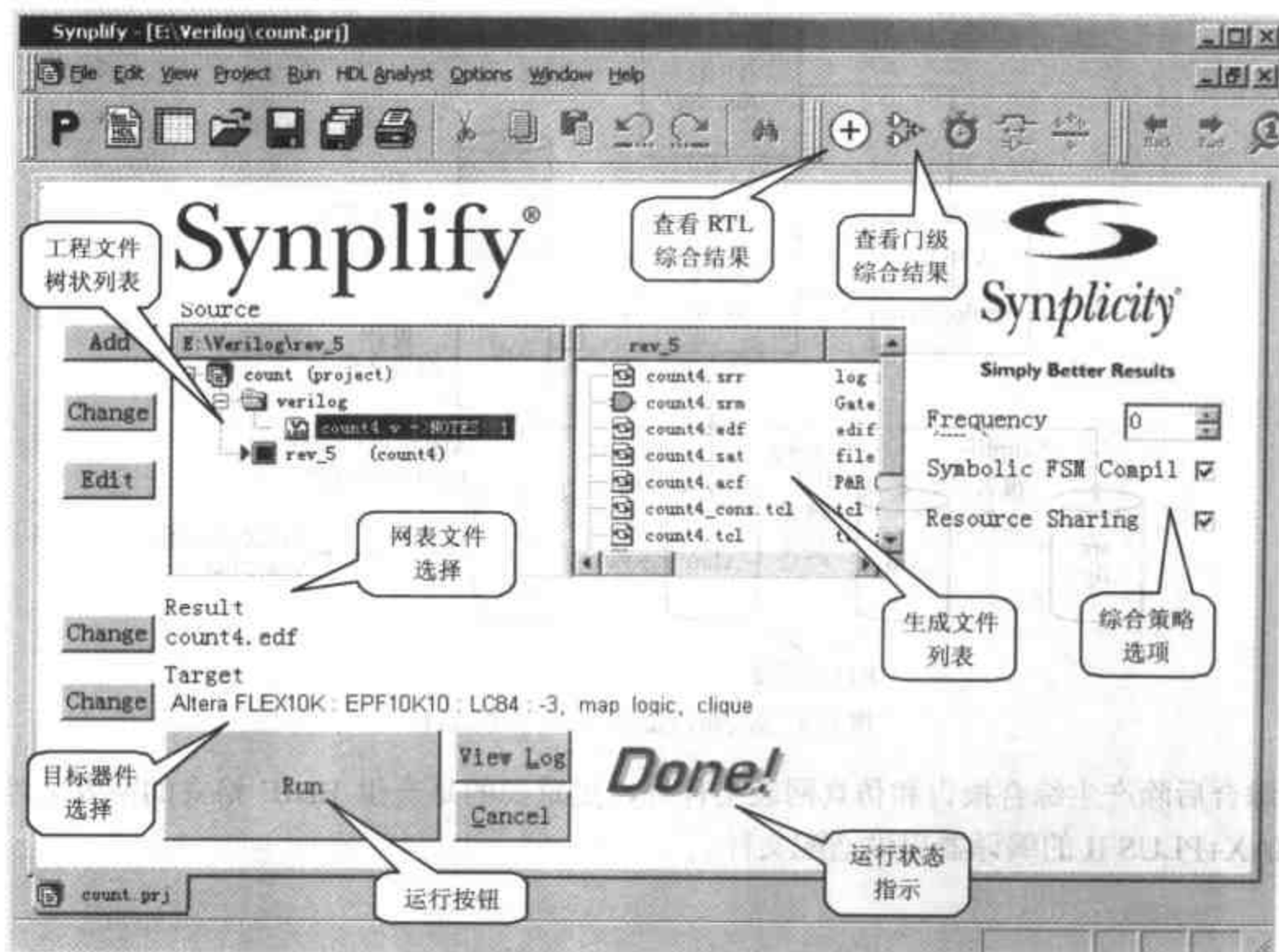


图 13.9 Synplify 的工作界面

④ 综合策略选项：“Symbolic FSM Compile”为符号化有限状态机编译选项，在含有状态机的设计中一般使能该选项；“Resource Sharing”为资源共享选项，一般建议使能该选项。

⑤ Run（运行按钮）：在各个选项都设计好以后，可以点击run按钮启动综合过程。

⑥ 运行状态指示：综合过程中，运行指示分别为“Compiling”、“Mapping”和“Done”，如果有错误“error”或者警告“Warning”，都会在“Done”的后面指出。

⑦ RTL View（查看 RTL 综合结果）：点击该按钮可将设计文件的 RTL 综合结果展示出来，RTL 视图与所选择的器件无关。

⑧ Technology View（查看门级综合结果）：该结果与所选用的器件有关系。

除了上面的几点之外，Synplify 综合器的 HDL 编辑器也是比较有特色的，体现在下面两点上：

(1) 编辑器对 HDL 关键词敏感：无论是 Verilog HDL 还是 VHDL，其关键字都以蓝色字体显示，注释以绿色字体显示。

(2) 可进行语法检查和综合检查：在综合的过程中，综合器能定位产生语法错误处，在“Run”菜单下选择“syntax check”可进行语法检查，选择“synthesis check”可进行综合检查。

Synplify 软件有助于帮助设计者建立“HDL 语言”和“硬件电路”两者之间的联系。

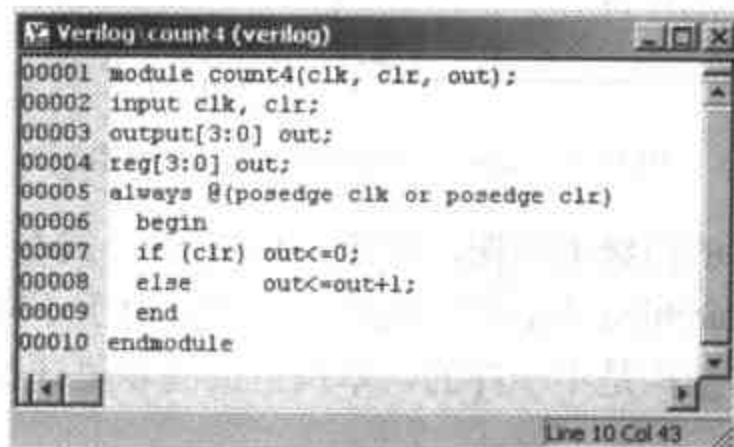
13.3.2 Synplify 使用举例

下面以 4 位计数器的综合为例介绍 Synplify 软件的具体使用。

1. 创建新工程

启动 Synplify 软件, 新建一个工程文件 (Project File), 选择 “File” 菜单下的 “New...” 命令, 出现与前面图 13.3 同样的对话框, 选择 “Project File (Project)” 选项, 填写 Project 的名字为 “count”, 之后就在项目浏览窗口出现了 “count” 项目的层次图。接着单击 **Add File...** 按钮, 将预先输入的 Verilog 设计文件 “count4.v” 加入到当前的 count 工程项目中。

也可以在文本编辑器中编辑 “count4.v” 文件, 编辑好的文本文件如图 13.10 所示。



```

00001 module count4(clk, clr, out);
00002 input clk, clr;
00003 output[3:0] out;
00004 reg[3:0] out;
00005 always @(posedge clk or posedge clr)
00006 begin
00007   if (clr) out<=0;
00008   else out<=out+1;
00009 end
00010 endmodule
  
```

图 13.10 count4.v 文本内容

2. 选择目标器件

按下 “Target” 左边的 **Change** 按钮, 在弹出的对话框 “Options for Implementation” 中的 “Device” 标签页中选择 “Technology”: “Altera” 的 “FLEX10K” 系列; Part (型号) 选择 “10K10”, Speed (速度) 为 -3; Package (封装) 为 LC84, 再按 “OK” 按钮, 表明此设计的综合目标器件为 Altera 公司的 EPF10K10LC84-3 (当然也可以选择其他器件系列)。

3. 综合方式选项

在对输入的文件进行综合前, 应根据源文件的不同设计特点作一些针对改善综合方式的控制。Synplify 软件提供了两个选项: “Symbolic FSM Compile” 为符号化有限状态机编译选项, 在含有状态机的设计中一般应选中该选项; “Resource Sharing” 为资源共享选项, 一般建议使用该选项, 它能够在不改变源文件的情况下对设计项目中的电路结构进行资源共享优化。

4. 开始综合

全部设置好后, 即可按下状态栏边上的 “RUN” 按钮, Synplify 即对当前项目 count.prj 中的 Verilog 文件进行综合。本例中只有一个文件 count4.v, 综合后在项目文件 count.prj 所在的目录生成 count4.edi (EDIF 格式的网表文件) 和 count4.acf (Altera 的项目配置文件)。在 count4.edi 中描述了对应器件的电路配置连接关系; 在 count4.acf 中, Synplify 对 Altera 的编译环境进行了优化配置。综合完成后, 将显示 “Done!”。

如果在综合中发现错误, 将显示于项目浏览窗口中, 用鼠标双击出错文件, 即可进入文件编辑器进行修改。

5. 查看综合结果

综合完成后, 可以点击按钮 , 或选择 “HDL Analyst” → “RTL” → “Hierarchical View” 菜单, 进入 RTL 级原理图观察窗, 通过此窗口可以浏览经过综合生成的 RTL (寄存器传输级) 方式的电路原理图。本例生成的 4 位计数器的 RTL 原理图如图 13.11 所示。仔细核对此电路与设计程序的功能描述是否吻合。这是根据源程序的逻辑描述生成的原理图, 可用于定性检

查原设计是否正确。此功能对调试 Verilog 设计非常有用。

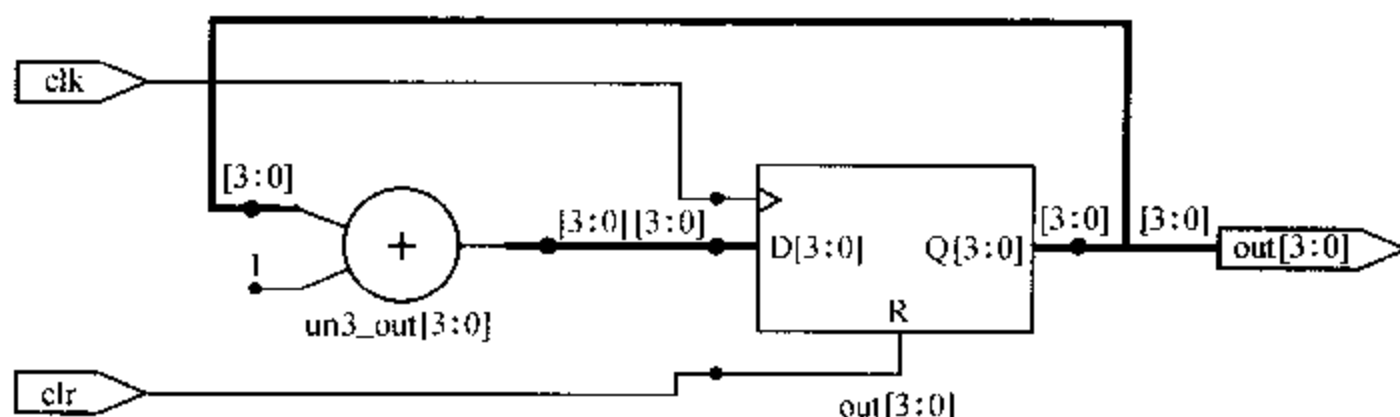


图 13.11 count4.v 综合后的 RTL 级原理图

为了得到此设计综合后的门级电路图，可点击按钮 ，或在“HDL Analyst”菜单中选择“Technology”→“Hierarchical View”菜单。注意，对于不同的综合目标器件，尽管逻辑功能一样，但其门级电路结构是不一样的，本例生成的 4 位计数器的门级结构原理图如图 13.12 所示。

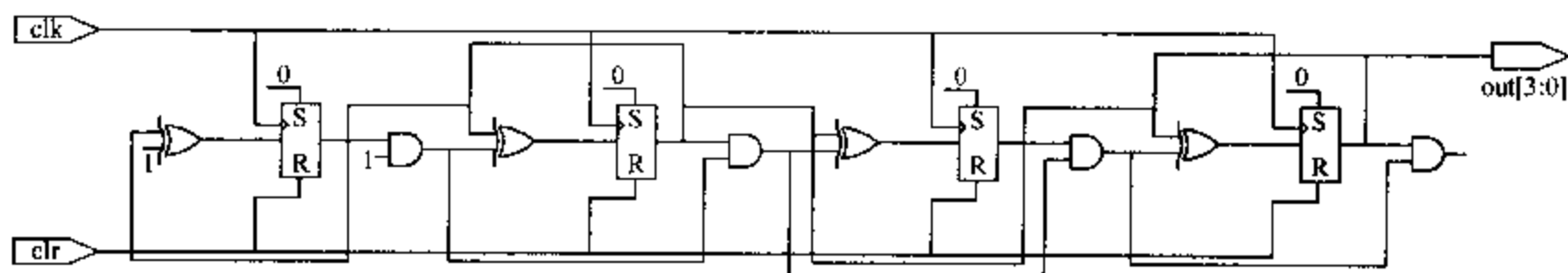


图 13.12 count4.v 综合后的门级结构原理图

6. 其他操作

至此，用 Synplify 进行综合的过程已结束，如要继续进行适配和下载等过程，可转到其他软件，如 MAX+PLUS II 进行进一步处理，具体操作可参考上一节的内容。

13.4 ModelSim 使用指南

ModelSim 是 Mentor Graphics 的子公司 Model Technology 的产品，属于编译型的 Verilog/VHDL 混合型仿真器。ModelSim 可以在同一个设计中单独或混合使用 Verilog 和 VHDL，允许 Verilog 模块调用 VHDL 的实体，或 VHDL 调用 Verilog。由于 ModelSim 是编译型仿真器，使用编译后的 HDL 库进行仿真，因此在进行仿真前，必须编译所有的待仿真的 HDL 文件成为 HDL 仿真库，在编译时使源文件获得优化，提高了仿真速度。

ModelSim 有不同的版本，主要有下面几种。同时，ModelSim 为不同芯片公司提供了 OEM 版本，以用于针对该公司芯片的时序仿真。

- ◆ ModelSim OEM
- ◆ ModelSim PLUS
- ◆ ModelSim SE

以 ModelSim SE 的功能最为全面，本节以 ModelSim SE 5.7C 的使用为例来介绍其使用方

法。ModelSim SE 5.7C 支持下列语言标准:

◆ Verilog HDL

IEEE Verilog 标准: IEEE 1364-'95。

IEEE Verilog2001 标准: IEEE1364-2001 (部分支持)。

OVI Verilog LRM 2.0 (大部分支持。OVI, Open Verilog International)。

PLI 1.0 (PLI, Programming Language Interface)。

VCD (Value Change Dump)。

◆ VHDL

IEEE VHDL'87 和'93 标准: IEEE Std. 1076-'87 &'93。

VHDL 多值逻辑系统标准: IEEE 1164-1993。

VHDL 标准数学程序包: IEEE 1076.2-1996。

◆ VITAL

VITAL2.2b 和 VITAL'95 (IEEE 1076.4-1995)。

VITAL 2000。

◆ SDF (Standard Delay Format, 标准延迟格式)

SDF 标准: SDF 1.0~3.0。

ModelSim 可以完成 3 个层次的 Verilog 仿真, 分别为 RTL 级仿真 (即将可综合的 Verilog 文件直接进入 ModelSim 进行功能仿真)、综合后门级仿真、适配后门级仿真 (时序仿真)。下面以 4 位计数器的仿真为例讲述 RTL 级功能仿真过程。

13.4.1 ModelSim 功能仿真

ModelSim 功能仿真过程可按下面的步骤进行。

1. 启动 ModelSim

图 13.13 是 ModelSim 的启动界面, 之后进入 ModelSim 工作界面, 如图 13.14 所示是 ModelSim 的主窗口 (Main Windows), 主要由下面几部分构成: 工作区 (Workspace)、命令窗口、工具栏等。

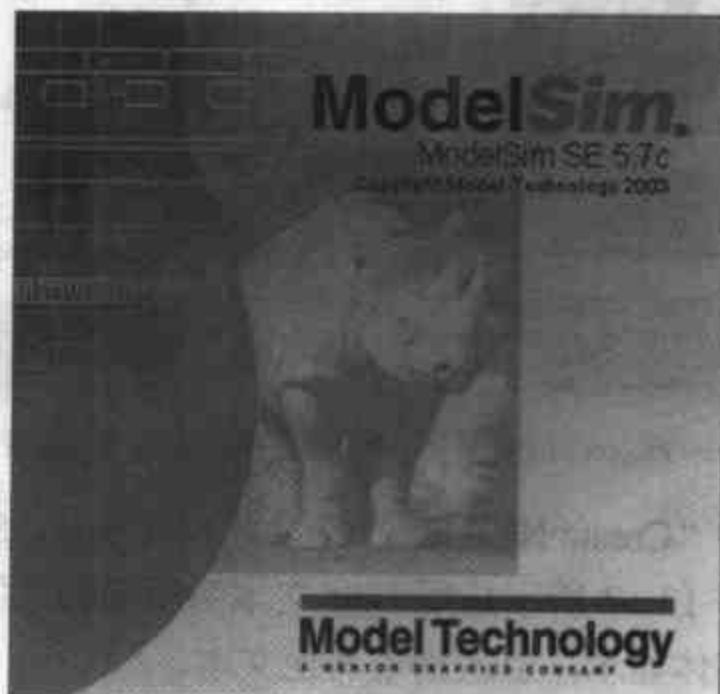


图 13.13 ModelSim 的启动界面

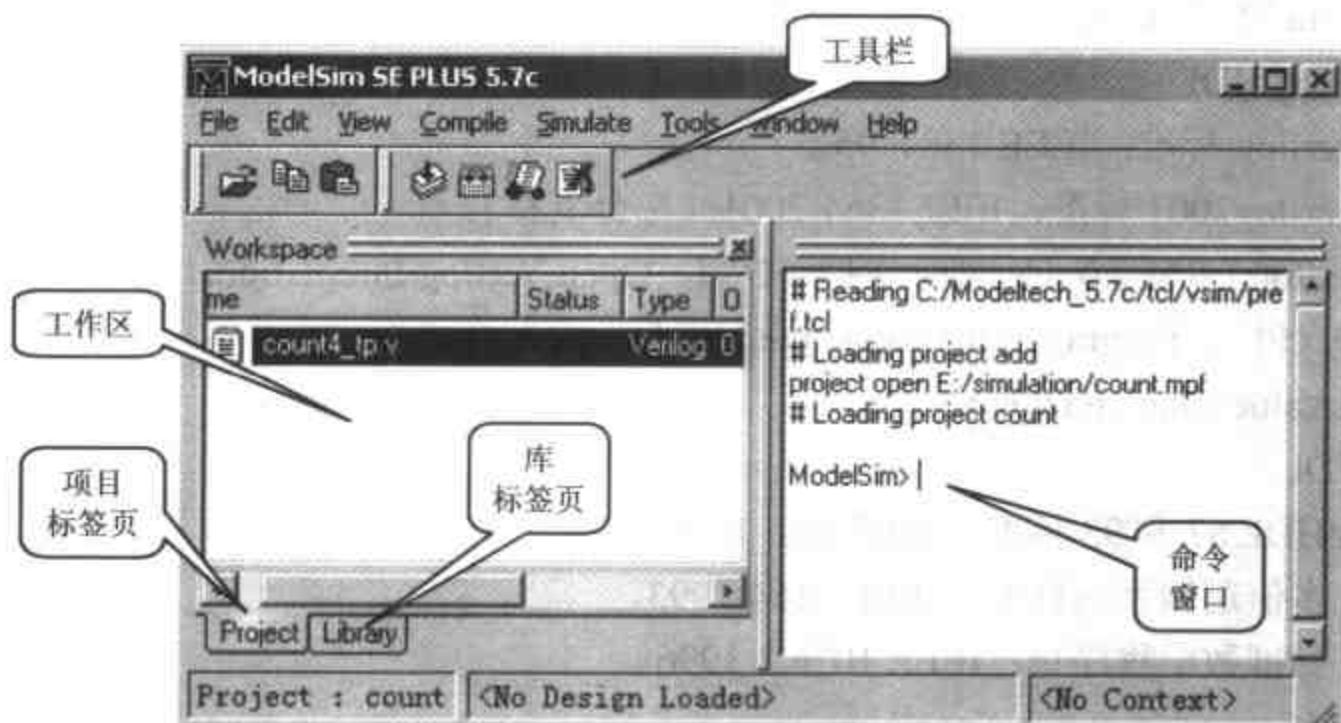


图 13.14 ModelSim 的主窗口

在命令窗口中可以输入 ModelSim 的命令（基于 TCL Script），并获得执行信息；在工作区用 Tree List（树状列表）的形式来观察 Library（库）、Project（项目源文件）、设计仿真的结构等。

2. 建立仿真工程项目

选择主窗口菜单“File”→“New”→“Project”，弹出建立新项目（Create Project）对话框（如图 13.15 所示）。填入要建的新项目名称以及所在文件夹。单击“OK”确定后，出现如图 13.16 所示的向当前项目添加“items”的对话框，点击相应的按钮可分别为当前项目创建新文件或将已存在的文件添加到当前项目中。也可以通过选择主窗口菜单“File”下的“Add to Project”→“New File”或“Existing File”为当前项目创建新文件或将已存在的文件添加到当前项目中，在弹出的“Add File to Project”对话框中选择加入的 Verilog 源文件的路径，即将已编辑好的 Verilog 源文件导入当前项目中。

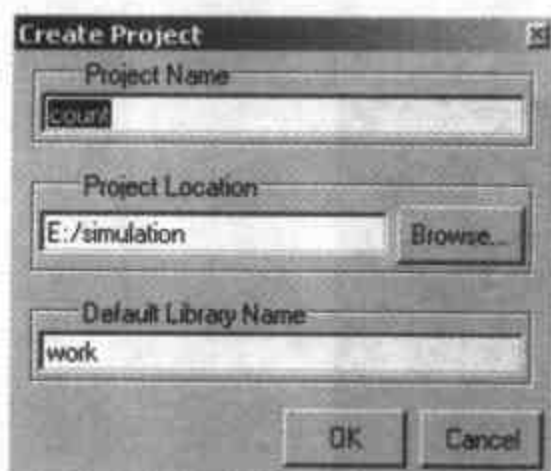


图 13.15 ModelSim 的 Create Project 对话框

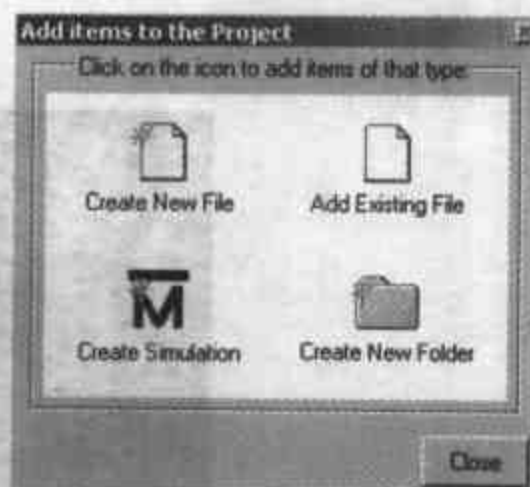


图 13.16 向当前项目添加文件（items）对话框

这里选择图 13.16 中的“Create New File”选项，打开 ModelSim 自带的文本编辑器 Edit 来输入和创建源文件，如图 13.17 所示是用 Edit 文本编辑器输入的 count_tp.v 的内容，所有的关键字都用特殊颜色的字体显示，以区别于其他文本。

单击“OK”确定后，在工作区的“Project”页中就出现了图表  count_tp.v，表示 count_tp.v

文件已加入到了 ModelSim 的当前仿真项目中。



图 13.17 在 Edit 文本编辑器中编辑 count_tp.v

3. 编译仿真文件

ModelSim 是一种编译型的仿真器,也就是说,ModelSim 在仿真前必须先编译仿真文件,即编译 Verilog 源文件进入“work”库。

在操作上,首先对 count_tp.v 进行编译,在工作区的 Project 页中选中  count_tp.v,然后单击鼠标右键,在出现的菜单中选择“Compile”→“Compile Selected”(如图 13.18 所示)即开始编译,在命令窗口中会出现编译的信息(如图 13.19 所示),如果编译通过,会显示“Compile of count_tp.v was successful.”,否则会显示出错信息,此时就应该在文本编辑器 Edit 中修改源文件,重新编译,直到通过为止。

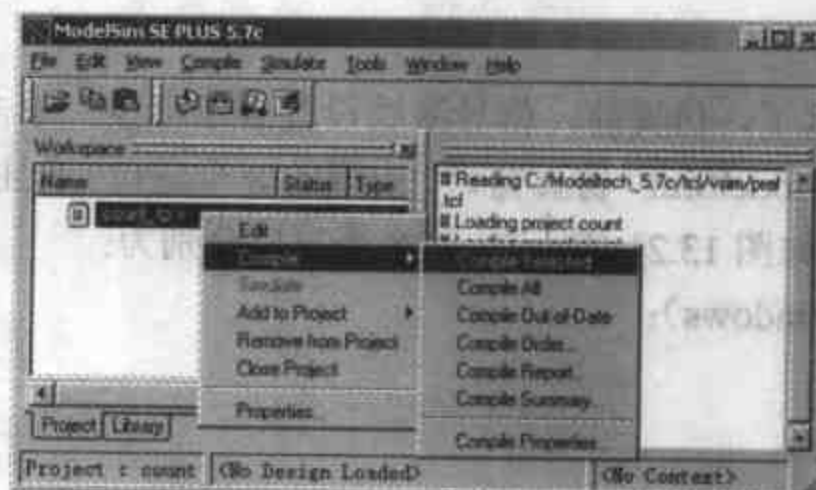


图 13.18 编译仿真文件

4. 装载仿真模块和仿真库

编译完成后,接着需要装载准备仿真的设计模块和仿真库。单击工作区中的“Library”标签页(如图 13.19 所示),把 Library 切换到 work 库,work 库就是当前的仿真工作库。我们可以看到,work 库中已经有 count_tp 仿真模块存在了,这是刚才编译的结果。选中该项目,单击鼠标右键,在出现的菜单中选择“Simulate”(如图 13.19 所示),当工作区中出现 Sim 标签页时(如图 13.20 所示),说明本仿真模块已装载成功。

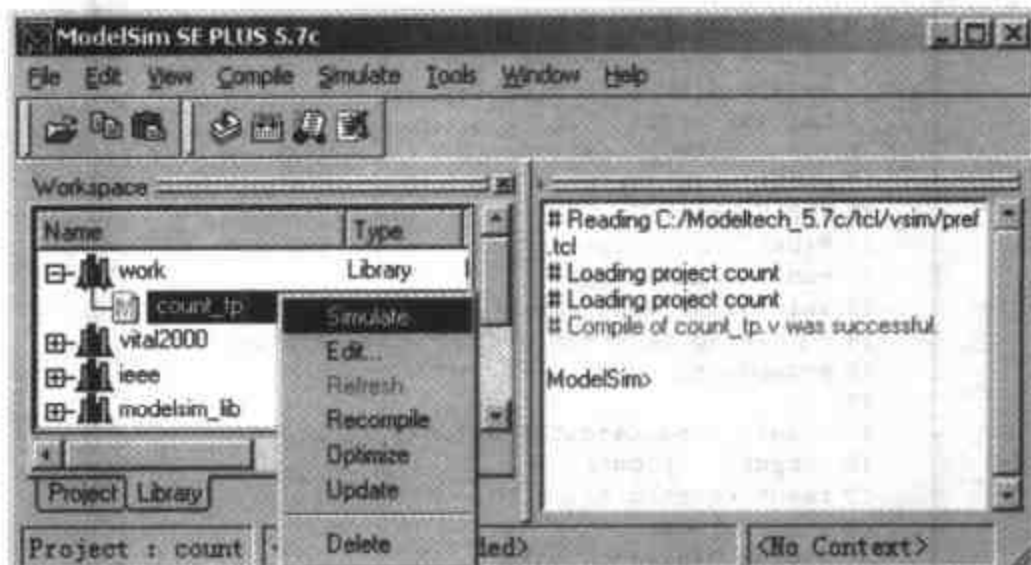


图 13.19 装载仿真模块

仿真模块加载前,主窗口中命令窗口的图标显示为: ModelSim>。

仿真模块加载后,主窗口中命令窗口的图标显示为: VSIM2>。

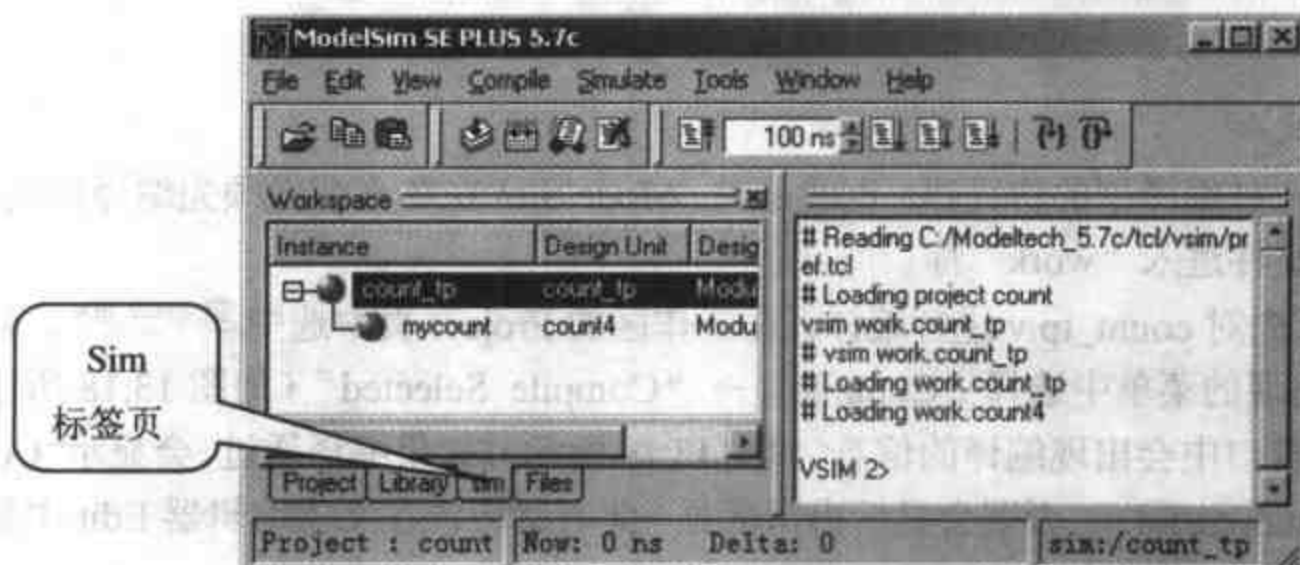


图 13.20 Sim 标签页

5. 执行仿真

下面就可以开始仿真了,仿真前,信号激励设置必不可少。单击菜单“View”→“All Windows”打开所有的“ModelSim”仿真窗口。出现的 8 个窗口(含主窗口)构成了 ModelSim 强大的 HDL 仿真环境(如图 13.21 所示)。这 8 个窗口分别为:

- ◆ 主窗口(Main Windows);
- ◆ 列表窗口(List);
- ◆ 波形窗口(Wave);
- ◆ 信号窗口(Signals);
- ◆ 源文件窗口(Source);

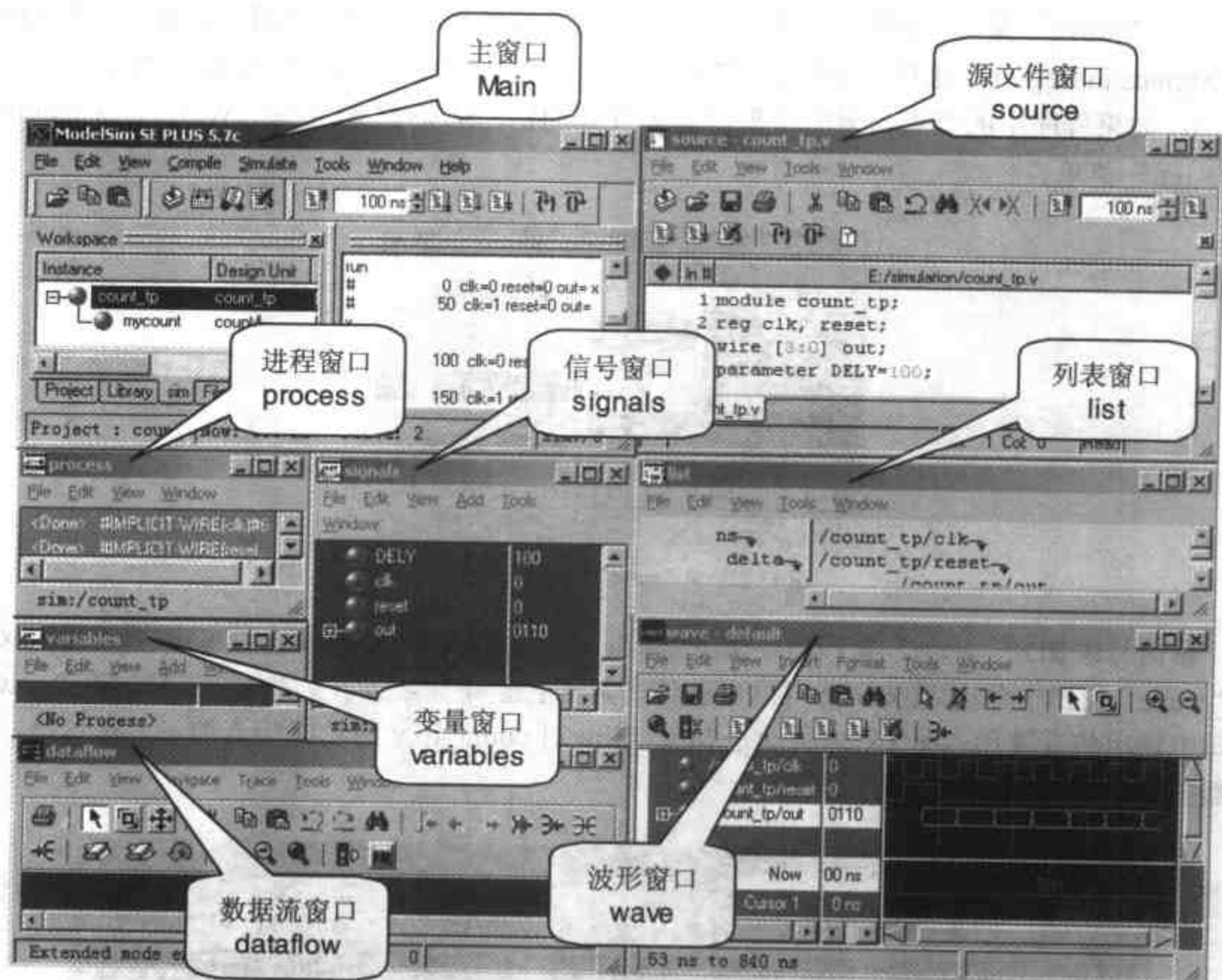


图 13.21 ModelSim 的 8 个仿真窗口

- ◆ 进程窗口 (Process);
- ◆ 变量窗口 (Variables);
- ◆ 数据流窗口 (Dataflow)。

由于 count_tp.v 文件中已经对激励信号和输出显示信号进行了设置, 因此主窗口的命令窗口中键入 Run 或者单击  按钮即可开始仿真。

如果是对设计模块本身 (如本例中的 count4.v 模块) 进行仿真, 应在 “Signals” 窗口中用该窗口下 “Edit” 菜单中的 “Force...”、“Clock...” 设置好信号的激励, 然后在主窗口的命令窗口中键入 Run 或者单击  按钮进行仿真, 并在适当的仿真时刻改变某些信号的激励。由于 ModelSim 是一个专业级的 HDL 仿真器, 仿真设置涉及步骤较多, 相关方法可参考该工具的帮助文档。

需要指出的是, 在 ModelSim 中对 Signal 的 Force 赋值有 3 个类 (Kind): Freeze、Drive 和 Deposit。其中 Freeze 的赋值强度最强, Drive 次之, Deposit 最弱。ModelSim 是一个严格的 HDL 仿真器, 对于寄存器不指定默认值 (MAX+PLUS II 有默认值), 在信号的初始化时要对寄存器赋值, 这时不可赋予 Freeze, 因为如果那样的话, 寄存器的值就不能被其他信号驱动而改变, 一般要用 “Deposit” 赋值。同样 inout 等类型信号一般情况下也不能用 Freeze 赋值。

要在“wave”窗口中观察信号波形,应首先在“Signals”窗口中选择“Add”→“Wave”→“Signals in Region”菜单(如图 13.22 所示),即将设计中的端口信号添加到“wave”窗口中显示。如果程序中的节点信号的波形也需要观察的话,应选择“Add”→“Wave”→“Signals in Design”菜单。

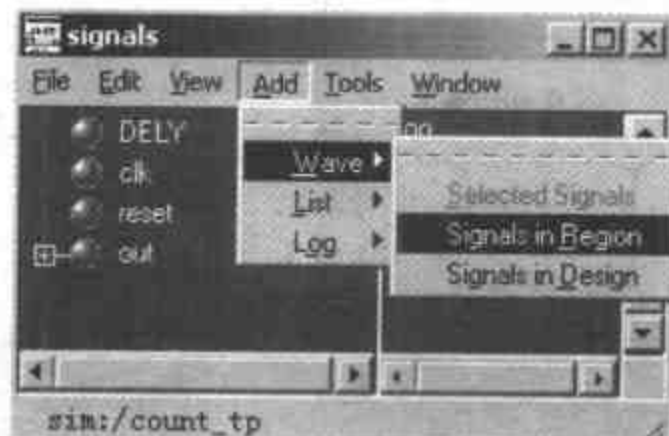


图 13.22 向“wave”窗口中添加信号

将信号添加到“wave”窗口后,在主窗口的命令窗口中键入 Run 或者单击  按钮,或者在“wave”窗口中单击  按钮都可以启动仿真,在主窗口中会显示文本输出,同时在“wave”窗口中输出仿真波形。图 13.23 所示是 ModelSim 对 count_tp.v 的功能仿真波形,在这里,计数器的输出没有延迟。

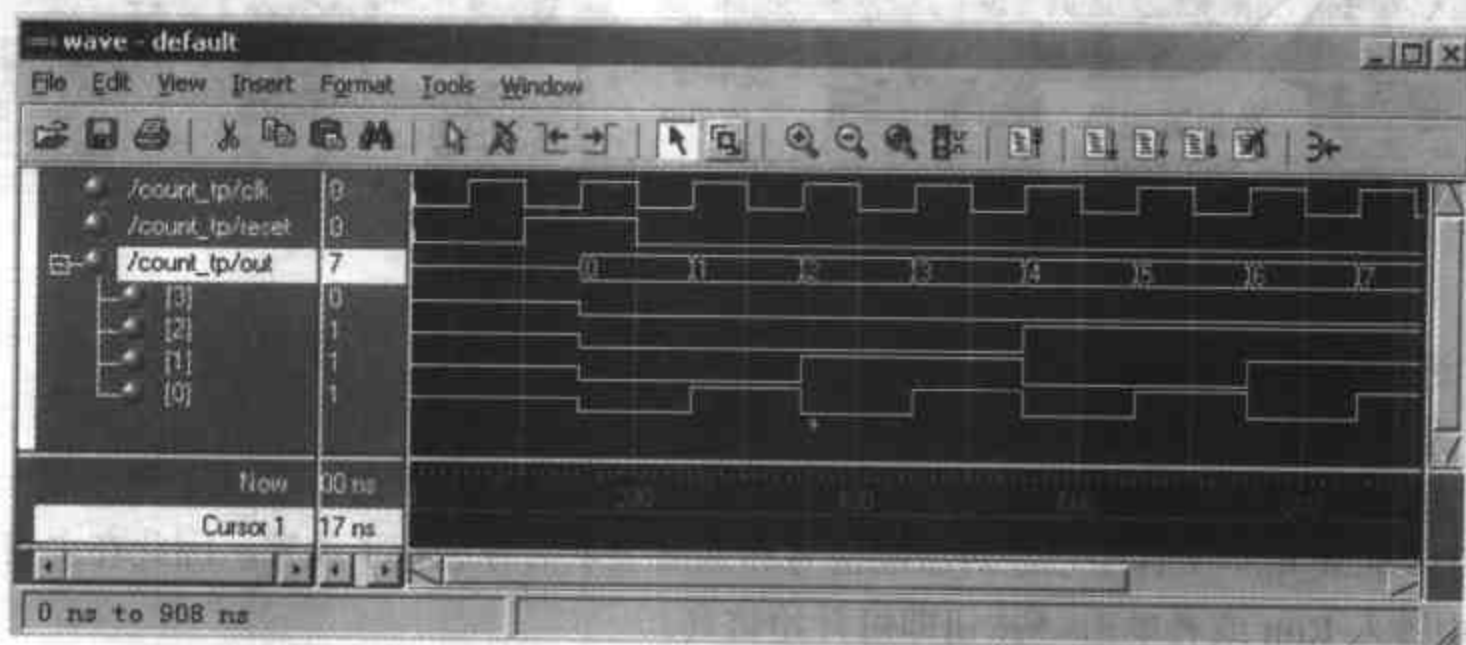


图 13.23 ModelSim 对 count_tp.v 的部分功能仿真波形

13.4.2 ModelSim 时序仿真

为了对设计程序进行门级时序仿真,必须用 MAX+PLUS II 对 count4.v 模块进行综合和适配,再把适配后的网表文件导入到 ModelSim 仿真环境中进行门级时序仿真。步骤如下:

1. MAX+PLUS II 编译前设置

MAX+PLUS II 可以输出用于仿真的 Verilog 或 VHDL 门级网表文件,输出类型可在设置中选择。由于 MAX+PLUS II 的菜单是动态的智能菜单,要找到这个设置,必须先在 MAX+PLUS II 中打开“Compiler”,然后在出现的“Interfaces”菜单项中选中“Verilog Netlist Writer”(如图 13.24 所示)。这时,Compiler 窗口界面中多了一项“Verilog Netlist Writer”模块,如图 13.25 所示。

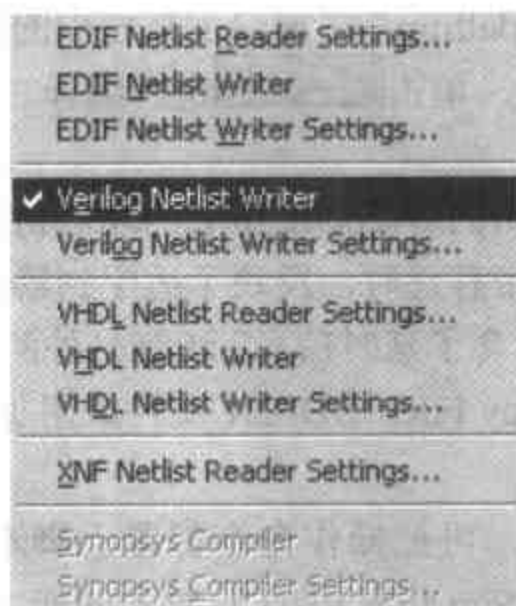


图 13.24 输出网表文件设置



图 13.25 Compiler 子窗口界面

MAX+PLUS II 支持多种 Verilog 网表格式，不同的 Verilog 网表格式，ModelSim 仿真处理有所不同，因此在 MAX+PLUS II 中还必须进行设置。在“Interfaces”菜单下选择“Verilog Netlist Writer Settings...”，将出现如图 13.26 所示的对话框。Verilog 网表延时信息的存放格式共有 3 种选择：包含在 Verilog 网表文件中，附加 SDF2.1（使用 VITAL 库）文件或附加 SDF1.0 文件。这里选择选项 1：Verilog Output File[.vo]。

2. 生成仿真文件

单击图 13.25 “Compiler”窗口中的“START”按钮，启动编译，编译结束后，MAX+PLUS II 会产生适配后的 Verilog 门级网表输出文件（*.vo）。

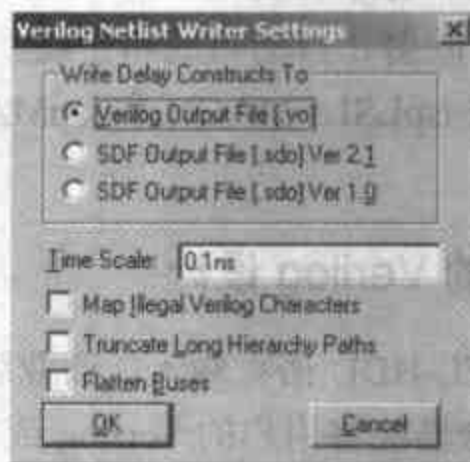
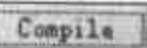


图 13.26 Verilog 网表格式设置对话框

3. 在 ModelSim 中作编译前设置

在 ModelSim 中新建一个项目（Project），在项目添加 MAX+PLUS II 输出的 Verilog 网

表文件 count4.vo, 然后单击“ModelSim”主窗口工具栏中的, 打开“Compiler HDL Source Files”对话框, 选择 count4.vo 后, 单击  进行编译。

4. 仿真

接下去的仿真步骤同前面介绍的对 count_tp.v 的功能仿真步骤一致, 在此不再赘述。需要指出的是, 最后的仿真波形中带有延时, 反映了适配到器件的延时情况。

前面提到的 count4.vo 已经包含了延时信息, 还有另外两种附加延时信息的仿真方式, 即附加标准延时文件 (Standard Delay File) SDF2.1 文件或附加 SDF1.0 文件。其操作步骤可参考相关的文档, 这里不再介绍。

ModelSim 带有 TCL Script, 可把操作命令都置于脚本中, 与第三方工具进行接口, MAX+PLUS II 不具备产生 ModelSim 仿真用 TCL 脚本的能力, 故不再介绍。

13.5 ispLEVER 使用指南

ispLEVER 是 Lattice 公司最新的集成开发工具, 能够完成包括设计输入、综合、仿真、适配和下载等整个流程, 设计输入可采用原理图、HDL 文本、混合输入三种方式, 能对所设计的数字系统进行功能仿真和时序仿真, 编译器能进行高效的逻辑优化, 并将逻辑映射到器件中去, 自动完成布局布线, 并生成编程所需要的熔丝图文件。软件支持原有 Lattice 公司的 GAL、ispLSI、MACH、ispGDX、ORCA2、ORCA3、ORCA4 和最新的 ispMACH、ispXPLD、ispXPGA、ispGDX2 等器件。

13.5.1 ispLEVER 的特点

ispLEVER 具有如下一些功能和特点。

(1) 支持多种输入方式, 包括:

- ◆ 原理图输入。
- ◆ HDL 文本输入: 支持 Verilog HDL、VHDL 和 ABEL-HDL 三种设计语言。
- ◆ EDIF 输入。

(2) 逻辑仿真: 包括功能仿真和时序仿真。

(3) 编译器: 包括结构综合、映射、自动布局和布线。

(4) 支持的器件: 支持所有的 ispLSI、MACH 和 ispMACH 器件, 含有支持 ispLSI 器件的宏库及 MACH 器件的 TTL 库。

13.5.2 基于 ispLEVER 的 Verilog 设计

ispLEVER 支持原理图和 ABEL-HDL 语言输入, 商业版的 ispLEVER 还支持 Verilog 和 VHDL 设计输入。ispLEVER 的综合器能将用户的 Verilog 或 VHDL 设计编译综合, 生成 EDIF 格式的网表文件, 然后进行仿真、适配, 生成可下载的 JEDEC 文件。

下面介绍 Verilog HDL 设计文件 (*.v) 输入、编译和下载整个操作过程, 其他语言的设计文本操作过程与此类似。

1. 设计输入

首先需要创建一个新的工程文件。在 ispLEVER Project Navigator 主窗口中, 按菜单“File”→“New Project”建立一个新的工程文件, 此时会弹出如图 13.27 所示的对话框选择设计类型, 在该对话框的 Project Type 栏中, 按照设计类型选择相应的工程文件的类型。本例中, 选择工程文件的类型为“Schematic/Verilog HDL”。若是 VHDL 设计输入, 则应选择 Schematic/VHDL 类型。

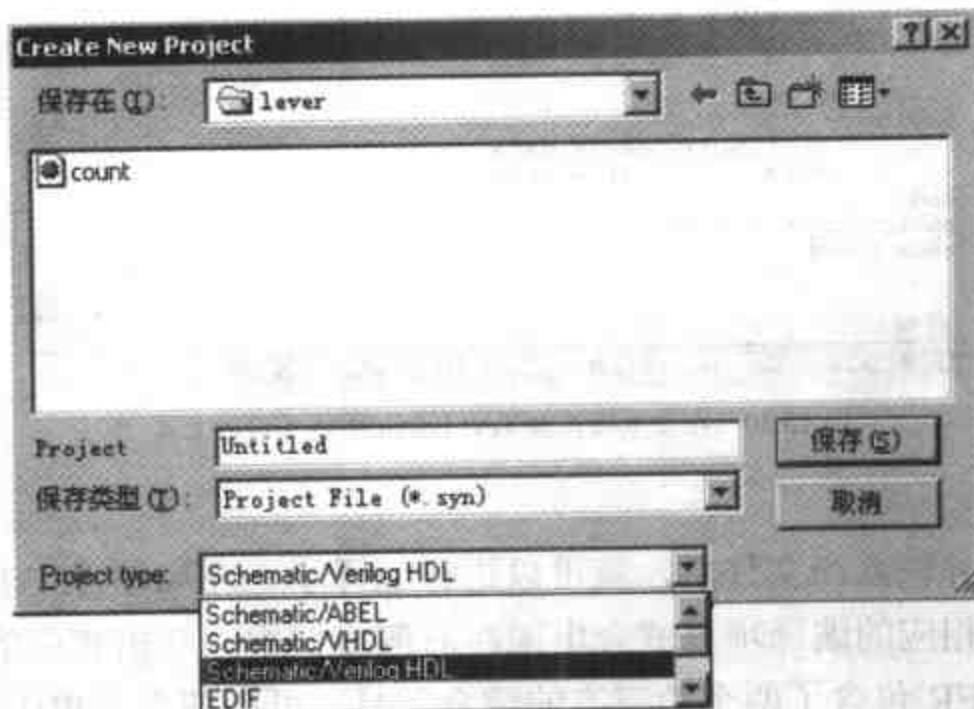


图 13.27 工程设计类型对话框

一个设计项目由一个或多个源文件组成。这些源文件可以是原理图文件 (*.sch)、Verilog HDL 设计文件 (*.v)、ABEL-HDL 文件 (*.abl)、VHDL 设计文件 (*.vhd)、仿真波形文件 (*.wdf)、测试向量文件 (*.abv) 或者是文字文件 (*.doc、*.wri、*.txt) 等。

下面将设计源文件添加到当前工程项目中, 在 ispLEVER Project Navigator 主窗口中选择“Source”→“New”菜单, 在弹出的如图 13.28 所示的“New Source”对话框中, 选择“Verilog Module”类型, 按“OK”后, 填写 Module Name (模块名)、File Name (文件名) 等信息, 在这里以设计一个 8 位计数器为例, 将 Module Name、File Name 均填写为 count8, 如图 13.29 所示, 按“OK”钮后, 进入文本编辑器 Text Editor, 就可以开始输入和编辑 Verilog 文本了。

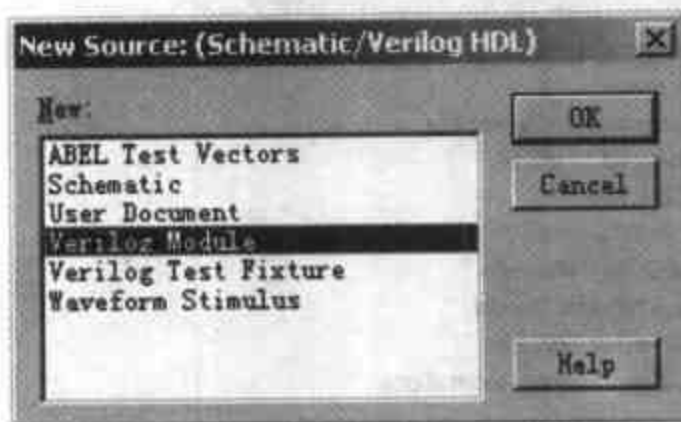


图 13.28 New Source 对话框

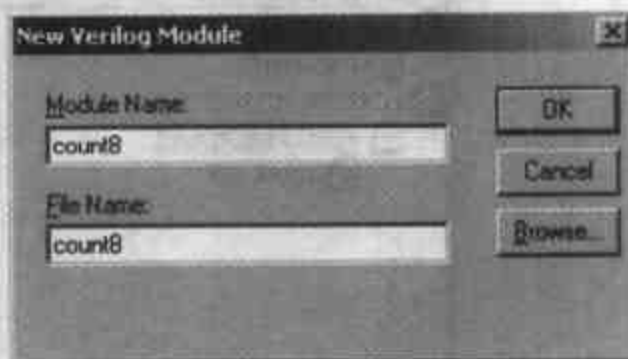


图 13.29 填写 Module Name 和 File Name 对话框

在本例中, 输入的 8 位计数器的 Verilog 设计文本如图 13.30 所示, Verilog 关键字皆显示为蓝色, 以与其他文本相区别。输入完毕后, 应保存, 此时, 在 ispLEVER Project Navigator 主窗口左侧的源程序区中, *.v 文件被自动调入。如果是将已经存在的 Verilog

源文件或其他源文件添加到当前工程中，应该选择“Source” → “Import”菜单，并执行相应的操作。

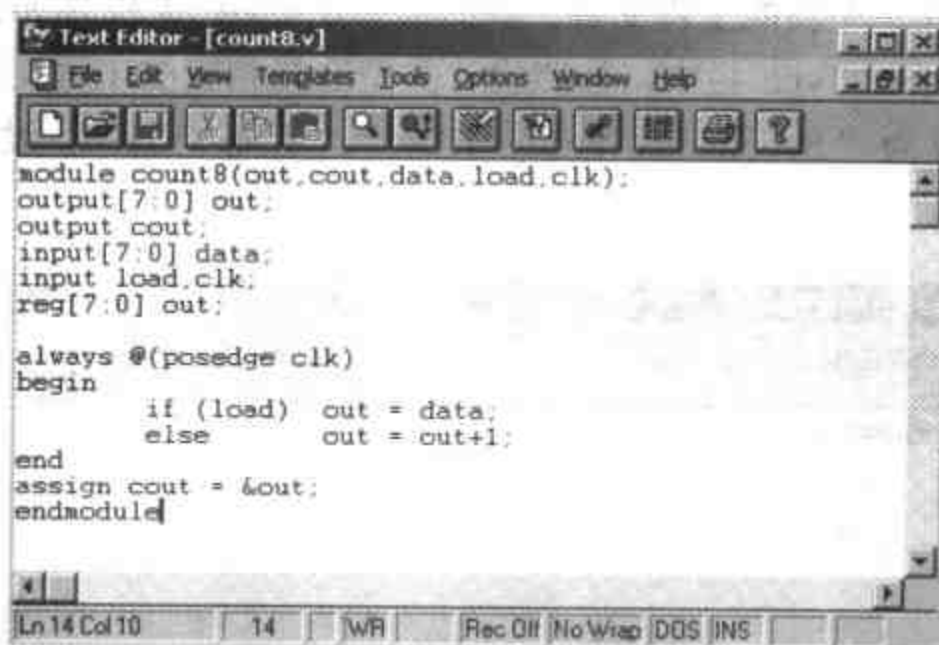


图 13.30 在文本编辑器 Text Editor 中输入设计文本

2. 综合

创建好工程文件并输入源文件后，就可以进行编译了。在如图 13.31 所示的主界面中单击源文件 count8.v，相应的编译项目就会出现在右侧窗口中，双击相应的项目软件自动执行相应的操作。ispLEVER 包含了两个第三方的综合工具，可以在主菜单中选择“Options” → “Select RTL synthesis”菜单，弹出如图 13.32 所示的对话框，从中选择 Synplify 或 Leonardo Spectrum。

执行编译后，若源文件 count8.v 没有语法错误和逻辑错误，则右窗口中相应的栏目均会顺利通过，并在相应的项目前用“√”表示（如图 13.31 所示）。如果代码中有错，软件会给出错误提示，双击状态窗口中的错误提示，在文本编辑器 Text Editor 中，光标会自动移到相应位置并高亮显示，便于查找错误。

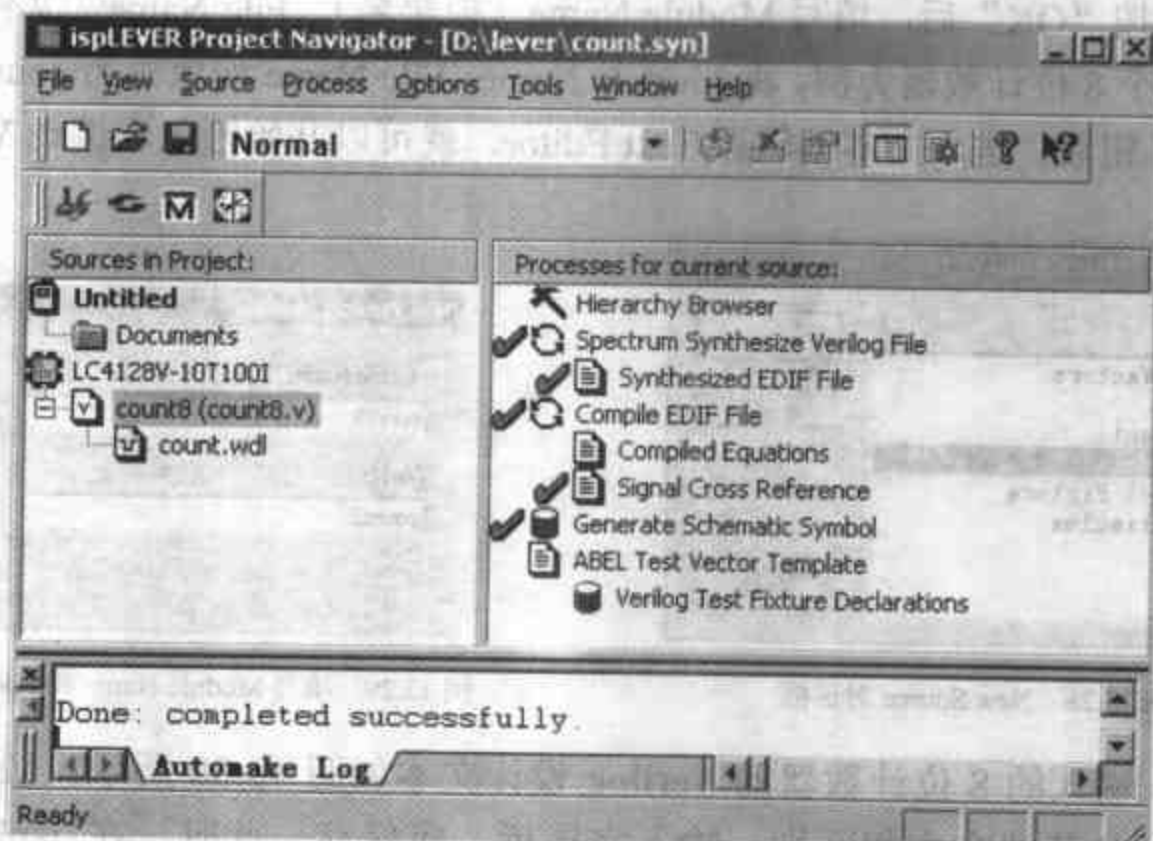


图 13.31 在主界面中对 count8.v 源文件进行综合



图 13.32 选择综合器对话框

综合通过后，可以继续添加源文件（Verilog 文本或原理图等），重复上述操作。源文件之间若有调用关系（Hierarchical Design），软件会自动理清层次。本例中由于只有一个源文件，因此编译过程已经结束了。

3. 仿真

仿真可以采用两种方式：文本仿真方式或者用波形仿真（Waveform Stimulus）。

使用文本仿真方式时，需编写测试程序，可以在文本编辑器中编辑*.abv 测试文件，完成后，存盘退出，然后双击 Functional Simulation，即开始功能仿真。为查看仿真波形，可进入 Simulation Control Panel 窗口，选择菜单“Windows”→“Waveform Viewer”，打开波形观测器查看仿真结果。此外，为了看波形，必须在 Simulation Control Panel 窗口中按 Debug 钮，使 Simulation Control Panel 窗口进入 Debug 模式。

在 Available Signals 栏中选择信号，并且按 Monitor 钮，这些信号名便可以在波形观测器中观察到，再按 Run 钮进行仿真。

如双击 Timing Simulation，即可进入时序仿真流程。

4. 适配

以上完成了源文件的输入、综合及仿真，剩下的步骤是将设计放入合适的 Lattice 器件中，即适配。

首先选择器件，选择菜单“Source”→“Select New Device”，出现如图 13.33 所示的对话框，从中选出合适的芯片，这里选择 ispMACH 4000 器件系列中的 LC4128V 芯片，速度等级选择 10，封装选择 100TQFP。

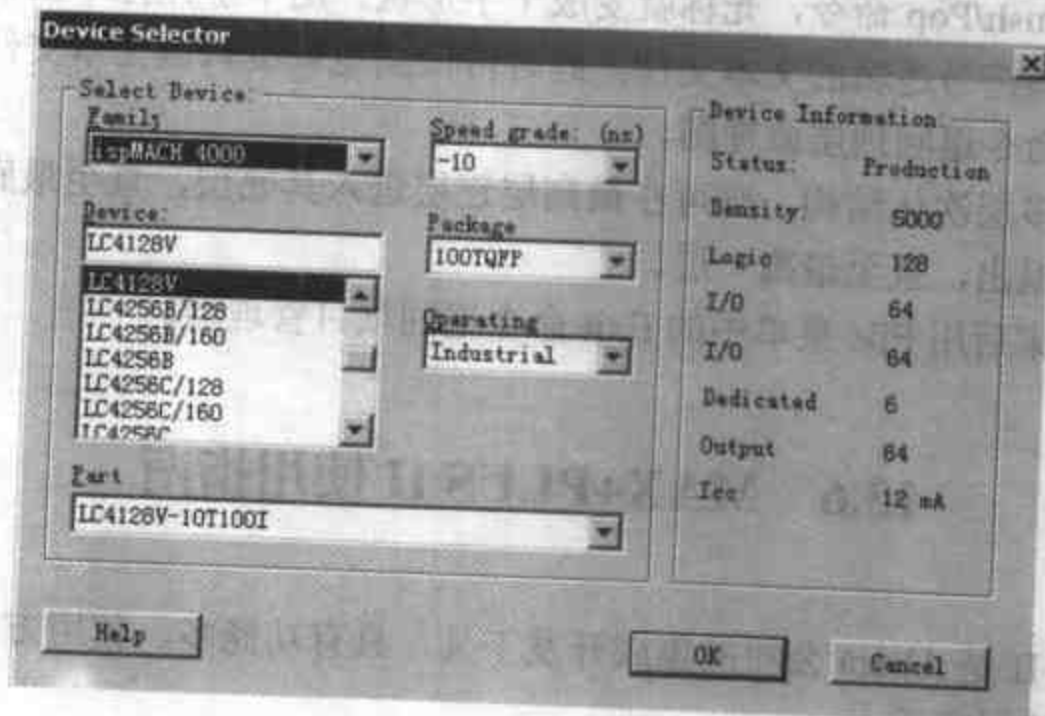


图 13.33 选择器件对话框

在如图 13.34 所示的主窗口中点击器件作为编译对象，在右侧的窗口中即出现对应的处理操作。双击“Fit Design”便启动了适配过程，软件将对项目中的所有源文件进行编译，然后连接所有的源文件，最后进行逻辑分割、布局和布线，将设计适配到所选择的 Lattice 器件中。若设计无误，则所有的相应的项目前均出现“√”，并最终生成 JEDEC 可下载文件。

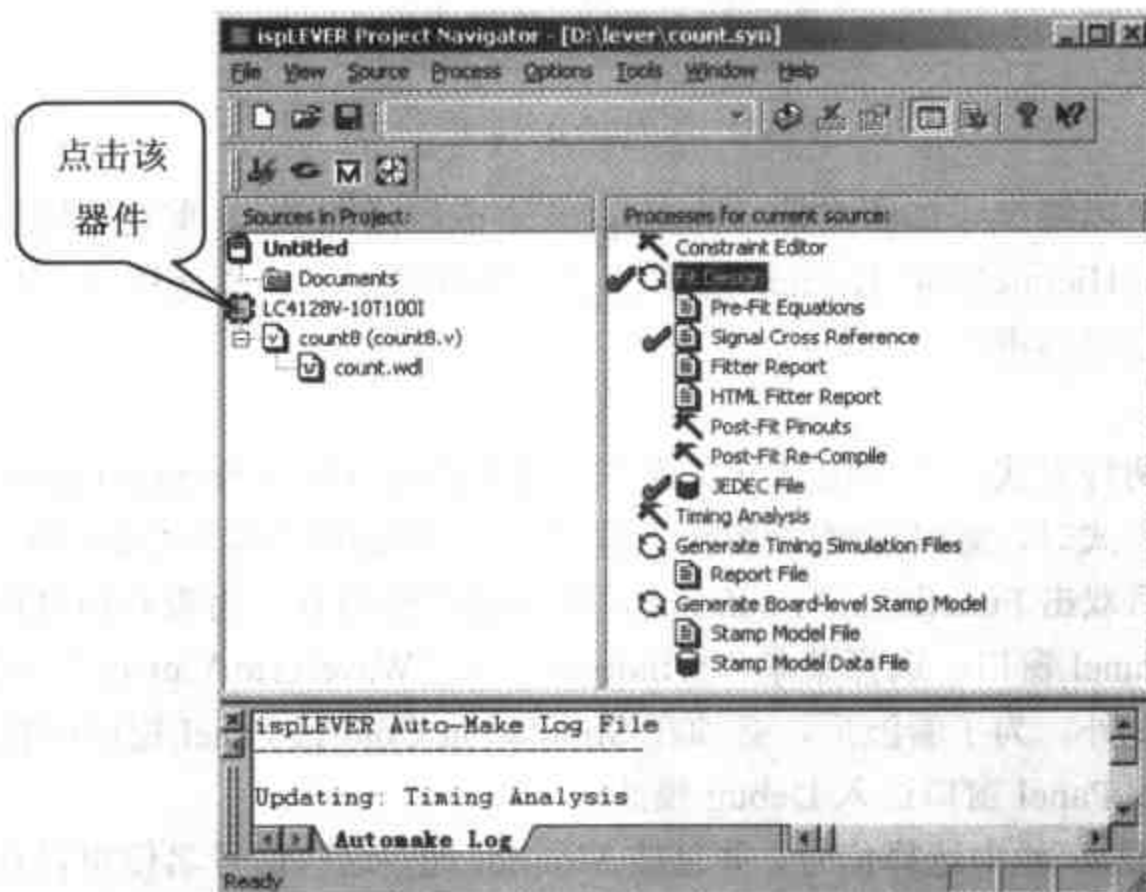


图 13.34 适配

当这些都完成后，可双击 Fitter Report 查看一下设计报告和有关统计数据。

5. 层次化操作方法

ispLEVER 还提供了层次化操作，以支持多层次的设计。

在项目管理器的源文件窗口中选择最顶层设计，此时在项目管理器右边的操作流程清单中必定有 Navigate Hierarchy 过程。

双击 Navigate Hierarchy 过程，即会弹出最顶层设计文件（一般多采用原理图方式）。选择 View 菜单中的 Push/Pop 命令，光标就变成十字形状。用十字光标单击顶层原理图中的符号，即可弹出描述该符号逻辑的文本文件。此时可以浏览或编辑该文本文件，浏览完毕后用 File 菜单中的 Exit 命令退回顶层原理图。

若某一设计为多层次化结构，则可在最高层逐层进入其底层，直至最底层；退出时亦可以从最底层逐层退出，直至最高一层。

层次化操作结束后用 File 菜单中的 Exit 命令退回项目管理器。

13.6 MAX+PLUS II 使用指南

MAX+PLUS II 是 Altera 公司的集成开发工具，具有功能多、使用方便等优点，是大众化的 EDA 平台，使用广泛。

MAX+PLUS II 支持 AHDL、Verilog HDL 及 VHDL 三种硬件描述语言。本节主要介绍 Verilog 程序的设计使用, 其他语言书写的程序的处理过程与此基本相同。

基于 MAX+PLUS II 软件的 Verilog 设计步骤大致如下所示。

- ◆ 建立设计项目, 创建设计文件;
- ◆ 输入设计文本并保存;
- ◆ 对 HDL 文件进行编译, 检查句法错误并修改;
- ◆ 进行功能仿真, 如有错误, 则修改源文件;
- ◆ 对设计指定器件并适配;
- ◆ 进行时序仿真, 如有错误, 修改源文件;
- ◆ 若以上步骤全通过的话, 可进行下载或其他操作。

下面通过 4 位加法器的设计来具体说明基于 MAX+PLUS II 软件的 Verilog 设计的步骤。

1. 设计输入

选择“File”→“New”, 在对话框中选择 Text Editor File, 打开一个 Text Editor 窗口, 输入 4 位加法器的 Verilog 设计文本, 如图 13.35 所示。选择“File”→“Save As”保存文件为“adder4.v”, 然后选择菜单“File”→“Project”→“Set Project to Current File”将该文件设为当前项目。

通过上述操作完成了 HDL 文本的输入, 以下是对设计文本的处理过程。

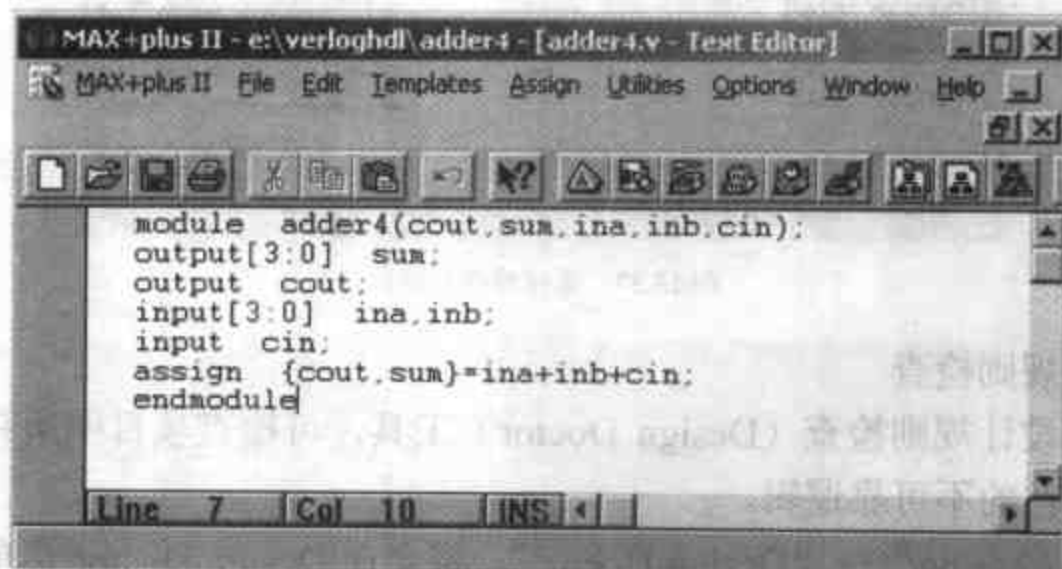


图 13.35 设计文本输入

2. 编译设定

HDL 文件输入后, 即可启动编译程序来编译项目。编译器将进行错误检查、网表提取、逻辑综合、器件适配、编程文件产生等操作。在编译前应完成以下操作:

- ◆ 为设计指定器件, 并锁定引脚;
- ◆ 选择设计规则检查;
- ◆ 设定全局逻辑综合有关选项;
- ◆ 设置全局定时要求;
- ◆ 打开功能仿真器或定时模拟器、网表文件提取器;
- ◆ 指定在.rpt 文件中报告的内容。

需注意的是, 在设置以下一些编译项目时, 应先进入编译界面, 选择菜单命令“MAX+PLUS II”→“Compiler”, 打开编译器窗口(如图 13.36 所示), 即出现了与编译设置

有关的菜单, 可开始进行如下一些设置和操作。



图 13.36 打开编译器窗口

(1) 指定器件

在编译项目前, 需要为设计项目指定器件, 否则编译器会自动选择一个器件。

选择菜单“Assign”→“Device”, 出现如图 13.37 所示的 Device 对话框。从“Device Family”下拉列表框中选择 MAX7000S 系列, 在 Devices 列表框中选择“EPM7064SLC44-5”, 最后点击“OK”(也可选其他器件)。

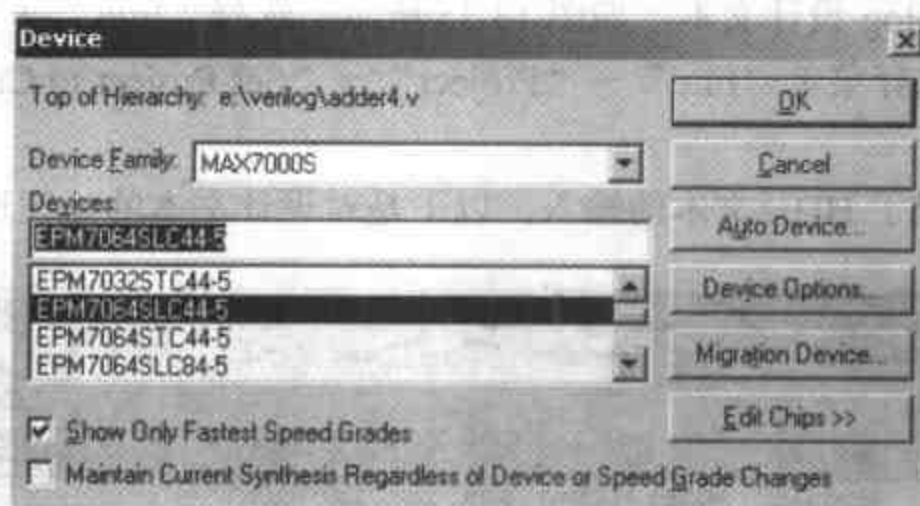


图 13.37 选择器件对话框

(2) 选择设计规则检查

编译时, 使用设计规则检查 (Design Doctor) 工具, 可检查项目中所有的设计文件, 以发现器件中可能存在的不可靠逻辑。

选择菜单“Processing”→“Design Doctor”, 将会在 Design Doctor 菜单项的左侧出现一个确认标记, 并且将会在 Compiler 窗口中 Logic Synthesizer 模块的下方显示  图标, 不选择“Design Doctor”时, 该图标不出现。

选择菜单“Processing”→“Design Doctor Settings”, 出现如图 13.38 所示的对话框。根据情况, 选择合适的设计规则。



图 13.38 设计规则选项

(3) 引脚锁定

只有在为设计项目选定器件以后,才可锁定顶层设计中各端口所对应的引脚号。锁定引脚既可以通过 Assign 菜单项或在项目顶层设计图中锁定引脚,也可在平面布局编辑器(或称底层编辑器)上通过编辑适配结果来修改锁定。一般可让编译器自动进行引脚分配,除非在输入、输出引脚已经由外围电路确定的情况下,一般可不锁定引脚。

锁定引脚选择菜单“Assign”→“Pin”→“Locate”→“Chip”,在弹出的对话框中进行引脚的分配和锁定。

(4) 设定全局逻辑综合选项

综合前应设定全局逻辑综合选项(Global Project Logic Synthesis),以便在编译的过程中指导编译器的逻辑综合过程。

选择菜单“Assign”→“Global Project Logic Synthesis”,出现如图 13.39 所示的对话框。

在“Global Project Logic Synthesis”下拉列表框中可以选择 NORMAL、FAST(快速)、WYSIWYG(所见即所得)等全局综合方式。默认的逻辑综合方式是 NORMAL(常规),该方式的逻辑综合目标是使逻辑单元耗用最少。

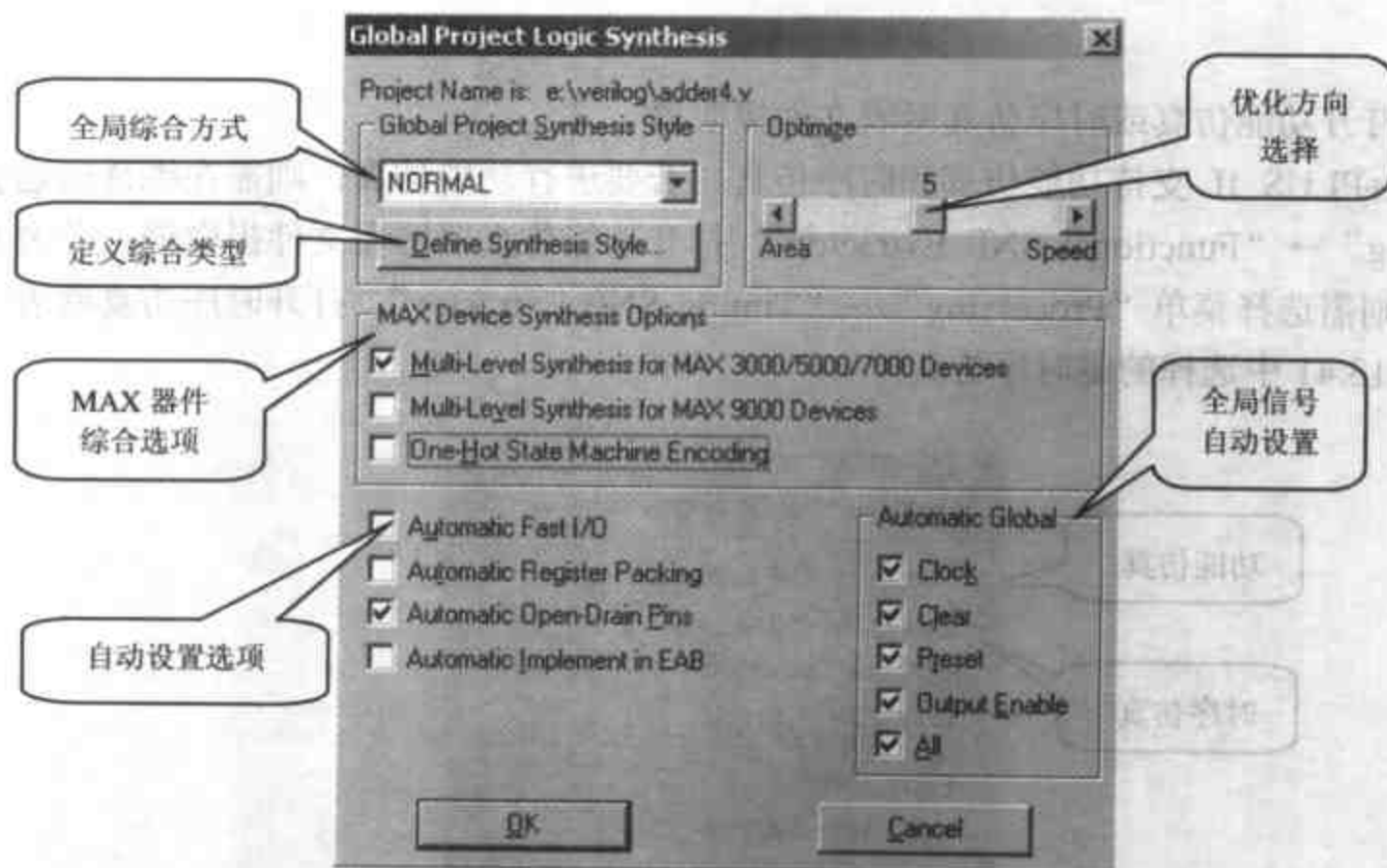


图 13.39 设置全局综合选项对话框

移动 Optimize(优化)栏中的滑动块可确定优化方向:若移动至 0,则逻辑综合时首先考虑减少器件的资源占用;若移动至 10,则优先考虑系统的执行速度;若在两者之间的话,则在资源和速度上折衷考虑。

对于 MAX 器件,设计人员可以选择多级逻辑综合方式选项。该方式可以充分利用所有可使用的逻辑单元,适用于处理含有复杂逻辑的设计项目,且不需要用户干预。对于 FLEX 系列器件,该选项自动有效。

当选择了 FAST 综合方式后,综合器会自动使用 FLEX/ACEX 等器件系列中的进位链、级连链等结构。

(5) 设置全局定时要求

可对整个项目设定全局定时要求,如传播时延、时钟到输出的延时建立时间和时钟频率等。全局定时要求仅对 FLEX 6000、FLEX 8000 和 FLEX 10K 等器件有效。设置定时要求的方法如下。

选择菜单“Assign”→“Global Time Requirements”,出现如图 13.40 所示的对话框。在相应的编辑框内输入所要求的定时时间,若将双向引脚反馈通路和清除/预置路径选项设置为关断,则不计这些延时。

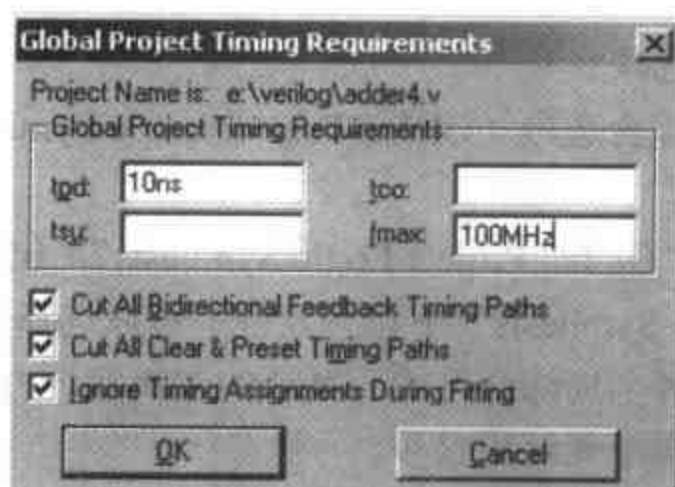


图 13.40 全局定时设置对话框

(6) 打开功能仿真或时序仿真网表文件提取器

MAX+PLUS II 支持功能仿真和时序仿真,若要进行功能仿真,则需在编译时选择菜单“Processing”→“Functional SNF Extractor”,打开功能仿真器网表文件提取器;若要进行时序仿真,则需选择菜单“Processing”→“Timing SNF Extractor”,打开时序仿真网表文件提取器。图 13.41 中选择的是时序仿真。

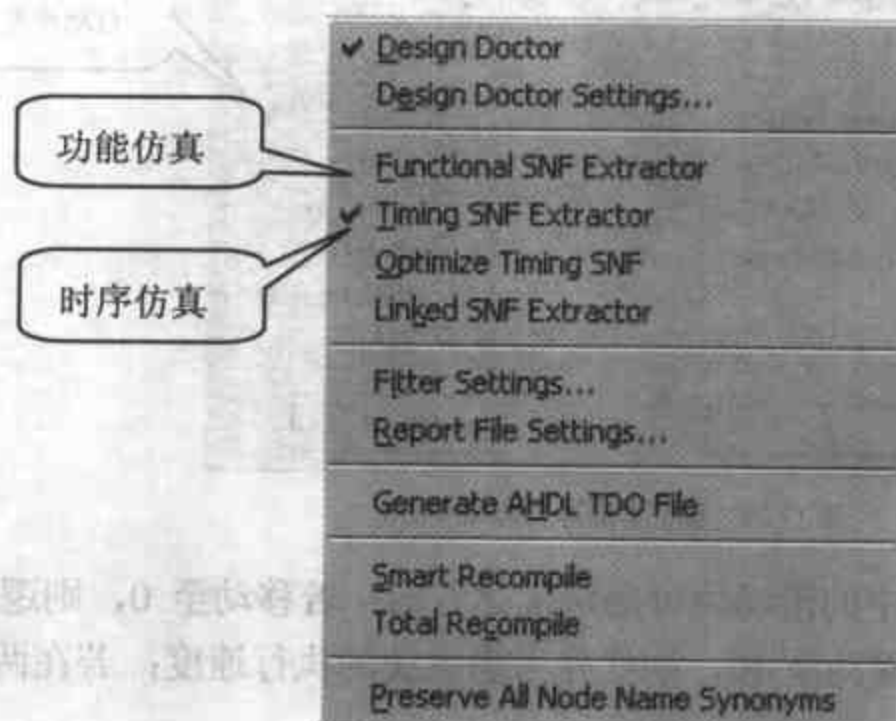


图 13.41 选择功能仿真或时序仿真

编译器能创建一个模拟器网表文件(.snf),该文件包含了许多逻辑信息和时间信息,并被用作 MAX+PLUS II 模拟器或定时分析器的输入。定时模拟器网表文件是一个二进制文件,包含了模拟、延时预估和定时分析所需的逻辑和时间信息。

(7) 指定报告文件中报告的内容

编译器适配(Fitter)模块产生报告文件(.rpt),它报告了器件内部资源的耗用情

况。编译器允许指定报告文件中所包含的内容。为指明报告文件的内容,可进行下述操作。

选择菜单“Processing”→“Report File”,出现如图 13.42 所示的对话框,设置需要产生的报告文件内容项。

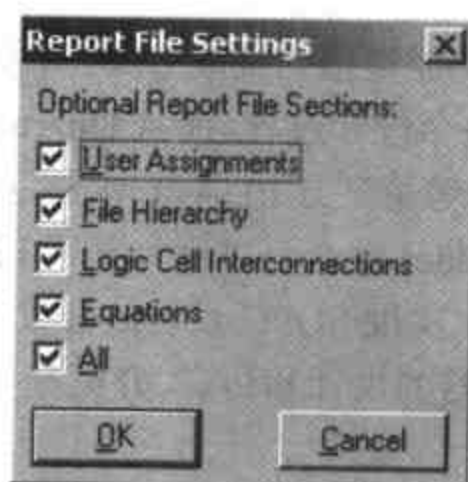


图 13.42 报告文件选项

在默认情况下,所有选项都被选中。如果某些项未被选中,打开 All 选项即可实现全选。

对于每个设计项目,都有一个配置文件(.acf),所有的配置参数都存在该文件中。主要的参数包括器件的类型、引脚的设置、速率及面积的比重、时间参数的要求、布线时的设置等。设置越多,则对布局、布线的限制越多,因此布通的概率就越低。所以,为保证布通,配置上应只对较重要的设置进行指定,以此来提高布通率。

3. 编译

设置完各种配置选项后,即可以开始编译设计项目。

在图 13.36 所示的编译器窗口中点击“Start”按钮即启动编译。在编译过程中,所有信息、错误和告警将会在自动打开的信息处理窗口中显示出来。如果有错误产生,选中该错误信息,然后按下 Locate 按钮可自动定位错误处。编译是在后台进行的,此时,计算机可从事其他工作。

编译结束后,那些由编译器产生的代表输出文件的图标将会出现在各模块框的下面,可通过双击适当的文件图标来打开这些文件察看。如有必要,可以选择“MAX+PLUS II”→“Message Processor”转到消息处理窗口,也可直接点击消息框中的一条消息或通过点击“Message”按钮左侧和右侧的标记来选择消息。点击“Locate”按钮或双击该条消息,消息处理器自动打开包含该条消息来源的设计文件,并高亮显示设计文件中产生该消息的位置。

编译完成后会产生如下一些重要的文件:

- ① 适配报告:包括芯片内部资源的利用情况、设计的布尔方程描述情况等。
- ② 面向其他 EDA 工具的输出文件,如 EDIF 文件等。
- ③ 延时信息:以便于进行精确的时序仿真,因为已经得到器件的实际硬件特性(如时延特性),所以仿真结果能比较精确地预测实际芯片的工作性能。如果仿真结果达不到设计要求,就需要修改源代码或选择不同速度的器件,直至完全满足设计要求为止。
- ④ 器件编程文件:如用于 CPLD 编程的.pof 文件、用于 FPGA 配置的.sof 文件等。

有关仿真、下载等方面的操作,限于篇幅,这里不作介绍,读者可参考相关资料。

13.7 Quartus II 使用指南

Quartus II 是 Altera 的新一代功能更强的集成 EDA 开发软件,使用 Quartus II 可完成从设计输入、综合适配、仿真到下载的整个设计过程,而且 Quartus II 也可以直接调用 Synplify Pro、Leonardo Spectrum 以及 ModelSim 等第三方 EDA 工具来完成设计任务的综合和仿真。此外,Quartus II 与 MATLAB 和 DSP Builder 结合可以进行基于 FPGA 的 DSP 系统开发,方便且快捷,还可以与 SOPC Builder 结合,实现 SOPC 系统的开发。

本节通过一个模 15 减法计数器的设计和仿真,介绍基于 Quartus II 软件的基本设计流程。

13.7.1 创建工程文件

在进行设计之前,首先应该建立工作目录,每个设计都是一项工程 (Project),一般都单独建一个工作目录,在此设立的工作目录为 E:\Verilog\count。

启动 Quartus II,出现如图 13.43 所示的界面,界面分为几个区域,分别是工作区、设计项目层次显示区、信息提示窗口、各种工具按钮栏等,也可以根据自己的喜好调整该界面。

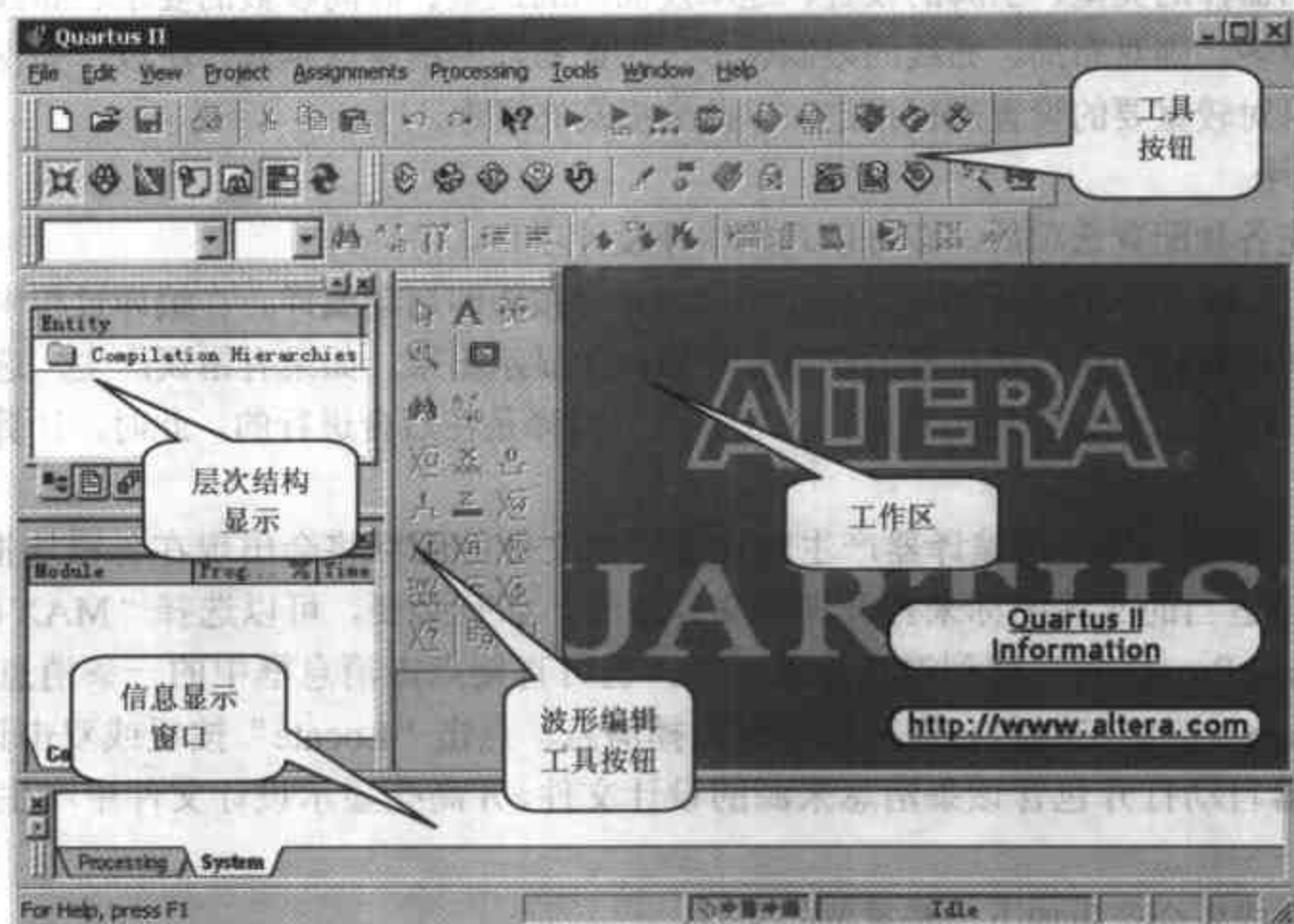


图 13.43 Quartus II 软件的界面

1. 输入源程序

选择菜单“File”→“New”,在弹出的“New”对话框的“Device Design Files”页面中选择源文件的类型,这里选择“Verilog HDL File”类型,如图 13.44 所示,然后就可以在文本编辑窗口中输入如下所示的 Verilog 源程序。

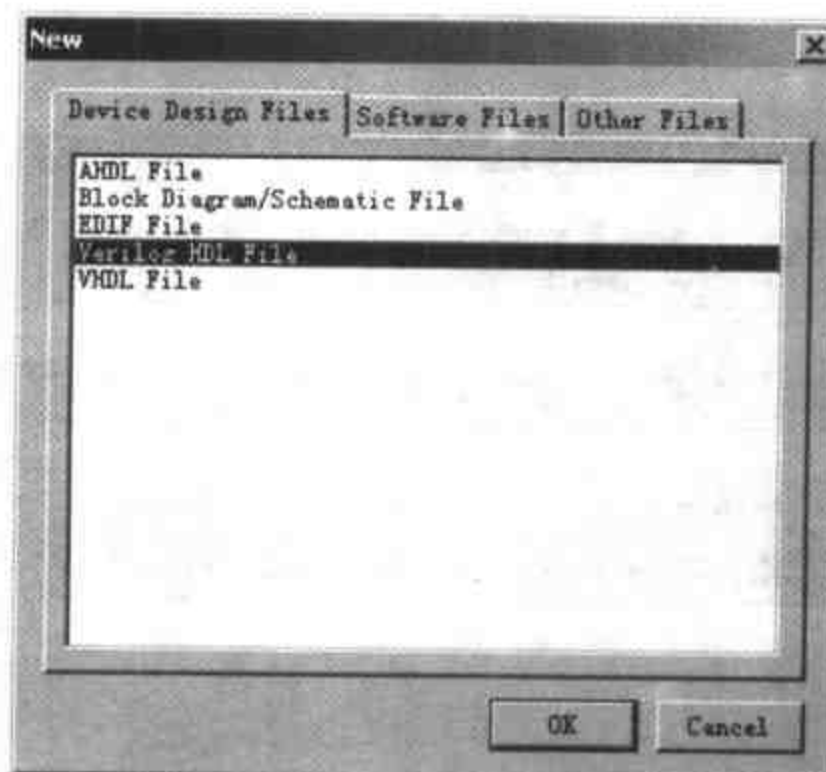


图 13.44 选择源文件类型对话框

```
module count15(out,cout,reset,clk);
```

```
output[3:0] out;
```

```
reg[3:0] out;
```

```
output cout;
```

```
input clk,reset;
```

```
always @(posedge clk)
```

```
begin
```

```
if (reset) out<=0;
```

```
else
```

```
begin
```

```
if(out==0) out<=14;
```

```
else out<=out-1;
```

```
end
```

```
end
```

```
assign cout=((out==4'd14)?1:0;
```

```
endmodule
```

完成输入后,选择“File”→“Save As”,将源文件存于已创建的文件夹 E:\Verilog\count 中,文件名应与模块名一致。

2. 创建工程

这里利用“New Project Wizard”建立设计工程,在此过程中要设定有关的内容,如工程名、目标器件、选用的综合器和仿真器等,其过程如下。

(1) 启动“New Project Wizard”

选择菜单“File”→“New Project Wizard”,弹出如图 13.45 所示的对话框。点击该框最上一栏右侧的按钮“...”,找到文件夹 E:\Verilog\count,作为当前的工作目录。第二栏的 count15 是当前工程的名字,一般将顶层文件的名字作为工程名。第三栏是顶层文件的实体名,此处

也命名为 count15。

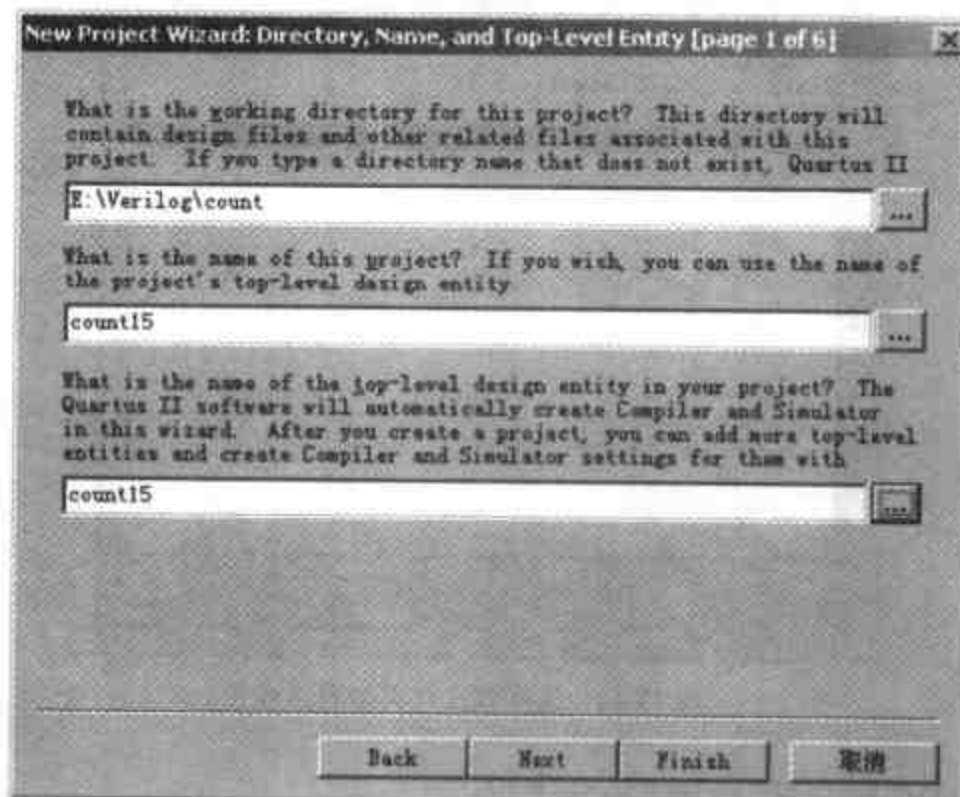


图 13.45 使用“New Project Wizard”创建工程

(2) 将设计文件加入工程中

点击图 13.45 中的“Next”按钮，弹出“Add Files”对话框，如图 13.46 所示，点击“Add All”，将所有有关的文件都加入到当前工程中。在本工程中，只有一个源文件 count15.v，因此只将该文件加入到当前工程中即可。

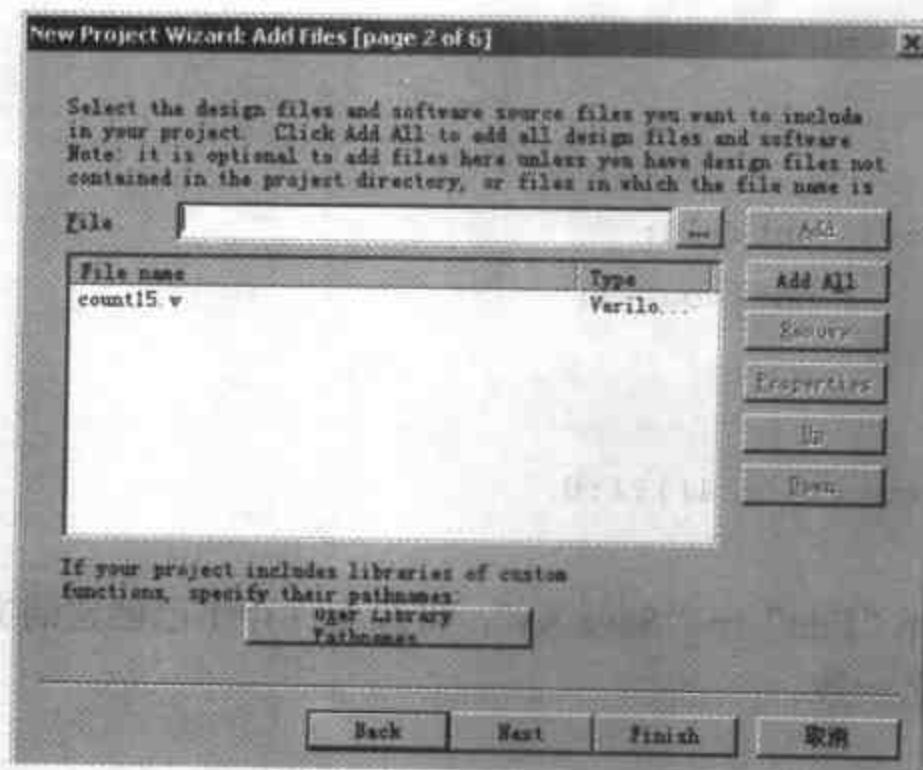


图 13.46 将设计文件加入到当前工程中

(3) 选择综合器和仿真器

点击图 13.46 中的“Next”按钮，弹出选择仿真器和综合器的对话框，如图 13.47 所示，如果选择默认的“None”，则表示选择 Quartus II 自带的综合器和仿真器。也可以选择其他第三方综合器和仿真器等专业 EDA 工具。

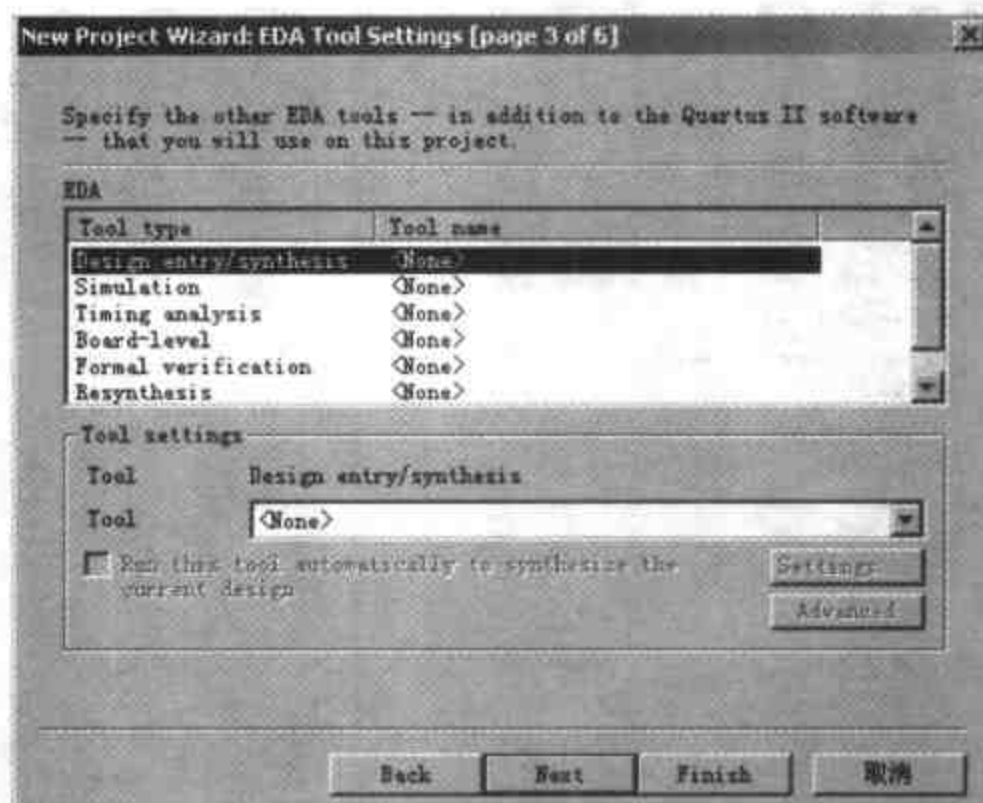


图 13.47 选择仿真器和综合器

(4) 选择目标器件

再次点击“Next”按钮，出现选择目标器件的窗口，此处选“ACEX1K”器件系列，在此栏下选“Yes”，表示要选择一个确定的目标器件。

点击“Next”按钮，出现如图 13.48 所示的选择具体器件的窗口，这里选择器件“EP1K10TC100-3”。

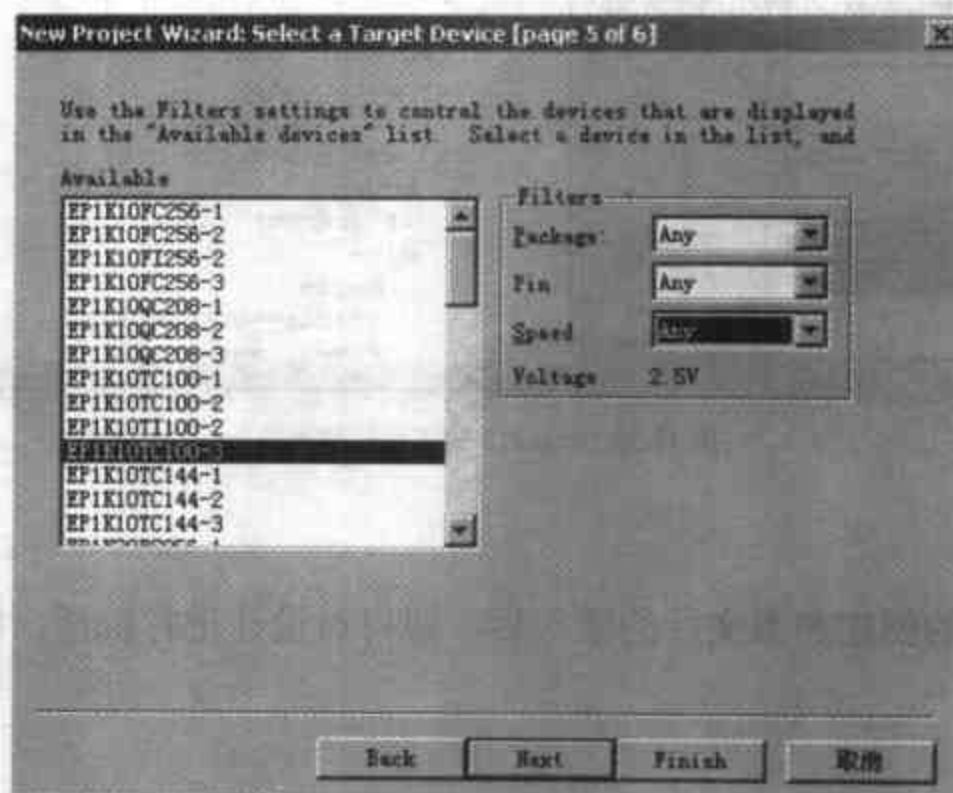


图 13.48 选择具体目标器件

(5) 结束设置

点击“Next”按钮，出现如图 13.49 所示的工程设置信息显示窗口，对前面所作的设置作了汇总。点击图中的“Finish”按钮，即完成了当前工程的创建。在工程管理窗口中出现当前工程的层次结构显示，如图 13.50 所示。

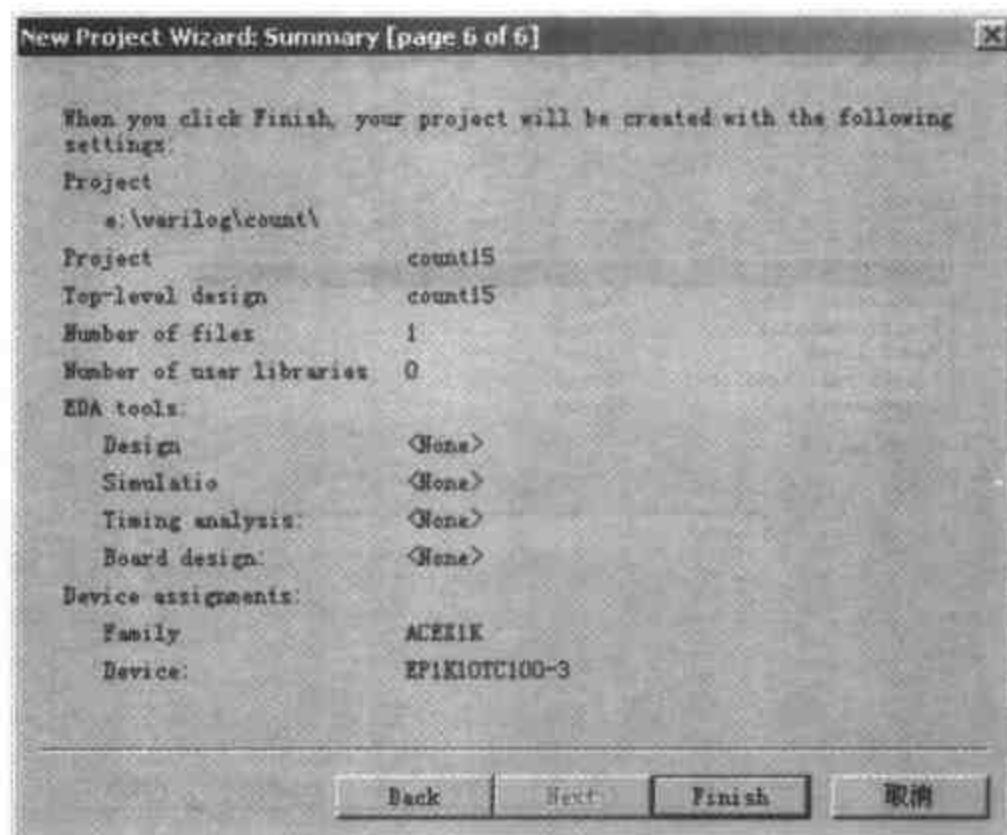


图 13.49 工程设置信息显示

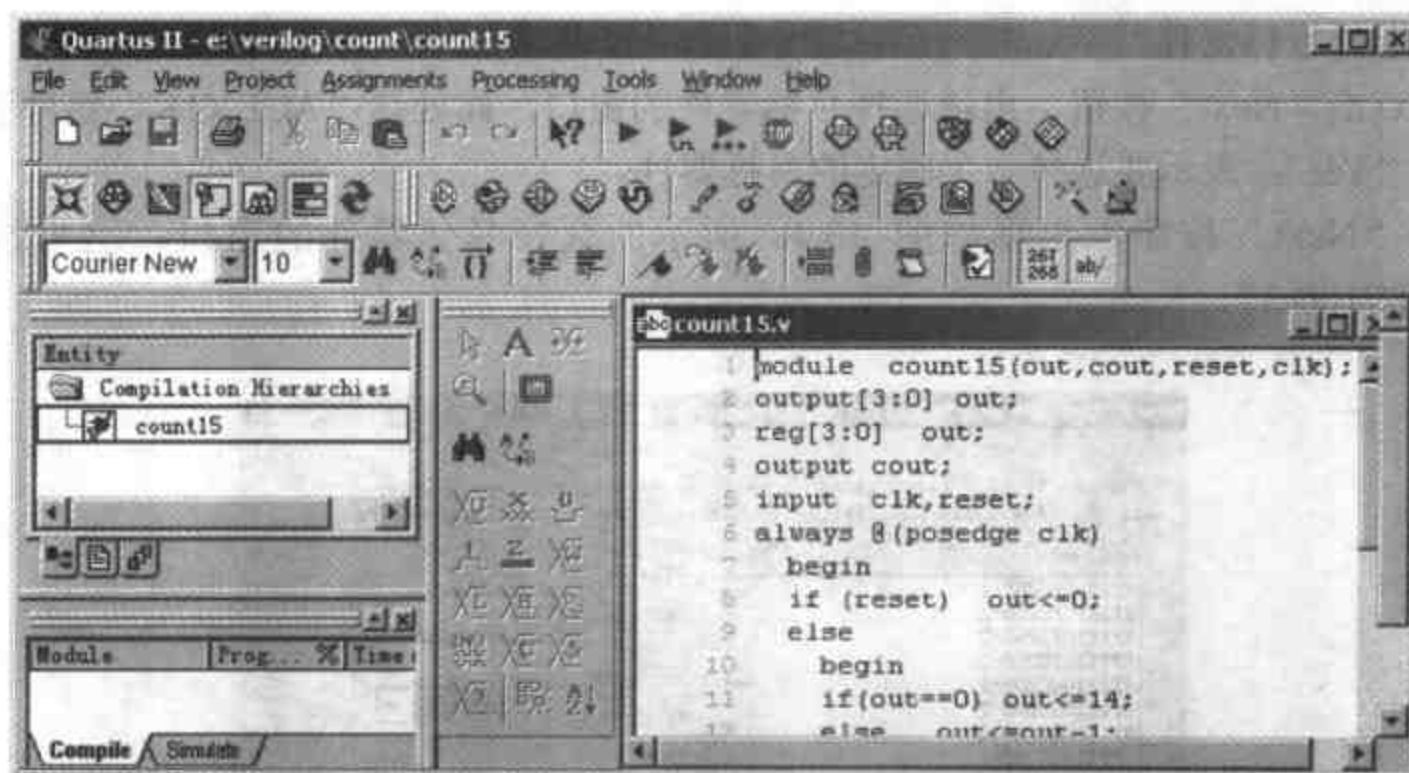


图 13.50 count15 的工程管理窗口

13.7.2 编译

完成了工程文件的创建和源文件的输入后,即可对设计进行编译,编译前,必须作好必要的设置。

1. 编译前设置

首先可以选择目标器件,在前面利用“New Project Wizard”建立设计工程时已经选定了目标器件,如果前面没有选定的话,可选择“Assignments”→“Settings”项,在弹出的对话框中选“Compiler Settings”项下的“Device”来选择器件。选择好器件后,再点击“Device & Pin Options”按钮,出现“Device & Pin Options”选择窗,从中选择“Configuration”页面,选择器件的配置方式,这里选择 Passive Serial (被动串行)方式,即由 PC 或 EPC 配置器件来对目标器件进行配置,配置器件选 EPC1,以上设置如图 13.51

所示。

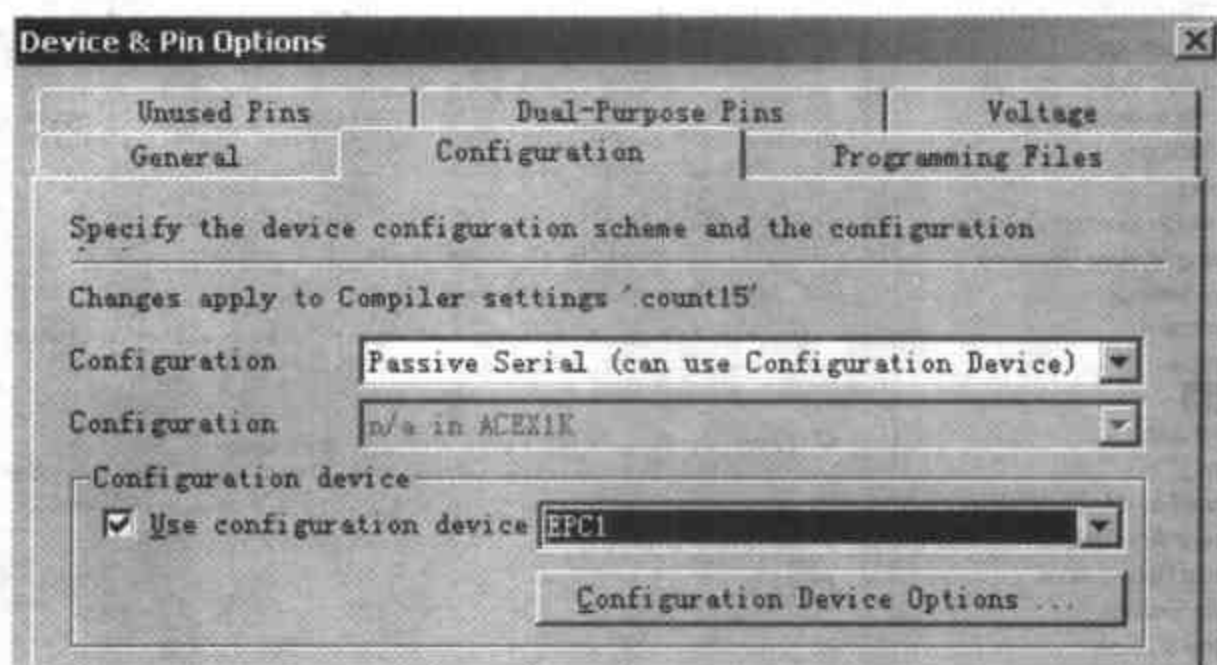


图 13.51 选择配置方式和配置器件

在图 13.51 中点击“Programming Files”，出现如图 13.52 所示的选择配置文件格式的页面，可用于器件配置编程的文件格式有 .ttf、.rbf、.jam、.jbc 和 .hexout 等，设计者可根据自己的需要选择其中的一种或几种文件格式。这样，编译器在编译后会自动生成该格式的配置文件供用户使用。

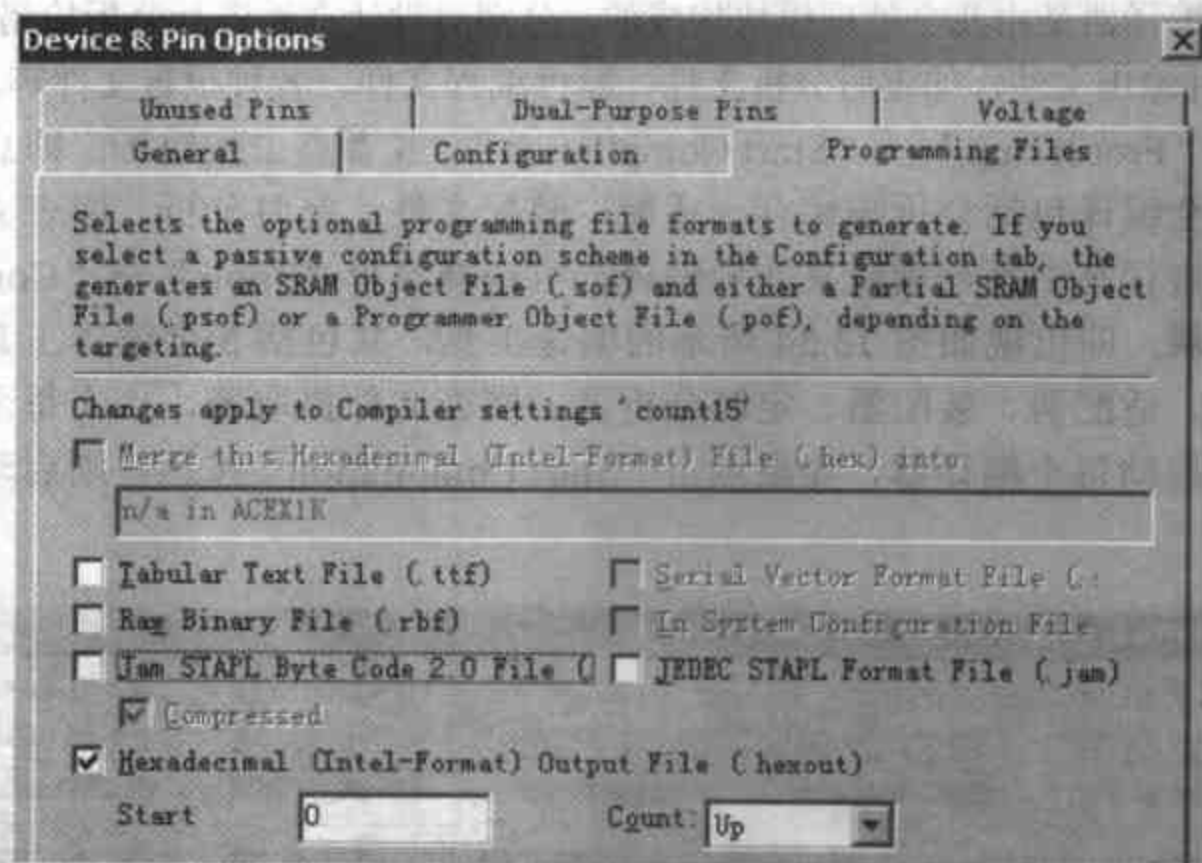


图 13.52 选择配置文件格式

在图 13.52 中点击“Unused Pins”，可设置目标器件闲置引脚的状态，可将闲置引脚设置为高阻输入状态，或低电平输出状态，或输出不定状态，也可以不作任何选择。

还可以选择编译模式，在如图 13.53 所示的“Settings”窗口中点击“Compiler Settings”项下的“Mode”，出现模式设置窗口。在图中，选择“Smart compilation...”和“Preserve fewer...”选项，这样可使得每次的重复编译能运行得更快。

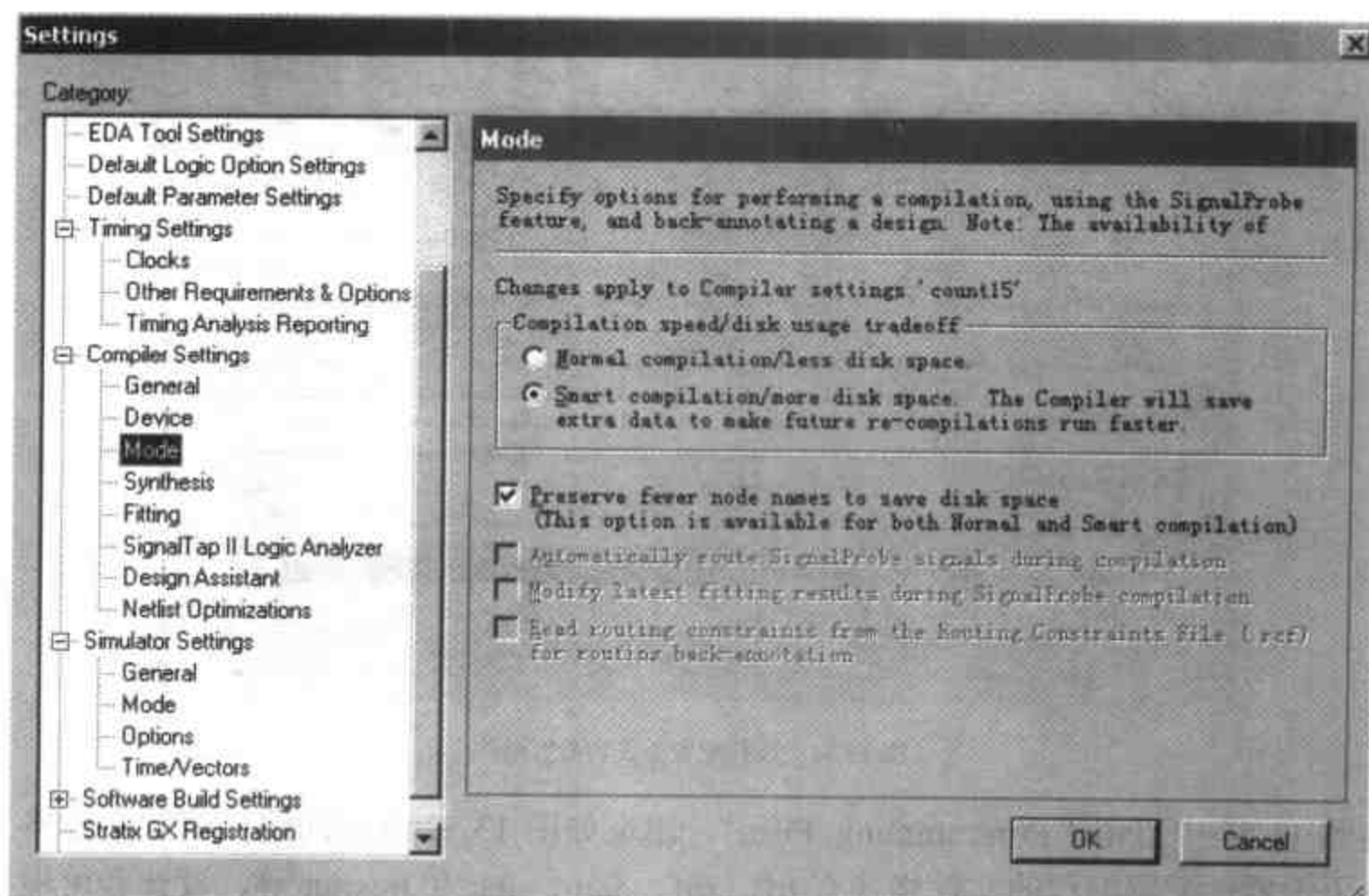


图 13.53 选择编译模式

2. 编译

Quartus II 编译器是由几个处理模块构成的，分别对设计文件进行分析检错、综合、适配等，并产生多种输出文件，如定时分析文件、器件编程文件、各种报告文件等。

选择菜单“Processing”→“Start Compilation”，或者点击按钮，即启动了完全编译，这里的完全编译包括分析与综合、适配、装配文件、定时分析、网表文件提取等过程。如果只想进行其中某一项或某几项编译，可选择菜单“Tools”→“Compiler Tool”，或者点击按钮，即出现如图 13.54 所示的编译工具，共包括 5 个编译工具，分别为：分析与综合器、适配器、装配器、定时分析器、网表文件提取器。点击每个工具前面的小图标可单独启动每个编译器，如果点击“Start Compilation”按钮，则启动了整个编译过程。



图 13.54 选择编译工具

3. 编译信息显示

编译完成后，可以看到如图 13.55 所示的界面，在其中点击报告选项中的某一条可查看相关内容的报告。

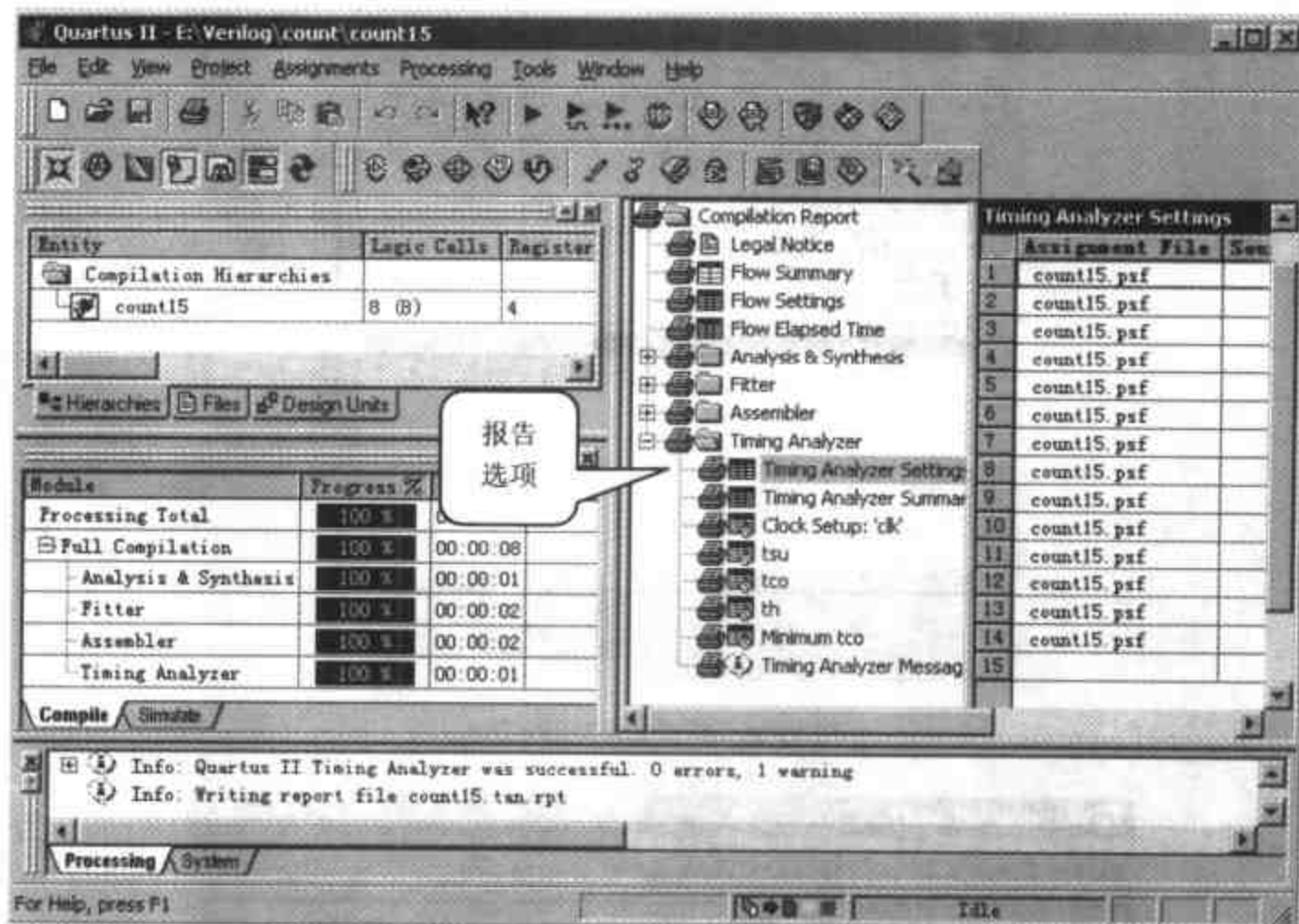


图 13.55 项目编译完成后的界面

13.7.3 仿真

Quartus II 支持功能仿真和时序仿真，功能仿真只检验设计项目的逻辑功能，时序仿真则将延时信息也考虑在内，更符合系统的实际工作情况。

Quartus II 允许对整个设计项目进行仿真测试，也可以只对该项目中的某个子模块进行仿真，其方法是选择菜单“Assignment”→“Simulator Settings Wizard”，在设置过程中指定仿真对象（Simulator Focus），并指定该对象的仿真类型、矢量激励源等。

矢量激励源可以是矢量波形文件.vwf（Vector Waveform File）、文本矢量文件.vec（Vector File）、矢量表输出文件.tbl 或功率输入文件.pwf 等，也可以通过 Tcl 脚本窗口来输入矢量激励源。

其中，.vwf 文件是 Quartus II 中最主要的波形文件；.vec 文件是 MAX+PLUS II 中的文件，主要是为了兼容而采用的文件格式；.tbl 文件则用来将 MAX+PLUS II 中的.scf 文件输入到 Quartus II 中。

下面着重介绍以矢量波形文件（.vwf）作为激励源进行仿真的步骤。

1. 打开波形编辑器

选择菜单“File”→“New”，在 New 窗口中选择“Other Files”页面中的“Vector Waveform File”选项（如图 13.56 所示），点击“OK”，即出现如图 13.57 所示的波形编辑窗口。

2. 输入信号节点

选择菜单“View”→“Utility Windows”→“Node Finder”，出现如图 13.58 所示的对话框，在“Filter”框中选择“Pins: all”，再点击“Start”按钮，即在下面的“Node Finder”框中出现本设计项目中的所有端口引脚列表，从端口列表中选择所需要的，并逐个拖到图 13.57 所示的波形编辑窗口中。

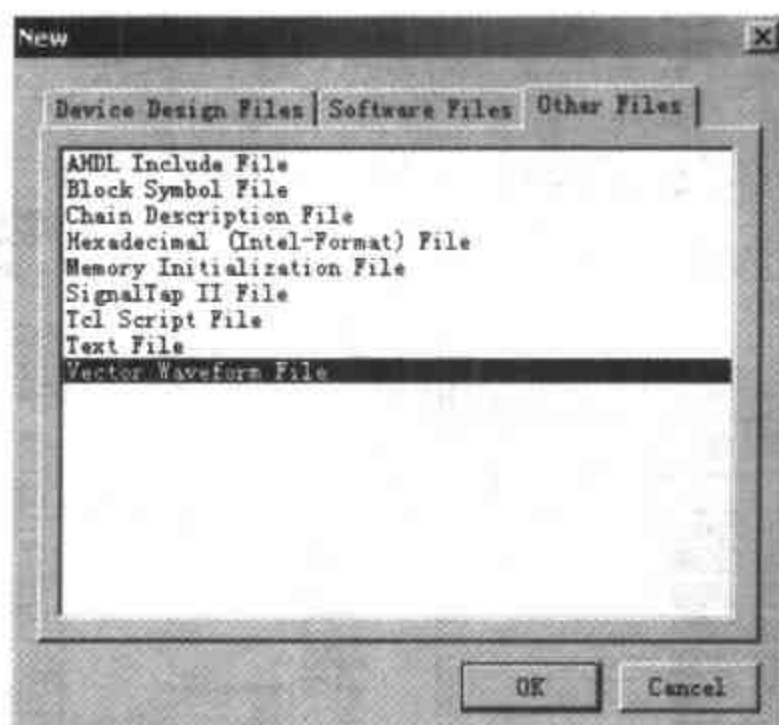


图 13.56 新建矢量波形文件

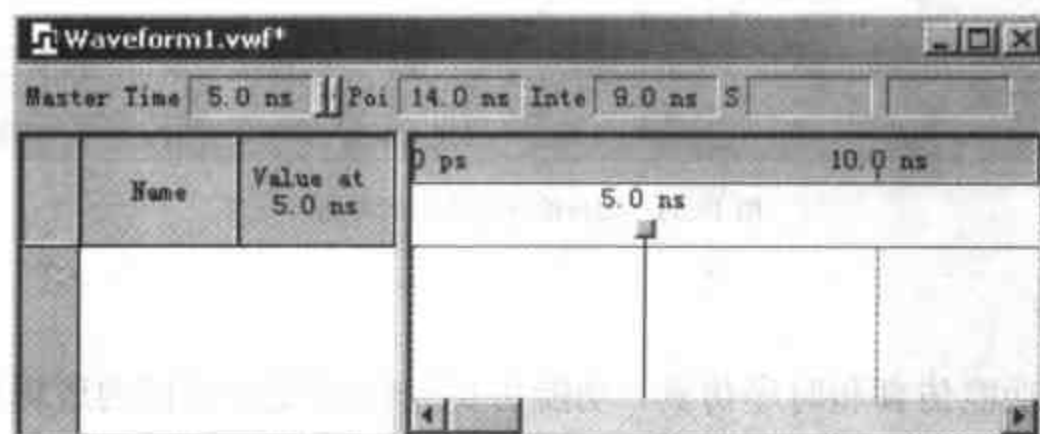


图 13.57 波形编辑窗口

3. 编辑输入信号波形

点击波形编辑窗口中的全屏显示钮，使窗口全屏显示，使用波形编辑窗口中的各种波形赋值快捷键，编辑各输入信号的激励波形，如图 13.59 所示。另外，还需设置仿真时间区域，以使仿真时间设置在一个合理的区域上，选择菜单“Edit”→“End Time”，在弹出的“Time”窗口中键入 10，单位选“ μs ”，即仿真时长为 $10\mu\text{s}$ 。

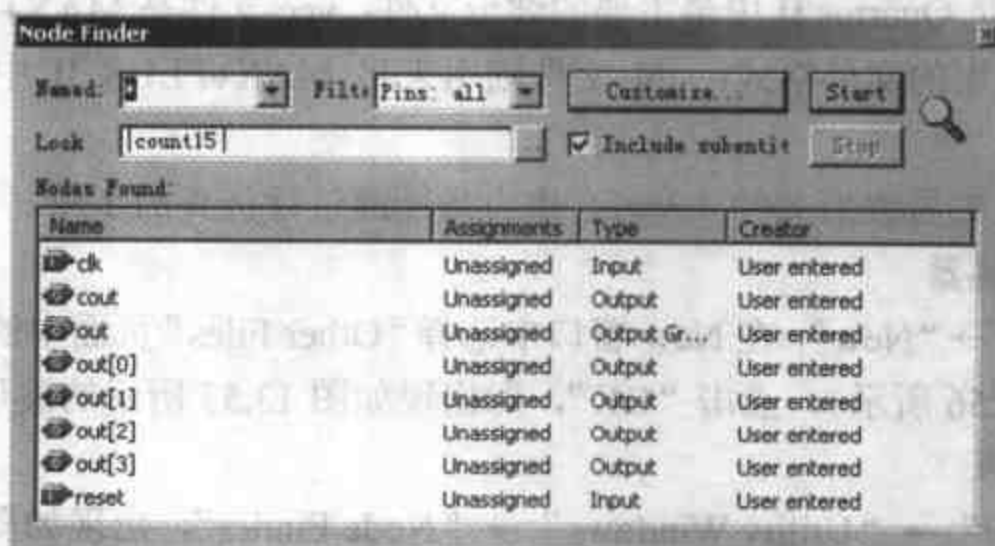


图 13.58 输入信号节点

图 13.59 中的 out 信号为总线型数据（信号左旁有符号“+”），如果点击信号“out”左旁

的“+”，则将展开该总线中的所有信号，如果双击“+”左旁的信号标记，则弹出信号数据格式设置对话框，如图 13.60 所示。该对话框中有 5 种数据格式可供选择，分别是 Binary：二进制；Hexadecimal：十六进制；Octal：八进制；Signed Decimal：有符号十进制；Unsigned Decimal：无符号十进制。这里选择 out 信号格式为无符号十进制，其余信号选择二进制格式。

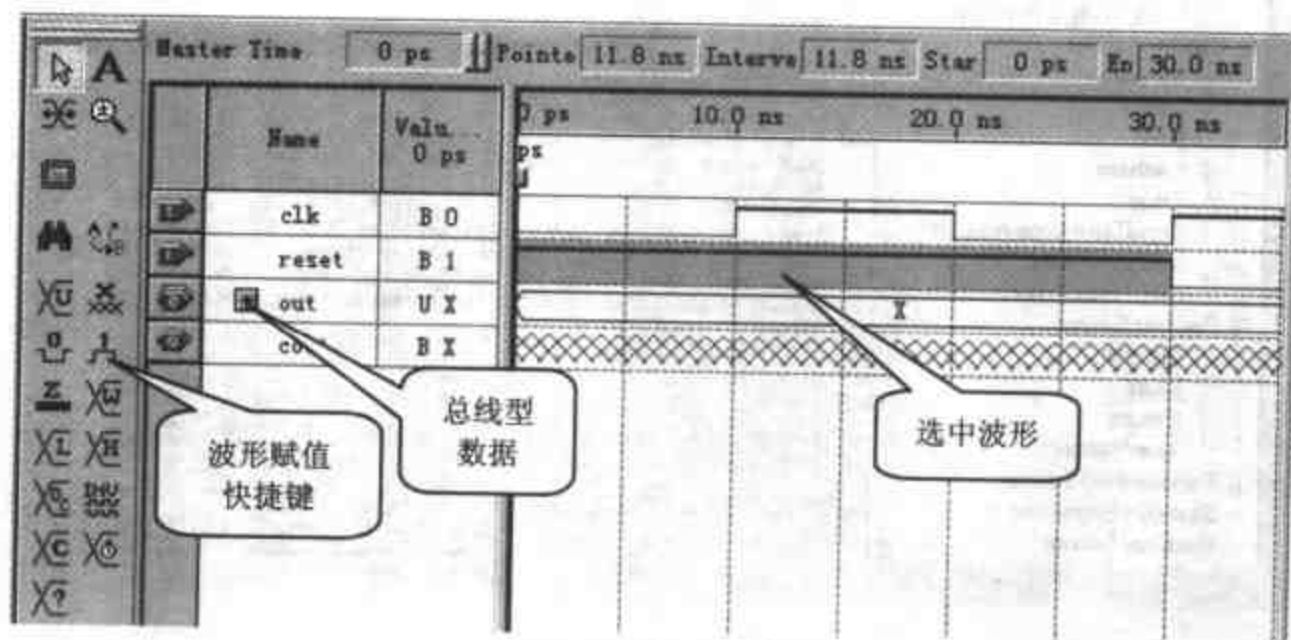


图 13.59 编辑输入信号波形

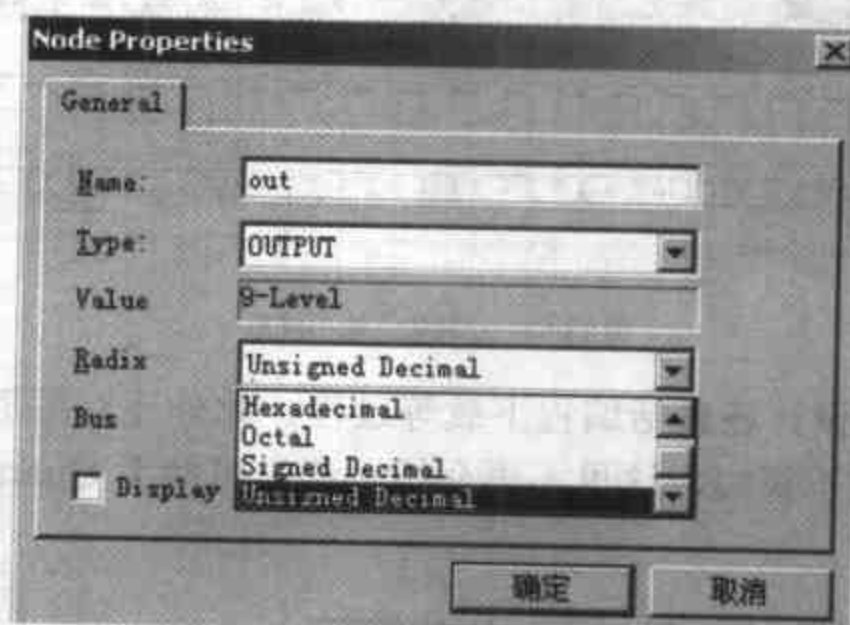


图 13.60 设置信号数据格式

4. 仿真器参数设置

选择“Assignments”→“Settings”项，在弹出的对话框中选“Simulator Settings”项下的“Mode”，以选择仿真模式，仿真模式有功能仿真（“Functional”）和时序仿真（“Timing”）两种，此处选择功能仿真（“Functional”），如图 13.61 所示。还可以设置其他一些选项，如在时序仿真模式下可选择毛刺检测的宽度等。

5. 观察仿真结果

选择菜单“Processing”→“Start Simulation”，或者点击按钮 ，即启动仿真器开始工作。仿真完成后，可通过查看输出波形，以检验所设计电路的功能是否正确。如不正确，可修改设计，重新执行以上的过程，直到完全满足自己的设计要求为止。模 15 减法计数器的功能仿真输出波形如图 13.62 所示。

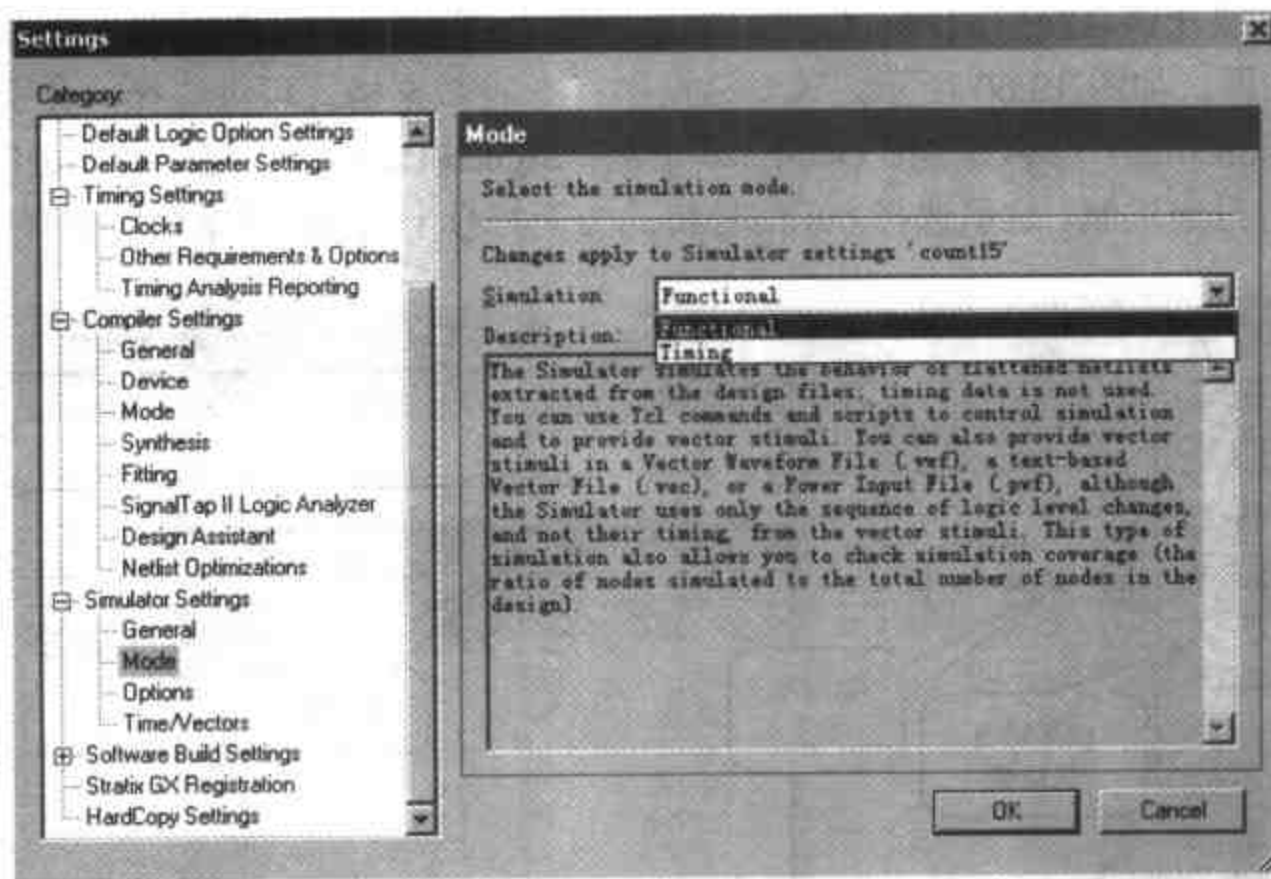


图 13.61 设置仿真模式

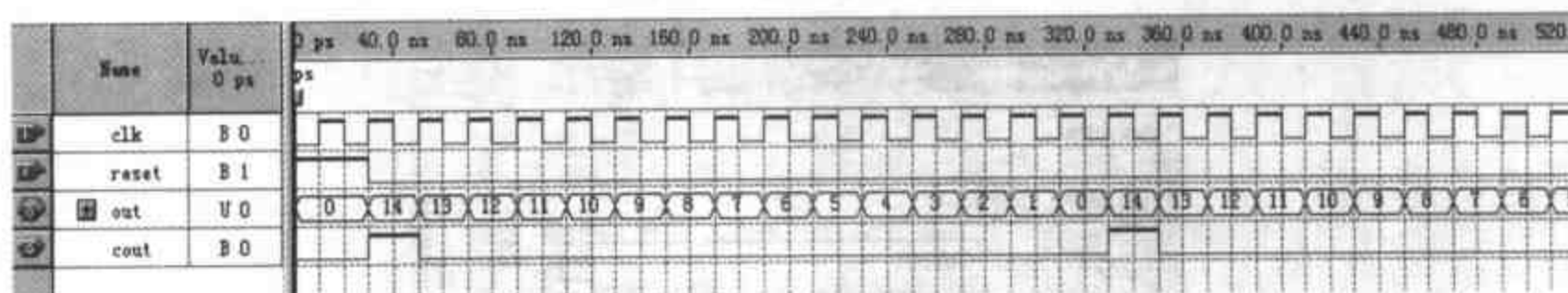


图 13.62 功能仿真输出波形

完整的 Quartus II 设计还包括编程下载等过程, 此外还有图形设计输入、使用宏功能模块进行设计等, 限于篇幅, 这里不再介绍, 读者可参考 Quartus II 帮助文档以及有关的资料。

13.7.4 Synplify Pro 与 Quartus II 的接口

Synplify Pro 只具有综合功能, 在综合后生成针对某种器件的门级结构网表文件, 针对 Altera 的器件, Synplify Pro 提供的门级结构网表文件一般是 Verilog Quartus Mapping File (*.vqm) 或者 EDIF File (*.edf), 可将该网表文件送 Quartus II 进行适配和下载等。

Synplify Pro 提供了多种方式与 Quartus II 接口, 最简单的就是在 Synplify Pro 中直接调用 Quartus II。

启动 Synplify Pro, 在创建了工程文件和源文件并设定了目标器件 (这里选择 Altera 的器件) 后, 选择菜单 “Option” → “Quartus” → “Run Foreground Compiler”, 如图 13.63 所示, Synplify 会自动打开 Quartus II 进行编译。

Quartus II 与其他第三方 EDA 工具也有很好的接口, 如 Leonardo Spectrum、ModelSim 等, 这些工具相互配合, 可有效提高数字系统设计的效率。

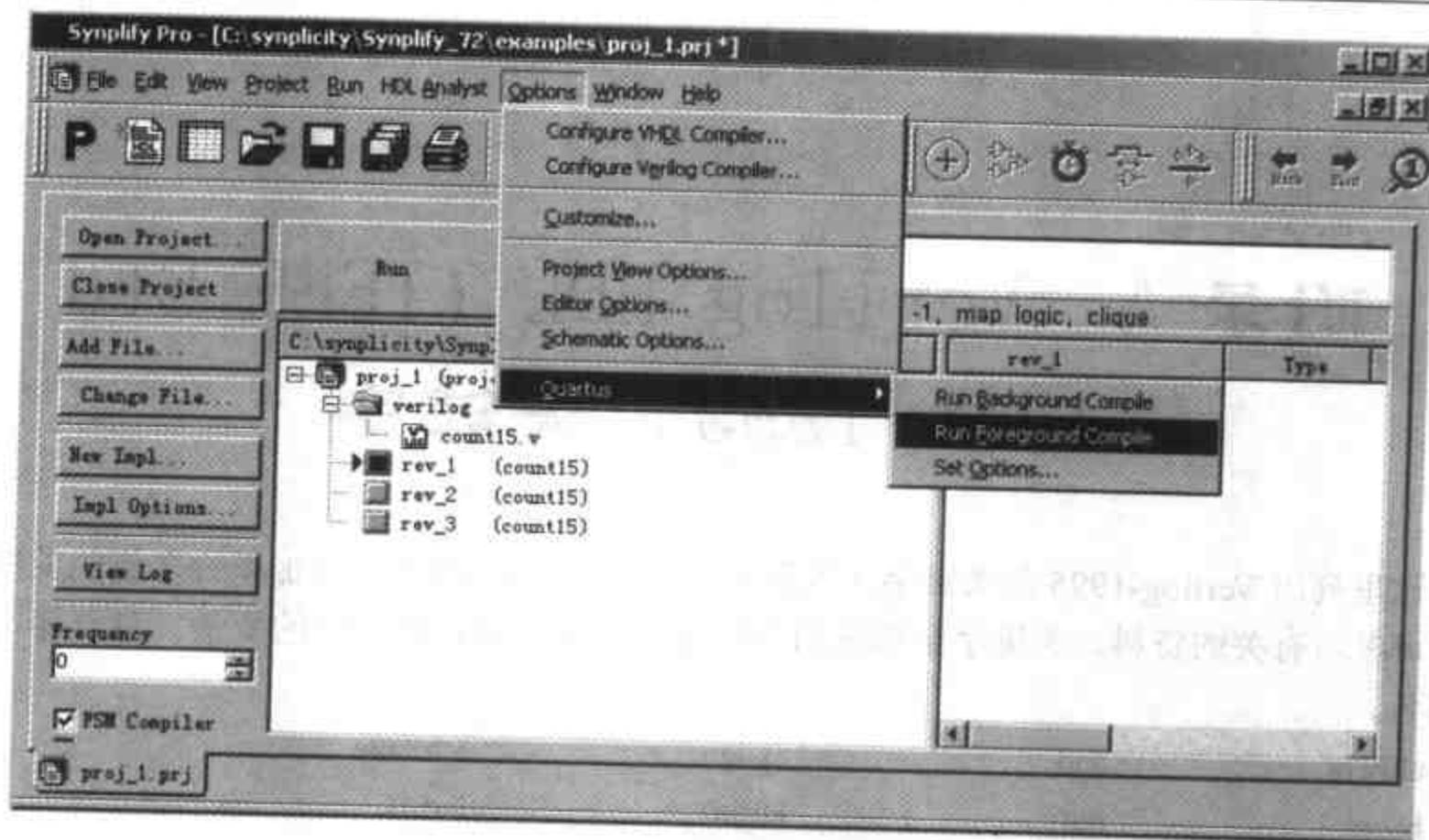


图 13.63 在 Synplify Pro 中调用 Quartus II

13.8 结 束 语

EDA 工具对于 HDL 设计开发具有非常重要的作用，好的 EDA 工具对于提高设计效率、改善优化质量、增强仿真测试的排错能力、加强设计成果的可移植性和可重用性以及扩展目标器件的选择范围等，都有着直接的影响。

EDA 技术的发展日新月异，包括了设计技术和设计方法的进步，设计工具和设计软件的改进，器件（FPGA 和 ASIC 等）性能的提升，此外，还有 HDL 语言的发展。

EDA 技术的发展是永无止境的，它随着计算机技术、设计方法学等技术的进步而不断进步。在 EDA 技术的支持下，数字系统的设计将会越来越方便。现代电子系统的设计已经离不开 EDA 工具的支持了，不管是各种普通消费类电子设备，还是各种军用、医用精密电子设备以及各种嵌入式设备，其设计与实现假如没有 EDA 软件工具的支持将是不可想象的。

HDL 语言也在不断地发展、完善，新的语言标准不断被制订和推出，其目的就是增强语言的功能，并使其更加好用。Verilog 自成为 IEEE 标准以来，先后通过了 IEEE 1364-1995 和 IEEE 1364-2001 两个标准，对 Verilog 语言进行了很多扩充和增强，也使它越来越好用。Verilog 是有着良好发展前景的硬件描述语言，正得到越来越广泛的应用。

附录 A Verilog HDL (IEEE Std 1364-1995) 关键字

这里列出 Verilog-1995 的关键字（保留字），在 Verilog-2001 中增加了许多新的关键字，读者可参考有关的资料。关键字全部采用小写字母构成，用户程序中的变量、节点等不可与其同名。

always	and	assign	attribute
begin	buf	bufif0	bufif1
case	casex	casez	cmos
deassign	default	defparam	disable
edge	else	end	endattribute
endcase	endfunction	endmodule	endprimitive
endspecify	endtable	endtask	event
for	force	forever	fork
function	hignz0	hignz1	if
ifnone	initial	inout	input
integer	join	large	macromodule
medium	module	nand	negedge
nmos	nor	not	notif0
notif1	or	output	parameter
pmos	posedge	primitive	pull0
pull1	pulldown	pullup	rcmos
real	realtime	reg	release
repeat	rnmos	rpmos	rtran
rtranif0	rtranif1	scalared	small
signed	specify	specparam	strength
strong0	strong1	supply0	supply1
table	task	time	tran
tranif0	tranif1	tri	tri0
tri1	triand	trior	trireg
unsigned	vectored	wait	wand
weak0	weak1	while	wire
wor	xnor	xor	

附录 B Synplify Pro/Synplify 可综合的 Verilog 结构

(1) Synplify Pro/Synplify 综合工具支持下面的 Verilog 结构

- ◆ 数据类型: net 型: wire, tri, supply1, supply0
register 型: reg, integer, time (64 bit reg); reg 阵列 (存储器)
- ◆ 端口类型: input, output, inout
- ◆ 持续赋值: assign
- ◆ 过程赋值: 阻塞赋值 “=” 和非阻塞赋值 “<=”, 但同一个寄存器变量不能同时使用 “<=” 和 “=” 赋值
- ◆ 门元件和模块例化
- ◆ always 过程块, 任务 (task) 和函数 (function)
- ◆ 所有的运算符: +, -, *, /, %, <, >, <=, >=, ==, !=, ===, !==, &&, ||, !, ~, &, ~&, |, ~|, ^~, ~^, ^, <<, >>, ?:, { }, { { } }

其中, 对 “/” 和 “and” 有所限制, 只适用于操作数为 2 的幂的情况下。

- ◆ 行为语句: if-else-if, case, casex, casez, for, repeat, while, forever, begin, end, fork, join
- ◆ 编译向导: `define, `ifdef, `else, `endif, `include, `undef 等等

(2) Synplify Pro/Synplify 不支持下面的 Verilog 结构, 如果遇到这些结构, 会报错并停止综合

- ◆ 数据类型: net 型: trireg, wor, trior, wand, triand, tri0, tri1 和电荷强度 (strength)
register 型: real
- ◆ 内置单向和双向开关, pull-up, pull-down
- ◆ 行为语句: deassign, wait
- ◆ 有名事件和事件触发
- ◆ 用户定义元件 (UDP) 和延时定义模块 (specify block)
- ◆ force, release 和层次 net 名 (仅用于仿真)

(3) Synplify Pro/Synplify 忽略的 Verilog 结构, 如果遇到下面这些结构, 不会报错, 也不会停止综合

- ◆ delay, delay 控制, 驱动强度
- ◆ initial
- ◆ 除 `define, `ifdef, `else, `endif, `include, `undef 外的编译向导语句
- ◆ 系统任务和系统函数

附录 C MAX+PLUS II 软件支持的 Verilog 结构

项 目	数据类型、语句	可综合性说明
数据类型	net 型	支持 wire 型, 不支持 tri0, tri1, wor, trior, wand, supply0, supply1, triand, trireg 型
	寄存器型	支持 reg 型, integer 只在 for 语句中支持, 不支持 real, time 型
	向量 (vectors)	支持
	parameter	支持
运算符	+, -, *, /, %, &, ~&, , ~ , ^, ^~, ==, !=, ===, !==, &&, , !, ~, &, , ^, ^~, >>, <<, ?, { }	除了 /, %, ==, != 外都支持 使用 >> 和 << 时, 右操作数必须是常数
门原语	and, nand, nor, or, xor, xnor, buf, not, bufif1, bufif0, notif1, notif0	支持
开关级原语	mos, cmos, pullup, pulldown 等	不支持
赋值语句	连续赋值 assign	支持
	过程赋值: 阻塞赋值 (=) 非阻塞赋值 (<=)	支持
条件语句	if-else, case	支持 (casez 不支持)
循环语句	for	支持
	forever, while, repeat	不支持
过程语句	always	支持
	function	支持
	initial, task	不支持
编译向导	`include, `define, `ifdef, `else, `endif	支持
系统任务和系统函数, 用户定义原语 (UDP)		不支持

附录 D 形式化句法定义

下面是用 BNF 的形式给出的 IEEE Std 1364-1995 Verilog HDL 形式化句法定义。BNF 是一种用来定义编程语言句法的标准形式，其规范如下所示：

- ◆ `::=` 为句法定义符，表示定义符左边的项由右边的表达式来定义。
- ◆ 符号 `|` 用于分隔多个选项。
- ◆ 方括号 `[]` 表示括号内的项是可选项。
- ◆ 大括号 `{ }` 表示括号内的项可以重复 0 次、1 次或多次。
- ◆ 关键字（保留字）、运算符和标点符号以加粗表示。
- ◆ 加粗的 `|`、方括号 `[]`、大括号 `{ }` 等，表示它们是句法的一部分。
- ◆ *name1*_*name2*：表示斜体的 *name1* 是对 *name2* 的注解。

D.1 Source Text

```
source_text ::= {description}
description ::=
    module_declaration
    | udp_declaration
module_declaration ::=
    module_keyword module_identifier [list_of_ports] ; {module_item}
    endmodule
module_keyword ::= module | macromodule
list_of_ports ::= ( port { ,port } )
port ::=
    [ port_expression ]
    | port_identifier ( [ port_expression ] )
port_expression ::=
    port_reference
    | { port_reference { , port_reference } }
port_reference ::=
    port_identifier
    | port_identifier [ constant_expression ]
    | port_identifier [ msb_constant_expression : lsb_constant_expression ]
```

```
module_item ::=  
    module_item_declaration  
    | parameter_override  
    | continuous_assign  
    | gate_instantiation  
    | udp_instantiation  
    | module_instantiation  
    | specify_block  
    | initial_construct  
    | always_construct  
module_item_declaration ::=  
    parameter_declaration  
    | input_declaration  
    | output_declaration  
    | inout_declaration  
    | net_declaration  
    | reg_declaration  
    | integer_declaration  
    | real_declaration  
    | time_declaration  
    | realtime_declaration  
    | event_declaration  
    | task_declaration  
    | function_declaration  
parameter_override ::= defparam list_of_param_assignments;
```

D.2 Declaration

```
parameter_declaration ::= parameter list_of_param_assignments ;  
list_of_param_assignments ::= param_assignment { , param_assignment }  
param_assignment ::= parameter_identifier = constant_expression  
input_declaration ::= input [ range ] list_of_port_identifiers ;  
output_declaration ::= output [ range ] list_of_port_identifiers ;  
inout_declaration ::= inout [ range ] list_of_port_identifiers ;  
list_of_port_identifiers ::= port_identifier { , port_identifier }  
reg_declaration ::= reg [ range ] list_of_register_identifiers ;  
time_declaration ::= time list_of_register_identifiers ;  
integer_declaration ::= integer list_of_register_identifiers ;
```

```

real_declaration ::= real list_of_real_identifiers ;
realtime_declaration ::= realtime list_of_real_identifiers ;
event_declaration ::= event event_identifier { , event_identifier } ;
list_of_real_identifiers ::= real_identifier { , real_identifier }
list_of_register_identifiers ::= register_name { , register_name }
register_name ::=
    register_identifier
    | memory_identifier [ upper_limit_constant_expression :
        lower_limit_constant_expression ]
range ::= [ msb_constant_expression : lsb_constant_expression ]
net_declaration ::=
    net_type [ vectored | scalared ] [ range ] [ delay3 ] list_of_net_identifiers ;
    | trireg [ vectored | scalared ] [ charge_strength ] [ range ] [ delay3 ]
        list_of_net_identifiers ;
    | net_type [ vectored | scalared ] [ drive_strength ] [ range ] [ delay3 ]
        list_of_net_decl_assignments ;
net_type ::= wire | tri | tril | supply0 | wand | triand | tri0 | supply | wor | trior
list_of_net_identifiers ::= net_identifier { , net_identifier }
drive_strength ::=
    (strength0, strength1)
    | (strength1, strength0)
    | (strength0, highz1)
    | (strength1, highz0)
    | (highz1, strength0)
    | (highz0, strength1)
strength0 ::= supply0 | strong0 | pull0 | weak0
strength1 ::= supply1 | strong1 | pull1 | weak1
charge_strength ::= ( small ) | ( medium ) | ( large )
delay3 ::= # delay_value | # ( delay_value [ , delay_value [ , delay_value ] ] )
delay2 ::= # delay_value | # ( delay_value [ , delay_value ] )
delay_value ::= unsigned_number | parameter_identifier |
    constant_mintypmax_expression
list_of_net_decl_assignments ::= net_decl_assignment { , net_decl_assignment }
net_decl_assignment ::= net_identifier = expression
function_declaration ::=
    function [ range_or_type ] function_identifier ;
    function_item_declaration { function_item_declaration }
    statement
    endfunction

```

```
range_or_type ::= range | integer | real | realtime | time
```

```
function_item_declaration ::=
```

```
    block_item_declaration
```

```
    | input_declaration
```

```
task_declaration ::=
```

```
    task task_declaration ;
```

```
    { task_item_declaration }
```

```
    statement_or_null
```

```
    endtask
```

```
task_item_declaration ::=
```

```
    block_item_declaration
```

```
    | input_declaration
```

```
    | output_declaration
```

```
    | inout_declaration
```

```
block_item_declaration ::=
```

```
    parameter_declaration
```

```
    | reg_declaration
```

```
    | integer_declaration
```

```
    | real_declaration
```

```
    | time_declaration
```

```
    | realtime_declaration
```

```
    | event_declaration
```

D.3 Primitive Instances

```
gate_instantiation ::=
```

```
    n_input_gatetype [ drive_strength ] [ delay2 ] n_input_gate_instance
```

```
        { , n_input_gate_instance } ;
```

```
    n_output_gatetype [ drive_strength ] [ delay2 ] n_output_gate_instance
```

```
        { , n_output_gate_instance } ;
```

```
    enable_gatetype [ drive_strength ] [ delay3 ] enable_gate_instance
```

```
        { , enable_gate_instance } ;
```

```
    mos_switchtype [ delay3 ] mos_switch_instance { , mos_switch_instance } ;
```

```
    pass_switchtype pass_switch_instance { , pass_switch_instance } ;
```

```
    pass_en_switchtype [ delay3 ] pass_en_switch_instance
```

```
        { , pass_en_switch_instance } ;
```

```
    cmos_switchtype [ delay3 ] cmos_switch_instance { , cmos_switch_instance } ;
```

```
    pullup [ pullup_strength ] pull_gate_instance { , pull_gate_instance } ;
```

```

| pulldown [ pulldown_strength ] pull_gate_instance { , pull_gate_instance };
n_input_gate_instance ::= [ name_of_gate_instance ] ( output_terminal , input_terminal
    { , input_terminal } )
n_output_gate_instance ::= [ name_of_gate_instance ] ( output_terminal { , output_terminal }
    , input_terminal )
enable_gate_instance ::= [ name_of_gate_instance ] ( output_terminal , input_terminal
    , enable_terminal )
mos_switch_instance ::= [ name_of_gate_instance ] ( output_terminal , input_terminal
    , enable_terminal )
pass_switch_instance ::= [ name_of_gate_instance ] ( inout_terminal , inout_terminal )
pass_en_switch_instance ::= [ name_of_gate_instance ] ( inout_terminal , inout_terminal ,
    enable_terminal )
cmos_switch_instance ::= [ name_of_gate_instance ] ( output_terminal , input_terminal ,
    ncontrol_terminal , pcontrol_terminal )
pull_gate_instance ::= [ name_of_gate_instance ] ( output_terminal )
name_of_gate_instance ::= gate_instance_identifier [ range ]
pullup_strength ::=
    ( strength0 , strength1 )
    | ( strength1 , strength0 )
    | ( strength1 )
pulldown_strength ::=
    ( strength0 , strength1 )
    | ( strength1 , strength0 )
    | ( strength0 )
input_terminal ::= scalar_expression
enable_terminal ::= scalar_expression
ncontrol_terminal ::= scalar_expression
pcontrol_terminal ::= scalar_expression
output_terminal ::= terminal_identifier | terminal_identifier [ constant_expression ]
input_terminal ::= terminal_identifier | terminal_identifier [ constant_expression ]
n_input_gatetype ::= and | nand | or | nor | xor | xnor
n_output_gatetype ::= buf | not
enable_gatetype ::= bufif0 | bufif1 | notif0 | notif1
mos_switchtype ::= nmos | pmos | rnmos | rpmos
pass_switchtype ::= tran | rtran
pass_en_switchtype ::= tranif0 | tranif1 | rtranif1 | rtranif0
cmos_switchtype ::= cmos | rcmos

```

D.4 Module Instantiations

```

module_ instantiation ::= module_ identifier [ parameter_ value_ assignment ] module_ instance
                        { , module_ instance } ;

parameter_ value_ assignment ::= # ( expression { , expression } )

module_ instance ::= name_ of_ instance ( [ list_ of_ module_ connections ] )

name_ of_ instance ::= module_ instance_ identifier [ range ]

list_ of_ module_ connections ::=
    ordered_ port_ connection { , ordered_ port_ connection }
    | named_ port_ connection { , named_ port_ connection }

ordered_ port_ connection ::= [ expression ]

named_ port_ connection ::= . port_ identifier ( [ expression ] )

```

D.5 UDP Instantiation

```

udp_ declaration ::=
    primitive udp_ identifier ( udp_ port_ list ) ;
    udp_ port_ declaration { udp_ port_ declaration }
    udp_ body
    endprimitive

udp_ port_ list ::= output_ port_ identifier , input_ port_ identifier { , input_ port_ identifier }

udp_ port_ declaration ::=
    output_ declaration
    | input_ declaration
    | reg_ declaration

udp_ body ::= combinational_ body | sequential_ body

combinational_ body ::= table combinational_ entry { combinational_ entry } endtable

combinational_ entry ::= level_ input_ list : output_ symbol ;

sequential_ body ::= [ udp_ initial_ statement ] table sequential_ entry
                    { sequential_ entry } endtable

udp_ initial_ statement ::= initial udp_ output_ port_ identifier = init_ val ;

init_ val ::= 1'b0 | 1'b1 | 1'bx | 1'B0 | 1'B1 | 1'Bx | 1'Bx | 1 | 0

sequential_ entry ::= seq_ input_ list : current_ state : next_ state ;

seq_ input_ list ::= level_ input_ list | edge_ input_ list

level_ input_ list ::= level_ symbol { level_ symbol }

edge_ input_ list ::= { level_ symbol } edge_ indicator { level_ symbol }

edge_ indicator ::= ( level_ symbol level_ symbol ) | edge_ symbol

```



```

current_state ::= level_symbol
next_state ::= output_symbol | -
output_symbol ::= 0 | 1 | x | X
level_symbol ::= 0 | 1 | x | X | ? | b | B
edge_symbol ::= r | R | f | F | p | P | n | N | *
udp_instantiation ::= udp_identifier [ drive_strength ] [ delay2 ] udp_instance { , udp_instance };
udp_instance ::= [ name_of_udp_instance ] ( output_port_connection,
                                             input_port_connection { , input_port_connection } )
name_of_udp_instance ::= udp_instance_identifier [ range]

```

D.6 Behavioral Statements

```

continuous_assign ::= assign [ drive_strength ] [ delay3 ] list_of_net_assignments ;
list_of_net_assignments ::= net_assignment { , net_assignment }
net_assignment ::= net_lvalue = expression
initial_construct ::= initial statement
always_construct ::= always statement
statement ::=
    blocking_assignment ;
    | non_blocking_assignment ;
    | procedural_continuous_assignment ;
    | procedural_timing_control_statement
    | conditional_statement
    | case_statement
    | loop_statement
    | wait_statement
    | disable_statement
    | event_trigger
    | seq_block
    | par_block
    | task_enable
    | system_task_enable
statement_or_null ::= statement | ;
blocking_assignment ::= reg_lvalue = [ delay_or_event_control ] expression
non_blocking_assignment ::= reg_lvalue <= [ delay_or_event_control ] expression
procedural_continuous_assignment ::=
    assign reg_assignment ;
    | deassign reg_lvalue ;

```

```

    | force reg_ assignment ;
    | force net_ assignment ;
    | release reg_ lvalue ;
    | release net_ lvalue ;
procedural_timing_control_statement ::= delay_or_event_control statement_or_null
delay_or_event_control ::=
    delay_control
    | event_control
    | repeat ( expression ) event_control
delay_control ::=
    # delay_value
    | # ( mintypmax_expression )
event_control ::=
    @ event_identifier
    | @ ( event_expression )
event_expression ::=
    expression
    | event_identifier
    | posedge_ expression
    | negedge_ expression
    | event_expression or event_expression
conditional_statement ::=
    if ( expression ) statement_or_null [ else statement_or_null ]
case_statement ::=
    case ( expression ) case_item { case_item } endcase
    | casez ( expression ) case_item { case_item } endcase
    | casex ( expression ) case_item { case_item } endcase
case_item ::=
    expression { , expression } : statement_or_null
    | default [ : ] statement_or_null
loop_statement ::=
    forever statement
    | repeat( expression ) statement
    | while( expression ) statement
    | for ( reg_assignment; expression ; reg_assignment ) statement
reg_assignment ::= reg_lvalue = expression
wait_statement ::=
    wait ( expression ) statement_or_null
event_trigger ::=

```



```

-> event_ identifier ;
disable_ statement : : =
    disable task_ identifier ;
    | disable block_ identifier ;
seq_ block : : = begin [ : block_ identifier { block_ item_ declaration } ] { statement } end
par_ block : : = fork [ : block_ identifier { block_ item_ declaration } ] { statement } join
task_ enable : : = task_ identifier [ ( expression { , expression } ) ] ;
system_ task_ enable : : = system_ task_ name [ ( expression { , expression } ) ] ;
system_ task_ name : : = $ identifier

```

D.7 Specify Section

```

specify_ block : : = specify [ specify_ item ] endspecify
specify_ item : : =
    specparam_ declaration
    | path_ declaration
    | system_ timing_ check
specparam_ declararion : : = specparam list_ of_ specparam_ assignments ;
list_ of_ specparam_ assignments : : = specparam_ assignment { , specparam_ assignment }
specparam_ assignment : : =
    specparam_ idenrifier = constant_ expression
| pulse_ control_ specparam
pulse_ control_ specparam : : =
    PATHPULSE $ = ( reject_ limit_ value [ , error_ limit_ value ] ) ;
    | PATHPULSE $ specify_ input_ terminal_ descriptor $specify_ output_ terminal_
    descriptor = ( reject_ limit_ value [ , error_ limit_ value ] ) ;
limit_ value : : = constant_ mintypmax_ expression
path_ declaration : : =
    simple_ path_ declaration ;
    | edge_ sensitive_ path_ declaration ;
    | state_ dependent_ path_ declaration ;
simple_ path_ declaration : : =
    parallel_ path_ description = path_ delay_ value
    | full_ path_ description = path_ delay_ value
parallel_ path_ description : : =
    ( specify_ input_ terminal_ descriptor [ polarity_ operator ] =>
    specify_ output_ terminal_ descriptor )
full_ path_ description : : =

```

```
( list_of_path_inputs [ polarity_operator ] * > list_of_path_outputs )  
list_of_path_inputs ::=  
    specify_input_terminal_descriptor { , specify_input_terminal_descriptor }  
list_of_path_outputs ::=  
    specify_output_terminal_descriptor { , specify_output_terminal_descriptor }  
specify_input_terminal_descriptor ::=  
    input_identifier  
    | input_identifier [ constant_expression ]  
    | input_identifier [ msb_constant_expression : lsb_constant_expression ]  
specify_output_terminal_descriptor ::=  
    output_identifier  
    | output_identifier [ constant_expression ]  
    | output_identifier [ msb_constant_expression : lsb_constant_expression ]  
input_identifier ::= input_port_identifier | inout_port_identifier  
output_identifier ::= output_port_identifier | inout_port_identifier  
polarity_operator ::= + | -  
path_delay_value ::=  
    list_of_path_delay_expressions  
    | ( list_of_path_delay_expressions )  
list_of_path_delay_expressions ::=  
    t_path_delay_expression  
    | trise_path_delay_expression , tfall_path_delay_expression  
    | trise_path_delay_expression , tfall_path_delay_expression ,  
      tz_path_delay_expression  
    | t01_path_delay_expression , t10_path_delay_expression ,  
      t0z_path_delay_expression , tz1_path_delay_expression ,  
      t1z_path_delay_expression , tz0_path_delay_expression  
    | t01_path_delay_expression , t10_path_delay_expression ,  
      t0z_path_delay_expression ,  
      tz1_path_delay_expression , t1z_path_delay_expression ,  
      tz0_path_delay_expression ,  
      t0x_path_delay_expression , tx1_path_delay_expression ,  
      t1x_path_delay_expression ,  
      tx0_path_delay_expression , txz_path_delay_expression ,  
      txx_path_delay_expression  
path_delay_expression ::= constant_mintypmax_expression  
edge_sensitive_path_declaration ::=  
    parallel_edge_sensitive_path_declaration = path_delay_value  
    | full_edge_sensitive_path_declaration = path_delay_value
```

```

parallel_edge_sensitive_path_declaration ::=
    ( [ edge_identifier ] specify_input_terminal_descriptor =>
        specify_output_terminal_descriptor [ polarity_operator ] :data_source_expression )

full_edge_sensitive_path_decicription ::=
    ( [ edge_identifier ] list_of_path_inputs * >
        list_of_path_outputs [ polarity_operator ] : data_source_expression )

data_source_expression ::= expression

edge_identifier ::= posedge | negedge

state_dependent_path_declaration ::=
    if ( conditional_expression ) simple_path_declaration
    | if ( conditional_expression ) edge_sensitive_path_declaration
    | ifnone simple_path_declaration

system_timing_check ::=
    $setup ( timing_check_event, timing_check_event ,
        timing_check_limit [ , notify_register ] ) ;
    | $hold ( timing_check_event, timing check event,
        timing_check_limit [,notify register]);
    | $period ( controlled_timing_check_event , timing_check_limit
        [,notify register]);
    | $width (controlled_timing_check_event , timing_check_limit,
        constant_expression[ , notify register]);
    | $skew ( timing_check_event ,timing_check_event,
        timing_check_limit[, notify register]);
    | $recovery ( controlled_timing_check_event , timing_check_event,
        timing_check_limit [, notify register]);
    | $setuphold( timing_check_event, timing_check_event,
        timing_check_limit , timing_check_limit [, notify register]);

timing_check_event ::=
    [ timing_check_event_control ] specify_terminal_descriptor[ & & &
        timing_check_condition]

specify_terminal_descriptor ::=
    specify_input_terminal_descriptor
    | specify_output_terminal_descriptor

controlled_timing_check_event ::=
    timing_check_event_control specify_terminal_descriptor [ & & &
        timing_check_condition]

timing_check_event_control ::=
    posedge
    | negedge

```

```
    | edge_control_specifier
edge_control_specifier ::= edge [ edge_descriptor [ , edge_descriptor ] ]
edge_descriptor ::=
    01
    | 10
    | 0x
    | x1
    | 1x
    | x0
timing_check_conditon ::=
    scalar_timing_check_condition
    | ( scalar_timing_check_condition )
scalar_timing_check_conditon ::=
    expression
    | ~ expression
    | expression == scalar_constant
    | expression === scalar_constant
    | expression != scalar_constant
    | expression !== scalar_constant
    timing_check_limit ::= expression
scalar_constant ::= 1'b0 | 1'b1 | 1'B0 | 1'B1 | 'b0 | 'b1 | 'B0 | 'B1 | 1 | 0
notify_register ::= register_identifier
```

D.8 Expressions

```
net_lavalue ::=
    net_identifier
    | net_identifier [ expression ]
    | net_identifier [ msb_constant_expression : lsb_constant_expression ]
    | net_concatenation
reg_lvalue ::=
    reg_identifier
    | reg_identifier [ expression ]
    | reg_identifier [ msb_constant_expression : lsb_constant_expression ]
    | reg_concatenation
constant_expression ::=
    constant_primary
    | unary_operator constant_primary
```

```

    | constant_expression binary_operator constant_expression
    | constant_expression ? constant_expression : constant_expression
    | string
constant_primary ::=
    number
    | parameter_identifier
    | constant_concatenation
    | constant_multiple_concatenation
constant_mintypmax_expression ::=
    constant_expression
    | constant_expression : constant_expression : constant_expression
mintypmax_expression ::=
    expression
    | expression : expression : expression
expression ::=
    primary
    | unary_operator primary
    | expression binary_operator expression
    | expression ? expression : expression
    | string
unary_operator ::=
    + | - | ! | ~ | & | ~& | || | ~|| | ^ | ~^ | ^~
binary_operator ::=
    + | - | * | / | % | == | != | === | !== | && | ||
    | < | <= | > | >= | & | || | ^ | ^~ | ~^ | >> | <<
primary ::=
    number
    | identifier
    | identifier [ expression ]
    | identifier [ msb_constant_expression : lsb_constant_expression ]
    | concatenation
    | multiple_concatenation
    | function_call
    | ( mintypmax_expression )
number ::=
    decimal_number
    | octal_number
    | binary_number
    | hex_number

```

```
    | real_number
real_number ::=
    [ sign ] unsigned_number . unsigned_number
    | [ sign ] unsigned_number [ . unsigned_number ] e [ sign ] unsigned_number
    | [ sign ] unsigned_number [ . unsigned_number ] e [ sign ] unsigned_number
decimal_number ::=
    [ sign ] unsigned_number
    | [ size ] decimal_base unsigned_number
binary_number ::= [ size ] binary_base binary_digit { _ | binary_digit }
octal_number ::= [ size ] octal_base octal_digit { _ | octal_digit }
hex_number ::= [ size ] hex_base hex_digit { _ | hex_digit }
sign ::= + | -
size ::= unsigned_number
unsigned_number ::= decimal_digit { _ | decimal_digit }
decimal_base ::= `d | `D
binary_base ::= `b | `B
octal_base ::= `o | `O
hex_base ::= `h | `H
decimal_digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
binary_digit ::= x | X | z | Z | 0 | 1
octal_digit ::= x | X | z | Z | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
hex_digit ::= x | X | z | Z | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f | A | B | C | D | E | F
concatenation ::= { expression { , expression } }
multiple_concatenation ::= { expression { expression { , expression } } }
function_call ::=
    function_identifier ( expression { , expression } )
    | name_of_system_function [ expression { , expression } ]
name_of_system_function ::= $identifier
string ::= " { Any_ASCII_Characters_except_new_line } "
```

D.9 Genaral

```
Comment ::=
    short_comment
    | long_comment
short_comment ::= // comment_text \n
long_comment ::= /* comment_text */
comment_text ::= { Any_ASCII_Character }
```

`identifier ::= IDENTIFIER [ { , IDENTIFIER } ]`

`IDENTIFIER ::=`

`simple_identifier`

`| escaped_identifier`

`simple_identifier ::= [ a-zA-Z ][ a-zA-Z_ $ ]`

`escaped_identifier ::= \ { Any_ASCII_Character_except_white_space } white_space`

`white_space ::= space | tab | newline`

附录 E 附带光盘内容及说明

光盘内容包括:

1. MAX+PLUS II Baseline 10.2。
2. Quartus II 3.0 web version。
3. 本书中所有例子的 Verilog 源程序。

其中 pdf 文档建议用 Acrobat Reader 5.0 以上版本打开。

下面所列程序分别以.v 文件的形式按章节存于 source 目录下, 读者如要编辑运行这些程序, 可将.v 文件拷贝至硬盘, 并将文件的属性从只读型修改为存档型。

附: 源程序目录

- 【例 3.1】4 位全加器 (adder4.v)
- 【例 3.2】4 位计数器 (count4.v)
- 【例 3.3】4 位全加器的仿真程序 (adder_tp.v)
- 【例 3.4】4 位计数器的仿真程序 (coun4_tp.v)
- 【例 3.5】“与—或—非” 门电路 (aoi.v)
- 【例 5.1】用 case 语句描述的 4 选 1 数据选择器 (mux4_1.v)
- 【例 5.2】同步置数、同步清零的计数器 (count.v)
- 【例 5.3】用 always 过程语句描述的简单算术逻辑单元 (alu.v)
- 【例 5.4】用 initial 过程语句对测试变量 A、B、C 赋值 (test.v)
- 【例 5.5】用 begin-end 串行块产生信号波形 (wave1.v)
- 【例 5.6】用 fork-join 并行块产生信号波形 (wave2.v)
- 【例 5.7】持续赋值方式定义的 2 选 1 多路选择器 (mux2l_1.v)
- 【例 5.8】阻塞赋值方式定义的 2 选 1 多路选择器 (mux2l_2.v)
- 【例 5.9】非阻塞赋值 (non_block.v)
- 【例 5.10】阻塞赋值 (block.v)
- 【例 5.11】模为 60 的 BCD 码加法计数器 (count60.v)
- 【例 5.12】BCD 码—七段数码管显示译码器 (decode4_7.v)
- 【例 5.13】用 casez 描述的数据选择器 (mux_casez.v)
- 【例 5.14】隐含锁存器举例 (buried_ff.v)
- 【例 5.15】用 for 语句描述的七人投票表决器 (voter7.v)
- 【例 5.16】用 for 语句实现 2 个 8 位数相乘 (mult_for.v)
- 【例 5.17】用 repeat 实现 8 位二进制数的乘法 (mult_repeat.v)
- 【例 5.18】同一循环的不同实现方式 (loop1.v, loop2.v, loop3.v)
- 【例 5.19】使用了 include 语句的 16 位加法器 (adder16.v, adder.v)

- 【例 5.20】条件编译举例 (compile.v)
- 【例 6.1】加法计数器中的进程 (count.v)
- 【例 6.2】任务举例 (alutask.v)
- 【例 6.3】测试程序 (alu_tp.v)
- 【例 6.5】用函数和 case 语句描述的编码器 (不含优先顺序) (code_83.v)
- 【例 6.6】阶乘运算函数 (funct.v)
- 【例 6.7】测试程序 (funct_tp.v)
- 【例 6.8】顺序执行模块 1 (serial1.v)
- 【例 6.9】顺序执行模块 2 (serial2.v)
- 【例 6.10】并行执行模块 1 (paral1.v)
- 【例 6.11】并行执行模块 2 (paral2.v)
- 【例 7.1】调用门元件实现的 4 选 1 MUX (mux4_1a.v)
- 【例 7.2】用 case 语句描述的 4 选 1 MUX (mux4_1b.v)
- 【例 7.3】行为描述方式实现的 4 位计数器 (count4.v)
- 【例 7.4】数据流方式描述的 4 选 1 MUX (mux4_1c.v)
- 【例 7.5】用条件运算符描述的 4 选 1 MUX (mux4_1d.v)
- 【例 7.6】门级结构描述的 2 选 1 MUX (mux2_1a.v)
- 【例 7.7】行为描述的 2 选 1 MUX (mux2_1b.v)
- 【例 7.8】数据流描述的 2 选 1 MUX (mux2_1c.v)
- 【例 7.9】调用门元件实现的 1 位半加器 (half_add1.v)
- 【例 7.10】数据流方式描述的 1 位半加器 (half_add2.v)
- 【例 7.11】采用行为描述的 1 位半加器 (half_add3.v)
- 【例 7.12】采用行为描述的 1 位半加器 (half_add4.v)
- 【例 7.13】调用门元件实现的 1 位全加器 (full_add1.v)
- 【例 7.14】数据流描述的 1 位全加器 (full_add2.v)
- 【例 7.15】1 位全加器 (full_add3.v)
- 【例 7.16】行为描述的 1 位全加器 (full_add4.v)
- 【例 7.17】混合描述的 1 位全加器 (full_add5.v)
- 【例 7.18】结构描述的 4 位级连全加器 (add4_1.v)
- 【例 7.19】数据流描述的 4 位全加器 (add4_2.v)
- 【例 7.20】行为描述的 4 位全加器 (add4_3.v)
- 【例 8.1】\$time 与 \$realtime 的区别 (time_dif.v)
- 【例 8.2】\$random 函数的使用 (random_tp.v)
- 【例 8.3】1 位全加器进位输出 UDP 元件 (carry_udp.v)
- 【例 8.4】包含 x 态输入的 1 位全加器进位输出 UDP 元件 (carry_udpx1.v)
- 【例 8.5】用简缩符“?”表述的 1 位全加器进位输出 UDP 元件 (carry_udpx2.v)
- 【例 8.6】3 选 1 多路选择器 UDP 元件 (mux31.v)
- 【例 8.7】电平敏感的 1 位数据锁存器 UDP 元件 (latch.v)
- 【例 8.8】上升沿触发的 D 触发器 UDP 元件 (dff.v)

- 【例 8.9】带异步置 1 和异步清零的上升沿触发的 D 触发器 UDP 元件 (dff_udp.v)
- 【例 8.12】延迟定义块举例 (delay.v)
- 【例 8.13】激励波形的描述 (test1.v)
- 【例 8.15】用 always 过程块产生两个时钟信号 (test2.v)
- 【例 8.17】存储器在仿真程序中的应用 (rom.v)
- 【例 8.18】8 位乘法器的仿真程序 (mult_tp.v)
- 【例 8.19】8 位加法器的仿真程序 (add8_tp.v)
- 【例 8.20】2 选 1 多路选择器的仿真 (mux_tp.v)
- 【例 8.21】8 位计数器的仿真 (count8_tp.v)
- 【例 9.1】基本门电路的几种描述方法 (gate1.v, gate2.v, gate3.v)
- 【例 9.2】用 bufif1 关键字描述的三态门 (tri_1.v)
- 【例 9.3】用 assign 语句描述的三态门 (tri_2.v)
- 【例 9.4】三态双向驱动器 (bidir.v)
- 【例 9.5】三态双向驱动器 (bidir2.v)
- 【例 9.6】3-8 译码器 (decoder_38.v)
- 【例 9.7】8-3 优先编码器 (encoder8_3.v)
- 【例 9.8】用函数定义的 8-3 优先编码器 (code_83.v)
- 【例 9.9】七段数码管译码器 (decode47.v)
- 【例 9.10】奇偶校验位产生器 (parity.v)
- 【例 9.11】用 if-else 语句描述的 4 选 1 MUX (mux_if.v)
- 【例 9.12】用 case 语句描述的 4 选 1 MUX (mux_case.v)
- 【例 9.13】用组合电路实现的 ROM (rom.v)
- 【例 9.14】基本 D 触发器 (dff.v)
- 【例 9.15】带异步清 0、异步置 1 的 D 触发器 (dff1.v)
- 【例 9.16】带同步清 0、同步置 1 的 D 触发器 (dff2.v)
- 【例 9.17】带异步清 0、异步置 1 的 JK 触发器 (jk_ff.v)
- 【例 9.18】电平敏感的 1 位数据锁存器 (latch_1.v)
- 【例 9.19】带置位和复位端的 1 位数据锁存器 (latch_2.v)
- 【例 9.20】8 位数据锁存器 (latch_8.v)
- 【例 9.21】8 位数据寄存器 (reg8.v)
- 【例 9.22】8 位移位寄存器 (shifter.v)
- 【例 9.23】可变模加法/减法计数器 (updown_count.v)
- 【例 9.24】4 位 Johnson 计数器 (异步复位) (johnson.v)
- 【例 9.25】256×8 RAM 模块 (ram256x8.v)
- 【例 9.26】256×16 RAM 块 (map_lpm_ram.v)
- 【例 9.27】4 位串/并转换器 (serial_pal.v)
- 【例 9.28】用函数实现简单的处理器 (mpc.v)
- 【例 9.29】微处理器的测试代码 (mpc_tp.v)
- 【例 9.30】乘累加器 (MAC) 代码 (mac.v)

- 【例 9.31】乘累加器的测试代码 (mac_tp.v)
- 【例 10.1】非流水线方式的 8 位全加器 (adder8.v)
- 【例 10.2】4 级流水线方式的 8 位全加器 (pipeline.v)
- 【例 10.3】两个加法器和一个数据选择器的实现方式 (resource1.v)
- 【例 10.4】两个数据选择器和一个加法器的实现方式 (resource2.v)
- 【例 10.5】状态机设计的例子 (fsm.v)
- 【例 10.6】自动转换量程频率计控制器 (control.v)
- 【例 10.7】8 位全加器 (add8.v)
- 【例 10.8】8 位寄存器 (reg8.v)
- 【例 10.9】累加器顶层连接文本描述 (acc.v)
- 【例 10.10】用 include 描述的累加器 (accn.v)
- 【例 10.11】阻塞赋值方式描述的移位寄存器 1 (block1.v)
- 【例 10.12】阻塞赋值方式描述的移位寄存器 2 (block2.v)
- 【例 10.13】阻塞赋值方式描述的移位寄存器 3 (block3.v)
- 【例 10.14】非阻塞赋值方式描述的移位寄存器 (block4.v)
- 【例 10.15】长帧同步时钟的产生 (longframe1.v)
- 【例 10.16】引入了 D 触发器的长帧同步时钟的产生 (longframe2.v)
- 【例 11.1】数字跑表 (paobiao.v)
- 【例 11.2】4 位数字频率计控制模块 (fre_ctrl.v)
- 【例 11.3】4 位数字频率计计数器模块 (count10.v)
- 【例 11.4】频率计锁存器模块 (latch_16.v)
- 【例 11.5】交通灯控制器 (traffic.v)
- 【例 11.6】“梁祝”乐曲演奏电路 (song.v)
- 【例 11.7】自动售饮料机 (sell.v)
- 【例 11.8】多功能数字钟 (clock.v)
- 【例 11.9】电话计费器程序 (account.v)
- 【例 12.1】8 位级连加法器 (add_jl.v)
- 【例 12.2】8 位并行加法器 (add_bx.v)
- 【例 12.3】8 位超前进位加法器 (add_ahed.v)
- 【例 12.4】8 位并行乘法器 (mult.v)
- 【例 12.5】4×4 查找表乘法器 (mult4x4.v)
- 【例 12.6】8 位加法树乘法器 (add_tree.v)
- 【例 12.7】11 阶 FIR 数字滤波器 (fir.v)
- 【例 12.8】16 位高速数字相关器 (correlator.v)
- 【例 12.9】(7, 4) 线性分组码编码器 (linear.v)
- 【例 12.10】(7, 4) 线性分组码译码器 (decoder1.v)
- 【例 12.11】(7, 4) 循环码编码器 (cycle.v)
- 【例 12.12】(7, 4) 循环码纠错译码器 (decoder2.v)
- 【例 12.13】CRC 编码 (crc.v)

参 考 文 献

- [1] Donald E. Thomas, Philip R. Moorby. The Verilog Hardware Description Language(Fourth Edition). Kluwer Academic Publishers, 1998
- [2] Doulos. The Verilog Golden Reference Guide. 1996
- [3] Synplify-Lite Synthesis User's Guide
- [4] Stuart Sutherland. The IEEE Verilog 1364-2001 Standard What's New, and Why You Need It. Sutherland HDL, Inc. 2001
- [5] 潘松,黄继业. EDA 技术实用教程. 北京: 科学出版社, 2002
- [6] 张明. Verilog HDL 实用教程. 成都: 电子科技大学出版社, 1999
- [7] 王金明, 杨吉斌. 数字系统设计与 Verilog HDL. 北京: 电子工业出版社, 2002
- [8] www.fpga.com.cn

[G e n e r a l I n f o r m a t i o n]

书名= V e r i l o g H D L程序设计教程

作者=

页数= 3 0 8

S S号= 0

出版日期=

封面
书名
版权
前言
目录
正文